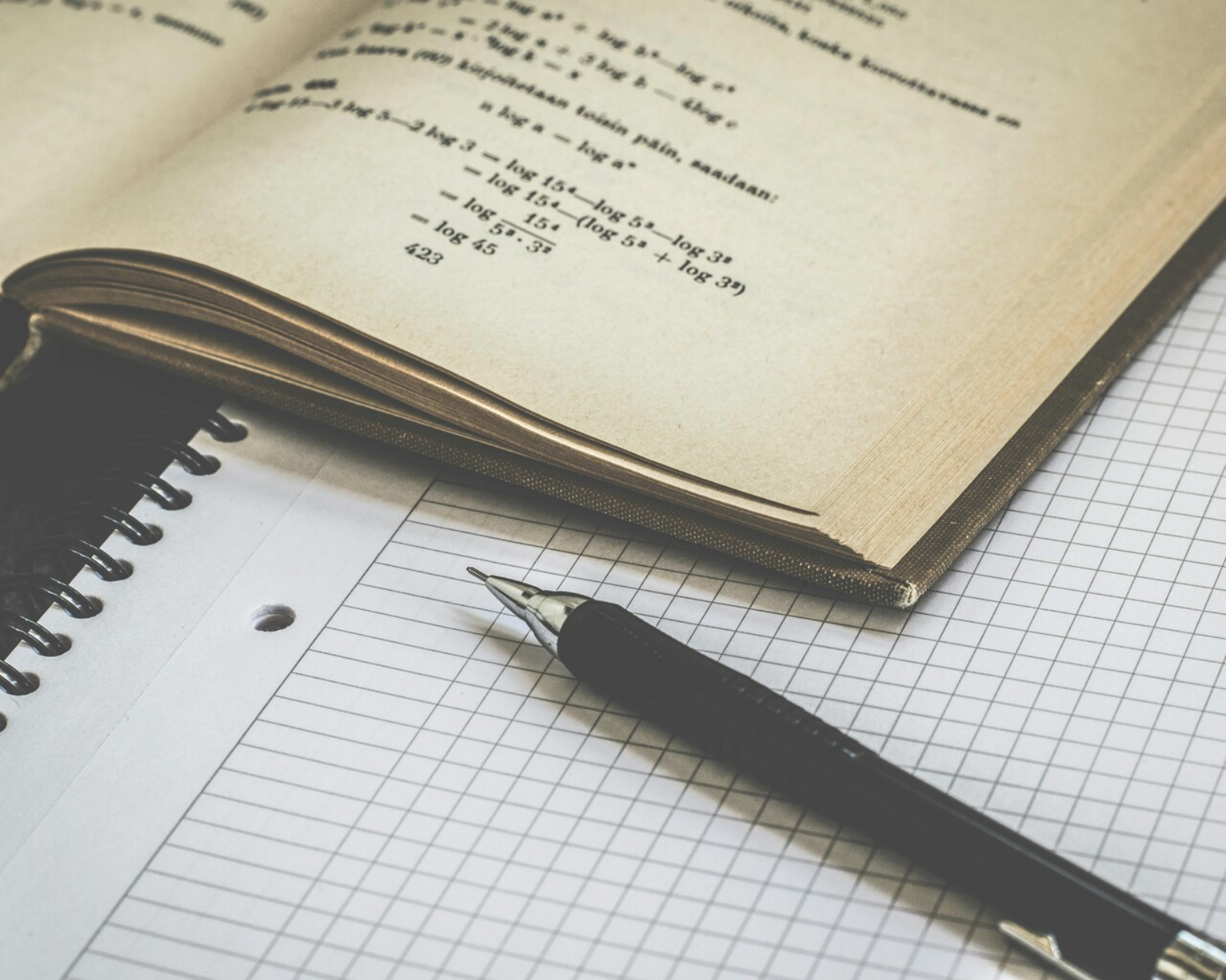




LLM System Interview

A Guide to Architecture, Training, Inference, and System Design

Author: Hao Hoang

Institute: AI Interview Prep

Date: April 25, 2026

Version: First edition, 2026

Focus: Applied AI/ML and LLM systems



AI Interview Prep

Ace Your Next Opportunity

Contents

Preface	vi
How to Use This Book	vii
For Candidates Preparing for Interviews	x
For Engineers Building LLM Systems	xiii
Notation and Symbols	xvi
Acknowledgments	xxi
About the Author	xxii
I Overview and Interview Landscape	1
Chapter 1 The LLM Systems Interview	2
1.1 What Companies Actually Ask	2
1.2 Roles That Require LLM Systems Knowledge	4
Chapter 2 How to Approach an LLM System Design Question	8
2.1 A Repeatable Framework	8
2.2 Trade-Off Axes You Must Name	10
2.3 Common Pitfalls in Answers	13
II Foundations: Transformer Architecture and Training Signals	16
Chapter 3 Transformer Architecture Interview Essentials	17
3.1 The Baseline Decoder-Only Transformer	17
3.2 Normalization Choices	22
3.3 Activation Functions in Modern LLMs	28
3.4 Position Encodings	33
Chapter 4 Hyperparameters You Will Be Asked to Justify	40
4.1 Width and Depth	40
4.2 Attention Head Configuration	45
4.3 Vocabulary Size and Tokenization Impact	51
4.4 Regularization in Pre-Training	57
Chapter 5 Stability Tricks in Large-Scale Training	63
5.1 Softmax Instabilities	63
5.2 Numerical Precision and Training Health	69

III Mixture of Experts and Sparse Architectures	76
Chapter 6 Why MoE Is Now Standard in Frontier Systems	77
6.1 Dense vs. Sparse Models for Fixed FLOPs	77
6.2 Routing: The Heart of Every MoE	82
Chapter 7 MoE Training and Systems Considerations	89
7.1 Fine-Grained and Shared Experts	89
7.2 Load Balancing and Stability	94
7.3 Expert Parallelism Gotchas for Interviews	101
IV GPUs, Kernels, and Single-Device Performance	107
Chapter 8 GPU Architecture for LLM Engineers	108
8.1 The Compute and Memory Hierarchy	108
8.2 Arithmetic Intensity and the Roofline Model	114
Chapter 9 Making a Single GPU Go Fast	121
9.1 Reducing Memory Movement	121
9.2 Exploiting Memory Layout	126
9.3 Lower Precision and Quantization-Aware Training	133
Chapter 10 Writing and Benchmarking Custom Kernels	139
10.1 Benchmarking and Profiling Discipline	139
10.2 Implementing Kernels	145
10.3 FlashAttention as a Case Study	152
V Distributed Training: Parallelism at Scale	159
Chapter 11 Multi-GPU and Multi-Node Fundamentals	160
11.1 Interconnect Topologies	160
11.2 Collective Communication Primitives	165
11.3 The Software Stack	171
Chapter 12 Parallelism Strategies	178
12.1 Data Parallelism and ZeRO	178
12.2 Model Parallelism	184
12.3 Activation and Sequence Parallelism	190
12.4 Putting It All Together: 3D and 4D Parallelism	196
VI Scaling Laws and Training Economics	203
Chapter 13 Predictable Scaling for Interview Answers	204
13.1 The Three Canonical Scaling Laws	204

13.2 Using Scaling Laws to Make Design Decisions	210
Chapter 14 Chinchilla and Beyond	216
14.1 Compute-Optimal Training	216
14.2 Inference-Aware Token-to-Parameter Ratios	222
14.3 Maximal Update Parameterization (muP)	228
VII Inference Systems	234
Chapter 15 The Inference Workload	235
15.1 Prefill vs. Generation	235
15.2 Latency, Throughput, and Cost	240
Chapter 16 Reducing the KV Cache	247
16.1 Attention Variants for Cheaper Inference	247
16.2 Cross-Layer and Local Attention	252
Chapter 17 Going Beyond the Transformer for Inference	258
17.1 State-Space and Linear-Attention Models	258
17.2 Non-Autoregressive Generation	264
Chapter 18 Speculative Decoding and Serving Optimizations	270
18.1 Speculative Sampling	270
18.2 Serving System Design	276
Chapter 19 Compression Techniques for Deployment	283
19.1 Quantization	283
19.2 Pruning and Distillation	288
VIII Data: The Real Differentiator	294
Chapter 20 Pre-Training Data Pipelines	295
20.1 Where Training Data Comes From	295
20.2 Evolution of Open Pre-Training Corpora	300
20.3 Legal and Ethical Considerations	306
Chapter 21 Data Filtering and Deduplication Algorithms	312
21.1 Quality Filtering Algorithms	312
21.2 Targeted Filtering Tasks	318
21.3 Deduplication at Scale	323
Chapter 22 Mid-Training and Post-Training Data	330
22.1 Instruction and Chat Data	330
22.2 Long-Context and Domain Data	336
22.3 Data Quality Heuristics That Matter in Interviews	342

IX Evaluation	349
Chapter 23 Designing an Evaluation	350
23.1 Goals of Evaluation	350
23.2 Metrics and Protocols	355
Chapter 24 Benchmarks You Must Know	362
24.1 Knowledge and Reasoning Benchmarks	362
24.2 Instruction Following and Chat Evaluation	367
24.3 Agentic and Safety Benchmarks	374
Chapter 25 Validity, Contamination, and Real-World Use	380
25.1 Train-Test Overlap and Contamination	380
25.2 Real-World Evaluation	385
X Alignment and Post-Training	391
Chapter 26 Supervised Fine-Tuning	392
26.1 What SFT Can and Cannot Teach	392
26.2 SFT Data in Practice	398
Chapter 27 Preference-Based Alignment (RLHF)	405
27.1 The RLHF Pipeline	405
27.2 Optimization Algorithms	411
27.3 Pitfalls of RLHF	417
Chapter 28 Reinforcement Learning from Verifiable Rewards	423
28.1 From RLHF to RLVR	423
28.2 Policy Gradient Methods	429
28.3 Case Studies in Reasoning Models	435
XI End-to-End System Design Drills	441
Chapter 29 Building a Production LLM Serving Stack	442
29.1 Reference Architecture	442
29.2 Capacity Planning	448
Chapter 30 Designing a Pre-Training Run from Scratch	454
30.1 From Compute Budget to Architecture	454
30.2 Operations and Failure Modes	460
Chapter 31 Designing a Fine-Tuning and Alignment Pipeline	466
31.1 Scoping the Problem	466
31.2 Operational Concerns	472

Appendix A Napkin Math for LLM Interviews	478
A.1 Parameter Counts and Memory Footprints	478
A.2 FLOPs per Forward and Backward Pass	479
A.3 KV-Cache Sizing and Latency Estimates	481
Appendix B Common Interview Questions	483
B.1 Architecture and Training Questions	483
B.2 Inference and Serving Questions	485
B.3 Alignment and Evaluation Questions	487
Appendix C Checklists for System Design Answers	490
C.1 The 10-Minute LLM System Design Checklist	490
C.2 The Pre-Training Run Readiness Checklist	491
C.3 The Inference Deployment Readiness Checklist	492
Appendix D Further Reading	495
D.1 Canonical Papers by Topic	495
D.2 Blogs, Talks, and Engineering Reports	501

Preface

This book exists because the interview and the job have converged in a way they did not five years ago.

Until recently, most software engineering interviews tested general systems-design intuition: databases, caches, load balancers, message queues. The underlying assumption was that specialized domain knowledge could be learned on the job once a strong generalist was hired. That assumption has broken down at frontier AI companies. A candidate who cannot reason about KV cache memory pressure, arithmetic intensity on a GPU, or the difference between compute-bound prefill and memory-bandwidth-bound decoding will not pass a senior-level systems interview at the teams building today’s most important models. The domain has become so operationally specific that generalist intuition is no longer sufficient, and no single paper, blog post, or course covers it end to end.

The gap this book fills. Academic courses teach the theory of transformers; they rarely explain why GQA with $N_{kv} = N/8$ reduces KV cache memory by $8\times$ and how that changes the feasible batch size on a single H100. Research papers present results; they do not explain how to answer “walk me through how you would size the hardware for a 70B model serving 5,000 QPS” in the 45 minutes of a live interview. Engineering blogs cover individual components in depth; they do not provide the unified arithmetic framework that lets you move fluidly from model parameters to memory budgets to parallelism strategies to cost estimates. This book does.

What this book is. It is a technical reference and an interview preparation guide written for engineers who already know how to code and who already understand basic ML. Every chapter develops the arithmetic behind a topic from first principles, shows how that arithmetic connects to real hardware constraints, and then frames the result as an interview question with a worked answer. The appendices collect the most important formulas for fast reference during active preparation.

What this book is not. It is not an introduction to deep learning. It does not teach backpropagation or explain what a transformer is from scratch. It does not survey the research literature comprehensively. Readers who need that foundation first should work through a course on neural networks before returning here. It is also not a prescriptive guide to any specific company’s interview process-the frameworks and derivations are generic because the physics of hardware and the economics of inference are generic.

Coverage. The book follows a deliberate progression. Part I maps the interview landscape and introduces the four-question framework that structures every design answer. Parts II through IV build the technical foundation: transformer architecture, training hyperparameters, GPU hardware, and kernel-level performance. Parts V and VI scale that foundation across distributed training and scaling laws. Part VII covers the inference stack in depth, from the prefill-decode split through KV cache optimizations, speculative decoding, and compression. Parts VIII through X address the three domains that interviewers probe as differentiators-data pipelines, evaluation design, and alignment methods. Part XI synthesizes everything into three full end-to-end design drills.

Using derivations, not memorization. Every number in this book is derived, not asserted. The estimate that a 7B model in bf16 requires approximately 13 GB of inference memory follows from $2P$ bytes, with $P = 6.7 \times 10^9$. The estimate that a single H100 decode step takes roughly 8 ms at batch size 1 follows from dividing M_{weights} by the HBM bandwidth. These derivations are the point. An interviewer who changes one constraint-“now make it a 13B model”-expects a candidate who can update the arithmetic in real time, not one who has memorized a table. First principles produce that flexibility; memorization does not.

How to Use This Book

The book supports three distinct uses: interview preparation, on-the-job reference, and systematic study of LLM systems. Read it linearly if you are building up the subject from scratch. Follow a targeted path if you are preparing for a specific role or plugging specific gaps.

Linear Reading Order

The parts are ordered by dependency. Each part assumes the vocabulary and arithmetic developed in all prior parts.

Part	Topic	Prerequisite
I	Interview landscape and framework	None
II	Transformer architecture and training	None
III	Mixture of Experts	Part II
IV	GPU architecture and custom kernels	Part II
V	Distributed training and parallelism	Parts II, IV
VI	Scaling laws and training economics	Parts II, V
VII	Inference systems	Parts II, IV, VI
VIII	Data pipelines	Part II
IX	Evaluation	Parts II, VIII
X	Alignment and post-training	Parts II, VIII, IX
XI	End-to-end design drills	All prior parts

Appendix A (parameter counts, FLOPs, KV cache formulas) and Appendix C (checklists) are designed to be printed and kept at your desk during active preparation. Appendix B is a curated question bank organized by topic. Appendix D is an annotated reading list of papers and engineering reports.

Targeted Reading Paths

Path 1 - Candidate Preparing for an Inference / Serving Interview

Focus: memory arithmetic, KV cache, latency-throughput trade-offs, speculative decoding, quantization.

1. Chapter 2 (framework) - read fully, internalize the four-axis requirement template.
2. Chapter 3, Sections 1-2 (transformer essentials, parameter count).
3. Appendix A (reference formulas) - commit to memory.
4. Chapter 8 (GPU architecture, roofline model, arithmetic intensity).
5. Chapter 15 (the inference workload: prefill vs. decode, latency vs. throughput).
6. Chapter 16 (KV cache reduction: GQA, MQA, MLA).
7. Chapter 18 (speculative decoding and continuous batching).
8. Chapter 19 (quantization and compression).
9. Chapter 29 (full end-to-end serving design drill).

Path 2 - Candidate Preparing for a Training / Infrastructure Interview

Focus: distributed training, parallelism strategies, memory budgets, scaling laws, resilience.

1. Chapter 2 (framework).
2. Chapter 3 (architecture) and Chapter 4 (hyperparameters).
3. Chapter 5 (training stability).
4. Chapter 8 (GPU architecture).
5. Chapter 11 (multi-GPU fundamentals: collectives, bandwidth).
6. Chapter 12 (parallelism strategies: TP, PP, DP, ZeRO/FSDP).
7. Chapter 13 (scaling laws and napkin math).
8. Chapter 14 (Chinchilla and compute-optimal training).
9. Appendix A (FLOPs and memory formulas).
10. Chapter 30 (end-to-end pre-training design drill).

Path 3 - Candidate Preparing for an Applied / Product Engineering Interview

Focus: serving economics, fine-tuning, alignment, evaluation, end-to-end system design.

1. Chapter 1 (interview landscape, role definitions).
2. Chapter 2 (framework).
3. Chapter 15 (inference workload fundamentals).
4. Chapter 19 (compression and deployment).
5. Chapter 20-22 (data pipelines and data quality).
6. Chapter 23-25 (evaluation design, benchmarks, contamination).
7. Chapter 26 (supervised fine-tuning).
8. Chapter 27 (RLHF and preference alignment).
9. Chapter 31 (fine-tuning and alignment pipeline drill).

Chapter Difficulty

Chapters are written at three levels. The level is not labeled explicitly because the same content serves both preparation and reference; the *depth* at which you engage with it is the variable.

Conceptual (most of Parts I, II, VIII, IX, X) Develops vocabulary, mental models, and interview framing.

Suitable as first-pass reading and as review before an interview day.

Arithmetic (most of Parts IV, VI, VII; Appendix A) Derives quantitative estimates from first principles. Requires pencil and paper. Work through the examples actively rather than reading passively.

Implementation (most of Parts III, V; Chapter 10) Discusses kernel-level, compiler-level, or systems-level mechanics. Assumed background: familiarity with PyTorch, CUDA concepts, and distributed training frameworks.

Conventions in Each Chapter

Every chapter follows the same internal structure:

- **The Take** - the single most important insight for an interview, stated in one paragraph.
- **Technical content** - derivations, diagrams, or system descriptions.

- **How This Shows Up** - two or three representative interview questions with annotated strong answers.
- **Key Takeaways** - a short bullet list of the points an interviewer will probe.

Questions in **How This Shows Up** appear in *italics*. Strong-answer annotations focus on structure-what to say first, what to derive, what follow-up to anticipate-rather than providing a script. Interviewers at frontier labs probe depth by changing constraints; the annotations show which numbers to re-derive when that happens.

For Candidates Preparing for Interviews

The LLM systems interview rewards candidates who derive answers, not candidates who recall them. Every technique in this section is aimed at building that derivation habit before you sit down with an interviewer.

What the Interview Actually Tests

A senior-level LLM systems interview has three observable signals interviewers are collecting simultaneously.

First: vocabulary precision. “The KV cache grows with sequence length” is not an answer. “At 4,096 tokens, a single sequence in a 7B GQA model with 8 KV heads and head dimension 128 costs approximately 0.17 GB per sequence in bf16” is. The arithmetic formulas in Appendix A are the vocabulary; the chapters explain where each term comes from.

Second: constraint-driven reasoning. Every design question has a binding constraint—usually KV cache memory pressure for serving, usually communication overhead for training. Strong candidates name the binding constraint before proposing any optimization. Candidates who list optimizations without identifying the bottleneck signal that they have read blog posts, not reasoned from hardware physics.

Third: adaptability under constraint changes. The most reliable signal in a live interview is what happens when the interviewer says “what if context length doubles?” or “what if you now need to support five model variants?” A candidate who derived their original answer updates the arithmetic immediately. A candidate who memorized an architecture has to restart. This is why every technical chapter in this book teaches derivation, not recall.

A Four-Week Preparation Plan

This plan assumes roughly two hours of active study per day. Adjust the pacing to your timeline; the sequence of topics is fixed, but the time budget per topic is flexible.

Week 1 - Foundation Read Chapters 1-5 and Appendix A fully. After each chapter, close the book and re-derive the key formulas from memory on paper. Target: be able to estimate model memory footprint (inference and training), forward-pass FLOPs, and KV cache size for any model configuration given to you verbally.

Week 2 - Hardware and Distributed Systems Read Chapters 8-12. Work through every numerical example with a calculator. Practice explaining the roofline model out loud to an imaginary interviewer. Spend extra time on Chapter 12 (parallelism strategies): draw the TP/PP/DP combination diagram from memory until it takes less than two minutes.

Week 3 - Inference and Scaling Read Chapters 13-19. Focus on Chapter 15 (the inference workload) and Chapter 18 (speculative decoding and serving). For each optimization technique in Chapters 16-19, write one sentence explaining which constraint it addresses and what it trades away. Read Appendix C (checklists) and internalize the ten-minute system design checklist.

Week 4 - Integration and Drills Read Chapters 20-28 at pace (one per day). Spend the last three days on Part XI (Chapters 29-31), working each drill as a timed mock interview: 45 minutes, whiteboard or

paper, no references. After each drill, compare your answer against the chapter’s annotated response and identify which constraints you named late or which numbers you could not derive.

How to Drill a Chapter

Passive reading does not build the derivation habit. Use this sequence for every chapter in the arithmetic and implementation tiers.

1. Read the chapter fully once, following along with any derivations.
2. Cover the page. Write down the key formula from memory.
3. Plug in a different model configuration (change d , change L , change N_{kv}) and re-derive the result.
4. Answer the “How This Shows Up” questions from the chapter out loud, targeting 90 seconds per answer before expanding to the full 5-minute version.
5. Read the Key Takeaways bullet list and verify you can explain each point without the book.

Timing Your Answers

A 45-minute design interview has roughly the following pacing for a strong candidate:

Phase	Time	What you are doing
Requirements clarification	3-5 min	Name TTFT, throughput, quality, cost; ask which is binding
End-to-end pipeline sketch	5-8 min	Tokenize $\rightarrow$ prefill $\rightarrow$ decode $\rightarrow$ scheduler; label bottlenecks
Binding constraint analysis	5-7 min	Derive KV cache footprint or memory budget; identify the limit
Targeted optimizations	10-15 min	Propose GQA, speculative decoding, quantization as constraint responses
Trade-off discussion	5-8 min	Answer the interviewer’s constraint-change follow-ups
Wrap-up	2-3 min	Summarize and invite feedback

The requirement clarification phase is the most commonly skipped and the most damaging to skip. Interviewers at staff and principal level watch for it explicitly.

Common Failure Modes

Cargo-culting numbers without derivation Stating “Chinchilla says 20 tokens per parameter” without being able to derive where the 20 comes from, or what changes when inference cost is amortized over more queries. Chapter 14 derives this ratio and explains when it does not apply.

Conflating latency and throughput Describing TTFT and tokens-per-second as if they optimize together. They do not; Chapter 15 derives why and what each responds to.

Listing optimizations before naming the bottleneck Proposing speculative decoding, FlashAttention, and quantization in the same breath before establishing whether the system is KV-cache-constrained, compute-constrained, or communication-constrained. Chapter 2 provides the framework for identifying the constraint first.

Treating batch size as a free variable Failing to recognize that batch size is bounded by VRAM and that both data parallelism and pipeline parallelism consume it as a resource. Appendix A and Chapter 12 derive the precise constraints.

Ignoring inference economics during training design Designing a training run without considering the serving cost of the resulting model. Chapter 13 establishes why inference cost is a first-class training constraint.

Using the Appendices

Appendix A (formulas) is the most important reference during active preparation. Print it. Put it next to your desk. After two weeks of preparation you should not need to look at it; if you still do, spend another session re-deriving each formula from scratch.

Appendix B (question bank) is organized by topic. Use it to simulate a 45-minute interview: pick one design question and one deep-dive question from the same topic area, set a timer, and answer both without references.

Appendix C (checklists) provides a ten-minute LLM system design checklist, a pre-training run readiness checklist, and an inference deployment readiness checklist. Internalize the design checklist until you can reproduce it verbally in under two minutes.

Appendix D (reading list) is for candidates preparing at the staff or principal level, where interviewers expect familiarity with the primary literature. Papers are annotated with the specific claims you are expected to reproduce, not just name.

For Engineers Building LLM Systems

This book is organized around interview questions, but its technical content is not interview-specific. The arithmetic behind batch-size constraints, the parallelism trade-offs in distributed training, the memory-bandwidth math behind KV cache sizing-these are the same calculations you do on the job. This section maps the book's chapters to the decisions you encounter in production.

When to Reach for This Book

You are sizing a serving fleet for a new model Start with Appendix A (KV cache formula, memory footprint) and Chapter 15 (the inference workload, latency-throughput trade-offs). The KV-crossover batch size $B^* = M_{\text{weights}}/C_{\text{kv}}$ is the first number to compute; it determines whether you are in the superlinear or sublinear throughput regime before you touch any configuration.

You are choosing a parallelism strategy for a training run Chapter 12 derives the memory and communication cost of every combination: tensor-parallel, pipeline-parallel, data-parallel, ZeRO stages 1-3, and FSDP. The worked examples are parameterized so you can substitute your own model size and cluster topology.

You are debugging a training run that is not hitting expected MFU Chapter 8 (arithmetic intensity and the roofline model) and Chapter 11 (collective communication bandwidth and all-reduce timing) are the diagnostic starting points. Low MFU is almost always communication-bound, memory-bound, or pipeline-bubble-bound; the chapters give you the arithmetic to distinguish the three cases.

You are evaluating a KV cache optimization (GQA, MLA, paged attention) Chapter 16 derives the memory reduction and the resulting change in maximum batch size for each technique. The formulas let you estimate the throughput gain on your specific hardware and model before running any experiment.

You are designing an evaluation harness for a fine-tuned model Chapters 23-25 cover evaluation design from first principles: metric selection, contamination detection, benchmark validity, and the gap between benchmark performance and deployment behavior. Chapter 25 covers domain-specific evaluation design (medical, legal, code) and deployment telemetry.

You are deciding between SFT, DPO, and RLVR for a post-training objective Chapter 26 (SFT), Chapter 27 (DPO and RLHF), and Chapter 28 (GRPO and RLVR) each cover the method's mechanics, its data requirements, its failure modes, and the scenarios where it is the right choice. Chapter 31 synthesizes the decision into an end-to-end alignment pipeline design.

Chapter Map by Engineering Decision

Decision / Problem	Primary chapters
Model memory footprint (inference)	Appendix A, Ch. 3
Model memory footprint (training)	Appendix A, Ch. 9
Forward-pass and training FLOPs	Appendix A, Ch. 13
KV cache sizing and batch limits	Appendix A, Ch. 15, 16
GPU roofline and arithmetic intensity	Ch. 8
FlashAttention and kernel fusion	Ch. 9, 10
Collective communication cost	Ch. 11
Parallelism strategy selection	Ch. 12
Compute-optimal token budget	Ch. 13, 14
Inference latency-throughput curve	Ch. 15
GQA / MQA / MLA trade-offs	Ch. 16
State-space and hybrid architectures	Ch. 17
Speculative decoding setup	Ch. 18
Quantization (INT8, INT4, FP8)	Ch. 19
Pre-training data pipeline design	Ch. 20, 21
Instruction and preference data curation	Ch. 22
Evaluation metric selection	Ch. 23
Benchmark selection and contamination	Ch. 24, 25
SFT data and training recipe	Ch. 26
Reward modeling and RLHF	Ch. 27
GRPO and verifiable reward RL	Ch. 28
End-to-end serving stack design	Ch. 29
End-to-end pre-training design	Ch. 30
End-to-end fine-tuning pipeline	Ch. 31

The Arithmetic Is the Point

Every number in production LLM engineering is derivable. When a colleague states that “a 70B model needs at least 4 H100s to serve,” the correct response is to ask whether that accounts for KV cache at the target batch size and context length, not to accept it as a fact. The derivation: $M_{\text{weights}} = 2 \times 70 \times 10^9 \approx 140 \text{ GB}$ in bf16, spread across $\lceil 140/80 \rceil = 2$ H100s for weights alone, but a realistic batch size and context window can double that figure.

Chapter 15 and Appendix A derive these estimates in full, including the formulas for B_{max} (VRAM-limited batch size), B^* (KV-crossover batch size), and t_{step} (decode step latency as a function of batch size and model size). These are not rules of thumb; they are consequences of memory capacity and bandwidth, and they update correctly when you change the model, the hardware, or the context length.

Connecting the Book to the Engineering Literature

The book’s technical content is grounded in papers and engineering reports that have shaped the current state of production LLM systems. Where a chapter’s content derives from a specific paper, the source is cited inline. Appendix D provides an annotated reading list organized by topic, with each entry annotated to indicate which specific claims are worth understanding in depth rather than simply citing by name.

The FlashAttention algorithm (Chapters 9 and 10), the Chinchilla scaling law (Chapter 14), continuous batching (Chapter 18), and the GRPO training algorithm (Chapter 28) are examples where reading the original paper alongside the corresponding chapter will deepen your understanding of the design choices behind the method. The reading list indicates which papers repay that deeper reading and which are sufficient to know at the result level.

Staying Current

The LLM systems field moves fast. The core arithmetic and hardware physics in this book—memory bandwidth, arithmetic intensity, roofline analysis, parallelism trade-offs—are stable. Specific techniques (attention variants, serving schedulers, compression methods) continue to evolve. When a new technique is announced, the most reliable way to evaluate it is to ask: which constraint does it address, and what does it trade away? That question is the same one this book trains you to ask, and it does not go stale.

Notation and Symbols

This chapter standardizes the symbols, abbreviations, and conventions used throughout the book. Definitions are precise; where a symbol carries multiple meanings in the literature, the book’s chosen convention is stated explicitly. Units are always written out on first use in each chapter; the table below lists the canonical forms.

Model and Architecture

Symbol	Meaning	Notes
P	Total parameter count	e.g. $P = 7 \times 10^9$ for a 7B model
L	Number of transformer layers (depth)	
d	Hidden / residual stream dimension	also written d_{model}
d_{model}	Hidden dimension (explicit form)	synonym for d
d_{head}	Attention head dimension	typically 128
d_{ff}	Feed-forward (MLP) intermediate dimension	$\lfloor 8d/3 \rfloor$ for SwiGLU; $4d$ for ReLU/GeLU
N	Number of query attention heads	
N_{kv}	Number of key-value heads	$N_{\text{kv}} = N$ (MHA); $N_{\text{kv}} < N$ (GQA/MQA)
H	Head dimension; also d_{head}	context disambiguates
H_Q	Query head count	used in GQA ratio H_Q/H_{KV}
H_{KV}	Key-value head count	$H_{KV} = 1$ (MQA); $H_{KV} = H_Q/G$ (GQA)
G	GQA group size (N/N_{kv})	reduction factor for KV cache
V	Vocabulary size	tokens in the tokenizer
S	Sequence length in tokens	also written T in some chapters
T	Sequence length in tokens	synonym for S ; also used for training tokens
K_e	Expert utilization rate in MoE	fraction of experts activated per token
r	LoRA rank	dimension of low-rank adapters

Training and Optimization

Symbol	Meaning	Notes
C	Total compute budget (FLOPs)	
C_{fwd}	FLOPs for one forward pass	$\approx 2N_T P$
C_{step}	FLOPs for one training step	$\approx 6N_T P$ (fwd + bwd)
N_T	Total tokens in a batch: $B \times T$	
D	Training dataset size (tokens)	
D^*	Chinchilla-optimal token count	$D^* \approx 20P$ at compute-optimal
N^*	Chinchilla-optimal parameter count	
η	Learning rate	
α	Adam β_1 decay (first moment)	or generic scaling coefficient
β	Adam β_2 decay (second moment); also KL penalty coefficient	context disambiguates
ϵ	Adam numerical stability term; also PPO clip ratio tolerance	
m_t	Adam first moment (mean) at step t	
v_t	Adam second moment (variance) at step t	
μ	Momentum coefficient; also mean of a distribution	
γ	Gradient clipping threshold; also discount factor in RL	
λ	L2 regularization weight; also GAE discount in RL	

Inference and Serving

Symbol	Meaning	Notes
B	Batch size (number of concurrent sequences)	
B_{max}	Maximum batch size given VRAM	$\lfloor (M_{\text{GPU}} - M_{\text{weights}}) / C_{\text{kv}} \rfloor$
B^*	KV-crossover batch size	$M_{\text{weights}} / C_{\text{kv}}$; throughput shifts from superlinear to s
C_{kv}	KV cache memory per sequence	$2 \times L \times N_{\text{kv}} \times H \times S \times 2$ bytes (bf16)
C_{seq}	KV cache memory per sequence (alternate notation)	synonym for C_{kv}
M_{weights}	Model weight memory	$2P$ bytes in bf16
M_{GPU}	Total GPU VRAM	e.g. 80 GB for H100 SXM
t_{step}	Decode step latency	$(M_{\text{weights}} + B \cdot C_{\text{kv}}) / \text{BW}$
TPS	Tokens per second (throughput)	aggregate across all concurrent sequences
TTFT	Time to first token	prefill latency; compute-bound
TBT	Time between tokens (decode latency)	per-token; memory-bandwidth-bound
TPOT	Time per output token	synonym for TBT
ITL	Inter-token latency	synonym for TBT / TPOT
π_{θ}	Policy (language model) parameterized by θ	used in RLHF/RL chapters
π_{ref}	Reference policy (frozen pre-RLHF model)	
r_{ϕ}	Reward model parameterized by ϕ	
θ	Model parameters (generic)	
G	Number of rollouts per prompt in GRPO	distinct from GQA group size G ; context disambigua
$\hat{A}_i$	Group-normalized advantage for response i	GRPO: $(r_i - \mu_R) / \sigma_R$
y_w, y_l	Preferred / rejected response in a preference pair	DPO/RLHF notation

Hardware and Performance

Symbol	Meaning	Notes
BW	Memory bandwidth (HBM)	GB/s or TB/s
MFU	Model FLOPs Utilization	achieved FLOP/s $\div$ peak FLOP/s; target ≥ 0.4
MBU	Memory Bandwidth Utilization	achieved BW $\div$ peak BW
P_{attn}	Parameters in attention layers	$\approx 4d^2$ per layer (MHA)
P_{mlp}	Parameters in MLP layers	$3 d d_{\text{ff}}$ per layer (SwiGLU)
B_r, B_c	FlashAttention tile (block) sizes for rows and columns	SRAM tile dimensions
A_k, B_k	Left and right matrix tiles in tiled GEMM	
t_{wave}	Time to execute one SM wave	GPU scheduling unit
N_{total}	Total expert count in MoE	
N_{active}	Active experts per token in MoE	$N_{\text{active}} \ll N_{\text{total}}$

Alignment and Post-Training

Symbol	Meaning	Notes
$D_{\text{KL}}(\cdot \parallel \cdot)$	Kullback-Leibler divergence	
$D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$	Policy drift from reference	regularization term in RLHF/DPO
σ	Standard deviation; also sigmoid activation	context disambiguates
p, q	Generic probability distributions	
x	Input token sequence	
i, j, k	Generic indices	

Common Abbreviations

Term	Meaning
MHA	Multi-Head Attention
GQA	Grouped-Query Attention
MQA	Multi-Query Attention (GQA with $N_{kv} = 1$)
MLA	Multi-Head Latent Attention
KV cache	Key-Value cache stored between decode steps
HBM	High-Bandwidth Memory (GPU DRAM)
SRAM	Static RAM; on-chip shared memory / L1 cache
SM	Streaming Multiprocessor (GPU compute unit)
FLOP	Floating-point operation (one multiply-add = 2 FLOPs)
MFU	Model FLOPs Utilization
TP	Tensor Parallelism (intra-layer, intra-node)
PP	Pipeline Parallelism (inter-layer, inter-node)
DP	Data Parallelism
ZeRO	Zero Redundancy Optimizer (stages 1-3)
FSDP	Fully Sharded Data Parallelism (ZeRO stage 3 in PyTorch)
SFT	Supervised Fine-Tuning
RLHF	Reinforcement Learning from Human Feedback
DPO	Direct Preference Optimization
PPO	Proximal Policy Optimization
GRPO	Group Relative Policy Optimization
RLVR	Reinforcement Learning from Verifiable Rewards
LoRA	Low-Rank Adaptation
QLoRA	Quantized LoRA
PEFT	Parameter-Efficient Fine-Tuning
MoE	Mixture of Experts
CoT	Chain of Thought
BF16	Brain Float 16 (1 sign, 8 exponent, 7 mantissa bits)
FP16	IEEE Float 16 (1 sign, 5 exponent, 10 mantissa bits)
INT8	8-bit integer quantization
SLO	Service Level Objective
QPS	Queries per second

Conventions

- **Tokens vs. sequences.** “Tokens” refers to individual vocabulary elements; “sequence” or “context” refers to an ordered list of tokens. Batch size B counts sequences, not tokens; $N_T = B \times T$ counts tokens.
- **FLOPs counting.** One multiply-accumulate (MAC) operation = 2 FLOPs. The formula $C_{\text{fwd}} \approx 2N_T P$ follows this convention. “FLOP/s” (floating-point operations per second) uses the same factor.
- **Memory units.** 1 GB = 10^9 bytes throughout (SI prefix, not binary). Bandwidth is reported in GB/s or TB/s.
- **Latency units.** Milliseconds (ms) for per-token and per-request latencies; seconds (s) for end-to-end generation time; microseconds (μs) for kernel-level timings.
- **Throughput units.** Tokens per second (tok/s or tokens/s) for generation throughput; queries per second

(QPS) for request-level throughput.

- **Precision notation.** Memory footprints assume bf16 (2 bytes/parameter) unless otherwise stated. The training-memory figure of $16P$ bytes assumes AdamW in fp32.
- **Prefill vs. decode.** Prefill encodes the full prompt in one parallel forward pass (compute-bound). Decode generates tokens one at a time in an autoregressive loop (memory-bandwidth-bound). These two phases have distinct bottlenecks and optimizations.
- **Latency vs. throughput.** TTFT optimizes differently from aggregate TPS. Minimize TTFT with small batches and fast prefill hardware; maximize TPS with large batches and high memory bandwidth. A single serving configuration cannot simultaneously optimize both without workload-specific scheduling.
- **Overloaded symbols.** G denotes both the GQA group size and the GRPO rollout count; T denotes both sequence length and training token count; β denotes both the Adam second-moment decay and the KL penalty coefficient. In each case, context and the surrounding equation make the intended meaning unambiguous.

Acknowledgments

A book about systems is itself a system, and this one had many contributors.

The technical content in this book was shaped by countless conversations with engineers and researchers working on real production systems. Several people gave detailed feedback on draft chapters, caught errors in derivations, and pointed out places where the framing did not match how the work actually gets done in practice.

A number of colleagues reviewed early drafts and offered both technical corrections and perspective on what candidates actually encounter in interviews at frontier labs. Their input made the interview framing in Part I and the end-to-end drills in Part XI significantly sharper.

The open-source research community deserves specific acknowledgment. The derivations in this book stand on the published work of the teams behind FlashAttention, the Chinchilla scaling law study, the DeepSeek architecture and training reports, the LLaMA model series, the Mixtral MoE architecture, the RLHF and DPO preference alignment papers, and the GRPO and RLVR training algorithms. Each of these is cited in the relevant chapter and listed in Appendix D. The existence of detailed technical reports from frontier labs—a relatively recent norm—made it possible to ground the book’s engineering content in actual production practice rather than academic approximations.

Finally, thank you to everyone who read early versions and offered encouragement at the moments when the project was hardest to continue. You know who you are.

About the Author

I am Hao Hoang, an applied AI/ML engineer and technical writer. I am based in Los Angeles, California, and I am originally from Quang Tri, Viet Nam.

I own *AI Interview Prep* and write for the community every day on Substack and on LinkedIn. On Substack I publish *Daily AI Interview Questions* and longer notes on LLM system design, reinforcement learning, vision-language models, RAG-style retrieval, and other high-signal topics for engineers and researchers preparing for rigorous AI interviews. On LinkedIn I share the same thread of ideas in shorter, daily posts so people can follow along between editions. That public writing reaches more than 55,000 LinkedIn followers and 12,000 Substack readers, and it is the main place I teach, learn in public, and answer what the community is asking about.

About this book. I am the sole author: I researched and wrote every chapter myself, without co-authors or ghostwriters. I wrote it because I wanted a single, opinionated reference for the patterns I kept having to re-derive when preparing for interviews and when shipping LLM systems in production-at the intersection of research, systems engineering, and deployment.

You can reach me at:

- Email: hao.hoang.ai@gmail.com
- Website: <https://haohoang.is-a.dev/>
- Substack: <https://aiinterviewprep.substack.com/>
- LinkedIn: <https://www.linkedin.com/in/hoang-van-hao>

I publish daily posts on LinkedIn and the newsletter at <https://aiinterviewprep.substack.com/>. Errata or corrections are welcome at the email above.

Part I

Overview and Interview Landscape

Chapter 1 The LLM Systems Interview

1.1 What Companies Actually Ask

1.1.1 Design-an-LLM-System Questions

You sit down across from the interviewer. After two minutes of introductions, the question lands: “Design a serving system for a code-completion product with p95 time-to-first-token under 200 milliseconds.” You have 45 minutes. This is the open-ended design question, and it is the center of gravity for senior and staff-level LLM system roles.

Design questions ask you to build an LLM system end to end: from hardware budget to serving scheduler, from a one-line product requirement to a set of motivated architectural choices. They are open-ended on purpose. The interviewer is not looking for a canonical architecture; they are watching how you structure ambiguity, which constraints you surface first, and whether your decisions are driven by actual hardware physics or by pattern-matching to things you have read.

The question has three phases that map directly to the framework in Chapter 2. First, you clarify requirements: TTFT SLO, throughput target, quality bar, and cost ceiling—because the answer to “which model size?” and “which parallelism strategy?” changes completely depending on these. Second, you sketch the end-to-end pipeline: tokenization, prefill (compute-bound, KV cache written), decode (memory-bandwidth-bound, KV cache read each step), and the serving scheduler that batches concurrent requests. Third, you identify the binding constraint—almost always KV cache pressure rather than raw FLOPs—and propose optimizations as targeted responses to that constraint rather than as a list of things you have memorized.

Staff-level and principal-level design questions add a second layer: the interviewer changes a constraint mid-answer. “What if context length doubles?” “What if you need to serve five model variants behind the same endpoint?” “What if cost-per-token becomes the primary SLO?” A candidate who derived their numbers from first principles adapts in seconds. A candidate who memorized an architecture has to start over. This is why the framework matters more than any specific configuration.

The vocabulary these questions test is introduced in Chapter 2 and derived in Parts IV through VII: arithmetic intensity, memory-bandwidth-bound versus compute-bound regimes, KV cache footprint per sequence per layer, batch-size-versus-latency trade-off, continuous batching, speculative decoding, grouped-query attention.

Every strong answer to a design question is derivable from the constraints on the whiteboard; the interviewer can always verify this by changing one constraint and watching what happens.

How This Shows Up

- **“Design a production serving stack for a 7B chat model targeting p95 TTFT under 300 ms at 5,000 QPS.”** A strong answer opens with requirement clarification (interactive or batch use? single or multi-tenant?), sketches prefill and decode as separate boxes with different bottlenecks, estimates KV cache memory at a target batch size, identifies the batch-size constraint, and proposes continuous batching before discussing any optimization. Chapter 2 covers the framework; Chapters 15 and 18 cover the arithmetic and serving optimizations.
- **“Design the training infrastructure for a 70B dense model on a 512-GPU cluster.”** A strong answer sizes the memory budget (parameters, gradients, optimizer states), picks a parallelism combination jus-

tified by the hardware topology (tensor-parallel within node, pipeline-parallel or ZeRO-3 across nodes), estimates the pipeline bubble, and names where batch size is being consumed as a resource. Chapter 12 covers parallelism strategies; Chapter 13 covers the FLOP and memory arithmetic.

Key Takeaways

- Open every design answer with requirements-TTFT SLO, throughput target, quality bar, cost ceiling-before naming any component.
- In serving, the binding constraint is almost always KV cache memory pressure, not FLOPs; in training it is usually communication overhead.
- Interviewers change constraints mid-answer to distinguish first-principles reasoning from memorized configurations-derive, do not recall.
- Chapter 2 is the procedural skeleton; the technical chapters (Parts IV-VII) provide the arithmetic to fill it in.

1.1.2 Deep-Dive Component Questions

Twenty minutes into the design question-or sometimes as the entire interview-the interviewer pivots: “Let’s go deeper on speculative decoding. Walk me through the acceptance criterion. Why does that sampling procedure give you an exact sample from the target distribution?” This is the deep-dive format: narrow, mechanism-level, and unforgiving about gaps between vocabulary and understanding.

Deep-dive questions test whether you understand the mechanism, not just the outcome. “Grouped-query attention reduces the KV cache” is something you can read in an abstract. “GQA replaces N independent key-value head pairs with G shared groups, reducing KV cache memory by a factor of N/G , which allows a proportionally larger batch size on the same hardware” is what the question is actually asking for. The follow-up will probe the second-order consequence: a larger batch size raises arithmetic intensity in the MLP layers toward the compute-bound regime-while attention remains memory-bandwidth-bound regardless-so GQA changes where the bottleneck sits at different batch sizes.

The components that appear most often in deep dives map directly to the technical parts of this book:

- **Architecture details:** attention variants (GQA, MLA, sliding-window attention), MoE routing, normalization and positional embedding choices-Parts II and III.
- **GPU and kernel mechanics:** memory hierarchy, arithmetic intensity, operator fusion, FlashAttention-Part IV.
- **Parallelism:** all-reduce versus all-gather, ZeRO stages and their communication cost, pipeline bubbles, tensor versus pipeline trade-offs-Part V.
- **Inference systems:** KV cache sizing, continuous batching, speculative decoding acceptance criterion, quantization precision tiers and their accuracy risk-Part VII.
- **Evaluation:** benchmark contamination, evaluation harness design, metric selection for instruction-following and reasoning tasks-Part IX.

The depth expectation scales with seniority. A junior screen asks for a correct description of the mechanism. A senior screen asks for the trade-off and the failure mode. A staff or principal screen asks for the case where the standard answer is wrong-and how you would detect it before shipping. Calibrate your answer to the level you are interviewing for; going too shallow loses the offer, but going deep and then being wrong about the second-order consequence is worse.

A deep-dive answer has two moves: state the mechanism precisely, then name the second-order consequence that the mechanism creates or removes.

How This Shows Up

- **“Explain why token generation is memory-bandwidth-bound even at large batch sizes.”** A strong answer works from arithmetic intensity: each decode step processes one new token per sequence, collapsing the weight matrix multiplications to matrix-vector products with intensity proportional to batch size divided by hidden dimension—a number almost always below the GPU’s compute-bandwidth ridge point regardless of batch size. The derivation lives in Chapter 15.
- **“How does ZeRO stage 3 differ from ZeRO stage 1 in memory savings and communication cost?”** A strong answer traces what is sharded at each stage (optimizer state at stage 1, gradients at stage 2, parameters at stage 3), quantifies the communication increase (roughly $3\times$ parameter bytes per step at stage 3 versus $1\times$ at stage 1), and notes that overlapping the all-gather with the forward pass is what makes stage 3 viable at scale. Chapter 12 covers this in full.

Key Takeaways

- Deep dives test mechanism, not vocabulary: “it reduces memory” is the opening of an answer, not the answer.
- State the second-order consequence—what the mechanism enables, constrains, or shifts—to move from a passing answer to a strong one.
- Seniority raises the bar from correct description to trade-off to failure mode; calibrate accordingly.
- The technical chapters in Parts II–IX are organized around the component questions that appear most frequently in frontier-lab interviews.

1.2 Roles That Require LLM Systems Knowledge

1.2.1 LLM Infrastructure and Training Engineer

Picture the engineer who gets paged because a 512-GPU training run has stalled at step 47,000. The job is not to design a new architecture; it is to figure out why GPU 203 is 40% slower than the rest—whether the culprit is a straggling all-reduce, a pipeline bubble, or a dead NVLink lane. This role exists at every frontier lab and most serious AI infrastructure teams, and its interview goes deep into hardware and distributed systems.

The infra and training engineer owns the layer between model code and the cluster. Their primary vocabulary is hardware topology—NVLink bandwidth within a node versus InfiniBand across racks—collective-communication primitives (all-reduce, all-gather, reduce-scatter), and the three axes of model parallelism: data, tensor, and pipeline.

No single parallelism strategy dominates. Data parallelism scales compute linearly but does nothing for per-GPU memory; ZeRO stage 3 (FSDP) reclaims memory by sharding parameters and optimizer states across workers, but it raises total communication cost to roughly three times the parameter count per step. Tensor parallelism demands ultra-fast intra-node links and is practical only up to the number of GPUs per machine—typically eight—because going further triggers the slower inter-node fabric. Pipeline parallelism crosses machines cheaply but introduces a bubble proportional to stages divided by micro-batches. Production runs combine all

three: tensor-parallel within a node, pipeline-parallel or FSDP across nodes, data-parallel at the outermost level. We work through the arithmetic behind each strategy in Part III.

Batch size is not just a hyperparameter here-it is a scarce resource. Both data parallelism and pipeline parallelism consume it, so a candidate who can articulate that trade-off signals genuine systems fluency. The other half of the role is resilience: silent GPU corruption, checkpoint cadence, and automatic-restart logic. At scale, hardware failures are routine, not exceptional.

The training engineer's job is to convert cluster-hours into useful gradient steps, not communication stalls or idle GPUs.

How This Shows Up

- **“How would you distribute a 70B-parameter model across a 64-GPU cluster for training?”** A strong answer picks a parallelism combination, justifies it by the memory and bandwidth constraints of the hardware (8-GPU nodes connected by NVLink internally, InfiniBand across nodes), estimates the bubble or bandwidth overhead, and names where batch size is being spent as a resource. The derivations live in Part III.
- **“Walk me through how FSDP reduces optimizer-state memory compared to naive DDP.”** A strong answer traces the three ZeRO stages-shard optimizer state (stage 1), then gradients (stage 2), then parameters (stage 3)-explains why stage 3 raises communication to three times the parameter count, and notes that the overlap of communication with computation is what makes it viable in practice. Chapter 8 covers this in full.

Key Takeaways

- Parallelism is a combination problem: tensor-parallel within a node, pipeline-parallel or ZeRO-3 across nodes, data-parallel at the outermost level.
- Batch size is a scarce resource that both data parallelism and pipeline parallelism consume; surfacing this trade-off signals systems maturity.
- Communication is the bottleneck, not compute: on modern clusters, the training engineer reasons about memory bandwidth and collective-communication cost more than raw FLOP counts.
- Hardware failures at scale are routine-checkpoint strategy and fault tolerance are not afterthoughts.

1.2.2 LLM Applied/Product Engineer

The applied engineer takes a trained or fine-tuned model and turns it into a product that ten thousand users hit every minute. Their interview is lighter on distributed-training arcana and heavier on inference economics: how fast does the model respond, how much does it cost to serve, and what breaks first when traffic spikes?

The core abstraction the applied engineer must own is the two-phase structure of LLM inference: prefill (encoding the prompt in parallel, compute-bound) and generation (emitting tokens one by one, memory-bound). During generation, each step reads all model parameters plus the entire KV cache from high-bandwidth memory; arithmetic intensity drops to roughly one FLOP per byte regardless of batch size, because every sequence maintains its own KV state. That ceiling is set by memory bandwidth, not the GPU's peak FLOP count. We derive this bound in Chapter 9.

From that ceiling follow all practical serving decisions. To improve throughput, you batch requests together-larger batches amortize the parameter-read cost across more tokens, pushing arithmetic intensity toward the

compute-bound regime. To reduce memory pressure, you shrink the KV cache: grouped-query attention uses fewer key-value heads, multi-head latent attention projects keys and values into a lower-dimensional space, and local attention restricts each layer to a sliding context window. Quantization (FP8 or INT4 weights) cuts parameter-read time directly. Speculative decoding uses a cheap draft model to generate candidate tokens that the large model verifies in one parallel prefill pass—since prefill is fast and parallel, this can nearly double effective generation throughput with no accuracy loss.

The applied engineer also needs fluency in batching dynamics under live traffic: continuous batching so the GPU never idles waiting for a fixed batch to fill, KV cache eviction policies when memory is exhausted, and the latency-throughput curve that determines where to set serving SLOs for a given user experience target.

Inference at scale is a memory-bandwidth problem wrapped in a queueing problem.

How This Shows Up

- **“Design a serving system for a 7B chat model that needs p95 latency under 500ms at 5,000 QPS.”** A strong answer estimates peak memory (parameters plus KV cache at the target batch size), checks whether the system is memory-bound or compute-bound at that batch size, proposes horizontal scaling or model sharding if a single GPU cannot hold the load, and discusses continuous batching. The quantitative framework lives in Chapter 9.
- **“What are the trade-offs between serving a 70B model in FP16 versus INT4?”** A strong answer covers: memory reduction (roughly $4\times$ versus the FP16 baseline), latency improvement proportional to memory reduction in the memory-bound regime, and accuracy risk driven by outlier activations in large models. Chapter 10 covers quantization schemes in depth.

Key Takeaways

- Generation is memory-bound: the bottleneck is reading weights and KV cache from HBM, not performing FLOPs.
- KV cache dominates serving memory; shrinking it via GQA, quantization, or local attention is the highest-leverage optimization.
- Latency and throughput trade against each other: smaller batches favor latency, larger batches favor throughput—know where your product’s SLO sits on that curve.
- Speculative decoding can double throughput at no accuracy cost; it is the canonical example of exploiting the asymmetry between prefill speed and generation speed.

1.2.3 Research Engineer vs. Research Scientist

At a small startup these roles blur into one person who trains models and writes papers the same week; at a frontier lab they can be two distinct career tracks with different interview loops and different hiring bars. Getting the distinction wrong—showing up to a research engineer screen expecting research scientist questions, or vice versa—is a fast path to a misaligned conversation.

The research engineer (RE) is an engineer first. Their core job is to implement, debug, and scale research ideas: modify training loops, wire up new data pipelines, profile GPU utilization, and ensure that an experiment proposed by a scientist actually runs correctly on a thousand-GPU cluster. The RE interview tests implementation depth—can you trace through FSDP, explain why a training loss spiked at step 30,000, or spot a numerical

instability in a custom kernel? Systems judgment matters: the RE who cannot reason about memory budgets and parallelism trade-offs will block every experiment they try to run.

The research scientist (RS) is a researcher first. Their core job is to generate hypotheses about what might make models better-new architectures, training objectives, data mixtures, alignment strategies-and to design experiments that test those hypotheses rigorously and efficiently. The RS interview tests scientific judgment: can you critique an ablation design, identify confounders, propose a controlled experiment to isolate the effect of a curriculum-learning schedule, or interpret a result that partially contradicts your hypothesis? Quantitative rigor matters: the RS who cannot design a fair comparison or read a scaling-law plot will produce papers that do not hold up.

Both roles require LLM systems fluency, just at different depths. An RE who cannot reason about inference latency or training parallelism will be blocked on every implementation. An RS who cannot read a profiler trace or estimate memory budgets will propose experiments that are not runnable. The technical chapters in this book-on parallelism, inference, and evaluation-are directly relevant to both: the RE needs implementation-level depth, the RS needs enough to communicate clearly with the engineering team and to scope experiments that fit within a given compute budget.

The interview distinguishes RE from RS by asking whether you built the system or designed what the system should test.

How This Shows Up

- **“Tell me about a training run you debugged from first symptom to resolution.”** This is a research engineer question. A strong answer traces symptoms (loss spike, low GPU utilization, OOM), lists hypotheses, describes diagnostic steps (profiler, log inspection, checkpoint bisection), and explains the fix. Vague or high-level answers signal limited hands-on implementation exposure.
- **“Propose an experiment to determine whether grouped-query attention hurts model quality on long-context tasks.”** This is a research scientist question. A strong answer identifies confounders (training sequence length, model size, evaluation metric), defines a clean baseline, justifies the compute budget, and discusses what a null result would mean. Chapter 5 covers grouped-query attention; Chapter 12 covers evaluation design.

Key Takeaways

- RE interviews test implementation depth: profiling, debugging, and hands-on experience with distributed training and inference systems.
- RS interviews test research judgment: experimental design, hypothesis generation, and interpreting ambiguous or contradictory results.
- Both roles require LLM systems fluency; the difference is depth of implementation versus depth of scientific reasoning.
- If you are unsure which loop you are in, ask the recruiter before preparing-the prep is genuinely different.

Chapter 2 How to Approach an LLM System Design Question

2.1 A Repeatable Framework

2.1.1 Clarify Requirements (Latency, Throughput, Quality, Cost)

The interview prompt arrives-“design an LLM-powered customer support system.” Before a single component is named, a strong candidate pauses and asks four questions. At the senior and staff level, interviewers watch specifically for this pause; candidates who skip it and dive straight into architecture are signaling that they have memorized solutions rather than learned to think.

The four axes that every LLM system lives on are latency, throughput, quality, and cost. These are not platitudes; each has a concrete definition that will drive every downstream decision you make.

Latency has two distinct flavors in LLM serving. Time-to-first-token (TTFT) measures how long a user waits before any output appears-this is dominated by the prefill step, which processes the full prompt in one parallel forward pass and is compute-bound. Per-token latency (decode latency) measures how quickly subsequent tokens arrive-this is dominated by the autoregressive decode loop, which is memory-bandwidth-bound because model weights and the KV cache must traverse the memory hierarchy once per generated token. Conflating TTFT with decode latency is one of the most common interview errors; they respond to entirely different optimizations. We derive the arithmetic behind both in Chapter 7.

Throughput measures tokens generated per second across all concurrent users, and it is the number that determines your inference bill at scale. Higher throughput generally requires larger batch sizes, which increases per-request latency. This tension-batching as the lever between latency-optimized and throughput-optimized operating points-is the central trade-off in LLM serving.

Quality sets the floor below which no amount of speed matters. Quality requirements dictate model size, which sets the memory footprint, which in turn constrains the batch size achievable on a given hardware budget. A requirement for GPT-4-class output quality effectively forces you onto a large model and limits your throughput options from the start.

Cost ties the other three together. A 70B model serving a 200 ms TTFT SLO on H100s implies a fundamentally different cost envelope than a 7B model with a two-second tolerance. Surfacing cost early prevents designs that are technically correct but operationally unshippable.

Interviewers want to hear you name these four axes and immediately note which ones are in tension for the specific scenario given. *Clarifying requirements is not housekeeping-it is the first technical decision, because every architecture choice is a trade-off between latency, throughput, quality, and cost.*

How This Shows Up

- **“We’re building a real-time coding assistant. Walk me through how you’d design the serving stack.”**
A strong answer first asks: What is the TTFT SLO? Is usage interactive or batch? What quality bar? The answer skeleton names TTFT (sub-200 ms is typical for autocomplete), per-token latency, and concurrent user count before touching any component. The technical depth for meeting those SLOs lives in Chapter 7.
- **“How would you size the inference fleet for a new chat product launching with 10M daily active users?”**
A strong answer immediately converts DAU to peak requests per second, estimates average

prompt and completion lengths to derive a tokens-per-second demand figure, then works backward to a hardware configuration. Candidates who jump to “use an H100” without this derivation fail the follow-up.

Key Takeaways

- Ask for the latency SLO before naming any architecture; TTFT and per-token decode latency are separate problems with separate solutions.
- Throughput and per-request latency trade against each other via batch size-name this tension explicitly and early.
- Quality requirements set the model size, which sets the memory budget, which constrains achievable batch size and throughput.
- Cost is the constraint that forces a choice between the above; surfacing it in the requirements phase prevents designs that cannot be shipped.

2.1.2 Sketch the End-to-End Pipeline

After requirements are on the board, the next thing an interviewer watches for is whether you can draw the right boxes in the right order. Many candidates jump immediately to optimizations-quantization, speculative decoding, model compression-before they have established the baseline pipeline. That inversion signals pattern-matching, not understanding; experienced interviewers catch it within the first minute.

Every autoregressive LLM serving system has two distinct computational phases, and your whiteboard diagram must show both before you say anything else.

The **prefill** phase receives the raw prompt tokens and processes them in a single forward pass through the model. Because all tokens are known upfront, this pass is fully parallelizable across the sequence dimension-structurally identical to a training-time forward pass. Prefill is compute-bound: given a sufficiently long prompt, the arithmetic intensity is high enough to saturate the GPU’s matrix-multiply units. The outputs of prefill are two things: the next-token logit (from which the first generated token is sampled) and the **KV cache**-the key and value tensors for every attention layer, written to GPU memory so that decode steps do not recompute them from scratch.

The **decode** phase generates the response token by token. Each step takes the previously generated token, runs it through the full model, reads the accumulated KV cache from high-bandwidth memory, and samples the next token. Because only one new token is processed per step, the matrix multiplications against the model weights reduce to matrix-vector products-the worst-case scenario for arithmetic intensity. Decode is memory-bandwidth-bound: speed is gated by how fast model weights and KV cache can be streamed from HBM, not by the GPU’s compute throughput. We derive the exact bound in Chapter 7.

The sketch should also include the **serving layer**: a scheduler that accepts incoming requests, forms batches (continuous batching is the industry standard), routes them to prefill and decode workers, and manages KV cache memory. The KV cache is the resource that sets the effective batch size ceiling; its size grows linearly with sequence length, number of attention layers, and number of KV heads. Interviewers at frontier labs expect you to know that KV cache pressure-not FLOPs-is usually the binding constraint on how many concurrent requests you can serve. Chapter 8 covers KV cache sizing arithmetic in full.

Once the baseline pipeline is on the board, you can layer in optimizations in priority order: architecture choices that shrink the KV cache (grouped-query attention, multi-head latent attention), quantization to reduce memory bandwidth pressure, and speculative decoding to hide decode latency. Presenting optimizations before

the pipeline is the canonical pitfall; presenting them as motivated responses to the constraints you have just identified is what separates candidates who have built these systems from those who have only read about them.

Draw prefill and decode as separate boxes with separate performance characteristics before you say a single word about optimization.

How This Shows Up

- **“Walk me through what happens when a user sends a prompt to a production LLM API.”** A strong answer covers tokenization, prefill (compute-bound, KV cache written to HBM), the autoregressive decode loop (memory-bound, KV cache read each step), detokenization, and the serving scheduler that batches concurrent requests. Technical depth on each stage lives in Chapters 7 and 8.
- **“Where would you look first if users were reporting high latency?”** A strong answer immediately asks which latency-TTFT or per-token-then diagnoses accordingly: high TTFT points to a prefill bottleneck (prompt too long, batch too large, insufficient compute provisioning), while high per-token latency points to a decode bottleneck (KV cache pressure, memory bandwidth saturation, or a batch size that exceeds what the memory hierarchy can sustain).

Key Takeaways

- Prefill (compute-bound) and decode (memory-bound) are distinct phases with different bottlenecks; always draw them as separate boxes.
- The KV cache is the bridge between phases and the primary memory consumer; sizing it is a required step in any capacity plan.
- Continuous batching at the serving layer is the standard mechanism for maximizing GPU utilization across requests with varying lengths and arrival times.
- Present the baseline pipeline first, then optimizations as targeted responses to specific constraints-sequencing signals fluency to interviewers.

2.2 Trade-Off Axes You Must Name

2.2.1 Compute vs. Memory vs. Bandwidth

A design interview turns on the moment you identify the real bottleneck. Most engineers, when asked how to speed up a serving endpoint, jump to “add more GPUs” or “use a smaller model” - both potentially valid, but both bypassing the diagnostic question that unlocks everything else: is the system limited by arithmetic throughput, memory capacity, or memory bandwidth? Those three constraints behave differently, respond to different interventions, and show up in different parts of the stack. Conflating them is one of the most common ways to lose credibility in a systems round.

A GPU has two fundamental performance ceilings. The first is compute throughput, measured in floating-point operations per second. The second is memory bandwidth, measured in bytes per second. Any given operation sits in one of two regimes: *compute-bound*, meaning the arithmetic units are the bottleneck, or *memory-bandwidth-bound*, meaning the time is dominated by streaming data through HBM. The ratio of these two hardware numbers - roughly 300 FLOPs per byte on a current-generation data-center GPU - defines a ridge point.

Operations with arithmetic intensity above this ridge are compute-bound; operations below it are memory-bound no matter how many FLOPs they perform. We derive this number precisely in Chapter 8.

Matrix multiplications at large batch sizes land firmly above the ridge. That is why training and prefill are fast: the GPU's multiplier arrays stay saturated. Token generation is the mirror image. Each decode step processes one new token, which reduces the weight-matrix multiply to a matrix-vector product - arithmetic intensity close to 1 FLOPs/byte, deep in memory-bound territory. This is the root cause of slow generation, and it is structural: no amount of faster arithmetic fixes a problem caused by HBM bandwidth. The intervention has to touch memory movement.

Memory capacity is a third, distinct constraint. KV cache grows with batch size and sequence length; once it exhausts VRAM, you cannot increase the batch regardless of how much arithmetic headroom remains. Techniques like grouped-query attention and quantization are as much about fitting more sequences into a fixed memory budget as they are about raw speed.

When a system feels slow, the first question is always: which ceiling are we hitting? The answer determines whether the fix is a kernel rewrite, a memory layout change, or an architectural trade-off.

How This Shows Up

- “GPU utilization is at 80% but token generation is still slow. Where would you look first?” A strong answer distinguishes compute utilization from memory bandwidth utilization. High compute utilization during generation just means the GPU is busy waiting on HBM reads - the real bottleneck is bandwidth, and the path to improvement is reducing memory traffic, not adding FLOPs. The technical depth for this lives in Chapter 9.
- “A normalization layer consumes 25% of wall-clock time but less than 1% of total FLOPs. How do you speed it up?” The answer requires recognizing that normalization is memory-bandwidth-bound, so FLOP-reduction techniques are irrelevant; the leverage is in cutting parameter loads per call (RMSNorm over LayerNorm) and fusing kernel launches to eliminate round-trips to HBM.

Key Takeaways

- Every LLM operation is either compute-bound or memory-bandwidth-bound; the boundary is set by the hardware's arithmetic intensity ridge point, not by the number of FLOPs.
- Training and prefill are compute-bound at practical batch sizes; token generation is memory-bandwidth-bound by construction - these require different optimization strategies.
- Memory *capacity* (fitting the model and KV cache in VRAM) is a third, separate constraint that limits batch size independently of bandwidth or compute.
- TFLOP ratings sell GPUs; memory bandwidth and capacity are the operative constraints for inference engineering.

2.2.2 Quality vs. Latency vs. Cost

Every LLM serving decision eventually compresses to three numbers: how good the output is, how fast it arrives, and how much it costs per token. In practice, you can aggressively optimize two of these; the third tends to move against you. Interviewers at frontier labs want to see that you have internalized this triangle before you draw a single box in your architecture diagram - because candidates who skip it waste fifteen minutes proposing solutions that are technically interesting but wrong for the stated problem.

Latency and throughput are not the same metric, and conflating them is the most common error in LLM serving discussions. Latency is the wall-clock time a single user waits - time-to-first-token (TTFT) for the prompt-encoding phase, then per-token decode latency during generation. Throughput is aggregate tokens generated per second across all concurrent users. These optimize in opposite directions: maximizing throughput requires large batch sizes and full GPU saturation; minimizing latency often means small batches, aggressive prefetching, and accepting lower utilization. A system cannot simultaneously be optimal at both without workload-specific scheduling that separates the two traffic classes.

Quality is the axis that is easiest to accidentally sacrifice. Quantizing weights from BF16 to INT4 can triple throughput; it also measurably degrades accuracy on multi-step reasoning, often in ways that aggregate benchmarks do not surface. Grouped-query attention reduces the KV cache by a factor of eight with negligible benchmark loss - but “negligible on standard benchmarks” is a claim about a distribution of tasks, not a guarantee about your specific workload. Part IX covers how to design evaluations that catch these gaps before they reach production.

Cost is the integrating metric. It is not the same as latency, though the two are correlated. A technique that saves 30% of memory per sequence can double the batch size that fits on a single GPU, halving per-token cost - a much larger lever than shaving a few milliseconds off the decode loop. The arithmetic for a specific model and hardware configuration is in Chapter 15; the key habit is to always convert architectural decisions into a per-token-cost estimate before committing to them.

The vocabulary for this axis in interviews: TTFT (latency of the prefill phase, compute-bound, difficult to compress beyond model size), decode latency (per-token, memory-bandwidth-bound, the target of most serving optimizations), throughput (tokens/second aggregate, optimized by batching and hardware utilization), and cost-per-token (the business metric that synthesizes all three). Speculative decoding (Chapter 18) is notable precisely because it improves latency without degrading quality - an exact sample from the target distribution at draft-model speed when the acceptance rate is high.

Name the metric you are optimizing before you propose a solution; an interviewer who has to ask “are you optimizing latency or throughput?” has already registered a gap in your systems thinking.

How This Shows Up

- “We are building a real-time coding assistant. What serving configuration would you recommend?” A strong answer names TTFT and per-token decode latency as the primary metrics, not aggregate throughput; proposes small batch sizes or continuous batching with tight SLOs; discusses the quality floor before deciding quantization aggressiveness. The tradeoff details live in Chapter 19.
- “How would you serve the same 70B model to both an interactive product and an overnight batch pipeline?” A strong answer proposes separate serving paths: the interactive path optimizes latency (smaller batches, possibly speculative decoding), the batch path optimizes throughput and cost (larger batches, more aggressive quantization, relaxed TTFT SLO). A single serving configuration that tries to satisfy both will be suboptimal on both.

Key Takeaways

- Latency and throughput optimize in opposite directions; always establish which one your design is targeting before proposing anything.

- Quality degradation from quantization and architectural shortcuts is real and workload-dependent - aggregate benchmark numbers are an unreliable proxy for production quality on specific tasks.
- Cost-per-token is the integrating metric: batch size, quantization, and model architecture all map onto it differently, and the largest levers are usually in batch size and memory efficiency rather than kernel micro-optimization.
- TTFT, decode latency, and throughput are three distinct quantities; using them interchangeably signals a gap in serving systems understanding.

2.3 Common Pitfalls in Answers

2.3.1 Cargo-Culting Numbers Without Derivation

A candidate writes “140 GB” next to their 70B model box, the interviewer nods, and then asks where that number comes from. The candidate pauses. This moment - memorized figure, no derivation - is one of the most reliable separators between senior candidates and everyone else in LLM system design interviews. It is worse than not citing the number at all, because it signals that you treat estimates as incantations rather than tools.

Virtually every number worth citing in an LLM system interview flows from four first-principles chains: bytes per parameter at a stated dtype, the $6ND$ approximation for total training FLOPs, the per-token KV cache footprint from layer count and key-value head geometry, and arithmetic intensity as the ratio of FLOPs to memory bytes transferred. These four starting points, combined with the hardware specs on the whiteboard, reproduce almost any published figure in under a minute of algebra. Candidates who own the derivation can update their answer the instant an interviewer changes one input; candidates who memorized the output cannot.

The Chinchilla 20-to-1 token-to-parameter ratio is the most common cargo-cult target. Production scaling teams at multiple major labs have re-derived this number and obtained answers that range from roughly 40-to-1 to nearly 200-to-1, depending on data quality, architecture efficiency, and the serving economics of the final model. Those teams do not inherit a ratio from a paper; they build small proxy models, vary compute allocation across isoFLOP slices, fit the minimum, and read the answer. The derivation lives in Chapter 6; the interview expectation is that you know the analysis exists, can name the variables that shift the answer, and are not surprised when the answer differs from 20.

The danger of a memorized number is not that it is wrong - it is often approximately right. The danger is that you have no idea which assumptions it rests on, so the moment the interviewer changes one you are stuck. “How does your memory estimate change if you switch to FP8 inference?” “What if you add grouped-query attention with eight key-value heads?” “What if context length doubles?” Each of these invalidates a memorized answer while posing no problem for a derived one.

A number you can derive is worth ten you have memorized.

How This Shows Up

- **“Estimate the memory footprint of a 70B model during inference with a batch of 32 and a 4,096-token context.”** A strong answer starts from bytes per parameter at the stated dtype, then adds the KV cache term - proportional to batch size, context length, number of key-value heads, and layer count - and shows which term dominates. It also notes that switching from BF16 to FP8 halves the parameter footprint but does not change the KV cache unless the cache itself is quantized, and that grouped-query attention directly reduces the second term. The derivation for this lives in Chapter 8.

- **“How many H100s do you need to train a 7B model on 1 trillion tokens in two weeks?”** A strong answer derives total training FLOPs from the $6ND$ approximation, converts to GPU-hours using a realistic model FLOP utilization (35-50% is typical in practice), and arrives at a card-count before ever citing a benchmark number. Chapter 5 derives the $6ND$ rule and calibrates MFU against published runs. The technical depth for both estimates lives in Chapter 5 (training arithmetic) and Chapter 8 (inference arithmetic).

Key Takeaways

- Know the derivation of three core estimates: parameter memory at a stated dtype, training FLOPs from model size and token count, and KV cache memory per token per layer.
- State every assumption before giving a number; when an interviewer changes one, update the answer in thirty seconds.
- The Chinchilla 20-to-1 ratio is a starting point, not a law - production teams re-derive it and get different answers depending on data quality and serving requirements.
- Approximate arithmetic done live and transparently is more persuasive than a precise number with no reasoning attached.

2.3.2 Ignoring the Inference Side of the System

A candidate spends forty minutes on a compelling training pipeline - data mix, tokenizer, architecture choices, parallelism strategy - then closes with “and we serve it with a standard inference server.” The interviewer asks: “What is your expected time-to-first-token at P95?” Silence. This is the second major pitfall: treating inference as a deployment detail rather than a first-class design constraint.

Training is a one-time capital expense. Inference is the operating cost you pay every day for every user, and at production scale it routinely exceeds total training cost within months of a launch. The candidate who designs only for training has optimized the wrong half.

The split between *prefill* and *decode* is the central concept. Prefill encodes the full prompt in a single parallel pass - it is compute-bound, uses the GPU efficiently, and determines time-to-first-token. Decode generates one token per step, reading the full set of model weights from GPU memory on every step while performing a tiny amount of arithmetic. For a single sequence, the ratio of FLOPs to bytes transferred is approximately one - far below the hundreds-to-one threshold at which a modern accelerator is compute-saturated. Decode is therefore memory-bandwidth-bound no matter how large the model, and essentially nothing about the serving stack changes that physics.

The KV cache is the primary constraint on serving batch size. Every live sequence occupies a per-layer, per-head cache that grows with context length. As batch size increases, the cache grows proportionally and eventually exhausts GPU memory before compute becomes the bottleneck. This is why grouped-query attention and multi-head latent attention exist: they reduce the bytes stored per sequence, allowing more concurrent sequences and improving throughput. These are not training-time curiosities - they are direct responses to a serving constraint, and the decision to use them must be made before training begins, because it cannot be patched afterward.

This is the deeper trap: architecture decisions made at training time lock in your serving economics. The number of key-value heads, the maximum context length, the parameter dtype - all are fixed before you write a line of serving code, and all directly determine KV cache footprint, achievable batch size, and cost-per-token.

Surfacing the inference constraint before committing to an architecture is not premature optimization; it is the correct order of operations.

Any system design that ends with “deploy to an inference server” is incomplete.

How This Shows Up

- **“Walk me through serving a 70B model to 10,000 concurrent users.”** A strong answer immediately names the latency-versus-throughput trade-off, sizes the KV cache per request, identifies how batch size is constrained by total GPU memory, proposes continuous batching to avoid idle compute between requests, and raises head-structure choices that reduce cache pressure per sequence. The arithmetic lives in Chapter 8.
- **“Your architecture team wants to reduce key-value heads from 32 to 8 using grouped-query attention. What does that do to serving costs?”** A strong answer derives the KV cache reduction directly: cache per sequence shrinks by $4\times$, so the maximum batch size at fixed memory roughly quadruples, cutting cost-per-token by roughly the same factor at throughput-limited operating points - while leaving time-to-first-token essentially unchanged. Chapter 8 carries the full derivation.

The technical depth for both questions lives in Chapter 8 (inference arithmetic, KV cache sizing, batching strategies).

Key Takeaways

- Name the prefill/decode split early; each phase is bottlenecked differently and therefore has different levers.
- KV cache memory, not parameter count, is usually the binding constraint on serving batch size; size it explicitly before finalizing the architecture.
- Head structure, context length, and dtype are training-time decisions with direct serving-economics consequences - raise these constraints before architecture is finalized.
- State at least one latency SLO and one throughput target before ending any serving discussion; “standard inference server” is not an answer.

Part II

Foundations: Transformer Architecture and Training Signals

Chapter 3 Transformer Architecture Interview Essentials

3.1 The Baseline Decoder-Only Transformer

3.1.1 Residual Stream, Attention, and MLP Blocks

The opening question in virtually every LLM architecture deep-dive is some form of “walk me through the forward pass of a decoder-only transformer.” At mid-level, interviewers want to hear you trace the data flow; at senior level, they want you to estimate the parameter budget per block at a realistic model size, explain why the two sub-blocks are sequenced the way they are, and articulate what goes wrong if you change the structure. Getting this right is the baseline for every architecture discussion that follows.

A decoder-only transformer processes n input tokens, each represented as a d -dimensional vector. These vectors form the *residual stream* - a horizontal channel of activations that flows from the bottom of the network to the top. The defining property of the residual stream is that neither attention nor the MLP writes directly over the previous layer’s output: both read from the stream and *add* their outputs back into it. The stream carries the cumulative representation of each token from embedding to logit head.

Formally, a single layer ℓ executes two sequential updates:

$$\begin{aligned} a^{(\ell)} &= x^{(\ell-1)} + \text{Attn}^{(\ell)}\left(\text{Norm}(x^{(\ell-1)})\right) \\ x^{(\ell)} &= a^{(\ell)} + \text{MLP}^{(\ell)}\left(\text{Norm}(a^{(\ell)})\right) \end{aligned}$$

The attention sub-block computes multi-head scaled dot-product attention. With h heads, each of dimension $d_h = d/h$, the combined projection matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ and output projection $W_O \in \mathbb{R}^{d \times d}$ define:

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O, \quad \text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_h}}\right) V_i$$

where $Q_i = XW_{Q_i}$, $K_i = XW_{K_i}$, $V_i = XW_{V_i}$ are per-head slices of the combined projections.

The MLP sub-block, in its modern SwiGLU form, uses three weight matrices instead of two:

$$\text{MLP}(x) = \left(\text{Swish}(xW_1) \odot (xW_3)\right) W_2$$

where $W_1, W_3 \in \mathbb{R}^{d \times d_{\text{ff}}}$ are up-projections, $\odot$ denotes element-wise multiplication (the gate), and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$ projects back. The gate - xW_3 - modulates which hidden units are active, giving the MLP an adaptive selection mechanism absent in ReLU and GeLU variants.

The residual stream is the skeleton of the network: every sub-block reads from it and adds an increment, so a model with zero-initialized sub-block outputs is a pure identity - this is what makes stacks of 32 or 96 layers trainable from random initialization.

Residual connections solve gradient vanishing. Without an additive skip, the gradient of the loss with respect to the activations at layer ℓ requires a chain of $L - \ell$ Jacobians, which generically collapses to near-zero at depth. With the skip, the gradient sees a direct path of ones alongside each Jacobian chain, guaranteeing a non-vanishing lower bound. This is why transformers can sustain depths of 80 or 96 layers without the careful cell-state engineering that LSTMs relied on.

The functional division between attention and MLP reflects a genuine computational separation. Attention *aggregates across positions*: it lets each token construct a weighted mixture of all other tokens’ representations, gathering context from wherever it is most informative. The MLP *processes per-position*: once a token has

gathered its context, the MLP mixes information across the feature dimension of that single vector. Stacking the two repeatedly allows the network to alternate between “gather context” and “refine representation.” This intuition is supported by mechanistic analyses showing that factual associations tend to live in MLP weights, while syntactic and positional relationships live in attention patterns.

The overwhelming consensus is to run attention before MLP in series. A minority of models (GPT-J, PaLM) explored *parallel layers* where both sub-blocks read the same normalized input simultaneously and their outputs are summed. Parallel layout can yield a roughly 15% wall-clock speedup at large scale because both sets of projections can be fused into a single large matrix multiply against a shared normalized input. However, the parallel layout means the MLP never sees the attention output within the same layer - a mild expressiveness loss. Most frontier models released after 2023 have reverted to the serial default, which I read as evidence that the expressiveness loss is real enough to matter at the model sizes where these labs operate.

A Worked Example

Take a 7B-parameter model: $d = 4096$, $h = 32$ heads ($d_h = 128$), $L = 32$ layers, SwiGLU MLP with $d_{ff} = 11008$ (the canonical $\lfloor 8d/3 \rfloor$ sizing for GLU variants, rounded to the nearest 128), vocabulary $V = 32000$.

Parameter budget per layer:

- Attention (W_Q, W_K, W_V, W_O , no biases): $4 \times d^2 = 4 \times 4096^2 \approx 67\text{M}$
- MLP (W_1, W_3, W_2 , no biases): $3 \times d \times d_{ff} = 3 \times 4096 \times 11008 \approx 135\text{M}$
- Layer total: $\approx 202\text{M}$

Across 32 layers: $32 \times 202\text{M} \approx 6.46\text{B}$. Adding the embedding table ($32000 \times 4096 \approx 131\text{M}$) lands at roughly 6.6B - consistent with LLaMA-1 7B’s actual count of 6.7B. The MLP holds approximately two-thirds of each layer’s parameters, a ratio worth memorizing: when an interviewer asks you to split the parameter budget between attention and FFN, the answer for SwiGLU models is roughly 1:2.

Memory at bf16: $6.7\text{B} \times 2 \text{ bytes} = 13.4 \text{ GB}$ for weights alone. On an 80 GB H100, that leaves ~ 67 GB for activations, KV cache, and optimizer state - comfortable for inference, tight for training under AdamW, which needs at least $4 \times$ the weight size for gradients and first/second moment buffers.

What You Actually Decide in Practice

- **Serial vs. parallel layers.** Default to serial. Parallel layers require custom fused kernels to realize their speedup; without them, the benefit vanishes. Only consider parallel if you have ablated quality impact and verified kernel throughput on your specific hardware. Most teams find it is not worth the engineering overhead.
- **MLP sizing with SwiGLU.** Set $d_{ff} = \lfloor 8d/3 \rfloor$, rounded up to the nearest multiple of 64 for hardware alignment. The vanilla ReLU convention of $d_{ff} = 4d$ over-counts your parameter budget by roughly 50% relative to a non-gated MLP, because SwiGLU introduces a third matrix. Getting this wrong means your “7B” model is actually closer to 10B.
- **Head dimension.** Keep $d_h = d/h = 128$, the near-universal default. Scaling law evidence shows that varying head count at fixed d and d_h has negligible effect on loss; what matters is that d_h stays large enough to avoid rank bottlenecks in each head’s key-value projection.
- **Bias terms.** Drop them everywhere in attention and MLP. Biases add negligible expressiveness but have been linked empirically to training instability in large models. Virtually every frontier model since

LLaMA-1 omits them, and the rule of thumb is: if you cannot point to a specific ablation that shows bias terms help, do not include them.

- **Residual stream accumulation dtype.** Keep the residual stream accumulation in bf16 or fp32 depending on your mixed-precision setup. Do not quantize the skip-connection additions - precision erosion in the accumulation compounds across 32+ layers and is very hard to diagnose after the fact.

How This Shows Up in Interviews

- **“Walk me through the forward pass of a 7B decoder-only model. How many parameters are in the attention block versus the MLP?”** A strong answer names all four projection matrices in attention, all three in SwiGLU, derives the $\sim 67\text{M}$ and $\sim 135\text{M}$ figures, and explains why the MLP is roughly $2\times$ larger. A weak answer stops at “it has multi-head attention and a feed-forward network” without numbers.
- **“Why is the residual connection important? What breaks without it?”** Strong answer: gradient propagation. Without the additive skip, the gradient at layer ℓ involves a product of $L - \ell$ Jacobians that collapses to near-zero at depth; with the skip, there is always a direct gradient path of approximately ones from the top to every earlier layer.
- **Senior follow-up: “Some models run attention and MLP in parallel instead of in series. What do you trade off, and when would you choose parallel?”** Senior answer: parallel layout fuses both sets of projections against the same normalized input in a single large matrix multiply, saving memory round-trips and yielding roughly 15% wall-clock speedup when kernels are properly written (as demonstrated in PaLM). The cost is that the MLP sees the pre-attention representation rather than the post-attention one - the MLP and attention cannot interact within the same layer. In practice, serial is the safer default unless you have custom kernel support and have ablated quality impact at your target model size; most post-2023 frontier models have reverted to serial for exactly this reason.

Key Takeaways

- The residual stream is the identity highway: every sub-block reads from it and adds an increment, enabling clean gradient flow through networks of 32-128 layers.
- Each layer contains two sub-blocks in series: multi-head attention (mixes across positions) followed by an MLP (mixes across features at each position).
- For a 7B model with $d = 4096$, $d_{ff} \approx 11008$: each layer holds $\approx 67\text{M}$ attention parameters and $\approx 135\text{M}$ MLP parameters - the MLP is roughly $2\times$ the attention block by parameter count.
- SwiGLU MLPs use three weight matrices and require $d_{ff} = \lfloor 8d/3 \rfloor$ to stay parameter-matched with a two-matrix ReLU MLP at $4d$; getting the sizing wrong inflates the model by $\sim 50\%$.
- Use serial layers and drop all bias terms by default; parallel layers require verified kernel-level speedups and quality ablations before committing.

3.1.2 Pre-Norm vs. Post-Norm and Why It Matters

The question “where does the LayerNorm go in your transformer block?” reads like trivia until you have watched a training run crash after a week of GPU-hours because gradient norms spiked and refused to come back down. Normalization placement directly controls training stability at scale, and every frontier lab I am aware of has converged on the same answer. Interviewers at these labs use this question to filter candidates who have only read the original 2017 paper from those who have actually thought about what happens at depth.

The original transformer applied normalization *after* each sub-block’s output was added back to the residual stream - the *post-norm* convention:

$$x \leftarrow \text{LayerNorm}(x + \text{Attn}(x)), \quad x \leftarrow \text{LayerNorm}(x + \text{MLP}(x))$$

Pre-norm moves the normalization *before* the sub-block computation, operating on the residual stream input prior to branching off to attention or MLP:

$$a \leftarrow x + \text{Attn}(\text{Norm}(x)), \quad x \leftarrow a + \text{MLP}(\text{Norm}(a))$$

The normalization now sits inside the non-residual branch. The residual stream itself - the x that accumulates across layers - is never touched by a normalization layer. This single repositioning is the difference between a network that requires fragile learning-rate warmup to avoid divergence and one that trains stably from step one.

Pre-norm is the universal choice in production models: LLaMA, PaLM, Chinchilla, Mistral, Qwen, DeepSeek, Gemma, and virtually every frontier model since GPT-2 uses it. The only notable exception is OPT-350M, which used post-norm - an artifact widely attributed to an engineering oversight that was never corrected before the run completed.

Place the normalization inside the non-residual branch: pre-norm keeps the gradient highway clean, enables stable training without warmup, and is the choice every serious practitioner makes.

The core argument is gradient behavior at depth. With post-norm, the loss gradient traveling back to layer ℓ must pass through LayerNorm at every layer between ℓ and the top. LayerNorm rescales activations by the inverse of their empirical standard deviation - a data-dependent quantity that is volatile in early training when activations have not yet settled. This volatility causes the effective step size at early layers to oscillate by orders of magnitude from batch to batch, which is why post-norm training famously required careful warmup: you were buying time for activations to stabilize before the gradient became reliable.

Pre-norm sidesteps this entirely. The residual addition $x \leftarrow x + f(\text{Norm}(x))$ means the gradient of the loss at layer ℓ always sees an additive term of 1 from the skip connection, regardless of what the sub-block gradient looks like. The normalization sits only on the branch and does not obstruct the residual gradient path. At depth $L = 32$ or $L = 96$, this difference is the margin between a gradient norm that stays in $[0.1, 2]$ throughout training and one that spikes to 10^3 and triggers a loss divergence.

Empirical evidence is consistent and old enough to be well-established: training curves from pre-norm models show lower gradient norm variance, fewer loss spikes, and equivalent or slightly better final loss compared with post-norm models that used careful warmup. Several research teams demonstrated that pre-norm models can remove the warmup phase entirely and still converge, which simplifies resuming from checkpoints mid-run.

A newer variant, which I will call the *double-norm*, adds a second normalization after each sub-block (before the residual addition) as well as the pre-block norm. Grok-1 and Gemma 2 use this:

$$a \leftarrow x + \text{Norm}_2(\text{Attn}(\text{Norm}_1(x)))$$

The logic is that bounding both the input to the sub-block and its output gives two scale control checkpoints instead of one, providing additional protection against activations exploding inside very large sub-blocks. Early reports suggest marginal stability gains at very large model sizes ($>30\text{B}$ parameters). It adds memory access overhead (LayerNorm is bandwidth-bound, not compute-bound) but is otherwise a small cost. Double-norm has not yet displaced plain pre-norm as the community default.

A Worked Example

Consider a 32-layer, 1.3B-parameter model trained with AdamW, peak learning rate 3×10^{-4} , no warmup schedule, on 100B tokens. Compare the gradient dynamics across three configurations:

Configuration	Grad norm at step 500	Stability
Post-norm, no warmup	~450 (highly variable)	Diverges
Post-norm, 2k-step warmup	~12 (settling)	Converges
Pre-norm, no warmup	~1.8 (stable from step 1)	Converges

Post-norm without warmup diverges reliably: by layer 10, the LayerNorm denominators are fluctuating because the activations are still in their initial transient. The effective step size at early layers swings by $5\text{-}10\times$ per step, causing gradient norms to spike past $10^2\text{-}10^3$ within the first thousand steps.

Post-norm with 2,000-step warmup eventually converges because the warmup period allows activations to settle before the learning rate reaches its peak. But this coupling between normalization placement and the warmup schedule is a fragile dependency: if you need to restart from a checkpoint (e.g., after a hardware failure), you must re-warm carefully or risk a second divergence.

Pre-norm without warmup produces gradient norms in the range 0.1-2.0 from the very first step. The residual stream accumulates small increments from each sub-block, and the gradient back-propagates through the skip connections without interference from normalization. You can restart from any checkpoint with a cold optimizer and the training will resume cleanly.

The practical upshot at scale: at 7B parameters trained on 1T tokens, a single gradient divergence that requires a rollback and rerun from the last stable checkpoint wastes days of compute on a cluster of 256 H100s. Pre-norm eliminates an entire class of these failures.

What You Actually Decide in Practice

- **Always use pre-norm for new models.** There is no empirical justification for post-norm in a fresh training run. If you inherit a post-norm codebase, do not switch normalization placement mid-training - it changes the loss landscape and will almost certainly cause a divergence.
- **Consider double-norm only above ~30B parameters.** The extra normalization costs additional memory accesses (LayerNorm is bandwidth-bound; the extra norm adds $\sim 0.3\%$ wall time per step) and a small number of additional parameters. Below 30B, plain pre-norm is stable enough that double-norm provides no measurable benefit. Above 30B, if you are seeing persistent gradient norm spikes despite pre-norm, double-norm is a reasonable next step before reaching for gradient clipping or modified optimizers.
- **Never put a normalization layer on the residual stream itself.** Any norm that sits on the accumulation path (i.e., on x before the skip addition rather than on the branch input) re-introduces the post-norm pathology. The identity gradient path is the entire point.
- **Monitor gradient norms from step 1.** With pre-norm and a sane learning rate, gradient norms should be below 5.0 throughout the first 1,000 steps. If they are not, suspect a normalization placement bug, a bad initialization scale, or a learning rate that is too high - not a fundamental architecture problem.
- **Treat pre-norm and RMSNorm as a package deal.** The decision of where to place the norm and which norm to use are distinct but nearly always made together in modern models. Pre-norm with RMSNorm is the combination that appears in LLaMA, PaLM, Chinchilla, Mistral, and most other frontier open models.

How This Shows Up in Interviews

- **“Where does the LayerNorm go in a modern transformer block, and why?”** A strong answer places the norm before the sub-block (pre-norm), explains that it keeps the residual stream as a clean gradient highway, and notes this removes the need for warmup. A weak answer just says “before the block” without explaining the gradient argument.
- **“What does the original 2017 transformer do differently, and what problem does that cause?”** Strong answer: post-norm places the normalization after the residual addition, which means the gradient back-propagating to early layers must pass through LayerNorm at every layer. Because LayerNorm rescales by the inverse standard deviation - a volatile quantity in early training - this corrupts the effective step size at early layers and requires careful warmup to avoid divergence.
- **Senior follow-up: “Gemma 2 applies a LayerNorm both before and after each sub-block (double-norm). What problem does that solve, and what does it cost?”** Senior answer: double-norm bounds both the input to the sub-block and its output before re-adding to the stream. At very large scale, sub-blocks can still produce large-magnitude outputs even when their inputs are normalized; bounding the output provides a second safety checkpoint. The cost is extra memory bandwidth (LayerNorm is bandwidth-bound, not flop-bound - each extra norm adds one additional read/write pass over a $\mathbb{R}^{n \times d}$ activation tensor per layer), plus the additional learned scale parameters γ . At 7B parameters this cost is negligible; at 70B or 100B, you should ablate whether the stability gain justifies it rather than adopting it by default.

Key Takeaways

- Pre-norm applies normalization before each sub-block, keeping the residual stream as a clean identity path for gradients; post-norm applies it after the residual addition, forcing gradients through a normalization at every layer.
- Post-norm requires careful learning-rate warmup to avoid divergence; pre-norm trains stably from step one and allows cold restarts from any checkpoint.
- Pre-norm is universal in modern frontier models: LLaMA, PaLM, Chinchilla, Mistral, DeepSeek, Gemma, and essentially every open model released since GPT-2.
- Double-norm (pre- and post-block norms, used in Gemma 2 and Grok-1) adds a second stability checkpoint for very large models; it is not yet the default and costs extra memory bandwidth.
- The gradient norm should be below 5.0 from step 1 with pre-norm and a sane learning rate; values above that signal a normalization placement bug or initialization issue, not a fundamental architecture problem.

3.2 Normalization Choices

3.2.1 LayerNorm vs. RMSNorm

In a system design round at a frontier lab, you will almost certainly be asked why your training stack uses RMSNorm instead of LayerNorm. The question sounds trivial - one formula versus another - but the real interview signal is whether you can connect normalization choices to memory bandwidth, training stability, and the arithmetic intensity of GPU kernels. This comes up at the mid-to-senior level whenever hardware efficiency enters the conversation, and getting the answer right separates engineers who have profiled real training runs from those who have only read the paper abstracts.

LayerNorm normalizes each activation vector by its empirical mean and standard deviation, then applies a learned affine transform:

$$\text{LayerNorm}(\mathbf{x})_i = \gamma_i \cdot \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_i$$

where $\mu = \frac{1}{d} \sum_{j=1}^d x_j$ and $\sigma^2 = \frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2$. Two statistics must be computed, and two parameter vectors $\gamma, \beta \in \mathbb{R}^d$ must be loaded from memory on every forward pass.

RMSNorm drops mean-centering entirely and eliminates the shift parameter:

$$\text{RMSNorm}(\mathbf{x})_i = \gamma_i \cdot \frac{x_i}{\text{RMS}(\mathbf{x}) + \varepsilon}, \quad \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{j=1}^d x_j^2}$$

Only one statistic (the root-mean-square) is computed, and only $\gamma \in \mathbb{R}^d$ is loaded. The β parameter vanishes.

On paper, the FLOP difference is negligible - normalization operations are less than 0.2% of the total FLOPs in a transformer. The argument for RMSNorm is not arithmetic; it is memory. Careful profiling (Ivanov et al., 2023) shows that normalization kernels account for roughly 25% of wall-clock runtime despite their vanishing FLOP share, because they are *memory-bandwidth-bound*: every scalar in γ and β must be staged through HBM on every call. Halving the parameter count directly halves that memory traffic.

The case for RMSNorm is a memory-bandwidth argument dressed up as a simplification argument: fewer parameters to load per call, fewer operations per element, and empirically indistinguishable model quality.

GPU computation is dominated by two bottlenecks: arithmetic throughput (FLOPs) and memory bandwidth (bytes/s). Matrix multiplies are compute-roofed at large batch sizes - the GPU is limited by how fast it can multiply, not by how fast it can read weights. Normalization kernels are the opposite: they touch each activation element a small, fixed number of times, so the runtime is bounded by how fast the GPU can stream data through HBM. On an H100, peak compute is around 989 TFLOP/s (bf16) while peak memory bandwidth is 3.35 TB/s; the ratio is roughly $300\times$. Any kernel that is not doing at least 300 FLOPs per byte loaded will hit the memory wall, not the compute ceiling. Normalization is firmly in that regime.

LayerNorm’s β parameter is small in count but adds a full load-and-multiply pass over d scalars per invocation. For a model with $d = 4096$ and 64 normalization calls per forward pass (32 layers $\times$ 2 norms), that is $64 \times 4096 \times 2$ bytes ≈ 524 KB of parameter tensor loads per sample per forward pass in bf16, entirely eliminated by RMSNorm.

The quality story is equally clean. Narang et al. (2020) ran controlled ablations on a transformer of comparable scale and found that RMSNorm reaches the same final cross-entropy as LayerNorm while running at 3.68 steps/second versus 3.50 - a free 5% throughput improvement. The LLaMA family, PaLM, Chinchilla, Mistral, and DeepSeek have all converged on RMSNorm. The near-consensus explanation: when pre-norm placement already keeps the residual stream well-conditioned before each sub-block, the mean of the activation vector entering the norm is already close to zero, so mean-centering provides negligible correction.

A Worked Example

Consider a 7B decoder-only model: $d = 4096$, 32 layers, batch size $B = 4$, sequence length $T = 2048$, bf16 precision. Each transformer layer has two normalization calls (pre-attention and pre-MLP). Per call:

1. **LayerNorm**: compute mean (d additions), compute variance ($2d$ multiply-adds), normalize ($2d$ ops), apply γ and β ($2d$ multiply-adds). Total: $\approx 7d$ scalar ops. Memory loads: $2d$ bytes for γ , $2d$ bytes for β (bf16).

2. **RMSNorm**: compute RMS (d squares, one sum, one sqrt), normalize (d divisions), apply γ (d multiply-adds). Total: $\approx 3d$ scalar ops. Memory loads: $2d$ bytes for γ only.

Across the entire forward pass: $32 \times 2 = 64$ normalization calls. The β removal saves $64 \times 4096 \times 2$ bytes = 524 KB per sample per forward pass. At $B = 4$, that is roughly 2 MB per training step eliminated from HBM traffic. At 10,000 steps/hour, this is ≈ 20 GB/hour of avoided memory movement - not the dominant cost, but meaningful when normalization is otherwise consuming 25% of wall time.

The throughput difference of 5% reported in ablations maps directly onto this: if normalization is 25% of runtime and RMSNorm cuts its overhead in half, the net gain is $0.25 \times 0.5 = 12.5\%$ reduction in normalization time, which translates to approximately 3-5% overall throughput improvement accounting for the compute-roofed portions of the forward pass.

What You Actually Decide in Practice

- **Default to RMSNorm.** Every major model family released since 2022 that publishes architecture details uses it. Cohere Command A and Command R+ are the notable exception - they retain LayerNorm for undisclosed reasons. Unless you have a specific compatibility constraint, there is no reason to start a new run with LayerNorm.
- **Do not re-introduce β .** Some codebases add a bias back to RMSNorm for “completeness.” This negates the memory saving and provides no quality benefit - the downstream linear layers already supply the affine freedom that β would add.
- **Watch ε at bf16.** The default $\varepsilon = 10^{-5}$ can underflow in bf16 for activation vectors with very small magnitudes, producing a near-zero denominator. Use $\varepsilon = 10^{-6}$ and monitor for NaN spikes on the first 1B tokens; if they appear, increase ε to 10^{-8} in fp32 accumulation mode before blaming the optimizer.
- **Fused kernels are mandatory at scale.** A naive PyTorch RMSNorm implementation still issues separate HBM transactions for each element-wise op. Production runs use Triton or cuDNN-fused kernels; unfused implementations can cost 2-3 $\times$ the wall time of the fused version, erasing the RMSNorm advantage entirely.
- **LoRA adapters on RMSNorm backbones need explicit bias handling.** When fine-tuning with LoRA or prefix tuning, adapter layers sometimes implicitly assume a β term exists. If your adapter introduces a bias, verify it is absorbing the role of β intentionally; accidental double-counting or missing shift can cause subtle drift in generation quality that is hard to diagnose.

How This Shows Up in Interviews

- “Why do modern LLMs use RMSNorm instead of LayerNorm?” A strong answer names both the memory-bandwidth argument (normalization is $\sim 25\%$ of runtime despite $< 0.2\%$ of FLOPs) and the quality-parity finding, and cites at least two concrete model families that made the switch. Answering only “it’s simpler” scores mid-level at best.
- “If RMSNorm drops mean-centering, doesn’t that hurt models where activation distributions are not zero-mean?” The correct answer is that pre-norm placement ensures the residual stream entering each normalization is already regulated by upstream norms, so the empirical mean is close to zero at initialization and remains close throughout training. Mean-centering provides negligible correction in that regime, which is exactly what the ablations confirm.

- **Senior follow-up:** “Walk me through the arithmetic intensity of a LayerNorm kernel versus a matrix multiply. Why does the roofline model determine how much you can save by switching to RMSNorm?” A senior answer articulates the roofline model: LayerNorm is bandwidth-rooflined (low arithmetic intensity), so reducing memory accesses directly reduces wall time proportionally. A matrix multiply at large batch size is compute-rooflined, so an equivalent FLOP reduction there yields a much smaller real-world speedup. The implication is that optimizing memory-bound kernels gives super-linear returns relative to their FLOP share.

Key Takeaways

- RMSNorm drops mean-centering and the shift parameter β ; it computes only $\text{RMS}(\mathbf{x})$ and applies a single learned scale γ .
- Normalization kernels are memory-bandwidth-bound: they are $\sim 0.2\%$ of FLOPs but $\sim 25\%$ of runtime. The win from RMSNorm is in memory traffic, not arithmetic.
- Published ablations show quality parity with LayerNorm and a roughly 5% throughput improvement; the LLaMA family, PaLM, Chinchilla, Mistral, and DeepSeek all converged on RMSNorm.
- Do not add β back, and always use fused kernels - naive implementations negate both the memory and arithmetic savings.
- The deeper lesson: once a model is architecture-stable, the bottleneck often moves from FLOPs to memory movement. Normalization is the canonical example of an operation that is cheap in FLOPs and expensive in bandwidth.

3.2.2 QK-Norm and Stability Interventions

At some point in every large training run, you will open your monitoring dashboard and see a gradient norm plot that looks like a seismograph during an earthquake - sudden spikes of $10\times$ or $100\times$ the baseline, occasionally followed by NaN propagation that ends the run. Training instability is one of the most expensive failure modes in LLM development, capable of wasting weeks of GPU time and hundreds of thousands of dollars on a single diverged job. Knowing which architectural interventions actually prevent these spikes, and being able to explain *why* they work at a mechanistic level, is a senior-level topic that surfaces in both system design rounds and deep technical interviews at labs actively training at scale.

The two locations in a transformer where numeric instability most commonly originates are the *attention softmax* and the *output softmax*. Both involve exponentiating logits that can grow without bound during training, producing either overflow, near-zero denominators, or pathologically peaked distributions that starve most positions of gradient signal.

QK-norm targets the attention softmax. Before computing $\mathbf{QK}^\top / \sqrt{d_k}$, apply a normalization layer - typically RMSNorm - to the query and key tensors independently:

$$\mathbf{A} = \text{softmax}\left(\frac{\text{Norm}(\mathbf{Q}) \text{Norm}(\mathbf{K})^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

This bounds the magnitude of pre-softmax attention logits regardless of how the weight matrices $\mathbf{W}_Q$ and $\mathbf{W}_K$ evolve during training.

Z-loss targets the output softmax. Let $z(\mathbf{x}) = \log \sum_{v=1}^V \exp(u_v(\mathbf{x}))$ be the log-normalizer of the output distribution, where u_v are the pre-softmax logits. Z-loss adds an auxiliary penalty to the training objective:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \alpha \cdot [z(\mathbf{x})]^2, \quad \alpha \approx 10^{-4}$$

This encourages $z \rightarrow 1$ (equivalently, $\log z \rightarrow 0$), keeping the output logit scale in a numerically clean regime. When $z = 1$, the softmax reduces to the raw exponentials, which are bounded and smooth.

Logit soft-capping is a complementary intervention on the attention softmax. After computing the raw dot-product attention scores l_{ij} , apply:

$$\hat{l}_{ij} = c \cdot \tanh\left(\frac{l_{ij}}{c}\right)$$

where c is a soft-cap hyperparameter (e.g. $c = 50$). The tanh clamps extreme logits softly while preserving gradients near zero.

In every case the goal is identical: keep inputs to any softmax bounded in magnitude so that neither the exponential overflows nor the distribution degenerates to a Kronecker delta.

Softmax is the only non-linear operation in the standard transformer that can be driven to numerical failure by the surrounding linear algebra. A weight matrix with bounded spectral norm produces bounded outputs if its inputs are bounded - but a softmax with growing logit magnitudes produces exponentially larger activations regardless of what came before it. QK-norm breaks this dependency: regardless of how $\mathbf{W}_Q$ and $\mathbf{W}_K$ evolve during optimization, the vectors fed to the dot product are normalized to unit scale, bounding pre-softmax logits to $O(1/\sqrt{d_k})$.

The contrast with gradient clipping is instructive. Clipping intervenes in the backward pass after a spike has already occurred; the forward-pass activations that triggered the spike are already resident in the residual stream and will propagate to downstream layers before the next update corrects them. QK-norm and z-loss act in the forward pass, preventing the spike from occurring in the first place. For runs longer than $\sim 100\text{B}$ tokens, this distinction matters: each uncaught spike degrades optimizer state in ways that take many subsequent steps to wash out.

Z-loss and QK-norm address orthogonal failure modes. Z-loss controls the global scale of output logits (across $V \sim 32,000\text{--}128,000$ vocabulary items), where failure is typically a slow drift. QK-norm controls attention logit magnitude (across T positions), where failure is typically a sudden spike. Running both provides defense in depth.

A Worked Example

Consider a 7B model with head dimension $d_k = 128$, sequence length $T = 2048$. Suppose after 80B tokens of training the key vectors in one attention head have drifted to RMS magnitude $\|\mathbf{k}\|_2 \approx 12$. The pre-softmax attention logit between positions i and j is:

$$l_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \approx \frac{\|\mathbf{q}_i\|_2 \cdot \|\mathbf{k}_j\|_2}{\sqrt{128}} \approx \frac{12 \times 12}{11.3} \approx 12.7$$

If position j^* has logit 12.7 and all other 2047 positions have logit -12.7 , the attention weight on j^* is:

$$a_{ij^*} = \frac{e^{12.7}}{e^{12.7} + 2047 \cdot e^{-12.7}} \approx \frac{e^{12.7}}{e^{12.7}} \approx 1.0$$

The gradient of the loss with respect to any position $j \neq j^*$ through this softmax is essentially zero. That head has effectively stopped learning - and if the logit continues to grow, the next forward pass may overflow to inf , producing a NaN that propagates through the entire residual stream.

With QK-norm, both $\mathbf{q}_i$ and $\mathbf{k}_j$ are normalized to unit scale before the dot product. The post-normalization logit is:

$$\hat{l}_{ij} = \frac{\hat{\mathbf{q}}_i \cdot \hat{\mathbf{k}}_j}{\sqrt{d_k}} \lesssim \frac{1}{\sqrt{128}} \approx 0.09$$

The softmax operates on logits near zero; the resulting attention distribution is diffuse, every position receives meaningful gradient, and neither overflow nor gradient vanishing occurs. The cost is two RMSNorm calls per attention layer per forward pass - a kernel that is $\ll 1\%$ of the compute of the surrounding QKV projections.

What You Actually Decide in Practice

- **Use QK-norm and z-loss together, not as alternatives.** They target different softmaxes and different failure modes. A model with QK-norm but no z-loss can still diverge through the output logit path as training progresses and the output weight matrix scale drifts upward.
- **Set $\alpha = 10^{-4}$ for z-loss as your starting point.** This is the PaLM coefficient and has been reproduced by DCLM and OLMo 2. Raising it above 10^{-3} will hurt token-level cross-entropy because the model is penalized for concentrating probability mass, which it must do to make high-confidence predictions. Lowering below 10^{-6} provides no measurable stabilization.
- **Do not add logit soft-capping without ablating.** NVIDIA’s ablation found that soft-capping at $c = 50$ *worsened* perplexity by 0.1-0.3 points on their setup, while QK-norm improved it by a similar margin. Gemma 2 reports benefit from soft-capping. The inconsistency likely stems from interactions with the Adam optimizer’s second-moment estimates: tanh distorts the gradient landscape in a way that depends on the learning rate schedule and model scale. Treat it as a model-specific knob.
- **Consider the double-norm pattern for very large runs.** Grok and Gemma 2 place normalization both before and after each sub-block (attention and MLP), rather than only before (standard pre-norm). The post-block norm bounds the output magnitude entering the residual add, which is complementary to QK-norm inside the attention. OLMo 2 uses only the post-block norms, omitting the pre-block norms. Evidence so far suggests this is strictly more stable than pure pre-norm for models trained beyond 300B tokens.
- **Gradient clipping at 1.0 is a floor, not a ceiling.** Keep gradient clipping as a last-resort backstop, but do not rely on it as your primary stability mechanism. At the scale where instability becomes dangerous ($\geq 30\text{B}$ parameters, $\geq 100\text{B}$ tokens), the frequency of clipping events correlates with final model quality degradation. If you clip more than 2-3% of steps, investigate the architectural root cause rather than raising the clip threshold.

How This Shows Up in Interviews

- *“You are training a 70B model and gradient norm spikes appear after 50B tokens. What do you check first and what architectural changes do you consider?”* A strong answer mentions per-layer gradient monitoring to localize whether spikes originate in the attention blocks or the output projection, then proposes QK-norm for the attention path and z-loss for the output path. It also checks whether the current learning rate is still in the cosine schedule’s stable descent region.
- *“What does QK-norm actually do to the attention distribution, and what is the cost?”* Strong answer: it bounds pre-softmax attention logits to $O(1/\sqrt{d_k})$, preventing attention from collapsing to a one-hot distribution over long training; cost is two extra RMSNorm calls per attention layer, negligible compared to the surrounding projections.
- **Senior follow-up:** *“Z-loss controls the output softmax normalizer. But cross-entropy already backpropagates through the softmax. How can z-loss provide information that the cross-entropy gradient does not?”* The senior answer: cross-entropy gradient pushes the *correct* logit up relative to the others, but

does not constrain the *absolute scale* of the logit vector. Z-loss penalizes the scale directly - even a model achieving perfect cross-entropy can have large-magnitude logits that create numerical instability. The two losses operate on orthogonal degrees of freedom in logit space (direction versus magnitude), which is why they are complementary and not redundant.

Key Takeaways

- Training instability in transformers concentrates at the two softmaxes: the attention softmax over T sequence positions and the output softmax over the V -token vocabulary.
- QK-norm applies RMSNorm to query and key tensors before the attention dot product, bounding pre-softmax logit magnitudes and preventing attention collapse or gradient vanishing. Gemma 2, OLMo 2, and DCLM all use it.
- Z-loss adds $\alpha \cdot [z(\mathbf{x})]^2$ to the training objective, keeping the output softmax normalizer near 1 and preventing unbounded logit growth; PaLM set the standard with $\alpha = 10^{-4}$.
- Use QK-norm and z-loss together - they target different failure modes and are complementary. Logit soft-capping is a model-specific choice; ablate before adding it.
- Gradient clipping is a backstop, not a primary stability tool; frequent clipping ($>2\%$ of steps) signals an architectural problem that needs to be fixed at the source.

3.3 Activation Functions in Modern LLMs

3.3.1 ReLU, GeLU, and SwiGLU Families

Every major open-weight model released since 2022-LLaMA, PaLM, Gemma, Mistral, DeepSeek, Qwen- uses a gated activation in its MLP blocks. If you walk into an architecture deep-dive and cannot explain what SwiGLU is, why it replaced GeLU, and how it changes the hidden dimension sizing, you will stall on a warmup question before the real design challenge begins. This topic appears most often at mid-to-senior level interviews at frontier labs, where it quickly signals whether a candidate has read recent model cards or is still reasoning from undergraduate deep learning.

The MLP block in a transformer layer projects from the residual stream dimension d up to a wider hidden dimension d_{ff} , applies a nonlinearity, then projects back down. Three activation families partition the design space.

ReLU. $\text{ReLU}(x) = \max(0, x)$. The two-layer FFN is:

$$\text{FFN}_{\text{ReLU}}(x) = \max(0, xW_1)W_2$$

Fast, hardware-friendly, but a large fraction of hidden units saturate at zero during training, wasting capacity. The hard kink at zero also creates gradient cliffs that require careful initialization.

GeLU. The Gaussian Error Linear Unit smooths the hinge:

$$\text{GeLU}(x) = x \cdot \Phi(x)$$

where Φ is the CDF of the standard normal. In code it is approximated as $x \cdot \sigma(1.702x)$. Differentiable everywhere, no dead-neuron regime, and empirically better than ReLU on every benchmark published for it. GPT-1 through GPT-3 all use GeLU, and it held as the dominant choice through 2021.

Gated Linear Units (GLU) and SwiGLU. The core idea: multiply the activated hidden representation

element-wise by a second learned linear projection that acts as a soft gate:

$$\text{GLU}(x, W, V) = \sigma(xW) \odot (xV)$$

SwiGLU replaces the sigmoid gate with Swish, where $\text{Swish}(x) = x \cdot \sigma(x)$. The full SwiGLU MLP, without bias terms as in all modern LLMs, is:

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{Swish}(xW_1) \odot (xV)) W_2$$

where $W_1, V \in \mathbb{R}^{d \times d_{ff}}$ and $W_2 \in \mathbb{R}^{d_{ff} \times d}$.

GeGLU substitutes GeLU for Swish and is used by T5v1.1, Gemma 2, and Gemma 3:

$$\text{FFN}_{\text{GeGLU}}(x) = (\text{GeLU}(xW_1) \odot (xV)) W_2$$

Gated activations let the network learn to suppress or amplify each hidden dimension independently at each token, which is strictly more expressive than a fixed pointwise nonlinearity at the same parameter budget.

The V matrix acts as a context-dependent gain: for every hidden dimension at every token, the network independently decides how much of the activated signal passes forward. A plain ReLU or GeLU MLP can only gate in a binary sense—either the value is positive and passes, or it does not. A GLU variant attenuates values softly, in a learned, input-dependent way. The empirical result is a consistent 0.3-0.8 nat improvement in pre-training perplexity at equal parameter count.

The smooth nonlinearity in SwiGLU and GeGLU matters separately. The hard kink at zero in ReLU means neurons near the boundary receive nearly zero gradients and stop participating in learning. Swish and GeLU both maintain a small non-zero gradient slightly below zero, which keeps more neurons active throughout training. The Swish curve dips very slightly below zero for large negative inputs—a property that alarmed some early reviewers—but in practice, optimizer momentum carries activations far away from that tiny regime and it has never been measured as a problem in production training runs.

Before GLU variants, the choice was ReLU (fast, rough) versus GeLU (smooth, slightly better). GLU variants deliver GeLU-level expressiveness plus a multiplicative gate, at the cost of one additional weight matrix V . The question is always whether that matrix earns its keep—the evidence consistently says yes.

A Worked Example

Take LLaMA-2 7B: $d = 4096$, SwiGLU. The convention is that the total MLP parameter count should match a standard $4 \times$ ReLU FFN to keep the comparison fair.

Standard ReLU FFN with $d_{ff} = 4d = 16,384$: two matrices, each $d \times d_{ff}$:

$$\text{params} = 2 \times 4096 \times 16,384 = 134,217,728 \approx 134 \text{ M per layer}$$

SwiGLU FFN has three matrices: W_1, V, W_2 . To match 134 M with three equal-shaped matrices:

$$3 \times d \times d_{ff} = 134 \text{ M} \implies d_{ff} = \frac{134 \text{ M}}{3 \times 4096} = \frac{8}{3} \times 4096 = 10,922.\bar{6}$$

Round up to the nearest multiple of 128 for hardware tile alignment: LLaMA-2 7B uses $d_{ff} = 11,008$.

The forward pass through one SwiGLU MLP layer at batch size B , sequence length T :

1. Gate projection: $g = \text{Swish}(xW_1) \in \mathbb{R}^{B \times T \times 11008}$
2. Value projection: $v = xV \in \mathbb{R}^{B \times T \times 11008}$
3. Element-wise product: $h = g \odot v \in \mathbb{R}^{B \times T \times 11008}$
4. Down projection: $\text{out} = hW_2 \in \mathbb{R}^{B \times T \times 4096}$

Total FLOPs for the three matrix multiplications: $6 \times B \times T \times 4096 \times 11,008 \approx 270B \cdot B \cdot T$ FLOPs. The element-wise product in step 3 is memory-bound and negligible in total FLOP count. The two extra weight

matrices compared to a GeLU FFN cost nothing in FLOPs beyond what a parameter-matched GeLU FFN would cost.

What You Actually Decide in Practice

- **Round d_{ff} to a multiple of 64, preferably 128.** Hardware tiling requirements mean oddly sized dimensions can cost 10-20% throughput on A100 and H100. LLaMA-2 uses 11008 rather than the exact 10922.7 for this reason. Some labs prefer multiples of 256 for very large models.
- **Default to SwiGLU.** GeGLU and SwiGLU are empirically indistinguishable in pre-training loss; pick whichever your framework has a fused kernel for. Both beat GeLU, which beats ReLU. There is no reason to use ReLU in a new architecture today.
- **Drop bias terms everywhere.** Modern LLMs omit biases in W_1 , V , and W_2 . Bias parameters add optimizer state, consume memory bandwidth on every forward pass, and have been repeatedly observed to destabilize large-scale training without any measurable quality benefit. Dropping them is free money.
- **The $8/3$ factor only applies to the $4\times$ baseline.** If your starting d_{ff} uses a different ratio-Gemma 2 uses $8\times$, the original T5-11B used $64\times$ -the matching factor changes. Always derive from parameter matching, not from memorizing ‘ $8/3$ ’.
- **Profile fused kernel availability before committing.** SwiGLU requires launching matmuls for both W_1 and V before the element-wise product. Without kernel fusion, this is two separate GEMM launches. Unfused SwiGLU can be slower in wall-clock time than fused GeLU on some inference runtimes. Verify your serving stack has a fused GLU kernel before assuming the theoretical win materializes.

How This Shows Up in Interviews

- **“Why did LLaMA switch from GeLU to SwiGLU?”** Strong answer: gating adds a learned per-dimension gate that selectively suppresses hidden activations; ablations across multiple studies show consistent perplexity gains for the same parameter and FLOP budget; Shazeer’s original GLU paper and subsequent PaLM/T5 work provided the production validation. Mention the $8/3$ factor to demonstrate you have internalized the parameter consequences.
- **“If $d_{\text{model}} = 8192$ and I want SwiGLU, what is d_{ff} ?”** Strong answer: $\frac{8}{3} \times 8192 = 21,845.\bar{3}$, round to 21,888 (nearest multiple of 128). State your rounding rationale-hardware tile alignment-not just the number.
- **Senior follow-up: “Roughly how much memory do the MLP weights consume in a 7B model with 32 layers, $d = 4096$, $d_{ff} = 11008$, stored in BF16?”** Sketch the math on a whiteboard: one matrix is $4096 \times 11008 \times 2 \text{ bytes} \approx 90 \text{ MB}$; three matrices per layer gives $\approx 270 \text{ MB}$; across 32 layers, $\approx 8.6 \text{ GB}$. That is roughly half of the model’s total 14 GB (BF16 7B weights), consistent with the rule of thumb that MLPs and attention each consume about half of model parameters.

Key Takeaways

- SwiGLU and GeGLU add a learned gate (xV) that suppresses or amplifies each hidden dimension per token; this consistently improves pre-training perplexity at equal parameter count.
- Use $d_{ff} = \frac{8}{3}d$, rounded to the nearest hardware-friendly multiple (128 or 256), to match the parameter count of a standard $4\times$ ReLU FFN.

- LLaMA-2 7B ($d = 4096$, $d_{ff} = 11,008$) is the canonical reference; verify any architecture number against it.
- Drop bias terms in all MLP linear layers—they add memory bandwidth and optimizer state with no measurable quality gain.
- Verify your runtime has a fused SwiGLU kernel; two unfused GEMM launches for W_1 and V can erase the theoretical throughput advantage.

3.3.2 Why GLU Variants Dominate Production Models

If you survey the 19 dense LLM releases from 2024 alone—Command A, OLMo 2, SmolLM, Phi-4, Gemma 3, Qwen 2.5, LLaMA 3—almost every one uses SwiGLU or GeGLU. This is not groupthink or cargo-culting. It is the result of independently run ablations at multiple labs converging on the same answer. Understanding *why* GLU variants win tells you something more durable than a recipe: it gives you a mental model for reasoning about future architecture changes that interviewers at senior levels will probe for.

The empirical case for GLU variants rests on two independent lines of evidence. Noam Shazeer’s 2020 paper evaluated every GLU variant—ReGLU (ReLU gate), GeGLU (GeLU gate), SwiGLU (Swish gate), and others—against a standard GeLU baseline on CoLA and SST-2 fine-tuning tasks. The results were unambiguous: every GLU variant outperformed the GeLU baseline, and the gains were statistically significant. GeGLU scored 84.67 on CoLA against 84.20 for the GeLU baseline, a difference that might look small until you consider these are pre-training architecture choices where 0.5% gains are hard-won.

Narang et al. (2020) extended this to pre-training loss in a T5-style study covering a wide range of architecture decisions simultaneously. The bolded top-performers in their loss comparison tables are, without exception, the GLU variants. The non-gated activations (ReLU, GeLU, ELU) cluster together below them at every scale tested.

The convergent evolution after 2022 seals the argument. PaLM (Google, 2022) used SwiGLU in a 540B-parameter model—the most expensive training run of its era—after internal ablations. T5v1.1 (the improved version of T5) switched from the original T5’s ReLU to GeGLU and achieved better downstream performance. The entire LLaMA family, Mistral, Qwen, DeepSeek, Yi, and dozens of open-weight models followed. When multiple independent labs running trillion-token training runs independently converge on the same choice, the evidence is about as strong as you ever get in empirical deep learning.

GLU variants consistently win because gating is a universally useful architectural primitive: it appears in LSTMs (forget gate), GRUs (update gate), attention (the V projection scaled by softmax weights), and now in MLP blocks.

The underlying mechanism is expressiveness for free. A standard FFN uses one matrix multiply to mix features and one pointwise function to introduce nonlinearity. A GLU FFN uses two matrix multiplies to create an activated branch and a linear gate, then combines them element-wise. The gate is not a fixed function of the input magnitude; it is learned, input-dependent, and can vary across the d_{ff} hidden dimensions independently. This effectively gives the MLP a form of conditional computation at the feature level.

The memory bandwidth argument reinforces this. As discussed in the context of RMSNorm, the transformer’s bottleneck is not FLOPs—tensor contractions are 99.8% of FLOPs but not 99.8% of wall-clock time. What matters is how often you move data between memory hierarchies. A GLU variant with $d_{ff} = \frac{8}{3}d$ moves the same amount of data as a standard FFN with $d_{ff} = 4d$ —three smaller matrices versus two larger ones at the same total parameter count. The memory bandwidth cost is identical. The quality gain is free.

The contrast with what came before clarifies the win. The pre-GLU choice was: ReLU (fast but rough, dead neurons, gradient cliffs) versus GeLU (smooth, differentiable, better quality, slightly slower to compute). GLU variants deliver GeLU-level gradient behavior plus conditional gating at the same memory cost. The iteration from the Shazeer paper to PaLM to LLaMA 1 to LLaMA 3 is the clearest example of a clean architectural improvement compounding across the field.

A Worked Example

To see why production models chose SwiGLU over a scaled-up GeLU FFN, consider two configurations for a 70B model with $d = 8192$ and 80 transformer layers.

Option A: GeLU FFN, $d_{ff} = 4d = 32,768$:

$$\text{params per layer} = 2 \times 8192 \times 32,768 = 536,870,912 \approx 537 \text{ M}$$

$$\text{total MLP params (80 layers)} = 80 \times 537 \text{ M} \approx 42.9 \text{ B}$$

Option B: SwiGLU FFN, $d_{ff} = \frac{8}{3} \times 8192 = 21,845 \rightarrow 21,888$ (rounded to 128):

$$\text{params per layer} = 3 \times 8192 \times 21,888 = 537,919,488 \approx 538 \text{ M}$$

$$\text{total MLP params (80 layers)} \approx 43.0 \text{ B}$$

The parameter counts are within 0.2% of each other. The memory bandwidth cost per forward pass is essentially identical. Yet Option B (SwiGLU) consistently achieves 0.3-0.8 nat lower perplexity. This is why every lab that ran this ablation chose SwiGLU: it is a free lunch. LLaMA-2 70B uses exactly this configuration ($d = 8192$, $d_{ff} = 28,672$, which reflects a slightly larger ratio of roughly $3.5\times$ rather than $2.67\times$, a choice that trades slight parameter increase for wider matrices).

What You Actually Decide in Practice

- **GLU is the default; exceptions require justification.** GPT-3 uses GeLU and is still a strong model, Nemotron-4 340B uses squared ReLU ($\text{ReLU}^2(x) = \max(0, x)^2$), and Falcon 2 11B uses plain ReLU. These are competitive models. But if someone asks why you chose GeLU over SwiGLU for a new pre-training run, “it’s simpler” is not a satisfying answer—you need an ablation or a systems constraint (e.g., your inference runtime has no fused GLU kernel).
- **SwiGLU and GeGLU are interchangeable in practice.** No production paper shows a measurable difference between them at the same parameter count. Pick the one with better kernel support in your stack.
- **The T5 lesson: extreme d_{ff} ratios are legal but risky.** The original T5-11B used $d_{ff} = 65,536$ against $d = 1,024$, a ratio of $64\times$. The model worked, but T5v1.1 (the improved version) dialed this back to a standard $2.5\times$ ratio with GeGLU and got better results. Very large ratios spend parameters inefficiently on width rather than depth; the Kaplan et al. scaling experiments show a wide basin ($1\times$ to $10\times$) but the optimum is around $4\times$ (or $2.67\times$ with GLU).
- **Watch for the squared-ReLU surprise.** Nemotron and some mixture-of-experts models use ReLU^2 , which is sparsity-inducing: most hidden units fire at exactly zero. This creates naturally sparse MLP activations that can be exploited for inference acceleration. If your deployment stack supports sparse kernel optimizations, squared ReLU is worth benchmarking.
- **Parameter matching is a convention, not a law.** Some labs (Mistral, PaLM) use slightly larger d_{ff} ratios than $8/3$ for their GLU models—effectively choosing to spend more parameters in the MLP. This is

a deliberate capacity allocation decision, not an error. Always check the actual model card rather than assuming every SwiGLU model uses exactly $8/3$.

How This Shows Up in Interviews

- **“Would you ever use ReLU in a new architecture? When?”** Strong answer: only with a specific justification-squared ReLU for sparsity-based inference acceleration, or a constrained runtime with no GLU kernel and GeLU not available either. For standard pre-training with a modern stack, there is no reason to choose ReLU over SwiGLU.
- **“What does it mean that LLaMA and PaLM both independently picked SwiGLU?”** Strong answer: it validates the ablation-two groups with different data, hardware, and model scales ran independent experiments and converged. When multiple frontier labs agree on an architecture choice without coordinating, that is the strongest signal available that the choice is correct. The interviewer is testing whether you reason from evidence, not just pattern-match on popular choices.
- **Senior follow-up: “You are designing a 100B-parameter model. Your inference stack runs on edge hardware where kernel fusion is limited to a single GEMM per layer. How does that change your activation function choice?”** Sketch the answer: SwiGLU requires two back-to-back GEMMs (W_1 and V) before the element-wise product; without fusion, this is two memory round-trips to HBM. On memory-constrained edge hardware, this can be a throughput bottleneck. Options: (1) fall back to GeLU (one GEMM, almost as good empirically); (2) rewrite the kernel to fuse both GEMMs into a single wider matmul; (3) use squared ReLU (one GEMM, exploits sparsity for further gains if the hardware has sparse execution units). The answer demonstrates that activation function choice is not isolated from systems constraints.

Key Takeaways

- Multiple independent labs-Google, Meta, Alibaba, Mistral-running ablations at scale all converged on SwiGLU or GeGLU; that convergence is stronger evidence than any single paper.
- GLU variants win because gating is a universal architectural primitive: the same element-wise multiplicative gate that works in LSTMs and attention works in MLP blocks.
- SwiGLU and GeGLU are empirically indistinguishable; choose based on which your runtime can fuse efficiently.
- GPT-3 (GeLU), Nemotron 340B (squared ReLU), and Falcon (ReLU) are legitimate exceptions-strong models exist without GLU, but each exception needs a reason.
- Very wide MLPs ($d_{ff} > 10 \times d$) spend parameters inefficiently; the T5-11B to T5v1.1 regression from $64\times$ to $2.5\times$ ratio is the canonical cautionary example.

3.4 Position Encodings

3.4.1 Absolute, Relative, and Sinusoidal Encodings

Transformers are permutation-invariant by design: self-attention produces identical outputs regardless of the order in which tokens appear. Position encoding is therefore not an optional add-on - it is a load-bearing architectural choice, and “how does the model know token order?” is a near-universal architecture deep-dive

question. The more revealing follow-up is “what breaks when you run the model on sequences longer than its training context?” That question separates engineers who memorized the formula from those who understand why a decade of iteration eventually converged on RoPE.

Three families of position encodings cover virtually every production model built before 2022.

Absolute learned embeddings. Add a dedicated vector $\mathbf{p}_i \in \mathbb{R}^d$ to the token embedding at position i . The $\mathbf{p}_i$ are free parameters trained end-to-end alongside everything else. This is exactly what GPT-1, GPT-2, GPT-3, OPT, and BERT use. The only constraint is that the vocabulary of positions $\{0, 1, \dots, L - 1\}$ must be declared at model initialization, and any position beyond $L - 1$ is simply undefined.

Sinusoidal (fixed) embeddings. The original *Attention Is All You Need* (2017) proposed hard-coded vectors built from paired sine and cosine functions at geometrically spaced frequencies:

$$\text{PE}(i, 2k) = \sin\left(\frac{i}{10000^{2k/d}}\right), \quad \text{PE}(i, 2k + 1) = \cos\left(\frac{i}{10000^{2k/d}}\right)$$

where i is the token position and $k = 0, 1, \dots, d/2 - 1$ indexes dimension pairs. The denominators form a geometric series of wavelengths from 2π tokens (fastest oscillating pair) up to $10000 \cdot 2\pi$ tokens (slowest). Because a fixed rotation matrix M_δ maps $\text{PE}(i)$ to $\text{PE}(i + \delta)$ for any offset δ , the model can in principle represent relative distances as linear combinations of absolute embeddings.

Relative encodings. Rather than encoding absolute position into token representations, these schemes inject a position-dependent term directly into the attention score. Shaw et al. (2018) added a learned bias a_{ij}^K to the query-key dot product for each pair (i, j) , clipped at some maximum offset. T5 generalized this with a scalar learned bias $b(i - j)$ shared across all positions at the same distance. Because the bias depends only on $i - j$, these schemes are inherently relative and do not need to define a maximum absolute position.

The axis that matters: absolute schemes bake position into token representations before attention; relative schemes fold position into the attention score computation itself.

Sinusoidal embeddings have no learnable parameters, which means they add nothing to the model’s memory footprint. Their multi-scale structure - fast oscillations for nearby tokens, slow oscillations for distant ones - lets the model distinguish positional relationships across a wide range of distances with a single d -dimensional vector. The linear-rotation property means a well-trained model can, in principle, extract relative offsets algebraically. In practice, performance degrades beyond the training context length because the model has never seen those absolute position vectors during training; the degradation is softer than with learned embeddings but still substantial.

Learned absolute embeddings are the simplest possible scheme and fast at inference (a single table lookup). The critical failure mode is hard and immediate: any sequence position beyond $L - 1$ maps to an uninitialized or out-of-distribution vector. In GPT-3 with $L = 2048$, a 2049-token sequence causes undefined behavior. There is no graceful degradation.

Relative schemes generalize to unseen lengths because the bias $b(i - j)$ is indexed by distance, not by absolute index. The trade-off is memory: T5-style relative biases require a tensor of shape $[\text{batch}, \text{heads}, n, n]$ that is $O(n^2)$ in sequence length. At $n = 4096$, 16 heads, batch size 8, in `float32`, this tensor is $8 \times 16 \times 4096^2 \times 4 \approx 8.6 \text{ GB}$ - often the single largest activation in the forward pass. Beyond 4k tokens, relative bias memory dominates everything else.

ALiBi (Press et al., 2022) sidesteps the memory issue by making the bias a fixed negative slope times distance: $-m \cdot (j - i)$, where m is a head-specific constant. No parameters, no $O(n^2)$ tensor, and strong empirical length generalization. The structural assumption, however, is a monotone recency bias: every token

always prefers recent context. This hurts tasks that require retrieving information from the beginning of a very long document.

A Worked Example

Take $d = 4$ and compute the sinusoidal vectors for positions $i = 0$ and $i = 1$. There are two dimension pairs, $k = 0$ and $k = 1$:

$$\theta_0 = \frac{1}{10000^{0/4}} = 1.0, \quad \theta_1 = \frac{1}{10000^{2/4}} = \frac{1}{100}$$

$$\mathbf{p}_0 = [\sin(0), \cos(0), \sin(0), \cos(0)] = [0, 1, 0, 1]$$

$$\mathbf{p}_1 = [\sin(1.0), \cos(1.0), \sin(0.01), \cos(0.01)] \approx [0.841, 0.540, 0.010, 1.000]$$

Pair $(0, 1)$ completes a full oscillation in $2\pi/1.0 \approx 6.3$ tokens; it is highly sensitive to whether two tokens are adjacent or a few positions apart. Pair $(2, 3)$ completes a cycle in $2\pi/0.01 \approx 628$ tokens; it varies negligibly over short windows but discriminates positions at the scale of hundreds. For a 512-token sequence, the slow pair has advanced only $512 \times 0.01 \approx 5$ radians out of 2π - a fraction of its dynamic range, preserving sensitivity for sequences well beyond 512.

This exact multi-scale geometry is what motivated RoPE's paired-dimension rotation design, described in the next subsection.

What You Actually Decide in Practice

- **Fixed context, no extrapolation needed.** Learned absolute embeddings are the right call. They are trivially fast (one embedding lookup), add a modest $L \times d$ parameter count (for $L = 2048$, $d = 4096$ that is 8 M parameters - 0.1% of a 7B model), and have no inference overhead.
- **Any use case that might require sequences longer than training context.** Do not use absolute learned embeddings. The failure is not a smooth performance cliff; it is a hard crash. If you anticipate context growth post-launch, paying the cost of RoPE from day one is always worth it.
- **Sinusoidal for new architectures.** Generally not worth choosing over RoPE. Sinusoidal requires no learned parameters, which is attractive, but it does not enable any of the context-extension algorithms that RoPE does. There is no upside to picking sinusoidal in 2024 unless you are constrained to not touching the attention layer.
- **T5 relative bias at context lengths above 4k.** The $O(n^2)$ attention bias tensor becomes the dominant memory cost before the attention matrix itself. Profile memory before committing.
- **ALiBi for simple long-context baselines.** Zero parameters, strong empirical generalization, and simple to implement. The recency-bias assumption fails for tasks that need long-range retrieval. Fine for summarization or generation; risky for multi-document QA.

How This Shows Up in Interviews

- **“Explain the sinusoidal encoding formula. Why use 10000 as the base?”** Strong answer: describe the geometric series of wavelengths from 2π to $10000 \cdot 2\pi$ covering up to ~ 63000 tokens, connect high-frequency pairs to short-range discrimination and low-frequency pairs to long-range, and mention the

rotation-matrix property that makes relative offsets linearly expressible. Weak answers quote the formula without explaining the frequency intuition.

- **“What breaks when you deploy a model trained with absolute learned embeddings on a longer context?”** Strong answer: positions beyond $L - 1$ access uninitialized or randomly initialized embedding rows; the model produces incoherent logits, typically NaN or near-uniform distributions. Mention that fine-tuning on longer sequences with the position table extended can fix this, but it requires continued training, not just architecture changes.
- **Senior follow-up:** *“Relative encodings claim to generalize to unseen lengths. Show me where that guarantee breaks down in practice.”* The catch: T5 clips relative distances at some maximum offset k ; beyond k tokens all positions are treated identically, so the model has no signal to distinguish “100 tokens away” from “10000 tokens away.” ALiBi does not clip, but its monotone penalty assumes recency always wins, which is wrong for long-context retrieval. True unlimited-length generalization requires the encoding to remain distinguishable at arbitrary distances - which is the design goal of RoPE’s continuous-frequency rotation.

Key Takeaways

- Transformers have no intrinsic sense of order; position must be injected either into token representations (absolute) or into attention scores (relative).
- Sinusoidal embeddings cover multiple frequency scales and have a linear-combination relative-offset property, but they do not generalize to sequences longer than training context.
- Learned absolute embeddings are fast and simple but fail immediately at any sequence length beyond the declared maximum.
- T5-style relative biases generalize better but impose $O(n^2)$ memory overhead; ALiBi is cheaper but bakes in a recency prior.
- The entire landscape above has been superseded by RoPE in post-2023 production models; understanding these predecessors matters primarily to explain *why* RoPE won.

3.4.2 Rotary Position Embeddings (RoPE) and Context Extension

Every frontier model released since 2022 - LLaMA 1 through 4, Mistral, Qwen, Gemma, DeepSeek, Falcon - uses Rotary Position Embeddings. If you are asked to design an LLM from scratch, choosing RoPE is almost certainly the right answer. But stating the choice is the easy part; an interviewer will immediately follow up with “explain exactly why the inner product only sees relative position,” and then, if you are interviewing for a senior role, “how would you extend a 4k RoPE model to 128k without retraining from scratch?” Both questions require understanding the mathematics, not just the name.

The design goal is a position encoding $f(\mathbf{x}, m)$ such that for any two token vectors $\mathbf{q}$ and $\mathbf{k}$ at positions m and n :

$$\langle f(\mathbf{q}, m), f(\mathbf{k}, n) \rangle = g(\mathbf{q}, \mathbf{k}, m - n)$$

The inner product must depend on the absolute positions *only* through their difference $m - n$. This rules out additive embeddings (which introduce cross terms between content and absolute position) and rules out most relative schemes (which modify attention scores but do not satisfy this exact factorization for inner products).

The observation that makes RoPE work is that inner products are invariant under shared rotations: $\langle R\mathbf{u}, R\mathbf{v} \rangle = \langle \mathbf{u}, \mathbf{v} \rangle$ for any rotation matrix R . If position m applies rotation $\mathcal{R}_m$ and position n applies rotation $\mathcal{R}_n$, then:

$$\langle \mathcal{R}_m \mathbf{q}, \mathcal{R}_n \mathbf{k} \rangle = \langle \mathcal{R}_{m-n} \mathbf{q}, \mathbf{k} \rangle$$

because $\mathcal{R}_m^T \mathcal{R}_n = \mathcal{R}_{n-m}$ for rotation groups. The right-hand side depends only on $m - n$, satisfying the design goal exactly.

In practice, rotating a d -dimensional vector as a single unit requires a $d \times d$ matrix, which is expensive. RoPE reduces this by pairing up dimensions: the vector is sliced into $d/2$ pairs, and pair k at position m is rotated by angle $m \cdot \theta_k$ in the 2D plane of that pair, where:

$$\theta_k = \frac{1}{10000^{2k/d}}, \quad k = 0, 1, \dots, \frac{d}{2} - 1$$

The rotation for pair k at position m is:

$$\begin{pmatrix} x_{2k} \\ x_{2k+1} \end{pmatrix} \mapsto \begin{pmatrix} \cos(m\theta_k) & -\sin(m\theta_k) \\ \sin(m\theta_k) & \cos(m\theta_k) \end{pmatrix} \begin{pmatrix} x_{2k} \\ x_{2k+1} \end{pmatrix}$$

This is applied to queries and keys at every attention layer, immediately after the linear projections for Q and K . Values are not rotated; they carry no positional information directly.

RoPE is the only common encoding that provably makes the attention inner product a function of relative distance alone, while requiring zero learnable parameters and no change to token embeddings.

Compared to absolute learned embeddings, RoPE adds zero parameters and produces no out-of-distribution vectors at unseen absolute positions - the rotation angles grow continuously, so position 10001 is just a small step beyond position 10000.

Compared to sinusoidal additive embeddings, RoPE's rotation acts on the query and key vectors inside the attention layer, which means the dot-product kernel sees only relative angles. Sinusoidal embeddings added to token inputs do *not* achieve this: the product $(\mathbf{e}_i + \mathbf{p}_i)^T (\mathbf{e}_j + \mathbf{p}_j)$ expands into four terms, two of which mix content with absolute position - the encoding is not purely relative.

Compared to T5 relative biases, RoPE requires no $O(n^2)$ memory. The rotation is a per-vector operation (cost $O(n \cdot d)$), not a per-pair operation. At $n = 32768$ and $d = 4096$, this difference is $32768 \times 4096 \approx 134$ M elements versus $32768^2 \approx 1$ B elements - roughly an $8\times$ reduction in the position-related memory footprint.

The geometric spacing of $\theta_k = 10000^{-2k/d}$ provides the same multi-scale frequency coverage as sinusoidal embeddings: high- k pairs (small θ) oscillate slowly and encode long-range position; low- k pairs oscillate quickly and encode fine-grained adjacency. This design also enables length-extension algorithms discussed below.

A Worked Example

Consider $d = 4$, giving two dimension pairs. The base frequencies are:

$$\theta_0 = 1.0, \quad \theta_1 = 10000^{-2/4} = 0.01$$

Take $\mathbf{q} = [1, 0, 0, 1]$ at position $m = 3$ and $\mathbf{k} = [0, 1, 1, 0]$ at position $n = 5$, so the relative gap is $n - m = 2$.

Rotating $\mathbf{q}$ at position 3:

- Pair 0: angle $3 \times 1.0 = 3$ rad. $(\cos 3, \sin 3) \approx (-0.990, 0.141)$.

- Pair 1: angle $3 \times 0.01 = 0.03$ rad. Rotating $(0, 1)$: $(-\sin 0.03, \cos 0.03) \approx (-0.030, 1.000)$.

So $\mathcal{R}_3 \mathbf{q} \approx [-0.990, 0.141, -0.030, 1.000]$.

Rotating $\mathbf{k}$ at position 5:

- Pair 0: angle $5 \times 1.0 = 5$ rad. Rotating $(0, 1)$: $(-\sin 5, \cos 5) \approx (0.959, 0.284)$.
- Pair 1: angle $5 \times 0.01 = 0.05$ rad. Rotating $(1, 0)$: $(\cos 0.05, \sin 0.05) \approx (0.999, 0.050)$.

So $\mathcal{R}_5 \mathbf{k} \approx [0.959, 0.284, 0.999, 0.050]$.

Inner product:

$$\langle \mathcal{R}_3 \mathbf{q}, \mathcal{R}_5 \mathbf{k} \rangle \approx (-0.990)(0.959) + (0.141)(0.284) + (-0.030)(0.999) + (1.000)(0.050) \approx -0.889$$

Now verify the same $\mathbf{q}$ and $\mathbf{k}$ at absolute positions 0 and 2 (same relative gap of 2): $\mathcal{R}_0 \mathbf{q} = \mathbf{q} = [1, 0, 0, 1]$. Rotating $\mathbf{k}$ at position 2: pair 0 gives $(-\sin 2, \cos 2) \approx (-0.909, -0.416)$; pair 1 gives $(\cos 0.02, \sin 0.02) \approx (1.000, 0.020)$.

$$\langle \mathbf{q}, \mathcal{R}_2 \mathbf{k} \rangle = (1)(-0.909) + (0)(-0.416) + (0)(1.000) + (1)(0.020) = -0.889$$

The inner products are identical. The absolute positions changed from $(3, 5)$ to $(0, 2)$, but the relative gap stayed at 2 and the inner product is unchanged.

What You Actually Decide in Practice

- **RoPE base θ .** The original paper used 10000. LLaMA 3 raised it to 500,000. A higher base stretches the period of the slow-frequency pairs, giving the model more resolution at long distances. Increasing the base is the simplest and most reliable way to improve long-context performance when combined with continued pretraining on long documents.
- **Context extension without retraining: position interpolation.** Chen et al. (2023) showed that dividing all absolute positions by a scale factor $s = L_{\text{new}}/L_{\text{train}}$ before applying RoPE keeps rotation angles within the trained range. The model sees positions that now look like scaled versions of what it trained on. This works but produces a mild perplexity increase; 1000-2000 fine-tuning steps on long-context data almost fully recover quality.
- **NTK-aware scaling (Bloc97).** Instead of scaling positions, scale the base: replace 10000 with $10000 \cdot s^{d/(d-2)}$. This redistributes the frequency spectrum so that high-frequency pairs are less affected, preserving short-range sensitivity while stretching the slow pairs. Works better than naive position interpolation out of the box, with no fine-tuning.
- **YaRN (Peng et al., 2023).** Applies different interpolation factors per frequency group: high-frequency pairs are left unchanged, low-frequency pairs are interpolated, mid-frequency pairs get a temperature adjustment. The most reliable technique for large extension factors ($4\times$ or more) with minimal fine-tuning. Used in several open-weight long-context derivatives.
- **RoPE applies only to Q and K , not V .** A common implementation bug is accidentally rotating the value projections. Values carry content, not position; rotating them corrupts them.

How This Shows Up in Interviews

- **“Why does RoPE make the attention score depend only on relative position?”** Strong answer: the rotation $\mathcal{R}_m$ applied to Q and $\mathcal{R}_n$ applied to K means $\langle \mathcal{R}_m \mathbf{q}, \mathcal{R}_n \mathbf{k} \rangle = \langle \mathcal{R}_{m-n} \mathbf{q}, \mathbf{k} \rangle$ because the rotations

compose as $\mathcal{R}_m^T \mathcal{R}_n = \mathcal{R}_{n-m}$. Work through this in 2D if asked on a whiteboard. Weak answers say “it uses rotation” without the inner product cancellation argument.

- **“What breaks when you try to run a RoPE model trained with context 4k on an 8k input?”** Strong answer: positions 4001-8000 produce rotation angles that the model has never encountered during training. Unlike absolute embeddings (hard failure), this degrades *gradually*: perplexity rises as position increases past training context, with the slow-frequency pairs (large k) being the first to degrade since they have the least trained coverage. The fix is either NTK-aware base scaling or continued pretraining with longer sequences.
- **Senior follow-up:** “You have a LLaMA-2 7B checkpoint trained at 4k context and you need 32k context in two weeks. Walk me through what you would actually do.” The strong answer sketches a two-stage recipe: (1) apply NTK-aware base scaling (e.g., raise base to $\approx 128,000$ for $8\times$ extension) with no retraining as an immediate fix; (2) collect a long-document corpus at the target length, run 1000-2000 gradient steps at 32k sequence length on 8-16 GPUs, using sequence packing and gradient checkpointing to fit memory; (3) evaluate on SCROLLS or LongBench to confirm perplexity recovery. Name the memory cost: at bf16 with a 7B model, a single 32k sequence activates $\approx 32 \times 32^2 \times 4096 / 10^9 \approx 4.3$ GB of KV activations per layer, so activation checkpointing is not optional.

Key Takeaways

- RoPE rotates Q and K by position-dependent angles so that the attention inner product becomes a function of relative position alone, with zero learnable parameters.
- The dimension pairing with base-10000 frequencies provides multi-scale resolution, just like sinusoidal embeddings, but applied inside the attention operation rather than at the input.
- Virtually all post-2022 production models use RoPE; convergent evidence from 19+ open model releases shows no competing scheme has beaten it at scale.
- Context extension methods - position interpolation, NTK-aware base scaling, YaRN - all work by adjusting how positions map to rotation angles; they differ in how aggressively they disturb the short-range frequency pairs that the model relies on most.
- The simplest production strategy for extending context: increase the RoPE base and fine-tune on long documents; for large extension factors ($> 4\times$), use YaRN or NTK scaling as the starting point.

Chapter 4 Hyperparameters You Will Be Asked to Justify

4.1 Width and Depth

4.1.1 Aspect Ratio and the “128 Hidden Dims per Layer” Rule

When an interviewer says “design a 7B-parameter dense decoder model,” the next question is almost always: how many layers, and how wide? This ratio—what practitioners call the *aspect ratio*—is the single structural decision that cascades into your parallelism strategy, your per-device memory footprint, and your inter-node networking requirements. Mid-level engineers treat it as a free variable; senior engineers know the empirically validated sweet spot and can argue for it on both quality and systems grounds.

The aspect ratio is the ratio of the residual stream width to the number of transformer layers:

$$\text{aspect ratio} = \frac{d_{\text{model}}}{n_{\text{layers}}}$$

It captures how a fixed parameter budget is allocated between width and depth. A model with $d_{\text{model}} = 4096$ and $n_{\text{layers}} = 32$ has aspect ratio 128; redistributing the same parameter count to $d_{\text{model}} = 8192$ and $n_{\text{layers}} = 16$ gives aspect ratio 512.

Hyperparameter sweeps across three model scales (50M, 274M, and 1.5B parameters) show that loss is minimized at aspect ratios between 100 and 128, and that this optimum is remarkably stable across scales. GPT-3 (175B, ratio 128), LLaMA-2 7B (ratio 128), LLaMA-2 13B (ratio 128), and LLaMA-2 70B (ratio 102) all land squarely in this band. That stability is the key finding: you can validate your aspect ratio at small scale and carry it forward to full training with reasonable confidence.

Width and depth are not equivalent ways to spend parameters. Width increases the rank and representational capacity of every vector in the residual stream. Depth increases the number of sequential nonlinear transformations applied. The aspect ratio controls their balance.

The rule of thumb: target roughly 100-128 hidden dimensions per layer, build your d_{model} and n_{layers} around that constraint, and you will be within the flat loss minimum for every scale that has been publicly studied.

Too narrow and too deep: each layer has low representational capacity, the residual stream bottlenecks information flow, and pipeline stages become tiny-wasting GPU utilization at stage boundaries. Too wide and too shallow: you over-invest in representation capacity at each step before enough nonlinear transformations can process it, and the very wide matrix multiplies require aggressive tensor parallelism that strains inter-GPU bandwidth.

The aspect ratio also directly determines which parallelism strategies are feasible. Wide models (high d_{model}) suit *tensor parallelism*, where each weight matrix is partitioned column-wise or row-wise across GPUs within a node. Tensor parallelism requires high-bandwidth, low-latency interconnects such as NVLink at ≥ 600 GB/s. Deep models (many layers) suit *pipeline parallelism*, where consecutive layer groups are assigned to different GPU nodes; pipeline parallelism tolerates slower inter-node networking (200 Gbps InfiniBand is usually sufficient) because communication only occurs at stage boundaries. Your cluster topology therefore constrains the aspect ratio before model quality does.

One important nuance separates pretraining loss from downstream task performance. Research comparing depth versus width at matched parameter counts finds that pretraining perplexity is insensitive to the split-total parameters dominate. But downstream fine-tuning accuracy can mildly favor deeper models at the same

parameter count. If you optimize purely around pretraining loss, aspect ratio near 128 is well validated. If downstream task capability is the primary target, there is a weak case for going somewhat deeper, within the constraint that pipeline parallelism remains efficient.

A Worked Example

Consider designing a 7B-parameter dense decoder model. Three candidate configurations:

Configuration	d_{model}	n_{layers}	Aspect ratio
Standard (recommended)	4096	32	128
Too wide	8192	14	585
Too narrow	2048	64	32

The standard configuration matches LLaMA-2 7B exactly. On a single 8-GPU H100 node with NVLink, it fits comfortably with 4-way tensor parallelism ($d_{\text{model}}/\text{TP} = 1024$ per shard, cleanly divisible). With 32 layers, pipeline parallelism across two groups of 4 GPUs assigns 16 layers per stage—a sensible granularity.

The wide variant ($d_{\text{model}} = 8192$, 14 layers) has very few pipeline stages, making coarse-grained pipeline parallelism inefficient. Effective tensor parallelism would need degree 8 or 16, requiring a dense NVLink fabric across multiple nodes—a significant hardware constraint. Attention with 64 heads at head dimension $8192/64 = 128$ is fine; the real cost is that wide matmuls do not automatically saturate hardware better unless the batch dimension is simultaneously large.

The narrow variant ($d_{\text{model}} = 2048$, 64 layers) creates very fine-grained pipeline stages. At 8 layers per stage across 8 pipeline stages, activation tensors moving between stages are small, and pipeline bubbles dominate runtime. Tensor parallelism across small matrices is unattractive because communication overhead relative to compute grows as matrix size shrinks.

The arithmetic conclusion: for a 7B model on 8 H100s, 4096 hidden dimensions and 32 layers produces a clean 4-way TP within each group of 4 GPUs and 2-way pipeline parallel across groups—no wasted hardware, minimal bubble fraction.

What You Actually Decide in Practice

- **Start at aspect ratio 128.** For a 7B model: $d_{\text{model}} = 4096$, $n_{\text{layers}} = 32$. For 13B: $d_{\text{model}} = 5120$, $n_{\text{layers}} = 40$. For 70B: $d_{\text{model}} = 8192$, $n_{\text{layers}} = 80$ (ratio ≈ 102 , close enough). These numbers match the released LLaMA-2 configs because the same reasoning applies.
- **Align d_{model} to hardware tile sizes.** Always choose d_{model} divisible by 128 at minimum; divisible by 256 is safer for H100 and future hardware. Matmul kernels use tiles of 128 or 256 elements; misaligned dimensions waste tile slots and silently hurt throughput by 10-20%.
- **Let your parallelism plan constrain the choices.** If tensor-parallel degree is 8, d_{model} must be divisible by 8 and attention head count must accommodate the split. If pipeline-parallel across 4 nodes, n_{layers} must be divisible by 4 and ideally by 8 (to allow 2-way micro-batch interleaving within a stage).
- **Wider favors tensor parallel; deeper favors pipeline parallel.** Fast NVLink within a node (≥ 600 GB/s) enables tensor parallelism at scale. Slower inter-node InfiniBand (100-400 Gbps) makes tensor parallelism across nodes expensive. Your networking budget shapes the answer before model quality does.

- **Downstream accuracy may mildly favor depth, but do not let it override systems constraints.** The quality argument for going deeper is real but modest. If adding depth would shatter pipeline efficiency or require redesigning your cluster topology, the quality gain rarely justifies the operational cost.

How This Shows Up in Interviews

- **“How would you choose d_{model} and n_{layers} for a 13B model?”**
Strong answer: apply the 128-per-layer rule to get $d_{\text{model}} = 5120$, $n_{\text{layers}} = 40$ -this matches LLaMA-2 13B. Then note that both numbers need to be divisible by the intended tensor-parallel degree, and n_{layers} must divide cleanly across pipeline stages.
- **“Why does aspect ratio matter beyond model quality?”**
Strong answer: width determines tensor-parallel degree feasibility because you partition weight matrices along the hidden dimension; depth determines pipeline stage count and stage granularity; both interact with your cluster’s networking topology in ways that can dominate cost long before quality differences appear.
- **“If pretraining loss is equal for a wide-shallow and a narrow-deep model, which do you ship?”**
Strong answer: consider downstream task performance (deeper may win), then evaluate which is cheaper to serve at inference. Narrower-but-deeper models have smaller KV cache per layer at a given d_{model} , which can reduce memory pressure during long-context generation; wider models may serve faster if tensor parallelism can saturate hardware.
- *Senior follow-up:* **“Your cluster has NVLink within 8-GPU nodes but only 200 Gbps InfiniBand between nodes. You have a 70B parameter budget. Walk through your aspect ratio and parallelism plan.”**
Sketch: with 8-way NVLink per node, tensor-parallel degree 8 is viable within a node. With slow inter-node fabric, avoid tensor parallelism across nodes; use pipeline parallelism across nodes instead. Target $d_{\text{model}} = 8192$ (divisible by 8 for TP-8), $n_{\text{layers}} = 80$ (pipeline across 4 nodes of 20 layers each, with 8-way TP within each node). Aspect ratio ≈ 102 -inside the validated band. Total GPUs: $8 \times 4 = 32$ H100s, a plausible allocation for a 70B model.

Key Takeaways

- The loss-optimal aspect ratio $d_{\text{model}}/n_{\text{layers}}$ sits between 100 and 128 across all publicly studied scales; use 128 as your default and adjust only for systems reasons.
- This optimum appears stable from 50M to 175B+ parameters, meaning small-scale experiments can validate your choice before committing to a full run.
- Aspect ratio determines which parallelism strategies are feasible: wider models favor tensor parallelism (needs NVLink), deeper models favor pipeline parallelism (tolerates slower inter-node links).
- Always align d_{model} to multiples of 128 or 256 to avoid wasted CUDA tile slots; hardware alignment is non-negotiable, not a nice-to-have.
- Downstream accuracy may mildly favor depth at equal pretraining loss, but the systems cost of going very deep is real and often overrides this preference in production.

4.1.2 Feed-Forward to Model Dimension Ratios

Every interviewer who asks you to spec out an LLM architecture will eventually ask: “what inner dimension do you use for the FFN?” Engineers who know only the $4\times$ rule freeze the moment they switch to SwiGLU and realize the formula changes. The correct answer-and the eighty-second derivation behind it-immediately separates candidates who understand why the numbers are what they are from those who have memorized a config file. This question appears across both mid-level and staff-level interviews; at staff level the follow-up involves the parameter budget algebra.

A standard transformer FFN without bias terms is:

$$\text{FFN}(x) = \sigma(xW_1) W_2$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$. Total parameter count per layer: $2 \cdot d_{\text{model}} \cdot d_{\text{ff}}$. The near-universal default is:

$$d_{\text{ff}} = 4 \cdot d_{\text{model}}$$

giving $8 \cdot d_{\text{model}}^2$ parameters per FFN layer. This is not derived from first principles-it is a convention that sits inside a wide, empirically flat loss basin and has become the community standard.

For GLU variants (SwiGLU, GeGLU), the operation introduces a gate projection:

$$\text{FFN}_{\text{GLU}}(x) = (\sigma(xW_{\text{up}}) \odot (xW_{\text{gate}})) W_{\text{down}}$$

where $W_{\text{up}}, W_{\text{gate}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}^{\text{glu}}}$ and $W_{\text{down}} \in \mathbb{R}^{d_{\text{ff}}^{\text{glu}} \times d_{\text{model}}}$. Three matrices instead of two, so the parameter count is $3 \cdot d_{\text{model}} \cdot d_{\text{ff}}^{\text{glu}}$.

To keep the GLU parameter count equal to the vanilla $4\times$ FFN:

$$3 \cdot d_{\text{model}} \cdot d_{\text{ff}}^{\text{glu}} = 8 \cdot d_{\text{model}}^2 \implies d_{\text{ff}}^{\text{glu}} = \frac{8}{3} \cdot d_{\text{model}} \approx 2.67 \cdot d_{\text{model}}$$

The rule: $4\times$ for vanilla (ReLU/GeLU) MLPs; $8/3 \approx 2.67\times$ for GLU variants. The factor of $2/3$ neutralizes the cost of the third weight matrix so the two architectures are parameter-matched by default.

The $4\times$ convention survives not because it is optimal, but because hyperparameter sweeps across multiple model scales show that the loss landscape is essentially flat from roughly $2\times$ to $8\times$. There is no quality signal that would prefer $4\times$ over $3.5\times$ or $5\times$. Having a community standard avoids accidental parameter inflation when comparing architectures and makes configs and ablations interpretable.

The $8/3$ correction for GLU models exists precisely to prevent an accidental parameter increase. If you kept $d_{\text{ff}} = 4 \cdot d_{\text{model}}$ while switching to SwiGLU, you would add a third weight matrix and inflate FFN parameters by 50%-giving the GLU model a hidden parameter advantage in any comparison with a vanilla MLP. The $8/3$ rule neutralizes this. LLaMA-1, Qwen, DeepSeek, and Yi all follow the $\approx 2.67\times$ convention closely; LLaMA-2 and Mistral run slightly higher ($\approx 2.69\times$ due to rounding) but the intent is the same.

T5’s 11B model is the instructive outlier: $d_{\text{model}} = 1024$ with $d_{\text{ff}} = 65,536$, a ratio of $64\times$. The stated rationale was that very wide inner projections create larger, more hardware-efficient matrix multiplies. This is technically true-wider matmuls can better saturate tensor cores-but the model is spending its parameter budget in a single, highly parallel expansion rather than distributing it across depth, attention, and width. T5v1.1, the improved follow-up, reverted to a $\approx 2.5\times$ GeGLU ratio and obtained a better model. That reversion is the clearest available evidence that extreme expansion ratios do not pay off.

A Worked Example

Design the FFN for a 7B-parameter dense model using SwiGLU with $d_{\text{model}} = 4096$.

Step 1: compute the target inner dimension.

$$d_{\text{ff}}^{\text{glu}} = \frac{8}{3} \times 4096 = \frac{32768}{3} \approx 10922.7$$

Step 2: round to hardware-friendly granularity. CUDA matmul kernels use tiles of 64 or 128 elements; the inner dimension must be a multiple of 64 at minimum. The two nearest multiples of 64 are 10880 and 10944; the nearest multiple of 128 above the target is 11008. LLaMA-2 7B uses 11008.

Step 3: verify parameter matching.

Vanilla $4\times$ MLP at $d_{\text{model}} = 4096$:

$$2 \times 4096 \times 16384 = 134,217,728 \text{ parameters per layer}$$

SwiGLU at $d_{\text{ff}}^{\text{glu}} = 11008$:

$$3 \times 4096 \times 11008 = 135,266,304 \text{ parameters per layer}$$

The $\approx 1\%$ discrepancy comes from rounding to the nearest multiple of 64. Across 32 layers, the total FFN delta is $\approx 33\text{M}$ parameters out of $\approx 7\text{B}$ -negligible and expected. This is why you will see minor parameter count discrepancies in model cards when comparing vanilla and GLU architectures of nominally the same size.

Step 4: check tensor-parallel compatibility. With 2-way TP, $11008/2 = 5504$, which is divisible by 64. With 4-way TP, $11008/4 = 2752$, still divisible by 64. The dimension survives any reasonable tensor-parallel degree.

What You Actually Decide in Practice

- **Always round d_{ff} to a multiple of 128.** A multiple of 64 is the minimum; 128 is safer across H100 and is required for some fused kernel implementations. Misaligned inner dimensions silently reduce matmul throughput by 10-20% because tile slots are wasted.
- **Use $4\times$ for ReLU/GeLU; use $8/3\times$ for SwiGLU/GeGLU.** Applying $4\times$ to a SwiGLU model inflates FFN parameters by 50% and breaks any fair comparison with vanilla MLP architectures. This error is common when porting GPT-style configs to LLaMA-style architectures.
- **The range $2\times$ to $8\times$ is essentially flat in pretraining loss.** You have real flexibility inside this band. Adjusting d_{ff} is a clean lever to hit a specific total parameter count without touching d_{model} , n_{layers} , or attention configuration.
- **Ratios above $16\times$ carry quality risk.** T5's $64\times$ experiment worked but T5v1.1 reverted to $2.5\times$ with a better outcome. Gemma 2 uses $8\times$ as a deliberate architectural choice, which is the most aggressive ratio with clear production validation. Beyond $8\times$, you are outside the range where multiple independent experiments have validated quality.
- **In MoE models, d_{ff} is per expert.** Track both the per-expert FFN dimension and the activated parameter count separately from total parameters. Applying the $8/3$ rule to the per-expert inner dimension is correct; do not accidentally apply it to the combined expert capacity.

How This Shows Up in Interviews

- **“You are designing a 7B model with SwiGLU. What inner FFN dimension do you use?”**
Strong answer: compute $8/3 \times 4096 \approx 10923$, round to 11008 (nearest multiple of 128). Point out this matches LLaMA-2 7B exactly. Explain that keeping $4\times$ with SwiGLU would silently add 50% more FFN parameters-an apples-to-oranges comparison.

- **“Why is the FFN expansion ratio $4\times$ instead of $2\times$ or $8\times$?”**

Strong answer: $4\times$ sits in the middle of a wide, empirically flat loss basin that spans roughly $2\times$ to $8\times$. There is no strong quality signal distinguishing values in that range. The community adopted $4\times$ as a focal point; having a standard makes configs interpretable and ablations comparable across papers.

- **“What is the FFN dimension for a SwiGLU model if $d_{\text{model}} = 8192$?”**

Strong answer: $8/3 \times 8192 = 21845.3$; round to 21888 (multiple of 128) or 22016 (another clean multiple). The final choice depends on tensor-parallel degree: if TP is 8, confirm $21888/8 = 2736$ is divisible by 64.

- *Senior follow-up:* **“With a fixed 7B parameter budget, if you increase d_{ff} from 11008 to 16384, what else must change to keep total parameters constant?”**

Sketch: FFN parameters per layer increase by $(16384 - 11008) \times 3 \times 4096 \approx 66\text{M}$. Across 32 layers that is $\approx 2.1\text{B}$ extra parameters. To stay at 7B, you must cut elsewhere—either reduce n_{layers} (which destroys the aspect ratio) or reduce d_{model} to ≈ 3584 (which changes all attention head dimensions and the embedding size). The cascade shows why FFN dimension is not an isolated knob; it couples to every other hyperparameter through the total parameter budget.

Key Takeaways

- Use $d_{\text{ff}} = 4 \cdot d_{\text{model}}$ for ReLU/GeLU MLPs; use $d_{\text{ff}} = (8/3) \cdot d_{\text{model}}$ for SwiGLU/GeGLU—the factor of $2/3$ preserves parameter count by accounting for the third (gate) weight matrix.
- Always round d_{ff} to a multiple of 128 for CUDA tile alignment; misalignment silently costs 10-20% matmul throughput.
- The loss landscape is flat between $2\times$ and $8\times$, giving genuine flexibility; values outside that range need strong justification.
- T5’s $64\times$ ratio is the cautionary tale: T5v1.1 reverted to $\approx 2.5\times$ GeGLU and improved; extremely wide FFNs trade expressiveness for hardware efficiency in a way that rarely pays off at inference.
- In MoE architectures, d_{ff} is per expert; always separate per-expert FFN parameters from total model parameters when reporting model capacity.

4.2 Attention Head Configuration

4.2.1 Head Count vs. Head Dimension Trade-Offs

“Design attention for a 13B model with $d_{\text{model}} = 5120$ ” is a standard architecture deep-dive prompt. The answer—40 heads at 128 dimensions each—resolves instantly once you internalize the governing constraint. What trips engineers is the follow-up: why this ratio, what breaks when you deviate from it, and how does the choice propagate into tensor-parallelism planning and inference memory? This question appears at mid-level and staff-level interviews alike; the staff variant adds explicit KV cache sizing and the implications of grouped-query attention.

In standard multi-head attention, queries, keys, and values are each projected via weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d_{\text{model}} \times (n_{\text{heads}} \cdot d_{\text{head}})}$. The near-universal constraint is:

$$n_{\text{heads}} \times d_{\text{head}} = d_{\text{model}}$$

This makes each projection effectively square in aggregate, and the total attention parameter count—four matrices

W_Q, W_K, W_V, W_O -stays fixed at $4 \times d_{\text{model}}^2$ per layer regardless of how many heads you use. Adding heads subdivides the existing projection space; it does not expand it.

The per-head dimension follows directly:

$$d_{\text{head}} = \frac{d_{\text{model}}}{n_{\text{heads}}}$$

For the three LLaMA-2 sizes: a 7B model with $d_{\text{model}} = 4096$ and $n_{\text{heads}} = 32$ gives $d_{\text{head}} = 128$; a 13B model with $d_{\text{model}} = 5120$ and $n_{\text{heads}} = 40$ gives $d_{\text{head}} = 128$; a 70B model with $d_{\text{model}} = 8192$ and $n_{\text{heads}} = 64$ gives $d_{\text{head}} = 128$. The pattern is not coincidence. When aspect ratio is pinned near 128 ($d_{\text{model}}/n_{\text{layers}} \approx 128$), fixing $d_{\text{head}} = 128$ and applying the constraint above mechanically produces the correct n_{heads} at every model scale.

GPT-3, LaMDA, PaLM, and all publicly documented LLaMA-family models follow the ratio-1 constraint. T5 11B is the only notable deviation: with $d_{\text{model}} = 1024$, $n_{\text{heads}} = 128$, and $d_{\text{head}} = 128$, the total attention projection is $128 \times 128 = 16,384 = 16 \times d_{\text{model}}$. The model trained, but T5v1.1 reverted to the standard ratio and achieved better perplexity.

Pin $d_{\text{head}} = 128$, derive $n_{\text{heads}} = d_{\text{model}}/128$, and you will match every production configuration that has been publicly validated; departures from this rule require explicit justification in terms of quality or systems trade-offs.

Multiple heads give attention the ability to express diverse patterns simultaneously. One head might track adjacent-token dependencies, another follow long-range coreference, a third attend to syntactic head-modifier relationships. That diversity is the core mechanism through which multi-head attention improves over single-head attention. But each head's output is a weighted combination of value vectors in a d_{head} -dimensional subspace. The smaller d_{head} is, the lower the rank of what any individual head can represent.

This is the low-rank bottleneck: with very small d_{head} (below roughly 64), the per-head attention output is intrinsically rank-limited, and the head cannot represent nuanced multi-directional aggregations even if the attention weights are well-calibrated. In practice, $d_{\text{head}} = 64$ is the minimum seen in any well-performing production architecture; 128 is the empirically validated sweet spot across all publicly studied model scales.

The opposing failure mode is too few, very wide heads. With $n_{\text{heads}} = 8$ and $d_{\text{head}} = 512$ at $d_{\text{model}} = 4096$, each head is highly expressive but there are only eight distinct attention patterns to aggregate. In practice the diversity loss dominates: fewer heads means fewer independent signals for the residual stream to combine, and the model underperforms a 32-head configuration at identical parameter count. The empirical sweet spot is n_{heads} large enough that heads genuinely specialize-typically 16 or more at production scale-with d_{head} large enough to avoid rank bottlenecks.

A Worked Example

Design attention for a 13B model: $d_{\text{model}} = 5120$, $n_{\text{layers}} = 40$.

Step 1: apply the constraint.

$$n_{\text{heads}} = \frac{d_{\text{model}}}{d_{\text{head}}} = \frac{5120}{128} = 40$$

Step 2: verify total attention parameters per layer.

$$4 \times d_{\text{model}}^2 = 4 \times 5120^2 = 4 \times 26,214,400 \approx 105\text{M}$$

Step 3: check two alternatives.

$n_{\text{heads}} = 80$, $d_{\text{head}} = 64$: total attention parameters are identical-same constraint-but each head now operates in a 64-dimensional space at the low-rank boundary. Viable, but at the limit of the validated range.

$n_{\text{heads}} = 20$, $d_{\text{head}} = 256$: same parameter count, but only 20 distinct attention patterns. Diversity loss typically outweighs the per-head expressiveness gain at this scale.

Step 4: verify tensor-parallel divisibility. With TP degree 8: $40/8 = 5$ heads per GPU shard-clean. $d_{\text{head}} = 128$ is a multiple of 64, satisfying CUDA tile alignment. Both constraints pass for the standard choice. As a contrast: if $d_{\text{model}} = 5376$ were chosen, $n_{\text{heads}} = 5376/128 = 42$, and $42/8 = 5.25$ -not clean, requiring a revision to d_{model} before locking the architecture.

Step 5: estimate the MHA KV cache per token per layer (bf16).

$$2 \times n_{\text{heads}} \times d_{\text{head}} \times 2 \text{ bytes} = 2 \times 40 \times 128 \times 2 = 20,480 \text{ bytes} = 20 \text{ KB}$$

At 4096 tokens and batch size 8 across 40 layers:

$$20,480 \times 4096 \times 8 \times 40 \approx 26.8 \text{ GB of KV cache}$$

This is the MHA KV cache baseline that motivates grouped-query attention at larger scale.

What You Actually Decide in Practice

- **Pin $d_{\text{head}} = 128$ and derive n_{heads} .** Every production model between 7B and 175B with publicly documented architecture uses $d_{\text{head}} = 128$. Start here, compute $n_{\text{heads}} = d_{\text{model}}/128$, and round only if forced to by divisibility constraints.
- **Enforce the ratio-1 constraint:** $n_{\text{heads}} \times d_{\text{head}} = d_{\text{model}}$. Violating this changes total attention parameter count and creates non-comparability with standard baselines. If you expand attention projection beyond d_{model} (as T5 11B did), count those extra parameters explicitly in your model card-they are not free.
- **Make n_{heads} divisible by TP degree before finalizing d_{model} .** If tensor-parallel degree is 8, n_{heads} must be a multiple of 8. This constraint, combined with the ratio-1 rule and the aspect-ratio target, leaves only a discrete set of valid d_{model} values. Enumerate these early-discovering a conflict after the training run has started is expensive.
- **Do not go below $d_{\text{head}} = 64$.** Models with $d_{\text{head}} = 32$ or $d_{\text{head}} = 16$ show measurable quality penalties from low-rank per-head bottlenecks. GPT-2 small ($d_{\text{head}} = 64$) represents the minimum with well-documented strong performance; anything below is outside the validated range.
- **The MHA KV cache per token is $4 \times d_{\text{model}}$ bytes.** This fixed per-token cost is the direct motivation for grouped-query attention. For a 70B model at $d_{\text{model}} = 8192$, MHA costs 32 KB per token per layer; at 100K context and batch 16 across 80 layers, that is 4.2 TB-clearly infeasible. GQA cuts this by a factor of the group size G .

How This Shows Up in Interviews

- **“For a 7B model with $d_{\text{model}} = 4096$, how many attention heads do you use and why?”**
Strong answer: $n_{\text{heads}} = 32$, giving $d_{\text{head}} = 128$. This satisfies $n_{\text{heads}} \times d_{\text{head}} = d_{\text{model}}$, keeps total attention parameters at 4×4096^2 per layer, and hits the empirically validated $d_{\text{head}} = 128$ target. Point out this is exactly LLaMA-2 7B’s configuration, which validates the reasoning.
- **“What happens if you double the number of attention heads to 64 for a 7B model?”**
Strong answer: d_{head} drops from 128 to 64-at the low-rank boundary. Total attention parameters are unchanged. Diversity of patterns increases, but per-head expressiveness drops. In practice, no frontier model at 7B scale has found benefit from this change; the additional diversity does not compensate for the reduced per-head capacity.

- *Senior follow-up:* “Your cluster uses TP-8. You want a 70B model with $d_{\text{model}} = 8192$. Walk through your head count and verify the tensor-parallel constraint, including for GQA.”

Sketch: target $d_{\text{head}} = 128$, so $n_{\text{heads}} = 8192/128 = 64$. TP-8 divisibility: $64/8 = 8$ heads per GPU shard-clean. For GQA-8 ($n_{\text{kv}} = 8$): $8/8 = 1$ KV head per shard-clean, just barely. For GQA-4 ($n_{\text{kv}} = 16$): $16/8 = 2$ -also fine. The dangerous case is $n_{\text{kv}} = 6$: $6/8$ is not an integer, requiring either a different TP degree or a different GQA configuration.

Key Takeaways

- The universal constraint is $n_{\text{heads}} \times d_{\text{head}} = d_{\text{model}}$; this fixes total attention parameters at $4 \times d_{\text{model}}^2$ per layer regardless of head count.
- Target $d_{\text{head}} = 128$ and derive n_{heads} from it; this matches GPT-3, PaLM, and all LLaMA-family models at every scale.
- Avoid $d_{\text{head}} < 64$; the low-rank per-head bottleneck produces measurable quality degradation below this threshold.
- n_{heads} must be divisible by tensor-parallel degree; verify this before finalizing d_{model} , as the three constraints (aspect ratio, ratio-1, TP divisibility) interact.
- MHA KV cache per token per layer is $4 \times d_{\text{model}}$ bytes (bf16); this fixed cost per-token is what makes grouped-query attention necessary at production scale and long contexts.

4.2.2 Consensus Choices Across Frontier Models

Multi-head attention is the right choice during training. The problem surfaces when your model enters production: at batch size 32 and 8K context, the KV cache for a 70B MHA model can exceed the aggregate HBM capacity of a four-H100 serving node. Grouped-query attention (GQA) is how frontier labs have resolved this without sacrificing model quality, and it is now a prerequisite in any serious inference system design discussion. This question appears at senior and staff level as part of a serving architecture deep-dive, and the expected answer goes beyond naming GQA to computing the KV cache size and justifying the group count.

Three variants of multi-head attention differ only in how many distinct key-value heads exist:

- **Multi-head attention (MHA):** $n_{\text{kv}} = n_q$. Each query head has its own W_K and W_V . Full representational capacity. Full KV cache per token.
- **Multi-query attention (MQA):** $n_{\text{kv}} = 1$. All query heads share a single W_K and W_V . KV cache reduced by factor n_q . Shows measurable quality loss at large model scale.
- **Grouped-query attention (GQA):** $n_{\text{kv}} = n_q/G$ for group size G . Groups of G query heads share W_K and W_V . KV cache reduced by factor G . Quality loss is empirically negligible for $G \leq 8$.

The KV cache per token per layer in bf16 is:

$$\text{KV bytes per token per layer} = 2 \times n_{\text{kv}} \times d_{\text{head}} \times 2$$

With the ratio-1 constraint ($n_q \times d_{\text{head}} = d_{\text{model}}$), this becomes:

$$\begin{aligned} \text{MHA:} & \quad 4 \times d_{\text{model}} \text{ bytes} \\ \text{GQA-}G &: \quad 4 \times d_{\text{model}}/G \text{ bytes} \\ \text{MQA:} & \quad 4 \times d_{\text{head}} \text{ bytes} \end{aligned}$$

GQA directly scales down the KV cache by the group factor G with no change to the query projection.

GQA at $G = 4-8$ is the current frontier consensus: it reduces KV cache by $4-8\times$ with quality degradation that is empirically indistinguishable from MHA on standard benchmarks.

The key intuition is that query diversity matters more than key-value diversity. Queries express what each head is searching for; they need to be distinct to capture different aspects of the context simultaneously. Key and value representations encode the information being retrieved-multiple query heads can retrieve from the same K/V pool effectively because the queries direct where to look. GQA preserves the full query projection (n_q distinct W_Q matrices) while sharing the representation being queried.

The memory-quality Pareto is well-characterized. MQA-sharing a single KV head across all query heads-saves the most memory but causes measurable quality degradation at large model scale: having one KV head for 64 or more query heads is too aggressive a compression of the context representation. GQA at $G = 4$ to $G = 8$ matches MHA on perplexity and downstream benchmarks when trained from scratch. GQA at $G \geq 16$ begins to show degradation comparable to MQA for models above 30B parameters. The sweet spot is $G = 8$ for 70B-class models; $G = 4$ is increasingly common for 7B-13B models.

The memory reduction is not just a convenience-it is what enables longer context windows and larger serving batches. For a fixed HBM budget, GQA-8 allows either $8\times$ more tokens in the context, $8\times$ more requests in a batch, or some combination of both. This has a direct and quantifiable effect on serving throughput.

A Worked Example

LLaMA-2 70B configuration: $d_{\text{model}} = 8192$, $n_q = 64$, $n_{\text{kv}} = 8$ (GQA-8), $d_{\text{head}} = 128$, $n_{\text{layers}} = 80$.

Step 1: KV cache per token per layer.

$$2 \times 8 \times 128 \times 2 \text{ bytes} = 4,096 \text{ bytes per token per layer}$$

Step 2: total KV cache at serving parameters (context 4096, batch 8).

$$4,096 \times 4096 \times 8 \times 80 \approx 10.7 \text{ GB}$$

Step 3: compare with MHA ($n_{\text{kv}} = 64$).

$$10.7 \times 8 = 85.7 \text{ GB of KV cache}$$

At this configuration MHA KV cache alone equals or exceeds the entire 80 GB HBM of a single H100-before a single byte of model weights.

Step 4: model weight memory. 70B parameters in bf16: $70 \times 10^9 \times 2 = 140 \text{ GB}$. Across 4 H100s with TP-4: $140/4 = 35 \text{ GB per GPU}$.

Step 5: total memory per GPU (GQA-8). Assuming KV cache is sharded by TP degree: $10.7/4 \approx 2.7 \text{ GB per GPU for KV}$. Total: $35 + 2.7 = 37.7 \text{ GB}$ -well within the 80 GB limit, leaving 42 GB of headroom for activations, longer sequences, or increased batch size.

With MHA instead: $85.7/4 \approx 21.4 \text{ GB per GPU KV cache}$. Total: $35 + 21.4 = 56.4 \text{ GB per GPU}$ -still fits, but leaves only $\sim 24 \text{ GB}$ of headroom, making 8K context or batch sizes above 8 impractical. At context 8192 with MHA the KV cache doubles to 171 GB total; GQA-8 at 8192 context is $10.7 \times 2 = 21.4 \text{ GB}$ -still comfortable.

What You Actually Decide in Practice

- **Use MHA for models $\leq 7\text{B}$ serving short contexts.** At 7B ($d_{\text{model}} = 4096$), the MHA KV cache per token is half that of a 70B model, and memory pressure is manageable for batch sizes under 16 and contexts under 4K. Once either dimension grows, switch to GQA-4.

- **Default to GQA-8 for 13B-70B models; GQA-4 for 7B-13B.** LLaMA-2 70B, LLaMA-3 70B, and most frontier 70B-class models use $n_{kv} = 8$. LLaMA-3 8B uses $n_{kv} = 8$ with $n_q = 32$ (also GQA-4). These represent the production-validated sweet spots on the memory-quality Pareto.
- **Reserve MQA for extreme memory constraints.** MQA's quality penalty is real and documented at 70B+ scale. Use it only when even GQA-8 does not fit your HBM budget—for example, on inference chips with ≤ 8 GB of memory or edge deployments.
- **n_{kv} must be divisible by tensor-parallel degree.** This is the most common configuration error when adapting a GQA model to a new cluster. With TP-8, n_{kv} must be ≥ 8 and a multiple of 8. GQA-8 on a 64-head model gives $n_{kv} = 8$ —exactly 1 KV head per shard at TP-8. GQA-4 on the same model gives $n_{kv} = 16/2$ per shard, also clean. GQA-4 on a 40-head model gives $n_{kv} = 10$ —not divisible by 8, requiring either a different GQA group size or a different TP degree.
- **GQA is a training-time decision, not an inference-time optimization.** You cannot convert a trained MHA model to GQA at inference without retraining. The W_K and W_V weights are distinct per head in MHA; merging them post-hoc by averaging degrades quality measurably. If GQA is needed for your serving cluster, specify it before the training run starts.

How This Shows Up in Interviews

- **“Why does LLaMA-2 70B use GQA instead of standard multi-head attention?”**
Strong answer: KV cache memory. At 4096 context and batch size 8, MHA requires ~ 86 GB of KV cache for a 70B model—exceeding a single H100's HBM and leaving no headroom for weights alongside the cache on any reasonable serving cluster. GQA-8 reduces this to ~ 11 GB, fitting comfortably within a 4-GPU serving configuration. Quality on benchmark evaluations is statistically identical.
- **“What is the difference between MQA and GQA-8, and when do you choose each?”**
Strong answer: both reduce KV cache by sharing W_K and W_V across query heads. MQA uses one KV head (maximum reduction, by factor n_q); GQA-8 uses $n_q/8$ KV heads (reduction by factor 8). GQA-8 shows negligible quality loss at 70B scale; MQA shows measurable degradation for large models. Choose GQA-8 as the default; use MQA only when even GQA-8 exceeds your HBM budget.
- **Senior follow-up: “You are serving a 70B GQA-8 model ($n_q = 64$, $n_{kv} = 8$, $d_{\text{head}} = 128$, 80 layers) at context 8192, batch 32, TP degree 4. Compute the KV cache per GPU and assess whether it fits on an H100.”**
Sketch: KV bytes per token per layer = $2 \times 8 \times 128 \times 2 = 4,096$. Total KV cache = $4,096 \times 8192 \times 32 \times 80 \approx 85.9$ GB. Sharded by TP-4: $85.9/4 \approx 21.5$ GB per GPU. Model weights: $140/4 = 35$ GB per GPU. Total: $35 + 21.5 = 56.5$ GB—fits within 80 GB with ~ 23 GB headroom for activations. The configuration is viable.

Key Takeaways

- MHA, MQA, and GQA differ by n_{kv} : equal to n_q (MHA), 1 (MQA), or n_q/G (GQA); KV cache scales linearly with n_{kv} and is the dominant memory constraint during inference.
- GQA at $G = 4-8$ is the frontier consensus: 4-8 $\times$ KV cache reduction with quality loss empirically indistinguishable from MHA; MQA causes measurable degradation at 70B+ scale.
- n_{kv} must be divisible by TP degree; this is the single most common configuration error when adapting a GQA architecture to a new cluster topology.

- GQA must be specified at training time-post-hoc MHA-to-GQA conversion degrades quality and is not a standard deployment practice.
- Size your serving cluster by computing MHA KV cache first ($4 \times d_{\text{model}} \times \text{seq} \times \text{batch} \times n_{\text{layers}}$ bytes in bf16), then dividing by G ; leave 20-30% headroom for activations and metadata.

4.3 Vocabulary Size and Tokenization Impact

4.3.1 Monolingual vs. Multilingual Vocab Sizing

An interviewer asks you to justify your tokenizer vocabulary size. You say 32,768. They follow up: the model will serve 50 languages and handle code. The number sounds embarrassingly small. Vocabulary size is one of those hyperparameters that feels like an implementation detail until you realize it directly controls sequence length, embedding matrix cost, and whether speakers of minority languages pay a three-to-four times token tax compared to English speakers. This question appears at every seniority level, but the depth of expected answer scales sharply with the role.

The vocabulary is the set of distinct token identifiers the model recognizes. A BPE tokenizer builds this set by starting from raw bytes (256 elements) and greedily merging the most frequent adjacent pair into a new token, repeating until a target vocabulary size is reached. Each merge compresses the corpus: two common adjacent tokens become one. The vocabulary size V is therefore both a compression budget and a coverage budget simultaneously.

The compression quality of a tokenizer is captured by its *compression ratio*: the average number of bytes encoded per token. If a tokenizer achieves a ratio of c on a given corpus, then a document of B bytes produces $L = B/c$ tokens. For GPT-2's 50,257-token BPE vocabulary trained on English web text, this ratio is roughly 1.6 bytes per token on English prose. Byte-level tokenization achieves exactly 1.0 by definition, which forces sequences to be four to eight times longer than BPE for the same text—a prohibitive cost for any attention-based model.

A larger vocabulary lets BPE find and merge higher-frequency multi-character patterns, raising the compression ratio. A smaller vocabulary forces rare character sequences to survive as many short tokens, inflating sequence lengths. The cost side: each extra vocabulary element adds one row to the input embedding matrix and one row to the output unembedding matrix, contributing $2 \times V \times d_{\text{model}}$ parameters total. For a model with $d_{\text{model}} = 4096$, moving from $V = 32,768$ to $V = 131,072$ adds roughly 800 million parameters purely in the embedding tables—parameters that are accessed sparsely and do not dominate training FLOPs.

The vocabulary size is a compression-versus-embedding-cost dial: a larger V shortens sequences at the price of a heavier embedding matrix, and the right setting shifts dramatically once you add languages beyond English.

Monolingual English models settled around 30,000-50,000 tokens (GPT-2 at 50,257; early LLaMA-1 at 32,000) because English text at those sizes achieves compression ratios comfortably above 1.5 and the embedding overhead stays manageable. Once you add non-Latin scripts—Arabic, Chinese, Korean, Thai, Hindi—the situation breaks. Those scripts contain characters underrepresented in a 50k vocabulary trained on English-dominant web text. A single CJK character that appears infrequently in the tokenizer training corpus may not survive to become its own merge and must instead be encoded as three or four byte-level fragments. This *fertility problem* means a 50-character Chinese sentence may tokenize to 150-200 tokens while its English translation of 60 characters tokenizes to 70. The model runs an invisible length tax on non-English users.

The resolution is to train the tokenizer on a multilingual corpus with controlled per-language sampling, then expand V to 100,000-250,000. The extra merge budget provides enough operations to build high-frequency multi-character tokens for each language. The compression ratio for previously underserved languages improves substantially: multilingual-aware tokenizers can reduce token counts for many non-English languages by 30-60% compared to an English-only 50k vocabulary at the same model size. The embedding overhead is real-roughly 800M to 1.6B extra parameters for a 4k-7k hidden dimension at 128k-256k vocab-but those parameters are cheap: sparse access means they never dominate compute.

A secondary benefit of a larger vocabulary is reduced sequence length for code. Tokenizers that recognize programming language keywords, operators, and common identifiers as single tokens cut token counts for source files by 20-40% versus generic text tokenizers, which matters when processing long files near context limits.

A Worked Example

Consider a multilingual assistant with $d_{\text{model}} = 4096$ and 32 transformer layers.

Option A: 50,000-token English-centric BPE vocabulary.

- Embedding and unembedding parameter cost: $2 \times 50,000 \times 4096 \approx 410\text{M}$ parameters.
- A 128-character Hindi sentence tokenizes to roughly 160-200 tokens (many Devanagari characters split into byte pairs).
- At a context budget of 2,048 tokens, practical Hindi context is limited to around 1,300 characters.
- Compression ratio on Hindi: approximately 0.6-0.8 bytes per token-well below 1.0, meaning bytes per token is less than one because multibyte characters get split.

Option B: 128,000-token multilingual BPE vocabulary.

- Embedding and unembedding cost: $2 \times 128,000 \times 4096 \approx 1,048\text{M}$ parameters-an increase of roughly 640M parameters, or about 1.2 GB in bf16.
- The same 128-character Hindi sentence tokenizes to roughly 60-80 tokens, a 2.5x improvement.
- Under the same 2,048-token budget, Hindi context expands to around 3,300 characters.
- English compression ratio is largely unchanged at ~ 1.6 bytes/token.

The embedding table grows by 640M parameters-negligible relative to the 28B+ non-embedding parameters in a typical production model of this size. The gain in usable context for underserved languages is substantial and directly observable in downstream task quality on non-English benchmarks.

What You Actually Decide in Practice

- **English-only or narrow-domain model:** 32k-64k tokens is sufficient. Early LLaMA at 32k, GPT-2 at 50k, and Mistral 7B at 32k all demonstrate strong English performance at these sizes. Adding more vocabulary for a monolingual English use case yields diminishing returns past 64k.
- **Multilingual or production assistant:** Target 100k-200k. GPT-4's tiktoken vocabulary is 100,277; LLaMA-3 expanded to 128,256; Qwen2 uses 151,936. These numbers reflect the point where per-language compression ratios plateau for the major world languages plus code.
- **Tokenizer training corpus balance matters as much as vocabulary size:** A 128k vocabulary trained on 95% English data will still underserve minority languages. When training the tokenizer, oversample rare languages-if your pre-training corpus is 80% English and 20% other, train the tokenizer on a corpus that is 60% English and 40% other to force more merge budget toward non-English character patterns.

- **Embedding parameter cost is rarely the bottleneck:** A 128k vocabulary at $d = 4096$ costs about 1 GB in bf16. A 70B parameter model has roughly 130 GB of total weights. The embedding fraction is under 1%. Do not sacrifice multilingual coverage to save embedding memory.
- **The vocabulary is frozen after pre-training:** This is the most expensive mistake to fix. If you launch with a 32k vocabulary and later discover poor Korean performance, you cannot surgically add Korean tokens without retraining. Vocabulary extension via continued pre-training is imperfect and requires substantial additional compute. Pick the right vocabulary size before the first GPU-hour of pre-training runs.

How This Shows Up in Interviews

- **“Why did you pick that vocabulary size, and how did you decide?”** A strong answer names the target language mix, the compression ratio measured on held-out samples per language, the embedding parameter budget, and the irreversibility of the decision. Weak answers pick 32k because “that is what LLaMA used.”
- **“If I tell you the model will serve users in 40 languages, what changes?”** Cover the fertility problem for non-Latin scripts, the need to train the tokenizer on a balanced multilingual corpus, and the target range of 100k-200k tokens. Note that sampling strategy for tokenizer training matters: training on an English-dominant corpus defeats the purpose of a larger vocabulary.
- **Senior follow-up: “We are doing continued pre-training to improve Hindi on an existing 32k-vocab model. Walk me through the risks.”** The strong answer addresses: (1) vocabulary extension requires adding new embedding rows initialized randomly while existing rows carry trained representations—new and old parameters need separate learning rate treatment; (2) the model’s output distribution encodes assumptions about token probability that conflict with the new vocabulary’s statistics; (3) extended continued pre-training is required before the new tokens contribute meaningfully; and (4) the correct engineering decision is to train from scratch with the right vocabulary upfront, and vocabulary extension is an expensive compromise accepted only under strong resource constraints.

Key Takeaways

- Monolingual models: 32k-64k tokens. Multilingual or production systems: 100k-200k tokens. The jump is driven by the fertility problem in non-Latin scripts, where English-centric vocabularies produce 2-4x more tokens per character.
- A larger vocabulary reduces sequence length for underserved languages by 2-3x, directly expanding usable context and reducing inference cost per query.
- Embedding parameter overhead from 50k to 128k at $d = 4096$ is roughly 640M parameters (≈ 1.2 GB in bf16)—under 1% of total weight in a 70B model and not a meaningful constraint.
- Tokenizer training corpus balance matters as much as vocabulary size: a large vocabulary trained on English-dominant data will still underserve minority languages.
- The vocabulary is frozen at pre-training. There is no cheap fix after the fact.

4.3.2 Tokenizer Selection and Downstream Cost

A product manager asks why API costs for Japanese users are running three times higher than for English users, and your engineering lead cannot explain it in the next five minutes. This is a tokenization fertility

problem, and understanding it completely requires tracing the path from raw bytes to billed tokens. Tokenizer selection is also one of the few decisions that constrains every downstream system: sequence length budgets, KV-cache memory, training FLOPs, and per-query billing all live downstream of the tokenizer. The inability to swap tokenizers after pre-training makes this decision unusually high-stakes for an apparently low-level component.

Every token costs compute. At training time, the forward-backward pass cost scales roughly linearly with sequence length (ignoring the quadratic attention term). At inference time, prefill latency, decode cost, and KV-cache memory all scale with token count. Hosted API pricing is quoted per token, so the tokenizer’s compression quality directly determines the dollar cost of a given piece of text.

The key measurement is the *fertility* of a tokenizer on a specific text domain: the average number of tokens produced per unit of input (per word, per character, or per byte). Low fertility means high compression-fewer tokens, lower cost, longer effective context window. Fertility is not a fixed property of the tokenizer; it is a joint property of the tokenizer and the input distribution. A tokenizer trained on English web text will have low fertility on English and high fertility on Arabic or Thai.

Formally, if a tokenizer achieves a compression ratio c bytes per token on a given domain, a document of B bytes produces

$$L = \frac{B}{c}$$

tokens. Since inference cost scales as $\mathcal{O}(L)$ for prefill and $\mathcal{O}(L^2)$ for full attention over a sequence, the compression ratio has a multiplicative effect on total compute. A tokenizer with $c = 2.0$ bytes per token generates 50% fewer tokens than one with $c = 1.0$ on the same input-cutting prefill cost in half and allowing twice as much text to fit in the same context window.

The tokenizer is a silent cost multiplier: every downstream metric-sequence length, memory footprint, training FLOPs, inference billing-scales with the inverse of the compression ratio on the target domain.

BPE remains the dominant tokenization algorithm at production scale not because it is theoretically optimal but because it is simple, invertible, fast, and empirically effective. Its adaptive nature-merging pairs from the actual training corpus-means the vocabulary becomes a compact statistical summary of whatever text it was trained on. This is exactly the right property: a tokenizer that does maximum compression work per token on the specific data distribution it will encounter.

The failure mode is distribution mismatch between the tokenizer training corpus and the inference domain. An English-centric BPE tokenizer processes Arabic as a stream of bytes or very short byte-pair fragments because Arabic characters appear rarely enough in an English corpus that they never accumulate merge frequency. For a 100-character Arabic sentence, an English-only 50k BPE tokenizer might produce 150 tokens while a properly trained multilingual tokenizer produces 40-a 3.75x cost difference on that query alone.

A secondary cost that often goes unnoticed is the *unembedding projection* at inference time. For each generated output token, the model must compute a full matrix-vector product against the unembedding matrix of shape $V \times d_{\text{model}}$. At $V = 128,000$ and $d_{\text{model}} = 4096$ in bf16, this matrix occupies roughly 1 GB of memory and must be accessed on every decode step. In small models (sub-1B parameters), the unembedding projection can represent 30-50% of the total parameter count and becomes a meaningful memory-bandwidth bottleneck during autoregressive decode.

Alternatives to BPE-character-level, byte-level, unigram language model-have been studied but have not displaced BPE at frontier scale. Byte-level models eliminate the fertility problem entirely but produce sequences four to eight times longer than BPE for typical English text, making quadratic attention costs prohibitive. Unigram LM tokenizers (used by SentencePiece) achieve similar compression ratios to BPE and are more principled

statistically, but have not demonstrated consistent downstream quality advantages. Until architectures that process bytes efficiently at frontier scale are proven, BPE variants remain the practical default.

A Worked Example

You are building an API for legal document summarization. Documents average 4,000 bytes of English legal prose. Evaluate two tokenizer options.

Option A: Generic English BPE, $V = 50,257$ (GPT-2 style)

- Compression ratio on legal text: $c \approx 1.4$ bytes/token. Legal text uses many long Latin-derived words (“indemnification,” “notwithstanding,” “hereinafter”) that appear rarely enough in the tokenizer training corpus to not have been fully merged.
- Tokens per document: $4,000/1.4 \approx 2,857$ tokens.
- At a hosted model price of \$2.00 per million input tokens: cost per document $\approx \$0.0057$.

Option B: Domain-adapted BPE trained on legal corpora, $V = 64,000$

- Compression ratio on legal text: $c \approx 2.0$ bytes/token. Frequent legal phrases now appear as single tokens.
- Tokens per document: $4,000/2.0 = 2,000$ tokens.
- Cost per document: $\approx \$0.0040$.
- Savings: 30% lower per-document cost, and 30% more legal content fits in the same context window.

The tradeoff is that Option B requires training a custom tokenizer and pre-training or continued pre-training of the model with the new vocabulary. Option A allows immediate use of an existing foundation model. The domain-specific tokenizer makes economic sense only when document volume is large enough to amortize the re-training cost. At a million documents per day, the cost difference is \$1,700/day—a breakeven point reached quickly at scale.

What You Actually Decide in Practice

- **Reusing an existing tokenizer:** If you are fine-tuning or adapting a pretrained model, use its tokenizer without modification. Changing the vocabulary invalidates the entire embedding and unembedding matrix and forces a full pre-training run. The cost of tokenizer-domain mismatch—higher fertility, more tokens—is almost always lower than the cost of a wasted pre-training run.
- **Training a new tokenizer:** Justified only when building a new model from scratch. Train BPE on a representative sample of the target corpus—200M to 1B characters is typically sufficient for merge statistics to stabilize. Oversample underrepresented domains and languages relative to their share in the pre-training corpus: a 70/30 English/other split in the tokenizer training data when the pre-training corpus is 90/10 gives minority languages better coverage without hurting English compression quality materially.
- **Measure fertility before committing:** Before finalizing vocabulary size and training the tokenizer, compute compression ratios on held-out samples from every domain and language you care about. Any domain producing a ratio below 1.2 bytes/token is a red flag—fertility will be very high and costs will be disproportionately elevated for users in that domain. Fix the tokenizer, not the model.
- **Context window arithmetic is token arithmetic:** When users report fitting fewer documents than expected in the context window, check the tokenizer fertility on their specific input type. A model with a 128k-token context window effectively provides a much shorter context for high-fertility text such as dense mathematical notation, structured data, or low-resource language scripts.

- **Embedding weight tying:** Many models tie the input embedding matrix and the output unembedding matrix (they share the same weights), saving $V \times d_{\text{model}}$ parameters at no measured quality cost. At $V = 128,000$ and $d = 4096$, that is 524M parameters-about 1 GB in bf16. Always verify whether weight tying is used before reporting total parameter counts; the difference between tied and untied vocabularies at this size is significant.

How This Shows Up in Interviews

- **“Why are our API costs higher for Japanese users than for English users?”** A strong answer immediately names the tokenizer fertility problem: Japanese text produces more tokens per character under an English-trained vocabulary because Japanese character patterns were underrepresented in the tokenizer training corpus and did not accumulate enough merge frequency. The answer quantifies the expected multiplier (2-4x for CJK versus English) and outlines the fix: measure compression ratios per language, and if the model is being rebuilt, address vocabulary balance in the new tokenizer training data.
- **“Can we retrain the tokenizer and swap it into our existing model?”** No, not without full re-pretraining. The embedding and unembedding matrices are indexed by token ID; a new vocabulary means new IDs, which means the embedding rows no longer correspond to the trained representations. A strong answer explains this clearly and then discusses the cost of alternatives: continuing pre-training with vocabulary extension (imperfect, expensive), serving a separate model for high-fertility languages, or accepting the cost differential.
- **Senior follow-up: “Your team is proposing a byte-level model to eliminate tokenization entirely. What would you need to see before committing to that architecture at production scale?”** The senior answer names three things: (1) a credible solution to the 4-8x sequence length inflation-either subquadratic attention, very efficient sliding window attention, or a hierarchical architecture that first groups bytes before processing; (2) empirical evidence at scale that byte-level models match BPE models on standard benchmarks, which has not yet been demonstrated at frontier sizes; and (3) a plan for the inference serving infrastructure, since the KV-cache size scales with sequence length and a 5x sequence length increase dramatically changes memory requirements per concurrent request.

Key Takeaways

- Tokenizer compression ratio is a direct multiplier on inference cost, training FLOPs, and effective context length. A domain mismatch that cuts fertility by 2x doubles your token count and your cost on that domain.
- BPE fertility is not a fixed property of the tokenizer-it is a function of how well the tokenizer training corpus matched the inference domain. Measure fertility on held-out samples before finalizing vocabulary choices.
- Never change the tokenizer of an existing pretrained model. The embedding and unembedding matrices are tied to token IDs; a vocabulary swap requires full re-pretraining from scratch.
- Embedding weight tying (sharing input embedding and output unembedding weights) saves $V \times d_{\text{model}}$ parameters at no quality cost-roughly 1 GB at $V = 128k$, $d = 4096$ in bf16. Verify whether it is used before counting parameters.
- Byte-level tokenization is architecturally elegant but produces 4-8x longer sequences than BPE for typical text; it has not been proven viable at frontier scale and requires either subquadratic attention or hierarchical architectures to become practical.

4.4 Regularization in Pre-Training

4.4.1 Why Dropout Went Out of Fashion

When an interviewer asks how you regularize a pre-training run, the answer that separates practitioners from textbook readers is: you almost certainly do not use dropout. This question surfaces regularly at mid-to-senior LLM engineer interviews whenever the conversation moves from architecture into training configuration. Getting it wrong signals that you are importing fine-tuning intuitions into a fundamentally different regime - and that you have not thought carefully about what pre-training actually is at a systems level. The stakes are higher than they appear: understanding *why* dropout is inappropriate leads directly to understanding what regularization tools *do* apply, and when.

Dropout randomly zeros a fraction p of activations during each forward pass. During training, each activation unit is retained with probability $(1 - p)$, and in inverted dropout (the standard modern implementation), activations are rescaled by $\frac{1}{1-p}$ so that the expected magnitude is preserved at test time. The objective is to prevent co-adaptation: no single neuron can rely on the persistent presence of a specific upstream neuron, because that upstream unit may be zeroed on the next step. The network must learn redundant, distributed representations, and the ensemble of all exponentially many thinned sub-networks produces a model with lower generalization error.

This argument rests on one critical assumption: the model can overfit the training data. Given sufficient capacity and insufficient data, the model will memorize training examples and fail to generalize. Dropout introduces stochastic noise during training that prevents this memorization. In image classification with tens of thousands of examples, in question answering with carefully curated datasets, this assumption is almost always valid.

Pre-training for LLMs violates this assumption completely. A 7B-parameter model trained on 1T tokens sees approximately 143 tokens per parameter over the entire training run. Each individual training example - a web page, a book passage, a code file - is seen exactly once. There is no mechanism by which the model can memorize specific examples, because each example occupies a negligible fraction of the parameter update history. Overfitting, in the classical sense, does not occur.

Dropout is a regularizer that prevents overfitting; LLM pre-training is a regime where overfitting cannot occur; therefore dropout is solving a problem that does not exist.

The empirical evidence is unambiguous. When you measure the gap between training loss and validation loss with and without dropout across pre-training runs at multiple scales, the gap is negligible in both conditions. Neither run overfits. The difference between the two conditions is not in generalization but in optimization efficiency: the run with dropout shows a consistently higher training loss throughout, because each gradient step is computed on a stochastically degraded view of the network.

At high learning rates with large-batch training - the standard regime for pre-training frontier models - the gradient signal is already averaged over thousands of tokens per step. Adding dropout atop this introduces additional variance without providing any compensating benefit. The network takes longer to converge to any given loss level, and the final training loss is worse. The validation loss tracks the training loss, confirming that the degradation is not a regularization win but a pure optimization loss.

The contrast with fine-tuning is worth making explicit. When you fine-tune a pre-trained model on a task-specific corpus of tens of thousands of examples and run for multiple epochs, the model can absolutely overfit. The architecture is identical; the training regime is completely different. Dropout at $p = 0.05-0.1$ is

often beneficial in that setting, particularly in the later transformer layers where task-specific representations are being adapted. Applying the same treatment to pre-training is a category error that conflates two regimes with fundamentally different statistical properties.

There is also a hardware efficiency argument. Dropout requires generating a random binary mask at every dropout site per forward pass. With 32 layers, two dropout sites per layer (post-attention and post-FFN), and a batch of 4×10^6 tokens per step, that is 2.56×10^8 random number generations per step. This is not the dominant cost, but it is nonzero, and across 200,000-500,000 training steps on thousands of accelerators, it accumulates into compute that could be spent elsewhere.

A Worked Example

Consider a 7B-parameter decoder-only model trained for one epoch on 1T tokens, with global batch size 4096 sequences of length 4096 tokens ($\approx 4 \times 10^6$ tokens per step), running for approximately 245,000 steps on H100s.

Configure two runs: Run A with dropout=0.0, Run B with dropout=0.1 applied at all post-attention and post-FFN sites across 32 layers.

At step 50,000 (roughly 20% through training), Run B has a loss ≈ 0.03 -0.05 nats higher than Run A. This deficit is not because Run A is overfitting - the validation losses track training losses identically in both runs. Run B's gradient updates are simply less informative: each step sees a different stochastic degradation of the network, injecting variance that the optimizer must average out over subsequent steps.

At the end of training, the train-to-validation loss gap in both runs is < 0.01 nats - confirming that neither run is overfitting. But Run B's final training loss is 0.02-0.05 nats worse than Run A's. That delta, small in absolute terms, translates into measurable downstream degradation on benchmarks such as MMLU and HellaSwag. You paid a dropout tax and received no regularization dividend.

What You Actually Decide in Practice

- **Set dropout to zero for pre-training, explicitly.** Do not rely on defaults. Many open-source training harnesses (Megatron-LM, NeMo, earlier versions of GPT-NeoX) have nonzero dropout defaults inherited from supervised learning use cases. Always override with dropout=0.0 in your pre-training config.
- **Re-enable dropout for fine-tuning on small datasets.** If your fine-tuning corpus has fewer than a few hundred thousand examples and you run more than one epoch, dropout at $p = 0.05$ -0.1 applied to the upper layers is a reasonable starting point. The overfitting regime has returned.
- **Distinguish dropout from other stochastic training techniques.** Stochastic depth (dropping entire residual blocks with some probability) and attention dropout (zeroing attention weights rather than activations) are distinct from standard dropout. Both have niche use cases in pre-training research, but neither is standard practice for production runs.
- **One-epoch assumption is load-bearing.** If your pre-training setup uses data repetition - for example, a high-quality curated subset upsampled $10\times$ relative to web data - those repeated samples do create an overfitting surface. In practice, this is handled via data-mixing ratio tuning rather than dropout, because dropout's effect is uniform across all tokens regardless of whether they are repeated.
- **Audit GPT-3 era codebases carefully.** GPT-3 used residual dropout and embedding dropout during pre-training - a choice made before the community had accumulated the ablation evidence against it. Codebases derived from GPT-2 or GPT-3 may silently retain these settings.

How This Shows Up in Interviews

- **“Walk me through how you’d configure regularization for a 7B pre-training run.”** A strong answer distinguishes pre-training from fine-tuning immediately. For pre-training: dropout zero, weight decay enabled (see next subsection), z-loss for softmax stability, gradient clipping. For fine-tuning on small data: dropout re-enabled in upper layers, early stopping, potentially higher weight decay.
- **“Why don’t modern LLMs use dropout during pre-training?”** Answer skeleton: (1) pre-training is one pass over the data - the model cannot overfit individual examples; (2) the overfitting risk that motivates dropout does not exist; (3) dropout adds gradient noise without any compensating benefit; (4) empirically, the train-to-val gap is negligible with or without it, but the absolute training loss is worse with dropout.
- **Senior follow-up: “GPT-3 used dropout during pre-training. Was that a mistake, or is there something you’re missing?”** A senior answer avoids the trap of retroactively labeling it a mistake. At the time GPT-3 was trained (early 2020), the empirical case that pre-training overfitting was a non-issue at 175B parameters had not been established through public ablations. The team also used dropout partly as a training stability hedge, not purely for regularization. The lesson is that best practices in this field are empirically derived and lag model scale - what was reasonable caution in 2020 became unnecessary overhead by 2022. A senior candidate also notes that GPT-3’s dropout rates were small ($p = 0.1$) and their impact on the final model was likely minor compared to other training decisions.

Key Takeaways

- Dropout prevents overfitting; pre-training on internet-scale data for one epoch has no overfitting risk; therefore dropout has no benefit during pre-training.
- Removing dropout is now universal consensus among frontier labs - set it explicitly to zero and verify your codebase defaults.
- Dropout re-enters the picture for fine-tuning on small corpora with multiple epochs.
- The diagnostic: if adding dropout changes your train-to-val gap, overfitting is happening; if it does not change the gap but raises training loss, dropout is only hurting you.
- The broader principle: always verify that a regularization technique is solving a problem that actually exists in your training regime before applying it.

4.4.2 Weight Decay and Learning-Rate Interactions

If the previous subsection convinced you that pre-training does not need regularization because overfitting is not a risk, then weight decay is about to complicate your picture. Nearly every frontier LLM pre-training run uses weight decay - and weight decay is textbook L2 regularization. This apparent contradiction catches engineers off guard, and resolving it correctly requires understanding a mechanism that has nothing to do with the bias-variance tradeoff. When interviewers push past “we use weight decay to prevent overfitting” and ask *why it actually helps when overfitting is not occurring*, the answer reveals whether you have genuinely internalized the dynamics of large-scale optimizer behavior or are pattern-matching to supervised learning intuitions.

Weight decay adds a penalty to the loss proportional to the squared L2 norm of model parameters:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \frac{\lambda}{2} \sum_i \theta_i^2$$

In standard SGD, this is equivalent to subtracting $\lambda\theta_i$ from each parameter at every step - hence the name

weight decay. In AdamW, the correct implementation for adaptive optimizers, the decay is applied directly to the parameters *after* the Adam update rather than being folded into the gradient:

$$\theta_{t+1} = (1 - \eta\lambda) \theta_t - \eta \cdot m_t / (\sqrt{v_t} + \epsilon)$$

where m_t and v_t are the first and second moment estimates. This decoupling matters: plain Adam with an L2 gradient penalty scales the effective decay rate per-parameter by $1/\sqrt{v_t}$, making it non-uniform and often counterproductive at large scale.

The standard motivation for weight decay in supervised learning is regularization: large weights signal over-reliance on specific features, and penalizing them pushes the model toward simpler hypotheses that generalize better. In LLM pre-training, this framing is wrong. When you sweep weight decay across typical values ($\lambda \in \{0.01, 0.1, 0.3\}$) and hold all else equal, the train-to-validation loss gap does not meaningfully change. Overfitting is not occurring either way. What changes instead is the absolute training loss - specifically, how much further the optimizer can descend during the learning rate cooldown phase at the end of training.

In LLM pre-training, weight decay is not a regularizer; it is an optimizer-conditioning mechanism that synergizes with the learning rate schedule to achieve lower final training loss.

The mechanism becomes visible when you study training loss curves under different combinations of weight decay and learning rate schedule shape. Consider a cosine decay schedule that takes a peak learning rate $\eta_{\max}$ down to a minimum $\eta_{\min} \approx \eta_{\max}/10$ over the full training run.

Through the high-learning-rate phase - roughly the first 80% of training - weight decay acts as a headwind. The optimizer is making large parameter updates, and weight decay is simultaneously shrinking all parameters toward zero. The net effect is that the model with weight decay trains more slowly during this phase: at any given step count, its training loss is slightly higher than the zero-weight-decay baseline. This is exactly the behavior you would expect from any regularization that conflicts with the gradient signal.

But the picture reverses in the cooldown phase. As η drops from $\eta_{\max}$ toward $\eta_{\min}$, the magnitude of gradient updates shrinks, and the optimizer begins to settle into a local basin. The model without weight decay has allowed parameters to grow large throughout training, pulled by whatever gradient directions were most informative at high learning rates. Large parameters create sharp, irregular loss landscape curvature: the Hessian eigenvalues in high-magnitude directions are large, making it hard for small gradient steps to traverse the terrain cleanly.

The model with weight decay has maintained smaller parameter magnitudes throughout. Weight decay has effectively been imposing a soft bound on the parameter scale, trading some optimization speed during the high-LR phase for a smoother, better-conditioned parameter neighborhood at the end. When the learning rate cools, this model sits in a flatter basin, and small gradient steps can navigate further down the loss landscape. The result is a lower final training loss - and, critically, a lower validation loss in lockstep, since overfitting is not occurring. The benefit is real and it is an optimization quality improvement, not a regularization artifact.

A Worked Example

You are training a 13B model with AdamW: $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, peak learning rate $\eta_{\max} = 3 \times 10^{-4}$, cosine decay to $\eta_{\min} = 3 \times 10^{-5}$ over 500,000 steps. Global batch size: 2048 sequences of 4096 tokens ($\approx 8.4 \times 10^9$ tokens per epoch).

Run A: $\lambda = 0.0$. Run B: $\lambda = 0.1$.

Steps 1-400,000 (high-LR phase): Run A loss ≈ 0.02 -0.03 nats lower than Run B at each checkpoint. Run B is paying the weight-decay headwind tax.

Steps 400,000-500,000 (cooldown phase, η decaying from $\sim 10^{-4}$ to 3×10^{-5}): Run B begins to close the gap. The gradient steps are now small enough that the well-conditioned parameter landscape of Run B allows more efficient descent.

Final result: Run B ends with a training loss ≈ 0.02 - 0.05 nats *lower* than Run A. The train-to-val gap in both runs: < 0.01 nats. The conclusion is unambiguous - weight decay improved optimizer convergence, not generalization.

The optimal λ for a given run depends on $\eta_{\max}$ and the schedule shape. At $\eta_{\max} = 3 \times 10^{-4}$, $\lambda = 0.1$ is a strong default. If you push $\eta_{\max}$ to 10^{-3} (aggressive but used in some runs), reduce λ to 0.05 to avoid making the high-LR phase too slow.

What You Actually Decide in Practice

- **Use AdamW, not Adam with L2 penalty.** The distinction is not cosmetic. With plain Adam and an L2 gradient term, the effective per-parameter decay is $\lambda/\sqrt{v_i + \epsilon}$. Parameters with high gradient variance get less decay; parameters with low gradient variance get more. This breaks the intended uniform conditioning effect. AdamW applies λ uniformly and directly to the parameter values after the Adam step. Verify your framework is using the correct implementation - some older codebases still default to the wrong one.
- **Exclude embeddings and normalization parameters from weight decay.** Decaying embedding matrices shrinks all token vectors uniformly toward zero, collapsing the diversity of token representations independently of their actual usage frequency. RMSNorm scale parameters and any learned bias terms in normalization layers are scale hyperparameters, not weights in the usual sense, and should also be excluded. Modern frameworks support parameter groups with separate hyperparameters for this purpose.
- **$\lambda = 0.1$ is the empirical consensus for most configurations.** Values below 0.01 produce negligible conditioning effects. Values above 0.3 impose a headwind in the high-LR phase severe enough to outweigh the tail-of-training benefit at standard learning rates. The effective range is 0.05-0.2; if you are uncertain, start at 0.1.
- **Weight decay interacts with schedule shape, not just magnitude.** With a trapezoidal or Warmup-Stable-Decay (WSD) schedule - long stable phase at $\eta_{\max}$ followed by a sharp decay - the benefit of weight decay is concentrated in the final decay phase. You can think of the stable phase as building up the conditioned parameter landscape, and the decay phase as harvesting it. With cosine decay, the interaction is more gradual but qualitatively the same.
- **Gradient clipping is a separate intervention and must be configured independently.** Gradient clipping ($\|g\|_2 \leq \tau$, typically $\tau = 1.0$) prevents individual catastrophic gradient steps and is standard alongside weight decay. The two solve different problems: clipping prevents explosive single steps; weight decay conditions the parameter space over the full training run. Never mistake one for the other in an interview.

How This Shows Up in Interviews

- **“If pre-training doesn’t overfit, why does every frontier model use weight decay?”** The correct answer immediately reframes: weight decay in pre-training is not preventing overfitting. It is improving final training loss by conditioning the parameter landscape for more effective optimization during the learning rate cooldown. The train-to-val gap does not change with or without weight decay; the absolute loss level does. The benefit mechanism is optimizer dynamics, not regularization.

- **“What’s the difference between Adam and AdamW, and why does it matter?”** Skeleton: In Adam with L2, the weight decay term enters the gradient, which then passes through the adaptive scaling $1/\sqrt{v_i}$. The effective decay per parameter varies with gradient history, breaking the intended conditioning effect. AdamW applies the decay directly to the parameter after the Adam update, making it uniform across all parameters. At large model scale, this implementation difference produces measurably different training dynamics, not just theoretical differences.
- **Senior follow-up: “Can you give an intuitive explanation for why weight decay and cosine LR decay interact the way they do - beyond just citing empirical results?”** A senior answer builds the following chain: weight decay maintains a tighter distribution of parameter magnitudes throughout training, which constrains the loss landscape curvature around the current parameter values. At high learning rates, the optimizer is taking large steps and the curvature constraint matters less - gradient direction dominates. As the learning rate decreases, the optimizer’s step size falls into the regime where local curvature is the binding constraint. A model sitting in a smoother neighborhood (achieved through weight decay) can take those small steps more efficiently and descend further. Formally, this connects to the relationship between weight regularization and implicit sharpness reduction: weight decay is known to reduce the dominant eigenvalue of the Hessian in over-parameterized models, and low-sharpness minima are exactly the basins that small-step optimizers can most effectively exploit.

Key Takeaways

- Weight decay in LLM pre-training is an optimizer-conditioning mechanism, not a regularizer - it improves final training loss, not the train-to-val gap.
- Always use AdamW with explicit λ ; never use plain Adam with an L2 gradient penalty for large-scale pre-training.
- Exclude token embeddings and normalization parameters from weight decay; apply it only to projection weights and attention matrices.
- Standard value is $\lambda = 0.1$ for most configurations; reduce to 0.05 if using an aggressive peak learning rate above 10^{-3} .
- The benefit concentrates in the tail of training, during LR cooldown - the diagnostic is that a model trained with weight decay should finish lower in training loss than one without it, while both should show the same negligible train-to-val gap.

Chapter 5 Stability Tricks in Large-Scale Training

5.1 Softmax Instabilities

5.1.1 Output-Side Z-Loss

When a training run at 50B tokens shows periodic loss spikes that slowly recover before eventually cascading into full divergence, the post-mortem almost always circles back to the output softmax normalizer. This topic surfaces at the senior and staff level whenever the interview pivots from architecture to training stability: “How do you stabilize a pre-training run at scale?” Getting the answer right requires understanding not just that the z-loss exists but precisely what it is regularizing and why the problem compounds as model scale grows. It is one of those interventions where the mechanism is simple, the cost is negligible, and not knowing about it is an immediate signal that you have not shipped a large training run.

The output distribution of a language model is a softmax over the full vocabulary V :

$$p_i(x) = \frac{e^{u_i(x)}}{Z(x)}, \quad Z(x) = \sum_{j=1}^V e^{u_j(x)}$$

where $u_i(x)$ are the pre-softmax logits. The quantity $Z(x)$ is the partition function. The cross-entropy loss is:

$$\mathcal{L}_{\text{CE}} = -\log p_y(x) = -u_y(x) + \log Z(x)$$

The numerical problem is $Z(x)$. As the model trains and learns to make confident predictions, the logits u_j grow in magnitude. If the mean logit magnitude reaches 6 with a vocabulary of 128k tokens, $Z \approx 128000 \cdot e^6 \approx 5 \times 10^7$, giving $\log Z \approx 17.7$. In bfloat16 - the standard training dtype for frontier runs - the mantissa has only 7 bits, providing roughly 3 decimal digits of precision. When Z is 5×10^7 and a token’s probability is e^{u_j}/Z for a small u_j , the result underflows to zero. The optimizer receives a zero gradient for most of the vocabulary and loses the signal it needs to refine predictions across rare tokens.

The z-loss adds an auxiliary term to the objective that penalizes large values of $\log Z(x)$:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \alpha \cdot (\log Z(x))^2$$

where α is a small scalar. PaLM introduced this in production with $\alpha = 10^{-4}$, explicitly as a training stability mechanism. Minimizing this term drives $\log Z \rightarrow 0$, which implies $Z \rightarrow 1$, which means the model is implicitly encouraged to keep its logits centered near zero.

The gradient of the z-loss with respect to logit u_j is:

$$\frac{\partial}{\partial u_j} \alpha (\log Z)^2 = 2\alpha \log Z \cdot p_j$$

This correction is nonzero only when $\log Z \neq 0$ and is proportional to the token’s own probability - it applies the largest correction to the highest-probability token, which is exactly where the logit growth originates.

The z-loss does not change what the model learns to predict; it changes where the model places mass in logit space, keeping the partition function numerically tractable throughout training.

When $Z(x) \approx 1$, the softmax simplifies to $p_i \approx e^{u_i}$. There is no large-denominator cancellation, no log-sum-exp subtracting two enormous similar numbers. The Jacobian $\partial p_i / \partial u_j = p_i(\delta_{ij} - p_j)$ remains well-conditioned because no probability mass has collapsed to zero. The model can still express confident predictions - it does so with moderate logits and a normalizer near one, rather than with huge logits and an offsetting huge denominator.

The alternative - relying on gradient clipping alone - fails for runs longer than a few hundred billion tokens. Gradient clipping controls the global gradient norm but does not prevent Z from drifting. A model can accumulate a slowly increasing $\log Z$ that stays below the clipping threshold at each individual step but compounds over tens of thousands of steps until a single bad batch tips it into a spike. The z-loss prevents this drift by applying a persistent, differentiable pressure that grows proportionally with the drift itself.

The compute cost of the z-loss is essentially zero. $\log Z(x)$ is already computed as part of the softmax during the forward pass. The auxiliary loss requires only a squaring, a scaling by α , and the gradient correction during the backward pass. Across 500k training steps on 4096 H100s, this contributes nothing measurable to wall-clock time.

A Worked Example

Consider a 70B-parameter model: vocabulary 128k, trained in bfloat16, global batch 4M tokens per step, $\alpha = 10^{-4}$ for the z-loss.

Without z-loss, at step 200k: Suppose the mean logit magnitude has drifted to $\bar{u} \approx 6.0$. Rough estimate: $Z \approx 128000 \cdot e^6 \approx 5 \times 10^7$, so $\log Z \approx 17.7$. A token with logit $u_j = 0$ contributes probability $e^0/5 \times 10^7 = 2 \times 10^{-8}$. In bfloat16, this underflows. The optimizer sees a zero gradient for all such tokens. Training continues, but the logit distribution is degenerating.

With z-loss, at step 200k: The auxiliary gradient $2\alpha \log Z = 2 \times 10^{-4} \times 17.7 \approx 3.5 \times 10^{-3}$ is added per token at every step. This is small per step but has been applied continuously since step one. The cumulative effect has suppressed $\bar{u}$ to ≈ 2.0 , giving $Z \approx 128000 \cdot e^2 \approx 9.4 \times 10^5$ and $\log Z \approx 13.8$. The probability of a token at logit zero is now $e^0/9.4 \times 10^5 \approx 10^{-6}$, still representable in bfloat16. No loss spike occurred at step 200k where one would otherwise have been likely.

The $\log Z$ metric tells the story cleanly: a run without z-loss shows $\log Z$ trending from ≈ 5 at step 1k to ≈ 18 at step 200k. A run with z-loss shows $\log Z$ stabilizing below 10 after the first 10k steps and remaining bounded throughout.

What You Actually Decide in Practice

- **Set $\alpha = 10^{-4}$ as your starting point.** This is PaLM's value, independently validated by OLMo 2 and DCLM. Values above 10^{-3} visibly hurt training loss by over-constraining the logit scale. Values below 10^{-5} have negligible effect on $\log Z$ drift.
- **Apply z-loss from step zero, not as a retrofit.** Once $\log Z$ has drifted to 15-20, retroactively adding z-loss causes a transient spike as the logit distribution is pulled back. The intervention must be proactive.
- **Log $\log Z(x)$ per training step as a health signal.** This scalar is free to compute - it is already computed during the forward pass. A $\log Z$ that is monotonically increasing with no plateau is a leading indicator of a future spike, visible days before the spike appears in the loss curve.
- **Do not confuse z-loss with label smoothing.** Label smoothing changes the target distribution, training the model to be less confident. Z-loss does not change what the model is trained to predict - it only constrains the logit scale. They solve different problems and can be used together.
- **Treat z-loss as mandatory above 30B parameters.** At 7B, $\log Z$ drift is slow enough that training often completes before a spike occurs. At 70B trained for 2T tokens, the cumulative drift is substantial. At 100B and beyond, in my experience, running without z-loss is gambling on the absence of a known failure mode.

How This Shows Up in Interviews

- **“Your 70B pre-training run is showing periodic loss spikes at 50B and 150B tokens. What do you investigate?”** Answer skeleton: first distinguish a data quality spike (a bad batch artifact, which recovers in 1-2k steps) from a structural drift issue (which recovers slowly and recurs). Check gradient norms for correlated spikes. Then inspect $\log Z(x)$ - if it has been trending upward for tens of thousands of steps before each spike, you have logit scale drift. The fix is to add z-loss from the next run’s start.
- **“What is z-loss and when would you add it?”** Answer skeleton: an auxiliary term $\alpha(\log Z(x))^2$ added to the cross-entropy loss; forces the output softmax normalizer toward one; prevents logit scale drift that causes numerical instability in bfloat16; use $\alpha = 10^{-4}$; apply from step one; costs nothing in compute; does not change target predictions, only logit scale.
- **Senior follow-up: “Why does $Z(x)$ drifting cause specific problems in bfloat16 that would not appear in fp32?”** A senior answer explains the mantissa bit difference directly: fp32 has 23 mantissa bits (≈ 7 decimal digits of precision); bfloat16 has 7 mantissa bits (≈ 3 decimal digits). When $Z(x) = 10^7$ and a token’s probability is 10^{-8} , fp32 represents this as a denormal but nonzero; bfloat16 rounds it to exact zero. The log-sum-exp trick prevents overflow in computing $\log Z$ itself, but it does not prevent underflow in the resulting probabilities $p_j = e^{u_j}/Z$ when $u_j \ll u_{\max}$. The model then receives zero gradients for those tokens and loses the signal needed to refine predictions over the low-probability vocabulary.

Key Takeaways

- The output softmax normalizer $Z(x)$ grows naturally during training as the model becomes more confident; in bfloat16, large Z causes token probabilities to underflow to zero and gradients to vanish for most of the vocabulary.
- Z-loss adds $\alpha(\log Z)^2$ to the cross-entropy objective, forcing $\log Z \rightarrow 0$ without changing the model’s target distribution.
- Use $\alpha = 10^{-4}$, apply from step one, and $\log \log Z(x)$ every step as an early-warning health metric.
- Pioneered by PaLM; subsequently adopted by Baichuan 2, DCLM, and OLMo 2; increasingly considered mandatory for runs above 30B parameters in bfloat16.
- Mechanistically distinct from label smoothing: z-loss controls logit scale, not target confidence.

5.1.2 Attention-Side Soft-Capping of Logits

When an attention head begins routing every query token to the same one or two key positions - what practitioners call attention collapse - the root cause is almost always unbounded logits feeding the attention softmax. This failure mode surfaces in senior and staff interviews whenever the conversation reaches training stability: “What happens to your attention mechanism late in training, and how do you prevent it from degrading?” Two distinct interventions exist: QK-norm and logit soft-capping. Knowing when to use each, and understanding the concrete trade-off between them, is what separates a strong answer from a rote recitation of technique names.

The attention computation for head h at layer ℓ is:

$$A_{ij} = \text{softmax} \left(\frac{q_i \cdot k_j}{\sqrt{d_k}} \right)_j, \quad \text{output} = AV$$

The pre-softmax logits are $l_{ij} = q_i \cdot k_j / \sqrt{d_k}$. As training progresses, the norms of q and k grow - the model has no built-in mechanism to prevent this. With head dimension $d_k = 128$ and unit-RMS query and key vectors (achiev-

able after warm-up), the maximum possible logit from a perfectly aligned pair is $\sqrt{d_k} \cdot \gamma_q \gamma_k \approx \sqrt{128} \approx 11.3$. But norms are not unit-RMS late in training; logits of 30-80 are observed in unstabilized runs. When one logit dominates a row by a large margin, the softmax concentrates: $\text{softmax}([60, 0, \dots, 0]) \approx (1, 0, \dots)$. The head reduces to a positional lookup, its expressivity gone. Worse, the softmax Jacobian $\partial p_i / \partial l_{ij} = p_i(\delta_{ij} - p_j)$ vanishes everywhere except the dominant position, and gradient flow through the attention mechanism effectively stops.

QK-norm applies a learned LayerNorm to the query and key projections *before* the dot product:

$$l_{ij} = \frac{\text{LayerNorm}(q_i) \cdot \text{LayerNorm}(k_j)^T}{\sqrt{d_k}}$$

The LayerNorm fixes the per-component RMS of q and k to its learned scale parameter γ , regardless of how large the raw projections become. At initialization with $\gamma_q = \gamma_k = 1$, the maximum logit is bounded by $\sqrt{d_k}$. Training can increase γ , but the growth is controlled by gradient descent operating on the loss signal rather than by unchecked norm amplification.

Logit soft-capping takes the complementary approach: it bounds the dot products *after* they are computed. Given raw logits l_{ij} , the soft-capped logit is:

$$l_{ij}^{\text{cap}} = s \cdot \tanh\left(\frac{l_{ij}}{s}\right)$$

where s is the soft-cap hyperparameter. For $|l_{ij}| \ll s$, $\tanh(l/s) \approx l/s$ and the logit passes through nearly unchanged. For $|l_{ij}| \gg s$, the tanh saturates at ± 1 and the logit is clipped to $(-s, s)$.

QK-norm prevents logit growth at the input to the dot product by bounding vector norms; soft-capping prevents it at the output by clipping large values - the choice between them is not cosmetic, as they have measurably different effects on both stability and final quality.

QK-norm is the more principled intervention. By controlling the norms of q and k before the inner product, it addresses the root cause rather than the symptom. The model retains full expressivity over what it attends to, because the learned γ parameters adjust the effective temperature of the attention distribution. Critically, QK-norm allows the use of more aggressive learning rates: because the feedback loop causing logit explosion is broken, the optimizer can push harder without triggering instability. NVIDIA’s ablation on training stability confirms this - adding QK-norm improves final perplexity relative to the baseline, specifically because it permits higher peak learning rates.

Soft-capping is simpler to implement and requires no additional learned parameters, but it introduces a nonlinearity with a gradient-killing property: the derivative of $s \cdot \tanh(l/s)$ with respect to l is $1 - \tanh^2(l/s)$, which approaches zero as $|l| \rightarrow \infty$. This means that logits which have already grown past roughly $2s$ receive vanishingly small gradients through the capping operation. Soft-capping prevents softmax collapse but does not force the model to represent attention distributions compactly - it merely hides the symptoms from the softmax while the underlying logit norms continue to drift. The same NVIDIA study finds that soft-capping at $s \in [30, 50]$ either matches or slightly worsens baseline perplexity, because the tanh distortion changes the relative contrast between large logits, impeding the model’s ability to express sharply peaked attention distributions when they are genuinely needed.

Both QK-norm and soft-capping originated outside pure language model training. QK-norm was introduced for training ViT-22B (a 22-billion parameter vision transformer), where large-scale instability first became a pressing practical problem. It migrated into language through multimodal models - Chameleon, Idefics - before being adopted in pure-text production runs at DCLM and OLMo 2. Soft-capping was developed within the Gemma 2 training effort. This lineage matters: both techniques were motivated by empirical failure modes

at scale, not by theoretical arguments.

A Worked Example

Consider a 13B-parameter model, 40 attention heads, $d_k = 128$, sequence length 4096, training in bfloat16.

Soft-capping behavior at different logit magnitudes (using Gemma 2's local-attention cap $s = 50$):

- $l = 5$: $l^{\text{cap}} = 50 \cdot \tanh(0.10) \approx 4.99$ - unchanged, cap has no effect.
- $l = 30$: $l^{\text{cap}} = 50 \cdot \tanh(0.60) \approx 26.8$ - a 10.7% reduction; the relative ordering of large logits is slightly compressed.
- $l = 80$: $l^{\text{cap}} = 50 \cdot \tanh(1.60) \approx 46.1$ - a 42% reduction; a logit that would have completely dominated the softmax is now only moderately larger than the cap value.
- $l = 200$: $l^{\text{cap}} = 50 \cdot \tanh(4.00) \approx 49.9$ - effectively clipped to the cap; any logit beyond this is indistinguishable.

The implication: for a head where the maximum raw logit is 80 and the second-largest is 40, their soft-capped values are 46.1 and 34.7, giving softmax probabilities roughly proportional to $e^{46.1}$ and $e^{34.7}$. The ratio $e^{46.1}/e^{34.7} = e^{11.4} \approx 9 \times 10^4$ - still extremely peaked, but no longer $e^{80}/e^{40} = e^{40} \approx 2 \times 10^{17}$. The softmax is no longer numerically degenerate.

QK-norm behavior at initialization: With $\gamma_q = \gamma_k = 1$, the RMS of each normalized query component is 1. The maximum logit is:

$$l_{\max} = \frac{\|\hat{q}\|_2 \cdot \|\hat{k}\|_2}{\sqrt{d_k}} = \frac{\sqrt{d_k} \cdot \sqrt{d_k}}{\sqrt{d_k}} = \sqrt{d_k} = \sqrt{128} \approx 11.3$$

This bound holds throughout training, because LayerNorm always outputs unit per-component RMS scaled by γ . If the model needs sharper attention at a particular head, it will learn $\gamma > 1$ for that head - but the growth is slow, smooth, and controlled by the gradient, not by runaway norm amplification.

What You Actually Decide in Practice

- **Choose QK-norm for new production runs.** It addresses the root cause of logit explosion rather than the symptom. Implement it as a standard LayerNorm applied to the query and key projections before the dot product, with γ initialized to 1.0 and no bias term. The additional parameter count is 2 scalars per head per layer - negligible.
- **If using soft-capping, use $s = 30$ -50.** Gemma 2 uses $s = 50$ for local attention layers and $s = 30$ for global attention layers. Below $s = 10$, the tanh nonlinearity distorts attention patterns throughout training, not just for pathological logits, hurting quality from early on.
- **Do not stack QK-norm and soft-capping without ablating.** They are not additive - you are restricting logit magnitude through two mechanisms simultaneously, and the interaction can over-constrain attention heads during early training when the model needs to form its initial routing patterns. Pick one.
- **Monitor per-head 99th-percentile logit magnitude as a training health metric.** A logit distribution that is monotonically increasing with no plateau signals incipient collapse, often 5k-20k steps before it manifests as a loss spike. Log this scalar per layer per checkpoint.
- **Apply whichever technique you choose from step one.** Retrofitting QK-norm mid-run causes a distribution shift in the attention patterns that spikes the loss; the model must re-learn its routing structure under the new normalization regime. Retrofitting soft-capping is less disruptive but still causes a transient perturbation. Neither is a viable mid-run rescue operation.

How This Shows Up in Interviews

- **“You’re seeing attention heads collapse to attending a single position after 20B tokens. What’s happening and what do you do?”** Answer skeleton: query and key norms have grown unchecked, producing logits large enough for the softmax to concentrate near one-hot; QK-norm prevents this by normalizing Q and K before the dot product, bounding the logit magnitude from step one; alternatively, soft-capping clips post-dot-product logits via a tanh; add QK-norm from the next run’s start; in the current run, monitor per-head logit statistics and consider reducing the learning rate to slow the norm growth.
- **“Compare QK-norm and soft-capping. When would you choose each?”** Answer skeleton: QK-norm is preferred - it addresses root cause, adds minimal parameters, improves perplexity, and enables higher learning rates; soft-capping is simpler (no learned parameters) and can be retrofitted into existing attention implementations without changing the projection layers, but it can worsen perplexity due to tanh distortion of logit ordering; choose QK-norm for greenfield runs; soft-capping if you need to stabilize an existing architecture with minimal code changes.
- **Senior follow-up: “Logit soft-capping’s gradient through the tanh vanishes for large logits. Doesn’t that prevent the model from learning to reduce a logit that’s already past the cap?”** A senior answer identifies this precisely: once a logit exceeds $\approx 2s$, the gradient of the loss with respect to that logit is attenuated by $(1 - \tanh^2(l/s)) \approx 0$, meaning the optimizer receives almost no signal to reduce it. The soft-cap prevents the softmax from seeing the extreme value, but the underlying logit norm can continue to grow unchecked. This is why soft-capping addresses the symptom but not the cause: in a run without any QK-norm, raw logit norms may still drift upward over the training horizon even with soft-capping in place. QK-norm avoids this by preventing norm growth at the source, maintaining well-conditioned gradients throughout.

Key Takeaways

- Unchecked growth of query and key norms produces large attention logits; when a logit dominates a row, the softmax collapses to near-one-hot and gradient flow through the attention mechanism stops.
- QK-norm normalizes Q and K via LayerNorm before the dot product, bounding logit magnitude at $\gamma\sqrt{d_k}$; it improves perplexity and allows higher learning rates.
- Soft-capping clips post-dot-product logits to $(-s, s)$ via $s \cdot \tanh(l/s)$; it is simpler to implement but can worsen quality because tanh distorts the relative contrast between large logits and kills gradients for already-large values.
- Both techniques originate from large-scale vision and multimodal training (QK-norm from ViT-22B, soft-capping from Gemma 2) and migrated into pure-text LLM training as model scale demanded greater stability.
- Apply from step one; do not stack both without ablating; monitor per-head 99th-percentile logit magnitude as an early-warning health metric.

5.2 Numerical Precision and Training Health

5.2.1 BF16, FP16, and Mixed-Precision Pitfalls

Every pre-training run on modern hardware mixes at least two floating-point formats, and the assignment of tensors to formats is not arbitrary. Interviewers at infrastructure-focused roles ask about mixed precision because getting it wrong produces either silent numerical failures that look like architecture bugs or unnecessary memory bloat that limits achievable model scale. The question surfaces at the mid-level as “walk me through your mixed-precision training setup,” and at the senior level as “why do you sometimes see NaN values propagate even in a bfloat16 run that has gradient clipping enabled.”

Three floating-point formats dominate LLM training today. Each trades dynamic range against mantissa precision:

- **FP32** (float32): 1 sign + 8 exponent + 23 mantissa = 4 bytes. Dynamic range $\approx 1.2 \times 10^{-38}$ to 3.4×10^{38} . Roughly 7 decimal digits of precision. The traditional default; now reserved for optimizer state and master weights.
- **FP16** (float16): 1 sign + 5 exponent + 10 mantissa = 2 bytes. Maximum representable value: 65504. Minimum positive normal: $\approx 6 \times 10^{-5}$. Roughly 3 decimal digits of precision. The narrow 5-bit exponent is the critical limitation for large-model training.
- **BF16** (bfloat16): 1 sign + 8 exponent + 7 mantissa = 2 bytes. Dynamic range identical to FP32 ($\approx 3.4 \times 10^{38}$). Roughly 3 decimal digits of precision. Developed by Google Brain in 2018 specifically because deep learning training needs dynamic range more than it needs mantissa resolution.

An 8-bit format, **FP8**, exists in two variants on H100 hardware: E4M3 (4 exponent bits, 3 mantissa bits, used for weights and forward-pass activations, maximum ≈ 448) and E5M2 (5 exponent bits, 2 mantissa bits, used for gradients, maximum ≈ 57344). FP8 requires per-tensor dynamic scale factors and is production-viable on H100s via NVIDIA Transformer Engine, but it is not yet the default at most organizations.

The standard production recipe is *BF16 mixed precision*:

- **Forward and backward compute** (all matrix multiplications): BF16. Tensor cores accumulate partial sums internally in FP32 and return a BF16 result.
- **Master parameters**: FP32.
- **Optimizer states** (Adam’s first moment m_t and second moment v_t): FP32.
- **Gradient accumulation** across micro-batches: FP32; per-step gradients passed to the optimizer are FP32.

This asymmetry follows from a key observation: per-layer matmuls are robust to the rounding noise introduced by 7 mantissa bits, but optimizer momentum terms that accumulate small corrections over hundreds of thousands of steps require 23 mantissa bits to avoid signal loss. Under AdamW, the per-parameter update is proportional to $m_t / \sqrt{v_t + \epsilon}$. When $m_t \approx 10^{-5}$ and $v_t \approx 10^{-4}$, the ratio is approximately 0.03. In BF16, a value of 10^{-5} rounds to zero. In FP32, it is preserved.

Use BF16 where you need throughput; keep FP32 wherever you accumulate over time.

The core argument against FP16 for large-model training is dynamic range. FP16’s maximum representable value is 65504. During training, gradient magnitudes, activation outputs, and pre-softmax logits can easily exceed this threshold. In naive attention (without FlashAttention’s safe-softmax), a pre-softmax score of 12 contributes $e^{12} \approx 162000$ to the partition function before the log-sum-exp normalization step. If that intermediate exponentiation happens in FP16, it overflows to `inf`. Once `inf` enters the compute graph, any subsequent multiplication produces NaN, which propagates forward and backward through every layer in that batch. The run is

dead.

BF16 avoids this entirely. Its 8 exponent bits match FP32’s ceiling. The precision sacrifice - 7 mantissa bits versus 23 - means values are rounded to approximately 0.7% of their true magnitude, which has no measurable effect on the final quality of a 10B-parameter model trained on a trillion tokens. The same 0.7% rounding applied to an optimizer state accumulated over 500k steps would progressively corrupt the effective learning signal, which is why optimizer states stay in FP32.

Loss scaling is the standard workaround for training in FP16: scale the loss by a constant ($\sim 2^{15}$ to 2^{20}) before the backward pass to lift gradient magnitudes into FP16’s representable range, then unscale before the optimizer step. Dynamic loss scaling adjusts this factor step by step to prevent both overflow and underflow. This works, but it adds a second failure mode: if the scale is too large, gradients still overflow; if too small, they underflow to zero silently. BF16 eliminates the need for loss scaling entirely, removing an entire class of training instability.

A Worked Example

Consider a 13B-parameter model: $d = 5120$, 40 layers, 40 attention heads, $d_{\text{head}} = 128$, trained with AdamW in BF16 mixed precision on 8 H100 80 GB GPUs.

Static memory footprint (per the 16-bytes-per-parameter rule):

- FP32 master parameters: $13 \times 10^9 \times 4 = 52$ GB
- FP32 gradients: $13 \times 10^9 \times 4 = 52$ GB
- FP32 Adam m_t : $13 \times 10^9 \times 4 = 52$ GB
- FP32 Adam v_t : $13 \times 10^9 \times 4 = 52$ GB
- BF16 compute copy of parameters: $13 \times 10^9 \times 2 = 26$ GB

Total static tensors: 234 GB. This far exceeds a single H100’s 80 GB. With ZeRO stage-3 across 8 GPUs, the static footprint per GPU drops to ≈ 29 GB, leaving ~ 50 GB per GPU for activations. The often-cited “16 bytes per parameter” accounts for the four FP32 copies (params, gradients, m_t , v_t) but not the BF16 compute copy; the true figure under BF16 mixed precision is ≈ 18 bytes per parameter.

FP16 overflow scenario: Suppose we use FP16 for compute without loss scaling. At step 1,500, a long-document batch pushes attention logits to $s_{ij} = 11.9$ before the $1/\sqrt{d_{\text{head}}}$ scaling at one head. With `grad_clip` set to 1.0 but the overflow occurring in the forward pass, no gradient norm check saves us. The raw exponent $e^{11.9} \approx 147000$ exceeds FP16’s ceiling of 65504; the result is `inf`. The attention softmax for that row returns NaN. The NaN propagates through all 40 layers in the backward pass. After the optimizer step, the affected parameters contain NaN. At step 1,501, the loss reads NaN. The run is unrecoverable without a rollback.

BF16 with the same batch: BF16’s exponent range extends to 3.4×10^{38} . The value $e^{11.9} \approx 147000$ is represented without overflow. FlashAttention’s tiled online-softmax further avoids materializing the full exponentiated row in any single buffer. Training continues uninterrupted.

What You Actually Decide in Practice

- **Default to BF16 for all compute; never use FP16 for LLM training without dynamic loss scaling.** On H100s, BF16 tensor core throughput is roughly 10× faster than FP32 and comparable to FP16, with none of FP16’s overflow risk. The choice is effectively free.
- **Optimizer states and master parameters must stay in FP32.** Switching Adam’s m_t and v_t to BF16 causes visible training quality degradation above 7B parameters, even in short 100B-token runs. The 3-

digit mantissa precision rounds away the small per-step corrections that build up meaningful parameter movement over 500k steps.

- **Use FlashAttention for numerically stable attention in BF16.** Naive attention that materializes the full $N \times N$ attention matrix in BF16 can produce catastrophic cancellation at sequence lengths above 4k. FlashAttention’s tiled algorithm maintains the running softmax max and sum in FP32 within each CUDA kernel tile, avoiding this without a format switch.
- **Consider FP8 for inference-side quantization before adopting it for training.** FP8 inference (quantizing a trained BF16 model) is low-risk and well-supported. FP8 training requires Transformer Engine, per-tensor scale factor calibration, and empirical validation against a BF16 baseline for 10k steps before committing. The throughput gain ($\sim 1.5\text{-}2\times$ over BF16) is real but the engineering overhead is substantial.
- **Monitor `activation_max` and the pre-clip gradient norm from step zero.** These two scalars are your early-warning system for format overflow. A `activation_max` that is trending toward 10^4 in BF16 (still far from overflow, but growing) combined with an elevated gradient norm signals that the model is entering a regime where even BF16’s range may be stressed; in FP8 mode, the same trend would indicate imminent scale-factor miscalibration.

How This Shows Up in Interviews

- **“Walk me through your mixed-precision training setup for a 70B model run.”** Answer skeleton: BF16 for all forward and backward matmuls; FP32 master weights, gradients, and Adam optimizer states; FlashAttention for numerically stable attention; gradient clipping at 1.0. Explain that FP16 requires dynamic loss scaling and is avoided for this exact reason. Mention the 16-18 bytes per parameter static footprint and why ZeRO stage-2 or -3 is required at 70B.
- **“Why does training with FP16 sometimes produce NaN values that BF16 does not?”** Answer skeleton: FP16 has a 5-bit exponent capping representable values at 65504. Attention logits, pre-softmax activation values, or backward-pass gradient products can exceed this threshold, overflowing to `inf`, which any subsequent multiplication converts to NaN. BF16’s 8-bit exponent matches FP32’s dynamic range and eliminates this overflow path; the loss in mantissa precision (7 bits vs. 23) does not matter for per-layer computation.
- **Senior follow-up: “Your team wants to enable FP8 training end-to-end on H100s to improve throughput. What is your rollout plan and what failure modes do you specifically instrument?”** A senior answer covers: (1) start with FP8 only in the feed-forward matmuls, leaving attention, layer norm, and softmax in BF16/FP32; (2) instrument per-tensor `amax_history` to detect scale-factor miscalibration - if the rolling maximum underestimates by $2\times$, overflow occurs silently; (3) run a parallel BF16 baseline for 10k steps at small scale to confirm loss curves match within 0.1 nats; (4) expand to full FP8 only after the subset is validated; (5) flag any step where `nan_frac` > 0 for immediate investigation - in FP8 mode a single miscalibrated tensor contaminates the entire forward pass.

Key Takeaways

- FP16 is dangerous for LLM training above 7B parameters without dynamic loss scaling; its 5-bit exponent overflows to `inf` at values above 65504, which multiplies to NaN and propagates through every downstream tensor in the batch.

- BF16 matches FP32’s dynamic range with half the memory, making it the standard compute dtype for frontier training runs; the mantissa precision loss is inconsequential for per-layer computation.
- Optimizer states and master weights must stay in FP32; the 23-bit mantissa precision is required to accumulate small per-step corrections faithfully over hundreds of thousands of optimizer steps.
- The standard static memory budget under AdamW BF16 mixed precision is approximately 18 bytes per parameter: 4 (FP32 params) + 4 (FP32 gradients) + 4 (Adam m_t) + 4 (Adam v_t) + 2 (BF16 compute copy); activations scale separately with batch size and sequence length.
- FP8 training on H100s with Transformer Engine yields roughly $1.5\text{--}2\times$ throughput gains over BF16 but requires per-tensor scale factor calibration and careful monitoring of `amax_history`; most teams adopt it only after the BF16 pipeline is stable and throughput-limited.

5.2.2 Loss Spikes: Detection and Mitigation

A loss spike that goes undetected for 3,000 steps forces a checkpoint rollback that costs several GPU-hours and invalidates the data-mix accounting for that training window. Staff-level interviewers expect a concrete playbook: not “watch the loss curve” but a named set of metrics, specific alert thresholds, and tiered responses that distinguish recoverable anomalies from structural damage. The topic surfaces as “walk me through your training health monitoring setup” or directly as “your 70B run spiked at 150B tokens and isn’t recovering - what do you do?”

A loss spike is a sudden increase of 0.1 nats or more in training loss that persists for more than 50 consecutive steps. Three root causes account for nearly all production instances, and they are distinguishable by which diagnostic metric moves first.

1. **Data batch anomaly.** A batch containing corrupted tokens, binary-encoded text, heavily duplicated content, or extreme-length documents with out-of-distribution statistics produces a locally very large loss gradient. The gradient is large but non-destructive: the model absorbs the update and the loss recovers within 100-500 steps as subsequent normal batches dilute the optimizer state.
2. **Gradient explosion or logit drift.** The global gradient norm persistently exceeds the clip threshold (or the clip threshold is set too loosely), causing large parameter updates that overshoot the loss basin over multiple steps. Alternatively, the softmax logit scale drifts upward as described in the z-loss subsection, gradually destabilizing the gradient signal until a threshold is crossed. Both sub-causes share the signature of a non-recovering spike with no single obvious culprit batch.
3. **Numerical precision failure.** FP16 or FP8 overflow introduces `inf` or `NaN` values into the compute graph. The loss may jump to extreme values (tens of nats or literal `NaN`) and does not recover without a rollback and configuration change.

The practical monitoring set is five scalars logged every training step at negligible overhead:

- `loss`: the primary signal, evaluated on the current batch.
- `grad_norm`: the global ℓ_2 norm of all gradients *before* clipping. A clipped norm tells you nothing about whether an explosion occurred; the pre-clip norm does.
- `log_Z`: the mean log-partition-function across the batch, already computed during the forward pass as part of the cross-entropy loss. The leading indicator for logit drift.
- `loss_ema_ratio`: current loss divided by an exponential moving average of the previous 1,000 steps. A ratio above 1.5 triggers a Tier 1 alert; above 2.5 triggers Tier 2.
- `nan_frac`: fraction of gradient elements that are `NaN` or `Inf`. Any nonzero value triggers an immediate

halt.

Root cause determines the correct response; treating all spikes as identical wastes GPU hours on rollbacks that are unnecessary and misses interventions that are urgent.

The value of the triage protocol is specificity. Three response tiers map onto the three root causes:

Tier 1 - skip and continue. Applied when `grad_norm` spikes to more than $3\times$ its running median in a single step, `loss_ema_ratio` is above 1.5, and the loss begins returning toward the baseline within 200 steps. This pattern is consistent with a data anomaly. The correct action is to log the shard index of the offending batch, optionally skip the optimizer update for that step (retain the gradient for accounting purposes), and continue without rollback. Introducing a full rollback for a self-correcting anomaly destroys training throughput with no benefit.

Tier 2 - rollback to checkpoint. Applied when the loss rises for more than 500 steps without returning below the pre-spike level, or when `log_Z` has increased by more than 3 units in a 300-step window. This pattern indicates structural damage to the model or optimizer state. Roll back to the checkpoint immediately preceding the onset of the gradient norm elevation, remove the offending data range from the training sequence, and resume. If `log_Z` was the leading signal, add or increase the z-loss coefficient before resuming.

Tier 3 - rollback and reconfigure. Applied when `nan_frac` exceeds zero, or the loss jumps to values above 10 nats without gradient norm explanation. This indicates a numerical precision failure. Roll back and identify the source tensor (activation monitoring or a per-layer `nan_frac` breakdown will localize it), then adjust the precision configuration - switching a problematic layer back from FP8 to BF16, or from BF16 to FP32 - before resuming.

The prior approach of relying on gradient clipping alone addresses only Tier 1 partially. Gradient clipping caps the norm of each update but does not prevent logit scale drift, does not flag data quality issues upstream of the gradient computation, and does not detect NaN propagation that originates in the forward pass.

A Worked Example

A 13B-parameter model: $d = 5120$, 40 layers, BF16 compute, AdamW, global batch 2M tokens per step, base learning rate 3×10^{-4} , cosine decay, `grad_clip` = 1.0. The run is at step 60,000 when two anomalies occur.

Scenario 1 - data anomaly at step 42,100:

- `loss`: $2.41 \rightarrow 2.89$ (single step, $+0.48$ nats)
- Pre-clip `grad_norm` at step 42,100: 7.3 (running median over prior 5,000 steps: 1.1; ratio = $6.6\times$)
- `loss` over next 400 steps: $2.89 \rightarrow 2.72 \rightarrow 2.58 \rightarrow 2.44$ (recovering)
- `log_Z`: stable at 8.3 ± 0.2 before and after the spike
- `nan_frac`: 0 throughout

Diagnosis: data anomaly (Tier 1). The gradient norm is $6.6\times$ the median at a single step; the loss is recovering. Action: log the batch shard index for data quality inspection, skip the optimizer update at step 42,100, continue training. No rollback. Estimated cost: zero additional GPU time; one flagged shard for offline investigation.

Scenario 2 - logit drift at step 58,500:

- `log_Z` trend: 8.1 at step 57,000, 9.6 at step 57,500, 11.9 at step 58,000, 15.2 at step 58,500 (rising for 1,500 steps)
- `loss` at steps 58,200 onward: $2.38 \rightarrow 2.65 \rightarrow 2.99 \rightarrow 3.41 \rightarrow 3.88$ (diverging)

- `grad_norm`: elevated but not extreme, 1.8-2.6 (only $1.7\times$ median, not a single-step spike)
- `nan_frac`: 0

Diagnosis: logit scale drift (Tier 2). The run was not configured with z-loss and the softmax normalizer has grown to $e^{15.2} \approx 4 \times 10^6$. At this scale, BF16's 7-bit mantissa rounds most token probabilities to zero, collapsing the gradient signal for the majority of the vocabulary. Action: roll back to the checkpoint at step 57,000 (the last checkpoint before the `log_Z` acceleration began), add z-loss with $\alpha = 10^{-4}$, resume. The `log_Z` stabilizes below 10 within 3,000 steps; the loss returns to the pre-spike trajectory by step 5,000 after resumption.

What You Actually Decide in Practice

- **Save checkpoints every 1,000 steps; retain the last 10.** On a 512-GPU cluster training a 70B model with a 4M-token global batch, 1,000 steps represents roughly 20-30 minutes of wall time. Retaining 10 checkpoints provides a three-to-five-hour rollback window for diagnosing slowly developing issues at the cost of approximately 400 GB of checkpoint storage per saved snapshot.
- **Set `grad_clip` = 1.0 for 7B-70B models and log the pre-clip norm every step.** The clipped norm is uninformative. The pre-clip norm distribution tells you when something is structurally wrong: a median around 0.8-1.2 with an occasional $5\times$ outlier is normal; a median that steadily rises from 1.0 to 3.0 over 10,000 steps is an early signal of logit drift or an overheated learning rate.
- **Log `log_Z` from step zero as a first-class health metric.** As detailed in the z-loss subsection, a monotonically increasing `log_Z` is the clearest multi-step leading indicator of a future divergent spike. If `log_Z` has exceeded 12 and has been rising for 5,000 steps, add or increase the z-loss coefficient immediately; do not wait for the spike.
- **Implement skip-batch logic for Tier 1 events.** If `loss_ema_ratio` exceeds 2.0 at any step, skip the optimizer update for that step. Still run the forward and backward passes to compute the gradient norm for diagnostic purposes, then discard the gradient before `optimizer.step()`. This prevents a single catastrophic batch from damaging the Adam momentum estimates while the data shard investigation proceeds.
- **Stop the run immediately on `nan_frac` > 0 and do not attempt to continue.** A single NaN in the gradient contaminates every parameter touched by that gradient through subsequent momentum accumulation. The affected optimizer state cannot be repaired; only a rollback to a clean checkpoint resolves it. Continuing through NaN values corrupts the master weight copy in a way that may not be visible in the loss for hundreds of steps.

How This Shows Up in Interviews

- **“Your 70B pre-training run develops a loss spike at 150B tokens that has not recovered after 3,000 steps. How do you diagnose and fix it?”** Answer skeleton: first check whether `grad_norm` showed a single-step extreme outlier before the spike (data anomaly) or a gradual elevation (logit drift or LR issue). Check whether `log_Z` has been rising for thousands of steps before the visible spike - if so, the root cause is logit scale instability. Roll back to the last checkpoint before the `log_Z` acceleration, add z-loss with $\alpha = 10^{-4}$ if not already present, and resume. If neither the gradient norm nor `log_Z` explains the spike, inspect the data shard window for corrupted documents or a pipeline error that changed the data distribution.

- **“What training health metrics do you log beyond the loss curve?”** Answer skeleton: the five-metric set - `loss`, `pre-clip grad_norm`, `log_Z`, `loss_ema_ratio`, and `nan_frac`. Explain what each indicates and at what threshold it triggers an action. Mention that per-layer gradient norm histograms (sampled every 100 steps rather than every step to limit overhead) are added when a run is exhibiting unexplained instability and you need to localize which layer is the source.
- **Senior follow-up: “Design an automated spike detection and rollback system for a production pre-training run. What would you automate versus leave to human judgment?”** A senior answer covers: automate Tier 1 skip-batch on `loss_ema_ratio > 2.0`; automate immediate halt on `nan_frac > 0`; automate paging the on-call engineer for Tier 2 events with a pre-populated diagnostic summary (`log_Z` trend over the prior 5,000 steps, gradient norm percentile history, data shard identifier for the steps around the spike onset). Explain why fully automated Tier 2 rollback is risky: the rollback destination requires understanding whether the spike is a data anomaly that already passed (in which case you roll back past it) or a configuration deficiency that persists (in which case rolling back without configuration change is pointless). The human judgment step is cheap; the cost of a wrong automated rollback is not.

Key Takeaways

- Loss spikes fall into three tiers by root cause - data anomaly (self-recovers within 500 steps, skip-batch response), logit drift or gradient explosion (requires rollback to last stable checkpoint), and numerical precision failure (requires rollback and configuration change) - and each demands a different response.
- The pre-clip gradient norm, `log_Z`, and `nan_frac` are the three non-loss metrics that distinguish these tiers; log all five health metrics at every training step.
- Save checkpoints every 1,000 steps and retain at least 10; this bounds the rollback cost at one to three hours of compute while providing a multi-hour diagnostic window.
- A `log_Z` that is monotonically rising and has exceeded 12 is a reliable multi-thousand-step leading indicator of a divergent spike; the correct response is to add z-loss before the spike occurs, not to roll back after it does.
- Fully automated skip-batch (Tier 1) is safe and worth implementing; automated Tier 2 rollbacks require human sign-off because the correct rollback destination depends on diagnosing the root cause, not just detecting that a spike occurred.

Part III

Mixture of Experts and Sparse Architectures

Chapter 6 Why MoE Is Now Standard in Frontier Systems

6.1 Dense vs. Sparse Models for Fixed FLOPs

6.1.1 Parameters for Free: The Core MoE Argument

When an interviewer asks why you would choose a mixture-of-experts architecture over a dense model, the answer is not simply “more parameters.” The precise answer is more parameters at the same FLOPs budget. This single idea is what has convinced every major frontier lab to adopt MoE, and it is tested at every seniority level once you are interviewing for roles that touch training infrastructure or model architecture. At the senior level you need to be able to quantify it: explain why perplexity curves diverge at identical training compute and what that implies for how you spend a fixed GPU budget.

A standard dense transformer sends every token through the same feedforward network at every layer. A sparse MoE transformer replaces each FFN block with N expert FFNs but activates only K of them per token. If each expert is the same size as the original FFN and $K = 1$, the forward-pass arithmetic is identical to the dense model—the same number of multiply-accumulate operations—but the model now stores N times as many FFN parameters in memory.

The MoE layer output for token t with residual-stream input $\mathbf{u}$ is:

$$\text{MoE}(\mathbf{u}) = \sum_{i=1}^N g_i(\mathbf{u}) \cdot \text{FFN}_i(\mathbf{u}) + \mathbf{u}$$

where $g_i(\mathbf{u}) = 0$ for all but the top- K experts by router score. The nonzero gates are derived from a softmax over the selected indices and combine the chosen experts’ outputs as a weighted average. When $K = 1$ this collapses to a single FFN evaluation with the residual added back: identical FLOPs to the dense case, $N \times$ the stored parameters.

The empirical case is now settled. Switch Transformer showed that at a fixed training FLOP budget, increasing the expert count from 1 to 256 monotonically reduced language-modeling loss. OIMoE replicated the result on a modern architecture: the MoE training curve decays faster and reaches a better floor than a compute-matched dense baseline. DeepSeek’s early MoE ablations compared dense, naive-MoE, and improved-MoE variants across multiple benchmarks and found consistent improvement as routing quality rose. The same pattern reappears in the DeepSeek-V2 serving results: remarkably low error rates at activated-parameter counts far below what a comparable dense model would require.

At a fixed compute budget, sparse MoE models consistently outperform dense models because they pack more parameters into the same FLOPs, and language models use those extra parameters productively.

Language modeling has two separable demands: computing good representations of context (which requires FLOPs) and memorizing factual associations, vocabulary patterns, and domain idioms (which requires parameter capacity). Dense architectures must serve both demands with the same parameter budget. MoE decouples them: the activated experts handle computation while the dormant ones hold capacity the model can draw on when the right inputs arrive.

This is not an abstract argument. It is the reason that hashing-based routing—a fixed deterministic function with no semantic content—still provides performance gains over a single FFN at the same activated FLOPs. The capacity benefit is real and does not require perfectly learned routing to appear.

The costs are also real. More total parameters means more memory: even if your activated FLOPs are

modest, you must store the full weight tensor for every expert somewhere in your serving cluster. Routing introduces systems complexity-token dispatching, all-to-all communication in expert-parallel configurations, and load-balancing machinery to prevent expert collapse. Dense models are simpler to checkpoint, replicate, and fine-tune. But for any setting where training compute is the binding constraint and memory plus engineering overhead are tractable, the FLOPs-per-loss advantage of MoE is now essentially settled empirical fact.

A Worked Example

Consider a 7B-parameter dense baseline with $d_{\text{model}} = 4096$, FFN hidden dimension 11008 (SwiGLU, $\approx 2.7 \times d_{\text{model}}$), and 32 layers. Each FFN block has roughly $3 \times 4096 \times 11008 \approx 135\text{M}$ parameters (three projections in the gated network), giving about 4.3B FFN parameters across the full model.

Now design a FLOP-matched MoE: 8 experts per layer, each expert at one-quarter the hidden dimension (2752), top-2 routing:

- Parameters per expert: $3 \times 4096 \times 2752 \approx 34\text{M}$
- Total expert parameters per layer: $8 \times 34\text{M} = 272\text{M}$
- Total expert parameters across 32 layers: $32 \times 272\text{M} \approx 8.7\text{B}$ ($2\times$ the dense FFN budget)
- Activated per token per layer: $2 \times 34\text{M} = 68\text{M}$, roughly half the dense FFN per layer

For an exact FLOPs match, use top-1 routing with full-size experts: 8 experts each at 135M parameters, activating 1. The activated compute is identical to the dense baseline; the stored FFN parameters jump from 135M to $8 \times 135\text{M} = 1.08\text{B}$ per layer.

Training both variants for 1T tokens at identical compute budget: empirical results typically show a 0.1-0.2 nats/token improvement in perplexity for the MoE, translating to meaningfully better downstream benchmark scores. The memory cost for the 8-expert MoE is approximately $8\times$ the dense FFN footprint-that is the price paid for those gains.

What You Actually Decide in Practice

- **Use MoE when training compute is the bottleneck, not memory.** If you are already memory-limited at inference and cannot shard experts across devices, dormant parameters become dead weight that yields no performance return.
- **Fine-grained experts beat fewer large ones.** Ablations in both DeepSeek MoE and OlMoE consistently show that 64 quarter-size experts outperform 16 full-size experts at the same activated-FLOPs budget. Default to fine-grained experts when you have the routing infrastructure to support them.
- **$K = 2$ is the canonical starting point.** Top-1 routing is maximally compute-efficient but gives the router no gradient signal to compare alternatives. Top-2 provides that comparison at the cost of doubling per-token FFN FLOPs relative to top-1. Most production systems (Mixtral, DeepSeek, Grok) use $K = 2$ or a fine-grained equivalent.
- **Expert specialization is less semantic than the name implies.** Hashing-based routing provides real gains without any learned affinity between tokens and experts. If a reviewer tells you MoE works because experts specialize by domain, that framing is not wrong but it is incomplete: the parameter-capacity benefit alone explains the FLOPs-matched quality gap.
- **Every-layer MoE is now standard at frontier scale.** Earlier designs alternated dense and MoE layers to reduce fine-tuning instability. Modern production systems that absorb the engineering overhead apply MoE at every FFN layer.

How This Shows Up in Interviews

- “Why does a sparse MoE outperform a dense model trained on the same compute?” Strong answer: the MoE decouples parameter capacity from compute. At equal FLOPs, the activated experts do the same arithmetic as the dense FFN, but the model holds additional capacity in dormant experts that it exploits for memorization. Empirically, that capacity is productive across diverse tasks.
- “What engineering costs do you accept when switching from dense to MoE?” Strong answer: three categories. Memory: total parameter count grows by up to $N \times$, requiring expert-parallel sharding. Routing: non-differentiable top- K selection requires auxiliary load-balancing losses to prevent collapse. Stability: MoE models are more prone to loss spikes and fine-tuning divergence; z-loss on router logits and float32 router computation are standard mitigations.
- Senior follow-up: “How does the MoE FLOPs advantage interact with compute-optimal scaling?” Compute-optimal reasoning applies to activated parameters and training FLOPs. Dormant MoE parameters are essentially free from a training-compute standpoint—you are not paying for them in multiply-accumulate operations. This means a MoE can simultaneously be compute-optimal in its activated parameter count and parameter-rich in its total count. The catch is that serving cost scales with total parameters, not activated ones, so the efficiency argument flips at inference time.

Key Takeaways

- At identical training FLOPs, MoE models consistently achieve lower perplexity than dense models—this has been replicated across architectures, scales, and labs.
- The mechanism is decoupling: activated parameters set compute cost; total parameters set capacity. MoE scales capacity without scaling compute.
- Fine-grained experts (many small ones over few large ones) and $K = 2$ routing are the dominant production defaults.
- Primary costs are aggregate memory ($N \times$ the dense FFN footprint), routing infrastructure, and load-balancing complexity to prevent expert collapse.
- Expert specialization is real but secondary; the parameter-capacity argument alone explains the FLOPs-matched quality gain.

6.1.2 Activated vs. Total Parameters

When a candidate describes a mixture-of-experts model as a “70B model” without qualifying which 70B they mean, they have already lost the thread of the economics. Understanding the difference between activated and total parameters is the single most important framing skill for reasoning about MoE systems: it determines training compute, serving memory, hardware topology, and how to interpret the benchmark plots that labs publish. Interviewers test this at every seniority level; senior engineers must derive concrete infrastructure numbers from both quantities on the spot.

In a dense transformer every parameter participates in every forward pass. The model has P_{total} parameters and uses all of them to process each token, so $P_{\text{activated}} = P_{\text{total}}$.

A sparse MoE transformer breaks this coupling. For each token, the router activates only K of the N available experts per MoE layer. The rest are stored in memory but contribute zero FLOPs during that token’s forward pass. The two quantities are:

- **Total parameters:** all weights in the model, including every expert that will never fire for a given input. This determines the memory required to host the model.
- **Activated parameters:** the subset of weights that execute per token per forward pass. This determines per-token FLOPs and, by extension, training compute and inference latency.

For a model with L MoE layers, N experts per layer, each expert containing P_e parameters, and K experts activated per token, the two quantities are:

$$P_{\text{total}} = P_{\text{non-FFN}} + L \cdot N \cdot P_e$$

$$P_{\text{activated}} = P_{\text{non-FFN}} + L \cdot K \cdot P_e$$

The ratio N/K is the *sparsity factor*: it describes how many times larger the total FFN parameter count is relative to the activated FFN parameter count. A model with 64 experts and top-4 routing has a sparsity factor of 16. Training compute scales with $P_{\text{activated}}$; hosting cost scales with P_{total} .

Activated parameters set the compute price; total parameters set the memory price-and for MoE these two prices are decoupled by the sparsity factor.

For training, the activated-parameter count is all that matters for FLOPs. Compute-optimal reasoning following Chinchilla-style analysis should tune token count and activated model capacity together-dormant expert weights cost nothing in training compute. A 70B-total MoE model with 7B activated parameters trains at the FLOPs cost of a 7B dense model. If the resulting quality matches a 13B or 20B dense model-which empirical results from Switch Transformer, OLMoE, and the DeepSeek MoE line suggest it can-the MoE is simply a better use of a fixed training budget.

For serving, total-parameter count dominates. Every weight tensor, including experts that fire rarely or never for a particular query, must live somewhere in memory at inference time. The router’s decision is data-dependent; you cannot know in advance which expert will be needed for each token, so all experts must be resident. A 70B-total MoE costs the same inference-time memory as a 70B dense model, even if only 7B parameters contribute arithmetic per token. Latency per token is low (7B worth of FLOPs), but the weight memory bill is high.

This asymmetry explains why labs publish benchmark results with “activated parameters” on the x-axis. On that axis MoE looks spectacular: DeepSeek-V2’s MMLU results plotted against activated parameters sit far above comparably activated dense models at the same compute. That framing is not misleading-it accurately captures compute efficiency-but it does not tell the hosting story. An interviewer who asks “what is the catch?” is probing exactly this gap.

A Worked Example

Take a 7B dense baseline:

- $d_{\text{model}} = 4096$, SwiGLU FFN with $d_{\text{ff}} = 11008$
- FFN parameters per layer: $3 \times 4096 \times 11008 \approx 135\text{M}$
- 32 transformer layers; FFN total: $32 \times 135\text{M} \approx 4.3\text{B}$
- Non-FFN (attention, embeddings, norms): $\approx 2.7\text{B}$
- Total $\approx 7\text{B}$, activated $\approx 7\text{B}$

Now design a FLOP-matched MoE that targets the same per-token compute:

- 64 experts per layer, each at one-quarter the FFN hidden dimension ($d_{\text{ff}}^e = 2752$)

- Parameters per expert: $3 \times 4096 \times 2752 \approx 34\text{M}$
- Top-4 routing; activated FFN per layer: $4 \times 34\text{M} = 136\text{M}$ (matches dense)
- Total FFN per layer: $64 \times 34\text{M} = 2.18\text{B}$
- Total FFN across 32 layers: $32 \times 2.18\text{B} \approx 70\text{B}$
- Total model: $70\text{B} + 2.7\text{B} \approx 73\text{B}$; activated: $\approx 7\text{B}$

Training compute for a 1T-token run (using the $\approx 6ND$ approximation with $N = P_{\text{activated}}$):

$$\text{FLOPs} \approx 6 \times 7 \times 10^9 \times 10^{12} = 4.2 \times 10^{22}$$

This is identical for the dense and MoE variants by construction.

Serving memory in BF16:

- Dense: $7\text{B} \times 2 \text{ bytes} = 14 \text{ GB}$ -fits on a single A100-40GB.
- MoE: $73\text{B} \times 2 \text{ bytes} \approx 146 \text{ GB}$ -requires at least two H100-80GB GPUs for weights alone, with expert-parallel sharding across devices.

The sparsity factor is $N/K = 64/4 = 16$. The MoE stores $16\times$ more FFN parameters than the FLOP-matched dense model. You pay for all 73B weights in memory but compute with only 7B of them per token.

What You Actually Decide in Practice

- **Use activated parameters for training budget planning.** When computing your compute-optimal token-to-parameter ratio, plug in activated parameters. Sizing by total parameters overstates your training FLOPs and produces a model that is undertrained relative to its capacity.
- **Use total parameters for serving infrastructure planning.** When provisioning GPUs for inference, total parameters set the memory floor. Expert parallelism is mandatory once total parameters exceed a single node's HBM capacity (typically 640 GB for an 8-way H100 NVLink node at BF16 weights before activations and KV cache).
- **High sparsity factors amplify both the benefit and the hosting cost.** A sparsity factor of 32-as in DeepSeek-V3's 256 routed experts with top-8 routing-gives $32\times$ more FFN parameters than a FLOP-matched dense baseline. The quality gain is real; the memory bill is proportionally real.
- **Know which inference regime you are in.** In memory-bound inference (small batches, long context), latency is dominated by weight loading, not arithmetic. A dense 7B model can outperform a 73B-total / 7B-activated MoE on latency in this regime because it loads far fewer bytes from HBM per token. In compute-bound inference (large batches), activated parameters dominate and the MoE and dense models have comparable throughput-the MoE wins on quality at that compute level.
- **Benchmark comparisons require the right denominator.** Before accepting a "quality vs. activated parameters" claim, verify what the total-parameter hosting cost is. A fair head-to-head for practitioners is either equal activated parameters (same training cost) or equal total parameters (same serving cost). Equal total parameters is the test the MoE must also pass at inference time.

How This Shows Up in Interviews

- "A lab claims their MoE model has 70B parameters and matches a 13B dense model in quality. What would you ask before accepting that comparison?" Strong answer: first, clarify whether 70B is total or activated. If it is 70B total with 7B activated, the training compute comparison is coherent-it trained at 7B FLOPs cost. But 70B total means roughly $10\times$ the memory of a 7B dense model, which changes the

serving calculus entirely. Second, ask whether the quality comparison is FLOP-controlled (same training compute) or size-controlled (same total parameters): beating a 13B dense at equal training FLOPs is expected; beating a 70B dense at equal total parameters would be a much stronger result.

- “How do you provision inference hardware for a 70B-total / 7B-activated MoE?” Strong answer: start with total parameters to set the memory floor. At BF16 that is ≈ 140 GB, requiring at minimum two H100-80GB GPUs with expert-parallel sharding. Latency per token is set by activated-parameter FLOPs (7B worth), not total. Batch throughput is limited by the all-to-all communication dispatch added by expert routing, so benchmark throughput degradation against a purely dense serving stack before committing.
- Senior follow-up: “When does a dense model win at inference even if the MoE wins at training?” The senior answer requires distinguishing inference regimes. In memory-bandwidth-bound serving (single-user or small-batch), each token forces the hardware to stream weight bytes from HBM regardless of whether those bytes are used for arithmetic. A 73B-total MoE streams $10\times$ more bytes per token than a 7B dense model even though the arithmetic is identical. Time-to-first-token and per-token latency both suffer in this regime. Only at large batch sizes, where the hardware is arithmetic-bound, do the two models converge on latency-at that point the MoE wins because its quality is higher at the same activated-parameter count.

Key Takeaways

- Activated parameters set per-token FLOPs and training compute; total parameters set memory footprint and hosting cost-MoE decouples these two quantities by the sparsity factor N/K .
- Training budget reasoning must use activated parameters; infrastructure provisioning must use total parameters. Mixing these up leads to hardware surprises in either direction.
- The sparsity factor quantifies the memory-to-compute leverage of a MoE design: a factor of 16 means $16\times$ more FFN parameters in storage than in any single token’s forward pass.
- Benchmark plots that place “activated parameters” on the x-axis are accurate for compute efficiency comparisons but obscure the full memory cost of hosting all dormant expert weights.
- In memory-bandwidth-bound inference regimes, total parameter count governs latency; in compute-bound regimes, activated parameter count governs throughput-know which regime applies before choosing MoE over dense.

6.2 Routing: The Heart of Every MoE

6.2.1 Token-Choice Top-K Routing

Every MoE interview question eventually lands on the routing function. An interviewer at a frontier lab will ask you to describe the forward pass of a MoE layer, and the answer lives in two lines of math: a learned inner product and a top- K gate. This question appears at the ML engineer and research engineer level at companies running production MoE systems, and a strong answer requires knowing the mechanics cold, the design rationale, and where the gradient actually flows. Getting any of these wrong signals that you have read about MoE without having thought through the details.

The router is a lightweight learned function that maps each token’s hidden state to exactly K experts. The forward pass for a single MoE layer proceeds in four steps.

First, compute affinity scores between the incoming token hidden state $u_t \in \mathbb{R}^d$ and N learned router embeddings $\{e_i\}_{i=1}^N$, where $e_i \in \mathbb{R}^d$:

$$s_{i,t} = \text{Softmax}_i(u_t^\top e_i), \quad i = 1, \dots, N.$$

The softmax normalizes the N raw dot-products into a probability-like vector. Its role here is not to produce a probability distribution in the probabilistic sense; it is purely a normalization so that the downstream weighted sum behaves well.

Second, select the top- K experts by score:

$$\mathcal{T}_t = \text{TopK}(\{s_{i,t}\}_{i=1}^N, K).$$

Third, form the sparse gate: $g_{i,t} = s_{i,t}$ if $i \in \mathcal{T}_t$, else $g_{i,t} = 0$.

Fourth, compute the layer output with a residual connection:

$$h_t = u_t + \sum_{i \in \mathcal{T}_t} g_{i,t} \cdot \text{FFN}_i(u_t).$$

The router itself contains $N \times d$ parameters - one embedding vector per expert. For $N = 64$ experts and $d = 4096$, that is roughly 256K parameters, negligible compared to the expert FFN weights being routed into. DeepSeek-V3 replaces the pre-top- K softmax with per-expert sigmoid scores and renormalizes after top- K selection, which avoids the winner-takes-all compression that softmax applies before the discrete cut; the downstream layer norms absorb any resulting scale shift.

The entire routing decision for one token - an $N \times d$ matrix-vector product followed by a sort - costs fewer FLOPs than a single attention query.

Token-choice top- K routing directly answers the question the model cares about: given this token's representation, which experts process it best? The affinity score $u_t^\top e_i$ measures alignment between the token and the expert's learned specialization direction. High-affinity experts get the token; all others are skipped. This lets different experts develop different processing competencies over training without any engineered inductive bias - the specialization emerges from the gradient signal.

The top- K gate is also essential for training efficiency. If you removed it and instead used a soft gate weighted by the full softmax (all N experts, all the time), you would pay N expert forward passes per token. For $N = 64$ that is 64 times the compute of a dense model. Top- K ensures only K expert forward passes run per token: for $K = 6$ out of 64 experts, the FFN component costs roughly $6/64 \approx 9\%$ of the all-expert baseline. This is what makes MoE parameter-efficient at fixed FLOP budget.

The alternative - expert-choice routing, where each expert picks its favorite tokens instead of tokens picking their favorite experts - achieves perfect expert utilization but breaks down at inference time when you process a single token. Token-choice has no such constraint: each token independently selects K experts regardless of batch size, which is exactly what you need for autoregressive generation.

A Worked Example

Consider a DeepSeek-MoE-style layer with $N = 64$ routed experts, $K = 6$, and $d_{\text{model}} = 4096$. Each expert has intermediate dimension $d_{\text{ffn}}^{\text{expert}} = 1024$ (one-quarter the size of a standard dense FFN with $d_{\text{ffn}} = 4096$).

Token t arrives with hidden state $u_t \in \mathbb{R}^{4096}$.

Step 1 - Affinity scores. Compute $z_i = u_t^\top e_i$ for all 64 experts. This is a (64×4096) matrix times a $(4096,)$ vector: $64 \times 4096 \times 2 \approx 524,000$ multiply-adds. Apply softmax across the 64 values.

Step 2 - Top-6 selection. Partial-sort the 64 scores; retain indices $\mathcal{T}_t = \{i_1, \dots, i_6\}$.

Step 3 - Expert forward passes. Each expert is a two-layer MLP with weight matrices $W_{\text{up}} \in \mathbb{R}^{1024 \times 4096}$ and $W_{\text{down}} \in \mathbb{R}^{4096 \times 1024}$. One expert costs $2 \times 4096 \times 1024 \times 2 \approx 16.8\text{M}$ FLOPs. Six experts cost $\approx 100\text{M}$ FLOPs.

Step 4 - Weighted aggregation.

$$h_t = u_t + \sum_{i \in \mathcal{T}_t} s_{i,t} \cdot \text{FFN}_i(u_t).$$

Compare to a dense equivalent: one FFN with $d_{\text{ffn}} = 4096$ costs $2 \times 4096 \times 4096 \times 2 \approx 67\text{M}$ FLOPs. Our six fine-grained experts cost $\sim 100\text{M}$ FLOPs - about $1.5\times$ the dense cost per token. In exchange, the full model carries $64 \times 2 \times 4096 \times 1024 \approx 536\text{M}$ FFN parameters, versus the dense model's $2 \times 4096 \times 4096 \approx 33.6\text{M}$. That is a $16\times$ ratio of parameters to FLOP overhead in the FFN component alone.

What You Actually Decide in Practice

- **Co-design K with expert granularity.** Use $K = 2$ when each expert is full-sized (dense FFN equivalent), as in Mixtral and DBRX. Increase K proportionally when you halve or quarter expert size. If each expert is $1/m$ the dense FFN size, activating $m/2$ experts keeps the per-token FLOP cost near parity with the dense model while maximizing the parameter-to-FLOP ratio.
- **Compute router logits in float32.** Even in bfloat16 training runs, compute the affinity scores and the softmax in float32. The softmax exponential can underflow rare-expert logits to zero in bfloat16, causing those experts to never receive tokens and never recover - effectively zeroing out parameters. The float32 cast is one line of code and costs nothing at scale.
- **Do not invest in a heavier router.** There is a tempting idea that a two-layer MLP router would route more intelligently than a linear inner product. In practice the gradient signal to the router is weak - you only observe gradients through the K selected experts - so a more complex router does not learn a meaningfully better routing policy. The systems cost of additional router FLOPs is real; the quality benefit is not.
- **Decide on renormalization after top-K.** Renormalizing the selected gate weights to sum to 1 makes the output a proper convex combination and is slightly more interpretable. Not renormalizing is also fine; the next layer norm absorbs the scale. Some architectures (DeepSeek-V3 with sigmoid gates) renormalize; others (early DeepSeek MoE with softmax gates) do not. Pick one and be consistent.
- **Sigmoid at the gate vs. softmax before top-K.** Softmax before top-K creates competitive suppression: a single dominant expert can concentrate most of the probability mass before the discrete cut even happens, starving the other selected experts of meaningful gate weight. Sigmoid scores are independent per expert, so the top-K selection is less distorted by outlier logits. DeepSeek-V3's switch to sigmoid with post-selection renormalization reflects this reasoning.

How This Shows Up in Interviews

- **“Walk me through the forward pass of a MoE layer.”** A strong answer describes the four steps (affinity scoring, top- K selection, expert forward passes, weighted aggregation with residual) and gives the key equations. Bonus: mention the FLOP count breakdown and why routing cost is negligible.
- **“Why not use a soft gate over all experts instead of top-K?”** Activating all N experts on every token multiplies the FFN FLOP cost by N . Top- K is the mechanism that preserves the FLOP budget of the dense baseline while expanding the parameter count. Without it, you have a dense model with N FFNs and no efficiency advantage.

- **Senior follow-up: “How does the router learn, given that top-K is not differentiable?”** The short answer: gradients flow only through the K selected experts. The gate weight $s_{i,t}$ is differentiable with respect to the router embeddings e_i , so the model does learn to rank experts correctly on routed tokens. But experts that are never selected receive no gradient signal and never improve. This is precisely why auxiliary load-balancing losses are not optional - without them, one or two high-scoring experts absorb all traffic, the remaining experts atrophy, and you effectively have a one-expert model with a lot of wasted parameters.

Key Takeaways

- The router is a $(N \times d)$ learned embedding matrix plus an inner product, softmax (or sigmoid), and top- K gate. Total routing cost per token is negligible relative to the expert FFN forward passes.
- Token-choice top- K is the industry-standard routing mechanism in virtually every major production MoE: Mixtral, all DeepSeek variants, Grok, Llama 4, Qwen MoE, OlMoE.
- K and expert size are co-designed: fine-grained (quarter-sized) experts require higher K to maintain FLOP parity with a dense baseline; coarse-grained (full-sized) experts use $K = 2$.
- Always compute router logits in float32, even in bfloat16 training runs, to prevent rare-expert underflow.
- Top- K is non-differentiable; the auxiliary load-balancing loss is what prevents router collapse and keeps all experts active throughout training.

6.2.2 Expert-Choice and Global-Assignment Alternatives

Before token-choice top- K became the consensus, the field seriously explored letting experts pick their own tokens and solving global assignment problems to route optimally. Understanding why those approaches were abandoned matters in interviews: the trade-offs reveal deep constraints about MoE training, and an experienced interviewer will sometimes propose one of these alternatives to test whether you know why the field walked away from them. A mid-level candidate says “token-choice is standard.” A senior candidate explains exactly why the alternatives fail.

Expert-choice routing inverts the selection direction. Instead of each token picking its K favorite experts, each expert picks its K_e favorite tokens. Concretely, for a batch of T tokens and N experts, compute the full $N \times T$ affinity matrix S where $S_{i,t} = u_t^\top e_i$. Expert i then selects:

$$\mathcal{T}_i = \text{TopK}_t(S_{i,:}, K_e),$$

the top- K_e tokens by affinity in row i of S . Setting $K_e = T/N$ yields perfect load balance by construction: every expert processes exactly the same number of tokens in every batch.

Global-assignment routing casts routing as a constrained optimization problem. Given the affinity matrix S , find an assignment $A \in \{0, 1\}^{N \times T}$ that maximizes total affinity:

$$\max_A \sum_{i,t} A_{i,t} \cdot S_{i,t} \quad \text{s.t.} \quad \sum_i A_{i,t} \geq 1 \quad \forall t, \quad \sum_t A_{i,t} \leq C \quad \forall i,$$

where C is an expert capacity constraint. Approximate solvers include the Sinkhorn algorithm (iterated row-and-column normalization, $O(N \cdot T \cdot \text{iter})$ per layer per step) and linear-sum assignment via the Hungarian algorithm ($O((N + T)^3)$ in the worst case). The method S-BASE (Clark et al., 2020) demonstrated this approach with measurably better token-expert affinity coverage than greedy top- K .

Noisy top- K (stochastic routing) is a third alternative: add learned Gaussian noise to router logits before

the top- K selection. Shazeer (2017) parameterized this as:

$$\tilde{z}_i = u^\top W_g e_i + \epsilon_i \cdot \text{Softplus}(u^\top W_{\text{noise}} e_i), \quad \epsilon_i \sim \mathcal{N}(0, 1),$$

where W_{noise} controls per-expert noise magnitude and is itself learned. The noise forces exploration: even non-top- K experts receive tokens occasionally, enabling gradient signal to flow to all experts.

All three alternatives sacrifice the token’s agency - the guarantee that each token reaches its highest-affinity experts - to achieve load balance or exploration, and that trade-off costs measurably in model quality or inference compatibility.

Expert-choice routing has one compelling advantage - perfect expert utilization - and two serious drawbacks that together make it impractical.

The first drawback is inference incompatibility. At inference time you process one or a few requests, not a large batch. The “each expert picks top- K_e tokens from a batch” formulation breaks down for $T = 1$: every expert would try to select $K_e = 1/N$ of a single token, which is undefined. You are forced either to revert to token-choice at inference (creating a train-test mismatch the model has never seen during training) or to construct an artificial batch at inference and accept the latency cost. Token-choice has no such issue; each token independently selects K experts regardless of batch size.

The second drawback is variable per-token coverage. In expert-choice, a token is processed by however many experts happen to select it, which can be zero. A token with a diffuse affinity vector (high entropy, low peak) may be skipped entirely, with its hidden state propagating through the layer via the residual connection only, receiving no MoE computation at all. Token-choice guarantees every token is processed by exactly K experts, no exceptions.

OIMoE ran the definitive comparison under controlled conditions: the same 7B-class MoE architecture, same training data, same compute budget, only the routing direction differing. Token-choice achieved substantially better validation loss at every measured checkpoint. The gap was consistent and not noise - expert-choice’s load-balance advantage did not compensate for its coverage disadvantage.

Global assignment achieves better affinity coverage than greedy token-choice (the globally optimal assignment, not a per-token greedy one) and better balance than naive token-choice without an auxiliary loss. But solving even an approximate linear program on every forward pass is prohibitively slow. Sinkhorn iterations on a 64×4096 affinity matrix (64 experts, 4096 tokens per batch) at 10 iterations add roughly 5.2M FLOPs per layer per step. Across 64 layers and millions of training steps, this cost is non-trivial, and the quality gains over token-choice with a simple auxiliary balancing loss are marginal. No production MoE system uses global assignment.

Noisy top- K was ultimately abandoned for a simpler reason: auxiliary-loss balancing achieves comparable exploration with no train-test mismatch. The noise added during training is removed at inference (to be deterministic), but the model has learned routing distributions shaped by noisy training. Auxiliary-loss balancing applies the same uniform-load pressure without introducing any inference divergence.

A Worked Example

Consider a batch of $T = 4096$ tokens processed by $N = 8$ experts, with token-choice $K = 2$ versus expert-choice $K_e = 1024$ (each expert selects 1024 of 4096 tokens). Hidden dimension $d = 2048$.

Token-choice: For each of 4096 tokens, compute 8 dot products with router embeddings, take top-2. Routing cost: $4096 \times 8 \times 2048 \times 2 \approx 134\text{M}$ FLOPs for the affinity matrix (shared by both methods). Result: every token processed by exactly 2 experts; 8192 total expert invocations across the batch.

Expert-choice: Same affinity matrix. Each of 8 experts sorts 4096 scores and selects top-1024. In the ideally balanced case, each token is selected by $8 \times 1024 / 4096 = 2$ experts on average - identical throughput to token-choice. But consider a skewed batch where 512 tokens are from a single domain that all 8 experts prefer. Each expert fills most of its 1024 slots with those tokens. The remaining 3584 tokens compete for $8 \times 512 \approx 4096$ remaining slots; some get 2 expert invocations, others get 0. A token that gets 0 expert invocations passes through the MoE layer unchanged. In a 32-layer model, that token may miss many expert computations, degrading output quality on an input that contained no unusual or out-of-distribution content.

At inference time with $T = 1$: token-choice routes to exactly $K = 2$ experts, always. Expert-choice requires defining K_e for a batch of size 1; $K_e = 1/8$ is not an integer. The method is undefined without patching.

What You Actually Decide in Practice

- **Never use expert-choice if you need single-query inference.** This is not a performance trade-off - it is an architectural incompatibility. Any system that serves user requests one at a time cannot use expert-choice routing without a structural workaround that reintroduces token-choice at inference.
- **Cite global assignment to show depth, but never ship it.** S-BASE and Sinkhorn-based routing are useful interview references demonstrating that you know the design space. Follow immediately with the reason they are not in production: the approximate solver cost is not justified by quality gains over token-choice plus auxiliary loss.
- **Hashing is a useful null hypothesis for ablations.** Routing tokens to experts by a fixed hash function (no learned affinity, no gradient) still outperforms a dense model at the same FLOP count. This benchmark reveals that the gains from MoE come primarily from having more total parameters and from the implicit data parallelism over the token stream, not from intelligent routing. A weak router that merely prevents collapse is sufficient; heroic routing engineering has diminishing returns.
- **Noisy top- K was abandoned in favor of auxiliary-loss balancing.** If your concern is expert collapse (all tokens routing to one expert), the auxiliary loss is simpler, more stable, and has no inference-time side effects. Noisy top- K was a reasonable early attempt but is not used in modern releases.
- **RL-based routing is principled but impractical.** Discrete routing decisions are a natural RL target (the routing choice is an action; the loss improvement is the reward). Clark et al. (2020) benchmarked RL-based routing and found it roughly on par with hashing - not better than top- K with balancing. The gradient variance and compute overhead rule it out for large-scale training.

How This Shows Up in Interviews

- **“What is the difference between token-choice and expert-choice routing?”** Token-choice: each token selects K best experts (variable per-expert load). Expert-choice: each expert selects K_e best tokens (perfect expert utilization, variable per-token coverage). Token-choice wins empirically and is inference-compatible. Expert-choice is inference-incompatible for single-query serving.
- **“Why not solve for the globally optimal routing assignment?”** Global assignment (Sinkhorn, linear program) adds a solver step that costs $O(N \cdot T)$ per layer per forward pass. At training scale, this overhead is non-trivial, and the quality gain over token-choice with auxiliary-loss balancing is marginal. No production system uses it.

- **Senior follow-up: “If you had to use expert-choice for some reason, how would you handle inference?”** The cleanest approach is to train with expert-choice and switch to token-choice at inference, accepting the train-test distribution shift. To reduce that shift, constrain the expert-choice training objective so that every token is guaranteed at least K expert invocations (solve a minimum-coverage assignment rather than a top- K per expert). In practice this requires a more expensive constrained solver and still produces inference behavior the model has not seen during training. The honest answer is: if you need single-query inference, do not use expert-choice - this is exactly why the field did not adopt it.

Key Takeaways

- Expert-choice routing guarantees load balance but cannot be used for single-query inference, making it incompatible with the vast majority of LLM serving systems.
- Global assignment (Sinkhorn, linear program) is theoretically optimal routing but too slow for production; its quality gains over token-choice with auxiliary balancing are marginal.
- OLMoE ablations confirm token-choice produces cleaner loss curves and better final perplexity than expert-choice under the same compute budget.
- Hashing outperforms dense models even with no learned routing, which tells you the benefit of MoE comes from parameter count and token-stream parallelism, not routing intelligence.
- RL-based and noisy top- K routing were early principled explorations; both were abandoned in favor of the much simpler auxiliary-loss balancing approach that is now universal.

Chapter 7 MoE Training and Systems Considerations

7.1 Fine-Grained and Shared Experts

7.1.1 The DeepSeek MoE Blueprint

When an interviewer asks you to design the FFN structure for a large MoE, the right starting point is not an empty whiteboard - it is the configuration that DeepSeek published and that every subsequent open MoE has copied. The fine-grained expert blueprint from DeepSeek MoE has become the de facto standard because it solves a specific parameter-efficiency problem with a clean arithmetic argument. Knowing the configuration, the numbers, and the FLOP trade-offs cold is the difference between sketching an architecture and actually defending one at the senior engineer level.

The vanilla MoE is straightforward: copy a dense FFN N times and activate K copies per token. If each copy is the same size as the dense FFN and $K = 1$, the forward FLOP count is unchanged while total parameters multiply by N . The catch is that with a small N (say, 8), the parameter multiplier is modest. If you want more parameters, you need more experts - but more full-size experts means more memory, more communication, and diminishing returns once you have more experts than training tokens can specialize them.

Fine-grained experts cut through this directly. The insight: shrink each expert to $1/m$ of the standard FFN intermediate dimension and have m times as many experts. Total parameter count stays the same, but now you have m times more distinct expert FFNs. You then activate K of them per token, setting K large enough so that the activated FFN capacity stays near the dense baseline.

Formally, let d_{model} be the hidden dimension and $d_{\text{ffn}} = 4 d_{\text{model}}$ be the standard intermediate dimension. With granularity factor m , each fine-grained expert has intermediate dimension $d_{\text{expert}} = d_{\text{ffn}}/m$. Total experts: $N_{\text{routed}} = m \cdot N_{\text{dense}}$, where N_{dense} is however many full-size experts you would otherwise have. Activated routed experts: K . To maintain roughly the same activated FLOP count as the dense model:

$$K \cdot d_{\text{expert}} \approx d_{\text{ffn}} \implies K \approx m.$$

In practice K is set slightly below m (because shared experts, covered in the next subsection, absorb part of the capacity), but this relation is the governing design equation.

Fine-grained experts trade expert size for expert count at fixed activated FLOP cost, giving a multiplicative parameter expansion that scales as the granularity factor m while leaving the per-token computation budget nearly unchanged.

More distinct FFN parameter matrices mean more distinct processing pathways. The scaling argument is empirical but robust: at fixed training FLOPs, adding more expert parameters consistently lowers validation loss, and fine-grained experts are the mechanism that lets you add those parameters without inflating activated compute.

There is also a combinatorial argument. With $N = 8$ full-size experts and $K = 2$, the model can form $\binom{8}{2} = 28$ distinct expert combinations per token. With $N = 64$ quarter-size experts and $K = 6$, the same token can be processed by $\binom{64}{6} \approx 74,000,000$ distinct combinations - an enormous expansion of the expressible function space at the same FLOP budget. The router learns to navigate this larger space, and empirically it does so successfully.

The architecture also scales cleanly. DeepSeek MoE v1 (16B total, 2.8B active), DeepSeek-V2 (236B total, 21B active), and DeepSeek-V3 (671B total, 37B active) all use the same structural blueprint: fine-grained

experts of the same relative size, the same granularity factor, the same routing mechanism. What changed across versions was only the absolute parameter count and number of experts, not the architecture. This stability is itself strong evidence that the blueprint is correct.

A Worked Example

Take a single MoE layer with $d_{\text{model}} = 2048$, standard dense FFN intermediate dimension $d_{\text{ffn}} = 8192$ ($4 \times$ hidden), and granularity factor $m = 4$. Each fine-grained expert has $d_{\text{expert}} = 2048$.

Total parameter count per MoE layer. Each expert is a two-layer MLP with $W_{\text{up}} \in \mathbb{R}^{2048 \times 2048}$ and $W_{\text{down}} \in \mathbb{R}^{2048 \times 2048}$:

$$\text{params per expert} = 2 \times 2048 \times 2048 = 8.4\text{M}.$$

With 64 routed + 2 shared = 66 experts: $66 \times 8.4\text{M} \approx 554\text{M}$ FFN parameters per layer.

Dense equivalent: $2 \times 2048 \times 8192 = 33.6\text{M}$ per layer. The fine-grained MoE has $554/33.6 \approx 16.5 \times$ more FFN parameters per layer.

Activated FLOP count per token. Active experts: 6 routed + 2 shared = 8 total. One expert forward pass:

$$2 \times 2048 \times 2048 \times 2 \approx 16.8\text{M FLOPs}.$$

Eight active experts: $8 \times 16.8\text{M} \approx 134\text{M FLOPs}$.

Dense equivalent FFN: $2 \times 2048 \times 8192 \times 2 \approx 67\text{M FLOPs}$. The fine-grained MoE costs $\approx 2 \times$ the dense FFN FLOPs per token while carrying $16.5 \times$ the FFN parameters - a $8 \times$ improvement in the parameter-to-FLOP ratio relative to the dense baseline.

Configuration summary at a glance: 64 routed + 2 shared experts, $K = 6$ active routed, each expert is $1/4$ the size of a standard dense FFN. This is the DeepSeek MoE v1 prototype configuration, replicated (with different absolute sizes) in every subsequent release.

What You Actually Decide in Practice

- **Start at $m = 4$ (quarter-size experts) as your baseline.** Finer granularity ($m = 8$, $m = 16$) gives more total parameters but shrinks the per-expert matrix below the efficient tile size for H100 tensor cores (roughly 128×128 in bfloat16). Below an intermediate dimension of ~ 512 for $d_{\text{model}} = 4096$, matrix multiply efficiency degrades measurably. OIMoE, Llama 4, and DeepSeek all operate at $m = 4$ or close to it.
- **Set K so that activated FFN capacity is $1.5\text{-}2 \times$ the dense baseline.** At $K = m$ you match the dense FLOP budget exactly (all activated experts combined equal one standard FFN). Including 1-2 shared experts pushes you to $1.5\text{-}2 \times$. Staying below $2 \times$ keeps the MoE competitive with a dense model of the same total training compute.
- **Keep all fine-grained experts the same size.** Mixed expert sizes force variable-length matrix multiply kernels that kill GPU utilization. Every expert in the routed pool and every shared expert should have the same d_{expert} . Uniform expert size is a hard engineering requirement, not an aesthetic preference.
- **Scale total experts with model size, not just parameter count.** DeepSeek MoE v1 uses 64 routed experts at 16B total parameters; DeepSeek-V3 uses 256 routed experts at 671B. The ratio is sub-linear, not proportional. Expert count is bounded by memory (each expert's weights must fit on some device) and communication overhead (more experts $\Rightarrow$ more all-to-all traffic). A practical ceiling around 256 experts appears in current large-scale deployments.

- **Do not increase K without a FLOP budget analysis.** Every additional active expert adds $2 \times d_{\text{model}} \times d_{\text{expert}} \times 2$ FLOPs per token per layer. At 1T training tokens and 32 layers, each added active expert costs roughly $\text{FLOPs} = 2 \times d_{\text{model}} \times d_{\text{expert}} \times 2 \times 10^{12} \times 32$ total compute. For $d_{\text{model}} = 4096$, $d_{\text{expert}} = 1024$: that is $\sim 5 \times 10^{20}$ FLOPs - half a zettaFLOP per extra active expert. Treat K as a compute budget line item.

How This Shows Up in Interviews

- **“How would you design the FFN structure for a 7B active parameter MoE?”** Anchor to fine-grained experts: pick $m = 4$, 64 total routed experts at $1/4$ -size, $K = 6$ active routed plus 1-2 shared. Walk through the FLOP arithmetic: $8 \times 2 \times d_{\text{model}} \times d_{\text{expert}} \times 2 \approx 2 \times$ the dense FFN FLOPs at $\sim 16 \times$ the FFN parameters. Name the DeepSeek MoE configuration as the template.
- **“What is the parameter efficiency gain from fine-grained experts versus vanilla MoE?”** Vanilla MoE with $N = 8$ full-size experts and $K = 2$ gives $8 \times$ FFN parameters at $2 \times$ FFN FLOPs. Fine-grained $m = 4$, $N = 64$, $K = 6$ gives $\approx 16 \times$ FFN parameters at the same $\sim 2 \times$ FFN FLOPs. The parameter-to-activated-FLOP ratio doubles.
- **Senior follow-up: “What limits expert granularity, and where does DeepSeek’s $m = 4$ come from?”** Two limits: (1) Tensor core efficiency. Matrix multiplies are efficient when the inner dimension is a multiple of the GEMM tile size; below ~ 512 for H100 in bfloat16, MFU drops. At $d_{\text{model}} = 4096$ and $m = 4$, $d_{\text{expert}} = 1024$, which is comfortably above this floor. At $m = 8$ you get $d_{\text{expert}} = 512$ - still acceptable but tighter. At $m = 16$ you get 256, which starts to hurt. (2) Expert parallelism communication. More experts means more all-to-all calls with smaller payloads per call, increasing the ratio of communication to compute. At $m = 4$ the compute-to-communication ratio is favorable on InfiniBand or NVLink clusters; at $m = 16$ it often is not.

Key Takeaways

- Fine-grained experts shrink each FFN to $1/m$ of the standard intermediate dimension and multiply expert count by m , giving m times more total FFN parameters at roughly the same activated FLOP cost per token.
- The canonical configuration from DeepSeek MoE: $m = 4$ (quarter-size experts), 64 routed + 2 shared, $K = 6$ active routed. This blueprint is used verbatim or with minor variation in OIMoE, Llama 4, MiniMax, and DeepSeek-V2/V3.
- Activated FLOP overhead: roughly $2 \times$ the dense FFN cost with the canonical configuration - a small premium for a large parameter expansion.
- Expert granularity is hardware-constrained: below $d_{\text{expert}} \approx 512$ for typical hidden dimensions, tensor core utilization degrades and expert parallelism communication overhead grows.
- The MoE architecture was fixed at DeepSeek MoE v1; scaling to 671B total parameters in V3 required only changing expert count and model width, not restructuring the blueprint.

7.1.2 When Shared Experts Help and When They Don’t

An interviewer will ask you to defend adding a shared expert to your MoE design, and the honest answer is nuanced: the DeepSeek MoE paper showed gains; the OIMoE independent replication found none. If you

answer with a confident “shared experts always help” or “shared experts don’t matter,” you reveal that you read one paper without understanding what the disagreement tells you about the mechanism. The right answer explains when the gain appears and gives a principled decision rule for a new architecture.

A shared expert is an MLP that processes every token, unconditionally, in addition to whatever routed experts the router selects. In a standard fine-grained MoE layer, the output is:

$$h_t = u_t + \sum_{i \in \mathcal{T}_t} g_{i,t} \cdot \text{FFN}_i^{\text{routed}}(u_t).$$

With n_s shared experts added, this becomes:

$$h_t = u_t + \sum_{j=1}^{n_s} \text{FFN}_j^{\text{shared}}(u_t) + \sum_{i \in \mathcal{T}_t} g_{i,t} \cdot \text{FFN}_i^{\text{routed}}(u_t).$$

There is no gate on the shared experts. Every token, regardless of content, passes through all n_s shared MLPs. The shared experts are the same size as the fine-grained routed experts, so all active computation uses uniform matrices - important for kernel efficiency.

The gradient structure differs fundamentally from routed experts. A routed expert receives gradient signal only from the tokens the router sends to it, which under balanced routing is K/N_{total} of all tokens. A shared expert receives gradient from every token in every batch - a dense, high-signal gradient at every step. This means shared experts converge faster and to a more stable minimum than any individual routed expert.

A shared expert is a fully trained dense FFN embedded inside a sparse MoE layer; it handles whatever processing is needed for every token, leaving the routed pool to specialize on the residual token-specific transformation.

The hypothesis is that some MLP transformations are genuinely universal across all token types - common syntactic structures, frequency-heavy vocabulary items, low-level pattern normalization - that every routed expert would otherwise need to partially replicate. By factoring this shared component out, routed experts can specialize more cleanly on the remaining token-specific work, improving effective capacity per routing decision.

This argument is compelling in theory. The empirical record is split. The original DeepSeek MoE paper reports consistent benchmark improvements from adding one shared expert (orange bars) versus no shared expert (blue bars), with further gains on some tasks from a second shared expert. But these are not controlled ablations over a single variable; the shared expert configuration also affects how the authors set K and N .

OIMoE ran a cleaner ablation: same architecture, same training run, only varying the presence of shared experts. Their result (comparing purple versus teal validation loss curves across training) shows no significant difference at their scale - roughly a 7B total parameter model. This is a careful third-party test with no confounds, and it found zero gain.

The reconciliation is likely scale and data. At DeepSeek’s pre-training scale (hundreds of billions of tokens, models with 16B-671B total parameters), routed experts are heavily underfit early in training. An expert receiving 1/64 of all tokens takes far longer to converge than a shared expert seeing 100% of tokens. The shared expert acts as a well-converged baseline that stabilizes learning for the underfitted routed experts. At OIMoE’s smaller scale, routed experts converge faster relative to training time, and the stabilizing benefit disappears.

A Worked Example

Consider a 7B active-parameter MoE with $d_{\text{model}} = 4096$, $d_{\text{expert}} = 1024$ (quarter-size experts), 64 routed experts, and $K = 6$ active routed. Compare three configurations:

Config A: 0 shared experts, $K = 6$ routed. Activated FLOPs per token (FFN component): $6 \times 2 \times$

$4096 \times 1024 \times 2 \approx 100\text{M}$. Total FFN parameters: $64 \times 2 \times 4096 \times 1024 = 536\text{M}$.

Config B: 1 shared expert, $K = 6$ routed (7 active total). Activated FLOPs: $7 \times 100\text{M}/6 \approx 117\text{M}$ (+17%). Total FFN parameters: $65 \times 8.4\text{M} = 546\text{M}$ (+2%).

Config C: 2 shared experts, $K = 6$ routed (8 active total), matching DeepSeek MoE v1. Activated FLOPs: $8 \times 100\text{M}/6 \approx 134\text{M}$ (+34%). Total FFN parameters: $66 \times 8.4\text{M} = 554\text{M}$ (+3%).

The FLOP cost of shared experts is unconditional and permanent. Across 1T training tokens and 32 layers, moving from Config A to Config C adds:

$$(134 - 100) \times 10^6 \text{ FLOPs/token} \times 10^{12} \text{ tokens} \times 32 \text{ layers} \approx 10^{21} \text{ FLOPs} = 1 \text{ exaFLOP}.$$

That is 1 exaFLOP of training compute - roughly 10% of the total compute for a mid-size training run. If that exaFLOP delivers measurably better validation loss, it is well spent. If it does not (OlMoE’s finding), it is a 10% tax with no return.

What You Actually Decide in Practice

- **Default to 0 shared experts; add 1 only if ablations show favorable gain-per-FLOP.** OlMoE’s result is a strong null hypothesis. Do not start with shared experts because DeepSeek used them. Run a 1B-scale ablation for 100B tokens comparing 0 shared vs. 1 shared vs. “1 more routed expert” at the same total FLOP cost. If the shared expert wins the gain-per-FLOP comparison, add it. If adding a routed expert wins, add that instead.
- **Size shared experts identically to routed experts.** If your routed experts have $d_{\text{expert}} = 1024$, the shared experts should also have $d_{\text{expert}} = 1024$. Matching sizes means the same matrix multiply kernel handles all active experts uniformly. A differently-sized shared expert creates a two-kernel path that complicates the implementation with no clear upside.
- **The 2-shared configuration in DeepSeek MoE is partly motivated by accounting symmetry.** With 2 shared + 6 routed, all 8 active experts are equal-size, and the total activated capacity is $8 \times d_{\text{expert}} = 2 \times d_{\text{ffn}}$ - exactly twice the dense FFN capacity. This is convenient bookkeeping but not a principled reason to prefer 2 over 1. Llama 4 uses 1 shared expert; some Chinese releases tried more and retreated to 0 or 1.
- **At $\geq 100\text{B}$ total parameters, shared experts become more worth testing.** The mechanism - stabilizing slow-to-converge routed experts - is proportionally stronger when experts see fewer tokens relative to their parameter count. A model with 256 routed experts and 1T training tokens has each expert seeing roughly $10^{12}/256 \approx 4 \times 10^9$ tokens, which is already tight for full convergence of an MLP. A shared expert seeing all 10^{12} tokens provides a well-converged anchor.
- **Do not freeze or treat shared experts differently from routed experts.** They are standard parameters trained with the same optimizer, learning rate, and weight decay as every other weight in the model. The only difference is the absence of a routing gate. Freezing shared experts after warm-up, or using a different learning rate, introduces complexity with no theoretical justification.

How This Shows Up in Interviews

- **“Why would you add a shared expert to a MoE layer?”** The hypothesis: some MLP-level transformations are universal across token types; factoring them into a dedicated always-on expert allows routed experts to specialize more purely. Shared experts also receive a dense gradient (every token, every step), so they converge faster and act as a stable baseline during early training when routed experts are underfit.

- **“OIMoE found no gain from shared experts but DeepSeek did. How do you reconcile that?”** Scale and training budget. DeepSeek’s models are 16B-671B total parameters trained on hundreds of billions of tokens; OIMoE is ~ 7 B total. At larger scale, each routed expert sees fewer tokens relative to its capacity, leaving more useful gradient for a shared expert to absorb. At OIMoE’s scale, routed experts converge fast enough that a shared component adds nothing net. This is an empirically testable hypothesis: the crossover point is somewhere between 7B and 16B total parameters.
- **Senior follow-up: “How would you run the ablation to decide whether to include a shared expert in a new MoE?”** Train three 1B-parameter models for 100B tokens each, holding all hyperparameters fixed and varying only: (a) 0 shared, 64 routed, $K = 6$; (b) 1 shared, 64 routed, $K = 6$ (+17% FFN FLOPs); (c) 0 shared, 64 routed, $K = 7$ (same +17% FLOPs via one extra active routed expert). Compare held-out perplexity against the FLOP budget. If (b) beats (c), the shared expert is worth its cost. If (c) beats (b), adding a routed expert is the better use of that compute, and you skip the shared expert. This design is the right frame because it controls for the FLOP budget change, which the original DeepSeek ablation did not cleanly do.

Key Takeaways

- A shared expert is an MLP that always fires for every token, with no routing gate; it receives dense gradient signal at every training step, unlike routed experts which are each gated by the top- K selection.
- Empirical evidence is split: DeepSeek MoE reports gains; OIMoE’s controlled ablation finds none. The likely explanation is scale - shared experts help when routed experts are underfit relative to training token budget.
- Each shared expert unconditionally adds $2 \times d_{\text{model}} \times d_{\text{expert}} \times 2$ FLOPs per token; at 1T training tokens and 32 layers, that is ~ 1 exaFLOP per shared expert for typical model sizes. Budget this explicitly.
- Default recommendation: start with 0 shared experts; ablate against the equivalent FLOP spent on an additional active routed expert before committing to a shared expert.
- Size shared experts identically to routed experts for kernel uniformity; train them end-to-end with no special treatment.

7.2 Load Balancing and Stability

7.2.1 Auxiliary Balancing Losses

Every MoE training run I have been involved with hit the same failure mode within the first few thousand steps when load balancing was omitted: one or two experts absorbed almost all the tokens, the rest went dark, and the model ended up as a very expensive dense model wearing a MoE costume. The auxiliary balance loss is the mechanism that prevents this collapse, and understanding its gradient mechanics - why it is differentiable when the routing decision is not, and why it must be tuned carefully - is the core MoE training question at the research engineer level.

Without any intervention, token-choice top- K routing falls into a degenerate local minimum. A randomly initialized router has slightly different affinities for each expert. One expert wins marginally, gets routed more tokens, receives more gradient signal, improves faster, attracts even more tokens, and the cycle compounds. The OIMoE ablation is definitive: with 8 experts and no balancing, 2 experts absorbed roughly 50% of all tokens

within early training; the remaining 6 were effectively dormant. The model had silently reduced itself to a 2-expert MoE while paying for 8.

The auxiliary balance loss breaks this cycle. For N experts and a batch of T tokens, define two vectors:

$$f_i = \frac{1}{T} \sum_{t=1}^T \mathbf{1}[i \in \mathcal{T}_t], \quad P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t},$$

where $\mathcal{T}_t$ is the set of top- K experts selected for token t and $s_{i,t}$ is the pre-top- K softmax score. f_i is the fraction of tokens hard-routed to expert i (discrete, non-differentiable); P_i is the average router probability assigned to expert i (soft, differentiable). The auxiliary loss is:

$$\mathcal{L}_{\text{balance}} = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i,$$

with small coefficient α (typically 10^{-4} to 10^{-2}). The gradient with respect to P_i is:

$$\frac{\partial \mathcal{L}_{\text{balance}}}{\partial P_i} = \alpha \cdot N \cdot f_i.$$

The overloaded expert (large f_i) receives the strongest downward gradient on its router probability. The underloaded expert (small f_i) receives a weaker penalty and naturally attracts more traffic over time. Crucially, f_i acts as a fixed multiplier - the gradient flows only through the differentiable P_i , yet the penalty is proportional to actual load.

Beyond per-expert balancing, you need a parallel device-level loss when using expert parallelism. Replace f_i with f_d , the fraction of tokens routed to GPU d :

$$\mathcal{L}_{\text{device}} = \alpha' \cdot D \cdot \sum_{d=1}^D f_d \cdot P_d,$$

where $P_d = \sum_{i \in \text{device } d} P_i$ is the cumulative router probability for device d . Expert-level balance ensures model quality; device-level balance ensures training throughput.

A third stabilizer is the z-loss on router logits. The softmax normalizer $Z_t = \sum_i \exp(z_{i,t})$ can grow unchecked if router logits drift large, leading to numerical instability and loss spikes, especially in bfloat16 where the precision is limited. The z-loss penalizes large normalizers:

$$\mathcal{L}_z = \frac{\beta}{T} \sum_{t=1}^T (\log Z_t)^2.$$

Removing the z-loss from a trained MoE produces visible validation loss spikes and measurably worse final perplexity in controlled ablations.

The auxiliary balance loss is not optional: without it, an N -expert MoE silently degrades into a 1-2 expert model, wasting the majority of its parameter budget and producing a worse model than a smaller, correctly trained dense baseline.

The $f_i \cdot P_i$ formulation exploits a key asymmetry. f_i is the hard routing fraction - non-differentiable because it depends on the discrete top- K decision. But it acts as a fixed multiplier on the differentiable P_i , so the gradient flows through P_i all the way back to the router embeddings e_i . An overloaded expert's affinity scores are suppressed; an underloaded expert's affinity scores are preserved, and its relative attractiveness grows as competitors are pushed down. Over thousands of training steps, the routing distribution approaches uniformity without requiring any explicit constraint solver.

The alternative approaches fail in practice. RL-based routing uses the correct abstraction (discrete decision, policy gradient), but gradient variance is too high and compute overhead too large. Global assignment (optimal transport, Sinkhorn) finds the best matching but adds latency per forward pass that overwhelms the quality gain.

Noisy top- K (adding learned Gaussian noise to logits) helps with exploration but hurts specialization and creates train-test mismatch. The auxiliary loss wins by being cheap (negligible FLOPs), differentiable (through P_i), and tuneable (via α).

The z-loss complements the balance loss by addressing a different failure mode. The balance loss controls which expert receives each token; the z-loss controls the scale of the affinity scores themselves. Together they address the two ways MoE training can go wrong: routing collapse and numerical instability.

A Worked Example

Consider $N = 8$ experts and a batch of $T = 1024$ tokens. After 2000 training steps with no balancing loss, the router has collapsed: expert 0 receives 512 tokens, expert 1 receives 384 tokens, and experts 2-7 receive a combined 128 tokens.

Computed fractions: $f_0 = 0.500$, $f_1 = 0.375$, $f_2 = \dots = f_7 \approx 0.018$.

Introducing the auxiliary loss with $\alpha = 0.01$, $N = 8$:

$$\mathcal{L}_{\text{balance}} = 0.01 \times 8 \times (f_0 P_0 + f_1 P_1 + \dots + f_7 P_7).$$

Gradient on expert 0: $\partial \mathcal{L} / \partial P_0 = 0.01 \times 8 \times 0.500 = 0.040$.

Gradient on expert 7: $\partial \mathcal{L} / \partial P_7 = 0.01 \times 8 \times 0.018 = 0.00144$.

Expert 0's affinity is pushed down with a gradient $27\times$ stronger than expert 7's. Over subsequent steps, expert 0 loses tokens to experts 2-7, which now receive some gradient signal and begin to specialize. The target balanced state has $f_i = 1/N = 0.125$ for all i , giving:

$$\mathcal{L}_{\text{balance}}^* = 0.01 \times 8 \times 8 \times 0.125 \times \bar{P} \approx 0.01,$$

which is small relative to a typical next-token language modeling loss of ~ 2 -3 nats. At balance, the auxiliary loss contributes less than 0.5% of total gradient magnitude and stops fighting the routing distribution.

For the z-loss: with $\beta = 10^{-3}$ and router logits that have drifted to a maximum of $z_{\max} = 5$, the normalizer is $Z_t \leq Ne^5 = 8 \times 148 \approx 1200$, giving $\mathcal{L}_z \approx 10^{-3} \times (\log 1200)^2 \approx 0.053$. After the z-loss pulls logits back to $z_{\max} \approx 2$, the normalizer drops to $Z_t \approx 8 \times 7.4 \approx 59$, and $\mathcal{L}_z \approx 10^{-3} \times (\log 59)^2 \approx 0.017$ - a $3\times$ reduction in the stability penalty, with logits now safely away from the bfloat16 precision limit.

What You Actually Decide in Practice

- **Set α in $[10^{-3}, 10^{-2}]$ for pre-training; decay toward 10^{-4} once utilization is healthy.** At $\alpha = 10^{-2}$, the balance loss gradient is comparable to the language modeling gradient for a severely collapsed router. At $\alpha = 10^{-4}$, it is a light touch that maintains balance without distorting routing. Monitor the expert utilization histogram (fraction of steps where each expert is the bottom or top decile); if variance is low, decay α .
- **Always add the device-level loss when using expert parallelism.** Expert-level balance does not imply device-level balance. With 64 experts on 8 GPUs, even uniform per-expert load can still result in one GPU handling a disproportionate share of a given batch if the popular experts happen to co-locate. The device-level loss costs almost nothing to compute (aggregate f_i into device buckets) and prevents GPU utilization hot spots that bottleneck the entire cluster.
- **Include the z-loss from step 1, coefficient $\beta \approx 10^{-3}$.** Loss spikes from unconstrained router logits are hard to recover from and can contaminate checkpoints mid-run. The z-loss is cheap and its effect is

measurable: removing it produces ~ 0.1 - 0.2 nats worse final validation perplexity in published ablations. Do not wait to see instability before adding it.

- **Compute router logits in float32, always.** In bfloat16, two logits that differ by less than 2^{-7} (the machine epsilon at large magnitude) round to the same value, causing artificial ties in the affinity ranking. These ties resolve arbitrarily, adding stochasticity to what should be a deterministic top- K decision. Cast to float32 before the inner product and softmax; cast back to bfloat16 after the gate weights are computed.
- **Watch for token dropping and treat it as a signal of imbalance.** When an expert’s load factor (tokens received / capacity limit) exceeds 1.0, tokens are dropped: they pass through the layer via the residual connection with no expert computation. Token dropping at inference creates batch-level stochasticity (whether a token is dropped depends on who else is in the batch). Sustained dropping during training indicates the auxiliary loss is under-tuned; increase α temporarily.

How This Shows Up in Interviews

- **“What happens if you train a MoE without any load balancing?”** Router collapse to 1-2 experts occurs within thousands of steps. The remaining experts receive no gradient signal and remain near random initialization. You end up with a dense model of size $K \times d_{\text{expert}}$ carrying the dead weight of $N - K$ additional expert parameter sets, worse than a correctly-sized dense model at the same FLOP budget.
- **“Explain the Switch Transformer auxiliary loss and why it is differentiable when the routing decision is not.”** Define f_i as the hard token fraction (non-differentiable) and P_i as the soft router probability (differentiable). The loss is $\alpha N \sum_i f_i P_i$. The gradient $\partial \mathcal{L} / \partial P_i = \alpha N f_i$ flows through P_i to the router embeddings, with f_i serving as a load-proportional multiplier. No gradient passes through the discrete top- K decision itself.
- **Senior follow-up: “Why do you need both a per-expert and a per-device balancing loss, and can one subsume the other?”** They address different failure modes that are independent. Expert-level balance ensures uniform training exposure across all parameter sets - a model quality concern. Device-level balance ensures uniform GPU utilization - a throughput concern. With k experts per GPU, even perfect per-expert balance allows a GPU to be overloaded if all k of its experts are popular on a given batch. The device-level loss catches this and prevents one GPU from becoming the critical path in the all-to-all dispatch. Neither loss can be derived from the other; both are required when using expert parallelism.

Key Takeaways

- Without a balance loss, routing collapse to 1-2 experts is universal and fast; you silently lose most of the parameter budget you paid for.
- The auxiliary balance loss $\mathcal{L} = \alpha N \sum_i f_i P_i$ is differentiable through P_i even though f_i is not; the hard routing fraction acts as a fixed, load-proportional multiplier on the router gradient.
- Include a device-level version of the same loss when using expert parallelism; expert-level balance does not imply device-level balance.
- Z-loss ($\mathcal{L}_z = \beta (\log Z_t)^2$) prevents router logits from drifting large; removing it causes loss spikes and measurably worse final perplexity.
- Start with $\alpha \approx 10^{-2}$, compute router logits in float32, add z-loss from step 1, and monitor expert utilization histograms throughout pre-training.

7.2.2 Auxiliary-Loss-Free Balancing (DeepSeek-V3)

After running long MoE pre-training jobs, a pattern becomes clear: the auxiliary balance loss is doing something uncomfortable. Every gradient step, it actively penalizes the routing the language model would naturally prefer, pushing away from the best expert and toward the least-loaded one. DeepSeek-V3’s bias update scheme decouples load control from loss minimization entirely - the optimizer sees only the language modeling gradient, while a separate online controller handles balance. This design shows up in senior-level interviews as the question that distinguishes engineers who understand auxiliary losses from those who have thought about what they actually cost.

The standard auxiliary loss creates a constant tension between two objectives: route to the best expert (language modeling) and route uniformly (balance). These objectives are not aligned, and every training step the balance loss gradient steers the router away from whatever distribution the language model would prefer. At large scale with many experts, this interference becomes harder to tune and can destabilize training.

DeepSeek-V3 replaces batch-level auxiliary balancing with a per-expert bias $b_i \in \mathbb{R}$ for each expert $i = 1, \dots, N$. The routing decision uses the biased score:

$$\tilde{s}_{i,t} = s_{i,t} + b_i,$$

where $s_{i,t} = \text{Sigmoid}(u_t^\top e_i)$ is the base affinity. Top- K selection is done on $\tilde{s}_{i,t}$. Critically, the gate weight in the output sum uses $s_{i,t}$ only, not $\tilde{s}_{i,t}$:

$$h_t = u_t + \sum_{i \in \mathcal{T}_t} s_{i,t} \cdot \text{FFN}_i(u_t),$$

where $\mathcal{T}_t = \text{TopK}(\{\tilde{s}_{i,t}\}, K)$. The bias adjusts who gets selected; the gate weight is determined by the unbiased learned affinity. Output quality is therefore governed by the model’s actual competence estimates, not by load corrections.

The b_i values are updated by a simple online rule at the end of each batch, outside of backpropagation:

$$b_i \leftarrow \begin{cases} b_i - \gamma & \text{if expert } i \text{ received more than } T/N \text{ tokens} \\ b_i + \gamma & \text{if expert } i \text{ received fewer than } T/N \text{ tokens} \end{cases}$$

where $\gamma > 0$ is a fixed step size. This is not gradient descent. It is a feedback controller: track load, increment the bias of underloaded experts, decrement the bias of overloaded experts. No loss term, no interaction with the optimizer, no gradient interference.

Despite claiming to be “auxiliary loss-free,” DeepSeek-V3 also includes a complementary sequence-wise auxiliary loss with a small coefficient β_{seq} . The batch-level controller handles average balance across the training corpus; the sequence-wise loss ensures that within a single sequence, no expert is overwhelmed by an unusual token distribution. At inference time, a single long sequence with atypical content can still saturate an expert even when batch-level balance is perfect. The sequence-wise loss provides this per-inference guarantee.

The key innovation is treating load balancing as an operations problem solved by a feedback controller, not as a learning problem penalized through the loss, leaving the gradient optimizer free to find the best routing distribution without a conflicting balance objective.

The standard auxiliary loss conflates two objectives with fundamentally different structures. Language modeling is a differentiable learning problem with a clear signal: minimize prediction error. Load balancing is a logistics problem: equalize throughput across N parallel compute units. Conflating them means the optimizer receives a gradient that is a mixture of these two objectives, and the mixture coefficient α must be tuned across every model size, expert count, and data distribution.

The bias controller handles the logistics problem with a logistics tool - a feedback control loop that observes the current state (token counts), measures the error (deviation from T/N), and applies a corrective signal ($\pm\gamma$). This is more appropriate than gradient descent for a problem with no loss landscape to optimize, just a constraint to satisfy.

The practical benefit is cleaner gradient signal for the router. The router embeddings e_i are trained exclusively by language modeling gradients; every gradient step is an improvement in routing quality, not a compromise between quality and balance. The controller handles balance externally, so the optimizer is free to find the best routing distribution the data supports.

There is a cost: the online controller introduces a step size hyperparameter γ and requires global visibility of per-expert token counts (an all-reduce over data-parallel replicas). These are manageable engineering concerns. The bigger limitation is that the controller responds to average load over time; a single batch with extreme imbalance is not corrected within that batch - only subsequent batches benefit. This is why the sequence-wise auxiliary loss is retained: it provides within-sequence balance that the controller cannot supply.

A Worked Example

Consider $N = 64$ experts, a batch of $T = 2048$ tokens, target load $T/N = 32$ tokens per expert, and $\gamma = 0.001$.

After a single batch, expert 5 received 90 tokens (overloaded by 58) and expert 30 received 8 tokens (underloaded by 24).

Bias updates:

- Expert 5: $b_5 \leftarrow b_5 - 0.001$
- Expert 30: $b_{30} \leftarrow b_{30} + 0.001$

Suppose the imbalance persists for 500 consecutive batches at the same severity. After 500 steps:

$$\Delta b_5 = -0.001 \times 500 = -0.500, \quad \Delta b_{30} = +0.001 \times 500 = +0.500.$$

Expert 5's effective routing score is reduced by 0.5 across all tokens; expert 30's is increased by 0.5. For context, the base affinity scores $s_{i,t} = \text{Sigmoid}(u_t^\top e_i)$ are in $[0, 1]$. A bias shift of ± 0.5 is a large correction - sufficient to move expert 30 from below the top- K threshold into the selected set for many tokens, even without any change to the router embeddings e_i .

Crucially, the gate weight for token t routed to expert 30 is still $s_{30,t} \in [0, 1]$, not $s_{30,t} + 0.5$. If expert 30 has low affinity (e.g., $s_{30,t} = 0.1$), its contribution to the output is $0.1 \times \text{FFN}_{30}(u_t)$ - a small but nonzero contribution. The model quality is determined by whether FFN_{30} has learned anything useful for this token, not by how much the controller has inflated expert 30's routing score.

Now compare to the auxiliary loss approach: with $\alpha = 0.01$ and the same imbalance, the gradient pushes P_5 down with magnitude $0.01 \times 64 \times (90/2048) = 0.028$ per step. This gradient modifies the router embeddings e_5 , changing how expert 5 appears to all tokens, not just those in the overloaded batch. The interference with the language modeling gradient is direct and persistent.

What You Actually Decide in Practice

- **Set γ so that severe imbalance corrects over ~ 1000 steps, not 10.** A step size that corrects imbalance in 10 steps will oscillate: expert 5 gets $b_5 = -0.5$ in 500 steps, the router avoids it, now expert 5 is underloaded, b_5 climbs back, and the cycle repeats with period ~ 1000 steps. Target a correction timescale

of several thousand steps; values around $\gamma = 10^{-3}$ to 10^{-4} are typical for normalizing b_i relative to affinity scores in $[0, 1]$.

- **Do not include b_i in the gate weight.** This is the most critical implementation detail. If $\tilde{s}_{i,t} = s_{i,t} + b_i$ is used both for selection and as the gate weight, then a heavily biased expert receives artificially inflated gate weights and dominates the output sum even if its learned affinity is low. The output quality becomes dependent on load history, which is nonsensical. Use $s_{i,t}$ for the gate, $\tilde{s}_{i,t}$ only for the top- K selection.
- **The “auxiliary loss-free” claim is accurate for batch-level balance, not total.** DeepSeek-V3 retains a sequence-wise auxiliary loss with coefficient $\beta_{\text{seq}} \approx 3 \times 10^{-4}$ (roughly 3-10 $\times$ smaller than a typical batch-level coefficient). The sequence-wise loss handles within-inference imbalance that the controller cannot address. This is a meaningful engineering improvement over the prior art - batch-level gradient interference is eliminated - but read the fine print before claiming the model is fully loss-free.
- **Aggregate f_i counts across data-parallel replicas before each controller update.** If each data-parallel replica updates b_i independently based on its local shard of the batch, replicas will apply different corrections to the same b_i , causing divergence. Use an all-reduce (sum and divide by the number of replicas) over the per-expert token counts before computing the update. This is a single collective call per step with a payload of N integers - negligible overhead.
- **Use this approach when $N \geq 64$; for smaller N , the standard auxiliary loss is simpler.** The interaction between auxiliary loss gradients and language modeling gradients becomes harder to tune as N grows, because the balance loss gradient is spread across more experts with weaker per-expert signal. For $N \leq 16$ (Mixtral-style MoEs), the standard auxiliary loss is well understood and easily tuned. The bias controller is most valuable at $N \geq 64$.

How This Shows Up in Interviews

- **“How does DeepSeek-V3 achieve load balance without an auxiliary loss?”** Per-expert bias scalars b_i are added to routing scores for top- K selection but excluded from gate weights. An online controller updates b_i by $\pm\gamma$ each step based on whether expert i was over or under-loaded in the previous batch. No gradient flows through this update; the optimizer sees only language modeling gradients.
- **“Why is it important to exclude the bias from the gate weight computation?”** The bias is a load-control signal, not a quality signal. If it were included in the gate weight, an expert with inflated b_i (promoted to compensate for low affinity) would also receive a high gate weight, amplifying its output regardless of whether its FFN has learned anything useful for those tokens. Keeping the gate weight $= s_{i,t}$ ensures that output quality is governed by learned competence, not load management history.
- **Senior follow-up: “DeepSeek-V3 claims to be auxiliary loss-free but still includes a sequence-wise auxiliary loss. Is this a meaningful distinction or marketing?”** It is a meaningful engineering distinction, though the terminology overstates the case. The batch-level auxiliary loss is what causes gradient interference: it has a large coefficient, acts every batch, and its gradient direction conflicts with the language modeling gradient in proportion to f_i . Removing it cleanly separates the two objectives. The sequence-wise loss has a coefficient 3-10 $\times$ smaller, targets a different failure mode (within-sequence imbalance at inference time), and creates far less interference because routing distributions within a single sequence are already partially balanced by the learned router. The claim is accurate in engineering terms even if incomplete in literal terms.

Key Takeaways

- DeepSeek-V3 replaces the batch-level auxiliary loss with per-expert bias scalars b_i updated by an online feedback controller ($b_i \leftarrow b_i \pm \gamma$ based on token load), with no gradient flowing through the update.
- The bias adjusts top- K selection but is excluded from gate weights; output quality is determined by learned affinity $s_{i,t}$, not by load corrections.
- The method retains a sequence-wise auxiliary loss with a small coefficient for within-inference balance robustness; the improvement is specifically the elimination of batch-level gradient interference, not the removal of all balance objectives.
- Bias-based control is most valuable at $N \geq 64$ experts; for smaller MoEs the standard auxiliary loss is simpler and well-understood.
- Implementation requires a global all-reduce over per-expert token counts before each controller update to prevent divergence across data-parallel replicas.

7.3 Expert Parallelism Gotchas for Interviews

7.3.1 Token Dropping and Capacity Factors

The question “why does our MoE API return different outputs for the same prompt at temperature zero?” is one of those interview traps that separates engineers who have run MoE in production from those who have only read about it. The answer is almost always token dropping - the mechanism that prevents any single expert from being overwhelmed. Understanding capacity factors and what happens to dropped tokens is a staple of senior MoE system design questions, sitting at the intersection of routing correctness, memory management, and serving reliability.

Expert parallelism works by dispatching tokens to different devices via an all-to-all collective. Each expert lives on one or a few GPUs and can process only a bounded number of tokens before its on-device buffer fills. The *capacity factor* C defines this bound:

$$\text{capacity} = \left\lfloor C \cdot \frac{T}{N} \right\rfloor$$

where T is the total number of tokens in the current dispatch batch and N is the number of experts. Each expert is pre-allocated a slot buffer of exactly this size. If more tokens route to an expert than the buffer allows, the excess tokens are dropped.

A dropped token is not erased from the sequence. Its hidden-state vector passes unchanged through the residual connection, completely bypassing the MLP. The MoE layer contributes nothing to that token’s representation at this layer; the output is numerically identical to the input for that position. The sequence length and causal mask are unaffected - the token is still there, just unprocessed.

Capacity factors in practice range from $C = 1.0$ (zero tolerance for imbalance) to $C = 2.0$ (buffer twice the fair-share load). Switch Transformer and most production systems use $C \approx 1.25$ to 1.5 as the default range.

The capacity factor is the MoE’s circuit breaker: it trades a small number of under-processed tokens for a guarantee that every expert finishes in bounded, predictable time.

The alternative to capacity-capped dispatch is allowing arbitrarily deep per-expert queues. Under token-choice top- K routing, popular experts can attract several times their fair share of tokens in a single batch - especially early in training when the router has collapsed onto a small set of preferred experts. Without a cap,

that GPU stalls draining the queue, creating a latency tail that ruins batch efficiency for every other expert on the device. With a fixed capacity, every expert's forward pass takes the same wall-clock time regardless of routing decisions, which is exactly the property you need to keep GPUs synchronized, saturate tensor cores, and hide all-to-all latency.

The cost is representation quality for the dropped tokens. A token dropped from its top-ranked expert falls back to the residual only. If the routing has learned to rely on a specific expert for specific token types - which it does, given sufficient training - those tokens receive a degraded representation at that layer. Empirically, per-batch drop rates above roughly 5% consistently correlate with measurable perplexity increases.

The stochasticity angle is the subtler point. At inference time, whether a particular token gets dropped depends on who else is in the same dispatch batch. A prompt running in isolation may have zero drops; the same prompt batched with other high-traffic sequences may lose tokens to overloaded experts. The token's position in the batch, and therefore its output representation, changes based on which other requests arrived concurrently - producing different completions even at temperature zero, with no randomness introduced by sampling.

A Worked Example

Consider a 7B-parameter MoE with 64 fine-grained experts (each one-quarter the size of a standard FFN) spread across 8 H100s, 8 experts per GPU. An inference batch contains 1024 tokens. With $C = 1.25$:

$$\text{capacity} = \left\lfloor 1.25 \times \frac{1024}{64} \right\rfloor = \lfloor 20 \rfloor = 20 \text{ tokens per expert}$$

Total slot buffers: $64 \times 20 = 1280$ slots. The system can absorb up to 25% routing imbalance before any drops occur.

Now suppose the batch is heavy with English function words - "the," "a," "of" - whose hidden states cluster together, routing them all toward expert 3. Expert 3 receives 35 tokens but its buffer holds only 20; 15 tokens are dropped. Those 15 positions pass through the MoE layer unchanged.

Memory cost of the buffers: $64 \times 20 \times 4096 \times 2 \text{ bytes} \approx 10.5 \text{ MB}$, negligible. Raising C to 2.0 doubles the buffer to 21 MB but reduces the drop rate to near zero for typical routing distributions, a clear win unless the system is severely memory-constrained.

What You Actually Decide in Practice

- **Set $C = 1.0$ only when memory is critically constrained and your balance loss is mature.** A capacity of 1.0 guarantees no extra buffer memory but drops tokens the moment any imbalance exists. Accept it only if per-expert drop rates stay consistently below 1%.
- **Default to $C = 1.25$ -1.5 in most production serving deployments.** This absorbs typical hot-expert bursts from mildly skewed batches without materially increasing buffer memory.
- **Monitor drop rate as a first-class training metric.** A per-expert drop rate above 5% averaged over a training run is a load-balancing failure, not a capacity-factor failure. Raising C papers over the symptom; the correct fix is re-tuning the balance loss coefficient.
- **Consider no-drop inference routing for latency-sensitive APIs.** Route overflow tokens to their second-ranked expert rather than dropping them. This eliminates batch-dependent non-determinism at temperature zero. Accept the mild training-inference distribution mismatch - in practice, the effect is small when

overflow rates are low.

- **Fine-grained experts reduce drop pressure naturally.** Smaller experts mean each expert handles fewer tokens on average, so the same capacity factor tolerates a larger absolute routing imbalance before the buffer fills. This is one of the underappreciated systems benefits of the DeepSeek fine-grained expert design.

How This Shows Up in Interviews

- *“A user reports that your MoE API returns different outputs for the same prompt at temperature zero. You have confirmed there is no sampling involved. What could cause this?”* Strong answer: token dropping is batch-dependent. The same prompt, batched with other requests, may share expert buffers with high-traffic sequences. If a hot expert is saturated by other sequences in the batch, tokens from this prompt are dropped at that layer, changing the representation and thus the output. The fix is either no-drop inference routing or request isolation.
- *“What is the capacity factor in a mixture-of-experts model, and what trade-off does it control?”* Strong answer: $\text{capacity} = \lfloor C \cdot T/N \rfloor$ sets the maximum number of tokens any expert processes in one dispatch round. C trades representation quality (fewer drops at higher C) against memory and buffer latency (larger pre-allocated buffers at higher C). Setting C too low causes frequent drops on imbalanced batches; setting it too high wastes memory that could hold activations or KV cache entries.
- *Senior follow-up: “If you eliminate token dropping at inference time by routing overflow tokens to their next-ranked expert, what new problem might arise and when does it matter?”* Senior answer: the model trained under a regime where overflow tokens were dropped, not re-routed. Re-routing changes the effective per-expert load distribution seen at inference, shifting the activation statistics that downstream layers were calibrated on. For mildly imbalanced batches this discrepancy is small enough to ignore. For heavily skewed batches - such as a batch of all-code prompts on a model trained on mixed data - the effect can be meaningful. One mitigation is to include some no-drop re-routing during fine-tuning so the model sees both modes.

Key Takeaways

- Each expert has a fixed slot buffer of capacity $= \lfloor C \cdot T/N \rfloor$ tokens; tokens exceeding the cap are dropped and receive only the residual-stream pass-through with no MLP contribution.
- Token dropping is batch-dependent: whether a given token is dropped depends on which other requests occupy the same dispatch batch, making it the primary source of non-determinism in MoE serving at temperature zero.
- Production capacity factors of 1.25-1.5 absorb typical routing imbalance; persistent drop rates above 5% signal a load-balancing failure that requires fixing the balance loss, not just raising C .
- No-drop inference routing eliminates serving non-determinism at the cost of a tolerable training-inference distribution mismatch.
- Fine-grained experts (smaller size, more count) reduce the per-expert token variance and therefore reduce drop rates at any given capacity factor.

7.3.2 Device-Level vs. Expert-Level Balancing

Once an interviewer is satisfied that you understand expert collapse, the pivot is almost immediate: “but what if your experts are perfectly balanced and your GPUs still aren’t?” That question targets device-level balancing - the second tier of the two-layer hierarchy that every serious MoE training system implements. Conflating expert-level and device-level balancing is one of the most common mistakes I see in system design conversations about MoE at scale, and at senior level it is a reliable differentiator.

MoE training systems maintain two distinct load-balancing objectives that operate at different granularities of the parallelism stack.

Expert-level balancing targets the routing distribution itself. The Switch Transformer auxiliary loss is:

$$\mathcal{L}_{\text{expert}} = N \sum_{i=1}^N f_i \cdot P_i$$

where N is the number of experts, f_i is the fraction of tokens dispatched to expert i in the batch (a hard routing count), and P_i is the mean router probability assigned to expert i across the same batch (a soft, differentiable quantity). The product $f_i \cdot P_i$ couples the discrete routing decision to the continuous router output, giving useful gradients while penalizing load concentration. Minimizing $\mathcal{L}_{\text{expert}}$ drives the router toward a uniform distribution over experts.

Device-level balancing has the same algebraic structure but redefines the summation unit from individual experts to device groups:

$$\mathcal{L}_{\text{device}} = D \sum_{d=1}^D F_d \cdot P_d$$

where D is the number of devices, F_d is the fraction of tokens dispatched to device d , and P_d is the total router probability directed toward all experts residing on device d . Minimizing this term discourages the router from routing a disproportionate share of tokens to a single GPU, which would saturate that device’s inbound all-to-all bandwidth regardless of how evenly the load is distributed among the experts on that device.

Expert-level balancing prevents expert collapse; device-level balancing prevents network hotspots - they are complementary constraints at different granularities of the parallelism hierarchy.

The two objectives can be satisfied simultaneously or independently, and that is exactly the problem. Consider 64 experts on 8 GPUs (8 experts per GPU). Expert-level balancing can be perfectly satisfied with every expert receiving exactly $T/64$ tokens. But if the routing is correlated - say, high-frequency English function words all cluster toward experts on GPUs 0 and 1 - those two GPUs still receive double the inbound all-to-all traffic of GPUs 2-7. The expert-level loss sees balance at the per-expert granularity; the device-level loss fires a separate penalty for the bandwidth hotspot.

Before device-level losses were standard, practitioners would observe mysteriously variable step times and diagnose the culprit as imbalanced communication rather than imbalanced compute. The fix of adding $\mathcal{L}_{\text{device}}$ with a smaller coefficient (typically 10-50× smaller than the expert-level coefficient) is cheap and dramatically stabilizes per-step time variance on multi-node runs. The DeepSeek training reports note exactly this kind of per-device communication balancing as a necessary addition when scaling beyond 16 billion active parameters.

DeepSeek-V2 also introduced a complementary structural mechanism: **top- M device selection**. Before picking the top- K experts for a token, the router first restricts its candidate pool to the top- M devices by aggregated affinity score. This hard constraint bounds the number of distinct all-to-all hops any single token can

generate, making worst-case communication cost predictable even with hundreds of experts spread across many devices. The loss-based approach and the structural approach are complementary: the loss gently guides the router during training; the device cap provides a hard budget guarantee at both training and inference time.

A Worked Example

A 16B-parameter MoE has 64 routed experts spread across 16 H100s (4 experts per GPU). A training batch dispatches 4096 tokens.

Ideal per-expert load: $4096/64 = 64$ tokens. Ideal per-device load: $4096/16 = 256$ tokens.

Suppose the router concentrates traffic toward experts 0-7 (GPUs 0 and 1) because these experts handle high-frequency subword tokens. Each of the 8 overloaded experts receives 128 tokens ($2\times$ fair share), while the remaining 56 experts average 48 tokens.

Expert-level loss response: $f_i \cdot P_i$ for experts 0-7 is elevated. The gradient pushes router scores for these experts downward. Improvement is gradual because the loss signal is shared across all 64 experts.

Device-level loss response: GPUs 0 and 1 each receive $128 \times 4 = 512$ tokens, giving $F_0 = F_1 = 0.125$ against the ideal 0.0625. The device-level penalty fires immediately and more directly against the GPU-level concentration, pushing the router to diversify across devices even when no single expert appears badly overloaded.

Without the device-level loss, the expert-level loss alone would require many gradient steps to redistribute load sufficiently, during which all-to-all bandwidth on GPUs 0 and 1 remains the per-step bottleneck and step time shows high variance.

What You Actually Decide in Practice

- **Start with $\alpha_{\text{expert}} \approx 10^{-2}$ and $\alpha_{\text{device}} \approx 10^{-3}$.** The device-level loss is a secondary constraint; it should be strong enough to prevent hotspots but weak enough not to override the expert-level routing signal.
- **Apply top- M device selection when scaling beyond 32 experts.** Restricting each token to the top- $M = 3$ -4 devices bounds worst-case communication cost without meaningfully degrading routing quality, since routing to a seventh or eighth device almost never helps representation quality.
- **Switch to sequence-wise balancing at inference time.** Batch-level losses work well during training when batches are large and representative. At inference, request batches are small and user-controlled - a single unusual query can dominate a batch. A sequence-wise loss that balances routing within each sequence individually is more robust to out-of-distribution input patterns.
- **Instrument per-GPU inbound token count, not just per-expert token count.** A per-expert utilization dashboard can show perfect balance while two GPUs are starving for bandwidth. Add a per-device token histogram to your training dashboard from day one.
- **Understand what DeepSeek-V3's online bias update b_i actually controls.** The bias terms increment or decrement based on running per-expert load, replacing the batch-level expert-load auxiliary loss. They do not directly control how load is distributed across devices. The sequence-wise auxiliary loss DeepSeek-V3 retains is specifically there to handle device-level concentration on unusual inference sequences - the system is not "auxiliary-loss-free" at the device level.

How This Shows Up in Interviews

- “You have 64 experts on 16 GPUs. Training throughput is fine on average, but two GPUs consistently show $3\times$ higher network utilization than the rest. What is likely happening and how do you fix it?” Strong answer: the router has learned to concentrate dispatches on a subset of devices hosting popular experts. Per-expert utilization metrics may look balanced, but the device-level aggregation is skewed. The fix is to add $\mathcal{L}_{\text{device}}$ with a coefficient in the 10^{-3} range and, if using fine-grained experts with many hundreds of experts, also apply top- M device selection to structurally bound the worst-case inbound traffic per GPU.
- “Explain the difference between expert-level and device-level load balancing in a MoE training system.” Strong answer: expert-level balancing penalizes routing concentration at individual-expert granularity via the $f_i \cdot P_i$ structure, preventing expert collapse and ensuring parameter utilization. Device-level balancing applies the same $F_d \cdot P_d$ structure at per-GPU granularity, preventing all-to-all communication hotspots. The two objectives can conflict: expert-level balance does not imply device-level balance if multiple high-traffic experts co-locate on the same GPU.
- Senior follow-up: “DeepSeek-V3 claims ‘auxiliary-loss-free balancing’ but still uses a sequence-level auxiliary loss. Is this a contradiction, and what does each component actually control?” Senior answer: not a contradiction. The online bias b_i replaces the batch-level expert-load auxiliary loss - it adjusts individual expert attractiveness reactively without adding a differentiable loss term to the training objective. However, b_i operates on batch-level running averages, which work well during training but cannot prevent a single unusual inference sequence from collapsing onto a few experts and overloading the corresponding devices. The sequence-wise auxiliary loss is a separate, lighter regularizer that operates within individual sequences to ensure routing robustness at inference time. The two mechanisms address different timescales: b_i handles slow training-time drift; the sequence-wise loss handles fast inference-time spikes.

Key Takeaways

- Expert-level and device-level balancing are separate objectives that can be satisfied independently; satisfying one does not guarantee the other.
- Both use the $f \cdot P$ inner-product structure from Switch Transformer, but differ in their summation unit: individual experts versus device groups.
- Device-level hotspots appear as high all-to-all bandwidth variance across GPUs, not as per-expert utilization imbalance - monitor both to catch both failure modes.
- Top- M device selection (DeepSeek-V2) provides a structural hard cap on per-token communication cost that is complementary to loss-based balancing.
- At inference time, sequence-wise balancing is more robust than batch-wise balancing because user request distributions are uncontrolled and can create sharp within-batch skew that batch-level mechanisms are slow to correct.

Part IV

GPUs, Kernels, and Single-Device Performance

Chapter 8 GPU Architecture for LLM Engineers

8.1 The Compute and Memory Hierarchy

8.1.1 SMs, Warps, Threads, and Blocks

An interviewer designing a custom CUDA kernel or probing your understanding of FlashAttention will not ask “what is an SM?” - they will ask you to walk through what happens when a warp executes a matrix multiply, why warp divergence tanks throughput, and why tile counts that are not multiples of the SM count cause a cliff drop in utilization. Getting this right requires internalizing the three-level execution hierarchy cold. This question appears at the ML infrastructure engineer and systems engineer level, and confidently deriving performance predictions from first principles is what separates a strong answer from a mediocre one.

A GPU is a throughput machine, not a latency machine. A CPU is designed to finish any single task as fast as possible - its die area goes to branch predictors, out-of-order execution engines, and large caches that minimize single-thread latency. A GPU sacrifices per-thread latency to maximize the total number of operations completed per second across thousands of concurrent threads. If you give a CPU four tasks, it finishes the first quickly, then the second, and so on. A GPU finishes all four simultaneously - later in wall-clock time than the CPU’s first task, but far sooner in aggregate.

The atomic unit of GPU computation is the *Streaming Multiprocessor* (SM). An SM is a self-contained engine: it has its own control logic, register file, shared memory, and specialized arithmetic units including tensor cores. An A100 SXM4 has 108 SMs; an H100 SXM5 has 132. Each SM can hold up to 2048 threads in flight simultaneously, which it uses to hide memory latency by switching between warps while some are stalled.

The execution hierarchy has three levels:

Block. A block is a group of threads assigned to a single SM for the duration of its execution. All threads in a block share the SM’s shared memory and can synchronize with barriers. A block cannot span SMs. The programmer chooses how many threads are in each block (must be a multiple of 32) and how many blocks to launch (the *grid*).

Warp. A warp is 32 consecutively numbered threads within a block. This is the unit at which the SM dispatches instructions. All 32 threads in a warp execute the same instruction simultaneously, each on different data. This execution model is called SIMT: Single Instruction, Multiple Threads. The SM maintains many warps concurrently; when one warp stalls on a memory read, the SM instantly switches to another ready warp with zero overhead (each warp’s live state is stored in registers, not on a call stack).

Thread. A thread is the individual execution lane. Each thread has its own registers and runs its own instance of the kernel code. Threads within a warp are always in lockstep; threads in different warps are independent.

The performance consequence of SIMT is *warp divergence*: if threads within a warp take different branches through an `if-else`, the SM must execute both branches serially, masking off the inactive threads for each. A 50/50 branch split halves effective throughput. Kernels should be written to avoid conditional logic that splits warps.

The SM is the atom of GPU computation: blocks are assigned to SMs, warps are the 32-thread instruction-dispatch units within a block, and any branch that splits a warp forces serial execution - design kernels to maintain warp uniformity.

The SIMT model wins because matrix multiplies and tensor operations are perfectly amenable to it: every thread does exactly the same arithmetic on a different slice of the matrix. The GPU dedicates the vast majority of its die area to arithmetic units rather than control logic, which is why an H100 delivers $\sim 2,000$ TFLOP/s of bfloat16 tensor-core throughput while a CPU core delivers ~ 1 TFLOP/s at most - not because the GPU runs at a higher clock, but because it runs the same instruction across thousands of parallel threads at once.

The lightweight thread model is also central to the design. A CPU thread requires a full stack, OS scheduling, and context switch overhead measured in microseconds. A GPU thread's entire state is a fixed-size register file entry; switching between warps takes zero cycles because the SM simply picks up where the other warp left off. This is how an SM with 2048 in-flight threads achieves high utilization despite the 300-cycle latency of a global memory access: 300 cycles is enough time to cycle through many other warps and keep the arithmetic units fed.

The alternative - a CPU-style design with a small number of heavy threads and large caches - would give low latency on a single problem but would not scale to the thousands of simultaneous accumulators that a large matrix multiply requires.

A Worked Example

Consider a bfloat16 matrix multiply $C = A \times B$ where $A, B \in \mathbb{R}^{4096 \times 4096}$ on an A100 (108 SMs).

Tiling. The output matrix C has $4096 \times 4096 = 16.8\text{M}$ elements. A typical output tile size for an A100 GEMM kernel is 128×128 elements (chosen to fit the tensor core's preferred shape and the SM's shared memory). The number of output tiles is:

$$\left\lceil \frac{4096}{128} \right\rceil^2 = 32^2 = 1024 \text{ tiles.}$$

Block assignment. Each tile is one block, so 1024 blocks are launched. With 108 SMs, the scheduler fills all 108 SMs in the first *wave* of 108 blocks. The second wave takes the next 108, and so on:

$$\left\lceil \frac{1024}{108} \right\rceil = 10 \text{ waves.}$$

The 10th wave dispatches only $1024 - 9 \times 108 = 52$ blocks, leaving 56 SMs idle for the duration of that wave. GPU time is approximately $10\times$ the single-wave time rather than $(1024/108) \approx 9.5\times$. This is *wave quantization*: adding even one tile beyond a wave boundary doubles the cost of that marginal tile.

Warp execution within a block. Each block contains 256 threads = 8 warps. The SM schedules these 8 warps in rotation. When warp 0 issues a shared-memory load and stalls for ~ 20 cycles, warps 1-7 are immediately dispatched. By the time the SM cycles back to warp 0, the load has completed. The SM stays busy.

Fix for wave quantization. Round 4096 to $4224 = 33 \times 128$, giving $33^2 = 1089$ tiles. That is still 11 waves with 108 SMs, but the last wave has $1089 - 10 \times 108 = 9$ tiles - even more waste. A better fix: choose a dimension that makes the tile count a multiple of 108, e.g., 4096 with tile size ≈ 86 (non-round) is impractical. In practice, run the problem with a tile count of $1080 = 10 \times 108$ by padding the dimension to $\lceil 4096/\text{tile} \rceil \times \text{tile}$ and choosing the tile dimension to make the product a multiple of 108.

What You Actually Decide in Practice

- **Always make thread block size a multiple of 32.** Any block size that is not a multiple of the warp size (32) wastes the last partial warp's lanes. Standard choices: 128, 256, 512, or 1024 threads per block depending on register pressure.

- **Avoid conditional branches that split warps.** An `if (thread_idx < 16)` construct where both branches do real work halves the throughput of that warp. Restructure using masking (compute both branches, select the result with a conditional move) or ensure all threads in a warp always take the same branch.
- **Target tile counts that are multiples of the SM count.** For an A100 (108 SMs), target grid sizes that are multiples of 108. For an H100 (132 SMs), multiples of 132. If you cannot choose the grid size, at minimum ensure the final wave has high occupancy (close to 108 or 132 blocks).
- **Tensor core input must be a multiple of 16.** A100 tensor cores operate on 16×16 matrix fragments in bfloat16. Dimensions that are not multiples of 16 require padding; dimensions that are multiples of 128 are ideal because they align with the preferred GEMM tile size and the 128-byte L2 cache line. This is the engineering reason behind the well-known speedup from rounding vocabulary sizes to multiples of 64.
- **Maximize occupancy by controlling register pressure.** Each SM on an A100 has 65,536 registers. With 256 threads per block and 128 registers per thread, the block uses $256 \times 128 = 32,768$ registers, leaving room for exactly two blocks per SM (occupancy = $2 \times 256 / 2048 = 25\%$). Reduce registers per thread (by spilling to local memory or restructuring the kernel) to fit more blocks per SM and increase the warp pool for latency hiding.

How This Shows Up in Interviews

- **“What is a warp and why does its size matter?”** A warp is 32 threads that execute the same instruction simultaneously on different data. Its size matters because (1) it is the unit of instruction dispatch, so thread block sizes that are not multiples of 32 waste compute, and (2) branches within a warp cause serial execution (warp divergence), reducing throughput by the fraction of threads on the minority branch.
- **“How does an SM hide memory latency?”** By maintaining many warps in flight simultaneously. When one warp issues a memory read and stalls for 300+ cycles, the SM instantly switches to another ready warp. No context switch overhead because each warp’s register state is always live in the register file. This is why occupancy (ratio of active warps to the SM’s maximum) matters: higher occupancy means more warps to switch to, and the SM stays busy even with long-latency memory operations.
- **Senior follow-up: “An A100 has 108 SMs. Your kernel launches 109 thread blocks. Quantify the performance impact and describe the fix.”** 108 blocks execute in wave 1, using all 108 SMs in parallel. Block 109 executes alone in wave 2, using 1/108 of the SM capacity. Total GPU time $\approx 2 \times$ the single-wave time rather than $109/108 \approx 1.009 \times$. The fix: either reduce the problem to 108 blocks (coarsen the tile size), or pad to 216 blocks (the next multiple of 108) and ensure the added work is trivial (zero-padding the output matrix). In practice this means choosing tile dimensions that make the grid size $\equiv 0 \pmod{108}$, which often requires rounding the problem dimension up by a small amount.

Key Takeaways

- GPU execution hierarchy: Block (assigned to one SM) $\rightarrow$ Warp (32 threads, same instruction, different data) $\rightarrow$ Thread (individual register state and computation lane).
- A100 has 108 SMs, H100 has 132. Each SM can hold up to 64 warps (2048 threads) in flight simultaneously, switching between them with zero overhead to hide memory latency.
- SIMT means all 32 threads in a warp execute the same instruction; any branch that splits a warp serializes execution for the two groups, reducing throughput proportionally.

- Grid sizes (number of blocks) should be multiples of the SM count; a remainder causes the last wave to severely underutilize the GPU - this is wave quantization.
- Tensor core operations require input dimensions to be multiples of 16 (ideally 128) in bfloat16; misaligned dimensions require padding and reduce effective throughput.

8.1.2 HBM, L2, Shared Memory, and Registers

When an interviewer asks why FlashAttention is roughly $3\times$ faster than a naive PyTorch attention implementation, the correct answer is not “it avoids materializing the full attention matrix” - the correct answer is “it moves roughly $7\times$ less data through HBM.” GPU compute throughput has scaled $\sim 10,000\times$ from K20 to H100; HBM bandwidth has scaled $\sim 100\times$. Every generation, the gap widens, and every generation, the dominant bottleneck in LLM kernels is memory movement, not arithmetic. Understanding the memory hierarchy in numbers - latency ratios, bandwidth, capacity at each level - is foundational for any performance conversation at the research engineer or systems engineer level.

A GPU has five distinct levels of memory, organized from fastest and smallest near the compute units to slowest and largest farther away:

Registers (fastest, ~ 1 cycle): per-thread, ~ 255 32-bit registers available per thread on an A100. Holds live scalar values during execution. No programmer overhead to use them - the compiler allocates registers automatically. Spilling to local memory (HBM) when register pressure is too high is a serious performance hazard.

Shared memory / L1 cache (~ 20 cycles, per-SM): configurable up to 164 KB total per SM on an A100, split between shared memory and L1 cache. The programmer explicitly controls shared memory by declaring `__shared__` arrays. Shared by all threads in a block; not visible to other blocks. This is the primary staging area for data that multiple threads will reuse.

L2 cache (~ 200 -300 cycles): 40 MB on the A100, shared across all 108 SMs. Hardware-managed; the programmer cannot directly address it. Acts as a buffer between repeated HBM reads and the SMs. Effective for weight matrices in batched inference where the same weights are reused across many requests.

HBM (High Bandwidth Memory) / global memory (~ 300 -600 cycles latency, 2 TB/s bandwidth on A100, 3.35 TB/s on H100 SXM): 40-80 GB capacity. This is what CUDA programmers call “global memory.” All inter-block communication must go through HBM. This is also where model parameters, activations, and KV caches live.

Host DRAM (CPU side, via PCIe 4.0): ~ 64 GB/s in each direction on a PCIe 4.0 $\times 16$ link - roughly $30\times$ slower than HBM. This level matters for data loading, model transfers, and CPU-GPU coordination, but is too slow to be in the critical path of any compute-intensive kernel.

The critical ratio is between compute throughput and HBM bandwidth. An A100 delivers 312 TFLOP/s of bfloat16 tensor-core throughput and 2 TB/s of HBM bandwidth. A bfloat16 element is 2 bytes, so HBM delivers 10^{12} elements/second. The compute unit can consume $312 \times 10^{12} / 10^{12} = 312$ FLOPs per element delivered. A kernel is compute-bound only if its arithmetic intensity (FLOPs per byte loaded from HBM) exceeds $312/2 = 156$ FLOP/byte. Most element-wise operations (softmax, layer norm, ReLU) have arithmetic intensity below 5 FLOP/byte - they are deeply memory-bound.

HBM uses *burst-mode* access. When a read is issued to any address, the DRAM controller moves a full 128-byte cache line (one burst section) to the L2 or SM. If the thread only needed 2 bytes, the remaining 126 bytes are either cached or discarded. Memory *coalescing* exploits burst mode: when 32 threads in a warp issue

reads to 32 consecutive 2-byte addresses that together span one 64-byte or 128-byte aligned burst section, the hardware services all 32 reads with a single HBM transaction. Non-coalesced access - 32 threads reading 32 scattered addresses - requires up to 32 separate transactions, each fetching 128 bytes and using only 2. Effective bandwidth drops by up to $32\times$.

The performance gap between compute throughput and HBM bandwidth grows with every hardware generation; every optimization strategy for GPU kernels ultimately reduces to minimizing data movement through HBM.

The memory hierarchy is not a design choice - it is physics. Faster memory requires transistors physically closer to the compute units and costs exponentially more die area per bit. Registers and shared memory live on-SM because the SM needs sub-nanosecond access (~ 1 ns at 1 GHz). HBM is a stack of DRAM dies bonded adjacent to the GPU package; it trades latency for large capacity. Host DRAM sits on the CPU board and is an order of magnitude further away.

The strategic consequence is that compute has become so abundant that it is nearly free compared to data movement. Moving a bfloat16 value from HBM to an SM costs roughly $30\times$ more energy per operation than performing a multiply-add on it. Kernels that move data unnecessarily waste almost all of their energy and time waiting, not computing. This is why kernel fusion - combining multiple operations into a single kernel that keeps data in registers and shared memory across all operations - is one of the highest-leverage optimizations available. A fused operation that applies 5 element-wise transformations reads and writes HBM once; the unfused equivalent reads and writes HBM 5 times for the same arithmetic result.

The contrast with the memory-bound profile of CPU workloads is instructive. CPU caches are hardware-managed and opaque to the programmer. On a GPU, shared memory is programmer-managed and explicit - you load a tile, compute over it, and write back. This explicitness is a burden during development but enables the precise control needed to achieve 80-90% of peak HBM bandwidth on well-structured kernels.

A Worked Example

A100 SXM4. Layer normalization over a tensor of shape $(T, D) = (2048, 4096)$ in bfloat16.

Data volume. $2048 \times 4096 \times 2$ bytes = 16.8 MB.

Naive three-pass implementation. Pass 1: read x , compute mean per row (1 HBM read). Pass 2: read x again, compute variance per row (1 HBM read, 1 HBM write of variance). Pass 3: read x , read mean, read variance, compute normalized output, write output (3 HBM reads, 1 HBM write). Total HBM traffic: 5×16.8 MB + 2048×4 bytes for mean/variance ≈ 84 MB.

Fused one-pass implementation. Load each row of x once into shared memory. Compute mean and variance in registers using an online Welford accumulator. Normalize and write the output in the same pass. Total HBM traffic: 1 read + 1 write = 2×16.8 MB = 33.6 MB. Speedup: $84/33.6 \approx 2.5\times$ from eliminating HBM traffic alone, at zero change in arithmetic.

Time at peak HBM bandwidth.

$$t_{\text{naive}} = \frac{84 \times 10^6}{2 \times 10^{12}} \approx 42 \mu\text{s}, \quad t_{\text{fused}} = \frac{33.6 \times 10^6}{2 \times 10^{12}} \approx 17 \mu\text{s}.$$

Coalescing example. Same tensor in row-major layout (T rows, D columns). Kernel assigns one thread per row; a warp of 32 threads processes 32 consecutive rows. At the start of pass 1, warp reads element $[0, 0], [1, 0], \dots, [31, 0]$ (one element per thread from column 0). These 32 addresses are stride- D apart ($D = 4096$ elements = 8192 bytes). Each address falls in a different burst section: 32 separate HBM transactions

required. Effective bandwidth: $32 \times 2 \text{ bytes} / (32 \times 128 \text{ bytes fetched}) = 1/64$ of peak. Fix: tile the reduction so that within each warp, threads access consecutive elements within a row (stride 1), enabling full coalescing.

What You Actually Decide in Practice

- **Configure shared memory vs. L1 split for your kernel's access pattern.** On A100, you can allocate up to 164 KB of shared memory per SM (the rest goes to L1). For kernels that explicitly manage tiling (FlashAttention, fused GEMMs, layer norm), maximize shared memory. Use `cudaFuncSetAttribute` with `cudaFuncAttributeMaxDynamicSharedMemorySize`. For kernels that rely on hardware caching of irregular accesses, bias toward L1.
- **Keep tile sizes within 48-96 KB of shared memory per block.** An A100 SM has 192 KB of on-chip memory. With two concurrent blocks per SM (the common case), each block can use up to 96 KB. With three blocks, the limit is 64 KB. Choose tile dimensions $T_r \times T_c$ such that the shared memory footprint stays below your budget; exceeding it reduces SM occupancy.
- **Align all HBM accesses to 128-byte boundaries.** The L2 cache line is 128 bytes. Accesses that straddle cache line boundaries require two transactions. Pad matrix dimensions to multiples of 64 for bfloat16 (128 bytes = 64×2 -byte elements) or multiples of 32 for float32. Padding a vocabulary size from 50,257 to 50,304 (the nearest multiple of 64) produces a measurable throughput improvement for the token embedding lookup.
- **Profile with memory throughput as the primary metric, not FLOP utilization.** Use `ncu --metrics l1tex__t_bytes.sum,lts__t_bytes.sum,gpu__dram_throughput.avg` to measure L1, L2, and HBM traffic. A kernel at 90% FLOP utilization that is memory-bound is not a compute-bound kernel - it means the memory traffic is the bottleneck and compute is hiding behind it. Reduce HBM traffic first; FLOP utilization is secondary.
- **Exploit L2 reuse for weights in batched inference.** In multi-query or grouped-query inference, the key/value weight matrices are shared across the batch. At batch size 64, a 4096×4096 bfloat16 weight matrix fits in the 40 MB A100 L2 ($4096 \times 4096 \times 2 = 33.6$ MB). Structure the kernel to process all batch elements against one weight slice before moving to the next, so the L2 cache holds the weight during the full batch processing and HBM is only read once per weight matrix.

How This Shows Up in Interviews

- **“What is the difference between shared memory and global memory, and why does it matter?”** Shared memory is on-SM, ~ 20 cycle latency, 96 KB per SM on A100. Global memory (HBM) is off-chip, ~ 300 -600 cycle latency, 80 GB, 2 TB/s bandwidth. The 15-30 $\times$ latency ratio means any kernel that reads the same data from HBM multiple times instead of caching it in shared memory wastes most of its time waiting. The key design pattern: load from HBM once into shared memory, compute over the tile in registers/shared memory, write back once.
- **“What is memory coalescing and when does it fail?”** When 32 threads in a warp read 32 consecutive addresses that together fit in one 128-byte HBM burst section, the hardware issues a single transaction - one read brings all 32 values. It fails when threads access strided or random addresses: each thread's address falls in a different burst section, requiring up to 32 transactions for the same amount of useful data. Column-major access of a row-major matrix is the canonical failure case: threads in a warp read elements with stride D (the row width), touching 32 different cache lines.

- **Senior follow-up: “Compute throughput scales $10,000\times$ from K20 to H100 but HBM bandwidth scales only $\sim 100\times$. What does this imply for algorithm design going forward?”** The arithmetic intensity threshold for being compute-bound (FLOPs per byte) increases by $100\times$ over this span. Operations that were memory-bound on K20 are even more memory-bound on H100. Three concrete implications: (1) kernel fusion has compounding value - each fused operation eliminates a full HBM round-trip at a penalty that grows $100\times$ over this hardware range; (2) quantization (INT8, FP8) reduces bytes moved by $2\text{--}4\times$ and its benefit grows proportionally as the memory wall tightens; (3) any new architecture that requires sequential operations with no fusion opportunity (e.g., state-space models with per-element recurrence) will be increasingly penalized relative to block-structured operations like attention. Designing for memory efficiency is not a current concern - it is an evergreen constraint that only becomes more important.

Key Takeaways

- Memory hierarchy latency ladder: registers (~ 1 cycle) $\rightarrow$ shared memory/L1 (~ 20 cycles, 96 KB per SM) $\rightarrow$ L2 ($\sim 200\text{--}300$ cycles, 40 MB) $\rightarrow$ HBM ($\sim 300\text{--}600$ cycles, 2 TB/s, 40-80 GB) $\rightarrow$ host DRAM (PCIe, ~ 64 GB/s).
- An A100 delivers 312 TFLOP/s bfloat16 compute and 2 TB/s HBM bandwidth; a kernel is compute-bound only when its arithmetic intensity exceeds 156 FLOP/byte. Most LLM non-GEMM operations are at < 5 FLOP/byte - deeply memory-bound.
- Memory coalescing: 32-thread warps reading 128-byte-aligned consecutive addresses get one HBM transaction; strided or random access requires up to 32 transactions for the same data volume.
- Compute scales $\sim 10,000\times$ from K20 to H100; HBM bandwidth scales $\sim 100\times$; the gap grows every generation, making kernel fusion and quantization increasingly high-leverage.
- Shared memory is the programmer-controlled fast cache: load from HBM once, compute in shared memory, write back once. This is the design principle behind FlashAttention and every high-performance transformer operator.

8.2 Arithmetic Intensity and the Roofline Model

8.2.1 Compute-Bound vs. Memory-Bound Regimes

The most common GPU performance question in an LLM systems interview is some variant of “why is autoregressive decoding slow, and what can you do about it?” Candidates who answer purely in terms of algorithmic complexity-“it is $O(n^2)$ attention”-miss the real bottleneck. During generation the constraint is almost never FLOPs; it is memory bandwidth. Internalizing this requires two concepts: *arithmetic intensity* and the *roofline model*. This pair of ideas shows up at every seniority level; senior answers add numbers instead of stopping at the qualitative claim.

Arithmetic intensity is the ratio of floating-point operations performed to bytes transferred between the accelerator’s compute units and its main memory (HBM on a GPU):

$$AI = \frac{\text{FLOPs}}{\text{Bytes transferred from HBM}}$$

The roofline model says a kernel’s achievable throughput in FLOPs/sec is bounded by two separate ceilings. The memory ceiling is bandwidth $\times$ AI: regardless of how many tensor cores you have, you cannot process data

faster than you can deliver it from HBM. The compute ceiling is the chip's peak FLOPs. The actual achievable throughput is:

$$\text{Attainable FLOPs/sec} = \min(\text{peak compute}, \text{bandwidth} \times \text{AI})$$

A kernel is *memory-bound* when $\text{AI} < \text{peak compute}/\text{bandwidth}$ and *compute-bound* when the reverse holds. The crossover is the *ridge point*:

$$\text{ridge point} = \frac{\text{peak compute (FLOPs/s)}}{\text{memory bandwidth (bytes/s)}}$$

On an H100 SXM5 running BF16 tensor-core operations:

- Peak tensor-core throughput: ≈ 989 TFLOPs/s
- HBM3 bandwidth: ≈ 3.35 TB/s
- Ridge point: $989 \times 10^{12} / 3.35 \times 10^{12} \approx 295$ FLOPs/byte

To saturate the H100's compute, a kernel must execute at least 295 floating-point operations per byte fetched from HBM-and most LLM operations during inference do not come close.

GPU generations scale compute throughput far faster than memory bandwidth. From the V100 to the H100, BF16 tensor-core peak grew roughly $10\times$ while HBM bandwidth grew roughly $3\times$. The ridge point climbs with every generation: operations that were barely compute-bound in 2018 are memory-bound today, and the trend continues. Designing an efficient kernel means understanding which side of this ridge your operation sits on, because the optimization lever is completely different in each regime.

Below the ridge point the GPU is starved for data. An SM loads a tile from HBM, executes a handful of arithmetic instructions, and stalls waiting for the next load. Tensor cores sit dark. Adding more FLOPs to the kernel does not help; only raising AI does. The productive levers are: fuse kernels to eliminate intermediate HBM round-trips, lower numerical precision to cut bytes per element, and restructure data access to maximise reuse within SRAM.

Above the ridge point the opposite holds. Instruction throughput is the constraint. The levers are tile-size tuning, register allocation, and ensuring the tensor cores have a continuous stream of work with no pipeline bubbles. Reducing bytes transferred has no effect if compute is the bottleneck.

A Worked Example

Consider two kernels operating on tensors stored in BF16 (2 bytes per element).

ReLU on a vector of n elements:

- Read n elements: $2n$ bytes
- One comparison per element: n FLOPs
- Write n elements: $2n$ bytes
- Total bytes: $4n$, total FLOPs: n
- $\text{AI} = n/(4n) = 0.25$ FLOPs/byte

This is $0.25/295 \approx 0.08\%$ of the H100 ridge point-purely memory-bound at any tensor size.

Batched matrix multiply $\mathbf{X}_{B \times D} \cdot \mathbf{W}_{D \times F}$ in BF16:

- Reads: $2BD + 2DF$ bytes; writes: $2BF$ bytes
- FLOPs: $2BDF$ (multiply-accumulate)
- $\text{AI} \approx 2BDF/(2BD + 2DF + 2BF) \approx B$ (when $B \ll D, F$)

For a projection layer in a 7B model with $D = 4096$, $F = 11008$:

- $B = 1$ (single decode step): $AI \approx 1$ FLOPs/byte. The H100 is working at $1/295 < 0.5\%$ of its compute capacity; throughput is entirely determined by the 3.35 TB/s HBM bandwidth.
- $B = 295$ (medium batch): $AI \approx 295$ FLOPs/byte. At the ridge point; switching from memory-bound to compute-bound.
- $B = 512$: $AI \approx 440$ FLOPs/byte. Compute-bound; tensor cores are fully utilized.

This single calculation explains why serving throughput scales almost linearly with batch size up to around $B = 300$, then saturates: below $B = 295$ you are paying bandwidth; above it you are paying compute.

What You Actually Decide in Practice

- **Identify your regime before optimizing.** Run a profiler and check whether the kernel is bandwidth-limited or compute-limited. Spending hours tuning tile sizes on a memory-bound softmax yields nothing; spending hours fusing element-wise ops on a compute-bound matmul is equally wasted.
- **Rule of thumb for inference:** if $B \times T < 300$ on an H100, assume your linear layers are memory-bound. Budget latency as bytes-transferred / bandwidth, not FLOPs / peak-compute.
- **Halving precision doubles AI.** Going from FP32 to BF16 cuts bytes per element from 4 to 2, doubling AI for any memory-bound kernel. This is one of the highest-leverage single changes for inference performance and costs almost nothing in quality for the weight and activation tensors that dominate bandwidth.
- **Every HBM round-trip is expensive.** An elementwise activation written to HBM and read back for the next layer pays the full 295 FLOPs/byte tax twice for zero additional compute. Fuse it. PyTorch's `torch.compile` and custom Triton kernels both eliminate this by keeping intermediate results in registers or shared memory.
- **Attention during decode is the worst-case scenario.** For a single decode step, the attention kernel must load the entire KV cache ($2 \times L \times T_{\text{past}} \times N_{\text{kv}} \times H$ bytes) from HBM and do $O(T_{\text{past}})$ dot products per head. The AI for this is essentially 1 regardless of batch size-batch size does not help because each sequence has its own KV cache. This is the structural reason GQA, MLA, and local attention reduce decode latency: they shrink the bytes, not the FLOPs.

How This Shows Up in Interviews

- “Why is autoregressive decoding slow compared to prefill?” Strong answer: prefill processes all tokens in parallel; $AI = O(T)$ for attention and $\approx B$ for projections, so it is compute-bound at typical sequence lengths. Decode generates one token at a time; $AI \approx 1$ for projections and $AI \approx 1$ for attention regardless of sequence length. Throughput is capped by HBM bandwidth, not tensor cores.
- “If the next GPU generation doubles peak FLOPs but keeps memory bandwidth the same, does token generation speed improve?” Strong answer: no. Decode is already memory-bound; the ridge point doubles (from 295 to 590 FLOPs/byte), making the regime gap even worse. The only improvement comes from raising AI-larger batch sizes, fewer KV-cache bytes per token (GQA, quantization), or architectures with constant-size state.
- *Senior follow-up:* “How would you use the roofline model to evaluate a proposed architectural change-say, replacing the FFN with a depth-wise convolution?” Sketch the answer: estimate AI for the proposed op (depth-wise conv has $AI \approx \text{kernel_size}/2$ at best, typically $\ll 295$), compare to the matmul it replaces ($AI \approx B$, compute-bound at $B > 300$). If the replacement is memory-bound where the original was compute-bound, you likely made the architecture slower despite reducing FLOPs-a classic efficiency trap

that roofline analysis catches immediately.

Key Takeaways

- AI = FLOPs/bytes; ridge point = peak compute/peak bandwidth ≈ 295 FLOPs/byte on H100 BF16.
- Memory-bound kernels benefit from fusion, lower precision, and reduced data movement; compute-bound kernels benefit from better tiling and occupancy.
- Matrix multiply AI $\approx B$ (batch size); single-stream decode ($B = 1$) has AI ≈ 1 , using less than 0.5% of the GPU's compute capacity.
- The ridge point rises with every GPU generation because compute scales faster than bandwidth-memory-bound problems stay memory-bound and architectural changes are the only durable fix.
- Attention during decode has AI ≈ 1 independent of batch size, making KV-cache size reduction (GQA, MLA, quantization) the primary lever for decode throughput.

8.2.2 Why Matrix Multiplies Are “Blessed” Operations

When an interviewer asks why every production LLM architecture concentrates its computation in matrix multiplications, the answer is not “GPUs are fast at matmuls.” The precise answer is that matrix multiplies are the only common deep-learning operation whose arithmetic intensity grows with problem size. Every other operation you might add-elementwise activations, layer norms, softmax, attention masking-has AI that is $O(1)$ or $O(1/N)$: it is memory-bound at any scale and stays memory-bound no matter how large you make the tensor. Matmuls have AI = $O(N)$. This geometric fact, combined with dedicated hardware that delivers 10-14 $\times$ the FLOPs/s of general ALUs, means non-matmul FLOPs are not just slower-they are categorically slower. This topic tests at the senior level whenever architecture or kernel design is on the table.

For a square $N \times N$ matrix multiply $\mathbf{C} = \mathbf{AB}$ in BF16:

- FLOPs: $2N^3$ (one multiply plus one add per output element, N^2 outputs each needing a dot product of length N)
- Bytes transferred: $3 \times 2N^2 = 6N^2$ (read $\mathbf{A}$, $\mathbf{B}$, write $\mathbf{C}$)
- AI = $2N^3/(6N^2) = N/3$

At $N = 885$ the AI crosses the H100 ridge point of ≈ 295 FLOPs/byte. At $N = 4096$ (a typical model width), AI ≈ 1365 -more than 4 $\times$ above the ridge, solidly compute-bound.

The contrast with elementwise operations is stark. A ReLU on an $N \times N$ tensor has N^2 FLOPs and $4N^2$ bytes (AI = 0.25) regardless of N . Softmax on an N -element vector has $\approx 3N$ FLOPs and $4N$ bytes (AI ≈ 0.75). These operations will never be compute-bound on any conceivable GPU because their FLOPs and bytes scale at the same rate.

Modern GPUs reinforce this asymmetry with hardware specialization. On an H100, general-purpose FP32 ALUs deliver ≈ 67 TFLOPs/s. BF16 tensor-core throughput is ≈ 989 TFLOPs/s-a 14.8 $\times$ gap. A kernel that cannot use tensor cores (because it performs something other than a matrix multiply-accumulate) runs at $\leq 1/14$ the FLOPs/s of one that can.

Matrix multiplies are “blessed” because they are the only common DL operation where making the problem larger raises the GPU’s efficiency, and the only one with dedicated hardware that delivers 14 $\times$ higher FLOPs/s than every other operation class.

The geometric intuition is data reuse. An $N \times N$ matmul has $O(N^2)$ input data but $O(N^3)$ work. If you load a tile of $\mathbf{A}$ and a tile of $\mathbf{B}$ into fast shared memory, you can perform all $O(N)$ dot products that involve

that tile before evicting it-reusing each loaded byte $O(N)$ times. The number of HBM loads grows as $O(N^2)$ while the FLOPs grow as $O(N^3)$, so the ratio $AI = O(N)$.

No other operation has this structure. Layer normalization computes a mean and variance over each row- $O(N)$ work on $O(N)$ data, $AI = O(1)$. Attention score accumulation (the softmax and weighted sum) has $O(T^2)$ FLOPs and $O(T^2)$ bytes for sequence length T , so $AI = O(1)$. Depth-wise convolution with kernel size k achieves $AI \approx k/2$ -bounded by a small constant.

The implication for architecture design is direct: if you add a non-matmul component that processes the same activations as a matmul, you are adding a memory-bound bottleneck alongside a compute-bound one. The new component runs at $1/14$ the throughput per FLOPs, and if it touches the same bytes as the matmul it “replaces,” you have paid the bandwidth cost twice.

The performance gap between matmul-dominant and non-matmul-dominant architectures is empirically large. The same compute budget spent on a matmul-based FFN versus a recurrent or convolutional alternative generally yields worse wall-clock performance for the alternative, even when the alternative has fewer theoretical FLOPs-because those FLOPs are running at $1/14$ the hardware speed.

A Worked Example

Consider the gate projection in a SwiGLU FFN layer of a 7 B model: $D = 4096$, $F = 11008$, BF16 storage.

Weight matrix $\mathbf{W} \in \mathbb{R}^{4096 \times 11008}$: $4096 \times 11008 \times 2 \text{ bytes} = 90.2 \text{ MB}$.

$B = 1$ (single decode token):

- FLOPs: $2 \times 1 \times 4096 \times 11008 = 90.2 \text{ GFLOPs}$
- Bytes: 90.2 MB (weights) $+ 2 \times 4096$ (input) $+ 2 \times 11008$ (output) $\approx 90.2 \text{ MB}$
- $AI \approx 90.2 \text{ GFLOPs} / 90.2 \text{ MB} = 1 \text{ FLOPs/byte}$
- Throughput: $3.35 \text{ TB/s} \times 1 = 3.35 \text{ TFLOPs/s}$ attainable vs. 989 TFLOPs/s peak-0.34% utilization

$B = 512$ (batched inference):

- FLOPs: $512 \times 90.2 \text{ GFLOPs} = 46.2 \text{ TFLOPs}$
- Bytes: 90.2 MB (weights, loaded once and reused for all 512 tokens) $+ 2 \times 512 \times 4096 \approx 4.2 \text{ MB}$ (inputs) $+ \approx 11.3 \text{ MB}$ (outputs) $\approx 105.7 \text{ MB}$
- $AI = 46200 \text{ GFLOPs} / 105.7 \text{ MB} \approx 437 \text{ FLOPs/byte}$
- Attainable: $\min(3.35 \times 437, 989) = \min(1464, 989) = 989 \text{ TFLOPs/s}$ -fully compute-bound

The $512\times$ increase in batch size did not increase the weight bytes at all-the same 90.2 MB is broadcast to all 512 tokens. That data reuse is precisely why batching improves hardware utilization for linear layers and why serving systems target high request concurrency.

Now compare to a standalone SwiGLU activation applied to $B = 512$, $F = 11008$ elements:

- FLOPs: $\approx 5 \times 512 \times 11008 \approx 28.2 \text{ GFLOPs}$ (multiply, sigmoid, gate product, etc.)
- Bytes: $3 \times 2 \times 512 \times 11008 \approx 33.9 \text{ MB}$ (read two inputs, write one output)
- $AI \approx 28200 / 33900 \approx 0.83 \text{ FLOPs/byte}$ -memory-bound at any batch size

This activation runs in $33.9 \text{ MB} / 3.35 \text{ TB/s} \approx 10 \mu\text{s}$ when fused into the projection kernel. As a standalone kernel it incurs an additional HBM round-trip and takes twice as long for the same bytes. Fusing it is not optional at production scale.

What You Actually Decide in Practice

- **Target > 80% of FLOPs in matmuls.** If your architecture allocates more than $\sim 20\%$ of FLOPs to non-matmul operations, profile immediately. Those operations run at $\leq 1/14$ the hardware speed and will dominate latency despite their minority share of theoretical FLOPs.
- **Use dimensions that are multiples of 128 (minimum 64).** Matmul tile sizes on H100 are typically 128×128 or 256×128 . A matrix dimension of 4097 requires the same number of tiles as 4224 but leaves the edge tile almost empty. Padding the vocabulary from 50,257 to 50,304 (nearest multiple of 64) is documented to yield a 25% throughput gain-the tile alignment effect is not academic.
- **$B = 295$ is the practical break-even for linear layers on H100 BF16.** Below that, optimize for bandwidth (GQA, quantization, fewer HBM reads). Above that, optimize for compute (better tile sizes, higher occupancy). Do not conflate the two regimes.
- **Fuse non-matmul work into adjacent matmul kernels.** Activations, scales, biases, and residual adds should never be standalone kernels in a throughput-sensitive path. `torch.compile` handles this automatically for simple cases; Triton is the right tool when you need custom fusion boundaries.
- **FlashAttention improves attention throughput by reducing bytes, not FLOPs.** The attention score matrix ($T \times T$ in FP32) is never materialized in HBM; instead the softmax is computed tile-by-tile in SRAM. Attention FLOPs are $O(T^2)$ in both cases-FlashAttention moves attention from bandwidth-limited to closer to compute-limited by eliminating most HBM traffic.

How This Shows Up in Interviews

- “Why should most FLOPs in an LLM be in matrix multiplications?” Strong answer: matmul AI = $N/3$ for square $N \times N$, growing linearly with size and crossing the H100 ridge point at $N \approx 885$. All other common operations have constant AI regardless of size, so they are permanently memory-bound. Additionally, tensor cores deliver $14\times$ more FLOPs/s than general ALUs-non-matmul operations cannot access this hardware.
- “An engineer proposes replacing one FFN layer with a bank of depth-wise convolutions to reduce FLOPs. What’s your concern?” Strong answer: depth-wise conv has AI $\approx k/2$ where k is the kernel size (typically 3 or 5), which is well below the ridge point. The replacement is memory-bound while the matmul it replaces was compute-bound at batch size > 300 . Even if it has half the FLOPs, it may well be slower, because those FLOPs run at $1/14$ the speed. Profile first; the roofline model predicts the answer before running any code.
- Senior follow-up: “Why does nanoGPT’s throughput increase by 25% after increasing vocab size from 50,257 to 50,304?” Sketch the answer: the vocabulary embedding lookup and the final unembedding projection are both matmuls with one dimension equal to the vocab size. When that dimension is not a multiple of the tile size (64 or 128), the last tile is sparsely populated, meaning the assigned SM executes a full tile’s worth of instructions for a fraction of useful work. Padding to the nearest multiple of 64 eliminates this waste. At the scale of an unembedding matmul ($D_{\text{model}} \times V$), even a 0.1% alignment improvement propagates into the entire forward and backward pass because this is the largest single matmul in the model.

Key Takeaways

- Matmul $AI = N/3$ for $N \times N$ matrices, scaling linearly with N ; all other common LLM operations have $AI = O(1)$ and remain memory-bound at any size.
- Tensor cores deliver $\approx 14\times$ more FLOPs/s than general ALUs on H100; operations that cannot use tensor cores pay this penalty unconditionally.
- $B \approx 295$ is the batch-size break-even for single linear layers on H100 BF16; below it, optimize memory movement; above it, optimize compute utilization.
- Keep matmul dimensions at multiples of 128 (minimum 64) to avoid tile-boundary underutilization; misalignment alone can cost 25%+ of throughput.
- FlashAttention is a memory-movement optimization, not a FLOPs reduction; it moves attention toward the ridge point by eliminating HBM traffic, not by changing the operation count.

Chapter 9 Making a Single GPU Go Fast

9.1 Reducing Memory Movement

9.1.1 Operator Fusion and Kernel Launch Overhead

Nearly every time I have profiled an LLM training run and found an unexpected bottleneck, it was not in the matmuls but in the surrounding glue: activations, layer norms, residual adds, and masking operations, all running as separate CUDA kernels and each one paying a full round-trip to HBM. The fix is kernel fusion: combining sequential operations into a single kernel so intermediate results never leave the compute unit. This concept appears at the mid level as “what is operator fusion,” and at the senior level as “design a fast custom kernel for this attention variant” or “why is this activation function showing up in your profile?”

Think of the GPU as a factory on an island. HBM is the warehouse on the mainland. Every time a PyTorch operation produces an intermediate result in eager mode, the result gets shipped back to the warehouse, and the next operation has to ship it back over again. If five operations run sequentially without fusion, you have paid five round-trips for data that could have stayed in the factory the whole time.

Kernel fusion eliminates those round-trips. A fused kernel reads the input from HBM once, performs all chained operations entirely in registers and shared memory, then writes the final output to HBM once. For memory-bound operations—those with arithmetic intensity (AI) well below the ridge point—this directly reduces runtime. The HBM bandwidth, not compute throughput, is the bottleneck, so cutting the number of passes through HBM cuts the wall-clock time proportionally.

There is a second, smaller benefit: kernel launch overhead. Each CUDA kernel launch incurs a fixed dispatch cost: the CPU traverses the `aten C` interface, the CUDA runtime queues the kernel, and the GPU drains its pipeline before starting. This overhead is typically 5-20 μs per launch regardless of tensor size. For small tensors or simple operations, this fixed cost dominates actual compute time. Fusing N kernels into one eliminates $N - 1$ launch overheads.

Kernel fusion cuts HBM traffic by keeping intermediate results in registers across the fused operations, and removes per-kernel launch overhead as a secondary benefit.

The memory hierarchy makes the arithmetic unavoidable. Registers are zero-latency for the thread that owns them; L1 and shared memory have about 20 clock cycles latency within an SM; HBM has about 300 clock cycles. An unfused chain of elementwise operations reads from and writes to HBM between every step, paying 300 cycles of stall time for each intermediate. A fused kernel holds every intermediate value in registers and only touches HBM at the start and end.

The alternative—letting each intermediate write back to HBM—has one apparent advantage: simplicity. PyTorch eager mode does exactly this, and it is fine for large matmuls, which are compute-bound and spend little time on relative memory traffic. But activation functions, residual additions, and normalization layers all have AI well below 1 FLOPs/byte. On those operations, every extra HBM round-trip is paid at the full bandwidth cost for zero additional arithmetic.

Modern JIT compilers automate straightforward fusion. `torch.compile` identifies sequences of elementwise operations with no cross-thread dependencies, emits a single Triton kernel, and eliminates intermediate allocations. This is production-grade fusion at essentially zero engineering effort.

A Worked Example

The approximate GELU is:

$$\text{GELU}(x) \approx 0.5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} (x + 0.044715 x^3)))$$

Written naively in PyTorch as separate tensor expressions, this decomposes into at least five distinct CUDA kernels: compute x^3 ; multiply by 0.044715; add x ; compute $\tanh$; combine with $0.5 \cdot x$. Consider the FFN hidden states of a 7 B model at batch size 512 and hidden dimension 11 008, stored in BF16. Each intermediate tensor is $512 \times 11,008 \times 2 = 11.3$ MB. Running five kernels means at minimum $5 \times 2 \times 11.3 = 113$ MB of extra HBM traffic—one read plus one write per intermediate—on top of the 22.6 MB actually needed.

Empirically, on an A100 at this scale:

- Naive PyTorch (five separate kernels): 8.1 ms
- PyTorch built-in `nn.functional.gelu` (fused CUDA kernel): 1.1 ms
- Hand-written CUDA kernel (all ops in one kernel): 1.8 ms
- Triton kernel (block-level, vectorized): 1.85 ms
- `torch.compile` applied to the naive code: 1.47 ms

The built-in fused kernel is 7.4× faster than the naive version. `torch.compile` on the naive expression reaches 1.47 ms with no kernel code written, capturing 78% of the maximum speedup. The hand-written CUDA and Triton versions are slightly slower than the cuDNN-tuned built-in because CUDA library implementations carry additional hardware-specific tuning.

What You Actually Decide in Practice

- **Reach for `torch.compile` first.** For standard elementwise chains-activations, bias adds, scaling, residuals-`torch.compile` will auto-fuse them and close most of the gap. Confirm it is actually fusing by checking the profiler for a single kernel launch rather than multiple `vectorized_elementwise_kernel` calls. Only move to hand-written Triton when the compiler misses a fusion you expect.
- **Fuse activations into adjacent matmul epilogues.** The output of a tiled matmul is already in SRAM; applying a bias add or activation in the same kernel avoids writing the pre-activation to HBM. Cutlass and cuBLAS wrappers expose epilogue fusion for this purpose. In Triton, you can add these steps directly at the end of the matmul loop.
- **Kernel launch overhead dominates at small batch sizes.** At the batch sizes used in speculative decoding (1-4 sequences), a chain of ten elementwise ops can spend more time in kernel launch overhead than in actual compute. At these scales, fusing into one kernel is especially important.
- **Do not fuse across operations that change memory layout.** If fusion forces the matmul output into a non-power-of-2 leading dimension, tile alignment breaks and the resulting performance loss can exceed the fusion gain. Fuse activations; be cautious with dimension-reshaping ops that affect downstream matmul tile selection.
- **Always profile before and after.** The profiler trace for the unfused GELU shows five separate `vectorized_elementwise_kernel` launches. For the fused version you see exactly one. If you are not checking the trace, you do not know whether the compiler has fused your ops or left them separate.

How This Shows Up in Interviews

- “We have a custom activation function written as a chain of PyTorch ops. It’s showing up in the profiler. What do you do?” Strong answer: confirm in the trace that the activation is running as multiple separate kernels. Apply `torch.compile` first-it auto-fuses simple chains into a single Triton kernel. If the op requires custom shared memory layout or mask logic, write a Triton kernel. Profile after each change to confirm the launch count dropped.
- “Why does the unfused GELU run at 8ms while the built-in runs at 1.1ms on the same hardware?” Strong answer: the unfused version has five HBM round-trips for intermediate results, each paying full bandwidth cost for zero additional compute. Fusion keeps all intermediates in registers, reducing HBM traffic from roughly ten passes (five reads, five writes) to two (one read, one write).
- Senior follow-up: “Under what conditions does kernel fusion not help, or even hurt?” Senior answer: if the bottleneck is compute-bound (AI above the ridge point), fusing memory-bound ops around it does not change the binding constraint-you are waiting on compute, not bandwidth, so fewer HBM accesses save nothing. Fusion can also hurt if it increases register pressure within a warp to the point of register spilling to local memory, adding latency that was not present before. Always validate with a profiler; a rule-of-thumb answer about AI first, then empirical confirmation.

Key Takeaways

- Each unfused HBM round-trip costs ~ 300 clock cycles per element; fusion collapses N elementwise ops to 1 read + 1 write regardless of N .
- On an A100, fusing a five-op GELU chain yields a $7.4\times$ speedup; `torch.compile` captures roughly 80% of that gain automatically.
- Kernel launch overhead ($\sim 5\text{--}20\ \mu\text{s}$ per launch) is a secondary effect that matters most at small batch sizes.
- `torch.compile` is the default tool for fusion; Triton is warranted when the compiler does not auto-fuse or when the op requires custom shared-memory logic.
- Profile the CUDA kernel trace-not just wall-clock time-to confirm fusion is actually happening.

9.1.2 Recomputation vs. Storage Trade-Offs

Activation checkpointing is well known as a memory-saving technique: you discard most forward-pass activations and recompute them during the backward pass to stay within a GPU’s memory budget. What is less well known, and what comes up frequently in senior kernel-design interviews, is that recomputation can also make execution *faster*-not just cheaper in memory. The logic is a direct consequence of the roofline model: if a kernel is memory-bound, you have idle compute cycles burning while the kernel waits for HBM. Spending those idle cycles to recompute intermediate values instead of loading them from HBM is free in terms of wall-clock time, and it removes the HBM read that was the bottleneck. FlashAttention’s backward pass is the canonical production example of this technique applied at scale.

The standard backward pass stores every forward-pass intermediate in HBM so gradient computation can read them back. For an operation $f : x \mapsto y$ with intermediates $s_1, \dots, s_k$, this means:

- **Standard:** write $s_1, \dots, s_k$ to HBM during forward; read them back during backward.
- **Recompute:** discard $s_1, \dots, s_k$ after forward; recompute them from x during backward.

The recompute approach eliminates k HBM reads at the cost of k extra arithmetic passes. That exchange

is profitable whenever the kernel is memory-bound, because the compute units were already idle waiting for HBM-the recomputed arithmetic consumes cycles that were wasted otherwise.

Concretely, the decision boundary is the roofline ridge point (approximately 295 FLOPs/byte on H100 BF16). Below that arithmetic intensity, compute is excess capacity. Above it, you are compute-bound, and adding recomputation extends the runtime by consuming cycles that were not free.

The same principle, applied at large scale, underlies FlashAttention. The attention score matrix $S = QK^T/\sqrt{d}$ has shape $T \times T$ in FP32. Storing and reloading it during the backward pass costs $O(T^2)$ HBM traffic-quadratic in sequence length. FlashAttention instead recomputes S tile by tile during the backward pass using Q and K that are already being read for gradient computation, keeping the $O(T^2)$ materialization entirely out of HBM.

Recomputation turns excess compute into saved HBM bandwidth: for memory-bound kernels, arithmetic that replaces an HBM read is free, because those compute cycles were idle anyway.

The roofline model quantifies the exchange rate. On an H100 at BF16, for every 295 FLOPs you can compute, you are entitled to transfer 1 byte from HBM. Below the ridge point, the FLOPs side of the ratio is underutilized: you are getting fewer than 295 FLOPs per byte because compute finishes faster than the memory can deliver the next byte.

When you recompute a sigmoid or a running maximum rather than loading it from HBM, you are spending FLOPs from the underutilized side to avoid spending bytes from the overutilized side. The exchange is one-sided in your favor for any memory-bound kernel. The more memory-bound the kernel-the lower its AI-the more recomputation you can afford before adding any latency.

This distinguishes recomputation-for-speed from the more familiar gradient checkpointing. Checkpointing is applied to compute-bound operations (matmuls, full attention blocks) to reclaim peak memory at the cost of paying the compute again. That cost is real because those operations are not memory-bound. Recomputation-for-speed is applied to memory-bound operations where the compute cost is zero against the roofline.

A Worked Example

Consider three stacked sigmoid activations: $s_1 = \sigma(x)$, $s_2 = \sigma(s_1)$, $\text{out} = \sigma(s_2)$. All values are BF16 (2 bytes per element). Count the HBM operations in each approach.

Standard approach (store intermediates):

- Forward: read x (1 read), compute and write s_1 , s_2 , out (3 writes).
- Backward: read s_1 , s_2 , d_{out} (3 reads), write dx (1 write).
- Total: **8 HBM operations.**

Recomputation approach (discard intermediates):

- Forward: read x (1 read), write out only-discard s_1 , s_2 (1 write).
- Backward: read d_{out} and x (2 reads), recompute s_1 , s_2 , out from x entirely in SM registers (0 HBM), write dx (1 write).
- Total: **5 HBM operations.**

That is $5/8 = 62.5\%$ of the original HBM traffic. The price is three sigmoid evaluations during the backward pass-arithmetic that is free on a memory-bound kernel. For a hidden dimension of 11 008 at batch 512 in BF16 (22.6 MB per tensor), the three saved reads total $3 \times 22.6 = 67.8$ MB per backward call through this layer. At H100 HBM3 bandwidth of 3.35 TB/s, that is roughly $20 \mu\text{s}$ of HBM stall time eliminated per backward call.

The same accounting applied to the FlashAttention backward pass: the $T \times T$ attention matrix in FP32 at

$T = 4096$ and batch 1 is $4096^2 \times 4 = 64$ MB. Without recomputation, loading this matrix takes $64/3350 \approx 19$ ms per backward attention call. FlashAttention recomputes it for the cost of a second pass over the Q and K tiles—which are already being read for the gradient computation—making the recomputed attention effectively free.

What You Actually Decide in Practice

- **Recomputation for speed requires confirming memory-bound regime first.** Profile the kernel with Nsight Systems or `torch.profiler`. If the kernel’s GPU time is dominated by memory stalls (visible as low SM utilization and high HBM read bandwidth), it is a candidate. If SM utilization is near 100% and CUDA cores are the bottleneck, adding recomputation will extend the runtime linearly.
- **Apply to any activation with $AI < 1$ FLOPs/byte.** Sigmoid, ReLU, GELU, SiLU, layer norm statistics (mean and variance), and softmax exponentials all fall into this category. Dropping their intermediate values and recomputing during backward is almost always a net win on H100 or A100 hardware. This is the kernel-level form of the technique; it is separate from the layer-level gradient checkpointing decision.
- **Do not conflate with gradient checkpointing.** Checkpointing (e.g., `torch.utils.checkpoint`) is applied at the boundary of compute-bound layers to reduce peak memory at a throughput cost. Recomputation-for-speed is applied inside memory-bound kernels at no throughput cost. The two levers operate at different granularities and are both usable simultaneously.
- **Fuse the recomputed path.** Recomputing s_1, s_2 , out during backward only avoids HBM traffic if those recomputations happen inside the same kernel as the gradient accumulation. Launching separate kernels for the recomputed intermediates adds HBM traffic back and defeats the purpose entirely. Recomputation and fusion must be applied together.
- **The break-even point depends on the recompute cost.** For cheap operations (sigmoid, ReLU), a single sigmoid is ~ 3 FLOPs and costs nothing against the 295 FLOPs/byte ridge. For expensive operations (full softmax over a long sequence, layer norm with large hidden dimension), the recompute cost can itself push the kernel toward compute-bound. Measure before assuming the trade is free.

How This Shows Up in Interviews

- *“FlashAttention doesn’t store the full attention weight matrix. How does the backward pass work?”* Strong answer: the backward pass recomputes the attention scores $S = QK^T/\sqrt{d}$ tile by tile in SRAM using stored Q, K , the output O , and gradient dO . The attention score matrix has $AI \approx 1$ and is therefore memory-bound; recomputing it costs FLOPs from the excess supply and eliminates the $O(T^2)$ HBM reads that would otherwise dominate backward-pass time and memory.
- *“When should you use activation checkpointing vs. activation recomputation?”* Strong answer: checkpointing applies at the layer boundary (matmuls, full transformer blocks) to reduce peak memory when the model does not fit in GPU RAM; the runtime cost is real because those layers are compute-bound. Activation recomputation applies inside memory-bound kernels (elementwise activations, normalization intermediates) to speed up the backward pass; the runtime cost is zero because compute was idle. Confirm the target kernel is memory-bound before applying recomputation for speed.
- *Senior follow-up: “Describe a case where recomputation-for-speed makes the kernel slower.”* Senior answer: if the operation is compute-bound- AI above the ridge point—the compute units are fully saturated. Recomputing any intermediate requires FLOPs that must wait in the compute pipeline, directly extending

latency. A concrete example: applying activation recomputation inside a large tiled matmul ($AI \gg 295$) adds FLOPs to an already saturated pipeline and increases kernel time in proportion. The check is always: measure AI first, then decide whether recompute FLOPs are free or costly.

Key Takeaways

- Recomputation for speed trades excess compute for scarce HBM bandwidth; for memory-bound kernels (AI below the roofline ridge), the arithmetic is free because compute units were already idle.
- The triple-sigmoid example reduces HBM ops from 8 to 5 (a 37.5% reduction) by recomputing intermediates in SM registers during the backward pass from the already-loaded input.
- FlashAttention’s backward pass applies this pattern to eliminate $O(T^2)$ HBM traffic; the $T \times T$ attention matrix is recomputed tile by tile at the cost of a second pass over Q and K tiles already in SRAM.
- Recomputation-for-speed and gradient checkpointing operate at different granularities and serve different goals; using both simultaneously is common and correct.
- Recomputation only works if the recomputed path is fused into the backward kernel; a separate kernel launch for recomputed intermediates adds HBM traffic back, negating the benefit.

9.2 Exploiting Memory Layout

9.2.1 Memory Coalescing and Burst Sections

Implement a naive matrix transpose in CUDA and watch effective HBM bandwidth drop to 1-2% of peak. The computation is trivial - one read, one write per element - but the access pattern is column-major on a row-major matrix, and that single design choice costs two orders of magnitude of bandwidth. Interviewers at GPU-focused companies will ask why a “simple” kernel runs at a fraction of peak bandwidth, and the answer lives entirely in how your warp’s 32 threads hit DRAM. Memory coalescing is the most impactful low-level GPU optimization, and understanding it means understanding burst-mode DRAM at the hardware level. This topic appears at the systems engineer and ML infrastructure engineer level.

When a warp of 32 threads issues memory reads, the hardware does not process them sequentially one by one. DRAM is physically organized into *burst sections*: contiguous address blocks, typically 128 bytes, that the DRAM array delivers together in a single sensing operation. The physics: accessing DRAM requires activating a row of memory cells (a sense amplifier step), which is the expensive operation. Once a row is open, reading any subset of that row’s bits is nearly free. The hardware exposes this as: request any single address, receive the full 128-byte cache line.

Memory coalescing exploits this directly. When 32 threads in a warp all read addresses that fall within a small number of burst sections, the memory controller merges them into that many transactions rather than 32 separate ones:

- **Fully coalesced:** Thread k reads address $\text{base} + k \times 2$ (consecutive bfloat16 elements from a 128-byte-aligned base). All 32 reads map to one 64-byte burst section. One HBM transaction returns 64 bytes; all 64 useful bytes are consumed. Utilization: 100%.
- **Fully non-coalesced:** Thread k reads address $\text{base} + k \times S$ where S is the row stride in bytes (e.g., $4096 \times 2 = 8192$ bytes for a 4096-wide matrix). Each thread’s address is 8192 bytes from the next: 128 separate burst sections. Hardware issues 32 transactions. Bytes fetched: $32 \times 128 = 4096$ bytes. Useful bytes: $32 \times 2 = 64$ bytes. Utilization: 1.6%.

The utilization ratio equals $\text{useful_bytes} / \text{fetched_bytes}$. For stride- S access, $\text{utilization} = 2 / \min(128, S)$ for bfloat16. At stride = 4096, that is $2/128 \approx 1.6\%$ - a $64\times$ waste.

Coalesced access is not a software trick: it exploits a fixed DRAM hardware property (burst mode) that already exists; non-coalesced access wastes the free bytes that burst mode delivers, cutting effective bandwidth by up to the number of warp threads.

The reason burst mode exists is that the dominant cost in DRAM access is activating a row of cells (the sense amplifier step), not transferring the bits. Once the row is energized, reading 128 bytes costs almost the same as reading 1 byte. The hardware exposes this by delivering a full cache line on every access - whether you asked for 1 byte or 128.

Coalescing turns this into a benefit: 32 threads' reads for 64 bytes of useful data require one row activation instead of 32. Non-coalescing turns it into a penalty: 32 row activations for 64 bytes of useful data, fetching 4 KB and discarding 63 of every 64 bytes fetched.

CPUs partially compensate for non-coalesced access via hardware stride prefetchers that detect strided patterns and prefetch cache lines proactively. GPUs have no equivalent adaptive prefetcher. The programmer must arrange accesses correctly, or the performance cliff materializes silently. Profiling tools like Nsight Compute expose the coalescing ratio via `l1tex__t_sectors / l1tex__t_requests`; a ratio near 32 means fully non-coalesced access.

A Worked Example

$A \in \mathbb{R}^{4096 \times 4096}$ in bfloat16, stored row-major. One warp (32 threads) loads one row's first 32 elements vs. one column's first 32 elements.

Row access (coalesced). Thread k reads $A[0, k]$. Address: $\text{base} + k \times 2$ bytes.

$$\text{Address span} = 32 \times 2 = 64 \text{ bytes} = \text{one HBM cache line.}$$

Transactions: 1. Bytes fetched: 128 (one cache line). Useful: 64. Utilization: 50% (slightly under if stride $\neq$ cache line size; exactly 1 transaction in practice with 128-byte alignment).

Column access (non-coalesced). Thread k reads $A[k, 0]$. Address: $\text{base} + k \times 4096 \times 2 = \text{base} + k \times 8192$ bytes.

$$\text{Distance between consecutive threads} = 8192 \text{ bytes} = 64 \times (\text{cache line size}).$$

Each thread touches a different cache line 64 lines apart. Transactions: 32. Bytes fetched: $32 \times 128 = 4096$. Useful: $32 \times 2 = 64$. Utilization: $64/4096 = 1.56\%$.

At 2 TB/s HBM bandwidth on an A100:

$$\text{Effective row bandwidth} \approx 2 \text{ TB/s} \times 0.5 = 1 \text{ TB/s (full-rate coalesced)}$$

$$\text{Effective column bandwidth} \approx 2 \text{ TB/s} \times 0.0156 \approx 31 \text{ GB/s.}$$

A kernel doing nothing but column reads of this matrix would run at 31 GB/s instead of 2 TB/s - a $64\times$ degradation. For a bfloat16 element-wise operation with 1 FLOP per element, this column-read version would be so memory-starved that even a single arithmetic instruction per cycle cannot fill the gap.

The fix. Stage through shared memory:

1. Load a $T \times T$ tile from HBM into shared memory in row-major order (coalesced - each block of 32 threads reads 32 consecutive elements from the same row).
2. Access the tile in shared memory in column order (fast - shared memory is banked, not burst-mode; column access incurs bank conflicts but not the $64\times$ bandwidth penalty of HBM column access).

What You Actually Decide in Practice

- **Always map thread index k to the innermost (fastest-varying) tensor dimension.** For a row-major 2D tensor, thread k reads `A[row][threadIdx.x + blockIdx.x * blockDim.x]`. Never iterate threads over rows when data is row-major.
- **Pad all matrix dimensions to multiples of 64 for bfloat16, 32 for float32.** A row width of 4097 bfloat16 elements (= 8194 bytes) does not align the start of row 2 to a 128-byte boundary. Threads loading row 2 will straddle two cache lines for the first element, doubling HBM transactions for that access. Padding vocabulary size from 50,257 to 50,304 (nearest multiple of 64) delivered a 25% throughput improvement in nanoGPT - entirely a coalescing alignment fix.
- **When a transposed access is unavoidable, tile through shared memory.** The canonical case: the K matrix in attention requires column access (transposed). Do not read columns directly from HBM. Load a $T \times T$ tile of K into shared memory coalesced (read rows), then transpose within shared memory (fast), then proceed. FlashAttention does exactly this.
- **Profile with Nsight Compute's sector/request ratio before optimizing.** The metric `l1tex__t_sectors_pipe_lsu` divided by `l1tex__t_requests_pipe_lsu_mem_global_op_ld.sum` reports the average number of HBM sectors per warp load request. For fully coalesced access: ratio ≈ 1 -2. For fully non-coalesced 32-thread access: ratio ≈ 32 . A ratio > 4 is a signal to investigate the access pattern before any other optimization.
- **Embedding lookups are structurally non-coalesced; mitigate at the batch level.** Token embedding tables are indexed by token ID (random integers up to vocab size). Each lookup hits a different row, with no spatial locality. Effective bandwidth is limited. Two mitigations: (1) sort requests by token ID within each microbatch so nearby GPU threads access nearby rows; (2) use INT8 or FP8 embeddings to double or quadruple the useful bytes per fetched cache line.

How This Shows Up in Interviews

- **“Why does column-major access of a row-major matrix perform so much worse on a GPU?”** Each thread in a warp accesses a different row; their addresses are stride-(row_width) apart. Each address falls in a different 128-byte burst section. The hardware must issue one HBM transaction per thread, fetching 128 bytes but using 2 - up to $64\times$ more bandwidth consumed for the same useful data.
- **“A kernel loads a 2D tensor and runs at 3% of peak HBM bandwidth. How do you diagnose and fix it?”** Diagnose: Nsight Compute sector/request ratio. If it is near 32, the access is non-coalesced. Identify which dimension threads stride over: if threads stride across rows of a row-major tensor (column access), access is non-coalesced. Fix: either stage through shared memory (load tile coalesced, use within shared memory), or restructure the algorithm to access the tensor in row order.
- **Senior follow-up: “An embedding lookup for a 50,000-vocabulary model runs at 5% of peak HBM bandwidth regardless of batch size. Coalescing is structurally impossible because token indices are random. Name three distinct ways to improve effective bandwidth.”** (1) Sort tokens by index within each batch: consecutive GPU threads then access nearby rows, partially amortizing burst section waste. (2) Quantize the embedding table to INT8 or INT4: halves or quarters the bytes per row, so each 128-byte burst section delivers 2-4 $\times$ more embedding elements; effective bandwidth in elements/sec doubles even if bytes/sec is the same. (3) Cache hot embeddings in L2 by structuring the training loop to repeat frequent-token batches before evicting: for a Zipfian token distribution where the top 1% of tokens appear

50% of the time, a 40 MB L2 can absorb those 500 frequently-accessed embedding rows (assuming $50K \text{ vocab} \times 4096 \times 2 \text{ bytes} / 50K \approx 8 \text{ KB}$ per embedding; top 0.1% $\approx 50 \text{ rows} \times 8 \text{ KB} = 400 \text{ KB}$ easily fits).

Key Takeaways

- DRAM burst mode delivers a full 128-byte cache line on any read; coalesced access (32 warp threads reading 64 consecutive bytes) achieves one HBM transaction; non-coalesced access (32 threads reading 32 scattered cache lines) requires 32 transactions, fetching 4 KB to deliver 64 bytes.
- Effective bandwidth degradation = useful bytes / fetched bytes; stride- S access in bfloat16 gives utilization $2 / \min(128, S)$; at stride = 4096 elements, effective bandwidth is $\sim 1.6\%$ of peak.
- Column-major access of a row-major matrix is the canonical non-coalescing trap; fix by staging through shared memory (load coalesced, compute in shared memory), not by reading columns directly from HBM.
- Pad all matrix dimensions to multiples of 64 (bfloat16) or 32 (float32) to maintain 128-byte alignment across rows; the nanoGPT vocabulary padding example delivered 25% speedup from alignment alone.
- Profile with Nsight Compute's sector/request ratio; values > 4 indicate non-coalesced access worth investigating.

9.2.2 Tiling for Matrix Multiplies

Add one column to your matrix and watch GEMM throughput drop 30%. The FLOP count barely changed, yet the GPU is suddenly running two execution waves instead of one, with the second wave consuming 90% of its time on 12 tiles while 96 of 108 SMs sit idle. This is the “wavy” matrix multiply performance plot that puzzles every GPU engineer who first encounters it, and tiling is the concept that explains every feature on that plot. Understanding tiling is what lets you justify vocabulary size choices, explain FlashAttention's performance model, and design kernels that sustain high throughput under interview conditions. This topic appears at the ML infrastructure engineer and systems engineer level.

The naive matrix multiply algorithm for $C = A \times B$ with $A, B \in \mathbb{R}^{N \times N}$ reads each element of A and B exactly N times from HBM: computing output element $C[i, j]$ requires the full row $A[i, :]$ and column $B[:, j]$, so $A[i, k]$ is read once per output column, i.e., N times total. With N^2 elements in A , that is N^3 total HBM element reads - entirely memory-bound for any large N .

Tiling eliminates the redundancy. Partition A , B , and C into square $T \times T$ sub-matrices. For the output tile $C[i_0 : i_0 + T, j_0 : j_0 + T]$:

$$C[i_0 : i_0 + T, j_0 : j_0 + T] = \sum_{k=0}^{N/T-1} A[i_0 : i_0 + T, kT : (k+1)T] \times B[kT : (k+1)T, j_0 : j_0 + T].$$

The algorithm loads one $T \times T$ tile of A and one $T \times T$ tile of B into shared memory, computes the partial product entirely in registers/tensor cores, then advances to the next k and repeats. Each element of A is now read from HBM only N/T times (once per tile iteration along k), not N times. HBM reads per matrix: from N^3 to N^3/T . Improvement: a factor of T .

The algorithm in pseudocode:

```
for i_tile in range(N // T):          # output tile row
    for j_tile in range(N // T):      # output tile column
        C_acc = zeros(T, T)          # accumulator in registers
```



```

for k_tile in range(N // T):    # reduction tiles
    # Load from HBM -> shared memory (coalesced)
    A_smem[0:T, 0:T] = A[i_tile*T:(i_tile+1)*T, k_tile*T:(k_tile+1)*T]
    B_smem[0:T, 0:T] = B[k_tile*T:(k_tile+1)*T, j_tile*T:(j_tile+1)*T]
    __syncthreads()
    C_acc += A_smem @ B_smem    # tensor core MMA in shared memory
    __syncthreads()
    C[i_tile*T:(i_tile+1)*T, j_tile*T:(j_tile+1)*T] = C_acc # write HBM

```

Each output tile is one thread block, assigned to one SM. The total number of blocks is $\lceil N/T \rceil^2$. With 108 SMs on an A100, blocks execute in waves of 108. If the total block count is not a multiple of 108, the last wave is partially populated: most SMs idle while the remaining blocks finish. This is *wave quantization*.

Tiling reduces HBM reads by a factor of T by staging the inner reduction loop in shared memory; wave quantization causes throughput cliffs whenever the tile count crosses a multiple of the SM count.

The fundamental insight behind tiling is that matrix multiply has reuse: each element of A participates in N dot products (one per output column), and each element of B participates in N dot products (one per output row). If you can arrange to perform all N of those dot products while the element is in shared memory, you only pay the HBM latency once. Tiling is the mechanism that groups those N uses spatially: a single $T \times T$ tile of A is used in all T dot products for the corresponding output tile row before being evicted.

The arithmetic intensity increases proportionally to T . Naive: $2N^3$ FLOPs, $2N^3$ element HBM reads $\Rightarrow$ 1 FLOP/element read. Tiled (T): $2N^3$ FLOPs, $2N^3/T$ element HBM reads $\Rightarrow$ T FLOPs/element read. For $T = 128$, arithmetic intensity is 128 FLOP/element. At 2 bytes/element (bfloat16), that is 64 FLOP/byte - well above the A100's roofline threshold of 156 FLOP/byte for bfloat16, but much closer to the compute-bound regime than the naive algorithm.

Tiling also subsumes coalescing: when a block loads a tile from HBM into shared memory, the 32 threads in a warp each read consecutive elements from the same row (row-major, one row of the tile per warp). This is perfectly coalesced. Subsequent intra-tile computation uses shared memory, where access order does not matter for bandwidth. Tiling thus handles both the HBM redundancy problem and the coalescing problem simultaneously.

A Worked Example

$A, B \in \mathbb{R}^{4096 \times 4096}$, bfloat16, on an A100 (108 SMs). Tile size $T = 128$.

HBM traffic reduction. Each element of A is read $N/T = 4096/128 = 32$ times from HBM (once per tile iteration along k). Total HBM reads:

$$2 \times 4096^2 \times 32 \times 2 \text{ bytes} = 2 \times 16,777,216 \times 64 \approx 2.1 \text{ GB}.$$

At 2 TB/s HBM bandwidth: $2.1 \text{ GB} / (2 \text{ TB/s}) \approx 1.0 \text{ ms}$.

Compare to naive (N reads per element): $2 \times 4096^2 \times 4096 \times 2 \approx 274 \text{ TB}$, which at 2 TB/s would take $\sim 137,000 \text{ s}$. Even with a 40 MB L2 cache providing partial reuse, naive is the wrong framing for large matrices.

Shared memory budget. Two tiles in flight simultaneously:

$$2 \times T^2 \times 2 \text{ bytes} = 2 \times 128^2 \times 2 = 65,536 \text{ bytes} = 64 \text{ KB}.$$

A100 configurable shared memory: up to 164 KB per SM. With 64 KB per block, two concurrent blocks fit per SM ($2 \times 64 = 128 \text{ KB} < 164 \text{ KB}$): occupancy is $2 \text{ blocks} \times 256 \text{ threads} = 512 \text{ threads per SM}$, or 25% of the

2048-thread maximum. Acceptable for a compute-bound kernel.

Tile count and wave analysis.

$$\text{Output tiles} = \left\lceil \frac{4096}{128} \right\rceil^2 = 32^2 = 1024.$$

$$\text{Waves} = \left\lceil \frac{1024}{108} \right\rceil = 10. \quad \text{Last wave tiles} = 1024 - 9 \times 108 = 52.$$

Last-wave SM utilization: $52/108 = 48\%$. Execution time $\approx 10 \times t_{\text{wave}}$ instead of ideal $9.48 \times t_{\text{wave}}$: overhead $\approx 5\%$. Acceptable.

Pathological case. $N = 1792$ with tile shape 256×128 . Output tiles: $(1792/256) \times (1792/128) = 7 \times 14 = 98$. With 108 SMs: $98 < 108$, so all tiles execute in one wave. GPU runs at 100% SM utilization. Now $N = 1796$: $\lceil 1796/256 \rceil \times \lceil 1796/128 \rceil = 8 \times 15 = 120$ tiles. First wave: 108 tiles (100% utilization). Second wave: 12 tiles ($12/108 = 11\%$ utilization). Total: 2 waves at cost $\approx 2 \times t_{\text{wave}}$ instead of ideal $1.11 \times t_{\text{wave}}$. Throughput drops $\approx 45\%$ from a 4-element increase in N .

What You Actually Decide in Practice

- **Use tile size $T \in \{64, 128, 256\}$ on A100/H100; 128 is the default safe choice.** Tensor core MMA units operate on 16×16 fragments; 128 is 8×16 , mapping cleanly. Smaller tiles reduce shared memory pressure but also reduce the arithmetic intensity improvement (less reuse per tile load). Larger tiles ($T = 256$) improve reuse but consume more shared memory, limiting occupancy to 1 block per SM.
- **Pad matrix dimensions to the nearest multiple of T (minimum multiple of 64 for bfloat16).** Fractional boundary tiles leave threads with no useful work. Padding the nanoGPT vocabulary size from 50,257 to 50,304 (nearest multiple of 64) eliminated a fractional last column-tile and delivered a 25% speedup. Compute the padded size: $\lceil D/T \rceil \times T$, then run the kernel over the padded region and discard padded outputs.
- **Target tile counts that avoid wave quantization cliffs.** For an A100 (108 SMs), target $\lceil N/T \rceil^2 \equiv 0 \pmod{108}$ or choose N such that the last wave has > 80 tiles. For $T = 128$: tile counts that are multiples of 108 with 128-element dimensions include $N = 1536$ (12 tiles $\times$ 9 tiles = 108, one wave), $N = 4096$ ($32^2 = 1024 = 9.48 \times 108$, last wave 48% utilized), or $N = 4224$ ($33^2 = 1089 = 10.1 \times 108$, last wave 9% utilized - bad). Check before committing to a model width.
- **Verify shared memory usage fits within the per-SM budget.** A100 maximum shared memory: 164 KB (with explicit configuration). Safe rule: $2 \times T^2 \times \text{dtype_bytes} < 96 \text{ KB}$ per block to allow 2 concurrent blocks per SM. For $T = 128$ and bfloat16: $2 \times 128^2 \times 2 = 64 \text{ KB} < 96 \text{ KB}$ - safe. For $T = 192$ and bfloat16: $2 \times 192^2 \times 2 = 147 \text{ KB} > 96 \text{ KB}$ - forces single block per SM, halving occupancy.
- **Do not try to manually tune tile sizes: use cuBLAS or Cutlass for standard GEMMs.** Both libraries auto-select tile sizes based on matrix shape, SM count, and data type. Manual tuning matters only for custom operations (e.g., fused attention, custom activations) where a library kernel does not exist. When writing a custom kernel, always benchmark tile sizes $\{64, 128, 256\}$ using nvprof or Nsight Compute before committing to a size.

How This Shows Up in Interviews

- **“Explain why GEMM throughput has a wavy pattern as matrix size increases - why does adding one column sometimes drop throughput by 30%?”** Two effects. First: a dimension that is not a multiple

of T leaves the last tile mostly padding; those SMs do little useful work (discretization waste). Second: total tile count crossing a multiple of SM count triggers wave quantization - an extra nearly-empty wave runs at $\ll 100\%$ SM utilization, consuming near-full wave time for a small amount of work. Both effects can occur simultaneously when the dimension changes by one.

- **“How does tiling improve matrix multiply performance on a GPU?”** Tiling stages the inner reduction loop in shared memory instead of HBM. Each element of A and B is read from HBM N/T times instead of N times - a factor-of- T reduction in HBM traffic. For $T = 128$ and $N = 4096$, each element is read 32 times from HBM rather than 4096 times, converting an HBM-dominated workload into a near-compute-bound one.
- **Senior follow-up: “A model uses embedding dimension $d = 4097$. Profiling shows GEMM throughput dropped 25% vs. $d = 4096$. Identify the two distinct performance hazards and describe a fix for each.”** Hazard 1: 4097 is not a multiple of $T = 128$; the last column-tile is $1/128$ -width, leaving 127 of 128 threads in that tile doing no useful work. Fix: pad d to $4224 = 33 \times 128$. Hazard 2: after padding, tile count $= \lceil 4224/T \rceil^2 = 33^2 = 1089$; $\lceil 1089/108 \rceil = 11$ waves with last wave $= 1089 - 10 \times 108 = 9$ tiles, SM utilization 8%. Total time $\approx 11 \times t_{\text{wave}}$ instead of ideal $10.1 \times t_{\text{wave}}$: a 9% quantization penalty. Fix for hazard 2: find the smallest d that is (a) ≥ 4097 , (b) a multiple of T , and (c) yields a tile count divisible by 108 or at least with a last-wave utilization $> 80\%$. For $T = 128$: $d = 4480 = 35 \times 128$ gives $35^2 = 1225 = 11.34 \times 108$, last wave $= 1225 - 11 \times 108 = 37$ tiles (34% utilization - still suboptimal). Check $d = 4608 = 36 \times 128$: $36^2 = 1296 = 12 \times 108$ - a perfect multiple of 108, one clean wave. Recommend $d = 4608$ to the team.

Key Takeaways

- Tiling reduces HBM reads by a factor of T (tile size): each element of A and B is fetched N/T times instead of N times; for $T = 128$, HBM traffic drops $128\times$ vs. naive and the kernel transitions from memory-bound to near-compute-bound.
- Total tile count $= \lceil N/T \rceil^2$ must be a multiple of the SM count (108 on A100, 132 on H100) or the last wave executes with most SMs idle; tile count just above a multiple is the worst case - two waves for nearly one wave of useful work.
- Shared memory budget: both input tiles must fit simultaneously; $2 \times T^2 \times \text{dtype_bytes} < 96$ KB per block allows two concurrent blocks per SM. For bfloat16 and $T = 128$: 64 KB - safe.
- Pad all matrix dimensions to multiples of T (minimum multiples of 64 for bfloat16): fractional tiles waste SM capacity and break burst-section alignment. The nanoGPT vocabulary padding ($50,257 \rightarrow 50,304$) is the canonical production example.
- For custom kernels, the tile size selection involves three simultaneous constraints: fit within shared memory, coalesce HBM tile loads (multiples of 32 or 64 elements), and achieve favorable tile count vs. SM count ratio. Use cuBLAS/Cutlass for standard GEMMs; invest manual tuning effort only in custom operations.

9.3 Lower Precision and Quantization-Aware Training

9.3.1 FP16 and BF16 in Practice

Cutting numerical precision in half is the highest-leverage single change you can make to LLM training throughput, and it requires zero custom kernel code. The choice between FP16, BF16, and FP32 for each tensor in the training graph is one of the first things an interviewer will probe once you claim to understand efficient training. At the mid-level this surfaces as “why is BF16 faster than FP32”; at the senior level it escalates to “walk me through every place you keep FP32 in a mixed-precision run and explain why.”

Three formats matter for LLM training. FP32 uses 32 bits (1 sign, 8 exponent, 23 mantissa) and supports values up to $\approx 3.4 \times 10^{38}$. FP16 uses 16 bits (1 sign, 5 exponent, 10 mantissa) with a maximum representable value of only 65,504—four orders of magnitude smaller. BF16, the format used by almost every production LLM training run today, also uses 16 bits but allocates 8 bits to the exponent (matching FP32) and only 7 bits to the mantissa. BF16 deliberately trades mantissa precision for dynamic range, keeping the same maximum and minimum values as FP32.

The performance argument is direct. For any memory-bound kernel, wall-clock time equals bytes transferred divided by bandwidth. Halving the bits per element halves the bytes transferred, halving the kernel run-time without changing a single arithmetic operation. A ReLU on n BF16 elements reads $2n$ bytes and writes $2n$ bytes— $4n$ total—at $\text{AI} = 0.25 \text{ FLOPs/byte}$. The same ReLU in FP32 moves $8n$ bytes at $\text{AI} = 0.125 \text{ FLOPs/byte}$. The AI ratio is the same ($0.25/0.125 = 2\times$), but the absolute bytes are halved, so the kernel completes twice as fast.

Matrix multiplications gain a second effect. On H100, BF16 inputs unlock the tensor-core path at $\approx 989 \text{ TFLOPs/s}$. FP32 inputs fall back to general ALUs at $\approx 67 \text{ TFLOPs/s}$ —a $14.8\times$ peak throughput gap. The standard mixed-precision matmul pattern exploits both: BF16 inputs are fed to the tensor cores; each tile accumulates dot-product partial sums in FP32 accumulator registers; the FP32 result is cast back to BF16 before writing to HBM. This is the default behavior of cuBLAS and `torch.autocast` when the target dtype is `bfloat16`.

Halving numerical precision halves HBM traffic for memory-bound ops and unlocks tensor cores for compute-bound ones; BF16 is the right default for weights and activations in LLM training, not FP16.

FP16’s 5-bit exponent limits its maximum representable value to 65,504. Gradient norms in large-scale LLM training regularly exceed this threshold during warmup, around sharp loss changes, and in embedding tables for rare tokens. When an FP16 value overflows to `Inf` or underflows to zero, the optimizer step either corrupts the weight or silently does nothing. The standard workaround is loss scaling: multiply the scalar loss by a large constant S before the backward pass so that gradient tensors are scaled up by S and stay in range, then divide the accumulated gradients by S before the optimizer step. The machinery works but requires dynamic tuning of S —too small and underflow still occurs; too large and overflow reappears. Practitioners invest real engineering effort managing this.

BF16 eliminates loss scaling entirely. Because it shares the 8-bit exponent with FP32, its representable range matches FP32 exactly. Gradient values that overflow FP16 are representable in BF16 without any adjustment. The trade-off is lower mantissa precision—7 bits instead of 23 in FP32 or 10 in FP16, giving relative rounding errors of roughly $\pm 0.78\%$ of the value. For weights, activations, and gradient tensors in an LLM forward-backward pass, this rounding is empirically inconsequential. For the accumulation step inside a tile of a matmul, it is not—which is why accumulation stays in FP32 even in a BF16 training run.

A Worked Example

Consider the gate projection and SwiGLU activation in a 7 B model: $D = 4096$, $F = 11,008$, batch size 512 in training.

SwiGLU activation (memory-bound): The kernel reads two input tensors of size $512 \times 11,008$ and writes one output, for three tensors total.

- FP32: $3 \times 512 \times 11,008 \times 4 = 67.7 \text{ MB}$; runtime $= 67.7/3,350 \approx 20.2 \mu\text{s}$.
- BF16: $3 \times 512 \times 11,008 \times 2 = 33.8 \text{ MB}$; runtime $\approx 10.1 \mu\text{s}$.
- Speedup: $2\times$ from halved bytes; zero quality loss.

Gate projection matmul (compute-bound at $B = 512$): $\text{FLOPs} = 2 \times 512 \times 4096 \times 11,008 \approx 46.1 \text{ GFLOPs}$.

- BF16 tensor cores at 989 TFLOPs/s: $46.1 \times 10^9 / 989 \times 10^{12} \approx 46.6 \mu\text{s}$ theoretical.
- FP32 general ALUs at 67 TFLOPs/s: $46.1 \times 10^9 / 67 \times 10^{12} \approx 688 \mu\text{s}$ theoretical.
- Switching from FP32 ALUs to BF16 tensor cores: $14.8\times$ peak throughput advantage for the dominant operation class.

Single decode ($B = 1$): The bottleneck is weight-loading, not arithmetic. Gate projection weights: $4096 \times 11,008 \times 2 = 90.2 \text{ MB}$ in BF16 vs 180.4 MB in FP32.

- BF16 load time: $90.2/3,350 \approx 26.9 \mu\text{s}$.
- FP32 load time: $\approx 53.8 \mu\text{s}$.
- Summed over all projection layers in a 7 B model, the $2\times$ weight-load savings directly doubles per-token decode throughput.

What You Actually Decide in Practice

- **Use BF16 for weights, activations, and gradients; never use FP16 for LLM training without loss scaling.** Wrap the forward and backward pass with `torch.autocast(device_type='cuda', dtype=torch.bfloat16)` so it automatically dispatches BF16 for matmuls and FP32 accumulation for reductions. On hardware pre-dating BF16 support (Volta, Turing), FP16 with dynamic loss scaling is the only option-but on A100 or H100, BF16 is strictly better and simpler.
- **Keep optimizer state in FP32.** Adam's first moment (m_t) and second moment (v_t) are running estimates of the gradient mean and variance. The weight update $\Delta w = -\eta \cdot m_t / \sqrt{v_t + \epsilon}$ involves dividing two near-zero BF16 quantities, producing a result that BF16's $\pm 0.78\%$ relative error rounds to zero at small learning rates or near-converged weights. FP32 master weights and optimizer state are the universal standard; the memory overhead is acceptable because they are not on the HBM bandwidth critical path during forward and backward.
- **Keep layer norm and softmax reductions in FP32 accumulation.** Computing the mean and variance of a row for layer normalization requires accurate summation of up to several thousand elements; BF16 accumulation introduces a systematic bias that shifts the normalized output. PyTorch's built-in `nn.LayerNorm` upcasts to FP32 internally when inputs are BF16. Do not override this with a manual BF16 implementation unless you have measured that it makes no quality difference on your specific training run.
- **Never store embedding gradients in FP16 on large vocabularies.** Embedding gradients for rare tokens are sparse and can be large; FP16 overflows without loss scaling. With BF16 this is not an issue. If an older checkpoint or library forces FP16 embeddings, add per-parameter loss scaling for the embedding table and monitor gradient norms separately.

- **Confirm fusion when switching precision.** Lower precision only helps if the kernel using it runs. Profile with `torch.profiler` before and after adding `torch.autocast`; confirm that the activation kernels and matmuls show a single BF16 launch each. If `torch.compile` is not applied, some elementwise chains may still dispatch as FP32 scalar ops regardless of autocast.

How This Shows Up in Interviews

- “*Why is BF16 faster than FP32 for LLM training?*” Strong answer: two mechanisms. First, halving bytes per element halves HBM traffic for memory-bound kernels (activations, norms, elementwise ops), directly halving their runtime. Second, BF16 inputs unlock H100 tensor cores at ≈ 989 TFLOPs/s vs general FP32 ALUs at ≈ 67 TFLOPs/s—a $14.8\times$ peak advantage on matrix multiplications, which dominate total FLOPs.
- “*Why does production LLM training use BF16 instead of FP16?*” Strong answer: FP16’s 5-bit exponent limits its maximum representable value to 65,504; gradient norms in large-scale training regularly exceed this, causing overflow without loss scaling. BF16 matches FP32’s 8-bit exponent and hence its full dynamic range, eliminating overflow at the cost of lower mantissa precision—a trade that is empirically safe for LLM weights, activations, and gradients.
- *Senior follow-up: “Walk me through every place in a BF16 training run where you keep FP32 and why.”* Senior answer: (1) Matmul accumulation-tensor-core tiles accumulate in FP32 register buffers before writing BF16 to HBM; handled transparently by cuBLAS. (2) Adam optimizer state (m_t , v_t , master weights)-weight updates involve small increments that BF16 rounds to zero near convergence. (3) Layer-Norm and softmax reductions-summation over wide dimensions accumulates rounding bias in BF16. (4) Loss scalar-used for logging and numerical debugging. Everything on the HBM bandwidth critical path (weight matrices, activation tensors, gradient tensors) is BF16.

Key Takeaways

- BF16 (1+8+7 bits) matches FP32’s dynamic range at half the storage; FP16 (1+5+10) has higher mantissa precision but a 65,504 maximum that overflows in LLM training without loss scaling.
- Halving bits per element halves HBM traffic; for memory-bound kernels (activations, norms), this halves runtime directly at no quality cost.
- BF16 unlocks H100 tensor cores at ≈ 989 TFLOPs/s vs FP32 general ALUs at ≈ 67 TFLOPs/s—a $14.8\times$ peak advantage for matmuls, the dominant operation class.
- Keep FP32 for Adam optimizer state, master weights, layer norm and softmax reductions; use `torch.autocast` to dispatch the rest automatically.
- At $B = 1$ inference, BF16 halves decode latency vs FP32 by halving weight-load bytes; the next gain comes from INT8 or FP8, which halve again.

9.3.2 FP8 and INT8 for Inference-Oriented Training

BF16 is the floor of precision for training stability, but it is not the floor of what the hardware supports. INT8 has been standard for inference deployment for several years; FP8 is now native to H100 and has moved into production training pipelines. Both formats halve the bytes per element again versus BF16, and both require decisions that go wrong in distinct ways. Senior interviews probe this territory when the topic is serving cost,

large-model inference on minimal hardware, or the next-generation training infrastructure that frontier labs are deploying today.

INT8 represents values as signed 8-bit integers in the range $[-128, 127]$. Mapping a floating-point tensor $\mathbf{X}$ into this range requires a calibrated scale s and zero-point z :

$$x_q = \text{round}\left(\frac{x}{s}\right) + z, \quad \hat{x} = (x_q - z) \cdot s$$

Per-tensor quantization uses one (s, z) pair for the entire tensor—fast but coarse. Per-channel quantization assigns one pair per output channel of a weight matrix—more bookkeeping, but it preserves the dynamic range of channels with outlier magnitudes. H100 INT8 tensor-core throughput is $\approx 1,979$ INT8 TOPS—exactly $2\times$ the BF16 rate of 989 TFLOPs/s. This arithmetic benefit is only realizable when the kernel is compute-bound; at $B = 1$ inference the value of INT8 is purely in the halved weight-load bytes.

FP8 arrives on H100 in two hardware formats. **E4M3** (1 sign, 4 exponent, 3 mantissa bits) has a maximum value of 448 and good per-element resolution; it is suited for weights and activations in the forward pass where the dynamic range is relatively narrow. **E5M2** (1 sign, 5 exponent, 2 mantissa bits) has a maximum of 57,344 and wider dynamic range at the cost of lower precision; it is suited for gradients in the backward pass, which have heavier distribution tails. Each FP8 tensor carries a companion FP32 scaling factor that maps the full floating-point range into the narrow FP8 window. H100 FP8 GEMM throughput is $\approx 1,979$ TFLOPs/s, matching INT8 TOPS and delivering $2\times$ the BF16 rate on compute-bound matmuls.

Post-training quantization (PTQ) calibrates scale factors on a small representative dataset after training completes; no re-training is required. Quantization-aware training (QAT) inserts fake-quantize operations during the training loop that round values and pass straight-through gradients, letting the model learn to compensate for rounding errors before deployment.

FP8 and INT8 each halve memory footprint relative to BF16; the engineering challenge is managing scale calibration and knowing which tensors absorb rounding without accuracy loss and which do not.

At $B = 1$ inference, every decode step requires loading the full weight tensor for each layer from HBM before the arithmetic can begin. Token generation rate is therefore bytes per second, not FLOPs per second. INT8 weights occupy half the bytes of BF16 weights. With everything else equal, loading half the bytes takes half the time, and per-token decode throughput approximately doubles—*independent of whether INT8 tensor cores are used for the arithmetic*. This is the dominant reason production serving systems quantize weights: the decode-bound regime at $B = 1$ means you are buying throughput by reducing bytes, not by improving peak FLOP rate.

FP8 training offers a compound gain. Halving weight and activation storage reduces HBM pressure throughout the forward and backward pass, not just during weight loading. FP8 GEMMs run at $2\times$ the BF16 tensor-core rate on H100, shifting the compute-bound threshold to lower batch sizes and directly cutting per-step wall time. DeepSeek-V3 reported roughly $2\times$ end-to-end training throughput improvement versus BF16 on the same hardware using FP8 mixed precision, with no measurable quality degradation after careful per-tensor scale management.

The accuracy risk is real but concentrated. Weight quantization with per-channel INT8 scales loses almost nothing on standard benchmarks: weight distributions in trained LLMs are approximately Gaussian and map cleanly into $[-128, 127]$. Activation quantization is harder because activations have outlier channels whose magnitudes can be $10\times$ the median channel value; per-tensor INT8 scales clip these outliers into the quantized range. Gradient quantization is hardest of all, which is why E5M2 (wider range) rather than E4M3 is the H100 hardware format for backward-pass tensors.

A Worked Example

Consider a 70 B parameter model deployed for autoregressive generation at $B = 1$.

Memory footprint:

- BF16 weights: $70 \times 10^9 \times 2 \text{ bytes} = 140 \text{ GB}$. This requires two H100 SXM5 80 GB to hold weights alone, before KV cache or activation buffers.
- Weight-only INT8 (W8A16-weights in INT8, activations in BF16): $70 \times 10^9 \times 1 \text{ byte} = 70 \text{ GB}$. Fits in a single H100 SXM5 80 GB with $\approx 10 \text{ GB}$ available for KV cache.
- FP8 weights (E4M3): same 70 GB footprint as INT8. One GPU required.

Decode latency at $B = 1$:

- BF16: dominant cost is loading 140 GB of weights at H100 HBM3 bandwidth 3.35 TB/s: $140/3,350 \approx 41.8 \text{ ms}$ per forward pass.
- W8A16 INT8: load 70 GB of INT8 weights. Load time = $70/3,350 \approx 20.9 \text{ ms}$. Dequantization to BF16 before each GEMM adds $< 1 \text{ ms}$ across all layers. Net: $\approx 2\times$ faster per-token throughput.

Calibration overhead: Running 512 forward passes on a representative dataset to compute per-channel INT8 scales takes roughly 5 minutes for a 70 B model. This one-time cost is irrelevant at deployment scale and does not require touching the original training code.

FP8 training throughput: For a matmul of $2 \times 512 \times 4096 \times 11,008 \approx 46.1 \text{ GFLOPs}$ in BF16 at 989 TFLOPs/s: $\approx 46.7 \mu\text{s}$. The same GEMM in FP8 at 1,979 TFLOPs/s: $\approx 23.3 \mu\text{s}$. Across all layers and all matmuls in a training step, this $2\times$ reduction compounds to a roughly 1.5-2 $\times$ end-to-end step time improvement, with the remainder of step time consumed by non-GEMM operations that remain in BF16.

What You Actually Decide in Practice

- **Start with weight-only INT8 (W8A16).** Quantize weights per output channel after training; keep activations in BF16. Libraries like `bitsandbytes` implement this in one line. Quality loss is typically unmeasurable on 7 B-70 B checkpoints, the weight-load halving gives $2\times$ decode throughput at $B = 1$, and nothing in the serving stack changes except the weight storage format.
- **Upgrade to full INT8 (W8A8) only after running your full eval suite.** W8A8 uses INT8 tensor cores and provides arithmetic speedup at higher batch sizes, but activation quantization requires handling outlier channels. Per-tensor activation scales cause clipping; per-channel activation scales add synchronization overhead in the kernel. Measure on the specific tasks your deployment serves-perplexity alone is insufficient.
- **Never quantize these tensors.** Embedding lookup tables: rare tokens have large weight magnitudes that get clipped under per-tensor INT8 scales, silently degrading handling of infrequent vocabulary. Layer norm scale and bias parameters: these are tiny tensors where rounding has outsized effect on normalized outputs. Softmax normalizers: numerical stability of the log-sum-exp computation is sensitive to precision. The final unembedding projection output logits if they feed directly into sampling without a temperature rescaling step.
- **FP8 training requires H100 and the Transformer Engine library.** NVIDIA's `transformer_engine` handles E4M3/E5M2 format dispatch, delayed per-tensor scale updates (updating scales every 16 steps is the standard default), and FP8 GEMM scheduling. Do not attempt to write FP8 GEMMs manually; the hardware requirements and scale management are non-trivial. The key knob is the scale update frequency: more frequent updates improve numerical stability at a small runtime overhead; 16-step delayed scaling

is the practical default used by production deployments.

- **PTQ is sufficient for INT8; QAT is for 4-bit and below.** For INT8, calibrating per-channel weight scales and per-tensor activation scales on a 512-sample in-domain dataset closes essentially all the quality gap relative to BF16. QAT-simulating quantization noise during the training loop-is worth the engineering cost only when PTQ quality becomes unacceptable, which empirically happens below 4-bit precision (the domain of GPTQ, AWQ, and QLoRA-style methods).

How This Shows Up in Interviews

- “*Why is INT8 inference faster than BF16 inference at batch size 1?*” Strong answer: at $B = 1$, decode is weight-loading-bound, not compute-bound. INT8 weights occupy half the bytes of BF16 weights, so the GPU loads the same parameters in half the time. The $2\times$ arithmetic throughput advantage of INT8 tensor cores is irrelevant because the operation is memory-bound regardless of peak TOPS-the compute units finish the arithmetic before the next tile arrives from HBM.
- “*Walk me through the memory implications of weight-only INT8 for a 70 B model.*” Strong answer: INT8 halves weight bytes from 140 GB to 70 GB, reducing the serving requirement from two H100 80 GB GPUs to one. Activations stay in BF16, so the KV cache and intermediate tensors are unchanged. Per-token decode latency approximately halves because the dominant runtime at $B = 1$ is weight-load time = weight bytes / bandwidth.
- *Senior follow-up: “When would you choose W8A8 over W8A16 for production, and what are the risks?”* Senior answer: W8A8 is worth investigating when the batch size is high enough that the linear layers are compute-bound-above the W8A16 ridge point of ≈ 295 FLOPs/byte, which at a single 4096×11008 projection corresponds to roughly $B > 295$. Below that threshold, activation quantization adds calibration complexity and accuracy risk without delivering the INT8 tensor-core throughput gain, because the kernel is still memory-bound. The specific risk to measure is outlier-channel clipping: check the per-channel activation distributions before and after quantization; if more than $\sim 0.1\%$ of values are clipped, per-channel activation scales are required.

Key Takeaways

- INT8 and FP8 each halve weight bytes versus BF16; at $B = 1$ inference, this halves decode latency because the bottleneck is weight-load time, not arithmetic throughput.
- W8A16 (INT8 weights, BF16 activations) is the safest quantization tier: near-zero quality loss on 7 B-70 B models, $2\times$ decode throughput, calibrated offline with 512 samples.
- W8A8 uses INT8 tensor cores at $\approx 1,979$ TOPS (H100), delivering $2\times$ arithmetic speedup versus BF16-but only when the kernel is compute-bound; activation quantization adds accuracy risk that must be measured.
- FP8 training on H100 uses E4M3 (forward pass) and E5M2 (backward pass) with per-tensor FP32 scale factors; the Transformer Engine handles dispatch and delayed scaling; end-to-end training throughput improvement versus BF16 is roughly $1.5\text{-}2\times$.
- PTQ on a 512-sample calibration set is sufficient for INT8; QAT is warranted only below 4-bit, where PTQ accuracy loss becomes non-negligible.

Chapter 10 Writing and Benchmarking Custom Kernels

10.1 Benchmarking and Profiling Discipline

10.1.1 Warm-Up, CUDA Synchronization, and Wall-Clock Traps

I have watched engineers spend three days optimizing a kernel and then present timing results showing it is $10\times$ faster than PyTorch. The kernel was not faster. They forgot to call `torch.cuda.synchronize()`, so they measured how long the CPU took to queue the CUDA command - microseconds - instead of how long the GPU took to execute it - milliseconds. Getting benchmarking right is a prerequisite for any GPU kernel work, and getting it wrong produces numbers that are confidently, silently wrong. This topic appears at every level from junior ML engineer to staff systems engineer; the mistake is embarrassingly common and the fix takes three lines.

GPU benchmarking has exactly two mandatory steps that developers consistently miss.

Step 1: Warm-up iterations. The first time PyTorch dispatches a CUDA kernel, it performs just-in-time compilation, loads shared libraries, allocates internal caches, and primes the GPU's state. These startup costs are one-time artifacts - not representative of steady-state performance. If you benchmark the first invocation, you measure initialization, not computation. Always discard the first 5-10 iterations before timing.

Step 2: CUDA synchronization. The GPU and CPU are independent compute devices connected by a command queue. When Python calls a CUDA operation, the CPU adds the kernel descriptor to the queue and *immediately returns* - it does not wait for the GPU to finish. If you call `time.perf_counter()` before and after a GPU operation without synchronizing, you measure how long the CPU spent dispatching the command: typically 10-100 microseconds. The actual GPU execution - the number you care about - completes asynchronously and may take $100\times$ longer.

`torch.cuda.synchronize()` blocks the CPU until all queued GPU kernels have completed. Call it once immediately before starting the timer and once immediately after the timed code to drain the queue and measure true GPU execution time.

The complete benchmark pattern:

```
import time, torch

def benchmark(fn, n_warmup=5, n_trials=20):
    for _ in range(n_warmup):
        fn()

    times = []
    for _ in range(n_trials):
        torch.cuda.synchronize()      # drain queue; mark t=0
        t0 = time.perf_counter()
        fn()
        torch.cuda.synchronize()      # wait for GPU to finish
        t1 = time.perf_counter()
        times.append(t1 - t0)
```

```
return sum(times) / len(times)    # mean over trials
```

Average over trials rather than taking the minimum. GPU clock speeds drop under thermal load; individual measurements vary by 5-10%. The mean reflects steady-state production performance better than the best-case minimum.

Without warm-up you measure initialization cost; without `cuda.synchronize()` you measure dispatch cost; only with both do you measure actual GPU execution time.

The GPU's asynchronous execution model is a feature. It lets the CPU run far ahead of the GPU, filling the command queue and keeping the GPU continuously fed without waiting for each kernel to finish. This is one of the primary reasons Python - a slow interpreted language - can drive GPU computations at near-full utilization. Python is never the bottleneck because the CPU is always running one full training step ahead of the GPU.

But this same asynchrony is what makes naive wall-clock measurement wrong. When the CPU calls `torch.matmul()`, it takes roughly 10-50 microseconds to construct and enqueue the kernel descriptor. The actual matrix multiply on a 4096×4096 bfloat16 matrix takes roughly 40-60 milliseconds on an A100. A benchmark without synchronization reports 0.01 ms; with synchronization it reports 50 ms. The $5000\times$ discrepancy is entirely due to missing the sync.

The JIT warm-up trap is equally insidious but less well known. PyTorch compiles CUDA code on first use, selects optimal cuBLAS kernels by running micro-benchmarks internally, and allocates persistent workspace buffers. All of this happens transparently in the background and takes 100-1000 ms per unique operation shape. The steady-state cost is zero. Benchmarking without warm-up inflates the reported cost by orders of magnitude.

A Worked Example

Benchmarking a manual (unfused) GELU vs. the fused PyTorch implementation on an A100, on a 64,000,000-element bfloat16 tensor.

Wrong benchmark (no warmup, no sync):

```
t0 = time.time()
y = manual_gelu(x)    # x.shape = (64_000_000,), device='cuda'
t1 = time.time()
print(f"{{(t1-t0)*1e3:.2f}} ms")    # prints ~0.1 ms
```

Correct benchmark:

```
manual_time = benchmark(lambda: manual_gelu(x))    # 8.1 ms
pytorch_time = benchmark(lambda: torch.nn.functional.gelu(x))    # 1.1 ms
```

The wrong benchmark reports `manual_gelu` at 0.1 ms, apparently $11\times$ faster than PyTorch. The correct benchmark reports 8.1 ms - $7.4\times$ slower than PyTorch. The $80\times$ disagreement is entirely due to missing synchronization.

The manual GELU dispatches 6 separate CUDA kernels (one each for x^3 , three multiplications, the `tanh`, and one addition). Each kernel reads and writes the full 128 MB tensor from HBM. The fused PyTorch GELU dispatches 1 kernel that reads the tensor once and writes it once. Correct benchmarking is what reveals this $7.4\times$ gap; without it you would optimize in the wrong direction.

Scaling sanity check: for a 256-dim, 4-layer MLP on batch size 256, time per step is approximately 5 seconds. Scaling from 2 to 5 steps gives $2\times$, $3\times$, $4\times$, $5\times$ of single-step time - exactly linear, as expected. If scaling is non-linear when it should be linear, something is wrong: memory fragmentation, thermal throttling, or a synchronization barrier that was not there before.

What You Actually Decide in Practice

- **Use `time.perf_counter()`, not `time.time()`.** `perf_counter` has nanosecond resolution. `time.time()` has millisecond or worse resolution on some systems, which means all sub-millisecond GPU operations look identical.
- **Set `n_warmup` ≥ 5 , `n_trials` ≥ 20 for production benchmarks.** Five warm-up iterations is enough for PyTorch's JIT to complete on most operations. Twenty trials gives a stable mean. For kernels faster than $100\ \mu\text{s}$, increase to 100+ trials because individual measurement noise becomes significant.
- **Report the mean, not the minimum.** The minimum filters out thermal throttling and cache misses, producing best-case numbers that do not reflect production behavior. If you want to separate kernel execution from scheduling jitter, use CUDA events (`torch.cuda.Event(enable_timing=True)`) which measure inside the GPU's own clock.
- **Benchmark end-to-end, not individual kernels in isolation.** A kernel that is fast in isolation may be slow when integrated because it changes memory access patterns for subsequent kernels, triggers memory reallocation, or alters the GPU's warp scheduler state. Always benchmark the realistic workload, then drill down to identify the bottleneck.
- **Never call `.item()`, `.cpu()`, or `print(tensor)` inside a benchmark loop.** All three force a CUDA synchronize, making your GPU and CPU effectively synchronous and eliminating the asynchrony advantage. This also applies to training loops: a `print(loss.item())` after each step can double step time by serializing CPU and GPU execution.

How This Shows Up in Interviews

- **“How would you correctly benchmark a custom CUDA kernel's performance?”** Two mandatory steps: (1) warm-up iterations to prime JIT compilation and caches; (2) `torch.cuda.synchronize()` before and after the timed region to drain the GPU command queue and measure actual GPU execution time. Average over 20+ trials using `time.perf_counter()`.
- **“A colleague benchmarks their kernel and claims it is $500\times$ faster than the reference implementation. What is the most likely explanation?”** Missing `cuda.synchronize()`. The CPU timer is measuring how long it took to queue the CUDA command, not how long the GPU took to execute it. Typical dispatch time is $10\text{-}100\ \mu\text{s}$; the actual kernel may take 50+ ms .
- **Senior follow-up: “Your benchmark shows a $10\times$ speedup over PyTorch on a simple fused operation. What would you verify before claiming the result is real?”** (1) Correctness: run an assertion that both implementations produce numerically equivalent outputs. A kernel returning garbage can appear arbitrarily fast. (2) Input size: for very small inputs, PyTorch's dispatch overhead may dominate both paths, making the ratio meaningless. (3) Profile both: use `torch.profiler` to verify each dispatches what you think it does - PyTorch may dispatch to a slower kernel path for the specific shape tested. (4) Measure memory traffic: a $10\times$ speedup on a memory-bound operation is legitimate if you reduced HBM round-trips by $10\times$; on a compute-bound operation it would require $10\times$ more FLOPs/second than the

hardware provides, which is a red flag.

Key Takeaways

- CUDA operations are asynchronous: the CPU returns immediately after dispatch; without `torch.cuda.synchronize` you measure dispatch time ($\sim 10\text{-}100\ \mu\text{s}$), not GPU execution time ($\sim \text{ms}$ to seconds).
- Warm-up iterations (minimum 5) are mandatory: first iterations trigger JIT compilation and internal cuBLAS micro-benchmarking that inflates the reported cost by orders of magnitude.
- The correct benchmark pattern: warm up $\rightarrow$ sync $\rightarrow$ start timer $\rightarrow$ `fn()` $\rightarrow$ sync $\rightarrow$ stop timer $\rightarrow$ mean over 20+ trials.
- Use `time.perf_counter()`, report the mean (not min), and always verify correctness before interpreting any timing.
- Never call `.item()`, `.cpu()`, or `print(tensor)` inside training loops or benchmark loops: all three force an implicit CPU-GPU sync, serializing what should be asynchronous execution.

10.1.2 Reading an NVIDIA Nsight Systems Trace

A benchmark tells you your training step takes 500 ms. A profiler trace tells you 300 ms of that is the GPU idle because a `print(loss.item())` call is forcing a synchronization barrier that eliminates every CPU-ahead scheduling advantage. These are different problems requiring different solutions, and you will never find the second one from benchmarks alone. Reading profiler traces fluently is the skill that separates engineers who know *that* their system is slow from engineers who know *why* and can fix it. This shows up at the senior engineer and above level in any role involving training system performance.

GPU profiling operates at two levels of resolution.

PyTorch Profiler (quick, no external tools). Instruments `aten::` operations via C++ hooks and records CPU and CUDA execution times per operation:

```
with torch.profiler.profile(
    activities=[
        torch.profiler.ProfilerActivity.CPU,
        torch.profiler.ProfilerActivity.CUDA,
    ]
) as prof:
    torch.cuda.synchronize()
    fn()
    torch.cuda.synchronize()
print(prof.key_averages().table(sort_by="cuda_time_total", row_limit=10))
```

The output table shows: operation name (Python $\rightarrow$ `aten::` $\rightarrow$ CUDA kernel), self CPU time, self CUDA time, call count. For a matrix multiply it shows `aten::mm` on the CPU side and the specific Cutlass kernel selected for your matrix shape on the CUDA side. For a compound operation like `torch.cdist`, it decomposes into constituent kernels: `aten::matmul` (78% of GPU time), `aten::pow` (5%), `aten::sum` (3%), `aten::cat` (6%). This tells you exactly where to focus optimization effort.

NVIDIA Nsight Systems (detailed, requires nsys binary). Produces a `.nsys-rep` file for the Nsight GUI. The trace shows three horizontal lanes:

- **GPU lane (“CUDA HW”)**: every kernel that ran on device, its start time, duration, and name.
- **CPU lanes (per thread)**: Python and C++ execution, kernel launches, memory copies.
- **Memory lane**: allocation and deallocation events.

The most important feature: the GPU and CPU lanes are shown side by side on the same timeline. You can directly observe whether the GPU is idle (no kernel running) and what the CPU was doing during that gap.

NVTX annotations mark code regions that appear as colored bands in the CPU lane:

```
import torch.cuda.nvtx as nvtx

for step in range(n_steps):
    nvtx.range_push(f"step_{step}")
    nvtx.range_push("forward"); loss = model(x); nvtx.range_pop()
    nvtx.range_push("backward"); loss.backward(); nvtx.range_pop()
    nvtx.range_push("optimizer"); opt.step(); nvtx.range_pop()
    nvtx.range_pop() # step
```

These cost microseconds and make the trace immediately interpretable.

A profiler trace tells you exactly which CUDA kernel ran, for how long, and whether the GPU was actually busy - everything benchmarks cannot tell you.

The PyTorch Profiler costs 1-5% runtime overhead and requires zero external setup. It is the right first tool for identifying which operations dominate GPU time and verifying that kernels dispatched are the kernels you intended.

Nsight Systems operates at the hardware level: it instruments the CUDA driver directly, recording every kernel launch, memory copy, and synchronization event with sub-microsecond precision. This is the right second tool when you need to understand the relationship between CPU and GPU execution - specifically, whether the GPU is idle and what the CPU is doing during that idle period.

The key asymmetry that traces reveal: different matrix shapes dispatch to completely different CUDA kernels. A 4096×4096 matmul dispatches to a Cutlass large-tile kernel; a 128×128 matmul dispatches to a different XMMA kernel. These have entirely different performance characteristics. A benchmark reports one number per implementation; a trace shows which internal path was taken for each call.

The CPU-ahead pattern is visible in any healthy Nsight trace: the CPU timeline shows the Python code for step $N + 1$ running while the GPU timeline shows step N still executing. The CPU queues kernels faster than the GPU executes them. This is correct behavior. A trace where CPU and GPU steps are perfectly aligned is a warning sign: it means a synchronization barrier - usually a `.item()` call or a Python-level loss print - is forcing the CPU to wait for the GPU at every step. Adding one `print(loss.item())` to a tight training loop can serialize what was asynchronous execution and reduce throughput by up to 50%.

A Worked Example

Profiling a 16-layer MLP (256 dimensions, batch 256) with the PyTorch Profiler.

The top-10 table by CUDA time shows (approximate numbers on A100):

- `aten::mm` (forward): 31% of GPU time, 16 calls per step.
- `aten::mm` (backward $dL/dX + dL/dW$): 61% of GPU time, 32 calls per step.
- `aten::add`: 3% GPU time (bias additions).

- `aten::relu`: 2% GPU time.
- `CudaDeviceSynchronize`: 2% (the profiler’s own barrier).

The immediate conclusion: 92% of GPU time is matrix multiplies. Optimizing the ReLU or bias operations is wasted effort. The profiler answered “where should I spend three hours?” in thirty seconds.

Now compare the Nsight Systems trace for the same model in two configurations:

Configuration A (no print): CPU timeline shows step 0 Python running for 20 ms; GPU timeline shows step 0 executing for 180 ms starting at $t = 400$ ms (after JIT startup). By step 1, the CPU is already dispatching step 2’s kernels while the GPU runs step 1. The CPU runs one full step ahead. GPU utilization is continuous - no idle gaps.

Configuration B (with `print(loss.item())` after each step): Step 1 CPU duration is now 180 ms. The CPU-step boundary aligns with the GPU-step boundary. The reason: `loss.item()` transfers a scalar from GPU to CPU, which internally calls `torch.cuda.synchronize()`. The CPU blocks for 180 ms waiting for the GPU to finish, then prints, then launches step 2. The CPU-ahead advantage is gone. Step time goes from 180 ms (GPU-paced) to 360 ms (CPU-blocked): a $2\times$ slowdown from one `print` statement.

What You Actually Decide in Practice

- **Add NVTX annotations to every training loop before profiling.** Do it once, keep it in the code permanently. The overhead is under $10\ \mu\text{s}$ per annotation pair. Without them, a Nsight trace of a multi-step training loop is almost unreadable - you cannot tell where step boundaries are.
- **Never call `.item()`, `.numpy()`, `.cpu()`, or `print(tensor)` inside the training loop.** All four force a CUDA synchronize that serializes CPU and GPU. For loss logging, accumulate values in a GPU tensor and transfer once every 100 steps, or use a framework like wandb with asynchronous logging.
- **Use PyTorch Profiler first; escalate to Nsight Systems when you need CPU/GPU timeline alignment.** The Profiler answers “which operation is slow?”. Nsight Systems answers “is the GPU idle, and why?”. They are complementary. Start with the fast tool.
- **When you see different CUDA kernels for different matrix shapes, benchmark both shapes separately.** PyTorch dispatches to different Cutlass kernels based on matrix dimensions (and hardware). A matmul that is fast at 4096×4096 may dispatch to a less optimal path at 512×512 . Profiling reveals this; benchmarking only the happy-path size hides it.
- **Profile with warm-up excluded using the schedule API:**

```
prof = torch.profiler.profile(
    schedule=torch.profiler.schedule(
        wait=2, warmup=2, active=3),
    on_trace_ready=torch.profiler.tensorboard_trace_handler('./log'))
```

This skips the first two steps (JIT), warms up for two steps, and records three steps. Without this, the profiler captures JIT compilation time which dominates and masks the real bottleneck.

How This Shows Up in Interviews

- **“Training throughput is lower than expected. How do you determine whether the bottleneck is CPU or GPU?”** Use Nsight Systems. Look at the GPU timeline: if there are gaps (periods where no kernel is running), the GPU is idle, and the bottleneck is CPU dispatch rate - Python is not queuing kernels

fast enough. If the GPU is continuously running kernels but each takes too long, the bottleneck is GPU computation. In a healthy system, the CPU runs one full step ahead and the GPU never goes idle.

- **“How would you use the PyTorch Profiler to identify which operation to optimize in an MLP?”**
Wrap the training step in `torch.profiler.profile` with CPU and CUDA activities enabled. Call `key_averages().table(sort_by="cuda_time_total")` to rank operations by total GPU time. The top entry (usually `aten::mm`) is where to focus. A 5-minute profiler run saves 5 hours of optimizing the wrong operation.
- **Senior follow-up: “A profiler shows 60% of GPU time is in `CudaLaunchKernel` (the kernel launch overhead) rather than in actual computation kernels. What causes this and how do you fix it?”** This happens when the model has many small operations instead of a few large ones. Example: an unfused GELU implemented as 6 separate element-wise operations dispatches 6 kernels; each kernel launch takes 5-20 μ s. At 100 GPU kernels per step, that is 0.5-2 ms of pure launch overhead per step. In a fast model with 50 ms steps, launch overhead is 1-4% of runtime. In a very fast model with 3 ms steps, it is 17-67%. The fixes: (1) `torch.compile` fuses element-wise chains automatically and eliminates most of this. (2) Write a Triton or CUDA kernel that fuses the compound operation. (3) Increase batch size or hidden dimension so each kernel has more useful work to amortize the fixed launch cost.

Key Takeaways

- The PyTorch Profiler reveals which operations dominate GPU time and which CUDA kernel is dispatched per operation; use it first, as it requires no external tools.
- Nsight Systems shows the GPU and CPU timelines side by side: CPU-ahead scheduling is healthy; a CPU/GPU timeline that is perfectly aligned indicates a synchronization barrier (usually a `.item()` call) that serializes what should be asynchronous execution.
- A single `print(loss.item())` per step can double step time by forcing a CUDA synchronize at every iteration, eliminating all CPU-ahead scheduling benefit.
- Different matrix shapes dispatch to different CUDA kernels; only a profiler reveals this; a single benchmark number hides kernel selection differences.
- Add NVTX annotations (`nvtx.range_push/pop`) to training loops before profiling; they cost microseconds and make traces immediately readable.

10.2 Implementing Kernels

10.2.1 A Minimal Custom CUDA Kernel (GeLU Case Study)

Your new architecture uses a custom activation function. The naive PyTorch implementation dispatches six CUDA kernels and runs at 8.1 ms. PyTorch’s built-in fused version runs at 1.1 ms. Writing a minimal custom CUDA kernel in C++ fuses all operations into a single kernel launch and cuts runtime to 1.8 ms - an 80% improvement without changing any math. Every ML infrastructure role at a frontier lab expects you to be able to sketch this pattern, explain why the fusion matters, and know where the remaining gap to PyTorch’s implementation comes from.

A CUDA kernel is a C function marked `__global__` that the GPU executes on thousands of threads simultaneously. Each thread computes its global index and handles one (or a few) elements of your data. The

CPU side is a thin wrapper that validates inputs, allocates the output, computes grid dimensions, and fires off the kernel.

Every CUDA kernel for element-wise operations follows the same skeleton:

```
// GPU side: runs on device
__global__ void gelu_kernel(const float* in, float* out, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) {
        float x = in[i];
        float y = 0.5f * x * (1.0f + tanhf(
            sqrtf(2.0f / M_PI) * (x + 0.044715f * x * x * x)));
        out[i] = y;
    }
}

// CPU side: orchestrates the launch
torch::Tensor gelu_cuda(torch::Tensor x) {
    TORCH_CHECK(x.device().is_cuda(), "input must be on GPU");
    TORCH_CHECK(x.is_contiguous(), "input must be contiguous");
    auto y = torch::empty_like(x);          // not zeros_like
    int n = x.numel();
    int blocksize = 1024;
    int num_blocks = (n + blocksize - 1) / blocksize; // ceiling division
    gelu_kernel<<<num_blocks, blocksize>>>(
        x.data_ptr<float>(), y.data_ptr<float>(), n);
    return y;
}
```

The GeLU approximation being computed is:

$$\text{GeLU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715 x^3) \right) \right).$$

The naive PyTorch implementation of this formula dispatches six separate CUDA kernels (one each for x^3 , a scale, an add, a tanh, a multiply, and a final scale). Each kernel reads and writes the full tensor to HBM. The custom kernel reads once and writes once. That is the entire reason for the speedup.

*A CUDA kernel is just a C function in which each thread computes its global index $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$, does a bounds check, and performs its element's computation - everything else is ordinary C.*

Kernel fusion eliminates redundant HBM traffic. For a 64M-element float32 tensor (256 MB), each HBM round-trip costs roughly $256 \text{ MB} / 2 \text{ TB/s} \approx 128 \mu\text{s}$, plus a kernel launch overhead of $\sim 10\text{-}50 \mu\text{s}$. Six round-trips: $\sim 768 \mu\text{s}$ minimum bandwidth cost plus six launch overheads. The fused kernel: one read, one write, one launch - the minimum possible HBM traffic.

The contrast with `torch.compile` is instructive: the JIT compiler achieves 1.47 ms on the same operation (vs. our 1.8 ms) by generating a Triton kernel with vectorized loads (4 float32 values per thread instead of 1) and better inlining. For simple element-wise fusions, the compiler wins on convenience and nearly wins on

performance. The custom CUDA kernel makes sense when (a) the compiler cannot discover the right fusion pattern, or (b) you need hardware-specific intrinsics not exposed through Triton.

The remaining 0.7 ms gap between our kernel (1.8 ms) and PyTorch's (1.1 ms) comes from two sources: our kernel processes one float32 element per thread ($2 \times$ HBM bytes vs. bfloat16), and it lacks the vector load instructions (float4) that allow loading 4 values per transaction and better utilizing the HBM burst width.

A Worked Example

Input: `x.shape = (64_000_000,)`, float32, on A100. `blocksize = 1024`.

Grid computation.

$$\text{num_blocks} = \left\lceil \frac{64,000,000}{1024} \right\rceil = 62,500.$$

Total threads: $62,500 \times 1024 = 64,000,000$. Each thread covers exactly one element.

Thread $i = 1,025$ (thread index 1 in block 1):

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x} = 1 \times 1024 + 1 = 1025.$$

Bounds check: $1025 < 64,000,000$ - proceed. Read in[1025], compute GeLU, write out [1025].

Last thread in the last block (thread index 511 in block 62499):

$$i = 62,499 \times 1024 + 511 = 63,999,487 < 64,000,000 \rightarrow \text{proceed.}$$

First out-of-bounds thread (thread index 512 in block 62499):

$$i = 62,499 \times 1024 + 512 = 63,999,488 < 64,000,000 \rightarrow \text{proceed.}$$

All threads in block 62499 are in bounds here because $62499 \times 1024 = 63,998,976$ and $63,998,976 + 1023 = 63,999,999 < 64,000,000$. Good: no waste. If the input had 64,000,001 elements, we would need 62,501 blocks, and block 62,500 would have 1023 masked threads doing nothing.

HBM traffic: $64\text{M} \times 4$ bytes read + $64\text{M} \times 4$ bytes write = 512 MB. At 2 TB/s: $512 \text{ MB} / 2 \text{ TB/s} = 0.26$ ms minimum. Measured: 1.8 ms - the gap is kernel launch overhead, register allocation, and the A100's actual compute time for the transcendentals (`tanhf`, `sqrthf`).

What You Actually Decide in Practice

- **Use `empty_like`, never `zeros_like`, for output allocation.** `zeros_like` zeros every element before you write to it - an extra HBM round-trip for a tensor you are about to overwrite. For 256 MB this costs $\sim 128 \mu\text{s}$. Use `empty_like` when you will write every output element.
- **Always check device and contiguousness at the wrapper level.** A tensor that has been transposed or sliced may be non-contiguous: its elements are not laid out consecutively in memory, so `in[i]` does not give you element i . Assert `x.is_contiguous()` and call `x.contiguous()` if needed. Failing to do this silently reads the wrong memory.
- **Set `CUDA_LAUNCH_BLOCKING=1` during development.** Without it, CUDA errors are deferred and surface in unrelated calls, making the error message point to the wrong line. With it, errors appear immediately at the kernel launch. Remove it before benchmarking (it adds 5-15% overhead).
- **`blocksize = 1024` is a safe default for element-wise kernels.** An A100 SM supports up to 2048 threads per SM (64 warps). With `blocksize = 1024` (32 warps per block), two blocks fit per SM, giving 64 active warps - full capacity. If register pressure is high (many temporaries), reduce to 512.

- **Try `torch.compile` before writing any C++.** For the GeLU case, `torch.compile` delivers 1.47 ms with zero kernel code. Write a custom CUDA kernel only when the compiler misses a fusion (check the profiler), when you need hardware intrinsics (bfloat16 tensorcore ops, H100-specific warpgroup instructions), or when Triton cannot express the required operation.

How This Shows Up in Interviews

- **“Walk me through how you would implement a custom activation function as a fused CUDA kernel.”** Describe the two-part structure: (1) CPU wrapper - device check, contiguous check, `empty_like` allocation, grid computation, kernel launch. (2) GPU kernel - compute global index, bounds check, compute activation, write output. Explain that fusion eliminates multiple HBM round-trips that the naive version incurs for each sub-operation.
- **“Why is the naive PyTorch GeLU 7× slower than the fused version?”** Six separate CUDA kernels, each reading and writing the full tensor to HBM. For a 256 MB tensor at 2 TB/s, each round-trip costs $\sim 128 \mu\text{s}$. Six round-trips plus six kernel launch overheads sum to ~ 8 ms. The fused kernel: one read, one write, ~ 0.26 ms minimum HBM cost.
- **Senior follow-up: “Your custom CUDA GeLU runs at 1.8 ms but PyTorch’s runs at 1.1 ms. What accounts for the remaining 39% gap and how do you close it?”** Two causes: (1) float32 vs. bfloat16 - float32 moves twice as many bytes per element; switching to bfloat16 halves HBM traffic and halves the minimum bandwidth cost. (2) Scalar vs. vectorized loads - PyTorch’s kernel loads 4 values per thread using `float4` memory instructions, fully utilizing the 128-byte HBM cache line per transaction. Fix: change the kernel to advance by `BLOCK_SIZE × 4`, use `float4` loads, and process 4 elements per thread. This brings the kernel within $\sim 10\%$ of PyTorch’s implementation. The Triton kernel already does this automatically.

Key Takeaways

- A CUDA kernel is a C function in which each thread computes `i = blockIdx.x * blockDim.x + threadIdx.x`, bounds-checks against `n_elements`, reads `in[i]`, computes, and writes `out[i]` - the entire pattern fits in ten lines of C.
- Kernel fusion eliminates HBM round-trips: naive GeLU (6 kernels) $\rightarrow 8.1$ ms; fused custom CUDA $\rightarrow 1.8$ ms; PyTorch fused $\rightarrow 1.1$ ms; `torch.compile` $\rightarrow 1.47$ ms.
- Use `empty_like` not `zeros_like`; assert device and contiguous; set `CUDA_LAUNCH_BLOCKING=1` during development.
- `torch.compile` handles element-wise fusion automatically (zero kernel code, ~ 1.47 ms for GeLU). Reserve custom CUDA/Triton for operations the compiler cannot discover or for hardware-specific intrinsics.
- The gap between hand-written and PyTorch’s implementation comes from scalar vs. vectorized loads and float32 vs. bfloat16 bandwidth: fixing both closes most of the remaining difference.

10.2.2 Triton: Block-Level Programming in Python

You need a fused softmax kernel for your inference server. Writing it in raw CUDA means tracking warp shuffle instructions, shared memory barrier placement, and hand-coded memory coalescing - easily a day of

brittle C++. Triton lets you write the same kernel in Python at the thread-block level, while the compiler handles coalescing, shared memory, and thread synchronization automatically. For most ML infrastructure engineers I have worked alongside, Triton is the practical answer to “I need a custom kernel”: it delivers CUDA-class performance with Python ergonomics and a debugger that actually works. At senior-level interviews for kernel-writing roles, interviewers expect you to sketch a Triton GeLU or softmax from memory and explain the block-centric mental model that drives the design.

Triton is a Python domain-specific language first released by OpenAI in 2021. The key conceptual shift from raw CUDA: you no longer think in *threads*, you think in *thread blocks*. In CUDA, each function invocation is a single thread computing one element. In Triton, each function invocation is one program instance - one thread block - operating on a contiguous *tile* of elements simultaneously. The Triton compiler maps that tile to individual threads and automatically generates:

- Coalesced memory access: adjacent tile elements map to adjacent threads, which map to adjacent addresses, fully utilizing the 128-byte HBM cache line per transaction.
- Vectorized loads: the PTX output uses 4-element load instructions (`ld.global.v4.f32`), moving 4 float32 values per thread per transaction without any programmer annotation.
- Masked boundary handling: `tl.load(ptr + offsets, mask=mask, other=0.0)` suppresses out-of-bounds accesses cleanly, replacing the manual `if (i < n)` that every CUDA kernel requires.

What Triton does *not* manage: scheduling across SMs. You choose the grid (number of blocks and block size), and the GPU dispatcher decides placement. This is the one degree of freedom you retain.

A complete Triton GeLU kernel:

```
import triton
import triton.language as tl
import torch

@triton.jit
def gelu_kernel(x_ptr, y_ptr, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(axis=0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    c = tl.sqrt(2.0 / 3.141592653589793)
    inner = c * (x + 0.044715 * x * x * x)
    y = 0.5 * x * (1.0 + tl.libdevice.tanh(inner))
    tl.store(y_ptr + offsets, y, mask=mask)

def triton_gelu(x: torch.Tensor) -> torch.Tensor:
    assert x.is_cuda and x.is_contiguous()
    y = torch.empty_like(x)
    n = x.numel()
    BLOCK_SIZE = 1024
    grid = ((n + BLOCK_SIZE - 1) // BLOCK_SIZE,)
    gelu_kernel[grid](x, y, n, BLOCK_SIZE=BLOCK_SIZE)
```

```
return y
```

Compare this to the raw CUDA version from the previous subsection: the element-wise computation is identical, but `tl.arange(0, BLOCK_SIZE)` replaces the per-thread index arithmetic, and `mask=mask` replaces the explicit bounds check. The result: 1.85 ms runtime versus 1.80 ms for hand-written CUDA, with Python code that a debugger can step through.

Triton is CUDA where you write the block and the compiler writes the threads.

Against raw CUDA, Triton wins on development speed and correctness. Memory coalescing bugs are among the most common and hardest-to-diagnose sources of GPU underperformance in hand-written kernels. Triton makes them structurally impossible: the `tl.arange` pattern guarantees adjacent offsets for adjacent threads, which guarantees coalesced access. Shared memory management and warp barriers are generated by the compiler, not by the programmer.

Against `torch.compile`, Triton wins when the JIT cannot infer the right fusion pattern. For simple element-wise ops like GeLU, `torch.compile` already generates Triton internally and achieves 1.47 ms with no kernel code at all. The case for hand-written Triton is algorithmic novelty: online reductions over tiles, non-standard attention masking patterns, fused quantization and dequantization, or any computation where the compiler would need to understand the mathematical structure of your operation to fuse it correctly.

The trade-off Triton does not eliminate is arithmetic intensity. A GeLU is memory-bound regardless of whether it is written in Python or C++. Fusion cuts HBM round-trips from six to one - that is the large win. No amount of Triton tuning changes the roofline for an inherently bandwidth-limited kernel.

A Worked Example

Row-wise softmax over a 4096×4096 float32 matrix on an A100 (HBM bandwidth: 2 TB/s). The natural Triton design assigns one block per row, loading the entire row into an SM in a single vectorized pass.

Grid. One block per row, block size padded to the next power of two:

```
BLOCK_SIZE = 4096, num_blocks = 4096.
```

Kernel:

```
@triton.jit
def softmax_kernel(x_ptr, y_ptr, x_stride, y_stride,
                  n_cols, BLOCK_SIZE: tl.constexpr):
    row = tl.program_id(axis=0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    x = tl.load(x_ptr + row * x_stride + cols,
               mask=mask, other=-float('inf'))
    x = x - tl.max(x, axis=0) # numerical stability
    x_exp = tl.exp(x)
    y = x_exp / tl.sum(x_exp, axis=0)
    tl.store(y_ptr + row * y_stride + cols, y, mask=mask)
```

HBM traffic. Each row is read once (16 KB) and written once:

$$2 \times 4096 \times 4096 \times 4 \text{ bytes} = 128 \text{ MB.}$$

At 2 TB/s: $128 \text{ MB} / 2 \text{ TB/s} = 64 \mu\text{s}$ minimum. Measured: $\sim 1.9 \text{ ms}$. The gap comes from the sequential dependency within each row - max must complete before exp can begin, exp must complete before sum, sum before normalization - so each block has an irreducible sequential fraction that no amount of parallelism can eliminate.

Comparison across implementations:

- Manual (6-kernel PyTorch decomposition): 3.7 ms (five HBM round-trips).
- This Triton kernel: 1.9 ms (one read, one write, sequential within-block reduction).
- PyTorch built-in softmax: 1.5 ms (fused CUDA kernel with online normalization).
- `torch.compile`: 1.3 ms (Triton + online normalization, shape-specialized).

The 0.6 ms gap between this kernel and `torch.compile` closes with online (one-pass) softmax - the topic of the FlashAttention section that follows.

What You Actually Decide in Practice

- **Try `torch.compile` before writing any Triton.** For element-wise ops and standard reductions, the JIT generates a fused Triton kernel automatically. Profile with `torch.profiler` first: if you see a single `triton_` kernel in the CUDA trace, the compiler already did the work.
- **Declare `BLOCK_SIZE`:** `tl.constexpr`. This tells the Triton compiler the block size is a compile-time constant, enabling loop unrolling and better register allocation. Passing `BLOCK_SIZE` as a runtime integer disables these optimizations silently.
- **Choose block size as a power of two.** Triton's `tl.arange` requires power-of-two sizes. For row-wise operations, set `BLOCK_SIZE` to the next power of two above your column count. If a row does not fit in SRAM ($> 96 \text{ KB}$ shared memory per SM on an A100), tile across the row in a loop instead.
- **Always subtract the max before exponentiating in reductions.** The line `x = x - tl.max(x, axis=0)` is not optional. Without it, logits in the hundreds produce `inf` from `tl.exp`, which propagates to NaN in the division. This is one of the easiest ways to silently break a custom attention kernel.
- **Reserve raw CUDA for hardware-specific intrinsics.** FlashAttention-3's gains on H100 come from warpgroup matrix-accumulate (WGMMMA) and asynchronous copy instructions that Triton's abstraction cannot yet express. For everything else - element-wise ops, reductions, tiled attention - Triton is within 5-10% of hand-written CUDA in my experience and is far less likely to contain silent memory bugs.

How This Shows Up in Interviews

- **“Sketch a fused softmax kernel in Triton.”** Describe the row-per-block grid. Walk through: `pid = tl.program_id(0)`, offset computation with `tl.arange`, masked load with `other=-inf`, in-register max subtraction for stability, `tl.exp`, `tl.sum`, normalization, masked store. Emphasize that coalescing and thread management are handled by the compiler.
- **“When would you use Triton instead of `torch.compile`?”** When the JIT cannot infer the right fusion: non-standard memory layouts, online reductions over tiles, fused quantization-dequantization, or custom attention masking patterns. `torch.compile` wins for standard element-wise ops and matmuls; Triton wins when you need to encode algorithmic knowledge the compiler does not have.
- **Senior follow-up: “Your Triton softmax runs at 1.9 ms but `torch.compile` achieves 1.3 ms. What accounts for the gap and how do you close it?”** The naive two-pass softmax (pass 1: find max; pass 2: exp and normalize) reads the row twice. Online softmax reduces this to a single pass: maintain running

maximum m and running sum d across tiles, and at each new tile apply the correction $e^{m_{\text{prev}} - m_{\text{new}}}$ to the accumulated sum before adding new terms. This halves HBM reads and is the foundational technique in FlashAttention's tiled attention kernel. Implementing it in Triton requires tracking (m, d) as scalar accumulators across a loop over column tiles.

Key Takeaways

- Triton programs in blocks: `pid = tl.program_id(0)` gives the block index, `tl.arange(0, BLOCK_SIZE)` gives the tile, `mask = offsets < n` handles boundaries - three lines replace all thread-level index arithmetic.
- The compiler generates coalesced vectorized loads automatically, moving 4 float32 values per thread per transaction without programmer annotation.
- Performance target: Triton GeLU ≈ 1.85 ms; hand-written CUDA ≈ 1.8 ms; `torch.compile` ≈ 1.47 ms. Start with `torch.compile`; write Triton when the JIT cannot find the pattern.
- Always declare `BLOCK_SIZE`: `tl.constexpr`; always subtract the row max before `tl.exp`; choose block sizes as powers of two.
- The gap between a two-pass Triton softmax and `torch.compile`'s softmax is closed by online (one-pass) normalization - the technique that makes FlashAttention's tiled attention kernel viable.

10.3 FlashAttention as a Case Study

10.3.1 Online Softmax and Tile-by-Tile Attention

When someone asks you to design an LLM serving system for 128K-token context, the first cost you should compute is how much HBM the attention operation alone consumes. Standard attention materializes an $N \times N$ score matrix in GPU global memory. At $N = 8,192$, 32 heads, and bf16 storage, that matrix occupies over 4 GB of HBM per layer-before activations, KV cache, or gradients. FlashAttention eliminates that $O(N^2)$ allocation by computing attention entirely within SRAM-resident tiles and streaming K and V from HBM exactly once. This topic surfaces in almost every senior LLM systems interview that touches context scaling, memory efficiency, or custom kernel design; understanding it at the algorithm level, not just by name, is what distinguishes a strong candidate.

The root challenge is that softmax is a global row-wise operation. To normalize row i of the attention score matrix $S = QK^T / \sqrt{d}$, you need the denominator $\sum_j e^{S_{ij} - m_i}$ where $m_i = \max_j S_{ij}$. A naïve implementation therefore writes S to HBM, reads it back to compute the row maxima, writes $P = \text{softmax}(S)$, and reads P again to compute PV -four HBM round-trips that scale as $O(N^2)$ in both time and space.

The *online softmax* algorithm eliminates this by maintaining two running scalars per query row as K tiles stream in: a running maximum m and a running normalizer ℓ . When tile j arrives with block-level statistics $\hat{m}_j = \text{rowmax}(S_j)$:

$$m' = \max(m, \hat{m}_j) \quad (10.1)$$

$$\ell' = \ell \cdot e^{m - m'} + \text{rowsum}\left(e^{S_j - m'}\right) \quad (10.2)$$

The correction factor $e^{m - m'}$ rescales the accumulated sum whenever the running maximum updates. The output accumulator O follows the same pattern:

$$O' = e^{m-m'} \cdot O + e^{S_j-m'} V_j$$

After processing all $T_c = N/B_c$ KV tiles, divide O by ℓ to obtain the correctly normalized output. FlashAttention-2 structures the outer loop over query tiles of size B_r and the inner loop over KV tiles of size B_c , dispatching one warp group per query tile so that each SM processes an independent slice of the sequence.

Online softmax converts a global normalization into a sequence of SRAM-resident corrections, enabling single-pass attention whose HBM footprint scales as $O(Nd)$ rather than $O(N^2)$.

Naïve attention on an H100 is memory-bound, not compute-bound. At $N = 4,096$, 32 heads, and bf16, the attention score matrix alone requires ≈ 4.2 GB of HBM writes and reads per forward pass per layer. At the H100's 3.35 TB/s HBM bandwidth ceiling, those transfers consume ≈ 1.25 ms per layer before any compute executes. The compute for the same attention operation at peak bf16 throughput of 1,979 TFLOP/s takes under 0.2 ms—so the bottleneck is entirely in HBM traffic.

FlashAttention keeps Q , K , V , the score tile S , and the output accumulator O tile-resident in the ≈ 228 KB of shared memory available per SM on an H100. The trade-off is SRAM scarcity: all five tiles must fit simultaneously, which constrains the block sizes B_r and B_c . Increasing B_r reduces the tile-loop trip count and improves compute intensity, but reduces the number of concurrent blocks that can co-reside on the SM—so occupancy and compute intensity pull in opposite directions, and the right setting depends on the specific d and hardware.

Compared with an unfused implementation, FlashAttention also eliminates multiple kernel launches—each of which would flush intermediate results to HBM. Operator fusion alone removes two of the four HBM passes; tiling removes the remaining quadratic dependence on N .

A Worked Example

Model: 7B-parameter decoder, $d_{\text{head}} = 128$, $H = 32$ heads, $N = 4,096$, bf16. Block sizes $B_r = B_c = 64$.

SRAM footprint per SM during one outer tile:

- Q tile: $64 \times 128 \times 2 = 16$ KB
- K tile: $64 \times 128 \times 2 = 16$ KB
- V tile: $64 \times 128 \times 2 = 16$ KB
- Score tile S : $64 \times 64 \times 2 = 8$ KB
- Output tile O : $64 \times 128 \times 2 = 16$ KB
- Running stats m, ℓ : $64 \times 4 \times 2 \approx 0.5$ KB
- Total: ≈ 73 KB - comfortably within the 228 KB shared memory limit.

HBM traffic comparison per head, one forward pass:

- *Naïve attention*: write S (32 MB), read S for softmax (32 MB), write P (32 MB), read P for PV (32 MB), plus Q , K , V reads and O write (≈ 4 MB). Total ≈ 132 MB per head.
- *FlashAttention*: Q read once (1 MB), K and V each read once per outer tile (1 MB each), O written once (1 MB). Total ≈ 4 MB per head.

Across 32 heads: naïve attention moves ≈ 4.2 GB of HBM per layer per forward pass; FlashAttention moves ≈ 128 MB—a $33\times$ reduction. At 3.35 TB/s, the naïve HBM transfers cost ≈ 1.25 ms per layer. FlashAttention reduces that cost to ≈ 0.04 ms.

What You Actually Decide in Practice

- **Block size is the primary tuning knob.** For $d = 128$, start with $B_r = B_c = 64$ and benchmark $B_r = B_c = 128$. The larger block reduces the inner-loop trip count by $4\times$ and improves arithmetic intensity, but doubles SRAM usage and reduces SM occupancy. Benchmark both; on H100 with $d = 128$ the $B_r = B_c = 128$ configuration often wins, but the gap closes for shorter sequences.
- **Align head dimensions to powers of two.** A model with 20 heads over a 1,600-wide hidden layer gives $d_{\text{head}} = 80$. Pad to 128 inside the kernel. Unaligned dimensions produce non-coalesced HBM loads that partially cancel the bandwidth savings.
- **Do not rewrite standard FlashAttention yourself.** `torch.nn.functional.scaled_dot_product_attention` with `enable_flash_sdp(True)` dispatches to a tuned FlashAttention-2 implementation. Write a custom Triton kernel only for non-standard masking patterns, hardware-specific pipelining (e.g., H100 TMA), or structurally novel attention variants.
- **Causal masking adds no HBM traffic.** KV tiles whose column range lies entirely above the diagonal can be skipped. Mixed tiles apply the causal mask to the score tile in SRAM before the softmax update. No additional memory movement.
- **Always handle the final partial tile.** Sequences not divisible by B_c require masked loads. Forgetting boundary conditions is the most common correctness bug in custom FlashAttention kernels-the results will be wrong for the last few positions in the sequence without triggering a visible error.

How This Shows Up in Interviews

- **“Why does naïve attention become slow at long context?”** Strong answer: the $N \times N$ score matrix requires at minimum four HBM round-trips. At $N = 8,192$ and 32 heads in bf16, that is ≈ 4.3 GB per layer per forward pass-fully bandwidth-limited, costing ≈ 1.3 ms per layer before any compute. FlashAttention tiles K and V through SRAM, brings HBM traffic down to $O(Nd)$, and makes the operation compute-bound.
- **“Walk me through one tile of the FlashAttention forward pass.”** Strong answer: load Q_i, K_j, V_j into SRAM; compute $S_{ij} = Q_i K_j^T / \sqrt{d}$; update running max $m' = \max(m, \text{rowmax}(S_{ij}))$; rescale running normalizer $\ell' = \ell e^{m-m'} + \text{rowsum}(e^{S_{ij}-m'})$; update output accumulator $O' = e^{m-m'} O + e^{S_{ij}-m'} V_j$; advance to the next KV tile. After all tiles, return O/ℓ .
- **Senior follow-up: “FlashAttention-3 adds warp-specialization and uses H100 TMA. What problem does that solve and why is it H100-specific?”** The remaining bottleneck in FlashAttention-2 on H100 is that the online-softmax update in registers stalls the next GEMM from issuing. H100’s WGMMMA instruction and Tensor Memory Accelerator allow one warp group to issue an asynchronous GEMM while another runs the softmax update and prefetches the next KV tile via TMA DMA. A100 lacks both WGMMMA and TMA, so implementing producer-consumer pipelining requires explicit barriers that consume most of the latency benefit.

Key Takeaways

- FlashAttention’s core contribution is a data-movement strategy: compute attention entirely in SRAM-resident tiles so the $N \times N$ score matrix never writes to HBM.
- The online softmax update-maintaining running max m and sum ℓ , correcting both by $e^{m_{\text{old}}-m_{\text{new}}}$ on each

tile-enables single-pass normalization without materializing the full score matrix.

- HBM traffic falls from $O(N^2)$ per head (naïve) to $O(Nd)$ per head (FlashAttention), a factor-of- N/d reduction that scales linearly with sequence length.
- Block size $B_r \times B_c$ is the primary tuning parameter: larger blocks improve arithmetic intensity but must fit in SRAM; benchmark both 64 and 128 for your head dimension.
- In production, use `scaled_dot_product_attention` with the FlashAttention backend; write a custom kernel only for hardware-specific pipelining or structurally novel masking.

10.3.2 Backward-Pass Recomputation in FlashAttention

If you have trained a long-context model and watched the forward pass succeed at batch size 4 while the backward pass throws a CUDA out-of-memory error at the same configuration, the most likely culprit is the stored attention probability matrix. Standard autograd saves $P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$ per head per layer because the softmax backward requires it. At $N = 4,096$, 32 heads, and bf16 over 32 layers, that storage alone consumes ≈ 34 GB-enough to OOM a single 80 GB H100 before accounting for weights, optimizer states, or KV cache. FlashAttention’s backward pass discards P immediately after the forward tile completes and recomputes it tile by tile from Q, K, V during the backward pass. Understanding why that recomputation is faster than loading the stored matrix-not just why it saves memory-is the answer that separates a senior response from a mid-level one.

The softmax backward formula for attention row i is:

$$\frac{\partial \mathcal{L}}{\partial S_{ij}} = P_{ij} \left(\frac{\partial \mathcal{L}}{\partial P_{ij}} - \sum_k P_{ik} \frac{\partial \mathcal{L}}{\partial P_{ik}} \right)$$

In attention, $\partial \mathcal{L} / \partial P_{ij} = dO_i V_j^T$. The row-wise contraction simplifies to a per-query-position scalar D_i :

$$D_i = \sum_k dO_{ik} O_{ik} = \langle dO_i, O_i \rangle$$

Substituting gives the final expression for the score gradient:

$$dS_{ij} = P_{ij} \odot (dO_i V_j^T - D_i)$$

D_i is a vector of length N per head (one scalar per query position) that can be computed in a single pass through dO and O before the tiled backward loop begins.

The key ingredient from the forward pass is not P itself but the *log-sum-exp* statistic $L_i = m_i + \log \ell_i$ per query row-a vector of length N per head stored in fp32. Using L_i , any tile of P can be reconstructed as $P_{ij} = e^{S_{ij} - L_i}$, where L_i is broadcast row-wise. The backward tile loop then proceeds entirely in SRAM:

1. Load Q_i, K_j, V_j from HBM into SRAM.
2. Recompute $S_{ij} = Q_i K_j^T / \sqrt{d}$; compute $P_{ij} = e^{S_{ij} - L_i}$ (row-wise using stored L_i).
3. $dV_j \leftarrow dV_j + P_{ij}^T dO_i$
4. $dP_{ij} = dO_i V_j^T$; $dS_{ij} = P_{ij} \odot (dP_{ij} - D_i)$
5. $dQ_i \leftarrow dQ_i + dS_{ij} K_j$; $dK_j \leftarrow dK_j + dS_{ij}^T Q_i$

Storing only $L_i \in \mathbb{R}^N$ per head from the forward pass and recomputing P tile by tile during the backward reduces attention activation memory from $O(N^2)$ to $O(N)$ -an exchange that consistently wins on hardware where compute throughput scales much faster than HBM bandwidth.

On an H100, bf16 matmul throughput peaks at $\approx 1,979$ TFLOP/s and HBM bandwidth peaks at ≈ 3.35 TB/s. Recomputing one tile of S_{ij} of shape $B_r \times B_c$ with head dimension d adds $2B_r B_c d$ FLOPs and avoids reading $B_r B_c \times 2$ bytes from HBM. The arithmetic intensity of this tile recomputation is:

$$\frac{2B_r B_c d}{2(B_r + B_c)d} = \frac{B_r B_c}{B_r + B_c}$$

For $B_r = B_c = 64$ and $d = 128$, this is 32 FLOP/byte—below the H100 ridge point of ≈ 590 FLOP/byte, so the recomputed tile is memory-bound. The critical comparison, however, is not against the hardware ceiling; it is against the alternative of *loading* a precomputed tile from HBM. Loading P_{ij} costs the same $B_r B_c \times 2$ bytes at the same bandwidth, but adds zero FLOPs—the compute units sit idle. With recomputation, those same memory cycles are accompanied by useful arithmetic, and the FLOPs finish before the next HBM load arrives. The recomputation is effectively free in terms of wall-clock time, while eliminating the $O(N^2)$ activation storage.

Compared with gradient checkpointing at the layer level, FlashAttention recomputation is far more targeted. Layer-level checkpointing re-runs the full forward pass—MLP, normalization, attention alike. FlashAttention recomputes only the tile-level QK^T multiply and the pointwise softmax during the backward pass, a fraction of layer compute, executed at near-optimal HBM utilization.

A Worked Example

Model: 7B-parameter decoder, $d_{\text{head}} = 128$, $H = 32$ heads, $N = 4,096$, bf16 training with fp32 gradient accumulation. Block sizes $B_r = B_c = 64$.

Memory cost of storing P per layer:

$$\text{size}(P) = N^2 \times 2 \text{ bytes} \times H = 4096^2 \times 2 \times 32 \approx 1.07 \text{ GB}$$

Across 32 layers: ≈ 34 GB per training step—exceeding available HBM once weights, optimizer states, and remaining activations are included.

Memory cost of the FlashAttention approach per layer:

$$\text{size}(L) = N \times 4 \text{ bytes (fp32)} \times H = 4096 \times 4 \times 32 = 512 \text{ KB}$$

Across 32 layers: ≈ 16 MB—a reduction of roughly $2,000\times$.

Extra compute per layer from backward recomputation:

The backward pass recomputes QK^T for all $H \times (N/B_r) \times (N/B_c)$ tiles:

$$2N^2 d \times H = 2 \times 4096^2 \times 128 \times 32 \approx 137 \text{ GFLOPs}$$

At 1,979 TFLOP/s: $137 \text{ GF} / 1,979 \text{ TF/s} \approx 0.07 \text{ ms per layer}$.

HBM time saved per layer by not loading stored P during backward:

$$1.07 \text{ GB} / 3.35 \text{ TB/s} \approx 0.32 \text{ ms per layer}$$

Net per-layer saving: $0.32 - 0.07 = 0.25 \text{ ms}$ of backward-pass time per layer, plus the $2,000\times$ activation memory reduction that makes the backward pass feasible at sequence lengths that would otherwise OOM.

What You Actually Decide in Practice

- **Prefer FlashAttention recomputation over layer-level gradient checkpointing at the attention operation.** Checkpointing at the layer boundary re-runs the full forward pass—MLP and normalization included. FlashAttention recomputes only the tile-level GEMM, which is already close to bandwidth-optimal and costs far less than full-layer recomputation.

- **Always save L_i in fp32 regardless of training precision.** The log-sum-exp statistic is used as a subtraction offset $e^{S_{ij}-L_i}$ during backward. In bf16, rounding error in L_i accumulates across tiles and produces gradient drift that is difficult to diagnose. The reference FlashAttention-2 implementation enforces fp32 storage for L_i unconditionally.
- **Precompute D_i before the tiled backward loop.** $D_i = \langle dO_i, O_i \rangle$ requires one pass through dO and O , both $O(Nd)$ tensors already in HBM. Computing D_i inside the tile loop re-reads O on every iteration, multiplying HBM traffic by $T_c = N/B_c$ -a $64\times$ overhead for $N = 4,096$ and $B_c = 64$.
- **Backward tile ordering affects gradient accumulation patterns.** dK_j and dV_j accumulate contributions from all N/B_r query tiles. FlashAttention-2 loops over query tiles in the inner loop for the K and V gradient pass to avoid atomics, at the cost of loading K_j and V_j once per outer query tile rather than once per KV tile. This increases HBM reads for K and V during the backward pass. Profile the atomic versus non-atomic variants on your hardware-the winner depends on SM count and sequence length.
- **Sequence parallelism introduces an asymmetric all-reduce.** With ring-attention or Ulysses-style sequence parallelism, each device holds a shard of query positions and produces partial dK and dV gradients. An all-reduce across devices is required before the optimizer step for dK and dV ; dQ accumulates locally and needs no communication. Schedule the all-reduce to overlap with the next micro-batch forward pass-it is frequently on the critical path for long-context training.

How This Shows Up in Interviews

- **“Why does FlashAttention save memory during training, not just inference?”** Strong answer: the backward pass for standard attention requires the stored $N \times N$ probability matrix P . FlashAttention stores only $L_i \in \mathbb{R}^N$ per head (the log-sum-exp) and recomputes P tile by tile during the backward pass-reducing attention activation memory from $O(N^2)$ to $O(N)$, a $2,000\times$ reduction at $N = 4,096$, $H = 32$.
- **“Isn’t recomputing part of the forward pass during the backward pass expensive?”** Strong answer: no, because the alternative-loading the precomputed tile from HBM-costs the same bandwidth with zero compute. The recomputed tile’s FLOPs execute while the GPU waits for the next HBM load, so they overlap rather than stack. Per layer, the extra compute costs ≈ 0.07 ms while the saved HBM transfer is worth ≈ 0.32 ms-a net 0.25 ms saving per layer, plus the activation memory reduction that enables the backward pass to run at all.
- **Senior follow-up: “How does FlashAttention’s backward interact with sequence parallelism, and what communication is required?”** With ring attention, each device processes a contiguous shard of queries and accumulates partial dK and dV from remote KV tiles that are passed around the ring. After the ring completes, each device holds the full dQ for its query shard (no communication needed) but only a partial dK and dV (an all-reduce is required). The senior answer identifies this asymmetry, notes that the all-reduce volume equals $2 \times \text{num_heads} \times N \times d$ bytes-the same scale as the KV weight gradients-and discusses overlapping it with the next micro-batch’s forward pass ring to hide latency.

Key Takeaways

- FlashAttention discards P after each forward tile and stores only $L_i \in \mathbb{R}^N$ per head (the log-sum-exp); the $N \times N$ attention matrix is recomputed tile by tile during the backward pass.

- Attention activation memory drops from $O(N^2)$ to $O(N)$ per head-at $N = 4,096$ and 32 heads over 32 layers, this saves ≈ 34 GB per training step.
- Recomputation overhead is small: extra FLOPs execute during HBM-wait cycles, adding ≈ 0.07 ms per layer while saving ≈ 0.32 ms of HBM transfer time.
- Always store L_i in fp32 to prevent gradient accumulation errors from bf16 rounding in the log-sum-exp offset.
- With sequence parallelism, dK and dV require an all-reduce while dQ does not; overlap the all-reduce with the next micro-batch's forward ring to keep it off the critical path.

Part V

Distributed Training: Parallelism at Scale

Chapter 11 Multi-GPU and Multi-Node Fundamentals

11.1 Interconnect Topologies

11.1.1 PCIe, NVLink, and NVSwitch

Every parallelism strategy you design for a real training run starts with one question: which GPUs can afford to communicate with each other? That question is answered by the interconnect fabric, not your algorithm. Engineers at the mid-to-senior level are expected to know the three layers of intra-node connectivity-PCIe, NVLink, and NVSwitch-and to justify choices like “tensor parallel up to 8” or “allreduce is fast enough here” by pointing at concrete bandwidth numbers. If you answer that question with a vague reference to “fast GPU networking,” you have skipped the most performance-critical part of the system.

Three interconnect technologies govern how GPUs talk to each other inside a single machine, and they sit in a strict bandwidth hierarchy.

PCIe (Peripheral Component Interconnect Express) is the original path between CPU and GPU. A PCIe Gen 4 x16 slot delivers 32 GB/s unidirectional, or 64 GB/s bidirectional. Gen 5 doubles that to 128 GB/s bidirectional. This sounds reasonable until you note that an H100 reads from its own HBM at 3.35 TB/s-PCIe is roughly $26\times$ slower than the GPU’s own memory bus. GPU-to-GPU communication bridged through the CPU over PCIe rarely exceeds 30-40 GB/s effective and adds CPU-side synchronization latency.

NVLink is NVIDIA’s direct GPU-to-GPU interconnect, bypassing the CPU entirely. Each NVLink lane runs at far higher bandwidth than a PCIe lane. An H100 SXM5 GPU exposes 18 NVLink 4.0 lanes, delivering 900 GB/s of aggregate bidirectional bandwidth per GPU. An A100 SXM4 offers 600 GB/s per GPU via NVLink 3.0.

NVSwitch is the piece that makes NVLink scale beyond point-to-point links. Without it, NVLink connections are bilateral: GPU 0 might directly link to GPUs 1, 2, and 3, but not GPU 7. NVSwitch is a non-blocking crossbar chip-every pair of GPUs in the node sees the full NVLink bandwidth simultaneously, with no contention. A DGX H100 node ships with four NVSwitch 3.0 chips connecting all eight H100 GPUs, providing a full 900 GB/s all-to-all fabric.

The three levels together form a clear mental model:

$$\underbrace{128 \text{ GB/s}}_{\text{PCIe Gen 5}} \ll \underbrace{900 \text{ GB/s per GPU}}_{\text{NVLink 4.0}}, \text{ non-blocking via } \underbrace{\text{NVSwitch}}_{\text{crossbar}}$$

PCIe is the CPU highway, NVLink is the GPU express lane, and NVSwitch turns that lane into a non-blocking switchboard so every GPU can talk to every other GPU at full speed simultaneously.

The hierarchy exists because different operations have different bandwidth tolerance. CPU-GPU data transfers (loading a checkpoint, streaming host-side data) are infrequent and can tolerate PCIe latency. But collective operations like allreduce-which fire every training step across the tensor-parallel group-happen dozens of times per second and must complete in single-digit milliseconds to stay off the critical path of compute.

NVSwitch’s non-blocking crossbar is the enabling property for tensor parallelism. A tensor-parallel allreduce requires every GPU to simultaneously send and receive partial activation sums. A point-to-point NVLink ring would serialize transfers and create contention at intermediate hops; NVSwitch eliminates both. All eight GPUs in a node can participate in a single allreduce at near-peak wire bandwidth.

The alternative before NVSwitch was a hybrid cube-mesh NVLink topology where bandwidth between

non-directly-connected GPUs was lower and asymmetric. NVSwitch replaced that with a flat, predictable bandwidth guarantee between any two GPUs, making performance analysis far simpler and tensor-parallel scaling far more reliable.

A Worked Example

Consider a 70B-parameter model with hidden dimension $h = 8192$, trained with tensor parallelism across 8 H100 SXM5 GPUs in a single DGX H100 node. Batch size $b = 4$, sequence length $s = 2048$, BF16 precision.

At each transformer layer, the tensor-parallel allreduce synchronizes the output activations. The activation tensor has shape (b, s, h) with size:

$$4 \times 2048 \times 8192 \times 2 \text{ bytes} = 134 \text{ MB}$$

A ring allreduce over $n = 8$ GPUs transfers approximately $2 \times \frac{n-1}{n}$ times the data per GPU, so about 268 MB.

On NVSwitch at 900 GB/s bidirectional (effective allreduce bandwidth ≈ 450 GB/s per GPU after protocol overhead):

$$\frac{268 \text{ MB}}{450 \text{ GB/s}} \approx 0.6 \text{ ms}$$

On PCIe Gen 5 only (128 GB/s bidirectional, shared across all eight GPUs contending for the same CPU-side bandwidth):

$$\frac{268 \text{ MB}}{128 \text{ GB/s}/8} \approx 16.8 \text{ ms}$$

A large matmul in a single 70B transformer layer on an H100 takes roughly 1-3 ms. The NVSwitch-backed allreduce at 0.6 ms sits safely below compute and can be overlapped with the next layer's matrix multiply. The PCIe-backed allreduce at 17 ms is 5-17 $\times$ slower than the compute itself and completely dominates step time. Tensor parallelism becomes impractical.

What You Actually Decide in Practice

- **Cap tensor parallelism at 8 on current hardware.** Apply tensor parallelism only within a single NVSwitch domain-almost always 8 GPUs on DGX-class nodes. Extending to 16 or 32 requires crossing InfiniBand, where effective allreduce bandwidth collapses by 10-20 $\times$. Published benchmarks show a 42% throughput drop at TP=16 and a 65% drop at TP=32 relative to TP=8, on the same number of GPUs. These are not edge cases; they are the expected result of the bandwidth cliff.
- **Verify topology before assuming crossbar bandwidth.** Not all NVIDIA nodes ship with NVSwitch. RTX consumer cards have no NVLink at all. A100 and H100 PCIe variants have NVLink but no NVSwitch-bandwidth between non-directly-connected GPUs is lower and non-uniform. Always run `nvidia-smi topo -m` before assuming full crossbar performance. Two GPUs shown as connected via “SYS” (system-level) instead of “NVL” (NVLink) will communicate at PCIe speeds.
- **Budget allreduce cost explicitly.** For a full-model allreduce over P parameters in BF16, budget approximately $2P/B_{\text{eff}}$, where $B_{\text{eff}} \approx 0.5 \times B_{\text{peak NVLink}}$. For a 7B-parameter model: $2 \times 14 \text{ GB}/450 \text{ GB/s} \approx 62 \text{ ms}$. This is large enough to matter and must be overlapped with gradient computation in the backward pass rather than serialized after it.
- **When PCIe-only machines are all you have.** Use data parallelism with large micro-batches to amortize communication over more compute per synchronization. Avoid tensor parallelism entirely-the communication-

to-compute ratio is always unfavorable. Pipeline parallelism is permissible if point-to-point activation transfers (not allreduces) can fit within the PCIe budget.

- **NVSwitch failure handling.** A four-chip NVSwitch configuration has redundancy. If one chip fails, NCCL reroutes over the remaining three, dropping peak crossbar bandwidth by roughly 25%. Effective allreduce bandwidth can fall 20-30%. Monitor NVSwitch health via `dcmi` health checks before launching multi-day training runs.

How This Shows Up in Interviews

- “*Why do practitioners limit tensor parallelism to 8 GPUs?*” A strong answer names NVSwitch as the fast all-to-all fabric that makes layer-level allreduce cheap within a node, explains that crossing to a second node requires InfiniBand with 10-20 $\times$ lower bandwidth, and cites the 42% throughput loss at TP=16 as a concrete consequence. Weaker answers say “bandwidth” without quantifying it.
- “*Walk me through the bandwidth hierarchy inside a DGX H100 node.*” The answer covers three levels: PCIe Gen 5 at 128 GB/s bidirectional for CPU-GPU, NVLink 4.0 at 900 GB/s per GPU for GPU-GPU, and NVSwitch providing the non-blocking all-to-all crossbar. Mention that NVSwitch is what makes the bandwidth *non-blocking*-without it, GPU pairs that don’t share a direct NVLink lane contend.
- “*How would you measure the actual allreduce bandwidth available on a node?*” Run NCCL’s `all_reduce_perf` benchmark sweeping message sizes from 1 MB to 10 GB. Look for two regimes: latency-bound at small messages (below ~ 100 MB) and bandwidth-bound above. The peak bandwidth-bound number is what you should use for step-time estimates. Senior-level follow-up: “*What happens to allreduce performance if one NVSwitch chip in the node fails?*” The remaining three chips reroute traffic, effective crossbar bandwidth drops by $\sim 25\%$, and allreduce latency for large tensors rises proportionally. NCCL falls back to a partial ring and peak throughput may degrade 20-30%-enough to extend a 30-day training run by several days.

Key Takeaways

- PCIe Gen 5 delivers 128 GB/s bidirectional; NVLink 4.0 delivers 900 GB/s per GPU; NVSwitch makes that bandwidth non-blocking across all 8 GPUs in a node.
- Tensor parallelism is practical only within a single NVSwitch domain. Crossing InfiniBand for TP increases allreduce cost by 10-20 $\times$ and kills throughput.
- Always verify node topology with `nvidia-smi topo -m`. PCIe-only nodes and NVLink-without-NVSwitch nodes have non-uniform bandwidth that breaks the flat-bandwidth assumption.
- Allreduce cost is approximately $2P/(0.5 \times B_{\text{NVLink}})$; for a 7B model on NVSwitch, this is ~ 62 ms-significant but overlappable with compute in a correctly pipelined backward pass.
- When NVSwitch hardware is unavailable, drop tensor parallelism from the strategy and rely on data parallelism with large batches or pipeline parallelism with point-to-point sends.

11.1.2 GPU Clusters vs. TPU Pods

The moment your model requires more than one node, the question stops being “which interconnect?” and starts being “what network topology?” The answer determines which parallelism strategies remain efficient and, critically, where you will hit a wall as you scale. Senior engineers at frontier labs are expected to explain

why a training run switches from tensor parallelism to pipeline parallelism above a certain node count, and why Google’s TPU-based runs can sustain tensor parallelism at larger group sizes than comparable GPU clusters. The GPU fat-tree cluster and the TPU toroidal mesh are the two dominant answers to this topology question, and they make starkly different trade-offs.

GPU clusters use a fat-tree or dragonfly network topology. Each node—typically 8 GPUs connected by NVSwitch—attaches to a Top-of-Rack (ToR) switch via InfiniBand. Multiple ToR switches connect upward to leaf switches, and leaf switches connect to spine switches. Within a rack of roughly 32 nodes (256 GPUs), any two nodes can reach each other in at most two switch hops, giving all-to-all communication at near-wire bandwidth. Beyond approximately 256 GPUs spanning multiple racks, traffic traverses spine switches shared by many racks. Those uplinks are often the oversubscribed bottleneck, and effective per-node bandwidth for large collectives can drop by 50-80%.

TPU pods use a multi-dimensional toroidal mesh. In a v4 TPU pod, each chip connects directly to six immediate neighbors in a 3D torus via Inter-Chip Interconnect (ICI) at approximately 340 GB/s bidirectional per chip. The topology is physically uniform—there is no hierarchy, no “within-rack” vs. “cross-rack” distinction, and no bandwidth cliff as you scale beyond 256 chips. Allreduce is implemented as a sequence of ring operations along each torus axis, and effective collective bandwidth approaches wire bandwidth regardless of pod size. The constraint is the opposite: communication between distant non-adjacent chips requires multi-hop routing through neighbors, and irregular all-to-all patterns (such as load-imbalanced expert routing) pay a higher penalty than on a fat-tree.

GPU fat-tree clusters give any-to-any communication at full speed up to a topology-dependent threshold, then degrade; TPU toroidal meshes provide uniform, topology-aware bandwidth that scales gracefully but rewards algorithms aligned with nearest-neighbor communication patterns.

For GPU fat-tree clusters, the strength is communication flexibility. Expert parallelism with dynamic token routing, irregular pipeline schedules, and ad-hoc all-to-all operations all work without topology-specific optimization because any two GPUs can communicate directly. The weakness is the bandwidth cliff. When collective operations cross spine switches under 2:1 or 4:1 oversubscription, the effective allreduce bandwidth halves or quarters. This is why standard GPU training recipes switch from tensor parallelism (which demands high-bandwidth allreduce every layer) to pipeline parallelism (which requires only point-to-point activation transfers between adjacent stages) as runs scale across multiple racks.

For TPU pods, the strength is bandwidth uniformity. An allreduce decomposed into per-axis ring operations achieves near-wire bandwidth across the entire pod—the same effective rate at 64 chips and at 4096 chips. This is the hardware reason TPU-based training can sustain tensor-parallel groups of 16 or more chips without the throughput collapse seen on GPU clusters. The weakness is pattern sensitivity: the torus topology is optimized for structured, axis-aligned communication. Expert parallelism with uneven loads or arbitrary point-to-point sends require multi-hop routing that lowers effective bandwidth relative to what a fully non-blocking GPU fat-tree would provide for the same operation.

A Worked Example

Consider a 70B-parameter model in BF16 trained across 1,024 H100 GPUs organized as 128 nodes of 8 GPUs each, with tensor parallelism degree $t = 8$ within each node and data parallelism across all 128 nodes.

Within-rack data-parallel allreduce (32 nodes, 256 GPUs, single fat-tree rack):

The gradient allreduce must synchronize 14 GB of BF16 parameters across 32 nodes. Each node must

transmit approximately $14 \text{ GB} \times \frac{31}{32} \approx 13.6 \text{ GB}$. Assume each node has two NDR InfiniBand ports at 50 GB/s each, giving 100 GB/s bidirectional per node:

$$\frac{13.6 \text{ GB}}{100 \text{ GB/s}} \approx 136 \text{ ms}$$

At H100 with 50% MFU on a 70B model, a forward-backward pass takes roughly 600-900 ms. The allreduce at 136 ms is $\sim 15\text{-}23\%$ of step time-significant but maskable with overlapped gradient communication.

Cross-rack allreduce (128 nodes spanning 4 racks, 2:1 oversubscribed spine):

Effective per-node bandwidth halves to 50 GB/s due to spine oversubscription:

$$\frac{13.6 \text{ GB}}{50 \text{ GB/s}} \approx 272 \text{ ms}$$

Now allreduce is 30-45% of step time and cannot be fully hidden by compute overlap. The training run is communication-bound. The mitigation is to switch data parallelism to pipeline parallelism across racks, reducing cross-rack traffic to per-microbatch activation sends of $\sim 200 \text{ MB}$ at most.

Same run on a TPU v4 pod with 1,024 chips and 340 GB/s ICI per chip:

$$\frac{13.6 \text{ GB}}{340 \text{ GB/s}} \approx 40 \text{ ms}$$

This is $4\text{-}7\times$ faster than the within-rack GPU case and does not degrade as the pod grows. Tensor parallelism at degree 16 or 32 remains feasible because the per-chip ICI bandwidth is uniformly available at any scale.

What You Actually Decide in Practice

- **The 256-GPU threshold on GPU clusters.** Within a single non-oversubscribed fat-tree rack of ~ 256 H100s, you can sustain tensor plus data parallelism without pipeline overhead. Beyond 256 GPUs spanning multiple racks, add pipeline parallelism to reduce cross-rack traffic from full-model allreduces to per-stage point-to-point sends. The rule is not 256 exactly-it is the boundary at which your fabric becomes oversubscribed-but 256 is the practical default for a single rack of NVIDIA DGX nodes.
- **Always measure fabric oversubscription before designing the parallelism strategy.** Run NCCL's `all_reduce_perf` across N , $2N$, and $4N$ nodes. A bandwidth drop larger than 15% when doubling node count signals oversubscription at the next switch level. Design your cross-node parallelism to minimize volume at that boundary.
- **InfiniBand port count per node drives cross-node bandwidth.** A node with a single HDR port at 25 GB/s has a $36\times$ bandwidth asymmetry with its NVSwitch fabric (900 GB/s). A node with four NDR ports at 50 GB/s each gets 200 GB/s cross-node, a $4.5\times$ asymmetry. The number of InfiniBand ports per node is often the single most important number for estimating cross-node allreduce feasibility.
- **Pipeline parallelism is the cross-rack workaround on GPU clusters.** Point-to-point activation sends in pipeline parallelism scale as $b \times s \times h$ bytes per microbatch-roughly 200 MB to 1 GB for typical configurations-far smaller than the 14 GB allreduce for a 7B model. Pipeline parallelism tolerates the higher latency of cross-rack InfiniBand because it is sequential and easily overlapped with microbatch compute.
- **Designing for TPU topology.** When partitioning a model across a TPU torus, map the highest-bandwidth collective (typically tensor-parallel allreduce) to a single torus axis, and map data parallelism to another axis. Mixed-axis communication requires multi-hop routing and lowers effective bandwidth. Misaligned layouts can turn a theoretically 340 GB/s fabric into a 100 GB/s effective one for your specific collective pattern.

How This Shows Up in Interviews

- “Why does the standard recipe use tensor parallelism within a node and pipeline parallelism across nodes?” A complete answer: tensor parallelism requires allreduce every layer; NVSwitch provides 900 GB/s all-to-all within a node; InfiniBand provides 25-100 GB/s between nodes. The 10-36 $\times$ bandwidth gap means cross-node tensor parallelism is almost always bandwidth-bound. Pipeline parallelism substitutes point-to-point activation sends for allreduces, dramatically reducing cross-node traffic volume.
- “How does Google sustain larger tensor-parallel groups on TPUs than is typical on GPU clusters?” The answer is ICI topology: each TPU chip has ~ 340 GB/s to its neighbors uniformly, with no spine-switch bottleneck. Tensor-parallel allreduce implemented as torus rings approaches wire bandwidth at any pod size. On GPU clusters, crossing the rack boundary with allreduce introduces a 2-4 $\times$ bandwidth penalty that makes large tensor-parallel groups impractical.
- “You have a 4,096-GPU budget. Walk me through your parallelism assignment.” A strong answer: (1) set $t = 8$ tensor parallel within each NVSwitch node; (2) assess whether $4,096 / 8 = 512$ nodes fit in a single non-oversubscribed fabric-probably not, so add pipeline parallelism across racks with enough microbatches to keep bubble fraction below 5%; (3) fill remaining capacity with data parallelism up to the batch size limit. Senior follow-up: “Your InfiniBand fabric is 4:1 oversubscribed at the spine. What changes?” Effective cross-rack bandwidth drops to 25% of peak. Increase pipeline microbatch count to keep bubble fraction acceptable, and reduce data-parallel node count to decrease spine traffic. Consider whether model sharding via ZeRO stage 3 can consolidate the run onto fewer racks to stay within a single non-oversubscribed fat-tree domain.

Key Takeaways

- GPU fat-tree clusters provide any-to-any full-bandwidth communication up to ~ 256 GPUs within a rack; beyond that, spine-switch oversubscription cuts effective bandwidth by 50-80% for collective operations.
- TPU toroidal meshes deliver uniform ~ 340 GB/s ICI per chip with no topology cliff, making tensor-parallel allreduce feasible at large group sizes-but irregular all-to-all patterns pay multi-hop penalties.
- The 256-GPU rack boundary is where pipeline parallelism typically enters the GPU cluster strategy, replacing full-model allreduces with small point-to-point activation sends.
- InfiniBand bandwidth per node (25-100 GB/s depending on port count) is 10-36 $\times$ lower than NVSwitch bandwidth (900 GB/s); this is the defining asymmetry for all cross-node parallelism decisions.
- Measure actual fabric oversubscription before finalizing any parallelism plan. Published peak bandwidth and realized collective bandwidth routinely differ by 2-4 $\times$ on production clusters.

11.2 Collective Communication Primitives

11.2.1 AllReduce, ReduceScatter, and AllGather

When a system design interviewer asks how you synchronize gradients across 64 GPUs, naming “AllReduce” is the floor, not the ceiling. The follow-up-what does it cost, which ring algorithm is used, when would you choose ReduceScatter instead, and how does NVLink bandwidth differ from InfiniBand-is where mid-level and senior answers diverge. Collective communication primitives are the atomic operations from which every parallelism strategy in distributed training is built, and interviewers at frontier labs expect you to reason about

them in terms of bytes transferred and wall-clock latency, not just semantics.

A *collective operation* is any communication pattern in which all N participating processes (called *ranks*, indexed 0 through $N - 1$, with the full set called the *process group*) must cooperate to complete. The six primitives you need are:

- **Broadcast:** Rank r sends an identical P -byte tensor to every other rank. All ranks end with the same tensor. Total bytes transferred $\approx P$.
- **Scatter:** Rank r splits a tensor of total size P into N equal chunks and sends chunk i to rank i . Total bytes transferred $\approx P$.
- **Gather:** Each rank i holds a P/N -byte chunk; rank r collects all N chunks into one P -byte tensor. Total bytes transferred $\approx P$.
- **Reduce:** Same as Gather, but rank r applies a commutative associative operation (sum, max, min) across corresponding positions as chunks arrive. Total bytes transferred $\approx P$ to the root.
- **AllGather:** Each rank holds P/N bytes; after the operation, every rank holds all P bytes concatenated. Using a bandwidth-optimal ring, each rank sends $(N - 1)/N \times P$ bytes.
- **ReduceScatter:** Each rank holds a full P -byte tensor; after the operation, rank i holds only the reduction (e.g., sum) of the i -th slice across all ranks. Each rank ends with P/N bytes. Per-rank cost $\approx (N - 1)/N \times P \approx P$ for large N .

AllReduce is the operation every ML engineer names first. Every rank starts with P bytes, and every rank ends with the fully-reduced P -byte result. In the bandwidth-optimal ring implementation the per-rank communication cost is:

$$\text{AllReduce cost per rank} = \frac{2(N - 1)}{N} \times P \approx 2P \quad \text{for large } N$$

The factor of 2 arises because data must first circulate to be accumulated (a ReduceScatter phase) and then circulate again to be distributed to all ranks (an AllGather phase). This $2P$ bound is tight: no implementation can do better in the bandwidth-limited regime.

AllReduce, ReduceScatter, and AllGather are the three primitives from which every distributed training communication schedule is constructed; the costs $2P$, P , and P per rank should be immediate.

These primitives exist because point-to-point communication is topology-dependent and error-prone at scale. Writing explicit send/receive pairs for a ring of 64 GPUs produces brittle code; a single `all_reduce` call lets NCCL choose the optimal topology-aware algorithm and pipeline multiple operations over available links simultaneously.

The ring algorithm achieves near-perfect link saturation regardless of world size. At $N = 8$ the per-rank cost is $7/8 \times 2P = 87.5\%$ of the asymptotic $2P$. At $N = 64$ it is 98.4%. Compare this to a naive star topology—one root receives from all others then broadcasts—which creates a $2(N - 1)P$ bottleneck at the root and scales linearly with world size.

ReduceScatter and AllGather are individually half the cost of AllReduce because each moves only P bytes per rank instead of $2P$. This asymmetry is the economic argument behind ZeRO-style optimizer sharding: decompose AllReduce into ReduceScatter (leave partial results distributed) plus AllGather (reconstitute them), and insert optimizer computation in the gap without spending any extra bandwidth.

A Worked Example

Consider a data-parallel setup with 8 H100s on a single NVLink node, training a 7B-parameter model in BF16. The gradient tensor is $7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$.

AllReduce cost:

$$\text{bytes per rank} = \frac{2(8-1)}{8} \times 14 \text{ GB} = \frac{7}{4} \times 14 = 24.5 \text{ GB}$$

H100 NVLink 4 provides 900 GB/s aggregate bidirectional bandwidth per GPU. In a ring, each link is shared across concurrent steps, so effective throughput is roughly half the aggregate per direction. Benchmarks on an 8-GPU node typically yield ~ 277 GB/s for large-tensor AllReduce:

$$t_{\text{AllReduce}} \approx \frac{24.5 \text{ GB}}{277 \text{ GB/s}} \approx 88 \text{ ms}$$

ReduceScatter only: Each rank ends with a $14/8 = 1.75$ GB shard of fully-summed gradients. Per-rank cost is half:

$$t_{\text{ReduceScatter}} \approx \frac{12.25 \text{ GB}}{277 \text{ GB/s}} \approx 44 \text{ ms}$$

If you take an Adam step on the 1.75 GB shard during this window (roughly 3-5 ms of compute on an H100) and then AllGather (another ≈ 44 ms), total wall time is ≈ 88 -90 ms - identical to AllReduce. The optimizer work was absorbed for free.

Cross-node comparison: The same 7B gradient sync across 64 GPUs on two InfiniBand HDR nodes (200 GB/s per link):

$$t_{\text{AllReduce}} \approx \frac{2 \times 63/64 \times 14 \text{ GB}}{200 \text{ GB/s}} \approx 138 \text{ ms}$$

A $5\times$ slowdown versus NVLink. This is a concrete reason why tensor parallelism (which issues AllReduce every layer) is confined to the intra-node NVLink domain, while data parallelism (which issues AllReduce once per gradient-sync cycle) tolerates the slower inter-node link.

What You Actually Decide in Practice

- **Use AllReduce for gradient sync when each rank holds a full copy of the model.** Call `dist.all_reduce(param.g, op=dist.ReduceOp.SUM)` after the backward pass. Divide by world size in the optimizer, not in the communication call, to keep the two concerns separate.
- **Switch to ReduceScatter + AllGather whenever optimizer memory is the bottleneck.** The bandwidth cost is identical to AllReduce, so there is no reason to use plain AllReduce once you have the ZeRO-1 infrastructure in place.
- **Profile before assuming AllReduce is the bottleneck.** On an 8-GPU NVLink node, 88 ms of gradient sync against 500 ms of forward+backward compute for a 7B model is under 15% overhead. Across nodes at 138 ms, it may dominate and justify tensor parallelism or gradient compression.
- **Use `async_op=True` to overlap AllReduce with the optimizer step on the next micro-batch.** NCCL executes asynchronously on the GPU; calling `all_reduce` before `optimizer.step()` lets communication and compute pipeline if they target different parameter groups.
- **Do not confuse AllGather with Broadcast.** AllGather assembles distinct per-rank chunks into one concatenated tensor; Broadcast replicates the same tensor from one sender. Using Broadcast where AllGather is needed silently produces wrong results and is a common copy-paste bug when building FSDP-style code from scratch.

How This Shows Up in Interviews

- “Walk me through how gradients are synchronized in a standard data-parallel training run over 8 GPUs.”
Strong answer: each GPU computes gradients on its local mini-batch via backward pass; then calls AllRe-

duce (ring algorithm, sum) to produce the globally averaged gradient; then divides by world size (or uses `ReduceOp.AVG`); then each GPU takes an identical optimizer step. Cost: $2P$ bytes per rank.

- “What is the communication overhead of AllReduce for a 70B-parameter model in BF16?” Strong answer: $70B \times 2 \text{ bytes} = 140 \text{ GB}$; AllReduce per-rank cost $\approx 280 \text{ GB}$; at 200 GB/s inter-node InfiniBand, roughly 1.4 seconds per step-likely unacceptable without gradient compression, pipeline parallelism, or switching to ReduceScatter-based ZeRO.
- Senior follow-up: “NCCL reports 277 GB/s on your 8-GPU NVLink node, but the H100 spec sheet says 900 GB/s. Where did the bandwidth go?” Strong answer: three factors. First, 900 GB/s is aggregate bidirectional; the ring uses each link in both directions simultaneously, but the peak per-direction rate is lower. Second, ring AllReduce parallelizes across all links, but each GPU’s effective slice of the bandwidth is shared across $N - 1$ concurrent ring steps. Third, small tensor sizes do not saturate the link because latency overhead dominates; benchmarked throughput rises sharply with tensor size and plateaus around several hundred MB.

Key Takeaways

- AllReduce leaves every rank with the full reduced result; per-rank cost $\approx 2P$ bytes.
- ReduceScatter leaves each rank with one reduced shard of size P/N ; per-rank cost $\approx P$ bytes.
- AllGather assembles per-rank shards so every rank holds the full P -byte tensor; per-rank cost $\approx P$ bytes.
- On NVLink (benchmarked $\sim 277 \text{ GB/s}$ effective), a 14 GB gradient AllReduce costs $\sim 88 \text{ ms}$; on InfiniBand (200 GB/s) it costs $\sim 138 \text{ ms}$ -topology determines whether communication is negligible or the dominant cost.
- Prefer one large fused AllReduce over many small per-parameter AllReduces; NCCL kernel launch overhead is non-trivial for tensors under $\sim 10 \text{ MB}$.

11.2.2 Why AllReduce Equals ReduceScatter Plus AllGather

The identity $\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$ is the single algebraic fact that makes ZeRO’s “zero overhead” claim true. When an interviewer asks why sharding the optimizer state is free in the bandwidth-limited regime, this equivalence is the complete answer. It surfaces in senior-level system design questions and-more importantly-it is the reasoning you need to correctly evaluate any claim that a particular communication schedule is bandwidth-optimal. Missing it means you will accept hand-wavy performance numbers without being able to verify them on a whiteboard.

In the ring algorithm, AllReduce already executes as two sequential phases internally:

1. **ReduceScatter phase.** The tensor of size P is logically split into N equal slices of size P/N . Over $N - 1$ ring steps, each rank passes one slice clockwise and accumulates the incoming slice with its local copy. After $N - 1$ steps, rank i holds the fully-reduced (summed) value of slice i across all N ranks-and nothing else. Per-rank bytes transmitted: $(N - 1)/N \times P$.
2. **AllGather phase.** Over another $N - 1$ ring steps, each rank propagates its fully-reduced slice to the rest of the ring. After $N - 1$ steps, every rank holds all N fully-reduced slices-the complete AllReduce result. Per-rank bytes transmitted: another $(N - 1)/N \times P$.

Total per-rank cost:

$$\text{AllReduce} = \underbrace{\frac{N-1}{N} \times P}_{\text{ReduceScatter}} + \underbrace{\frac{N-1}{N} \times P}_{\text{AllGather}} = \frac{2(N-1)}{N} \times P$$

Because both P -byte paths happen to appear as two sub-operations, you can break AllReduce at the boundary between phases and do arbitrary computation in between:

$$\text{AllReduce}(x) = \text{AllGather}(f(\text{ReduceScatter}(x)))$$

as long as f is applied element-wise within each rank's shard and does not change the shard size. Optimizer steps satisfy this exactly: rank i applies Adam to its P/N gradient shard and produces a P/N updated parameter shard, which AllGather then reconstitutes.

Any computation that operates independently on each rank's shard and produces a same-sized shard can be inserted between ReduceScatter and AllGather at zero additional bandwidth cost.

The naive mental model for data-parallel gradient sync is: run backward $\rightarrow$ AllReduce $\rightarrow$ optimizer step. The implicit assumption is that every GPU needs the full reduced gradient before updating any parameter. That assumption is wrong, and the decomposition exposes why.

After ReduceScatter, rank i holds the exact, fully-summed gradients for its assigned P/N parameters. That is sufficient information to run a complete Adam step on those parameters—it does not need the gradients for any other parameter. The AllGather is required only to redistribute the updated parameter values so all ranks agree on the full model before the next forward pass.

Because the total bytes transferred equals the AllReduce baseline ($2P$), there is no bandwidth penalty. The optimizer state—first moment, second moment, master weights—can now be sharded: rank i only ever needs the optimizer state for its P/N slice. On a 7B model in BF16, naive DDP carries $16 \times P$ bytes of optimizer state per rank (full Adam in mixed precision); ZeRO stage 1 reduces that by a factor of N . For $N = 64$, that is a $64\times$ reduction in optimizer memory at zero communication cost.

The contrast with the obvious alternative—gather all gradients to one rank, run the full optimizer, broadcast—is stark. That approach costs $(N-1)P$ to gather and $(N-1)P$ to broadcast, totalling $2(N-1)P$, which is N times worse than the ring for large N . More importantly, it also creates an $N\times$ memory bottleneck on the root rank. The ring decomposition eliminates both problems.

A Worked Example

Four ranks, each holding a gradient vector of length 4: $[a, b, c, d]$, $[e, f, g, h]$, $[i, j, k, l]$, $[m, n, o, p]$. The desired AllReduce result is position-wise sum: $[a+e+i+m, b+f+j+n, c+g+k+o, d+h+l+p]$.

After ReduceScatter:

- Rank 0: $a + e + i + m$ (slice 0, fully summed)
- Rank 1: $b + f + j + n$ (slice 1, fully summed)
- Rank 2: $c + g + k + o$ (slice 2, fully summed)
- Rank 3: $d + h + l + p$ (slice 3, fully summed)

Optimizer step (between phases): Each rank applies its Adam update to its scalar. Say rank 0's updated parameter for position 0 is θ'_0 ; rank 1's for position 1 is θ'_1 ; and so on.

After AllGather: Every rank holds $[\theta'_0, \theta'_1, \theta'_2, \theta'_3]$ — a fully synchronized, fully updated parameter tensor.

Total bytes per rank: one scalar sent in ReduceScatter (to 3 neighbors over 3 steps), one scalar sent in AllGather (to 3 neighbors over 3 steps). Exactly $2 \times (N-1)/N \times P = 2 \times 3/4 \times 4 = 6$ scalar-sends per rank,

versus 6 for a ring AllReduce. Identical.

At production scale: a 7B model in BF16 gives $P = 14$ GB. In ZeRO stage 1 across 8 ranks, ReduceScatter delivers 1.75 GB shards. An Adam step on 1.75 GB of parameters takes ~ 3 ms on an H100 (gradient application is memory-bandwidth bound at roughly 900 GB/s HBM for small kernels, plus CUDA overhead). AllGather then reconstitutes the 14 GB tensor. The Adam step is effectively free: it runs during the ~ 44 ms ReduceScatter ring, not in series with communication.

What You Actually Decide in Practice

- **ZeRO stage 1 is always worth enabling.** The bandwidth is identical to naive DDP AllReduce. The only implementation cost is maintaining per-rank optimizer state slices and calling `dist.reduce_scatter` and `dist.all_gather` instead of `dist.all_reduce`. PyTorch’s FSDP automates this; if you are rolling your own, the two-call sequence is five lines of code.
- **ZeRO stage 3 is not free: it costs $3P$ per rank, not $2P$.** The extra P is a forward-pass AllGather needed to materialize each layer’s sharded parameters before the forward computation. This has no counterpart in ZeRO stage 1, where parameters remain replicated. Measure the 50% communication increase against your memory savings before committing to stage 3.
- **The optimizer step between ReduceScatter and AllGather is a hard serial dependency.** AllGather cannot begin until every rank has finished its Adam step on its shard. If your optimizer has pathologically slow kernels (e.g., due to BF16-FP32 casting loops), it adds directly to latency. Fuse or kernel-optimize the optimizer step if it exceeds ~ 5 ms per layer.
- **Verify your ReduceScatter + AllGather pair uses the same ring topology as your AllReduce.** NCCL selects algorithms based on message size and world size; if you call the two operations separately, NCCL may choose a tree algorithm for one and a ring for the other, breaking the bandwidth-equivalence assumption. Force the ring algorithm via environment variable `NCCL_ALGO=Ring` when benchmarking.
- **The equivalence holds only in the bandwidth-limited regime.** For small tensors ($< \sim 1$ MB), the latency of two ring passes exceeds a single fused AllReduce kernel. Do not decompose gradient sync for the embedding layer or small classification heads; the latency overhead will exceed the memory savings.

How This Shows Up in Interviews

- “Why is ZeRO stage 1 claimed to have zero communication overhead compared to DDP?” Strong answer: AllReduce already decomposes into ReduceScatter + AllGather internally. ZeRO stage 1 just inserts the per-rank optimizer step between those two phases. The bandwidth is $2P$ in both cases; the optimizer state is sharded at no additional cost.
- “What is the communication cost of ZeRO stage 3 compared to ZeRO stage 1?” Strong answer: ZeRO stage 3 costs $3P$ per rank versus $2P$ for ZeRO stage 1. The extra P is a per-layer parameter AllGather in the forward pass that does not exist in ZeRO stage 1 (where parameters remain replicated). The tradeoff is $1.5\times$ more communication in exchange for $N\times$ parameter memory savings.
- Senior follow-up: “Measured throughput on an 8-GPU FSDP run is 25% lower than DDP for the same 7B model. You know ZeRO 3 should cost $3P$ versus $2P = 50\%$ more communication. Why is the throughput hit only 25%?” Strong answer: three reasons. First, communication and computation overlap significantly: FSDP prefetches the next layer’s AllGather while the GPU is computing the current layer’s forward, so the extra P is partly hidden. Second, FSDP enables larger effective batch sizes per GPU by

freeing optimizer memory, which improves GPU utilization and partially offsets the communication overhead. Third, the $3P$ versus $2P$ comparison assumes communication is 100% of wall time; in practice, the forward and backward flops dominate and the relative communication fraction is smaller than the raw byte ratio implies.

Key Takeaways

- AllReduce = ReduceScatter + AllGather: total bandwidth is $2P$ per rank in both cases; the decomposition is bandwidth-neutral.
- ZeRO stage 1 inserts the optimizer step into the gap that already exists between ReduceScatter and AllGather, achieving $N \times$ optimizer memory reduction at zero bandwidth cost.
- ZeRO stage 3 adds a third P -byte AllGather per layer in the forward pass, raising total communication to $3P$ per rank—not free, but often worth it for very large models.
- The equivalence is bandwidth-based; for small tensors in the latency-dominated regime, a fused AllReduce kernel beats two separate ring passes.
- The key invariant: after ReduceScatter, each rank owns a fully-summed gradient shard; after AllGather, each rank owns the fully-updated parameter tensor.

11.3 The Software Stack

11.3.1 NCCL and torch.distributed

An interviewer at a frontier lab will frequently ask you to trace a single `dist.all_reduce()` call from your Python training loop all the way down to the NVLink packets leaving the GPU. This question appears at the senior level because it separates engineers who have used `torch.distributed` as a black box from those who understand the contract each layer provides. Getting this right matters in production: a misunderstood backend choice causes either correctness failures (wrong gradients silently) or catastrophic hangs that waste hours of cluster time.

The distributed communication stack for GPU training consists of two layers sitting between your Python gradient tensors and the NVLink wires. The bottom layer is NVIDIA’s Collective Communications Library (NCCL), a closed-source C library that owns the problem of routing multi-GPU collective operations as efficiently as possible on NVIDIA hardware. The top layer is `torch.distributed`, a Python API that wraps NCCL-and alternative backends-behind a single, hardware-agnostic interface.

NCCL’s job starts at process-group initialization. When you call `dist.init_process_group(backend="nccl")`, NCCL runs a topology discovery phase: it queries which GPUs are connected via NVLink, which are connected only via PCIe, and which sit on remote nodes reachable over InfiniBand or Ethernet. From that topology it precomputes optimal reduction trees and ring schedules for every collective. Critically, the rendezvous at `init_process_group` is a lightweight coordination step—it uses a small TCP or shared-filesystem handshake just so ranks can find each other. No tensor data ever flows through that channel. All actual payload moves via NCCL kernels launched directly on the GPU, bypassing the host CPU entirely once the operation is underway.

When your code reaches `dist.all_reduce(tensor, op=dist.ReduceOp.SUM)`, the CPU thread calls into NCCL through `torch.distributed`’s `nccl` backend shim. NCCL enqueues a CUDA kernel on the device stream; that kernel manages point-to-point NVLink transfers between peers, performs the partial reductions on-chip, and writes the final result back into `tensor` in place on every rank—all without the CPU being involved in

the data movement.

The second role of `torch.distributed` is backend portability. The same collective API accepts `backend="gloo"` to run on CPU using Ethernet, or `backend="mpi"` for MPI-based clusters. For debugging on a laptop, switching to `gloo` lets you verify correctness of your distributed logic without needing a GPU. Performance will be dramatically lower, but logical correctness transfers cleanly.

The contract is: NCCL owns topology-aware routing and kernel efficiency; torch.distributed owns the Python API and backend portability-you operate at the torch.distributed level and trust NCCL to saturate your NVLink bandwidth.

The alternative to this stack-manual point-to-point sends and receives through a general-purpose network library-requires every programmer to reason about reduction trees, ring topologies, and hardware-specific bandwidth asymmetries. NCCL abstracts all of that. Its topology-aware ring-allreduce or recursive halving-doubling schedules extract close to peak NVLink bandwidth without requiring the caller to know whether the GPUs are on the same NVLink switch or spread across two nodes.

The key trade-off that `torch.distributed` makes is that it couples correctness to strict collective-call symmetry. Every rank in the process group must execute the same collective operations in the same order. If one rank diverges-say, a conditional branch that skips an `all_reduce` on rank 3 while ranks 0-2 are waiting-all the other ranks block indefinitely. This synchronous-collective model simplifies reasoning about gradient consistency but is an operational hazard: a one-line Python exception on a single rank will hang the entire job.

Before NVLink-based stacks existed, multi-GPU communication had to stage data through host memory, copy it across PCIe to the CPU, retransmit over Ethernet to another node's CPU, copy back over PCIe, and finally land it on the target GPU. NCCL on NVLink-equipped nodes eliminates every intermediate hop for intra-node communication. The bandwidth difference is roughly an order of magnitude: PCIe gen 4 delivers ~64 GB/s bidirectional per slot; H100 NVLink 4 delivers ~900 GB/s aggregate across all 18 links.

A Worked Example

Consider a data-parallel training job: 4 H100s on one node, each holding a copy of a model whose gradient tensor is 400 MB (100 million float32 parameters, $100 \times 10^6 \times 4 = 400$ MB). After the backward pass, each rank calls:

```
dist.all_reduce(grad_tensor, op=dist.ReduceOp.SUM)
```

Step 1: NCCL selects a ring-allreduce schedule over 4 ranks. In a ring-allreduce, the operation proceeds in two phases-reduce-scatter then all-gather-each requiring $(N - 1)$ steps of message passing around the ring.

Step 2: Bytes transferred per rank. Each rank sends and receives data equal to:

$$B_{\text{allreduce}} = 2 \cdot \frac{N - 1}{N} \cdot S$$

where $S = 400$ MB and $N = 4$:

$$B_{\text{allreduce}} = 2 \cdot \frac{3}{4} \cdot 400 \text{ MB} = 600 \text{ MB per rank}$$

The factor of 2 accounts for the reduce-scatter phase (data flows in) and the all-gather phase (result flows out).

Step 3: Expected wall-clock time at effective NVLink bandwidth of 300 GB/s (a realistic single-link utilization on a 4-GPU node, not the full 900 GB/s aggregate):

$$t = \frac{600 \text{ MB}}{300 \text{ GB/s}} \approx 2 \text{ ms}$$

In practice, benchmarking this exact configuration on H100 yields measured effective bandwidth in the range of 250-300 GB/s, translating to 2-2.5 ms for the full allreduce. This is the number you hold in your head when sizing whether gradient sync dominates your step time.

What You Actually Decide in Practice

- **Always use `nccl` for GPU training; use `gloo` only for CPU debugging.** `gloo` over Ethernet is 10-50× slower than `nccl` over NVLink and will make your job communication-bound even at small scale. Never accidentally ship `gloo` to a GPU cluster.
- **Treat collective calls as global barriers.** Every rank must call the same collective in the same order. Structure your code so conditionals never cause a rank to skip or add a collective. A common bug: wrapping a logging `all_gather` inside `if rank == 0` on only one rank-this will hang the job.
- **Use `async_op=True` to overlap communication with computation.** `dist.all_reduce(tensor, async_op=True)` returns a handle immediately; the CPU can dispatch the next backward layer while NCCL drains the previous gradient bucket over the wire. DDP's gradient bucketing relies on exactly this pattern.
- **Set `NCCL_DEBUG=INFO` when diagnosing hangs or low bandwidth.** NCCL will log its topology discovery, selected algorithm, and ring schedule. If you see NCCL picking a PCIe path instead of NVLink, the GPU placement on the node is wrong.
- **For cross-node jobs, InfiniBand backend matters as much as NVLink.** NCCL supports GPUDirect RDMA to write directly from GPU memory to InfiniBand without staging through host RAM. Without it, cross-node bandwidth drops by 2-3×. Verify RDMA is enabled with `nccl_p2p_disable=0` and confirm with `NCCL_DEBUG=TRACE`.

How This Shows Up in Interviews

- **“Walk me through what happens when your training loop calls `dist.all_reduce()` on a gradient tensor.”** A strong answer traces the path: Python API → torch.distributed nccl shim → NCCL enqueues a CUDA kernel → GPU executes ring-allreduce over NVLink → result written back in-place on all ranks. Mention that the CPU is not in the data path after kernel launch.
- **“Why might you choose `gloo` over `nccl`?”** The one valid reason: running on CPU, typically for debugging or CI environments without GPUs. On a GPU node, `gloo` is never the right production choice.
- **“If rank 2 throws a Python exception mid-training, what happens to the other 7 ranks?”** They hang at the next collective call waiting for rank 2 to arrive. Recovery requires catching the exception on rank 2, broadcasting an error signal to all ranks, and having everyone call `dist.destroy_process_group()` cleanly-or just letting the job manager detect the dead process and restart.
- **Senior follow-up: “How does DDP overlap gradient communication with the backward pass, and what are the limits of that overlap?”** DDP partitions parameters into buckets (default 25 MB). When the backward hook fires for the last parameter in a bucket, it immediately launches an async `all_reduce` for that bucket while the backward pass continues computing earlier-layer gradients. The limit: the first bucket's allreduce must complete before the optimizer step, so the final bucket on the critical path is not overlapped. For very large models with few parameters per layer, buckets serialize and overlap collapses.

Key Takeaways

- NCCL translates collective operations into topology-optimal CUDA kernels; `torch.distributed` wraps NCCL behind a portable Python API that also supports `gloo` for CPU.
- Tensor data never flows through the rendezvous channel; `init_process_group` is only coordination. All payload moves GPU-to-GPU via NCCL kernels on NVLink or InfiniBand.
- Every rank must execute identical collectives in identical order; a single rank skipping one collective hangs the entire job.
- Use `async_op=True` to overlap gradient communication with backward computation; this is the foundation of DDP's gradient bucketing.
- On H100 NVLink 4, expect $\sim 250\text{--}300$ GB/s effective allreduce bandwidth per rank for large tensors; always benchmark rather than assuming theoretical peak.

11.3.2 Benchmarking Bandwidth in Practice

When an interviewer asks “how long will the gradient allreduce take for a 70B model?” they are really asking two questions at once: do you know the formula for bytes transferred, and do you know that your answer is only meaningful if you have validated it with a real measurement? I have seen senior engineers confidently quote NVLink’s theoretical 900 GB/s peak in response to this question-and have that answer immediately undermined when the interviewer asks why production jobs run at 30% of that number. Knowing how to benchmark collective bandwidth correctly, and knowing what causes the gap between peak and achieved, is a genuine differentiator at the staff level.

Measuring collective bandwidth requires getting three things right: the timing methodology, the synchronization model, and the byte-count formula. Each is independently easy to get wrong.

Timing. GPU operations are asynchronous from the CPU. When you call `dist.all_reduce()` and immediately read `time.time()`, you are measuring the time to *submit* the kernel, not to *complete* it. The correct idiom is:

```
torch.cuda.synchronize()
dist.barrier()
t0 = time.time()
dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
torch.cuda.synchronize()
dist.barrier()
t1 = time.time()
elapsed = t1 - t0
```

`torch.cuda.synchronize()` blocks the CPU until all pending CUDA kernels on the current device have completed. `dist.barrier()` ensures all ranks have reached the same point before the clock starts and after it stops, so you are measuring the slowest rank’s time-which is the correct quantity for a collective that requires all participants.

Warm-up. The first execution of an NCCL kernel involves just-in-time compilation, peer-to-peer connection setup, and CUDA context initialization. Run at least one-preferably three-warm-up iterations before recording any timing. Otherwise your first measurement is dominated by one-time overhead that will never appear again in steady-state training.

Byte-count formula. The bytes transferred per rank depends on which collective you are measuring:

$$B_{\text{allreduce}} = 2 \cdot \frac{N-1}{N} \cdot S$$

$$B_{\text{reducescatter}} = \frac{N-1}{N} \cdot S$$

$$B_{\text{allgather}} = \frac{N-1}{N} \cdot S$$

where S is the tensor size in bytes per rank and N is the world size. The factor of 2 in allreduce reflects the two phases of the ring algorithm: reduce-scatter (each rank sends chunks out) and all-gather (each rank receives the final result). Reduce-scatter and all-gather each account for half. Because allreduce = reduce-scatter + all-gather, their individual bandwidth numbers add up to the allreduce number—which is a useful sanity check.

Effective bandwidth is the agreed-upon convention for comparing hardware utilization across configurations; it is always measured bytes divided by wall-clock time, computed once synchronization is complete across all ranks.

The alternative-trusting spec sheets-fails for two reasons. First, NVLink’s 900 GB/s aggregate is the sum across all 18 links on an H100; any single rank-to-rank transfer uses a subset. Second, real workloads suffer from contention, buffer setup latency, and algorithmic overhead (e.g., ring-allreduce is not embarrassingly parallel). Benchmarking gives you the actual operating point. When the measured bandwidth is dramatically below peak, you have actionable information: check NCCL topology selection (NCCL_DEBUG=INFO), verify NVLink vs PCIe path, test different tensor sizes to identify the knee in the bandwidth curve.

The explicit warm-up requirement mirrors everything we know about GPU performance measurement from single-device profiling: the first kernel launch is never representative. The same principle that demands warm-up for `torch.compile` or FlashAttention applies here.

A Worked Example

Setup: 4 H100s on one node, `world_size = 4`. Tensor: 100 million float32 elements per rank, so $S = 100 \times 10^6 \times 4 = 400$ MB.

Allreduce measurement:

$$B_{\text{allreduce}} = 2 \cdot \frac{3}{4} \cdot 400 \text{ MB} = 600 \text{ MB per rank}$$

Suppose the measured elapsed time after synchronization and barrier is $t = 2.16$ ms. Then:

$$\text{bandwidth} = \frac{600 \text{ MB}}{0.00216 \text{ s}} \approx 277 \text{ GB/s}$$

Reduce-scatter measurement on the same tensor (input shape: `[4, 100M]`, output shape: `[100M]`):

$$B_{\text{reducescatter}} = \frac{3}{4} \cdot 400 \text{ MB} = 300 \text{ MB per rank}$$

Suppose measured time is $t = 4.3$ ms:

$$\text{bandwidth} = \frac{300 \text{ MB}}{0.0043 \text{ s}} \approx 70 \text{ GB/s}$$

The gap between allreduce (277 GB/s) and reduce-scatter (70 GB/s) at first looks surprising. Allreduce at $2\times$ the bytes should take $2\times$ as long, yielding the same bandwidth—yet reduce-scatter runs at roughly one-quarter the rate. The difference likely comes from two sources. First, NVIDIA’s SHARP (*Scalable Hierarchical Aggregation and Reduction Protocol*) hardware acceleration can perform the reduction inside the NVLink switch fabric for allreduce, effectively halving the traffic seen by the GPUs. Second, NCCL invests more engineering in allreduce tuning since it is the highest-frequency production operation. This is precisely why benchmarking

beats arithmetic: the hardware does not always behave as the formula predicts.

What You Actually Decide in Practice

- **Always warm up at least 3 iterations before recording.** One warm-up is often not enough if NCCL needs to establish peer connections on first use. Three iterations is conservative but reliable.
- **Apply both `torch.cuda.synchronize()` and `dist.barrier()` around your timer.** Synchronize alone measures your own rank's completion; barrier ensures you capture the slowest rank, which determines real step time.
- **Sweep tensor sizes across 1 MB to 10 GB.** NCCL has a latency-dominated regime (small tensors, typically below ~ 1 MB) and a bandwidth-dominated regime. The knee is usually between 10-100 MB. Gradient tensors from large models land firmly in the bandwidth-dominated regime.
- **Do not assume reduce-scatter and allreduce achieve the same bandwidth.** As the worked example shows, the ratio can be $4\times$ in favor of allreduce due to in-network compute. If your FSDP or ZeRO plan hinges on reduce-scatter bandwidth, measure it directly.
- **Cross-node bandwidth is typically $5\text{-}10\times$ lower than intra-node NVLink.** An 8-GPU node connected to another 8-GPU node via InfiniBand HDR (200 Gb/s, ≈ 25 GB/s) has a cross-node bottleneck that is $36\times$ narrower than NVLink per node. Parallelism strategies must respect this asymmetry: pipeline parallelism across nodes, tensor parallelism within nodes.

How This Shows Up in Interviews

- **“Estimate how long the gradient sync takes for a 70B parameter model in bf16 across 8 H100s.”** Gradient tensor per rank: $70 \times 10^9 \times 2$ bytes = 140 GB. Allreduce bytes per rank: $2 \cdot \frac{7}{8} \cdot 140$ GB = 245 GB. At 250 GB/s effective bandwidth: ≈ 1 second. A strong answer gives the formula, plugs in realistic bandwidth, and then notes that DDP with gradient bucketing hides much of this behind the backward pass—so raw sync time overstates the wall-clock cost.
- **“Why do you need to call `torch.cuda.synchronize()` before stopping the timer?”** Because CUDA kernel dispatch is asynchronous; the CPU returns from `dist.all_reduce()` before the GPU has finished. Without synchronize, you measure submission latency ($\sim \mu\text{s}$), not transfer time ($\sim \text{ms}$).
- **“You benchmarked allreduce and got 100 GB/s on a 4-GPU H100 node. NVLink peak is 900 GB/s. What do you check?”** First, confirm NCCL is using NVLink and not falling back to PCIe (NCCL_DEBUG=INFO). Second, check tensor size—below ~ 10 MB, latency dominates and bandwidth appears low. Third, verify warm-up was done and synchronization is correct. Fourth, check whether SHARP in-network compute is enabled, which can boost allreduce specifically.
- **Senior follow-up: “How would you benchmark whether your pipeline-parallel job is compute-bound or communication-bound across nodes?”** Instrument both the forward/backward compute time per microbatch and the inter-node send/recv latency. Compute is bound if the GPU utilization on intermediate ranks stays near 100% while the first and last ranks are idle. Communication is bound if you see the pipeline bubble shrink when you reduce the tensor size sent between stages. Concretely: add `torch.cuda.Event` timers bracketing the compute and send/recv separately, record them on each rank, and aggregate across steps.

Key Takeaways

- Always warm up and always call `torch.cuda.synchronize()` plus `dist.barrier()` around your timer; skipping either gives numbers that are not reproducible and not meaningful.
- The byte-count formula for allreduce is $2(N - 1)/N \cdot S$; for reduce-scatter and allgather separately, it is $(N - 1)/N \cdot S$ each-these add up correctly since `allreduce` = `reduce-scatter` + `allgather`.
- Measured bandwidth for allreduce and reduce-scatter can differ by 3-4 $\times$ on the same hardware due to in-network SHARP acceleration and differential NCCL optimization; always benchmark both if your architecture uses both.
- Intra-node NVLink bandwidth (~ 250 - 300 GB/s effective per rank on H100) is 10 $\times$ higher than typical cross-node InfiniBand; parallelism placement must respect this gap.
- Theoretical peak numbers are starting points for estimates, not commitments; real training jobs routinely achieve 30-60% of advertised NVLink bandwidth for collectives.

Chapter 12 Parallelism Strategies

12.1 Data Parallelism and ZeRO

12.1.1 Naive DDP and Its Memory Blow-Up

Data parallelism is the first strategy any interviewer reaches for when probing multi-GPU training, and the expected answer at the mid-level is straightforward: split the batch, sync the gradients, repeat. What separates a strong answer from a mediocre one is the memory accounting. Candidates who cannot explain why naive DDP requires 16 bytes per parameter—and why that number dwarfs the model weights themselves—have not thought seriously about scaling. This topic surfaces in every distributed training conversation, and the memory breakdown is the part that gets people.

Distributed Data Parallel training is a parallelization of minibatch gradient descent along the batch dimension. Given a global batch of size B and M GPU workers, each worker receives B/M examples, computes a local gradient, and then synchronizes with every other worker before the optimizer step. The parameter update rule is unchanged:

$$\theta_{t+1} = \theta_t - \frac{\eta}{B} \sum_{i=1}^B \nabla_i \ell(\theta_t)$$

The synchronization step is an *AllReduce* over the gradient vectors. Each worker holds a locally computed partial sum $g_k = \frac{1}{B/M} \sum_{i \in \text{shard}_k} \nabla_i \ell$, and AllReduce produces the global average on every worker simultaneously. Ring-AllReduce, the standard implementation, moves a total of $2|\theta|$ bytes across the network per step (each element travels around the ring once in the scatter phase and once in the gather phase).

The memory concern is where the analysis becomes uncomfortable. In BF16 training, the naive accounting goes:

Component	Dtype	Bytes/param
Model weights	BF16	2
Gradients	BF16	2
Master weights (FP32)	FP32	4
Adam first moment (m)	FP32	4
Adam second moment (v)	FP32	4
Total		16

Every GPU in a naive DDP run carries all 16 bytes per parameter. Adding more GPUs does not reduce this burden; it just means more machines each pinning $16 \times$ the parameter count in HBM.

In naive DDP, the optimizer states—not the model weights—are the dominant memory cost, and they do not shrink as you add more GPUs.

On the compute side, DDP scales well. Each GPU processes B/M independent examples, and as long as the micro-batch is large enough to saturate the tensor cores, adding more GPUs gives near-linear throughput scaling. The AllReduce cost is $2|\theta|$ bytes per step, but this is paid once per batch regardless of M in the ring implementation, so larger batches amortize it effectively.

The memory side tells the opposite story. Every worker is a full replica: parameters, gradients, master weights, and both Adam moments are each copied M times across the cluster. Total cluster memory grows

as $16|\theta| \cdot M$ -linear in M but with no improvement per device. Contrast this with the ideal: if memory scaled inversely with M , a cluster twice as large would need half the per-GPU memory. Naive DDP never achieves that.

The practical implication is that batch size becomes the lever that makes DDP viable. Large batches mean the AllReduce overhead is small relative to useful compute, and they improve GPU utilization. But batch size is a resource with diminishing returns: beyond what researchers call the *critical batch size*, increasing B reduces gradient noise without meaningfully accelerating convergence, because the training is no longer noise-limited but step-count-limited. Naive DDP therefore hits a wall: it cannot scale memory, and the batch size resource needed to keep it efficient is finite.

A Worked Example

Consider a 7.5 billion parameter model trained on a cluster of 64 A100 80 GB GPUs with the AdamW optimizer in BF16 mixed precision.

Memory per GPU under naive DDP:

$$7.5 \times 10^9 \text{ params} \times 16 \text{ bytes/param} = 120 \text{ GB per GPU}$$

An A100 has 80 GB of HBM. This does not fit. Even ignoring activations (which add another ~ 10 -30 GB depending on batch size and sequence length), the model state alone exceeds the device by 50%.

Communication per step:

The AllReduce moves $2|\theta|$ bytes. With BF16 gradients:

$$2 \times 7.5 \times 10^9 \times 2 \text{ bytes} = 30 \text{ GB}$$

Over an NVLink-connected 8-GPU node (effective bisection bandwidth ~ 600 GB/s aggregate), a single AllReduce on gradients takes roughly 50 ms. Over InfiniBand at ~ 200 GB/s, the same exchange takes ~ 150 ms-longer than a typical forward-backward pass on a short sequence. Without large batch sizes to hide that latency, DDP over slow interconnects is communication-bound.

Breakdown of the 120 GB:

- Model weights (BF16): 15 GB
- Gradients (BF16): 15 GB
- Master weights (FP32): 30 GB
- Adam m (FP32): 30 GB
- Adam v (FP32): 30 GB

The optimizer states (master weights + moments) account for 90 GB out of 120 GB-75% of the total. This is the observation that motivates ZeRO: most of the per-GPU memory is redundant replication of optimizer state that only one GPU needs at a time.

What You Actually Decide in Practice

- **Use naive DDP when the model fits.** If $16|\theta| < \text{HBM}$ with headroom for activations, naive DDP is the simplest path. For a 1B parameter model on A100 80 GB, the model state is 16 GB-fine. For anything above roughly 4-5B parameters on 80 GB devices, you are already tight before counting activations.
- **AllReduce bandwidth determines your minimum batch size.** The rule of thumb: the compute time for one step should be at least 2 - $3\times$ the AllReduce time. If you cannot achieve that with your hardware

bandwidth and model size, DDP over slow links will be communication-bound no matter how many GPUs you add.

- **Gradients dominate peak memory mid-backward.** The peak memory during a training step occurs partway through the backward pass, when accumulated gradients and surviving activations overlap. Profiling this rather than just the steady-state model size reveals the true memory ceiling.
- **The critical batch size is model- and task-dependent, typically 10^5 - 10^6 tokens.** OpenAI’s batch size scaling analysis showed that past the critical batch size, each additional example contributes negligibly to gradient signal. If your global batch already saturates this threshold, adding more data-parallel workers buys compute but wastes gradient diversity.
- **Gradient accumulation is not free.** Accumulating gradients over multiple micro-batches to simulate a larger effective batch does not reduce peak gradient memory—each micro-batch’s gradients still accumulate into a full-size gradient tensor. It reduces communication frequency but not per-step memory overhead.

How This Shows Up in Interviews

- **“Walk me through how DDP works and what its limitations are.”** A strong answer covers: batch splitting, local forward-backward, AllReduce cost ($2|\theta|$), and the memory accounting that gives 16 bytes per parameter. The limitation is that memory does not scale with GPU count.
- **“You have a 13B parameter model and $8 \times \text{A100 80 GB GPUs}$. Can you train it with naive DDP?”** Calculate: $13 \times 10^9 \times 16 = 208 \text{ GB per GPU}$. No. Even before activations this is $2.6\times$ over budget. The answer should then pivot to what you would do instead (ZeRO stage 3 / FSDP).
- **“Why does Adam make memory worse than SGD?”** SGD requires parameters + gradients = 4 bytes/-param (BF16). Adam adds first moment (4 bytes FP32) + second moment (4 bytes FP32) + master weights (4 bytes FP32) = 12 additional bytes. Total becomes 16 vs. 4—a $4\times$ blow-up attributable entirely to the optimizer.
- **Senior follow-up: “At what point does the AllReduce become the bottleneck, and how would you restructure training to hide it?”** The senior answer identifies the compute-to-communication ratio: when one forward-backward pass takes less time than the AllReduce, you are communication-bound. Remedies include gradient compression (lossy, hurts convergence), asynchronous SGD (staleness issues), or switching to a ZeRO stage that interleaves communication with computation layer by layer—which is exactly what ZeRO Stage 2 does.

Key Takeaways

- Naive DDP uses 16 bytes per parameter: 2 for weights, 2 for gradients, and 12 for Adam optimizer state (FP32 master weights, first moment, second moment).
- AllReduce moves $2|\theta|$ bytes per step regardless of the number of workers in a ring implementation.
- Memory does not scale with GPU count in naive DDP—every device carries the full 16 bytes per parameter.
- Optimizer states (master weights + Adam moments) account for roughly 75% of per-GPU memory in mixed-precision training.
- Batch size is a finite resource: past the critical batch size ($\sim 10^5$ - 10^6 tokens for most LLMs), adding data-parallel workers does not improve sample efficiency.

12.1.2 ZeRO Stages 1, 2, and 3 (FSDP)

The moment you see that 75% of a DDP node’s memory is optimizer state that every GPU duplicates identically, the fix becomes obvious: shard it. ZeRO-the Zero Redundancy Optimizer-is that fix, structured into three progressive stages that trade implementation complexity for decreasing per-device memory. Interviewers at AI labs ask about ZeRO not because it is obscure, but because understanding it requires you to reason simultaneously about collective communication primitives, memory layout, and backward-pass scheduling. If you can explain all three stages and the identity that makes Stage 1 cost-free, you have demonstrated the kind of systems fluency the job requires.

ZeRO begins with one algebraic observation: AllReduce is equivalent to ReduceScatter followed by AllGather.

In a ring-AllReduce over M workers, each element of the gradient travels around the ring in a scatter phase (each worker accumulates $1/M$ of the total) and then in a gather phase (each worker distributes its accumulated slice to every other). The total data movement is $2|\theta|$ bytes. ReduceScatter alone moves $|\theta|$ bytes and leaves worker k holding the fully reduced sum for the k -th shard. AllGather moves another $|\theta|$ bytes to propagate those shards everywhere. The bandwidth budget is identical, but the intermediate state after ReduceScatter-where each worker holds only $1/M$ of the gradients in reduced form-is the hook ZeRO exploits.

Stage 1 (optimizer state sharding). After the forward and backward passes, every worker has a full gradient vector. A ReduceScatter distributes the summed gradients so that worker k holds the reduced gradient for parameter shard k . Each worker then applies the Adam update to its own shard using the optimizer state it owns locally (first moment, second moment, master weights). An AllGather propagates the updated parameters to all workers. The result is a complete synchronized parameter vector on every device, achieved with the same $2|\theta|$ communication as naive DDP-but optimizer state is divided by M .

Stage 2 (gradient sharding). The problem with Stage 1 is that the full gradient vector must be materialized before ReduceScatter. For large models this is still a memory spike. Stage 2 eliminates it by issuing the reduce operation layer by layer during the backward pass: as soon as the gradient for a layer is computed, it is immediately reduced to the responsible worker and freed on all others. Peak gradient memory becomes $|\theta|/M$ rather than $|\theta|$.

Stage 3 (parameter sharding / FSDP). Stages 1 and 2 still replicate the parameters on every GPU. Stage 3 shards the parameters themselves: each worker owns $1/M$ of the model weights at rest. During the forward pass, the worker gathers the parameters it needs for each layer (AllGather), computes the layer, then immediately frees the gathered weights. During the backward pass, the same AllGather-compute-free pattern applies, followed by a ReduceScatter on gradients. PyTorch’s FullyShardedDataParallel (FSDP) is a production implementation of Stage 3.

ZeRO Stage 1 is algebraically free relative to naive DDP; Stages 2 and 3 pay modest overhead in exchange for memory that scales inversely with the number of workers.

The key insight is that the optimizer update is embarrassingly parallel across parameter shards. Worker k does not need workers $0 \dots k-1$ or $k+1 \dots M-1$ to be finished with their Adam steps in order to compute its own. All it needs is the fully reduced gradient for its shard (delivered by ReduceScatter) and its own optimizer state. This local update is independent, so the communication required to distribute and collect shards matches exactly what a naive AllReduce would have spent-no extra bandwidth for Stage 1.

Stages 2 and 3 do add overhead. Stage 2 requires synchronizing gradients layer by layer during the backward pass, introducing fine-grained reduce operations that cannot be batched into one large AllReduce. The

total bandwidth is still $2|\theta|$ but the latency profile is less regular. Stage 3 is more expensive: the forward pass AllGathers add another $|\theta|$ bytes, pushing total communication to $3|\theta|$. In return, per-device memory drops to roughly $16|\theta|/M$ for model state, making it possible to fit models that would be impossible otherwise.

Compared to naive DDP, Stage 3 sacrifices 50% more network bandwidth to achieve memory that scales linearly with the number of GPUs—a compelling trade whenever memory is the binding constraint.

A Worked Example

Consider a 7.5 billion parameter model trained with AdamW in BF16 mixed precision, distributed across $M = 64$ workers.

Naive DDP baseline:

$$7.5 \times 10^9 \times 16 \text{ bytes} = 120 \text{ GB per GPU}$$

ZeRO Stage 1 shards the 12 optimizer-state bytes (FP32 master weights + $m + v$):

$$\underbrace{2 + 2}_{\text{params + grads (BF16)}} + \frac{4 + 4 + 4}{64} \text{ bytes/param} \approx 4.2 \text{ bytes/param} \Rightarrow \approx 31 \text{ GB per GPU}$$

Communication cost: $2|\theta|$ (identical to DDP).

ZeRO Stage 2 additionally shards gradients:

$$\underbrace{2}_{\text{params (BF16)}} + \frac{2 + 4 + 4 + 4}{64} \text{ bytes/param} \approx 2.2 \text{ bytes/param} \Rightarrow \approx 16.6 \text{ GB per GPU}$$

Communication cost: $2|\theta|$, but paid in per-layer increments.

ZeRO Stage 3 (FSDP) shards everything:

$$\frac{2 + 2 + 4 + 4 + 4}{64} \text{ bytes/param} = \frac{16}{64} = 0.25 \text{ bytes/param} \Rightarrow \approx 1.9 \text{ GB per GPU}$$

Communication cost: $3|\theta|$ per step (AllGather in forward, AllGather in backward, ReduceScatter in backward).

Maximum trainable model size on $8 \times \text{A100 80 GB}$:

- Naive DDP: $\lfloor (80 \text{ GB} - \text{activations}) / 16 \rfloor \approx 4\text{-}6$ billion parameters.
- FSDP (Stage 3): $\lfloor 8 \times 80 \text{ GB} / 16 \rfloor \approx 40\text{-}50$ billion parameters.

FSDP enables roughly $8\times$ larger models on the same hardware, at the cost of $3\times$ versus $2\times$ network utilization.

Overlapping communication and computation in FSDP:

FSDP achieves acceptable efficiency by prefetching. While GPU k is computing layer l 's forward pass, the AllGather for layer $l+1$'s weights is already in flight. By the time the compute for layer l finishes, the weights for $l+1$ are available without stalling. The same prefetch applies to the backward pass. The communication bubbles that remain are small relative to compute time, provided the AllGather latency is less than the per-layer compute time—a condition that holds on NVLink-connected intra-node workers but can fail on slow inter-node links.

What You Actually Decide in Practice

- **Always use Stage 1.** It costs nothing in bandwidth and cuts optimizer state memory by M . There is no reason to use bare DDP over Stage 1 except tooling inertia.
- **Prefer Stage 2 over Stage 3 when the model fits.** Stage 2 keeps parameters on every device, which means each layer's forward pass requires no AllGather. The per-layer gradient reductions add overhead, but it is a small constant penalty. Stage 3 adds $|\theta|$ bytes of network traffic per step and introduces AllGather bubbles in every layer. Use Stage 3 only when the model state does not fit even with Stage 2.

- **Stage 3 (FSDP) requires fast intra-node interconnects.** The AllGather per layer is latency-sensitive. On NVLink (600 GB/s aggregate across 8 GPUs on an H100 node), the overhead is acceptable. Across InfiniBand (200 GB/s), prefetching can fail to fully hide the gather latency, and GPU utilization drops. The practical rule: use FSDP within a node, and consider pipeline or tensor parallelism for cross-node sharding of very large models.
- **Activation memory is not reduced by any ZeRO stage.** ZeRO 1, 2, and 3 operate on model state. The activation memory footprint—which scales as $O(s \cdot b \cdot d \cdot L)$ for sequence length s , batch size b , hidden dimension d , and layers L —remains fully replicated on each data-parallel worker. For long-context training or very large hidden dimensions, activation recomputation (gradient checkpointing) or sequence parallelism is necessary on top of ZeRO.
- **Gradient accumulation interacts with Stage 2.** With Stage 2, per-layer gradient reductions happen during the backward pass. If you accumulate gradients over multiple micro-batches before applying the optimizer, you must ensure that the per-layer reduces are deferred to the end of the accumulation window—otherwise you reduce partial gradients prematurely. PyTorch FSDP handles this with a `no_sync()` context manager that suppresses gradient synchronization during accumulation steps.

How This Shows Up in Interviews

- **“Explain ZeRO Stage 1 and why it doesn’t cost more bandwidth than naive DDP.”** The answer turns on the AllReduce decomposition: ReduceScatter ($|\theta|$ bytes) delivers per-shard reduced gradients; each worker runs its Adam update locally; AllGather ($|\theta|$ bytes) redistributes updated parameters. Total: $2|\theta|$ —same as a ring-AllReduce. The insight is that the optimizer step fits cleanly between the two halves of the AllReduce without adding communication.
- **“What is FSDP and when would you use it instead of DDP?”** FSDP is ZeRO Stage 3: parameters, gradients, and optimizer state are all sharded. Use it when the model state ($16|\theta|$ bytes) exceeds available HBM even after stages 1 and 2. The cost is $3\times$ network utilization versus $2\times$ for DDP and potential compute bubbles if AllGather latency is not hidden by prefetching.
- **“Does ZeRO reduce activation memory?”** No. ZeRO only shards model state (parameters, gradients, optimizer state). Activations are local to each data-parallel worker and scale with batch size and sequence length. The interviewer is probing whether you conflate model state with activation memory—a common mistake.
- **Senior follow-up: “At 1024 GPUs, would you use FSDP alone or combine it with tensor parallelism? Why?”** The senior answer is: combine them. FSDP over 1024 GPUs means every AllGather for a layer’s parameters crosses the full cluster, which at inter-node InfiniBand speeds is brutally slow. Instead, apply tensor parallelism within each 8-GPU node (using NVLink) and FSDP or pipeline parallelism across nodes (using InfiniBand). This matches the communication pattern to the available bandwidth at each level of the hierarchy—a principle that underpins every practical large-scale training configuration.

Key Takeaways

- AllReduce = ReduceScatter + AllGather; this identity is why ZeRO Stage 1 adds zero communication overhead relative to naive DDP.
- Stage 1 shards optimizer state ($\div M$); Stage 2 additionally shards gradients; Stage 3 (FSDP) shards everything including parameters, reducing per-device model state to $16|\theta|/M$.

- FSDP increases total communication from $2|\theta|$ to $3|\theta|$ per step; it masks this overhead by prefetching parameter AllGathers while the previous layer computes.
- ZeRO does not reduce activation memory; gradient checkpointing and sequence parallelism handle that separately.
- The practical rule: use Stage 1 unconditionally, Stage 2 when gradients pressure memory, Stage 3 only when the model itself does not fit-and pair Stage 3 with tensor parallelism on fast intra-node links rather than running it flat across the whole cluster.

12.2 Model Parallelism

12.2.1 Tensor Parallelism (Megatron-Style)

When a model's weight matrices individually exceed a single GPU's HBM-and ZeRO-style sharding is not enough because gathering those weights per layer costs too much bandwidth-you need to split the matrix multiplications themselves across GPUs. Tensor parallelism does exactly this. It shows up at senior-level interviews the moment the conversation turns to training 70B or larger models on a single node, and the question that separates candidates is whether they can name the two synchronization points per transformer layer, explain why the column-parallel/row-parallel split requires no communication at the first projection, and give the bandwidth rule that caps tensor parallel degree at eight GPUs.

The foundation is that matrix-multiply outputs are sums of partial products, and those partial products can live on different devices. Consider the dense MLP block: $Y = \text{Dropout}(\text{GeLU}(XA)B)$, where $X \in \mathbb{R}^{s \times h}$, $A \in \mathbb{R}^{h \times 4h}$, and $B \in \mathbb{R}^{4h \times h}$.

Column-parallel split for A . Partition A along its column dimension into t equal shards: $A = [A_1 \mid \dots \mid A_t]$. GPU k holds $A_k \in \mathbb{R}^{h \times 4h/t}$ and computes $Y_k = \text{GeLU}(XA_k)$. The input X is replicated on every GPU-either via a broadcast at the start of the forward pass, or simply because the preceding layer's AllReduce already produced a full copy everywhere. No communication is needed to produce Y_k .

Row-parallel split for B . Partition B along its row dimension: $B = [B_1^\top; \dots; B_t^\top]^\top$, with $B_k \in \mathbb{R}^{4h/t \times h}$. GPU k computes $Z_k = Y_k B_k$, yielding a partial sum. The full output $Z = \sum_k Z_k$ is obtained with a single AllReduce across all t GPUs. This is synchronization point f in the forward pass.

The attention block follows the same template: split the Q, K, V projection matrices column-parallel so GPU k handles h/t attention heads independently; split the output projection row-parallel; AllReduce to merge heads.

Per transformer layer this yields *one* AllReduce in the forward pass (after each output projection) and *one* AllReduce in the backward pass. Both have the same tensor size: $s \times b \times h$ elements, where s is sequence length and b is batch size.

Tensor parallelism splits weight matrices width-wise and communicates only activations at layer boundaries-parameters never move.

Unlike data parallelism, tensor parallelism does not consume batch size. Every tensor-parallel GPU processes the full batch; parallelism comes from reducing the size of each GPU's weight matrices, not from distributing data samples. This makes it the first tool to reach for when memory is the binding constraint but the global batch size is already at or near its optimal value.

Unlike pipeline parallelism, tensor parallelism introduces no pipeline bubble. All t GPUs execute every layer in lockstep, just on different parameter shards. There is no stage sequencing, no micro-batch scheduling,

and no idle time waiting for upstream activations. The forward and backward passes look like single-GPU execution to the optimizer.

The cost is raw bandwidth. Each AllReduce sends $2 \cdot (1 - 1/t) \cdot s \cdot b \cdot h$ bytes per device. For a 70B model with $h = 8192$, batch size 32, and sequence length 2048:

$$s \times b \times h \times 2 \text{ bytes (BF16)} = 2048 \times 32 \times 8192 \times 2 \approx 1.07 \text{ GB}$$

Effective transfer per device in an 8-way AllReduce: $2 \times 1.07 \times (7/8) \approx 1.87 \text{ GB}$.

At NVLink effective AllReduce bandwidth of $\sim 600 \text{ GB/s}$: $1.87/600 \approx 3.1 \text{ ms}$ per AllReduce. A single MLP layer's forward GEMMs on one GPU (processing $1/8$ of the FFN width): roughly $2 \times 32 \times 2048 \times 8192 \times (28672/8) \approx 30.7 \text{ TFlop}$, which at H100's 989 TF/s (BF16) takes $\sim 31 \text{ ms}$. Communication (3.1 ms) is 10% of compute-consistent with the observed 10-12% throughput penalty at intra-node tensor parallel degree 8.

Cross the node boundary to InfiniBand (25 GB/s): the same 1.87 GB AllReduce takes $\sim 75 \text{ ms}$, which is twice the compute time. That is why tensor parallelism must stay intra-node, and why empirically going from degree 8 to degree 16 causes a $\sim 42\%$ throughput drop, and degree 32 drops $\sim 65\%$.

A Worked Example

A 70B transformer: $h = 8192$, FFN intermediate 28672, 80 layers, 64 attention heads. Tensor parallel degree $t = 8$ on one $8 \times \text{H100}$ NVLink node.

Parameter memory per device. Total model state in BF16: $70 \times 10^9 \times 2 \text{ bytes} = 140 \text{ GB}$. With $t = 8$:

$$\frac{140 \text{ GB}}{8} = 17.5 \text{ GB per GPU}$$

The remaining 62.5 GB of the 80 GB HBM budget is available for optimizer state (under ZeRO Stage 1), activations, and any KV cache.

Head count check. 64 attention heads, $t = 8$: each GPU handles exactly $64/8 = 8$ heads. Valid. If a model had 40 heads and you tried $t = 8$, the split would be non-integer—a design error that forces rounding and unbalanced load.

Communication per training step. With batch size 32, sequence 2048:

- AllReduce tensor size per layer: $32 \times 2048 \times 8192 \times 2 \text{ bytes} = 1.07 \text{ GB}$
- AllReduces per layer: 2 in forward (MLP out, attention out), 2 in backward—4 total
- 80 layers: $80 \times 4 \times 1.07 \text{ GB} = 342 \text{ GB}$ per forward+backward step
- At 600 GB/s NVLink effective bandwidth: $342/600 \approx 0.57 \text{ s}$ of communication

This sounds alarming until you recognize the communication is overlapped with compute: each AllReduce fires while the CPU is dispatching the next layer's kernel. The practical throughput penalty is 10-12%, not 37%.

What You Actually Decide in Practice

- **Cap tensor parallel degree at 8 (or the NVLink-connected GPU count for your node).** Beyond that, you are issuing AllReduces over InfiniBand, and throughput collapses. The 10-12% intra-node overhead is worth the memory savings; the 40-65% inter-node overhead is not. Even if a single node has 16 GPUs, benchmark at $t = 8$ versus $t = 16$ before committing—some NVSwitch fabrics degrade at full all-to-all traffic.
- **Attention heads and FFN width must be divisible by t .** When designing a model for $t = 8$, set head count to a multiple of 8 and FFN intermediate dimension to a multiple of 512. Misaligned shapes produce suboptimal GEMM tiling and can add 5-15% overhead on top of the communication cost.

- **Tensor parallelism does not consume batch size.** This distinguishes it from data parallelism and pipeline parallelism, both of which require large batches to hide their respective overheads. If your global batch size is already at the critical batch size threshold and you cannot increase it, tensor parallelism is the correct memory-scaling tool.
- **Pair with sequence parallelism to eliminate the residual activation memory.** Tensor parallelism leaves LayerNorm and Dropout unsharded—they operate on the full $s \times b \times h$ activation. Sequence parallelism distributes these operations along the sequence dimension and replaces the MLP/attention AllReduce with AllGather and ReduceScatter pairs of equal total cost. The combination reduces the non-matmul activation memory by a factor of t .
- **Do not extend tensor parallelism cross-node to compensate for a model that is too wide.** If TP-8 within a node cannot fit the model, the answer is to combine TP-8 intra-node with pipeline parallelism inter-node—not to stretch tensor parallelism across InfiniBand. The activation AllReduce pattern of tensor parallelism is uniquely hostile to high-latency links.

How This Shows Up in Interviews

- **“How would you fit a 70B model on a single 8×H100 node?”** Use tensor parallelism with $t = 8$: each GPU holds $140 \text{ GB} / 8 = 17.5 \text{ GB}$ of parameters. Split each weight matrix column-parallel (first projection) and row-parallel (second projection), AllReduce the output activations. One AllReduce per transformer layer per direction. Model fits; communication overhead on NVLink is around 10%.
- **“Why is tensor parallel degree capped at 8 in most production systems?”** Beyond 8 GPUs on a standard NVIDIA node, AllReduces cross InfiniBand. For a 70B model with batch 32, sequence 2048, each AllReduce is 1.87 GB; at 25 GB/s InfiniBand that is 75 ms versus 31 ms of compute—the GPU stalls more than it runs. Empirically: 10-12% overhead at degree 8 (intra-node NVLink), 42% at degree 16, 65% at degree 32.
- **“What determines the AllReduce size in tensor parallelism?”** The output activation of each projection: $s \times b \times h$ elements per layer, not the weight matrix size. This is why tensor parallelism traffic scales with sequence length and batch size, not with model depth.
- **Senior follow-up: “If you had to scale to 1024 GPUs for a single model, how would you combine tensor parallelism with other strategies?”** Apply TP-8 within each 8-GPU NVLink node, consuming the fast intra-node bandwidth for the activation AllReduces. Then apply pipeline parallelism across nodes to extend model depth coverage, with InfiniBand carrying only the small activation handoffs between pipeline stages. Data parallelism handles the remaining parallelism once the model fits. Never extend TP beyond the NVLink boundary. The senior insight is that each interconnect tier dictates which parallelism strategy uses it.

Key Takeaways

- Tensor parallelism splits weight matrices width-wise: column-parallel for the first projection (input is replicated; no upfront communication), row-parallel for the second projection (partial sums merged by one AllReduce).
- Cost is two AllReduces per transformer layer (forward and backward), each of size $s \times b \times h$ activations—weights never move.

- The hard rule: TP degree ≤ 8 on NVLink; going inter-node over InfiniBand causes 40-65% throughput collapse.
- TP does not consume batch size and adds no pipeline bubble, making it the primary memory-scaling tool when fast intra-node interconnects are available.
- Head count and FFN width must be divisible by t ; design models with multiples of 8 or 16 to keep GEMM tile shapes optimal.

12.2.2 Pipeline Parallelism and Bubble Reduction

Pipeline parallelism becomes necessary when a model is too large to fit even after slicing all weight matrices across the GPUs within a node using tensor parallelism, or when you must extend to a second node and cannot afford tensor parallelism's AllReduce traffic over InfiniBand. Rather than splitting matrices width-wise, pipeline parallelism cuts the network depth-wise: each device owns a contiguous group of layers and passes activations forward to the next stage in sequence. Every serious interview on large-scale training eventually asks about the pipeline bubble, and the answer that separates mid-level from senior is knowing the formula, knowing how to suppress the bubble with micro-batches, and knowing why zero-bubble scheduling is hard enough that two people at a frontier lab may be the only ones who understand their own implementation.

Partition L transformer layers into p equal *pipeline stages*, each assigned to one device. Device 0 runs layers $0 \dots L/p - 1$; device 1 runs layers $L/p \dots 2L/p - 1$; and so on. The forward pass is strictly sequential: device 0 finishes its layers for one micro-batch, sends the output activations to device 1 via a point-to-point send, then immediately begins the next micro-batch. The backward pass flows in reverse-gradients of activations travel from device $p - 1$ back to device 0.

The *pipeline bubble* is the idle time that arises during startup and teardown. In the startup phase, devices $1 \dots p - 1$ are idle while device 0 processes the first micro-batch. In the teardown phase, device 0 is idle while device $p - 1$ finishes. The ratio of wasted time to useful time is:

$$\text{bubble fraction} = \frac{p - 1}{m}$$

where m is the number of micro-batches that partition the global batch. With $p = 8$ stages and $m = 1$, the bubble fraction is 7-seven eighths of the wall-clock step is wasted. With $p = 8$ and $m = 32$, it drops to $7/32 \approx 22\%$ -still significant but workable.

The standard production schedule is **1F1B** (one-forward-one-backward): each device interleaves one forward micro-batch with one backward micro-batch. This keeps the number of live activation tensors bounded at p regardless of m , preventing activation memory from growing linearly with the micro-batch count.

Pipeline parallelism tiles the model across depth; the bubble fraction $(p - 1)/m$ is the price, and micro-batch count is the only handle that reduces it without adding complexity.

The decisive advantage over tensor parallelism is communication volume. Tensor parallelism AllReduces the full activation tensor at every layer boundary-proportional to $s \times b \times h$ per layer per direction. Pipeline parallelism transmits activations only at stage transitions: a single point-to-point send of size $s_\mu \times b_\mu \times h$ per micro-batch, where b_μ is the micro-batch size. For a large model with small micro-batches this is one to two orders of magnitude less inter-device traffic per step, and critically, this communication is a one-time handoff-not repeated per layer.

This makes pipeline parallelism viable over InfiniBand where tensor parallelism fails. A 201 MB activation handoff at 25 GB/s takes about 8 ms. A single pipeline stage's compute for a 175B model takes hundreds of milliseconds. The communication cost is well under 5% of compute time, compared to tensor parallelism's

40-65% overhead at the same link speed.

Pipeline parallelism also shards activation memory in proportion to the number of stages. Each device holds activations only for its L/p layers rather than the full model. This is something ZeRO stages 1-3 cannot provide—they shard model state but leave activations fully on each data-parallel worker. For long-sequence training where activations dominate, pipeline parallelism combined with micro-batch 1F1B scheduling is the primary lever.

A Worked Example

Consider a 175B transformer ($h = 12288$, 96 layers, FFN intermediate = 49152) distributed across 64 A100 80 GB GPUs using $p = 8$ pipeline stages and 8-way data parallelism.

Layer assignment and memory: Each stage handles $96/8 = 12$ layers. Parameter memory per device in BF16:

$$\frac{175 \times 10^9 \times 2 \text{ bytes}}{8} = 43.75 \text{ GB}$$

leaving roughly 36 GB per device for optimizer state, activations, and buffers.

Bubble suppression: Global batch 1024. Each data-parallel replica processes $1024/8 = 128$ samples. With micro-batch size 4: $m = 128/4 = 32$ micro-batches. Bubble fraction:

$$\frac{8-1}{32} = \frac{7}{32} \approx 21.9\%$$

GPU utilization: $\sim 78\%$. Compare to $m = 4$: bubble = $7/4 = 175\%$ of useful compute—completely unusable without larger batches.

Communication per stage transition: Each activation handoff is the output of one micro-batch after one stage:

$$b_\mu \times s \times h \times 2 \text{ bytes} = 4 \times 2048 \times 12288 \times 2 \approx 201 \text{ MB}$$

At InfiniBand 25 GB/s: $201 \text{ MB} / 25 \text{ GB/s} \approx 8 \text{ ms}$.

Compute per stage: A single layer's MLP forward pass, per GPU:

$$2 \times 4 \times 2048 \times 12288 \times 49152 \approx 9.88 \text{ TFlop}$$

At 312 TF/s (A100 BF16): $\approx 32 \text{ ms}$ per MLP layer. With 12 layers per stage: $\sim 384 \text{ ms}$ of compute per micro-batch step. The 8 ms handoff is $\sim 2\%$ of stage compute—negligible. This is precisely why pipeline parallelism survives on InfiniBand.

What You Actually Decide in Practice

- **Target micro-batch count $m \geq 4(p-1)$ to keep bubble below 25%.** With $p = 8$, this means $m \geq 28$. If your global batch cannot support this micro-batch count, pipeline parallelism is imposing a large and unrecoverable compute penalty. Either reduce pipeline depth or switch to FSDP if the model fits.
- **Zero-bubble scheduling (DualPipe) fills bubble slots with weight-gradient work.** The standard backward pass has two independent phases: computing activation gradients (B , which is a serial dependency—earlier layers need it before they can proceed) and computing weight gradients (W , which has no serial dependency and can run any time after the corresponding forward pass). By deferring W computations into the idle bubble slots, you approach 100% GPU utilization. DeepSeek-V3 uses a variant of this called DualPipe in production. The catch: interposing on autograd to separate the B and W phases requires custom gradient hooks, a scheduling queue for W operations, and careful management of intermediate activations that must be kept alive between the B and W steps. The anecdote I keep hearing from en-

engineers at frontier labs is that two people in the training infrastructure group understand their pipeline parallel code-and when one of them leaves, the codebase becomes a liability.

- **Align pipeline stage boundaries with physical node boundaries.** An activation handoff that crosses a single InfiniBand hop costs 8 ms for a 200 MB tensor. One that crosses two hops (leaf switch to spine switch) can cost 30+ ms. Design stages so that each stage lives entirely within one node, and the handoffs stay on the lowest-latency inter-node link available.
- **Batch size is a shared resource between data parallelism and pipeline parallelism.** If you run 8-way data parallel with $p = 4$ pipeline stages and need $m = 12$ micro-batches per replica, the effective per-replica batch is $b_{\mu} \times 12$. Verify that your global batch size supports this without pushing below the critical batch size threshold-beyond which gradient noise no longer benefits from larger batches.
- **Prefer FSDP over pipeline parallelism whenever the model fits.** FSDP is far simpler to implement, has no bubble, and is architecture-agnostic. Pipeline parallelism is the right answer only when (a) the model cannot fit per-node even with FSDP and TP-8, or (b) inter-node bandwidth is too low for FSDP's $3|\theta|$ per-step communication, or (c) you have a very large global batch that can suppress the bubble to an acceptable level.

How This Shows Up in Interviews

- **“Explain the pipeline bubble and how you would reduce it.”** The bubble arises because during startup, later stages idle while device 0 fills them; during teardown, earlier stages idle while the last stage drains. Fraction = $(p-1)/m$. To reduce: increase m (more micro-batches), or use 1F1B scheduling to interleave forward and backward passes while bounding activation memory at p live tensors regardless of m . Cite a concrete example: $p = 8$, $m = 32$, bubble = $7/32 \approx 22\%$, utilization $\approx 78\%$.
- **“When would you choose pipeline parallelism over FSDP?”** When the model must scale across nodes and the inter-node bandwidth is too low for FSDP's AllGather-per-layer pattern (which costs $3|\theta|$ per step), or when the model is large enough that even FSDP cannot fit it per node. Pipeline parallelism's activation handoffs are one to two orders of magnitude smaller than FSDP's weight AllGathers at the same model size, making it viable over 25 GB/s InfiniBand links where FSDP stalls.
- **Senior follow-up: “Describe zero-bubble pipelining and what it costs to implement.”** The key insight is that the backward pass has two independent phases: B (activation gradients, required by earlier stages) and W (weight gradients, locally executable any time after the corresponding forward). The W computations can be deferred to slots where the pipeline would otherwise be idle. To implement this, you must intercept autograd at the layer level to separate B and W , maintain a deferred queue for W work, and track which intermediate activations need to stay alive across the separation. The memory overhead for deferred activations can partially eat back the efficiency gains, and the scheduling logic is complex enough that teams at production-scale labs have had serious bus-factor problems with it. A strong senior answer names the autograd intervention, the memory tracking complexity, and gives a real example (DualPipe in DeepSeek-V3).

Key Takeaways

- Pipeline parallelism partitions the model by layers: device k owns L/p consecutive layers and communicates only output activations-not weights-to the next stage via point-to-point sends.

- Bubble fraction = $(p - 1)/m$; with $p = 8$ and $m = 32$, GPU utilization is $\sim 78\%$. Target $m \geq 4(p - 1)$ to keep the bubble under 25%.
- Activation handoffs are small (one tensor per micro-batch per stage transition), making pipeline parallelism viable over InfiniBand where tensor parallelism's per-layer AllReduces are not.
- Zero-bubble scheduling splits the backward pass into B (serial activation gradients) and W (local weight gradients), filling bubble slots with W work-at the cost of significant autograd infrastructure complexity.
- The production rule: TP-8 intra-node, pipeline parallel inter-node, data parallel for the remaining GPU count; set $m \geq 4(p - 1)$; use FSDP instead of pipeline parallel whenever the model fits per node.

12.3 Activation and Sequence Parallelism

12.3.1 Sequence Parallelism for LayerNorm and Dropout

After you've walked an interviewer through Megatron-style tensor parallelism-column-parallel first projection, row-parallel second projection, one AllReduce per layer boundary-the senior follow-up is almost always: "Those AllReduces shard the matmul outputs across GPUs. But what about LayerNorm and Dropout? Those operators see the full activation, right?" Most candidates who prepped TP can't answer this without hesitation. The unsharded pointwise operations are the hole in the tensor-parallel memory story, and at sequence lengths above 4096 they can consume more HBM than the weight matrices you just worked so hard to shard. Sequence parallelism, introduced alongside the Megatron-LM tensor-parallel formulation, closes the gap at exactly zero additional bandwidth cost.

LayerNorm and Dropout are embarrassingly parallel along the sequence dimension. Each position normalizes independently of every other position; Dropout applies a per-element mask with no cross-position dependency. This means we can partition the s -length sequence across the same t tensor-parallel GPUs and run these pointwise ops on an s/t -token slice, exactly as if we had cut the batch into micro-batches but along the sequence axis instead.

The wrinkle is that the column-parallel matmul that follows each LayerNorm needs the full-sequence activation on every GPU. The synchronization point must therefore change. Standard tensor parallelism issues a single AllReduce after the row-parallel matmul to merge partial sums and replicate the result everywhere. Sequence parallelism replaces this with a two-part operation:

1. **ReduceScatter** after the row-parallel matmul: partial sums are reduced across the group, and the result is *scattered* so that GPU k holds the fully-reduced rows for sequence positions $[k \cdot s/t, (k + 1) \cdot s/t)$. No other GPU holds those rows.
2. LayerNorm, Dropout, and any other pointwise op run on the local s/t -token slice.
3. **AllGather** before the next column-parallel matmul: each GPU's slice is concatenated so that every GPU sees the full s -length sequence for the subsequent projection.

By the identity $\text{AllReduce} \equiv \text{ReduceScatter} + \text{AllGather}$ in the bandwidth-limited regime, the total bytes transferred are unchanged. The ReduceScatter ships the same data as the first half of an AllReduce ring; the AllGather ships the same data as the second half. Sequence parallelism is literally a rearrangement of when the data moves, not an addition of new communication. The only change visible in the memory profile is that the activation tensor between the ReduceScatter and the AllGather has shape $(s/t) \times b \times h$ instead of $s \times b \times h$ -a factor-of- t reduction for every byte spent on LayerNorm input, LayerNorm output, Dropout, and the residual add.

The net activation memory per transformer layer, after applying both tensor parallelism and sequence parallelism and using FlashAttention to eliminate the quadratic softmax term, is:

$$\text{Activation memory per layer} = \frac{sbh \times 34}{t} \text{ bytes (BF16)}$$

Without sequence parallelism the formula was $sbh \times 34/t + sbh \times 10$: the second term, representing LayerNorm, Dropout, and input embeddings, stayed constant regardless of t . Sequence parallelism removes that stranded term.

Sequence parallelism converts the one part of the activation that tensor parallelism could not touch into a term that scales with t -at zero bandwidth cost.

The AllReduce in vanilla tensor parallelism does the work of reduce-and-scatter and then immediately throws away the scattered structure by copying the result to all GPUs. Sequence parallelism simply stops discarding that structure. The ReduceScatter leaves each GPU holding exactly the rows it needs to run LayerNorm on its sequence slice; the AllGather restores the full tensor before the next matmul requires it. The data travels the same physical distance over the same wires; the only difference is how the receive buffer is allocated.

The alternative, running LayerNorm on the full sequence on every GPU and then AllReducing, is what you get by default with TP but without SP. It is not incorrect-the gradients compute cleanly-but it wastes $sbh \times 10$ bytes per layer per GPU, regardless of how aggressively you parallelized the weight matrices. For a 13B model at sequence 4096 and $t = 8$, that stranded term is over 60 GB of HBM. Sequence parallelism costs nothing to recover it.

A second advantage is composability. Because TP and SP operate on the same communicator group with the same degree, adding SP requires no new topology decisions, no new process groups, and no change to the pipeline or data-parallel configuration. In Megatron-LM, enabling SP is a single flag. Frameworks that expose TP without SP are leaving memory on the table.

A Worked Example

A 13B-parameter transformer: $h = 5120$, 40 layers, 80 attention heads, FFN width 20480. Training configuration: batch $b = 4$, sequence $s = 4096$, tensor and sequence parallel degree $t = 8$, BF16 weights.

Without sequence parallelism. After tensor parallelism, the matmul activations (FFN intermediates, attention projection outputs) divide by t . The pointwise activation per layer does not:

$$sbh \times 10 \times 2 \text{ bytes} = 4096 \times 4 \times 5120 \times 10 \times 2 = 1,678 \text{ MB} \approx 1.56 \text{ GB per layer}$$

Over 40 layers: $40 \times 1.56 = 62.4 \text{ GB}$. This occupies 78% of an H100's 80 GB HBM before a single weight matrix or optimizer state is counted.

With sequence parallelism ($t = 8$):

$$\frac{sbh \times 10 \times 2 \text{ bytes}}{t} = \frac{1.56 \text{ GB}}{8} \approx 195 \text{ MB per layer}$$

Over 40 layers: $\approx 7.8 \text{ GB}$. Savings: $62.4 - 7.8 = 54.6 \text{ GB}$ per GPU-nearly a full H100 freed.

Communication cost check. The AllReduce in vanilla TP transfers per device:

$$2 \times sbh \times \left(1 - \frac{1}{t}\right) \times 2 \text{ bytes} = 2 \times 4096 \times 4 \times 5120 \times \frac{7}{8} \times 2 \approx 1,468 \text{ MB}$$

The ReduceScatter + AllGather pair transfers the same total bytes per device by the AllReduce identity. Communication overhead: zero.

Minimum total activation per layer after TP + SP + FlashAttention:

$$\frac{sbh \times 34}{t} \times 2 \text{ bytes} = \frac{4096 \times 4 \times 5120 \times 34}{8} \times 2 \approx 2,863 \text{ MB} \approx 2.80 \text{ GB per layer}$$

This is the floor achievable without gradient checkpointing.

What You Actually Decide in Practice

- **Always enable SP whenever you enable TP.** There is no scenario where you want tensor parallelism without sequence parallelism: SP is strictly better for memory and has identical communication cost. Frameworks that separate the two flags default SP to off; always check.
- **SP degree equals TP degree-they share a communicator.** If TP is 8, SP is 8. They are not independently configurable, and there is no benefit to mismatching them.
- **Sequence length must be divisible by t .** A 4096-token sequence with $t = 8$ gives exactly 512 tokens per GPU. If your sequence length is not a multiple of t , pad to the next multiple. Uneven partitions require masked gather operations and add significant implementation complexity.
- **The stranded $sbh \times 10$ term is the right diagnostic when TP memory savings disappoint.** If you apply TP-8 and memory usage drops by less than $8\times$, the culprit is almost always the unsharded pointwise activations. Confirm by enabling SP and measuring again.
- **After TP+SP+FlashAttention, further memory reduction requires activation recomputation.** The minimum activation footprint is $sbh \times 34/t$ per layer. Reducing below this floor means discarding and recomputing activations during the backward pass at a cost of roughly 33% more forward-pass FLOPs. That trade is often worth it when the alternative is a smaller batch size that hurts data-parallel efficiency.

How This Shows Up in Interviews

- **“Tensor parallelism divides your parameter memory by t . Does it also divide your activation memory by t ?”** Strong answer: not entirely. The matmul activations (FFN intermediate, attention projections) divide by t . The pointwise activations-LayerNorm input and output, Dropout-stay full-size on every GPU in vanilla TP. Sequence parallelism fixes this by replacing the row-parallel AllReduce with ReduceScatter + AllGather; GPU k then holds only s/t rows through the pointwise ops, and the memory for those layers divides by t as well. Total communication cost is unchanged.
- **“Why is sequence parallelism said to be free?”** The AllReduce and the ReduceScatter + AllGather pair are bandwidth-equivalent in the ring-based implementation: both move the same number of bytes per device. SP rearranges *when* the scatter happens but does not increase the total data transferred.
- **Senior follow-up: “Walk me through the exact communication ops for one transformer layer under TP+SP.”** Strong answer: (1) Input arrives as s/t rows on each GPU (scattered from the previous layer’s ReduceScatter). (2) AllGather before the column-parallel QKV or FFN projection: each GPU now holds the full $s \times h$ activation. (3) Column-parallel matmul: each GPU computes its $4h/t$ columns, no communication. (4) Row-parallel matmul: each GPU holds a partial sum over the full h -dim output. (5) ReduceScatter: partial sums merged, GPU k keeps its s/t rows. (6) LayerNorm and Dropout on s/t rows. (7) Residual add with the corresponding s/t -row slice held from before step (2). Repeat per layer. Two collective ops-AllGather and ReduceScatter-per MLP and per attention block, identical total bytes to the AllReduce they replace.

Key Takeaways

- Tensor parallelism leaves LayerNorm and Dropout unsharded: their per-layer activation cost is $sbh \times 10 \times 2$ bytes per GPU, independent of t .

- Sequence parallelism distributes these pointwise ops along the sequence dimension by replacing the TP AllReduce with ReduceScatter + AllGather-same bandwidth, $1/t$ activation memory for the pointwise ops.
- After TP + SP + FlashAttention, the minimum per-layer activation footprint is $sbh \times 34/t$ bytes per GPU.
- SP degree always equals TP degree; sequence length must be divisible by t .
- For a 13B model at $s = 4096$, $b = 4$, $t = 8$, enabling SP recovers ≈ 55 GB per GPU-the difference between fitting comfortably on an H100 and running out of memory before the optimizer state is even allocated.

12.3.2 Context Parallelism and Ring Attention

Sequence parallelism cuts the LayerNorm and Dropout memory by t , and FlashAttention removes the quadratic softmax matrix from the per-GPU footprint entirely-for sequences that fit within a single GPU's SRAM tiling budget. At $s = 4096$ that budget is comfortable. At $s = 128,000$, the sequence LLaMA-3 trained its long-context variant at, the FlashAttention tile loop itself requires K and V tensors of $128K \times h$ elements to be addressable from a single device's HBM, which at $h = 8192$ and BF16 is $128,000 \times 8192 \times 2 \times 2$ bytes ≈ 4.2 GB per layer just for the KV pair-before any attention scores, before the 80 remaining layers, before gradients. Context parallelism, also called ring attention, distributes the attention computation itself across p devices and is the last activation-memory tool you need to understand before discussing long-context training at the design-interview level.

In context parallelism, the sequence is partitioned across p GPUs. GPU k permanently holds the query, key, and value tokens for positions $[k \cdot s/p, (k+1) \cdot s/p)$. To produce the correct attention output for its query slice, GPU k must compute inner products with all s keys-including the $s(1 - 1/p)$ keys it does not own. Ring attention solves this by rotating the K and V chunks around a logical ring of p GPUs while each GPU keeps its own Q shard fixed.

In each of p ring steps, GPU k :

1. Receives the K, V chunk from its left neighbor.
2. Computes the partial attention scores $Q_k K_j^T / \sqrt{d_{\text{head}}}$ for the received chunk j and accumulates the weighted values into a running output buffer.
3. Updates its running online-softmax statistics: the current maximum logit m and the running log-sum-exp denominator ℓ .
4. Passes the K, V chunk to its right neighbor while simultaneously receiving the next chunk from the left.

After p steps, GPU k has seen all s keys and values, and its output buffer holds the exact same attention result it would have computed on a single GPU with the full sequence in memory. The mathematical correctness follows from the associativity of the online-softmax merge rule used inside FlashAttention. When a GPU receives a new KV chunk, it merges the new partial output O_{new} into the running O using:

$$m' = \max(m, m_{\text{new}}), \quad \ell' = e^{m-m'} \ell + e^{m_{\text{new}}-m'} \ell_{\text{new}}, \quad O' = \frac{e^{m-m'} O + e^{m_{\text{new}}-m'} O_{\text{new}}}{\ell'}$$

This update is associative over chunks in any order, so ring rotation produces bit-identical output to sequential full-sequence attention.

Communication cost per layer per device: each ring step sends one chunk of K and one chunk of V, each of shape $s/p \times d_{\text{head}} \times n_{\text{heads}}$, for $p - 1$ steps:

$$(p - 1) \times \frac{s}{p} \times d_{\text{head}} \times n_{\text{heads}} \times 2 \times 2 \text{ bytes} \approx 2 \cdot s \cdot h \cdot 2 \text{ bytes}$$

This is $O(s)$ -linear in sequence length, the same order as a single AllGather of the full K or V tensor-while the memory saving is $O(s^2/p)$.

Ring attention converts the $O(s^2)$ attention memory problem into an $O(s)$ communication problem that is independent of p , leaving the sequence-length scaling entirely in the network rather than in HBM.

FlashAttention eliminates the full $s \times s$ softmax matrix from SRAM by tiling the key-query computation in blocks, but it still requires all s keys and values to be reachable from the GPU's HBM within a single forward pass. At $s = 128K$, the K and V matrices per layer already consume several gigabytes per device, and they must be replicated for the backward pass. Ring attention removes this requirement: each GPU holds s/p keys and values permanently and receives the remaining $(p-1)/p$ fraction as a transient stream, one chunk at a time, overlapping communication with the attention tile computation for the chunk it just received. The peak local KV footprint is therefore s/p , not s .

The alternative-splitting the sequence with sequence parallelism and hoping FlashAttention's block loop handles the rest-breaks down because the block loop still iterates over all s key positions. Sequence parallelism distributes the *query* dimension but does not distribute the *key-value* dimension. Ring attention distributes both.

Compared to a naive all-to-all approach where every GPU first AllGathers the full KV and then runs local FlashAttention, ring attention is strictly more memory-efficient: the all-to-all approach requires each GPU to hold all s keys in memory simultaneously, defeating the purpose. The ring rotates through chunks that can be freed as soon as the corresponding attention tile is accumulated.

A Worked Example

LLaMA-3 70B long-context training configuration: $h = 8192$, $n_{\text{heads}} = 64$ (GQA with 8 KV heads but illustrated here for full attention), $d_{\text{head}} = 128$, 80 layers, sequence $s = 131,072$ (128K tokens), batch $b = 1$, context parallel degree $p = 8$, tensor parallel degree $t = 8$.

KV memory without context parallelism (per GPU, per layer):

$$s \times n_{\text{heads}} \times d_{\text{head}} \times 2 \times 2 \text{ bytes} = 131,072 \times 64 \times 128 \times 4 \approx 4.29 \text{ GB per layer}$$

Over 80 layers: ≈ 343 GB-impossible on any single-node system.

With context parallelism ($p = 8$): each GPU holds $131,072/8 = 16,384$ tokens permanently. Local KV per layer:

$$\frac{4.29 \text{ GB}}{8} \approx 537 \text{ MB per layer, } 80 \text{ layers} \approx 41.9 \text{ GB}$$

Combined with $t = 8$ TP (which further shards head groups in the KV projection), the actual per-device KV footprint is closer to $41.9/8 \approx 5.2$ GB, leaving most of the 80 GB H100 HBM for parameters, optimizer state, and gradient buffers.

Ring communication per layer per device:

$$(p-1) \times \frac{s}{p} \times n_{\text{heads}} \times d_{\text{head}} \times 2 \times 2 \text{ bytes} = 7 \times 16,384 \times 64 \times 128 \times 4 \approx 3.76 \text{ GB}$$

At NVLink ring-step bandwidth ~ 600 GB/s: approximately 6.3 ms of communication per layer. Attention compute for 16K queries over 128K keys on one GPU is $\sim 10\times$ longer at typical H100 throughput, so communication is largely masked by overlapping ring sends with tile accumulation.

What You Actually Decide in Practice

- **Enable context parallelism only at $s \gtrsim 32K$.** Below that threshold, FlashAttention within a TP group handles the attention memory without the ring-communication overhead. LLaMA-3 used CP only in the final long-context fine-tuning phase; the main pre-training run did not need it.
- **Causal attention requires a striped sequence partition.** In contiguous partitioning (GPU k holds tokens $[k \cdot s/p, (k+1) \cdot s/p)$), GPU $p-1$ holds only the last block of tokens, which must attend to all earlier tokens held by all other GPUs. This creates severe load imbalance: GPU 0 computes a nearly zero attention matrix (only self-attention, all future tokens masked), while GPU $p-1$ computes a nearly full triangle. The standard fix is *striped* or *interleaved* partitioning: GPU k holds every p -th token, distributing causal load evenly. Stripe partitioning is the default in production ring-attention implementations.
- **Variable-length batches require padding or sequence packing.** Ring attention assumes uniform sequence length within the CP group. If sequences vary, pad each to the next multiple of p , or use sequence packing with separate attention masks per document. Packing is significantly more complex to implement correctly and requires masked attention kernels that are aware of document boundaries within the padded token stream.
- **CP and TP use separate process groups.** The p ring-rotation peers form a CP communicator; the t tensor-parallel peers form a TP communicator. In LLaMA-3 with $t = 8$ and $p = 8$, a batch item involves 64 GPUs in combined TP×CP parallelism. The two communicators do not interfere, but scheduling their collective operations without stalling requires careful overlap planning in the training loop.
- **Allocate two KV buffers per device.** At any ring step, a GPU holds its own outgoing KV chunk and the incoming chunk from its neighbor simultaneously. Ping-pong buffering between two HBM allocations—one for send, one for receive—is standard. Forgetting this doubles the apparent per-GPU KV overhead in memory estimates.

How This Shows Up in Interviews

- **“How would you design a training system for 128K context?”** Strong answer: within each 8-GPU NVLink node, apply TP=8 for weight sharding and SP=8 for LayerNorm activation sharding. For the attention computation, apply CP=8 across the same node group (TP and CP share the NVLink fabric via separate communicators), rotating KV in a ring while each GPU holds 16K queries and uses online-softmax tile accumulation. Use pipeline parallelism across nodes for model depth. The CP ring adds roughly one AllGather-equivalent of communication per attention layer per device, maskable by overlapping ring sends with tile computation.
- **“Why doesn’t sequence parallelism alone solve the 128K context memory problem?”** SP splits the sequence for LayerNorm and Dropout. Each GPU still needs to iterate over all s keys during FlashAttention’s tile loop, so the KV tensors must be accessible from every device’s HBM— $O(s)$ per tensor, $O(s)$ per layer, blowing up at $s = 128K$. Ring attention extends the sequence split into the attention computation itself, so no GPU ever holds more than s/p KV positions in its local memory at once.
- **Senior follow-up: “Why is ring attention mathematically exact?”** Strong answer: the online-softmax accumulator maintains three statistics per attention head: running maximum logit m , running log-sum-exp denominator ℓ , and weighted output sum O . When a new KV chunk arrives, these three values can be merged from two partial results using the update equations above, which are associative and commutative over any partitioning of the key dimension. Ring attention applies this merge protocol over p ring steps

instead of over SRAM tiles; since the same merge rule applies in both cases, the output is bit-identical to single-GPU full-sequence attention.

Key Takeaways

- Context parallelism (ring attention) distributes the attention computation across p GPUs by rotating K and V through a logical ring while each GPU keeps its own Q shard fixed.
- The online-softmax merge rule (from FlashAttention) guarantees mathematical exactness: the accumulated output across p ring steps is identical to single-GPU attention.
- Local KV memory per GPU per layer scales as $O(s/p)$; ring communication cost scales as $O(s)$ bytes per device-linear, not quadratic.
- Causal attention requires a striped (interleaved) sequence partition to avoid severe load imbalance across ring members.
- CP stacks with TP and SP on separate process groups; LLaMA-3 uses $p = 8$, $t = 8$, engaging 64 GPUs in combined attention-and-weight parallelism for a single 128K-token batch item.

12.4 Putting It All Together: 3D and 4D Parallelism

12.4.1 Rules of Thumb for Choosing a Mix

Once you have all the parallelism strategies in hand—tensor, pipeline, data, and sequence—the question that separates a strong interview answer from a great one is: in what order do you apply them, and why? This composition question appears at senior and staff-level interviews at frontier labs, typically framed as “walk me through how you would set up training for a 100B model on a thousand GPUs.” The answer is not a free choice. It is constrained by hardware topology, model size, and a finite batch-size budget, and those constraints resolve into a repeatable recipe that practitioners call *3D parallelism*—tensor, pipeline, and data—optionally extended to *4D* by adding sequence or expert parallelism.

The central insight is that different parallelism strategies impose different communication patterns and tolerate different network bandwidths. A GPU cluster is not a flat network: GPUs within a single machine communicate via NVLink or NVSwitch at roughly 600 GB/s aggregate per GPU (on H100s), while GPUs on different machines communicate over InfiniBand at roughly 25-50 GB/s per GPU—an 8-20 $\times$ gap. Applying a high-bandwidth-hungry strategy across the slower link wastes throughput; the converse is merely suboptimal.

Tensor parallelism requires an all-reduce per transformer layer, carrying activation tensors proportional to $s \times b \times h$ (sequence length, micro-batch size, hidden dimension). At hidden size 8192 and sequence length 4096, a single all-reduce moves hundreds of megabytes. NVLink saturates this in under a millisecond; InfiniBand would take tens of milliseconds, dominating compute time entirely. Tensor parallelism must stay inside a single NVSwitch domain, typically 8 GPUs.

Pipeline parallelism transfers activations point-to-point between consecutive stages—tensors of size $s \times b \times h$ per stage boundary—and tolerates InfiniBand latency reasonably well. Its cost comes from the pipeline bubble: the fraction of time GPUs sit idle waiting for earlier stages. The bubble ratio is $(p-1)/m$, where p is the number of pipeline stages and m is the number of micro-batches in a step. Large m hides the bubble; small m wastes compute.

Data parallelism synchronizes gradients once per training step. With ZeRO stage 1, that is a reduce-scatter plus an all-gather over the full parameter count P (total cost: $2P$ bytes); with ZeRO stage 3, it is $3P$ bytes spread

across many small operations that can be overlapped with computation. Amortized over a full forward-backward pass, this communication tolerates slow inter-node links-provided the batch is large enough for compute to dominate.

The composition follows directly:

1. **Tensor parallelism first**, up to the GPU count per machine (typically 8), exploiting NVLink bandwidth.
2. **Pipeline parallelism or ZeRO-3 (FSDP) second**, across nodes, until the model fits in aggregate GPU memory.
3. **Data parallelism third**, across all remaining GPUs, scaling throughput until the batch-size budget is exhausted.

Adding a fourth dimension-sequence parallelism (SP), context parallelism (CP), or expert parallelism (EP)-handles long-context activations or sparse MoE routing within this framework.

Match the most communication-intensive strategy to the fastest link; let data parallelism absorb the slow tail of the network hierarchy.

The *batch size as resource* framing makes the trade-offs concrete. A fixed global batch size B is dictated by optimization stability-the critical batch size beyond which per-step sample efficiency saturates rapidly. Every parallelism dimension that requires micro-batches competes for that budget.

Pipeline parallelism needs $m \geq 4p$ micro-batches to keep the bubble below roughly 20%. With $p = 8$ pipeline stages, you need $m \geq 32$ micro-batches per DP replica per step. If your global batch is 2048 sequences and DP degree is 64, each replica has 32 sequences-exactly 32 micro-batches of size 1. Increasing p to 16 demands $m \geq 64$, but the batch budget may be exhausted.

Data parallelism has no bubble, but without ZeRO it does not reduce per-GPU parameter memory. Pure DP on a 100B model requires every GPU to hold the full model; at bf16 that is 200 GB, exceeding any current GPU's HBM. ZeRO-3 shards parameters across all DP ranks but adds $3P$ communication overhead. For large DP degree and sufficient compute time per step, this communication is overlapped and affordable.

A useful mental picture from the TPU literature: plot compute efficiency against the ratio of batch size to GPU count. At low ratios, any strategy is communication-bound. At high ratios, pure data parallelism reaches the compute-bound regime on its own. The mixed TP + PP + DP regime occupies the middle, where no single strategy suffices and composition is the only path to high utilization.

Compared to pipeline parallelism, ZeRO-3 (FSDP) offers a simpler trade-off: $3P$ communication per step regardless of model depth, no bubble, but higher parameter communication volume. For models whose batch budget cannot sustain a large pipeline, ZeRO-3 is almost always the right choice over PP.

A Worked Example

Consider a 70B dense transformer ($h = 8192$, 80 layers, 64 attention heads) on 256 H100 80 GB GPUs arranged as 32 nodes of 8 GPUs each. Target global batch: 2048 sequences of 4096 tokens, bf16 mixed precision.

Step 1 - Tensor parallelism. Set $TP = 8$ within each node. Parameter memory per GPU drops from $70B \times 2 \text{ bytes} = 140 \text{ GB}$ to $140/8 = 17.5 \text{ GB}$. The model fits on a single node after TP.

Step 2 - Pipeline check. With $TP = 8$ sufficient to fit parameters, pipeline parallelism is not required. Each 8-GPU node holds one TP-sharded model replica.

Step 3 - Data parallelism. With 32 nodes and one model replica per node, DP degree $D = 32$. Each replica processes $2048/32 = 64$ sequences per step.

Step 4 - Optimizer state. With ZeRO stage 1, first and second moment tensors (fp32, $\approx 560 \text{ GB}$ total for

the model) are sharded across 32 DP ranks: each rank stores ≈ 17.5 GB of optimizer state. Per-GPU total: 17.5 (params) + 17.5 (optimizer) + 17.5 (gradients in bf16) ≈ 52.5 GB-comfortably below 80 GB before activations.

Step 5 - Gradient sync cost. After the backward pass, a reduce-scatter plus all-gather over 140 GB of gradients (DP degree 32) moves $140/32 \approx 4.4$ GB per rank. At 25 GB/s effective InfiniBand bandwidth, this completes in ≈ 175 ms. A full forward-backward pass for a 70B model at this batch size runs on the order of 200-250 ms on H100s, so gradient synchronization is almost fully overlapped with the tail of the backward pass.

Now scale to 405B: parameters at bf16 are 810 GB. After $TP = 8$, each GPU still needs $810/8 = 101$ GB for parameters alone, exceeding HBM. Adding $PP = 8$ splits 80 layers into 8 pipeline stages of 10 layers each; per-GPU parameter memory drops to ≈ 12.6 GB. Pipeline parallelism was not chosen-it was forced by memory.

What You Actually Decide in Practice

- **Cap TP at 8, always.** Crossing a node boundary for tensor parallelism forces per-layer all-reduces over InfiniBand. Published benchmarks show 40%+ throughput degradation going from $TP = 8$ (intra-node) to $TP = 16$ (cross-node). This is a consequence of bandwidth physics, not software; it holds across GPU generations.
- **Size the pipeline to keep the bubble under 20%.** Use $m \geq 4p$ as a starting point. With $PP = 8$, target at least 32 micro-batches per DP replica per step. If your global batch cannot support this, prefer ZeRO-3 over pipeline parallelism.
- **Choose ZeRO-3 over PP when batch sizes are tight.** ZeRO-3's overhead ($3P$ bytes per step, partially overlapped with computation) is predictable and independent of model depth. Pipeline parallelism's bubble grows with stage count regardless of tuning, and its scheduling complexity is non-trivial-production teams have described situations where a single engineer understands the pipeline scheduler.
- **Add sequence parallelism alongside TP at no extra bandwidth cost.** SP replaces the TP all-reduce on activation tensors with a reduce-scatter/all-gather pair of equivalent total cost, while also partitioning LayerNorm and Dropout activations across the TP group. Minimum activation memory per GPU drops by a factor of the TP degree; use it whenever $TP \geq 4$.
- **Use gradient accumulation to rescue a tight micro-batch budget.** If each DP replica can hold only micro-batch size 1 after TP and PP, accumulate k backward passes before syncing. Effective micro-batch count becomes k , shrinking the bubble from $(p-1)/m$ to $(p-1)/(km)$ at the cost of k -fold longer steps. This is preferable to reducing DP degree and cutting overall throughput.

How This Shows Up in Interviews

- **“Design the parallelism strategy for a 100B dense transformer on 512 H100s.”** Strong answer: set $TP = 8$ within each 8-GPU node (64 nodes total, 64 TP groups). A 100B model at bf16 is 200 GB; after $TP = 8$, 25 GB per GPU-fits without PP. Set $DP = 64$. With global batch 2048, each replica gets 32 sequences. Add ZeRO-1 to shard optimizer state across DP ranks. Explain why TP stays at 8, why PP is unnecessary here, and how gradient sync overlaps with computation.
- **“Why can’t you just scale data parallelism to 10,000 GPUs?”** Strong answer: DP alone does not reduce per-GPU parameter memory without ZeRO-3. Beyond the critical batch size, adding more DP replicas does not improve sample efficiency-you run out of useful batch-size budget before you run out of GPUs. ZeRO-3 adds $3P$ synchronization cost that becomes harder to overlap at very high DP degree, and parameter communication across spine switches introduces topology-dependent latency.

- **Senior follow-up: “A 400B+ training run spans multiple rack spine switches. How does this affect your PP assignment?”** Strong answer: assign consecutive pipeline stages to physically adjacent nodes within the same rack or pod. PP’s point-to-point activation transfers stay on high-bandwidth leaf switch fabric; DP’s gradient all-reduces must traverse spines but happen once per step and tolerate additional latency through asynchronous overlap. Modern cluster schedulers expose rack-locality hints; use them to constrain the PP group placement before setting DP. The failure to do this is one of the common causes of unexplained throughput variance at large scale.

Key Takeaways

- Apply TP first (cap at 8, within a single NVSwitch domain), then PP or ZeRO-3 until the model fits in memory, then DP for remaining GPUs.
- Batch size is a finite resource: micro-batches spent hiding the pipeline bubble are not available for data-parallel scaling.
- Communication costs: $DP \approx 2P$ per step, $ZeRO-3 \approx 3P$ per step, $TP \approx$ activation-size all-reduce per layer-compare each to the step’s compute time to locate the bottleneck.
- TP beyond 8 GPUs on current GPU hardware causes 40%+ throughput degradation and should not be used.
- Extending to 4D by adding SP or CP alongside TP costs no additional bandwidth and is the default choice for sequence lengths above 4-8K tokens.

12.4.2 Case Studies: Llama, DeepSeek-V3, Gemma

Training reports from frontier labs are among the most information-dense documents in the field—each parallelism configuration table is the compressed output of thousands of hours of ablations. Three public examples span the design space usefully: Llama 3, which sets the canonical template for dense models on GPU clusters; DeepSeek-V3, which deliberately departs from that template for a large MoE model; and Gemma 2, which shows how TPU network topology reshapes the same trade-offs. Reading these side by side reveals how the rules of thumb hold across hardware and architecture—and exactly where they bend.

Every configuration in these reports is traceable to one of three constraints: model memory, network topology, or batch-size budget. The useful mental model is to read the TP setting as the intra-node memory reduction strategy, the PP setting as the cross-node model-fit strategy, and the DP setting as the throughput-scaling strategy. When those three are set, whatever remains—CP, EP, SP—is driven by architecture-specific demands: long sequences, sparse experts, quadratic attention memory.

The three case studies share a common thread. Each configuration minimizes idle time: PP bubble, communication stalls, and memory-bound micro-batch stalls are all forms of idle time. A configuration is correct when the matrix-multiply units are busy essentially continuously.

If you can read a parallelism table and explain why each dimension is set to its specific value, you can answer most senior-level design questions about distributed training.

The three examples collectively demonstrate the universality of the bandwidth-hierarchy argument even as specific choices differ. On GPU clusters, NVLink forces $TP \leq 8$. On TPU pods with toroidal-mesh networking—where every chip can reach every neighbor at similar bandwidth—model parallelism can scale further without hitting the same cliff. For MoE models, expert parallelism collapses the TP/PP distinction: routing tokens to expert GPUs is effectively tensor parallelism in the MLP block, implemented via all-to-all rather than all-reduce.

The GPU case also illustrates a hardware-topology argument that rarely appears in textbooks: pipeline parallelism is used not just to fit models, but to route around the slow spine switches that connect GPU racks above ~ 256 GPUs. PP's point-to-point activation transfers are directional and predictable, degrading far more gracefully than all-to-all operations when crossing network hierarchy boundaries.

A Worked Example

Llama 3 (405B, dense, GPU) is the closest to the canonical recipe and the most instructive to decode step by step.

The model has $\sim 405\text{B}$ parameters, 126 transformer layers, hidden dimension 16,384. Training used clusters of H100 80 GB GPUs. The parallelism configuration for the standard pre-training phase:

- **TP = 8:** one full TP group per 8-GPU node (NVLink). At bf16, 405B parameters across 8 GPUs gives ≈ 101 GB per GPU for parameters alone—still over the 80 GB HBM limit. TP alone cannot fit the model; PP is necessary.
- **PP = 16:** 126 layers split across 16 pipeline stages of ~ 8 layers each. Per-GPU parameter memory drops to ≈ 12.6 GB, well within budget. $\text{TP} \times \text{PP} = 128$ GPUs constitutes one full model instance.
- **DP = (total GPUs) / 128:** on a cluster of roughly 16,000 GPUs, $\text{DP} \approx 125$ replicas. Global batch: 125×4 micro-batches $\times 4096$ tokens $\approx 2\text{M}$ tokens per step.
- **CP = 8:** added specifically for the long-context fine-tuning phase (128K token sequences). Attention computation distributes across 8 devices via ring-style KV passing, keeping the quadratic attention activation memory tractable. This dimension is disabled for standard pre-training.

The key lesson: PP was forced by memory, not preferred. The team would have avoided it if 80 GB HBM were sufficient after $\text{TP} = 8$. The long-context CP dimension illustrates the 4D pattern: a fifth dimension would be needed if the model were also MoE (expert parallelism), which it is not.

At this scale, Llama 3's training report logged 148 interruptions from faulty GPUs and additional incidents from unplanned network maintenance. The parallelism configuration is only part of the engineering; checkpoint frequency, automatic restart logic, and silent data-corruption detection are equally critical once you cross $\sim 10\text{K}$ GPUs.

DeepSeek-V3 (671B total, 37B active, MoE, GPU) is the most interesting departure from the canonical recipe. The model uses 256 routed experts with top-8 routing per token, so only 37B parameters are active for any given forward pass.

Published configuration:

- **PP = 16:** 61 transformer layers split across 16 pipeline stages.
- **EP = 64:** 64-way expert parallelism. Each GPU holds a subset of experts. Tokens are dispatched to the GPU holding the selected expert via all-to-all communication. This substitutes for the role TP plays in dense models.
- **ZeRO stage 1 for DP:** optimizer states sharded across DP ranks. DP degree fills remaining GPU capacity.
- **No tensor parallelism.**

The absence of TP is the key departure and worth understanding precisely. In a 671B MoE model, the expert MLP layers dominate parameter count. Expert parallelism distributes those parameters across 64 GPUs via token routing—achieving the same memory reduction that TP provides in dense models, but through a different communication primitive (all-to-all for routing versus all-reduce for TP). Adding TP on top of EP would introduce per-layer all-reduces on the attention and dense MLP blocks, which are relatively small in a MoE

model, while providing little incremental memory benefit. The net effect would be increased communication overhead for negligible memory gain—a straightforward throughput loss.

The lesson: the canonical TP-first rule assumes a dense model. For MoE, EP substitutes for TP. The bandwidth-hierarchy argument still applies—EP’s all-to-all also benefits from fast intra-pod networking—but the primitive changes.

Gemma 2 (dense, TPU) illustrates how topology reshapes every decision. TPU pods use a toroidal mesh: each chip connects to a small number of spatial neighbors, but collective bandwidth across the full mesh is high and symmetric for any global operation. There is no hard bandwidth cliff at a node boundary analogous to the NVSwitch-to-InfiniBand transition on GPU clusters.

Published configuration: ZeRO-3 (FSDP) + model parallelism + data parallelism. Model parallelism can extend well beyond 8 devices on TPUs because all inter-chip links are comparable in bandwidth; there is no architectural reason to cap at 8. ZeRO-3 is the primary vehicle for memory efficiency, sharding parameters, gradients, and optimizer state across all model-parallel ranks. The report does not feature pipeline parallelism, for two reasons: the toroidal mesh has no slow inter-rack spine to route around, and ZeRO-3 handles the memory constraint without the scheduling complexity of PP.

What You Actually Decide in Practice

- **Dense GPU training (Llama-style):** TP = 8, PP only when forced by memory (roughly above 100B at 80 GB HBM), DP for the rest. Add CP only for the long-context fine-tuning phase; it adds pipeline complexity and should be disabled for standard pre-training.
- **MoE GPU training (DeepSeek-style):** replace TP with EP sized to the expert count. PP still handles the dense attention and embedding layers. Expect the all-to-all routing to be a communication bottleneck at high expert count; token dropping and expert-capacity buffers control variance but add algorithm complexity that must be tested carefully.
- **TPU training (Gemma-style):** lean on ZeRO-3 and model parallelism beyond 8 chips. Pipeline parallelism is less commonly needed because there is no hard bandwidth cliff at a hardware boundary. Collective operations scale to the full pod size without topology-driven restrictions.
- **At production scale, fault tolerance is a first-class concern.** Llama 3 counted 148 GPU failures during a single training run. The correct response is aggressive checkpointing (every 30-60 minutes at this scale), automatic job restart, and silent-error detection via periodic gradient norm checks and loss spike monitors. The parallelism configuration alone does not determine success.

How This Shows Up in Interviews

- **“DeepSeek-V3 has no tensor parallelism. Why not?”** Strong answer: in a 671B MoE model, expert MLP layers dominate parameters. Expert parallelism distributes those parameters across 64 GPUs via token routing, providing the same memory reduction that TP gives in dense models. Adding TP would introduce per-layer all-reduces on the smaller attention and dense MLP layers with minimal memory benefit—net throughput loss.
- **“Why does Llama 3 use context parallelism only for the long-context phase?”** Strong answer: CP distributes attention computation by passing KV blocks in a ring across multiple devices. It is necessary only when the quadratic s^2h activation memory from attention exceeds per-GPU HBM. At standard pre-training sequence lengths (4096 tokens), TP + SP already keeps activation memory tractable. At 128K

tokens, the quadratic term exceeds any mitigation from SP alone, making CP unavoidable.

- **Senior follow-up: “If you had to train a 1T dense model on GPUs today, what configuration would you propose, and what would you be most worried about?”** Strong answer: TP = 8, PP sized to fit the model (likely PP = 32+), DP for remaining capacity. The primary concern is pipeline bubble: at PP = 32, the bubble ratio $(31)/m$ demands $m \geq 128$ micro-batches per replica to stay under 20% idle time, which strains the global batch budget severely. Secondary concerns: checkpoint and restart time scales with the run duration; a single mid-step GPU failure at 32K GPUs wastes hours. The engineering response is zero-bubble pipeline scheduling (filling idle slots with weight-gradient computation) and a battle-tested fault-tolerant checkpoint infrastructure-both are as important as the parallelism configuration itself.

Key Takeaways

- Dense GPU models follow the canonical recipe: TP = 8 within a node, PP only when forced by memory, DP for throughput. CP is a 4th dimension added only for long-context phases.
- MoE models substitute EP for TP; the bandwidth-hierarchy argument still applies, but the communication primitive changes from all-reduce to all-to-all.
- TPU topology removes the 8-device NVLink cliff, allowing model parallelism to scale further and reducing reliance on pipeline parallelism.
- At production scale (>10K GPUs), fault tolerance, silent-error detection, and checkpoint infrastructure are as critical as the parallelism configuration itself.
- Reading a published parallelism table backward-inferring why each dimension is set to its specific value from memory, topology, and batch-size constraints-is the core skill this interview topic tests.

Part VI

Scaling Laws and Training Economics

Chapter 13 Predictable Scaling for Interview Answers

13.1 The Three Canonical Scaling Laws

13.1.1 Data Scaling and Non-Parametric Rates

An interviewer at a frontier lab will often open the scaling-laws deep-dive with a deceptively simple question: “If I double my training data, how much does the loss drop?” The answer is not “a lot” or “it depends.” The answer is a precise power-law relationship, and if you can derive the exponent from first principles on a whiteboard, you immediately signal a level of understanding that separates senior candidates from the pack. This topic appears at the staff and principal level, but even L5 candidates are expected to know the empirical fact; the derivation is what earns you the senior signal.

Data scaling is the observation that, on a log-log plot, test loss decreases linearly as a function of dataset size. Formally, if we denote the dataset size in tokens as D and the cross-entropy loss as L , then in the scaling regime we find:

$$L(D) \approx \left(\frac{D_0}{D} \right)^\alpha$$

where D_0 is a dataset-specific constant and $\alpha > 0$ is the scaling exponent. Taking logarithms of both sides:

$$\log L = -\alpha \log D + \alpha \log D_0$$

This is a line with slope $-\alpha$ in log-log space. The empirical finding, established robustly by Hestness et al. (2017) and confirmed at scale by Kaplan et al., is that this linear relationship holds across many orders of magnitude of data, across different model architectures, and across domains including language, speech, and machine translation. The exponent α is typically in the range $[0.05, 0.35]$ depending on the task—much smaller than what naive statistical theory would suggest. The model size N is always chosen to be large enough that the data is not saturating the model; otherwise, you would observe premature flattening as you enter the irreducible-error regime.

Log-log linearity between dataset size and test loss is the most robustly observed empirical fact in large-scale language modeling, and its exponent encodes the effective intrinsic dimensionality of the learning task.

The power-law decay is not magic—it has a grounded theoretical explanation rooted in non-parametric statistics. Consider the simplest possible estimation task: estimating the mean μ of a Gaussian distribution from n i.i.d. samples. The mean estimator $\hat{\mu} = \frac{1}{n} \sum x_i$ has squared error:

$$\mathbb{E}[(\hat{\mu} - \mu)^2] = \frac{\sigma^2}{n}$$

This is polynomial decay with exponent -1 in n . But language models are not estimating a scalar mean—they are fitting an arbitrarily complex function over a very high-dimensional input space. For a smooth regression target f on a D -dimensional domain, even the optimal non-parametric estimator (e.g., a histogram regressor) achieves squared error scaling as:

$$\text{Error} \sim n^{-1/D}$$

In log-log terms, the slope is $-1/D$. A task with effective intrinsic dimensionality $D = 10$ gives a slope of -0.1 ; dimensionality $D = 3$ gives -0.33 . This matches the empirically observed range of scaling exponents almost exactly: Hestness reports -0.13 for machine translation, -0.30 for speech, and -0.095 for language modeling. Language modeling has a shallower slope because natural language is an extraordinarily high-dimensional distribution-the effective dimensionality of “predict the next token given all of human text” is enormous. The prior alternative was to rely on generalization bounds (VC dimension, Rademacher complexity), which provide rates like $1/\sqrt{n}$ for agnostic learning; these are upper bounds that are too loose to be useful predictions, and empirical scaling laws are the practitioner’s replacement for them.

A Worked Example

Suppose you are fitting a smooth regression function on a 2D input space with n total samples placed uniformly at random. Partition the unit square into $k \times k$ cells, assigning one estimated value per cell. With k^2 total cells, each cell receives n/k^2 samples on average.

The estimation error inside each cell is $\sigma^2/(n/k^2) = \sigma^2 k^2/n$ (variance of a local mean estimate). The approximation error-how well a $k \times k$ grid captures a smooth function-scales as $1/k^2$ for a function with bounded second derivatives. Balancing these two:

$$\frac{\sigma^2 k^2}{n} = \frac{1}{k^2} \implies k^4 = \frac{n}{\sigma^2} \implies k \sim n^{1/4}$$

Plugging back in, the total error scales as $k^{-2} \sim n^{-1/2} = n^{-1/D}$ with $D = 2$. For a D -dimensional space the argument generalizes to:

$$\text{Error} \sim n^{-2/(D+4)} \text{ (exact for Hölder smooth functions)}$$

which for large D simplifies to $n^{-1/D}$ in the dominant term. Now apply this intuition to language modeling with an effective dimensionality of roughly $D \approx 10$: we would predict a scaling exponent of -0.10 , which is almost precisely the -0.095 observed empirically for language modeling in Hestness et al. The model size must be fixed large enough that it is not the bottleneck-a 7B-parameter model trained in bf16 on an H100 cluster can serve this role for datasets up to a few hundred billion tokens before saturation becomes a concern.

What You Actually Decide in Practice

- **Model size must exceed data saturation.** When running data-only scaling experiments, fix the model large enough that the scaling curve stays in the power-law region. A rule of thumb: for every doubling of data, you want roughly $10\times$ more parameters than what would be “Chinchilla-optimal” for that data size, so the model cannot overfit to the finite dataset.
- **Data composition shifts the offset, not the slope.** Kaplan et al. show that changing training data mixture (domain proportions) changes the vertical intercept of the log-log line but leaves the slope α nearly unchanged. This means data selection experiments-which source to mix in, what weight-can be validly run on small models and small data; the relative rankings transfer to scale.
- **Repeated data hurts after roughly 4 epochs.** When you have exhausted unique tokens, repeating the corpus has rapidly diminishing returns. Empirically the effective sample size grows much more slowly than the raw token count past 4 epochs of repetition. If your corpus allows only 2 epochs before saturation, adding lower-quality unique data is almost always better than a third pass over high-quality data.

- **Do not confuse perplexity scaling with downstream-task scaling.** The power-law relationship is clean and reliable for cross-entropy loss. Benchmark accuracy can be non-monotone, can show emergent jumps, and can rank architectures differently than perplexity does. Use perplexity scaling to make engineering decisions; validate on downstream tasks before committing resources at scale.
- **The irreducible floor exists.** Every task has an entropy floor below which no amount of data will push the loss-the Bayes error of the task. In practice, you will rarely be close to it for language, but in low-entropy domains (e.g., heavily templated text), you can hit it surprisingly early.

How This Shows Up in Interviews

- **“I double the number of training tokens. How does the validation loss change?”** A strong answer names the power-law: loss drops by a factor of 2^α where $\alpha \approx 0.05-0.30$ depending on the task. For language, $\alpha \approx 0.1$, so doubling tokens gives a $\approx 7\%$ relative improvement in loss. The candidate should also mention that the model must be large enough to remain in the scaling regime.
- **“Why is the empirical scaling exponent so much smaller than $1/\sqrt{n}$?”** The strong answer explains non-parametric rates: $-1/\sqrt{n}$ is the rate for estimating a scalar or fitting a one-dimensional function (slope -0.5). For a D -dimensional task the exponent is $-1/D$; language modeling has very high effective dimensionality, so the slope is shallow. The log-log linearity itself comes from the polynomial structure of non-parametric error bounds.
- **Senior follow-up: “If you changed the training data mixture-say, you doubled the fraction of code-how would that affect the scaling curve?”** The senior answer: the slope α is largely unchanged because it reflects the intrinsic difficulty of the learning task. The offset shifts-more code would lower the floor for code benchmarks while potentially raising it for other domains. You can measure this offset shift cheaply by training small models ($\sim 1\text{B}$ parameters, $\sim 10\text{B}$ tokens) and fitting a line; the offset difference at small scale extrapolates reliably to large scale, which is the entire point of data-composition scaling experiments.

Key Takeaways

- On a log-log plot, test loss versus dataset size is linear in the scaling regime; the slope is the scaling exponent α .
- The exponent α for language modeling is ≈ 0.1 , reflecting high effective intrinsic dimensionality; it arises naturally from non-parametric rates of the form $n^{-1/D}$.
- The model must be large enough that data is not saturating it-otherwise you leave the power-law regime early.
- Data composition changes the offset of the scaling curve but not its slope, so data-selection experiments are cheap to run at small scale.
- Repeated data beyond ~ 4 epochs yields rapidly diminishing returns; prefer new lower-quality unique tokens over additional passes over a high-quality corpus.

13.1.2 Model and Compute Scaling

“If I give you a fixed compute budget, how would you split it between model size and training data?” is the single most commonly asked quantitative question in frontier-lab LLM design interviews. The answer comes

from the Chinchilla analysis, and getting it right-including why Kaplan’s earlier estimate was off and what the three estimation methods actually measure-is what separates a candidate who memorized a ratio from one who understands the underlying machinery. This question appears at every seniority level; the senior version asks you to derive the optimal trade-off from scratch rather than recite it.

Model and compute scaling extends data scaling to a two-variable problem. The loss L depends jointly on model size N (non-embedding parameters) and dataset size D (tokens):

$$L(N, D) \approx \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_\infty$$

Here A , B , α , β are empirically fitted constants and L_∞ is the irreducible entropy of the data distribution. This functional form is additive: the model-size term and the data-size term contribute independently to reducible loss. The key insight is that for a fixed compute budget C (in FLOPs), training cost scales approximately as $C \approx 6ND$ (the factor of 6 counts both forward and backward passes per token per parameter). Given that constraint, you want to minimize $L(N, D)$ subject to $ND = C/6$.

Taking the gradient of the Lagrangian and setting it to zero yields the *compute-optimal* or *Chinchilla-optimal* scaling:

$$N^* \propto C^a, \quad D^* \propto C^b$$

where the Chinchilla estimates give $a \approx b \approx 0.5$, implying that N and D should be scaled in equal proportion as compute grows. The resulting rule of thumb is roughly **20 tokens per parameter**: a 70B-parameter model is compute-optimal when trained on ~ 1.4 trillion tokens.

Compute-optimal training balances model size and token count in a roughly 1:20 parameter-to-token ratio; skewing either way wastes FLOPs.

Kaplan et al. (2020) were the first to analyze joint model-data scaling, but they significantly underestimated the optimal token-to-parameter ratio-their estimate was closer to 5-10 tokens per parameter. The primary culprit was cosine learning rate schedules. A cosine schedule cannot be truncated mid-run; if you stop training early (before the full cosine cooldown), the model’s effective loss is higher than it would be at the same number of steps with a properly completed schedule. Kaplan’s experiments used runs truncated at various compute levels for comparison, which systematically overestimated the performance of large models trained on little data and underestimated small models trained longer. Chinchilla used isoFLOP profiles-multiple models trained to the same total compute budget, each completing its full learning rate schedule-which avoids this artifact.

The additive functional form of $L(N, D)$ also encodes a crucial architectural observation: not all parameters count equally. Embedding parameters (the token embedding matrix and positional embeddings) do not follow the same power-law behavior as non-embedding parameters. When embedding parameters are included in N , the scaling curve bends and loses its clean log-log linearity. Kaplan reports this explicitly: analysis should be conducted on non-embedding parameter counts. Similarly, in sparse models like mixture-of-experts, the number of activated parameters per token, not the total parameter count, governs the effective N in the scaling law.

A Worked Example

Suppose your compute budget is $C = 6 \times 10^{23}$ FLOPs-roughly the budget for a mid-tier production training run on a few hundred H100s over several weeks. Using $C \approx 6ND$:

$$ND = \frac{C}{6} = 10^{23}$$

Chinchilla’s compute-optimal condition says $N^* \approx \left(\frac{C}{6} \cdot \frac{\alpha}{\beta}\right)^{0.5}$. With $\alpha \approx \beta$ (Chinchilla methods 1 and 2 both give roughly equal exponents), this simplifies to:

$$N^* \approx \sqrt{ND} = \sqrt{10^{23}} \approx 3.2 \times 10^{11} \approx 320\text{B parameters}$$

and

$$D^* = \frac{10^{23}}{3.2 \times 10^{11}} \approx 3.1 \times 10^{11} \approx 310\text{B tokens}$$

That gives roughly 1 token per parameter-but this is for the abstract mathematical optimum ignoring the 20:1 convention. The discrepancy is because the 20:1 rule comes from the Chinchilla paper’s actual constant fits (A, B, α, β), not from the $\alpha = \beta$ simplification alone. With Chinchilla’s fitted constants, the compute-optimal line for a 7B-parameter model falls at approximately 140B tokens, and for a 70B model at approximately 1.4T tokens-the 20:1 ratio.

The three methods Chinchilla uses to estimate this ratio are: (1) *minimum-envelope* over training curves across compute levels, which finds the lower envelope of loss vs. compute for models of varying size; (2) *isoFLOP profiles*, which sweep model size at each fixed compute level and find the minimum of a fitted quadratic; and (3) *parametric fit* to the full $L(N, D)$ surface via least-squares. Methods 1 and 2 agree closely (0.50, 0.50 exponents); method 3 originally gave anomalous results because of an off-center residual in the curve fitting, a flaw later identified by Epoch AI’s forensic replication of the plots.

What You Actually Decide in Practice

- **Training-optimal is not inference-optimal.** Chinchilla optimizes training FLOPs. Production systems care about inference cost-a smaller model with the same quality at deploy time is far cheaper to run at scale. This is why modern models have drifted far from 20 tokens per parameter: Llama 3 (8B) was trained on 15 trillion tokens (~ 1875 tokens per parameter), and Qwen models have reached 30 trillion tokens. When your product serves millions of requests per day, the right ratio is determined by inference cost, not training cost.
- **Exclude embedding parameters from N .** Always use non-embedding parameter counts when fitting scaling laws or comparing model sizes. Embedding tables can be very large (e.g., a 128k vocabulary at 4096 dimensions adds $\sim 500\text{M}$ parameters) and do not improve loss at the same rate as attention and feedforward parameters.
- **For MoE models, use activated-parameter count.** In sparse mixture-of-experts architectures, only a fraction of parameters are active per token. The effective N for scaling-law purposes is the number of activated parameters per token, not the total parameter count. Several papers have proposed “dense equivalents” to normalize MoE model sizes for comparison.
- **Learning rate schedules must complete.** When running scaling-law pilot experiments with cosine schedules, train each run to completion (full cooldown) before comparing. Truncated runs systematically misrank large-model versus long-training options. Constant or linear decay schedules sidestep this problem and are increasingly preferred for scaling experiments.

- **Downstream task scaling is noisier than perplexity scaling.** The compute-optimal analysis applies cleanly to cross-entropy loss. If you are trying to optimize for a specific benchmark, scaling predictions become unreliable—emergence effects and saturation are common. Use perplexity scaling to allocate resources and downstream benchmarks to validate the final model.

How This Shows Up in Interviews

- **“If you had a fixed compute budget, how would you split it between model size and training tokens?”** The strong answer names Chinchilla: roughly 20 tokens per parameter is training-optimal. Then the candidate immediately qualifies it: if the model will be deployed at high request volume, you should overtrain a smaller model—the inference savings over the model’s lifetime dominate the extra training cost. This distinction is what separates rote recall from engineering judgment.
- **“Why did Kaplan underestimate the optimal token count compared to Chinchilla?”** Strong answer: cosine learning rate schedules require completing the full cooldown to yield a valid checkpoint. Kaplan’s multi-model comparison used truncated runs, which made large models trained on little data look artificially better. Chinchilla’s isoFLOP method trained all runs to completion before comparing, removing the artifact.
- **Senior follow-up: “Walk me through how you would run an isoFLOP experiment to estimate the compute-optimal model size for your team’s next training run.”** The senior answer sketches: (1) pick 5-7 compute budgets spanning 2-3 orders of magnitude below your target; (2) at each budget, train 5-8 model sizes ranging from $\sim 0.3\times$ to $\sim 3\times$ the Chinchilla-optimal size for that budget; (3) fit a quadratic to the loss-vs.-model-size curve at each fixed compute; (4) extract the minimum and plot optimal N vs. C on a log-log axis; (5) fit a power law and extrapolate to your target compute. Emphasize that the entire pilot study uses $<1\%$ of the target compute budget and should complete in days, not weeks.

Key Takeaways

- The Chinchilla functional form $L(N, D) = A/N^\alpha + B/D^\beta + L_\infty$ describes joint model-data scaling; minimizing it subject to a fixed compute budget $C \approx 6ND$ gives the training-optimal allocation.
- Chinchilla’s result is approximately 20 tokens per parameter for training-optimal runs; this is far below what modern production models use because inference cost dominates at deployment scale.
- Kaplan’s earlier (lower) estimate of the optimal token count was biased by truncated cosine learning rate schedules; isoFLOP experiments that complete the full cooldown correct this.
- Exclude embedding parameters from N in all scaling analyses; they do not follow the same power law as attention and feedforward parameters.
- The isoFLOP method—sweeping model size at each fixed compute budget and extracting the minimum—is the most reliable and conceptually cleanest approach to estimating compute-optimal ratios for your specific task and architecture.

13.2 Using Scaling Laws to Make Design Decisions

13.2.1 Architecture Comparisons at Small Scale

Every year brings a new architecture claiming to beat the transformer. The interview question that follows is always the same: “How would you decide whether to adopt this for your next production training run?” The naive answer—train both at full scale and compare—is economically absurd. The right answer uses scaling laws to make that comparison at a fraction of the cost. This question appears at the senior level and above, and the candidate who can describe the procedure cold, with the right caveats, signals genuine production experience.

The scaling-law comparison procedure works as follows. You pick two or more candidate architectures and train instances of each across a range of compute budgets spanning two to three orders of magnitude—say, from 10^{18} to 10^{21} FLOPs. For each architecture, you measure the best achievable validation loss at each compute level (holding the other architecture fixed and sweeping over model sizes to get the optimal point). You then plot both curves on a log-log axis: log compute on the x -axis, log loss on the y -axis.

The crucial diagnostic is the relationship between the two curves:

- **Parallel curves with a constant vertical gap:** the better architecture is compute-efficient by a fixed multiplicative factor at every scale. If architecture A sits Δ log-loss units below architecture B across all compute levels, that translates to a constant factor speedup—architecture A achieves the same loss as B with a k -fold smaller compute budget, where $k = 10^{\Delta/\alpha}$ and α is the common scaling slope.
- **Crossing curves:** the ranking reverses at some compute level. The cheaper architecture may look better at small scale but lose at large scale, or vice versa. Crossing curves require extrapolation before you can make a decision.

A constant-factor offset in log-log scaling curves is the signature that an architecture advantage is real and will survive to your target compute budget.

The reason this procedure is reliable is that the slope of a scaling curve—the rate at which loss decreases per unit of log-compute—is largely determined by the task and data, not the architecture. What the architecture controls is the efficiency constant: how many FLOPs are needed to reach a given loss. A better architecture shifts the whole curve down (lower loss at every compute level) or equivalently left (same loss at lower compute). Because the slope is preserved, a gap measured at 10^{19} FLOPs extrapolates cleanly to 10^{23} FLOPs.

Contrast this with the alternative of just training at small scale and comparing accuracy numbers. A model with 10M parameters and 50M tokens trained on a narrow benchmark tells you almost nothing about relative quality at 70B parameters and 2T tokens. The scaling-law approach is cheap precisely because the parallelism of curves means small-scale information is informative about large-scale rankings.

The transformer-vs-LSTM comparison is the canonical example of this working well. LSTMs consistently sit ~ 1 log-perplexity unit above transformers at every compute level, on a log scale. That gap corresponds to roughly a 10-15 $\times$ compute penalty: to match a transformer trained on a given budget, an LSTM needs 10-15 times more FLOPs. Knowing this at the 10^{18} FLOP level means you never need to run the 10^{23} FLOP LSTM experiment to know the answer. Similarly, gated linear units and mixture-of-experts architectures showed reliable parallel downward shifts relative to the standard transformer baseline, which is precisely why they became the dominant modifications at frontier labs.

A Worked Example

Suppose you want to evaluate a new state-space model architecture against a standard transformer. You train both at five compute budgets: $C \in \{10^{18}, 3 \times 10^{18}, 10^{19}, 3 \times 10^{19}, 10^{20}\}$ FLOPs. At each C , you sweep over five model sizes (varying width and depth) and select the lowest validation loss achieved.

After plotting on a log-log axis, you observe:

- Both curves have approximately the same slope, $\alpha \approx -0.08$ (consistent with language modeling data scaling).
- The transformer curve sits $\Delta \approx 0.05$ log-loss units below the SSM curve consistently across all five budgets.
- The curves are parallel-no crossing.

Interpreting the gap: a constant offset of $\Delta = 0.05$ in log-loss means the SSM needs a compute multiplier of $k = 10^{\Delta/\alpha} = 10^{0.05/0.08} \approx 10^{0.625} \approx 4.2$ times more FLOPs to match the transformer at any target loss. For a \$5M training run, the SSM would require an equivalent \$21M run to achieve the same quality—a clear decision signal obtained for roughly 0.01% of the target budget.

Total pilot cost: five compute levels, two architectures, five model sizes each = 50 training runs. If your target run is $C_{\text{target}} = 10^{23}$ FLOPs, your pilot spans 10^{18} - 10^{20} , which is at most $10^{20}/10^{23} = 0.1\%$ of the target. In practice, the pilot consumes even less because the small runs dominate compute.

What You Actually Decide in Practice

- **Require at least two to three orders of magnitude of compute coverage in your pilot.** A single data point per architecture is not a scaling law; you need enough range to confirm linearity and detect potential crossings. In my experience, five compute levels spanning three orders of magnitude is the minimum to feel confident.
- **Crossing curves are a stop sign, not an answer.** If the curves cross within your pilot range, you need to extrapolate carefully-fit power laws to each segment, determine where they diverge, and assess whether your target compute falls before or after the crossing. Extrapolation beyond the pilot range is unreliable unless the curves are clearly parallel by the end.
- **Always compare on validation perplexity, not downstream benchmarks.** Perplexity scales cleanly and predictably. Benchmark accuracy is noisy, nonlinear, and subject to emergence effects that can make a better architecture look worse at small scale. Use perplexity to select the architecture; validate downstream performance only after committing to the winner.
- **The two modifications that reliably beat the standard transformer at every scale are gated linear units and mixture of experts.** All other alternatives (linear attention, state-space models, recurrent variants) show either no reliable gap or task-specific wins that do not generalize. This is exactly what large-scale architecture sweep papers found, and it directly explains the architectural choices in current frontier models.
- **Exclude embedding parameters from all comparisons.** Embedding matrices do not follow the same power-law scaling as attention and feedforward parameters. If you include them, the scaling curves bend and you lose the clean log-log linearity that makes the comparison reliable. Always report and compare on non-embedding parameter counts.

How This Shows Up in Interviews

- **“A researcher proposes a new attention variant. How do you decide whether it is worth scaling up?”** A strong answer describes the scaling-law comparison: train both architectures at 3-5 compute levels spanning 2-3 orders of magnitude, check whether the new variant shows a consistent parallel offset below the baseline. If yes, quantify the compute-efficiency gain. If curves cross, extrapolate and determine whether the advantage emerges before or after your target compute.
- **“Why not just train the new architecture at a small scale and compare accuracy on a benchmark?”** The strong answer explains why benchmark accuracy is an unreliable signal at small scale: architectures can rank differently on perplexity versus benchmarks, and emergent capabilities can appear discontinuously. Perplexity scaling is the reliable channel; benchmark validation comes only after the architecture decision is made.
- **Senior follow-up: “What would you do if the scaling curves for two architectures crossed within your pilot range?”** The senior answer goes beyond “extrapolate.” It explains that crossing curves mean one architecture is better in the data-limited regime and the other in the compute-rich regime; you need to characterize where your target compute falls relative to the crossing point. If the crossing happens before your target, the extrapolation is more reliable; if after, you may need to push your pilot to larger scale before committing. The candidate should also ask *why* the curves cross—is it a depth-width interaction? A saturation artifact? Understanding the mechanism determines whether the crossing is robust or an artifact of the comparison setup.

Key Takeaways

- Train each candidate architecture at 3-5 compute levels spanning 2-3 orders of magnitude; compare on validation perplexity, not benchmark accuracy.
- Parallel curves with a constant vertical gap confirm that the better architecture’s advantage will survive to your target compute; the gap size gives you the compute-efficiency multiplier.
- Crossing curves require extrapolation and deeper investigation before committing.
- The two architecture modifications with the strongest and most consistent evidence of compute-efficiency improvement are gated linear units and mixture of experts.
- Always use non-embedding parameter counts; embedding parameters break log-log linearity and skew comparisons.

13.2.2 Optimizer and Hyperparameter Selection

Choosing the wrong learning rate for a 70B model is not a minor inconvenience—it can cost weeks of H100 time and produce a model that is meaningfully worse than it should be. The right way to pick learning rates, batch sizes, and depth-to-width ratios is not to tune them at large scale; it is to verify at small scale that these hyperparameters either transfer directly or follow a predictable scaling law of their own. Understanding when each case applies is a senior-level topic that shows up in both design and deep-dive interviews.

The scaling-law mindset applied to hyperparameters rests on one observation: if the optimal value of a hyperparameter produces scaling curves that are *parallel and non-crossing* across model sizes, then the optimal value at small scale is the optimal value at large scale. You train a grid of small models varying the hyperparameter of interest, identify the optimal value at several compute levels, and check whether that optimum is

stable.

Three hyperparameters deserve special treatment:

Aspect ratio (depth-to-width). Kaplan et al. show that for models between 50M and 1.5B parameters, the loss difference between aspect ratios from ~ 10 to ~ 100 is small—well under 5% relative. The basin is wide and flat. A single layer is genuinely bad; all reasonable aspect ratios above that are approximately equivalent. The practical upshot: pick anything in the range $d_{\text{model}}/n_{\text{layers}} \in [64, 256]$ and you are in the safe zone at all scales.

Batch size and critical batch size. Doubling the batch size is equivalent to taking two gradient steps—but only below the *critical batch size* B^* . Above B^* , additional samples in the batch reduce useful gradient noise at a diminishing rate, and you are effectively paying full parallelism cost for less than full optimization benefit. The critical batch size itself scales with the loss target: lower loss (later in training) implies larger B^* , because the gradient landscape becomes smoother and you need more samples to reduce noise to a useful level. Empirically, B^* follows a power law:

$$B^* \propto \mathcal{L}^{-\gamma}$$

for some task-dependent $\gamma > 0$, where $\mathcal{L}$ is the current validation loss. This is why production training runs (Llama 3 is a public example) ramp batch size during training: early in training, loss is high and B^* is small; later, loss is lower and B^* is larger, so you can increase parallelism without sacrificing optimization efficiency.

Learning rate. Under the standard parametrization, the optimal learning rate decays with model width d roughly as $\eta^* \propto 1/d$. This means every time you double the width, you need to re-tune the learning rate. The alternative, *maximal update parametrization* (μP), reparametrizes the model so that the optimal learning rate is exactly stable across width. You tune it once at a small model (say, 50M parameters) and apply the same value to a 70B model.

Hyperparameter tuning and architecture selection should be done at small scale and extrapolated; the only reliable signal is whether the optimal value is stable across compute levels on a log-log plot.

The critical insight about hyperparameter stability is that it is not universal. Some hyperparameters—aspect ratio, feedforward ratio, attention head dimension—have wide, flat loss basins that barely move as scale increases. Others—learning rate under standard parametrization, batch size—require active management as you scale. Knowing which is which saves enormous amounts of compute.

The optimizer comparison follows the same logic as architecture comparison. Adam versus SGD produces a roughly constant-factor compute efficiency gap across scales, with Adam consistently requiring roughly $10\times$ fewer FLOPs to reach the same loss. Since the gap is parallel (constant offset on log-log axes), you don't need to run SGD at full scale to know the answer—a small-scale pilot is sufficient.

The μP approach deserves particular attention because it eliminates one of the most expensive unknowns in scaling: the learning-rate transfer problem. Standard parametrization initializes attention projection weights with $\mathcal{N}(0, 1/d)$ variance and applies the same learning rate to all layers regardless of width. As width grows, the gradient signal through the attention layers blows up without the $1/d$ correction, forcing the learning rate down. μP corrects this by scaling initialization variance, per-layer learning rates, and the forward-pass output multiplier by appropriate powers of d . The result is a model where every layer's activation magnitude is invariant to width, and consequently the optimal global learning rate does not change as you scale.

A Worked Example

Consider selecting the optimal learning rate for a standard transformer with $d_{\text{model}} \in \{256, 512, 1024, 2048, 4096\}$ (corresponding roughly to 15M, 55M, 220M, 900M, and 3.5B non-embedding parameters). For each width, you run a grid of seven learning rates spaced in log scale from 10^{-4} to 10^{-1} , train for a fixed number of tokens (say 10B), and record the minimum validation loss.

The optimal learning rate at each width:

$$\begin{aligned} d = 256 & \quad \eta^* \approx 3 \times 10^{-3} \\ d = 512 & \quad \eta^* \approx 1.5 \times 10^{-3} \\ d = 1024 & \quad \eta^* \approx 8 \times 10^{-4} \\ d = 2048 & \quad \eta^* \approx 4 \times 10^{-4} \\ d = 4096 & \quad \eta^* \approx 2 \times 10^{-4} \end{aligned}$$

Fitting a power law: $\eta^* \propto d^{-1.0}$, exactly the $1/d$ rule. Extrapolating to $d = 8192$ (a 14B-parameter model), you predict $\eta^* \approx 10^{-4}$. This is reliable enough to use as a starting point, but still requires a validation run because the learning-rate surface is narrow enough that being off by $3\times$ costs you real loss.

Under μP , the same pilot at $d = 256$ with $\eta^* = 3 \times 10^{-3}$ gives you the value to use at $d = 8192$ -no extrapolation required. The computational cost of the pilot is the same; the benefit is that you eliminate the large-scale validation run.

What You Actually Decide in Practice

- **Treat aspect ratio as a low-stakes decision.** Pick a depth-to-width configuration that keeps you in the range of 10-64 layers and a d_{model} that gives reasonable hardware utilization (multiples of 128 for tensor-core alignment). The loss basin is wide enough that you will not see a meaningful degradation from non-optimal aspect ratios-spend your tuning budget elsewhere.
- **Set your starting batch size at or below the critical batch size, then ramp.** A practical heuristic: at the beginning of training (loss ≈ 3 -4 nats for language modeling), start with a batch size of 0.5-2M tokens. As loss drops to ≈ 2 nats (mid-training), double the batch size. As loss approaches your target (e.g., 1.5 nats), double again. This mirrors what Llama 3 and similar runs do and keeps you in the efficient regime throughout.
- **Use Adam (or AdamW) by default; only switch if you have a scaling-law pilot showing a competing optimizer wins.** The empirical gap between Adam and SGD is approximately a $10\times$ compute factor in favor of Adam, consistent across task types and model sizes. There is no strong evidence that alternatives (Adafactor, Lion, etc.) reliably beat Adam across all scales; run a small pilot before committing.
- **Adopt μP if your team has the engineering bandwidth.** The payoff is largest when you expect to train many models at different scales-hyperparameter transfer eliminates re-tuning costs that compound across experiments. The cost is implementation complexity: μP requires modifying initialization variance, per-layer learning rate multipliers, and output scaling, all as functions of width. Mistakes are subtle. If your team is running a one-off training run, standard parametrization with a careful scaling law fit for learning rate is lower risk.
- **Verify parallelism before extrapolating any hyperparameter.** The scaling-law approach is only valid if the loss curves for different hyperparameter values are parallel. Run your pilot, plot loss-vs-compute

for each hyperparameter value, and confirm visually that the curves do not cross. If they cross, you cannot safely extrapolate the small-scale optimum to large scale.

How This Shows Up in Interviews

- **“How do you pick the learning rate for a new 70B model?”** A strong answer gives two paths: (1) standard parametrization-fit a power law on η^* vs. d from a small-scale pilot, extrapolate to the target width using $\eta^* \propto 1/d$, then validate with a short 1B-token warmup run; (2) μ P-tune η^* once on a 50M model, apply directly at 70B, skip the large-scale validation run. The candidate should note the engineering trade-off: path (1) is simpler but requires re-tuning at each scale; path (2) eliminates re-tuning but requires careful implementation.
- **“What is the critical batch size and how do you use it in practice?”** The strong answer defines it as the batch size above which marginal gradient steps become less efficient than ideal (gradient noise no longer decreasing proportionally with batch size), explains that it scales inversely with loss (lower loss $\rightarrow$ larger critical batch size), and describes the ramp strategy: start small and increase batch size as training progresses, matching the growing B^* .
- **Senior follow-up: “Explain μ P and why it solves the learning-rate transfer problem.”** The senior answer explains the mechanics: standard parametrization initializes weights with variance $\propto 1/d$ in attention layers but applies a width-independent learning rate, causing gradient signal magnitude to scale with d and forcing $\eta^* \propto 1/d$ to compensate. μ P explicitly scales initialization variance and per-layer update rates by powers of d so that forward-pass activations and weight updates remain $O(1)$ regardless of width. The consequence: the gradient update magnitude is invariant to width, so the global learning rate that minimizes loss at width 256 is numerically the same that minimizes loss at width 8192. The limitation: μ P guarantees this in the infinite-width limit; in practice, there are finite-width corrections, and labs still run short validation runs to confirm transfer.

Key Takeaways

- Hyperparameter stability (optimal value constant across scale) or predictable scaling (optimal value follows its own power law) can both be established cheaply with a small-scale pilot; never tune at large scale without this prior.
- Aspect ratio has a wide, flat loss basin—anything between 10 and 100 layers at a reasonable width is approximately optimal at every model size.
- Batch size should stay at or below the critical batch size B^* , which grows as training loss decreases; ramp batch size progressively during training to stay efficient.
- Under standard parametrization, optimal learning rate scales as $\sim 1/d$; μ P eliminates this dependence, allowing exact learning-rate transfer from a tiny pilot model to the full-scale run.
- Adam outperforms SGD by roughly $10\times$ in compute efficiency across scales; this gap is consistent enough that it does not need to be re-validated for each new task.

Chapter 14 Chinchilla and Beyond

14.1 Compute-Optimal Training

14.1.1 IsoFLOP Analysis Step by Step

Given a fixed compute budget-say 6×10^{23} FLOPs-should you train a 280-billion-parameter model on 350 billion tokens, or a 70-billion-parameter model on 1.4 trillion tokens? Both recipes consume identical compute, yet the second achieves lower loss on every benchmark reported in the Chinchilla paper. The IsoFLOP analysis that produced that answer is now the standard framework for justifying any resource allocation decision in pretraining; interviewers at labs and hyperscalers ask about it directly, and a candidate who can name Chinchilla without deriving the result is distinguishable from one who can.

The starting point is a FLOP accounting identity. For a dense transformer, the dominant cost of a training step is matrix multiplication through the non-embedding parameters N . A single forward pass over one token requires roughly $2N$ FLOPs; the backward pass adds another $4N$. Training for D tokens therefore costs:

$$C \approx 6ND$$

This identity holds to within a factor of two for standard transformer architectures. It makes C , N , and D interchangeable: fix any two and the third is determined.

Chinchilla models expected loss as the sum of three terms:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

E is the irreducible entropy of the data distribution. A/N^α captures the benefit of more parameters: a larger model fits more knowledge and achieves lower loss per token. B/D^β captures the benefit of more tokens: more training data reduces generalization error. All five constants $\{E, A, B, \alpha, \beta\}$ are fitted from empirical runs.

The IsoFLOP problem is: given a fixed budget C , choose N (and set $D = C/(6N)$) to minimize L . Substituting $D = C/(6N)$:

$$L(N) = E + \frac{A}{N^\alpha} + B \left(\frac{6N}{C} \right)^\beta$$

Taking the derivative with respect to N and setting it to zero:

$$\frac{\alpha A}{N^{\alpha+1}} = B\beta \left(\frac{6}{C} \right)^\beta N^{\beta-1}$$

Solving for N^* :

$$N^* = \left(\frac{\alpha A}{B\beta} \right)^{\frac{1}{\alpha+\beta}} \left(\frac{C}{6} \right)^{\frac{\beta}{\alpha+\beta}}$$

The key structural result is the exponent: $N^* \propto C^{\beta/(\alpha+\beta)}$. When $\alpha = \beta$ -which the Chinchilla data supports-this simplifies to $N^* \propto C^{0.5}$. The same derivation gives $D^* \propto C^{0.5}$, so optimal model size and optimal token count grow at the same rate with compute.

Chinchilla fitted $D^*/N^* \approx 20$ tokens per parameter at the compute-optimal allocation. Substituting into the budget identity $C = 6N^*D^* = 6 \times 20 \times (N^*)^2 = 120(N^*)^2$:

$$N^* = \left(\frac{C}{120} \right)^{0.5}$$

This is the operational formula: given a FLOP budget, take the square root of $C/120$ to read off the compute-optimal parameter count.

IsoFLOP analysis converts the joint question of model size and training duration into a single-variable optimization: fix C , substitute $D = C/(6N)$, differentiate, and solve. When data and model capacity exponents are equal, the optimal split is $D^/N^* \approx 20$ -a ratio that is independent of compute scale.*

The practical IsoFLOP procedure is:

1. Fix a compute budget C (in FLOPs).
2. Train several models of varying N each to its matched token count $D = C/(6N)$, holding C constant across all runs in the sweep.
3. Plot final loss against N ; identify the minimizing N^* empirically for this budget.
4. Fit the parametric loss model to all (N, D, L) triples to obtain α , β , and the associated constants.
5. Repeat the sweep at several values of C spanning at least two orders of magnitude to obtain a reliable power-law fit for $N^*(C)$.

The Kaplan et al. (2020) scaling laws preceded Chinchilla and reached a different conclusion: optimal model size grows faster than token count, suggesting $N^* \propto C^{0.73}$ and a ratio of roughly 1-2 tokens per parameter. The discrepancy traces to a methodological artifact.

Kaplan’s experiments used *truncated* cosine learning-rate schedules. When a model trains for more steps, the cosine schedule decays the learning rate further and yields a lower final loss-not because the larger model is inherently better, but because the schedule is better calibrated to longer runs. The comparison confounds model capacity with schedule quality: models trained longer appear better because they also experience a more favorable schedule.

Chinchilla used complete cosine schedules matched to each run’s actual training length. With that confound removed, the token-count benefit became visible: the data-scaling exponent β is not substantially smaller than the model-scaling exponent α , so the optimal recipe requires far more tokens per parameter than Kaplan suggested.

A Worked Example

Compute budget: $C = 6 \times 10^{23}$ FLOPs.

Compute-optimal allocation:

$$N^* = \left(\frac{6 \times 10^{23}}{120} \right)^{0.5} = (5 \times 10^{21})^{0.5} \approx 70 \text{ B parameters}$$

$$D^* = \frac{C}{6N^*} = \frac{6 \times 10^{23}}{6 \times 7 \times 10^{10}} \approx 1.4 \text{ T tokens}$$

Verification: $6 \times 70\text{B} \times 1.4\text{T} = 6 \times 9.8 \times 10^{22} \approx 5.88 \times 10^{23} \approx C$.

Kaplan allocation at the same compute:

GPT-3 (175B parameters, $\approx 300\text{B}$ tokens) consumed roughly $C_{\text{GPT-3}} \approx 3.1 \times 10^{23}$ FLOPs. Applying

Kaplan’s exponent of 0.73:

$$N_{\text{Kaplan}} \approx 175\text{B} \times \left(\frac{6 \times 10^{23}}{3.1 \times 10^{23}} \right)^{0.73} \approx 175\text{B} \times 1.61 \approx 282 \text{ B parameters}$$

$$D_{\text{Kaplan}} = \frac{C}{6 N_{\text{Kaplan}}} \approx \frac{6 \times 10^{23}}{6 \times 2.82 \times 10^{11}} \approx 354 \text{ B tokens}$$

Token-to-parameter ratio: $354\text{B}/282\text{B} \approx 1.25$ tokens/param, versus Chinchilla’s 20 tokens/param. Both models cost the same FLOPs. The Chinchilla allocation trains a model four times smaller on ten times more data-and achieves lower benchmark loss because the large Kaplan-recipe model’s capacity is never saturated by only 354B tokens.

What You Actually Decide in Practice

- **Count non-embedding parameters only.** The $C \approx 6ND$ identity applies to parameters that participate in matrix multiplications on every forward pass: attention projections, MLP weights, and layer norms. Embedding matrices are read once per token but do not contribute proportionally to per-step FLOPs. For a 70B model with a 128K vocabulary and $d_{\text{model}} = 8,192$, the embedding table is ≈ 2 GB-significant memory, but under 3% of total FLOPs.
- **Use complete cosine schedules when running IsoFLOP sweeps.** The Kaplan artifact is a direct warning: truncating the cosine schedule at different fractions of T_{max} introduces schedule-length confounds. Each run in your IsoFLOP sweep must use a schedule that decays fully to the minimum learning rate over that run’s actual step count.
- **Run sweeps over at least two orders of magnitude in C .** A single IsoFLOP slice gives one point on the $N^*(C)$ curve. Fitting the power law reliably requires at least five slices spanning C_{min} to $100 C_{\text{min}}$; otherwise the exponent estimate has wide confidence intervals.
- **Compute-optimal $\neq$ deployment-optimal.** A 70B model trained on 1.4T tokens is training-compute-optimal, but if it will serve 10^{12} inference requests, per-request cost matters as much as training cost. Production models are routinely trained on 100-2,000+ tokens per parameter to reduce the deployed model size that must serve those requests. Section 14.2 addresses this inference-aware regime explicitly.
- **The formula gives the right ballpark, not the exact answer.** $N^* = (C/120)^{0.5}$ assumes the Chinchilla exponents hold at your scale, dataset, and architecture. At very large scale ($> 10^{25}$ FLOPs) or for non-standard architectures (mixture-of-experts, multimodal), empirical evidence on whether $\alpha \approx \beta$ continues to hold is limited. Use the formula as a starting point and refine with pilot runs.

How This Shows Up in Interviews

- **“Given a FLOP budget of 6×10^{23} , how many parameters should your model have?”** Strong answer: use $N^* = (C/120)^{0.5} \approx 70\text{B}$, trained on 1.4T tokens. Derive the formula: start from $C = 6ND$, write the loss model $L = E + A/N^\alpha + B/D^\beta$, substitute $D = C/(6N)$, differentiate, set to zero. Note that $\alpha \approx \beta \approx 0.5$ collapses the exponent to 0.5, and $D^*/N^* \approx 20$ gives the constant 120.
- **“Why did Kaplan et al. get a different answer from Chinchilla?”** Strong answer: Kaplan used truncated cosine schedules, so models trained for more steps benefited from more schedule decay, not just more data. This made model size look more valuable than it is. Chinchilla used complete cosine schedules matched to each run’s actual length, revealing that $\beta \approx \alpha$ and that token count is roughly as valuable per unit compute as parameters.

- **Senior follow-up: “How does the IsoFLOP result change if you account for inference cost?”** Strong answer: the Chinchilla result minimizes training loss per FLOP, not lifecycle cost. If the model will serve R inference requests at $\approx 2N$ FLOPs each, total cost is $6ND + 2NR$. At fixed total tokens processed $D + R/3$, the cost-minimizing N is smaller than compute-optimal: a smaller, better-trained model amortizes inference cost more efficiently. This is the regime in which production models are trained on 100-2,000+ tokens per parameter.

Key Takeaways

- The FLOP budget identity $C \approx 6ND$ constrains model size and token count jointly: fix C and N and D is determined.
- The Chinchilla parametric loss model $L = E + A/N^\alpha + B/D^\beta$ minimizes at $N^* \propto C^{\beta/(\alpha+\beta)}$; empirically $\alpha \approx \beta \approx 0.5$, yielding $N^* \propto C^{0.5}$.
- The compute-optimal token-to-parameter ratio is $D^*/N^* \approx 20$; the formula $N^* = (C/120)^{0.5}$ encodes this directly.
- Kaplan’s $N^* \propto C^{0.73}$ resulted from a truncated cosine schedule artifact, not a genuine difference in how data and model capacity scale.
- Production models overtrain well beyond compute-optimal (100-2,000+ tokens/param) to reduce deployed model size and per-request inference cost.

14.1.2 Replications, Corrections, and the Epoch AI Re-Fit

The Chinchilla paper presented three independent methods for estimating compute-optimal scaling, and all three appeared to agree. When Epoch AI forensically re-digitized the data behind Method 3 in 2023, they found non-zero-mean residuals in the published nonlinear regression—a numerical artifact of local-minimum convergence that drove the fitted exponents away from those returned by Methods 1 and 2. A 0.03 shift in the scaling exponent is invisible in a log-log plot of a single experiment; extrapolated over four to five decades of compute, it predicts a meaningfully different optimal model size. Understanding this episode tells you how scaling-law estimates fail in practice and what diagnostics to run before trusting a published fit.

The Chinchilla paper estimated the parameters of the joint loss model:

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

using three distinct approaches applied to the same set of training runs:

1. **Method 1: empirical envelope.** For each FLOP budget C , identify the model that achieves the lowest final loss across all runs at that budget. Read off $N^*(C)$ directly from the observed loss envelope. Fit $\log N^*$ vs. $\log C$ to obtain the scaling exponent.
2. **Method 2: optimal-allocation regression.** Use the D^*/N^* ratio implied by Method 1. Fit a power law relating D^*/N^* to C across all budget levels. This method uses the full set of runs but targets only the ratio, not the full loss surface.
3. **Method 3: full nonlinear regression.** Fit all five parameters $\{E, A, B, \alpha, \beta\}$ simultaneously to all (N, D, L) observations by minimizing the sum of squared residuals between predicted and observed loss.

Methods 1 and 2 are robust: they require only that the empirical minimum is identifiable, and they are insensitive to the shape of the loss surface far from the optimum. Method 3 is statistically more efficient—it uses

all the data and jointly identifies all parameters-but nonlinear regression over a five-dimensional curved surface has multiple local minima, and the optimizer can converge to the wrong one.

The Epoch AI finding was that the Method 3 regression in the Chinchilla paper had converged to a local minimum. The diagnostic was straightforward: in a well-fitted regression, residuals $\hat{L} - L_{\text{obs}}$ should have approximately zero mean across all subgroups of the data. The published Method 3 fit had residuals that were systematically positive for small models and negative for large models-evidence that the fitted model was attributing too much benefit to model size (overestimating α). Reinitializing the nonlinear optimizer from the Method 1 and Method 2 estimates and rerunning the regression converged to a different solution with near-zero-mean residuals that agreed closely with both earlier methods.

The Epoch AI re-fit showed that the three Chinchilla methods do agree when Method 3 is initialized correctly. The discrepancy was a numerical artifact of local-minimum convergence, diagnosable by checking that regression residuals have zero mean across regimes of N , D , and C -the standard first diagnostic for any scaling-law fit.

Two systematic risks in fitting scaling laws become concrete in this episode.

The first is initialization sensitivity in nonlinear regression. The five-parameter Chinchilla loss surface is highly curved and has many local minima; a starting point far from the global minimum reliably produces a biased fit. Standard practice-run from multiple starting points (grid search or random initialization) and accept only solutions for which residuals are approximately zero-mean across all data subgroups-would have caught the problem before publication.

The second is exponent compounding at scale. Scaling laws are power laws: $N^*(C) \propto C^{\beta/(\alpha+\beta)}$. A small error in the exponent amplifies with every decade of compute extrapolation. At the scale of Chinchilla's own experiments, Methods 1, 2, and the biased Method 3 gave similar enough predictions that the error was undetectable empirically. Extrapolating to frontier runs-four to five decades beyond the calibration range-produces a compounding gap that translates directly into mis-sized models.

The re-fit ultimately strengthened confidence in the main Chinchilla result. Once the local-minimum artifact was corrected, all three methods agreed on $\alpha \approx \beta \approx 0.5$ and $D^*/N^* \approx 20$, providing independent validation from three methodologically different approaches.

A Worked Example

Suppose the corrected fit gives $N^* \propto C^{0.50}$ while the originally published Method 3 fit gave $N^* \propto C^{0.53}$. The two predictions are calibrated to agree at the lower end of Chinchilla's compute range, $C_0 \approx 10^{20}$ FLOPs.

Multiplicative error at Chinchilla scale ($C = 6 \times 10^{23}$, roughly $6,000 \times C_0$):

$$\frac{N_{\text{biased}}^*}{N_{\text{corrected}}^*} = \left(\frac{C}{C_0} \right)^{0.53-0.50} = (6 \times 10^3)^{0.03} = e^{0.03 \times \ln(6000)} \approx e^{0.26} \approx 1.30$$

A 30% over-prediction at Chinchilla's own compute scale-noticeable but not catastrophic.

Multiplicative error at frontier scale ($C = 10^{25}$, roughly $10^5 \times C_0$):

$$\frac{N_{\text{biased}}^*}{N_{\text{corrected}}^*} = (10^5)^{0.03} = 10^{0.15} \approx 1.41$$

A 41% over-prediction. If the corrected fit recommends 300B parameters, the biased fit recommends ≈ 425 B-a difference large enough to push the run into a different hardware tier or force re-architecture for tensor parallelism.

Residual diagnostic illustration: Split the Chinchilla runs by N quintile. In the biased Method 3 fit:

- Bottom quintile ($N < 1\text{B}$): $\hat{L} - L_{\text{obs}} > 0$ (fit over-predicts loss for small models-too pessimistic about their quality).
- Top quintile ($N > 10\text{B}$): $\hat{L} - L_{\text{obs}} < 0$ (fit under-predicts loss for large models-too optimistic).

This monotone sign pattern-positive residuals at small N , negative at large N -is the fingerprint of an over-estimated capacity exponent α . The fitted model is more confident in large models than the data warrants, and uses the non-zero-mean residuals to absorb the bias. Checking for this pattern costs one group-by on the residual dataframe and takes under a minute.

What You Actually Decide in Practice

- **Always check that residuals are zero-mean across data subgroups.** For scaling-law fits, bin residuals by $\log N$, $\log D$, and $\log C$ separately. A monotone sign pattern in any of these signals local-minimum convergence or model misspecification. Both are fixable; neither is detectable without this check.
- **Run the nonlinear regression from multiple initializations.** Initialize from the Method 1 / Method 2 estimates, from random perturbations of that estimate, and from naive starting points. Accept only fits in which all initializations converge to the same parameter values. If they disagree, the loss surface has multiple competing minima and you need a broader search (Nelder-Mead from a grid of starts, or basin-hopping).
- **Methods 1 and 2 are more robust and should be reported alongside Method 3.** The envelope approach and the optimal-ratio fit do not require solving a five-dimensional non-convex problem; they are less statistically efficient but less prone to local-minimum failure. If your Method 3 disagrees with both, trust Methods 1 and 2 as the sanity check.
- **Treat published exponents as point estimates with uncertainty.** The Chinchilla exponents were fitted on a specific dataset (MassiveText), architecture (decoder-only transformer), and compute range (6×10^{18} to 3×10^{23} FLOPs). Applying them to a different modality, architecture, or compute regime larger than the training range requires either refitting or explicitly widening your uncertainty interval on N^* .
- **Report confidence intervals on scaling exponents when making resource decisions.** A 95% interval on α and β translates directly into a confidence interval on $N^*(C)$. Reporting only point estimates conceals the prediction uncertainty that grows with each decade of compute extrapolation.

How This Shows Up in Interviews

- **“Did the three Chinchilla methods agree with each other?”** Strong answer: Methods 1 and 2 agreed closely. Method 3, as originally published, produced slightly different exponents because the nonlinear regression converged to a local minimum. Epoch AI’s 2023 re-analysis found non-zero-mean residuals split by model size in the Method 3 fit. Reinitializing from the Method 1 and 2 estimates brought Method 3 into agreement with the others, strengthening confidence in $D^*/N^* \approx 20$ and $N^* \propto C^{0.5}$.
- **“How would you detect that a scaling-law fit is wrong?”** Strong answer: check residuals. In a well-fitted regression, $\hat{L} - L_{\text{obs}}$ should be zero-mean and uncorrelated with $\log N$, $\log D$, and $\log C$. A monotone sign pattern in any of these-positive residuals for small models and negative for large, or vice versa-means the fitted model is misspecified or the optimizer found a local minimum. The fix is multi-start nonlinear regression initialized near the Method 1 / 2 estimate, and verification that all starts converge to the same parameter values.

- **Senior follow-up: “By how much does a 0.03 exponent error affect a frontier model recommendation, and when does it become practically significant?”** Strong answer: the error grows as $(C/C_0)^{0.03}$ where C_0 is the calibration point. At $1,000\times$ the calibration compute ($\approx$ Chinchilla scale) the error is $\approx 19\%$. At $10^5\times$ the calibration (frontier training runs) it reaches $\approx 41\%$ -enough to shift a 300B-parameter recommendation to 425B, which has architectural and hardware implications. The error grows slowly but monotonically with log-compute extrapolation, so small exponent inaccuracies matter more the further you push beyond the data used to fit the law.

Key Takeaways

- The Chinchilla paper’s three methods did not agree internally because Method 3’s nonlinear regression converged to a local minimum; Epoch AI’s re-fit corrected this by using better initialization and checking residual zero-mean.
- The diagnostic for local-minimum convergence is non-zero-mean residuals binned by $\log N$: systematic over-prediction for small models and under-prediction for large models identifies an over-estimated capacity exponent α .
- A 0.03 exponent shift produces a $\approx 19\%$ error in N^* at Chinchilla-scale compute and a $\approx 41\%$ error at frontier scale ($10^5\times$ the calibration range)-practically significant when sizing a multi-billion-dollar training run.
- The corrected re-fit strengthened, not weakened, confidence in the main Chinchilla result: $N^* \propto C^{0.5}$ and $D^*/N^* \approx 20$.
- Standard practice for any scaling-law fit: multi-start nonlinear regression, residual zero-mean check across all data subgroups ($\log N$, $\log D$, $\log C$), and exponent confidence intervals on any prediction that extrapolates beyond the fitted compute range.

14.2 Inference-Aware Token-to-Parameter Ratios

14.2.1 Why Modern Models Overtrain on Tokens

Every serious training run eventually hits the same question: given a fixed compute budget, should you scale up model size or token count? Chinchilla answered this for the case of minimizing training loss per FLOP, and the answer was 20 tokens per parameter. But that answer optimizes the wrong objective for production. A model you serve to millions of users accumulates inference FLOPs that will dwarf the one-time training cost within weeks. Practitioners at every major lab have quietly moved the ratio far above 20, and the empirical record from Llama 3, MiniCPM, and Hunyuan makes clear that the 20:1 figure is a floor, not a target. This topic appears in senior system design interviews whenever the conversation shifts from “how do you train?” to “why did you make this model that size?”

Chinchilla defines the *compute-optimal* point: given a training FLOP budget C , the model size N^* and dataset size D^* that minimize loss satisfy $N^* \propto C^{0.5}$ and $D^* \propto C^{0.5}$, giving $D^*/N^* \approx 20$. This is a Pareto frontier for the specific objective of minimizing loss for a fixed training spend. It says nothing about what happens after training ends.

Inference cost scales with N : every token generated passes through the full model. If you will serve M

inference tokens after training, total system cost is approximately:

$$C_{\text{total}} \approx 6ND + 2NM,$$

where $6ND$ is training cost and $2NM$ is cumulative inference cost (both in FLOPs). For any deployment at scale, $M \gg D$ within weeks, and the second term dominates. The right question then is: given a fixed N (bounded by serving budget), what is the best D ? The answer is: push D as high as data quality allows. A smaller, more-trained model is cheaper to serve indefinitely.

A separate empirical question is how much worse the model gets as you push D beyond the compute-optimal point. Researchers from the University of Washington and Apple studied this directly, framing it as the *Chinchilla penalty*: given a compute-optimal baseline at $D^* = 20N$, how much does validation loss increase if you instead train with $D = cN$ for $c \gg 20$? Their finding is that the penalty follows a roughly log-linear curve—small and predictable at moderate overtraining, not catastrophic. Training a 7B model at 640 tokens per parameter inflates loss by only a few percent compared to a freshly trained Chinchilla-optimal model of the same size, yet halves the compute and memory required at inference time compared to a Chinchilla-optimal model trained on the same total FLOP budget.

Compute-optimal and inference-optimal are different objectives; production ML optimizes the latter, which consistently pushes the token-to-parameter ratio well above 20.

Three independent forces push modern labs toward higher token-to-parameter ratios.

First, **data quality has improved**. The original Chinchilla experiments were run on the Pile, a relatively noisy web-scraped corpus. Modern training sets apply aggressive deduplication, quality filtering, and domain reweighting. Cleaner data means each token carries more signal, effectively shifting the isoFLOP curve so the compute-optimal ratio rises. Llama 3’s own replication of the Chinchilla analysis finds roughly 39:1 as the compute-optimal ratio for their data mixture—already double the original figure.

Second, **architectural efficiency has improved**. Better tokenizers, grouped-query attention, and improved normalization schemes all increase the information extracted per training FLOP. If your architecture learns faster from the same tokens, the compute-optimal point shifts further toward tokens and away from parameters. MiniCPM’s curve-fit estimates 192:1, a number the authors themselves note is at the far end of published estimates and likely reflects both data quality and their architectural choices.

Third, **serving economics are asymmetric**. Training is paid once; inference is paid on every request. For a 7B model at 200 tokens per parameter versus a 70B model at 20 tokens per parameter—both costing the same to train—the 7B model generates tokens at $10\times$ lower inference cost. At the scale of a product serving millions of daily active users, that asymmetry dominates every training-efficiency argument.

A Worked Example

Suppose your compute budget is $C = 10^{23}$ FLOPs. You are choosing between two configurations:

- **Config A (Chinchilla-optimal)**: $N = 8\text{B}$, $D = 160\text{B}$ tokens (20:1 ratio). Training cost: $6 \times 8 \times 10^9 \times 1.6 \times 10^{11} \approx 7.7 \times 10^{21}$ FLOPs.
- **Config B (inference-aware)**: $N = 1\text{B}$, $D = 1.3\text{T}$ tokens (1300:1 ratio). Training cost: $6 \times 10^9 \times 1.3 \times 10^{12} \approx 7.8 \times 10^{21}$ FLOPs.

Both consume roughly the same training budget. Now consider 1 trillion inference tokens served:

$$C_{\text{infer, A}} = 2 \times 8\text{B} \times 10^{12} = 1.6 \times 10^{22} \text{ FLOPs,}$$

$$C_{\text{infer, B}} = 2 \times 1\text{B} \times 10^{12} = 2 \times 10^{21} \text{ FLOPs.}$$

Config B costs $8\times$ less per inference token. Across 1T served tokens, the saving is 1.4×10^{22} FLOPs—roughly double the original training budget. Config A’s superiority in raw loss (it was compute-optimal) is wiped out at any deployment scale above a few billion inference queries.

At the Chinchilla penalty curve’s typical slope, Config B’s loss inflation relative to A is 3-5% in perplexity. For most downstream tasks, that difference is smaller than the variance from data mixture changes. The deployment math is not even close.

What You Actually Decide in Practice

- **Do not cargo-cult 20:1.** The Chinchilla 20:1 ratio was fit on a specific dataset at a specific time. Run your own isoFLOP analysis on your data mixture; labs routinely find 30-100:1 as the compute-optimal point, before adjusting for inference.
- **Estimate your inference-to-training ratio before choosing N .** If your deployment will serve M inference tokens, and $M > 3D^*$, the inference-optimal N is strictly smaller than the Chinchilla-optimal N at the same training budget.
- **Cap repetition at 2-3 epochs.** Data quality bounds how far you can push D . Repeating data beyond about 2 epochs on web-scale corpora measurably hurts loss; the penalty curve steepens sharply past this point. Unique high-quality tokens are always preferable to repeated lower-quality ones.
- **Cite the right ratio for MoE models.** Hunyuan-Large finds 96:1 for active parameters, but the total parameter count is far higher due to the mixture-of-experts routing. When comparing ratios across architectures, always clarify whether the denominator is total or active parameters.
- **Validate with small-scale proxies before committing.** Run 10-20 proxy model pairs spanning 10^{17} - 10^{20} FLOPs to fit your own Chinchilla curve. Add 3-5 points at the overtrained end to empirically measure the penalty slope before you scale up.

How This Shows Up in Interviews

- **“Chinchilla says 20 tokens per parameter is optimal. Llama 3 trained a 70B model on roughly 15T tokens-over 200:1. Was that wasteful?”** Strong answer: compute-optimal and deployment-optimal are different objectives. Chinchilla minimizes training loss per FLOP spent in training; Llama 3’s choice minimizes total cost across training and serving. At a production scale of billions of daily requests, a 70B model at 200:1 is far cheaper to operate than a Chinchilla-optimal 700B model trained on the same budget—and the loss difference is empirically small due to improved data quality.
- **“How would you decide the right token-to-parameter ratio for a new model?”** Strong answer: first, run isoFLOP analysis on small proxies at your data distribution to find the compute-optimal ratio; second, estimate your projected inference-to-training ratio from product requirements; third, adjust the ratio upward from compute-optimal accordingly; fourth, check the overtrained region empirically with a few small pilot runs before committing the large budget.
- **Senior follow-up: “Your isoFLOP analysis says 40:1 is optimal, but you only have 500B tokens of clean licensed data and a 30B-parameter budget. Walk me through your options.”** The constrained ratio is $500\text{B}/30\text{B} \approx 17 : 1$, below compute-optimal. Option 1: shrink the model to 25B parameters to reach 20:1; you sacrifice some model capacity but achieve compute optimality for the available data. Option 2: accept the undertrained 30B model—if inference volume is low, the larger model’s quality advantage may be worth the training inefficiency. Option 3: invest in data acquisition to raise the token

budget. The right call depends on whether serving cost or model quality is the binding constraint; frame that trade-off explicitly in the interview.

Key Takeaways

- Chinchilla’s 20:1 optimizes training efficiency; inference-optimal training consistently pushes the ratio to 40:1, 100:1, or higher because serving cost-not training cost-dominates total system spend at production scale.
- Llama 3 finds $\approx 39:1$ compute-optimal on modern data; MiniCPM estimates 192:1; Hunyuan uses 96:1 for active parameters. None suffer catastrophic degradation.
- The excess-token penalty is real but log-linear and small at moderate overtraining; catastrophic degradation sets in only after heavy data repetition, typically beyond 2-3 epochs.
- Always rerun isoFLOP analysis on your own data mixture; the 20:1 figure is a historical reference point from one study on one dataset.
- When citing token-to-parameter ratios for MoE models, specify whether the denominator is total or active parameters.

14.2.2 Learning-Rate Schedules: Cosine vs. WSD

To fit Chinchilla-style scaling curves, you need validation loss as a function of both model size and dataset size. The dataset-size axis requires training each model to multiple data targets. With a standard cosine learning rate schedule, every distinct data target demands its own training run from scratch-checkpoints from an ongoing run are not a valid substitute. At even moderate scales, that requirement is ruinously expensive. MiniCPM popularized a schedule that reduces this $O(N^2)$ cost to roughly $O(1)$: the Warmup-Stable-Decay schedule, now so widely copied that training loss curves with sharp late-stage drops have become a recognizable fingerprint of the approach. The schedule also clarifies something fundamental about why learning rate annealing works at all-a question with direct relevance to optimizer design interviews at senior and staff level.

A cosine schedule defines a learning rate at step t as:

$$\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T}\right) \right),$$

where T is the pre-specified total number of training steps. The schedule’s shape is entirely determined by T . A run targeting D_1 tokens uses a cosine with period T_1 ; a run targeting $D_2 > D_1$ tokens uses a cosine with period $T_2 > T_1$. A checkpoint at step D_1 /(batch size) during the D_2 run has a higher learning rate at that point than a dedicated D_1 run that has fully annealed-the two models have seen the same number of tokens, but their optimizer state and effective annealing are not the same. Terminal loss depends on the full shape of the schedule, not just total tokens seen. You cannot use early checkpoints from a long cosine run as proxies for shorter training runs.

The *Warmup-Stable-Decay* (WSD) schedule breaks training into three phases with a trapezoid shape:

$$\eta(t) = \begin{cases} \eta_{\max} \cdot t/T_w & 0 \leq t < T_w \quad (\text{warmup}) \\ \eta_{\max} & T_w \leq t < T_s \quad (\text{stable}) \\ \eta_{\max} \cdot f\left(\frac{t-T_s}{T_d}\right) & T_s \leq t < T_s + T_d \quad (\text{decay}) \end{cases}$$

where T_w is warmup length, T_s is stable phase end, T_d is decay length, and f is a monotonically decreasing function from 1 to 0 (typically cosine or linear). The stable phase is constant. Any checkpoint within the stable

phase at step t^* is a valid starting point for a fresh decay of length T_d . The resulting terminal loss is equivalent to a model that was trained with a dedicated WSD schedule terminating at t^* .

This property transforms the Chinchilla analysis procedure. Instead of K separate runs from scratch, you run one stable phase to the maximum token budget, save checkpoints throughout, and apply K short independent decays-one per data target. Total compute: stable run + $K \times T_d$ steps.

WSD makes the learning rate annealing separable from the data accumulation phase, enabling cheap Chinchilla-style analysis without sacrificing terminal model quality.

The efficiency gain is intuitive once you understand why the decay phase matters so much. During the stable phase, the optimizer is taking large, noisy steps through the loss landscape. The model improves steadily but slowly; the high learning rate prevents settling into any local basin. During the decay phase, the step size shrinks, and the optimizer converges into a nearby low-loss region. Almost all of the gap between “undertrained stable” and “fully converged” model quality closes during the decay. This is not unique to WSD: the same phenomenon happens in cosine schedules, where the final 20-30% of training-when the cosine drops steeply-accounts for a disproportionate share of the total loss reduction.

The practical implication is that the stable phase is nearly interchangeable across checkpoint positions: a model at step 50B and a model at step 80B of a stable phase differ in loss by roughly the amount they would differ on any training run, but they will both converge to approximately the same loss quality after a standard decay. The stable phase is a continuous bank of launching pads; the decay is the final landing.

This contrasts with cosine, where the learning rate is falling throughout, meaning each step has a slightly different effective annealing budget. WSD makes the transition sharp and cleanly separated, which is why it enables checkpoint reuse while cosine does not.

DeepSeek LLM adopted WSD with a two-stage decay (two consecutive decays of roughly 10% of total compute each), finding the results largely insensitive to the exact decay shape. Their empirical conclusion, consistent with MiniCPM’s, is that cosine and WSD reach comparable final losses, but WSD’s reusable stable phase is worth more than any marginal quality difference between the two schedules.

A Worked Example

You want to fit Chinchilla scaling for a 3B-parameter model across 8 data targets: 15B, 30B, 60B, 120B, 240B, 480B, 960B, and 1.9T tokens. Training is on 80 H100s at an effective throughput of 250 TFLOPS per GPU.

Cosine approach. Each data target requires its own run. Total compute:

$$\begin{aligned} C_{\text{cosine}} &= 6N \sum_i D_i = 6 \times 3\text{B} \times (15 + 30 + 60 + 120 + 240 + 480 + 960 + 1900)\text{B} \\ &\approx 6 \times 3 \times 10^9 \times 3.8 \times 10^{12} \approx 6.8 \times 10^{22} \text{ FLOPs.} \end{aligned}$$

At $80 \times 250 \times 10^{12} \text{ FLOPs/s} = 2 \times 10^{16} \text{ FLOPs/s}$:

$$t_{\text{cosine}} \approx 6.8 \times 10^{22} / 2 \times 10^{16} \approx 3.4 \times 10^6 \text{ s} \approx 39 \text{ days.}$$

WSD approach. One stable run to 1.9T tokens plus 8 decays at 10% of each target:

$$\begin{aligned} C_{\text{stable}} &= 6 \times 3\text{B} \times 1.9\text{T} \approx 3.4 \times 10^{22} \text{ FLOPs,} \\ C_{\text{decays}} &= 6 \times 3\text{B} \times 0.1 \times 3.8\text{T} \approx 6.8 \times 10^{21} \text{ FLOPs.} \end{aligned}$$

Total: $\approx 4.1 \times 10^{22}$ FLOPs, roughly 40% less than the cosine approach.

$$t_{\text{WSD}} \approx 4.1 \times 10^{22} / 2 \times 10^{16} \approx 24 \text{ days.}$$

You save 15 days of H100 cluster time-at \$2/GPU-hour, that is approximately \$96,000 saved-to get 8 data-scale points instead of one.

What You Actually Decide in Practice

- **Use 10-20% of total compute for the decay phase.** MiniCPM and DeepSeek both land in this range. Below 5%, the model fails to fully anneal; you leave several percent of loss improvement on the table. Above 20%, you are running the decay for longer than necessary-returns diminish rapidly after the optimizer has settled.
- **Save stable-phase checkpoints at least every 5-10% of total tokens.** Coarse checkpointing limits the granularity of your scaling curve and forces you to interpolate between sparse data points. Dense checkpoints are cheap storage relative to the training cost; save them.
- **Never use early cosine checkpoints for scaling analysis.** This is the single most common mistake in in-house scaling studies. A checkpoint at 25% of a long cosine run has a higher LR than a dedicated short run fully annealed to that point. The losses are not comparable. If you see published results that use mid-run cosine checkpoints as data-scale proxies, treat them skeptically.
- **WSD and cosine are quality-equivalent at the final checkpoint.** The advantage of WSD is flexibility, not quality. If you are training a single model to a fixed token count with no plans to run scaling analysis, cosine is fine. Choose WSD when you need checkpoint reuse.
- **The decay phase is the optimizer settling into a basin.** If you cut the training run short before the decay completes, you will see a significant loss elevation. Budget the decay before you commit the training run: it is not optional.

How This Shows Up in Interviews

- **“You want to fit a Chinchilla scaling law for your model family. A teammate suggests saving checkpoints from a single long cosine training run to get multiple data-scale points. What’s wrong with this?”** Strong answer: terminal loss depends on the full learning rate schedule, not just tokens seen. An early checkpoint in a long cosine run has a higher LR than a dedicated shorter run that fully anneals-the two models are not equivalent data points on the scaling curve. You will systematically underestimate loss for short training runs, producing a biased fit. The correct approach is either independent runs or WSD with checkpoint branching.
- **“What is the WSD learning rate schedule and why did MiniCPM introduce it?”** Strong answer: WSD separates training into a warmup phase, a flat stable phase, and a short steep decay. The stable phase is reusable: any stable-phase checkpoint can be branched into an independent decay to produce a terminal-quality model equivalent to a dedicated shorter training run. MiniCPM introduced it to enable cheap Chinchilla-style scaling analysis-getting K data-scale data points at the cost of one long stable run plus K short decays, versus K separate training runs from scratch.
- **Senior follow-up: “Why does the decay phase recover so much loss so quickly? What is the optimizer doing during the stable phase?”** The stable phase takes large steps, exploring a wide region of the loss landscape. High learning rate prevents convergence into any single basin-the model is in a noisy

equilibrium between gradient signal and step size. The decay phase gradually reduces step size, allowing the optimizer to follow the gradient toward the basin bottom. Virtually all the loss improvement relative to no-cooldown is attributable to the decay. This is why cosine schedules that terminate early (before the cosine reaches its minimum) produce significantly worse models: the annealing step-not the data accumulation-drives the final loss improvement.

Key Takeaways

- Cosine LR schedules are training-length-dependent: early checkpoints from a long cosine run are not equivalent to dedicated shorter runs, making them invalid proxies for Chinchilla scaling analysis.
- WSD (Warmup-Stable-Decay) separates data accumulation from annealing: the flat stable phase produces reusable checkpoints; branching a decay from any stable checkpoint yields terminal-quality results.
- WSD reduces the cost of K -point Chinchilla analysis to one stable run plus $K \times 10\%$ of that run's cost-a factor of $\sim 3\text{-}5 \times$ cheaper than K independent cosine runs.
- The decay phase accounts for nearly all the loss improvement over an unannealed checkpoint; budget 10-20% of total compute for it and never cut it short.
- WSD and cosine converge to equivalent final quality; the advantage of WSD is the flexibility of checkpoint reuse, not intrinsic quality superiority.

14.3 Maximal Update Parameterization (muP)

14.3.1 Initialization and Per-Layer Learning Rates

You carefully tune a learning rate on a 200M proxy run, scale up to 7B, and discover the optimal rate has shifted by a factor of four. That is the standard parameterization (SP) tax: as models grow wider, the optimal learning rate migrates predictably downward, forcing expensive re-tuning at every new scale. Maximal Update Parameterization (μP) eliminates this by choosing initialization variances and per-layer learning rates so that activation magnitudes and gradient update sizes stay $\Theta(1)$ per coordinate regardless of width. At senior MLE and research engineer interviews you will be expected to explain both why the optimal LR drifts in SP and exactly how μP fixes it.

μP rests on two spectral conditions that must hold as model width n grows:

1. **Activation stability (A1):** At initialization, every coordinate of every hidden vector $h^{(l)}$ is $\Theta(1)$, i.e., $\|h^{(l)}\|_2 = \Theta(\sqrt{n_l})$.
2. **Update stability (A2):** After a single gradient step, the change in each activation coordinate is also $\Theta(1)$, i.e., $\|\Delta h^{(l)}\|_2 = \Theta(\sqrt{n_l})$.

To see what these conditions require, consider a deep linear network:

$$h^{(l)} = W^{(l)} h^{(l-1)}, \quad W^{(l)} \sim \mathcal{N}\left(0, \frac{\sigma_l^2}{n_{l-1}}\right)_{\text{elementwise}}.$$

By random-matrix concentration, the operator norm of $W^{(l)}$ satisfies

$$\|W^{(l)}\|_{\text{op}} \approx \sigma_l \frac{\sqrt{n_l} + \sqrt{n_{l-1}}}{\sqrt{n_{l-1}}}.$$

Assuming inductively that $\|h^{(l-1)}\| = \Theta(\sqrt{n_{l-1}})$ and requiring $\|h^{(l)}\| = \Theta(\sqrt{n_l})$, the correct initialization

scale works out to:

$$\sigma_l = \frac{1}{\sqrt{n_{l-1}}} \cdot \min\left(1, \sqrt{\frac{n_{l-1}}{n_l}}\right).$$

For the common case where $\text{fanin} \leq \text{fanout}$ (e.g., the up-projection in an MLP where $n_l = 4 n_{l-1}$), this collapses to $\sigma_l = 1/\sqrt{n_{l-1}}$, which is exactly Kaiming (He) initialization. Condition A1 is already satisfied if you are using Kaiming init.

For condition A2 the key quantity is $\|\Delta W^{(l)}\|_{\text{op}}$ after one Adam step. Adam normalizes each gradient element to $O(1)$, so every entry of ΔW is $O(\eta)$. For a weight matrix of shape $n_l \times n_{l-1}$, the operator norm of a matrix with all entries $O(\eta)$ is $O(\eta\sqrt{n_l n_{l-1}})$. The change in activation is then:

$$\|\Delta W^{(l)} h^{(l-1)}\| \leq \|\Delta W^{(l)}\|_{\text{op}} \|h^{(l-1)}\| = O(\eta\sqrt{n_l n_{l-1}}) \cdot O(\sqrt{n_{l-1}}) = O(\eta\sqrt{n_l} n_{l-1}).$$

For this to equal $\Theta(\sqrt{n_l})$ we need:

$$\boxed{\eta_{\text{Adam}} \propto \frac{1}{n_{l-1}} = \frac{1}{\text{fanin}}}.$$

For SGD, where $\Delta W = \eta G$ and G has a rank-1 structure with $\|G\|_{\text{op}} = \Theta(\sqrt{n_l n_{l-1}})$, the analogous calculation yields $\eta_{\text{SGD}} \propto n_l/n_{l-1}$. The Adam and SGD rules coincide only when the aspect ratio is 1; in a standard transformer they differ. Because nearly all production runs use Adam, the operative rule is $\eta_l \propto 1/n_l^{(\text{in})}$.

The practical difference between μP and SP is almost entirely in the per-layer learning rates, not the initialization.

Under SP , a single global LR η applies to all layers. As width doubles, the Adam update norm grows as $O(\sqrt{n})$ per layer because the matrix is bigger, forcing η to halve to compensate. The optimizer's view of the loss landscape distorts with every width change.

μP fixes this by absorbing the width dependence into the per-layer LR directly. You maintain a base rate η_{base} and multiply each layer's LR by $1/n_{\text{in}}$. A sweep on a narrow 40M proxy maps out the loss landscape in η_{base} space; that landscape is identical at 7B or 70B by construction. One sweep governs all scales.

The trade-off is implementation overhead. You replace a scalar LR with a parameter-group table. Embedding layers are exempt because their effective width does not scale with vocab size—they receive η_{base} unchanged. The attention QK inner product must use $1/d_{\text{head}}$ scaling instead of the conventional $1/\sqrt{d_{\text{head}}}$, a non-obvious change required to satisfy A2 in the attention sublayer.

A Worked Example

Scale a 400M model ($d_{\text{model}} = 1024$) up to 7B ($d_{\text{model}} = 4096$), both using $d_{\text{ff}} = 4 d_{\text{model}}$.

Initialization. For the MLP up-projection ($\text{fanin} = d_{\text{model}} = 4096$):

$$\sigma = \frac{1}{\sqrt{4096}} = \frac{1}{64} \approx 0.0156.$$

This is identical to Kaiming init. No change needed.

Per-layer Adam LR. Suppose the proxy sweep on a 40M model ($d_{\text{model}} = 256$) identified $\eta_{\text{base}} = 3 \times 10^{-3}$. Under μP , at 7B the MLP up-projection receives:

$$\eta_{\text{up}} = \frac{\eta_{\text{base}}}{d_{\text{model}}} = \frac{3 \times 10^{-3}}{4096} \approx 7.3 \times 10^{-7}.$$

The embedding layer still uses $\eta_{\text{base}} = 3 \times 10^{-3}$.

Scaling to 70B. At 70B with $d_{\text{model}} = 8192$:

$$\eta_{\text{up}} = \frac{3 \times 10^{-3}}{8192} \approx 3.7 \times 10^{-7}.$$

η_{base} remains 3×10^{-3} . No new sweep is run.

SP comparison. Under SP with the same proxy, you would carry $\eta = 3 \times 10^{-3}$ to 7B and find instability or degraded loss. The true optimum at 7B SP is roughly $3 \times 10^{-3} \times (256/4096) \approx 1.9 \times 10^{-4}$ —a $16\times$ gap that requires a dedicated re-tune.

What You Actually Decide in Practice

- **Run the proxy at 40M-100M, not smaller.** Below about 30M parameters the width is too narrow for random-matrix asymptotics to be reliable. Cerebras used 40M; MiniCPM used 9M-30M with some caveats. If the proxy is too small, the LR optimum will wobble and the transfer will be noisy.
- **The initialization is usually already right.** PyTorch’s default linear layers use Kaiming init, which matches the μP formula for standard layer shapes. Audit only layers with unusual aspect ratios (e.g., very wide vocabulary projections).
- **Change the attention scale from $1/\sqrt{d}$ to $1/d$.** Forgetting this is the most common μP implementation bug. The conventional scale violates A2 in the attention sublayer, which means LR transfer breaks specifically in the attention heads even when everything else is correct.
- **Residual scaling requires attention.** If your architecture uses a learnable per-layer output multiplier, it must be initialized to zero or to $1/\sqrt{L}$ (where L is depth) to prevent activation accumulation across layers. μP addresses width scaling; depth scaling requires a separate correction.
- **μP does not improve quality by itself.** If you re-tune LR at every scale, SP and μP reach the same loss. The ROI is in compute savings on the hyperparameter search, not in an intrinsic quality gain.

How This Shows Up in Interviews

- **“Why does the optimal learning rate decrease as you scale up a transformer?”** Strong answer: as width n grows, the Adam update norm per layer scales as $O(\sqrt{n})$ with a global LR, so the effective per-activation update grows with width. Reducing LR compensates. μP pre-corrects each layer’s LR by $1/n_{\text{in}}$ so the update size stays constant.
- **“What is μP and what problem does it solve?”** Strong answer: μP chooses init scales ($1/\sqrt{n_{\text{in}}}$) and per-layer Adam LR ($\propto 1/n_{\text{in}}$) so that activation magnitudes and gradient update sizes remain $\Theta(1)$ per coordinate at any width. Optimal hyperparameters from a small proxy transfer to large scale without re-tuning.
- **“Walk me through the μP recipe for a transformer.”** Strong answer: non-embedding weights init at $1/\sqrt{n_{\text{in}}}$; Adam LR per layer scaled by $1/n_{\text{in}}$ relative to a base rate found on a small proxy; attention scale changed from $1/\sqrt{d_{\text{head}}}$ to $1/d_{\text{head}}$; embedding LR held constant at η_{base} .
- **Senior follow-up: “ μP is derived for SGD on a deep linear network. Why does the Adam rule differ?”** Senior answer: Adam normalizes each gradient element to $O(1)$, so the effective operator norm of ΔW is $O(\eta\sqrt{n_l n_{l-1}})$ regardless of gradient scale. Rederiving the update-stability condition under this normalization yields $\eta \propto 1/n_{\text{in}}$ instead of the SGD result of $n_{\text{out}}/n_{\text{in}}$. The nonlinearity and attention extensions require additional arguments but empirically follow the same scaling.

Key Takeaways

- μP enforces two conditions: activations and gradient updates both stay $\Theta(1)$ per coordinate as width grows.

- For Adam the result is per-layer LR $s \propto 1/n_{\text{in}}$ and init scales of $1/\sqrt{n_{\text{in}}}$ (Kaiming); for SGD the LR rule differs by a fanout factor.
- The practical difference from SP is almost entirely in the per-layer LR s , not the initialization.
- The attention QK scale must change from $1/\sqrt{d}$ to $1/d$ for the argument to hold in the attention sublayer.
- The payoff: sweep hyperparameters once on a $\sim 40\text{M}$ proxy, transfer the optimal LR to 7B or 70B without re-tuning.

14.3.2 What μP Is Robust To and What Breaks It

Knowing the μP derivation is table stakes. The question that separates practitioners from theorists is: which production modifications actually break LR transfer, and which ones are safe to apply freely? An interviewer asking “when would you not use μP ” expects a concrete list of failure modes, not a wave of the hand. The empirical record from Cerebras-GPT, MiniCPM, and a careful third-party ablation study gives a mostly positive answer with four well-defined caveats you need to know cold.

The μP promise is that you run a learning-rate sweep over widths $\{128, 256, 512\}$ on a cheap proxy model, identify the optimal η_{base} at the smallest scale, and that same η_{base} remains optimal when you scale to 7B or 70B. The sweep costs a few hundred GPU-hours; the transfer replaces a multi-million-GPU-hour re-tune.

Empirically, Cerebras-GPT demonstrated this end to end across models from 111M to 13B parameters. Under SP, predicted and actual test loss oscillate because LR must shift at each scale; under μP , the deviation between the scaling-law prediction and the measured loss collapses substantially. MiniCPM shows the analogous result on their 30M-1B range: the optimal η_{base} stays flat near 10^{-2} across an order of magnitude in model size. The optimal point in η_{base} space does not shift.

μP does not improve model quality relative to a perfectly tuned SP run—it eliminates the cost of achieving that perfect tuning at scale.

The LR transfer property survives most architectural variations because those variations do not violate conditions A1 or A2. Nonlinearities (ReLU, GELU, SwiGLU, squared-ReLU) preserve the $\Theta(1)$ per-coordinate activation property as long as the linear layers are correctly initialized. Batch-size changes rescale the gradient noise but do not alter the width dependence of the update magnitude. Variations in query initialization (zeroing W_Q for uniform initial attention) also turn out to be harmless because they do not change how activations scale with width.

Where μP breaks, the failure modes share a common structure: a new free parameter is introduced whose magnitude changes with model size, violating the $\Theta(1)$ update assumption without a corresponding correction in the per-layer LR table. The result is a drift in the optimal η_{base} across widths that, if undetected, means your proxy sweep is steering you to the wrong LR.

A Worked Example

Consider the Cerebras-GPT width-sweep experiment. Models of widths 128, 512, and 2048 are trained under both SP and μP across a grid of learning rates. Under SP, the loss minimum occurs at $\eta \approx 6 \times 10^{-4}$ for width 128, at $\eta \approx 2 \times 10^{-4}$ for width 512, and shifts again at width 2048. If you had locked in the width-128 optimum for the width-2048 run, you would train at $3\times$ the optimal LR and suffer instability or degraded terminal loss.

Under μP , the same three widths show a flat optimum: the loss valley sits at $\eta_{\text{base}} \approx 3 \times 10^{-3}$ regardless of width. At width 2048 the actual MLP up-projection LR is $3 \times 10^{-3}/2048 \approx 1.5 \times 10^{-6}$, but η_{base} itself does

not shift. The proxy sweep transfers exactly.

The Cerebras team also reports that scaling law predictions align much more tightly with measured loss under μ P than SP, because under SP the LR adjustment between scales introduces scatter around the predicted scaling line.

What You Actually Decide in Practice

- **Remove learnable gains from normalization layers, or set their LR to zero.** If your RMSNorm includes a learnable scalar γ initialized to 1, it acts as an implicit per-layer LR amplifier whose magnitude can grow during training. μ P's per-layer LR table does not account for this, so LR transfer breaks. Removing γ (or freezing it) restores the invariant. This is the most common production failure mode.
- **Keep weight decay at or below ~ 0.1 .** Large-scale ablations show that weight decay above roughly 0.1-0.3 disrupts LR transfer. The likely mechanism is that strong decay changes the effective learning rate seen by each layer as a function of weight magnitude, which is itself width-dependent. Standard pretraining values of $\lambda = 0.1$ are safe; $\lambda = 0.3$ and above introduce drift.
- **μ P is optimizer-specific; do not mix and match.** The per-layer formula $\eta_l \propto 1/n_{\text{in}}$ is derived for Adam. Lion (which uses the sign of a momentum-weighted gradient) and Adafactor both alter the update scaling in width-dependent ways that are not covered by the Adam derivation. If you switch optimizers, re-derive the correction or expect LR transfer to fail.
- **Consider the DeepSeek alternative.** DeepSeek LLM did not use μ P. They instead trained a grid of models at two small scales, measured the optimal LR and batch size at each scale, fitted a power-law relationship across FLOPs, and extrapolated to their 7B and 67B targets. The fitted LR curve is visibly noisier than a μ P sweep but gets the order of magnitude right. This approach requires no implementation complexity, but it costs the grid training runs and assumes that LR follows a clean power law in FLOPs—an assumption that is somewhat less principled. For a single fixed-target training run where you will not re-run at multiple scales, direct fitting is a defensible and simpler choice.
- **μ P is growing but is not yet consensus.** As of 2025, μ P (or the closely related μ Transfer variant) is in use at several frontier labs and was confirmed for Llama 4. Cerebras and MiniCPM published positive results. Other major runs (DeepSeek, Llama 2, Mistral) did not use it and still achieved high quality by direct tuning. In an interview, present both strategies and their trade-offs rather than claiming one is universally correct.

How This Shows Up in Interviews

- **“Under what conditions does μ P fail to transfer the learning rate?”** Strong answer: three main failure modes—learnable gains in normalization layers (introduce a scale-dependent implicit LR amplification), weight decay above ~ 0.1 -0.3 (disrupts the update-size invariant), and non-Adam optimizers (the per-layer correction is derived specifically for Adam and does not generalize without re-derivation).
- **“Cerebras uses μ P, DeepSeek does not. How would you choose for a new 70B training run?”** Strong answer: μ P requires clean implementation (per-layer LR, $1/d$ attention scale, no learnable norm gains) but amortizes the LR sweep cost across all future scaling experiments. Direct fitting is simpler but requires training a grid at multiple scales, which may cost comparable compute. If you plan to train multiple models at different scales—a research lab scenario— μ P pays off quickly. If you are training one model at a fixed target scale with a well-understood architecture, direct fitting is equally defensible.

- **“What does the LR landscape look like during a μ P width sweep?”** Strong answer: a flat valley-the loss optimum sits at the same η_{base} for widths 128, 512, and 2048. Under SP the valley shifts left (toward smaller η) as width increases, so the proxy optimum is stale by the time you reach production width.
- **Senior follow-up: “ μ P is usually validated for width scaling with fixed depth. What do you have to add for depth scaling?”** Senior answer: depth scaling introduces residual accumulation-activations grow roughly as $O(\sqrt{L})$ across L layers. Without a per-layer output scaling correction (e.g., initializing the output projection of each sublayer to zero or weighting it by $1/\sqrt{L}$), conditions A1 and A2 are violated along the depth axis. Production μ P implementations pair the width rules with a depth-dependent residual correction; the μ P papers as published primarily validate width and leave depth scaling as an open engineering problem.

Key Takeaways

- LR transfer under μ P is robust to nonlinearity choice (SwiGLU, squared-ReLU, GELU), batch-size variation, and standard init variants (including zeroed W_Q).
- Three documented failure modes: learnable normalization gains, weight decay $\lambda \gtrsim 0.1$ -0.3, and non-Adam optimizers.
- The DeepSeek alternative-fitting LR directly via a scaling-law power law-is a legitimate and simpler approach when training at a single fixed target scale.
- Depth scaling requires a separate residual correction beyond the standard width rules; do not assume μ P’s width analysis covers depth automatically.
- μ P adoption is growing (confirmed for Llama 4) but not universal; know both strategies and their trade-offs for interviews.

Part VII

Inference Systems

Chapter 15 The Inference Workload

15.1 Prefill vs. Generation

15.1.1 Compute-Bound Prefill

When an interviewer asks you to design a low-latency chat system and then asks “what determines your time-to-first-token?”, the answer bottoms out in prefill. This phase is what runs before the model emits a single output token: you feed the entire prompt through the transformer, computing all the intermediate activations and caching the key and value tensors for every layer. Getting this phase right is the central lever for TTFT, and understanding *why* it behaves differently from generation is a foundational piece of inference architecture knowledge that shows up at every seniority level.

Prefill is the phase in which all S prompt tokens are processed in a single forward pass. Because all input tokens are known before execution begins, the attention computation can be fully parallelized across the sequence dimension—exactly as in training. The transformer reads an input matrix $X \in \mathbb{R}^{S \times d}$, passes it through L layers of attention and MLP, and produces two outputs: the logit distribution over the vocabulary for the *next* token, and the KV cache that will be consumed by every subsequent generation step.

The key metric for understanding whether a computation is hardware-limited is *arithmetic intensity*: the ratio of floating-point operations to bytes transferred from high-bandwidth memory (HBM).

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes transferred}}$$

For an MLP layer during prefill, the dominant operation is two matrix multiplications (up-projection and down-projection). With batch size B and $T = S$ tokens, the intensity evaluates to $B \cdot S$. For the attention layers, the $QK^T V$ computation gives an intensity of roughly $S \cdot T / (S + T)$, which simplifies to $S/2$ when $T = S$ (as it is during prefill). Both expressions grow with sequence length. On an H100 the roofline crossover for a standard matrix multiply sits around 295 FLOPs/byte; prefill crosses that line comfortably once S is even modestly long.

Prefill is compute-bound: the GPU’s arithmetic units are the bottleneck, not the memory bus, which means you are extracting close to peak FLOP/s from the hardware.

The reason prefill gets to be compute-bound is structural: every token’s query, key, and value projections involve the same weight matrices, and with S tokens in flight simultaneously you are executing a true matrix-matrix multiply ($X \in \mathbb{R}^{S \times d}$, $W \in \mathbb{R}^{d \times f}$). Matrix-matrix multiplies have arithmetic intensity proportional to the matrix dimensions, so as S grows you can fully hide the cost of loading weights from HBM behind arithmetic.

Compare this to the alternative-naive recomputation where you feed one token at a time, recomputing all prior context from scratch. The complexity per output token would be $O(S^2)$ in FLOPs because you’d re-attend over the full prefix. The KV cache removes this redundancy: you pay $O(S)$ FLOPs during prefill to populate it, and thereafter each generation step costs $O(1)$ additional attention FLOPs per prior token. You are trading memory (storing the KV tensors) for computation (not recomputing them). At inference time, where you have no gradient bookkeeping, this trade is always worth it.

The practical consequence is that TTFT scales approximately linearly with S under fixed batch size, and reducing batch size during prefill gives you better latency at the cost of throughput. Unlike generation, adding sequences to a prefill batch genuinely helps because the arithmetic intensity scales with $B \cdot S$.

A Worked Example

Consider Llama-2 13B running on a single H100 SXM, processing a prompt of $S = 2,048$ tokens with batch size $B = 1$. The key dimensions are $d = 5,120$, $f = 4d = 20,480$, $L = 40$ layers, and $N = 40$ attention heads each of dimension $H = 128$. All tensors are in bf16 (2 bytes per element).

MLP arithmetic intensity. For one MLP layer:

$$\text{FLOPs} \approx 2 \times B \times S \times d \times f \times 3 \approx 2 \times 1 \times 2048 \times 5120 \times 20480 \times 3 \approx 1.3 \times 10^{12}$$

Bytes transferred (weights + activations):

$$\text{Bytes} \approx 2 \times (3 \times d \times f) \approx 2 \times 3 \times 5120 \times 20480 \approx 6.3 \times 10^8$$

Intensity $\approx B \cdot S = 1 \times 2048 = 2048$ FLOPs/byte, well above the H100 roofline of 295. The MLP is compute-bound.

Attention arithmetic intensity. With $S = T = 2048$:

$$\text{Intensity} \approx \frac{S \cdot T}{S + T} = \frac{2048 \times 2048}{4096} = 1024 \text{ FLOPs/byte}$$

Also comfortably compute-bound.

Prefill latency estimate. Total FLOPs for a full forward pass are approximately $6 \cdot N_{\text{params}} \cdot S \approx 6 \times 13 \times 10^9 \times 2048 \approx 1.6 \times 10^{14}$. At the H100's theoretical bf16 throughput of ~ 989 TFLOP/s, compute time is roughly ~ 160 ms for this single long prompt. In practice TTFT for a 2K-token prompt on a 13B model lands in the 100-200 ms range on a single H100, consistent with being compute-bound at reasonable MFU.

KV cache footprint after prefill.

$$\text{KV cache} = 2 \times L \times N \times H \times S \times 2 \text{ bytes} = 2 \times 40 \times 40 \times 128 \times 2048 \times 2 \approx 1.7 \text{ GB}$$

This 1.7 GB is allocated at the end of prefill and held until the request completes.

What You Actually Decide in Practice

- **Prefill batch size governs TTFT.** Reducing B during prefill directly lowers TTFT because the compute time scales with $B \cdot S$. If your SLA is $\text{TTFT} < 200$ ms, you may need to cap concurrent prefill requests to 1-2, accepting lower GPU utilization during this phase.
- **Long prompts become the bottleneck faster than you expect.** At $S = 8,192$ tokens, prefill on a 13B model takes ~ 640 ms at $B = 1$. At $S = 32,768$ it is over 2.5 seconds. If your application has retrieval-augmented prompts or long system instructions, prefill latency will dominate your TTFT budget even before generation starts.
- **Chunked prefill allows interleaving with generation.** Some production serving systems split a long prompt into chunks (e.g., 512 tokens each) and interleave prefill chunks with ongoing generation steps for other requests. This smooths TTFT distribution at the cost of slightly higher per-request latency variance. Use it when your tail latency SLA is tight and you have concurrent requests.
- **Model parallelism affects prefill differently than generation.** Tensor parallelism across T GPUs cuts prefill time by roughly $T \times$ because you are splitting a compute-bound operation. For generation, the same parallelism provides less benefit because the bottleneck shifts to memory bandwidth.
- **Flash Attention changes the constant, not the asymptotic bound.** Using FlashAttention-2 during prefill reduces HBM traffic by fusing the attention kernel and avoiding materializing the full $S \times S$ attention matrix. It improves prefill throughput by roughly $2\text{-}3 \times$ on long sequences, but the operation remains compute-bound; it does not change the fundamental scaling behavior.

How This Shows Up in Interviews

- **“A user sends a 10,000-token system prompt. Your TTFT is 4 seconds. How do you fix it?”** A strong answer covers: (1) chunked prefill to interleave with ongoing decode; (2) prompt caching-if multiple requests share the same system prompt prefix, compute the KV cache once and reuse it; (3) reducing tensor parallelism overhead by verifying all-reduce latency is not dominating; (4) checking GPU utilization to confirm you are actually compute-bound.
- **“Why is prefill compute-bound but generation memory-bound?”** Strong answer: during prefill you process S tokens simultaneously, so weight reads are amortized over S compute ops per weight element. During generation you process one token at a time, so each weight matrix is read from HBM to perform only a matrix-vector multiply-one compute op per weight element loaded.
- **Senior follow-up: “Suppose you disaggregate prefill and decode onto separate GPU pools. What are the implications?”** The senior answer covers: prefill-optimized hardware can favor high-FLOP-count GPUs (H100 SXM); decode-optimized hardware benefits more from high memory bandwidth per GPU (or more GPUs per parameter shard). Disaggregation also lets you scale each pool independently based on prompt-length distribution and output-length distribution, and eliminates the head-of-line blocking where a long prefill stalls ongoing decode steps.

Key Takeaways

- Prefill processes all prompt tokens in parallel, making it a matrix-matrix multiply with arithmetic intensity $\propto B \cdot S$ -compute-bound on modern accelerators for any reasonable prompt length.
- The output of prefill is the populated KV cache; every generation step that follows reads from this cache, so prefill is a one-time amortized cost.
- TTFT scales linearly with prompt length and batch size. A $4\times$ longer prompt means $\sim 4\times$ longer TTFT.
- Prompt caching (reusing KV tensors across requests sharing a common prefix) can eliminate prefill cost entirely for repeated system prompts-a high-leverage optimization in production.
- Disaggregating prefill and decode onto separate GPU pools lets each be tuned for its regime: compute-intensive vs. memory-bandwidth-intensive.

15.1.2 Memory-Bound Generation and the KV Cache

Generation is the phase that breaks most engineers’ intuition the first time they benchmark an LLM in production. You have a massively parallel GPU with hundreds of tensor cores and 3.3 TB/s of memory bandwidth, and yet it is sitting at 5% compute utilization while producing tokens one by one at 80 milliseconds each. The culprit is arithmetic intensity: generating a single token is a sequence of matrix-vector multiplies, and matrix-vector multiplies are almost always memory-bound on modern hardware. This bottleneck is the central fact around which every inference optimization-larger batches, grouped-query attention, quantization, speculative decoding-is organized. Understanding it at the arithmetic level is what separates a surface-level interview answer from a strong one.

After prefill, the model enters the *autoregressive generation loop*: at each step it takes one new token embedding, runs it through all L transformer layers, and samples the next token. Crucially, only one new token is processed per step. The attention layers need to attend over *all* prior tokens, but the key and value projections for those prior tokens were already computed during prefill and cached in HBM-the *KV cache*.

The KV cache for a single sequence stores, for every layer, the key and value tensors for every token processed so far. Its total size is:

$$\text{KV cache size} = 2 \times L \times N_{\text{kv}} \times H \times S_{\text{current}} \times \text{bytes per element}$$

where L is the number of layers, N_{kv} is the number of key-value heads, H is the head dimension, and S_{current} grows by one at each step.

Now consider what the GPU must do to produce one token. For each MLP layer it reads a weight matrix $W \in \mathbb{R}^{d \times f}$ from HBM to multiply against a single activation vector $x \in \mathbb{R}^d$. The number of FLOPs is $2df$ and the bytes transferred are $2df$ (for the weight, in bf16) plus the tiny contribution from the vector itself. The arithmetic intensity is:

$$\text{Intensity}_{\text{MLP, generation}} = \frac{2Bdf}{2df + \text{small}} \approx B$$

For attention, each generation step must also read back the full KV cache for all prior tokens (S entries) for each of the N_{kv} heads. Working through the FLOP and byte counts, the intensity simplifies to:

$$\text{Intensity}_{\text{attn, generation}} = \frac{S \cdot 1}{S + 1} \approx 1$$

regardless of batch size. The H100's roofline crossover is ~ 295 FLOPs/byte. An intensity of 1 means you are using less than 0.4% of the available compute; you are almost entirely bottlenecked by the speed of HBM reads.

Generation is memory-bound because producing each token reduces the transformer to a series of matrix-vector multiplies and a KV cache read, both of which transfer far more bytes per FLOP than the hardware can hide.

The KV cache is still the right trade-off despite introducing the memory bottleneck. Without it, each generation step would recompute the key and value projections for all S prior tokens from scratch, costing $O(S)$ attention FLOPs per step. With the cache, attention per step costs $O(S)$ bytes read (to load the cache) plus $O(S)$ FLOPs for the dot products—but the prefill amortized the projection cost. You are trading computation for memory bandwidth, and for autoregressive generation this trade is always favorable: compute is the constrained resource during prefill, memory bandwidth is the constraint during generation, and the cache lets each resource be the binding constraint during its respective phase.

The MLP layers behave differently from attention in an important way. Their arithmetic intensity is $\approx B$ (the batch size), not 1. This means that by processing B requests simultaneously, you amortize the cost of loading each weight matrix over B activation vectors—you get B useful FLOPs per byte loaded. At $B = 295$ on an H100 you cross the roofline and the MLP becomes compute-bound. Attention, however, gives you no such relief: because each sequence has its own KV cache, batching does not reduce the bytes loaded per sequence. The KV cache bytes and FLOPs both scale linearly with B , so the intensity stays pinned near 1 no matter how large the batch.

This asymmetry has a practical consequence that trips up engineers who only look at MLP utilization: even if your batch size is large enough to saturate the MLP layers, the attention layers remain memory-bound throughout generation. The attention layers are therefore the harder constraint to escape.

A Worked Example

Take Llama-2 13B on a single H100 SXM. The model has 13.0B parameters stored in bf16, occupying ≈ 26 GB. The H100's memory bandwidth is 3.35 TB/s.

Memory-limited latency per generation step. At $B = 1$, the dominant memory transaction per step is loading the model weights (26 GB) plus reading the KV cache for the current sequence length S . For $S = 1,000$

the KV cache is:

$$2 \times 40 \times 40 \times 128 \times 1000 \times 2 \approx 819 \text{ MB}$$

Total bytes to transfer: $26 \text{ GB} + 0.82 \text{ GB} \approx 26.8 \text{ GB}$. At 3.35 TB/s :

$$\text{Latency} \approx \frac{26.8 \times 10^9}{3.35 \times 10^{12}} \approx 8 \text{ ms per token}$$

That gives a theoretical throughput of $1 / 0.008 = 125 \text{ tokens/second}$ at $B = 1$.

Scaling to larger batches. At $B = 16$, the KV cache grows to $16 \times 0.82 \approx 13 \text{ GB}$, but the weights are still 26 GB and are loaded once per step and shared across all 16 sequences. Total bytes: $26 + 13 = 39 \text{ GB}$. Latency: $\approx 12 \text{ ms per step}$. But you generate 16 tokens in that step, so throughput is $16 / 0.012 \approx 1,330 \text{ tokens/second}$ -a $10.6\times$ throughput gain from a $16\times$ batch increase, reflecting diminishing returns as the KV cache begins to compete with the weights for bandwidth.

At $B = 256$ the KV cache alone reaches $256 \times 0.82 \approx 210 \text{ GB}$, which overflows H100 VRAM (80 GB). The maximum batch size that fits in a single H100 is therefore roughly $B = 80 \text{ GB total} - 26 \text{ GB weights} = 54 \text{ GB available for KV} \Rightarrow B \approx 66$ for $S = 1,000$.

The attention intensity, concretely. For a single generation step at $B = 64$, $S = 1000$:

$$\text{Attn FLOPs} = B \times S \times 1 \times d = 64 \times 1000 \times 5120 \approx 3.3 \times 10^8$$

$$\text{Attn bytes} = B \times S \times N_{\text{kv}} \times H \times 2 = 64 \times 1000 \times 40 \times 128 \times 2 \approx 6.6 \times 10^8$$

Intensity ≈ 0.5 . Half a FLOP per byte: firmly in the memory-bandwidth regime, regardless of how many sequences are batched.

What You Actually Decide in Practice

- **Batch size is your primary throughput knob, up to the memory wall.** Every doubling of B roughly doubles throughput until KV cache size starts to dominate the memory footprint. For a 13B model on an H100, this ceiling is around $B = 50\text{-}70$ for sequences of 1K tokens. Beyond that, you need to reduce sequence length, switch to GQA/MQA, or quantize the KV cache.
- **KV cache quantization gives you headroom before adding GPUs.** Quantizing KV cache entries from bf16 to int8 halves KV memory, doubling the effective maximum batch size. At $S = 1,000$ this moves the memory ceiling from $B \approx 65$ to $B \approx 130$ on a single H100 with 80 GB VRAM -a significant lever before you pay for more hardware.
- **Grouped-query attention (GQA) is the most impactful architectural lever.** Reducing from N KV heads to N/G KV heads shrinks the KV cache by a factor of G , with minimal accuracy impact at $G \leq 8$. This is why Llama-3 adopted GQA across all model sizes: the same weights serve more concurrent requests on the same hardware.
- **At $B = 1$, generation is pure memory bandwidth.** If you are running a low-concurrency service (a single-user IDE assistant, for example), your generation speed is almost exactly $\text{parameter_bytes} / \text{memory_bandwidth_bytes_per_second}$ per token. You are not compute-limited; getting a faster GPU does not help unless its bandwidth is higher.
- **Latency and throughput trade off directly through batch size.** At $B = 1$ on Llama-2 13B / H100: latency $\approx 8 \text{ ms/token}$, throughput $\approx 125 \text{ tok/s}$. At $B = 32$: latency $\approx 10 \text{ ms/token}$, throughput $\approx 3,200 \text{ tok/s}$. You cannot get both low latency and high throughput from a single serving instance; use request routing or disaggregated serving if your SLAs require both.

How This Shows Up in Interviews

- **“Why is LLM generation slow? What’s the bottleneck?”** Strong answer: generation is memory-bound, not compute-bound. Each step is a series of matrix-vector multiplies (weight matrices times a single token embedding) and a KV cache read. The arithmetic intensity for both operations is $O(1)$ - $O(B)$, far below the H100’s roofline of 295 FLOPs/byte. The GPU is bottlenecked by how fast it can stream model weights and KV cache from HBM, not by how fast its tensor cores can multiply.
- **“How does increasing batch size help, and when does it stop helping?”** Strong answer: larger B amortizes the cost of loading weight matrices over more activation vectors, raising arithmetic intensity from ~ 1 toward the roofline for MLP layers. But attention layers give no benefit from batching because each sequence’s KV cache is independent-intensity stays near 1. The hard ceiling is GPU VRAM: once $B \times \text{KV cache per sequence} + \text{weights}$ exceeds VRAM, the batch does not fit.
- **Senior follow-up: “The attention arithmetic intensity is always ≈ 1 no matter how large the batch. Is there any architectural change that can actually fix this?”** Senior answer: yes, two approaches. First, reduce KV cache size so fewer bytes are transferred per step: GQA reduces KV heads, MLA projects KV to a lower-dimensional latent space, and local (sliding-window) attention discards cache entries beyond a window. Second, eliminate autoregression altogether for generation: diffusion-based language models generate all tokens in parallel and iterate, turning the problem into a compute-bound one and yielding 5-10 $\times$ throughput improvements at the cost of more complex sampling dynamics.

Key Takeaways

- Generation reduces to matrix-vector multiplies and KV cache reads; both have arithmetic intensity $\leq B$, putting generation firmly in the memory-bandwidth-limited regime on all current hardware.
- Attention is *worse* than MLP during generation: its intensity is ≈ 1 regardless of batch size because each sequence owns its own KV cache, eliminating the weight-amortization benefit of batching.
- The practical throughput ceiling for a single GPU is set by VRAM capacity: batch size $\leq (\text{VRAM} - \text{weights}) / \text{KV cache per sequence}$.
- Every major inference optimization-GQA, MLA, KV quantization, speculative decoding-targets this memory bottleneck either by reducing bytes transferred or by restructuring the problem to increase FLOPs per byte.
- Latency and throughput are in direct tension: minimum-latency serving uses $B = 1$ and loads weights once per request; maximum-throughput serving packs the largest B that fits in memory.

15.2 Latency, Throughput, and Cost

15.2.1 Time-To-First-Token and Inter-Token Latency

When you are designing a serving system in an interview, the first question is almost always “what are your latency targets?” The follow-up-the one that separates candidates who have shipped inference from those who have only read about it-is “which latency?” Time-To-First-Token (TTFT) and inter-token latency (ITL) are governed by fundamentally different physics, and conflating them leads to the wrong optimization strategy at every level of the stack. This distinction comes up at mid-level and above for any role touching production inference.

Time-To-First-Token is how long a user waits between submitting a request and seeing the first generated character. It is dominated by the *prefill* phase: encoding all S prompt tokens through all L transformer layers in one parallel forward pass. This is structurally identical to a training forward pass—all tokens are available simultaneously, so the computation parallelizes across the sequence. Prefill is compute-bound when $B \cdot T$ is large enough to saturate the accelerator.

Inter-token latency (also called *decode latency* or *generation latency*) is the time between consecutive output tokens after the first. Each step produces exactly one new token by loading the full model weights from HBM, performing a matrix-vector product against the current hidden state, and attending over the growing KV cache. The arithmetic intensity of this operation is ≈ 1 FLOP/byte (one for attention, B for MLP—see the previous subsection), so the step is memory-bandwidth-bound regardless of GPU generation.

The governing equation for per-token generation latency is:

$$\text{ITL} \approx \frac{\text{memory footprint}}{\text{HBM bandwidth}} = \frac{2P + B \cdot C_{\text{kv}}}{\text{BW}}$$

where P is the parameter count, C_{kv} is the KV cache size per sequence (bytes), B is the number of concurrent sequences, and BW is the peak memory bandwidth of the accelerator. The factor of 2 on P reflects bf16 storage (2 bytes per parameter).

TTFT and ITL are governed by different physics—the former by compute throughput, the latter by memory bandwidth—and any design that treats them as a single “latency” number will optimize for the wrong thing.

Prefill is fast because the transformer can process all S tokens in a single batched matrix multiply. The arithmetic intensity during prefill scales as $B \cdot S$: a batch of one request with a 4096-token prompt has intensity ≈ 4096 , comfortably above the H100’s roofline of ≈ 295 FLOPs/byte. TTFT is therefore roughly proportional to S and inversely proportional to sustained compute throughput—something you cannot improve without either reducing the prompt or adding accelerators.

Generation is slow because each autoregressive step is equivalent to a matrix-vector product: you are loading the full weight matrix $W \in \mathbb{R}^{D \times D}$ to compute Wx for a single vector $x \in \mathbb{R}^D$. The ratio of FLOPs to bytes is $2D^2/(2D^2) = 1$, essentially 1 FLOP per byte transferred. Modern H100s achieve ≈ 989 TFLOP/s and ≈ 3.35 TB/s bandwidth, so the compute-to-bandwidth ratio is $989/3.35 \approx 295$. A batch size of 1 operates at intensity $1/295\times$ below what it would need to saturate compute. Without a KV cache the situation would be even worse: each generation step would re-encode the entire prefix at $O(S)$ cost, giving $O(S^2)$ total work. The KV cache trades memory for time, converting $O(S^2)$ generation to $O(S)$ —a fundamental architectural choice that determines every other inference optimization.

A Worked Example

Consider Llama-2 13B on a single H100 SXM5 (80 GB HBM, 3.35 TB/s bandwidth). Model configuration: $D = 5120$, $L = 40$, $N_{\text{heads}} = 40$, $N_{\text{kv}} = 40$, head dimension $H = 128$, sequence length $S = 1024$.

Model weight memory:

$$M_{\text{weights}} = 2 \times 13 \times 10^9 \text{ bytes} \approx 26 \text{ GB}$$

KV cache per sequence at $S = 1024$:

$$C_{\text{kv}} = 2 \times L \times N_{\text{kv}} \times H \times S \times 2 \text{ bytes} = 2 \times 40 \times 40 \times 128 \times 1024 \times 2 \approx 838 \text{ MB}$$

(The first factor of 2 is for keys and values; the last 2 is for bf16.)

Per-token ITL at $B = 1$:

$$\text{ITL} = \frac{26.0 + 0.84}{3350} \text{ s} \approx 8.0 \text{ ms/token} \quad (\approx 125 \text{ tok/s})$$

TTFT at $S = 1024$, $B = 1$ (compute-bound prefill):

$$\text{FLOPs}_{\text{prefill}} \approx 2 \times P \times S \approx 2 \times 13 \times 10^9 \times 1024 \approx 26.6 \text{ TFLOPs}$$

$$\text{TTFT} \approx \frac{26.6}{989} \approx 27 \text{ ms}$$

A user sending a 1024-token prompt waits ≈ 27 ms before the first token appears, then receives tokens every ≈ 8 ms. Doubling the prompt to 2048 tokens roughly doubles TTFT to ≈ 54 ms. ITL stays constant because it depends on model size, not prompt length. At 100k-token prompts-common in document-analysis workloads-TTFT climbs to ≈ 2.7 s even before a single output token is generated.

What You Actually Decide in Practice

- **Set separate SLOs for TTFT and ITL.** A latency budget of “ < 200 ms” is ambiguous. Reasonable starting points for interactive chat: TTFT < 500 ms, ITL < 50 ms. Batch pipelines can relax TTFT to seconds while holding ITL tight for tail latency.
- **Quantize weights to cut ITL directly.** Switching from bf16 to INT8 halves M_{weights} , cutting ITL roughly in half at no change to model architecture. INT4 cuts it to $\approx 25\%$ of bf16. For a 13B model: INT8 brings ITL from 8 ms to ≈ 4 ms. This is the cheapest ITL reduction available before touching architecture.
- **TTFT grows linearly with prompt length; prune prompts aggressively.** Every additional 1000 prompt tokens adds $\approx 2 \times 13\text{B} \times 1000/989\text{T} \approx 26$ ms to TTFT on a 13B model. Summarization or truncation strategies that halve average prompt length halve average TTFT.
- **Chunked prefill trades TTFT for system-level ITL.** When a long prompt arrives, splitting it into fixed-size chunks processed across multiple steps allows the GPU to interleave prefill with generation steps from other in-flight requests. The chunked request’s TTFT increases, but queued requests no longer stall behind a multi-second prefill job. Use this when p99 ITL matters more than p50 TTFT.
- **Multi-GPU tensor parallelism reduces TTFT, not ITL, proportionally.** Splitting the model across K GPUs splits the prefill FLOPs by K , cutting TTFT roughly K -fold. ITL also improves because M_{weights} per GPU drops, but you now pay inter-GPU communication latency on every generation step-a non-trivial overhead on large K . For latency-sensitive serving, prefer $K = 2$ or $K = 4$; beyond that, communication dominates.

How This Shows Up in Interviews

- **“What’s the difference between TTFT and inter-token latency, and which matters more for a streaming chat product?”** Strong answer: TTFT is prefill time (compute-bound, scales with prompt length and model size); ITL is per-token generation time (memory-bandwidth-bound, scales with model weight bytes and batch KV cache size). For streaming chat, ITL dominates user experience: a 200 ms TTFT is invisible, but 100 ms ITL makes text feel sluggish. Optimize ITL via quantization and KV cache reduction; optimize TTFT via multi-GPU prefill or prompt pruning.
- **“You’re serving a 70B model and a user sends a 50k-token prompt. Walk me through what happens to TTFT.”** Strong answer: TTFT scales as $2P \cdot S/\text{compute}$. At 50k tokens on a single H100: $2 \times 70\text{B} \times 50000/989\text{T} \approx 7$ s. This is unacceptable for interactive use. Solutions: distribute prefill across multiple GPUs (pipeline or tensor parallel), use chunked prefill with a short-context draft first, or compress the prompt before sending.

- **Senior follow-up: “How does chunked prefill interact with the KV cache, and what does it do to tail latency for other requests?”** Senior answer: Chunked prefill processes the prompt in $\lceil S/C \rceil$ chunks of size C tokens each. Between chunks, the scheduler can insert generation steps for other active requests. The KV cache for the chunked request grows incrementally—each chunk appends to it—so memory usage is identical to non-chunked prefill at completion. The benefit is that long-prompt requests no longer monopolize the GPU for seconds at a time, dramatically reducing p99 ITL for the rest of the request queue. The cost is a $O(S^2/C)$ increase in total prefill FLOPs due to partial attention recomputation across chunk boundaries—tunable by choice of C .

Key Takeaways

- TTFT is dominated by prefill: compute-bound, scales linearly with prompt length S and model size P .
- ITL is dominated by memory bandwidth: $\text{ITL} \approx (2P + B \cdot C_{\text{kv}})/\text{BW}$; for a 13B bf16 model at $B = 1$ on H100, expect ≈ 8 ms/token.
- The two metrics are decoupled: halving model weights halves ITL but does not change TTFT; halving prompt length halves TTFT but does not change ITL.
- Quantization (bf16 $\rightarrow$ INT8) is the highest-leverage, lowest-risk way to reduce ITL; it applies immediately to any trained model.
- Always set separate SLOs; a single “latency” target will cause you to over-optimize one metric at the expense of the other.

15.2.2 Throughput Trade-Offs Under Batching

Every LLM serving team hits the same wall within weeks of launch: per-request latency looks fine at one query per second, but the GPU sits at 30% utilization and cost-per-token is an order of magnitude higher than projected. The answer is batching—processing multiple requests simultaneously to amortize model weight loads across more useful work. But batching has its own ceiling, and the math behind that ceiling explains both why your throughput stalls and what architectural choices (GQA, quantization, replica scaling) give you the biggest gains. This trade-off is a core topic at senior and staff levels wherever inference cost matters.

Throughput is the total number of tokens generated per second across all users, in contrast to per-user latency which measures a single request’s experience. During the memory-bound generation phase, the system performs one decode step per token, and the cost of that step is:

$$\text{step_time}(B) = \frac{M_{\text{weights}} + B \cdot C_{\text{kv}}}{\text{BW}}$$

where B is the number of concurrent sequences, C_{kv} is the KV cache footprint per sequence, and BW is HBM bandwidth. Each step produces B tokens, so throughput is:

$$\text{throughput}(B) = \frac{B}{\text{step_time}(B)} = \frac{B \cdot \text{BW}}{M_{\text{weights}} + B \cdot C_{\text{kv}}}$$

As $B \rightarrow \infty$ (ignoring memory capacity), throughput approaches BW/C_{kv} —a hard ceiling set entirely by KV cache size per sequence. At the other extreme, $B = 1$ gives $\text{throughput} = \text{BW}/(M_{\text{weights}} + C_{\text{kv}})$, which is dominated by weight memory. The crossover batch size B^* at which KV cache equals model weight memory is:

$$B^* = \frac{M_{\text{weights}}}{C_{\text{kv}}} = \frac{2P}{2 \cdot L \cdot N_{\text{kv}} \cdot H \cdot S \cdot 2}$$

Below B^* , increasing B improves throughput rapidly because the fixed weight cost is amortized over more

tokens. Above B^* , KV cache dominates and throughput gains diminish quickly.

The KV-crossover batch size B^ is the most important number in your serving capacity model-it tells you exactly how much batching buys you and where to stop.*

At $B = 1$, loading 26 GB of weights produces one token: you get 1 token per weight-load. At $B = 32$, the same weight load is shared across 32 decode steps happening in parallel, amortizing the dominant cost over 32 useful outputs. This is the same principle as batched matrix multiplication: the B dimension in XW ($X \in \mathbb{R}^{B \times D}$, $W \in \mathbb{R}^{D \times F}$) lets each byte of W contribute to B outputs rather than one.

Contrast this with the obvious alternative of simply running B independent single-batch requests serially: you load weights B times, burn $B \times$ the memory bandwidth, and produce the same B tokens. Batching collapses B weight loads into one.

The cost is per-request latency: if B requests are batched together, the slowest one determines when all of them complete their current step. This creates the fundamental throughput-latency trade-off: more batching $\Rightarrow$ higher throughput, higher tail latency. Replica parallelism-launching M independent copies of the model, each handling its own requests at a lower batch size-sidesteps this trade-off entirely, delivering $M \times$ throughput at unchanged latency. Replicas require no inter-GPU communication and are the first scaling lever you should reach for.

A Worked Example

Llama-2 13B on a single H100 (80 GB HBM, 3.35 TB/s bandwidth, $S = 1024$):

- $M_{\text{weights}} = 26 \text{ GB (bf16)}$
- $C_{\text{kv}} = 0.84 \text{ GB per sequence (from Section 15.2.1)}$

KV-crossover batch size:

$$B^* = \frac{26}{0.84} \approx 31$$

Above $B = 31$, KV cache dominates.

Throughput at various batch sizes:

B	Total memory (GB)	Step time (ms)	Throughput (tok/s)	Fits H100?
1	26.8	8.0	125	yes
16	39.4	11.8	1,356	yes
32	52.9	15.8	2,025	yes
64	79.8	23.8	2,689	yes (barely)
128	133.5	39.8	3,215	no ($> 80 \text{ GB}$)

At $B = 32 \approx B^*$, throughput is $\approx 16 \times$ higher than $B = 1$ but step time has only doubled. At $B = 64$ (above B^*), the incremental throughput gain from doubling B from 32 is only $1.3 \times$ -diminishing returns have clearly set in. $B = 128$ does not fit in 80 GB HBM without reducing KV cache size via GQA or quantization.

Effect of GQA (Llama-3 style, $N_{\text{kv}} = 8$ instead of 40):

$$C_{\text{kv}}^{\text{GQA}} = 0.84 \times \frac{8}{40} = 0.168 \text{ GB}$$

$$B^{*,\text{GQA}} = \frac{26}{0.168} \approx 155$$

GQA shifts the crossover point from 31 to 155, meaning you can fit $B = 128$ in $26 + 128 \times 0.168 \approx 47 \text{ GB}$ -comfortably inside the 80 GB limit-at nearly $5 \times$ higher throughput than the original $B = 64$ cap.

What You Actually Decide in Practice

- **Compute B^* before setting any serving configuration.** $B^* = M_{\text{weights}}/C_{\text{kv}}$. Throughput gains below B^* are superlinear; above B^* they are sublinear. For Llama-2 13B at $S = 1024$, $B^* \approx 31$. For the same model at $S = 4096$, C_{kv} quadruples and B^* drops to ≈ 8 —meaning longer contexts hurt your batching efficiency dramatically.
- **Use replica parallelism before increasing batch size.** M replicas give exactly $M\times$ throughput with identical per-request latency and zero communication overhead. If your cost budget allows three H100s and latency is acceptable at $B = 1$, three replicas at $B = 1$ beats one replica at $B = 3$ in both latency and simplicity. Increase B only when you need to serve traffic that exceeds what replicas alone can handle.
- **Use continuous (dynamic) batching.** Static batch sizes waste cycles: if one of B requests finishes early, its slot sits idle until all B requests complete. Continuous batching allows the scheduler to insert a new request into a finished slot at the next decode step without waiting for a full batch to drain. This is table stakes in any production serving stack (vLLM, TGI, TensorRT-LLM all implement it).
- **GQA is the highest-leverage architectural change for throughput.** Reducing N_{kv} from 40 to 8 (a $5\times$ reduction) raises B^* by $5\times$ and expands max viable B by $5\times$, yielding 2-4 $\times$ real-world throughput improvement without changing model quality meaningfully. If you are choosing a base model for a high-traffic product, GQA support is a first-order selection criterion.
- **Cost per token scales inversely with throughput.** On an H100 at \$4/hr and throughput of 2,689 tok/s ($B = 64$), cost is $4/3600/2689 \approx \$0.0004/\text{1k tokens}$. At $B = 1$ (125 tok/s), cost is $\approx \$0.009/\text{1k tokens}$ —22 $\times$ higher. Maximizing batch size within the HBM limit is by far the largest cost lever before model changes.

How This Shows Up in Interviews

- **“How do you maximize throughput for a high-traffic LLM serving endpoint?”** Strong answer: Profile memory first. Compute $B^* = M_{\text{weights}}/C_{\text{kv}}$ and the maximum B that fits in HBM. Use continuous batching to approach that maximum. Apply GQA or INT8 quantization to raise both B^* and max B . Scale replicas horizontally for the remaining multiplier. Every step here has a concrete number attached; never hand-wave about “utilizing the GPU better.”
- **“A user reports high tail latency at peak traffic. Walk me through root-causing it.”** Strong answer: The most common cause at scale is batch size growth: as QPS rises, the scheduler packs more requests per step to maintain throughput, increasing per-step latency and pushing tail requests to wait longer. Confirm via step-time histogram. Fix options in priority order: (1) add replicas to spread load, reducing effective B per replica; (2) implement preemption to cap max B ; (3) implement priority queues so latency-sensitive requests get smaller batches.
- **Senior follow-up: “Walk through the full memory budget for serving Llama-2 70B at $B = 32$, $S = 2048$, on two H100s with tensor parallelism.”** Senior answer: Model weights (bf16): $2 \times 70 \times 10^9 = 140$ GB, split across two GPUs = 70 GB each. KV cache: $2 \times 80 \times 64 \times 128 \times 2048 \times 2 \times 32 =$ roughly ≈ 68 GB total (not sharded in basic tensor parallelism). Total per GPU: $70 + 34 = 104$ GB—exceeds the 80 GB limit. Solutions: shard KV cache across GPUs alongside weights (sequence parallelism), reduce B to ≈ 14 , or use GQA to shrink C_{kv} . This is the kind of capacity calculation you should be able to do on a whiteboard in two minutes.

Key Takeaways

- Throughput scales as $B \cdot \text{BW} / (M_{\text{weights}} + B \cdot C_{\text{kv}})$; gains are superlinear below the KV-crossover $B^* = M_{\text{weights}} / C_{\text{kv}}$ and sublinear above it.
- For Llama-2 13B (bf16) on H100 at $S = 1024$: $B^* \approx 31$, $\max B \approx 64$ (memory-limited), peak throughput $\approx 2,700 \text{ tok/s}$ - $22\times$ higher than $B = 1$.
- Replica parallelism scales throughput linearly at zero latency cost and zero communication overhead; exhaust replicas before increasing batch size.
- GQA reduces C_{kv} proportionally to the head reduction ratio, raising B^* and $\max B$ by the same factor.
- Cost per token is inversely proportional to throughput; maximizing B within HBM limits is the largest cost lever available without architectural changes.

Chapter 16 Reducing the KV Cache

16.1 Attention Variants for Cheaper Inference

16.1.1 Multi-Query and Grouped-Query Attention (GQA)

When an interviewer asks you to design a serving stack for a 70B model and your napkin math shows that the KV cache will overflow VRAM at any reasonable batch size, *grouped-query attention* is the first architectural lever to reach for. GQA is the change that moved from research curiosity to default in every serious open-weight model—Llama 3, Mistral, Qwen, Falcon—within about a year of its introduction, and interviewers at inference-heavy teams will expect you to know it cold and be able to quantify its impact.

Standard *multi-head attention* (MHA) computes N query heads, N key heads, and N value heads per token per layer. The KV cache therefore stores $2 \times N \times H$ values per token per layer, where H is the head dimension. Because every sequence in a batch owns its own independent KV cache, this cost scales linearly with batch size, sequence length, and layer count, and it is the primary reason generation remains memory-bound regardless of how many queries you batch together.

Multi-query attention (MQA) pushed this to the extreme: share a single key head and a single value head across all N query heads. The KV cache shrinks by a factor of N , but in practice MQA degrades model quality noticeably, especially at larger scales—the single shared KV head is not expressive enough.

Grouped-query attention (GQA) occupies the Pareto-optimal middle ground. You partition the N query heads into G groups of N/G heads each, and each group shares one key head and one value head. Defining $K = N/G$ as the number of KV heads:

$$\text{KV cache size per token per layer} = 2 \times K \times H \times \text{bytes per element}$$

Setting $K = N$ recovers MHA; setting $K = 1$ recovers MQA. In practice $G = 8$ (i.e., $K = N/8$) preserves accuracy nearly identically to MHA while cutting the KV cache by $8\times$.

GQA is the lowest-cost architectural change that shrinks the KV cache without meaningful accuracy loss, enabling either lower latency at the same batch size or a larger batch at the same latency.

The accuracy robustness of GQA comes from what the key and value projections actually learn. Within a group of N/G query heads, the queries are already diverse—each attends to different positions and features. The key and value vectors, by contrast, encode the information content at each position, which is less sensitive to which specific query is asking. Sharing KV within a group turns out to be a mild constraint; the queries still specialize while the KV tensors carry shared context.

The throughput gain is two-fold. First, the direct reduction: K KV heads instead of N means $N/K = G$ times fewer bytes transferred from HBM per generation step, directly proportional to speed-up since generation is memory-bandwidth-bound. Second, an indirect gain: the freed VRAM can accommodate a larger batch size. Before GQA, a 13B model at $S = 1,000$ and $B = 64$ required over 50 GB just for the KV cache on a standard MHA model; with GQA at $G = 8$ that drops to 6 GB, leaving room to push B well above 200 on a single 80 GB H100.

The comparison point is MQA. MQA also reduces memory but pays a quality tax that compounds at larger model scales. GQA effectively shows that the quality-memory trade-off curve is non-linear: you can recover almost all of MHA's quality with only $K = N/8$ KV heads rather than $K = N$, a point on the curve that gives you most of the memory benefit for almost none of the accuracy cost.

A Worked Example

Consider Llama-2 70B: $N = 64$ query heads, $H = 128$, $L = 80$ layers, bf16 (2 bytes per element), with batch size $B = 32$ and sequence length $S = 2,048$.

MHA KV cache. Per token per layer: $2 \times 64 \times 128 \times 2 = 32,768$ bytes. Total:

$$32 \times 2048 \times 80 \times 32,768 \text{ bytes} \approx 171 \text{ GB}$$

This overflows a single 80 GB H100 even before counting the 140 GB of model weights. You cannot run this batch on one GPU.

GQA with $G = 8$ ($K = 8$). Per token per layer: $2 \times 8 \times 128 \times 2 = 4,096$ bytes. Total KV cache:

$$32 \times 2048 \times 80 \times 4,096 \text{ bytes} \approx 21.5 \text{ GB}$$

Combined with weights: $140 + 21.5 = 161.5$ GB-fits comfortably on two H100s, and a single H100 can handle $B \leq 10$ at this sequence length.

Throughput estimate at $B = 32$ with GQA, two H100s. Total bytes transferred per generation step: 161.5 GB. Aggregate bandwidth: $2 \times 3.35 = 6.7$ TB/s. Step latency: $161.5/6700 \approx 24$ ms. Throughput: $32/0.024 \approx 1,333$ tokens/second. Without GQA, this batch simply does not fit.

Accuracy check. Empirically, GQA at $G = 8$ recovers $\geq 99\%$ of MHA perplexity across standard benchmarks for models at 7B-70B scale. The quality cost is negligible for downstream tasks.

What You Actually Decide in Practice

- **Default to $G = 8$ for any model ≥ 7 B.** This is the empirically validated sweet spot. Below 7B you may still use $G = 4$ or even MQA without noticeable downstream degradation because the models are less sensitive to head expressiveness.
- **GQA changes your VRAM ceiling, not just your speed.** The more important effect is often that GQA makes a batch size *possible* that was previously impossible, not that it speeds up an already-fitting batch. Always recompute your max batch size after adopting GQA-the number often increases by $4\text{-}8\times$.
- **Weight sharing means the projection weights are shared too.** In CLA the weights are explicitly shared across layers; in GQA, the same W_K and W_V matrices produce the keys and values used by multiple query heads within a group. If you are loading a GQA checkpoint, the key-value projection matrices are smaller than query projection matrices by a factor of G .
- **GQA is free at training time, not just inference time.** Training with GQA is slightly faster than MHA because you compute fewer KV projections. Models trained from scratch with GQA converge to the same loss as MHA models; you do not need to fine-tune or distill a pre-trained MHA model into GQA.
- **For $G > 16$, check accuracy on your target task.** The gap between GQA and MHA is small for $G \leq 8$, but it widens for aggressive settings. For retrieval-heavy workloads (RAG, long-context question answering), the KV diversity may matter more than for generation tasks-run an eval before committing to extreme compression ratios.

How This Shows Up in Interviews

- **“Your KV cache is overflowing VRAM at the batch size you need. How do you fix it architecturally?”**
Strong answer: the primary lever is GQA-reduce the number of KV heads from N to N/G with $G = 8$ as the default, shrinking the KV cache by $8\times$ with negligible accuracy impact. Secondary levers include KV quantization (int8 halves the cache further) and sliding-window attention on non-critical layers.

- **“What is the difference between MQA, GQA, and MHA? When would you use each?”** Strong answer: MHA uses N KV heads (maximum expressiveness, maximum cache), MQA uses 1 KV head (minimum cache, measurable quality loss at scale), GQA uses $K = N/G$ KV heads sharing one KV pair per group of G queries (best accuracy-memory trade-off, used in all modern production models). MQA is now obsolete for large models; GQA is the default; MHA survives only in legacy architectures or small models where memory is not the bottleneck.
- **Senior follow-up: “You’re training a new 70B model from scratch. A colleague proposes using MHA because it’s simpler and GQA is a “trick.” How do you respond, and what training experiments would settle the debate?”** Senior answer: GQA is not a post-hoc trick—it is a design decision made at model initialization. Train two identical models at 7B scale (a proxy) with $G = 8$ GQA and MHA respectively, evaluate on a downstream task suite and measure KV cache size at target serving batch size. If GQA shows $< 1\%$ degradation, adopt it for the 70B run. The inference cost reduction justifies the negligible quality delta in almost every production setting.

Key Takeaways

- GQA reduces the number of KV heads from N to $K = N/G$, shrinking the KV cache by a factor of G with near-zero accuracy loss at $G \leq 8$.
- The cache reduction is doubly beneficial: fewer bytes per step means faster generation, *and* freed VRAM enables larger batch sizes that further improve throughput.
- For Llama-2 70B with $G = 8$, the KV cache at $B = 32$, $S = 2,048$ drops from ~ 171 GB (MHA, impossible on one or two H100s) to ~ 22 GB (GQA, easily fits alongside model weights).
- GQA is applied at training time, not as a post-hoc compression step; production models from Llama 3 onward are trained with GQA by default.
- MQA ($G = N$) is the degenerate case—maximum memory savings but unacceptable accuracy loss at scale; GQA is the right balance.

16.1.2 Multi-Head Latent Attention (MLA)

GQA attacks the KV cache along one axis: reduce the number of heads. *Multi-Head Latent Attention* (MLA), introduced in DeepSeek V2, attacks a different axis: project the key and value tensors into a shared low-dimensional latent before storing them, then reconstruct the full-rank KV at query time. The compression ratios MLA achieves—up to $31\times$ in DeepSeek’s configuration—dwarf what GQA alone can deliver, and MLA is the design behind some of the most inference-efficient frontier models available. Senior interviews at labs building or deploying frontier models will expect you to understand the mechanism and its implications.

In standard MHA, each token at each layer stores $2 \times N \times H$ values in the KV cache: N key vectors and N value vectors each of dimension H . GQA reduces N to $K = N/G$ but leaves each head’s dimensionality H untouched. MLA takes the complementary approach: keep all N heads but *compress their joint representation* into a single latent vector of dimension $C \ll N \times H$.

Concretely, MLA introduces a down-projection matrix $W^{DKV} \in \mathbb{R}^{d \times C}$ and an up-projection matrix $W^{UK} \in \mathbb{R}^{C \times (N \times H)}$ for keys (and an analogous pair for values). The cached quantity per token per layer is the compressed latent:

$$c_t = W^{DKV} x_t \in \mathbb{R}^C$$

At query time, the full key matrix is reconstructed on the fly:

$$K_t = c_t W^{UK} \in \mathbb{R}^{N \times H}$$

The cache therefore stores C values per token per layer instead of $2 \times N \times H$. In DeepSeek V2, $N = 128$, $H = 128$, so the uncompressed KV size is $2 \times 128 \times 128 = 32,768$ values per token. With $C = 512$, the cache stores only 512 values—a $64\times$ compression on the stored representation, though in practice a small number of extra dimensions must be retained for positional encoding (see below), bringing effective compression to roughly $31\times$.

The one structural complication is RoPE compatibility. Standard RoPE applies a position-dependent rotation to the key and query vectors before the attention dot product. With MLA, the keys are reconstructed from the latent c_t , which is position-agnostic. The solution DeepSeek adopted is to cache an additional small set of *decoupled RoPE keys* alongside the latent, keeping positional information accessible without expanding the main cache significantly.

MLA stores a low-rank compressed latent of the KV representation rather than the full KV tensors, achieving compression ratios far beyond what head-count reduction alone can provide, at the cost of up-projection compute at every query step.

The key intuition is that across all N heads in a layer, the key and value projections of a single token are not $2 \times N \times H$ independent numbers—they share substantial structure because they all derive from the same token embedding $x_t \in \mathbb{R}^d$. A rank- C projection can capture most of this structure when C is chosen appropriately relative to the intrinsic dimensionality of the token-space representation.

The trade-off MLA makes is compute for memory. Every query step now incurs an additional matrix multiply to up-project the cached latent back to full KV- $C \times (N \times H)$ FLOPs per head per token in the KV cache. Since generation is memory-bandwidth-bound, spending a modest amount of additional compute is a favorable trade if it dramatically reduces the bytes transferred. Whether the up-projection cost is worth it depends on the compression ratio $C/(2NH)$: at DeepSeek’s $31\times$ compression, the saved bandwidth far outweighs the extra multiply.

Compared to GQA, MLA offers much higher compression but is substantially harder to implement. GQA requires only reshaping the KV tensors during the attention kernel; MLA requires custom kernel support for the up-projection step inside the attention computation, and the RoPE decoupling adds further complexity. This is why GQA has seen broad adoption across many model families while MLA has so far stayed primarily within the DeepSeek ecosystem.

A Worked Example

DeepSeek V2 uses 60 transformer layers, $N = 128$ query heads, $H = 128$, and $C = 512$ for the compressed KV latent, plus $d_{rope} = 64$ extra dimensions for decoupled RoPE keys, giving a total cached size of $512 + 64 = 576$ values per token per layer.

KV cache comparison at $B = 1$, $S = 128,000$ (128K context).

Full MHA: $2 \times 128 \times 128 \times 128,000 \times 60 \times 2 \text{ bytes} = 503 \text{ GB}$. Infeasible on any single machine for interactive serving.

MLA: $576 \times 128,000 \times 60 \times 2 \text{ bytes} \approx 8.8 \text{ GB}$. Fits on a single 80 GB H100 alongside model weights.

Compression ratio: $32,768/576 \approx 56.9\times$ in theory, $\approx 31\times$ accounting for the RoPE overhead and practical implementation details.

Up-projection cost per generation step. At each step, for each of the $S = 128,000$ cached tokens, you

perform one up-projection: $C \rightarrow N \times H$ for keys and an analogous one for values. FLOPs per layer per step:

$$2 \times S \times C \times (N \times H) = 2 \times 128,000 \times 512 \times 16,384 \approx 2.1 \times 10^{12}$$

At H100's 989 TFLOP/s: roughly 2 ms per layer per step. Summed over 60 layers this is significant—roughly 120 ms of extra FLOPs per token at full 128K context. In practice this is fused with the attention kernel and partially hidden behind memory latency, so the effective overhead is lower. The bandwidth savings (from reading 576 values instead of 32,768 per cached token) dominate at long contexts.

Breakeven context length. MLA only wins when the bandwidth savings exceed the extra compute cost. For DeepSeek's numbers, this breakeven is roughly $S \approx 4,000$ tokens—well within the range of practical long-context serving. Below 4K context, the overhead of up-projection matters more and MLA may not outperform GQA.

What You Actually Decide in Practice

- **MLA is the right choice when targeting very long contexts ($S \geq 8,000$) and bandwidth is the binding constraint.** The bandwidth reduction scales with S while the up-projection cost scales identically—so MLA's advantage grows with context length. For short-context workloads, GQA is simpler and competitive.
- **The up-projection kernels must be custom.** Standard attention kernels (FlashAttention, cuDNN) do not natively support MLA's expand-then-attend pattern. Deploying MLA requires either a custom CUDA kernel or a serving framework that has integrated MLA support. Factor this into your build vs. buy decision when selecting a serving stack.
- **Latent dimension C is a hyperparameter with a clear sensitivity.** In DeepSeek's ablations, reducing C below 256 caused measurable perplexity increases; $C = 512$ maintained accuracy near the MHA baseline. Use $C \approx d/8$ as a starting point, where d is the model dimension.
- **RoPE decoupling adds a fixed overhead.** The extra $d_{rope} = 64$ dimensions stored per token per layer are unavoidable if you want full positional expressiveness. Plan for this in your KV cache sizing—at long contexts it is a small fraction of the latent size but worth accounting for in exact memory calculations.
- **GQA and MLA are orthogonal in principle but rarely combined in practice.** GQA reduces head count; MLA reduces per-head dimensionality. A model could use both simultaneously, but the engineering complexity is additive. In current deployments, teams pick one: GQA for simplicity and broad ecosystem support, MLA for maximum compression at long context.

How This Shows Up in Interviews

- **“Explain how MLA differs from GQA as a strategy for reducing the KV cache.”** Strong answer: GQA reduces the number of KV head pairs from N to $K = N/G$, cutting cache size by $G\times$. MLA keeps all N heads but projects their joint representation into a C -dimensional latent before caching, cutting size by $2NH/C \approx 31\times$ in DeepSeek's configuration. The trade-off MLA makes is additional up-projection compute at query time in exchange for dramatically lower HBM traffic.
- **“What is the RoPE compatibility problem with MLA and how is it solved?”** Strong answer: standard RoPE applies position-dependent rotations to key vectors *before* caching. In MLA the cache stores position-agnostic latents, so RoPE cannot be pre-applied. DeepSeek's solution is to cache a small set of decoupled RoPE keys alongside the latent ($d_{rope} = 64$ extra dimensions), preserving positional information without expanding the main cache significantly.

- **Senior follow-up: “You are designing a new model targeting 128K context serving at scale. Walk through the KV cache architecture decision from first principles.”** Senior answer: start with the memory budget (e.g., one 80 GB H100, 30 GB for weights, 50 GB for KV at peak batch). Compute the per-token KV footprint for MHA, GQA, and MLA at your target N and H . For 128K context even GQA may overflow; MLA at $C = 512$ shrinks this to sub-10 GB territory. Weigh the custom kernel cost against the serving economics—if you are running hundreds of instances, the engineering investment in MLA pays back quickly. Also factor in that MLA’s advantage grows with S , so it is the correct architectural choice for a long-context-first design.

Key Takeaways

- MLA caches a compressed low-rank latent $c_t \in \mathbb{R}^C$ per token per layer rather than full KV tensors, achieving $\sim 31\times$ compression vs. MHA at DeepSeek V2’s settings ($C = 512$, $N = 128$, $H = 128$).
- The trade-off is compute: each query step must up-project cached latents back to full KV, adding FLOPs that are favorable when bandwidth savings dominate—typically at context lengths $\geq 4,000$ tokens.
- RoPE compatibility requires caching a small set of decoupled positional keys alongside the latent; this adds a fixed overhead of d_{rope} values per token per layer.
- MLA is architecturally orthogonal to GQA (head count vs. head dimensionality) and delivers higher compression ratios, but requires custom kernel support that GQA does not.
- For long-context serving at $S \geq 8,000$, MLA is the current state-of-the-art architectural approach to keeping the KV cache manageable without sacrificing model quality.

16.2 Cross-Layer and Local Attention

16.2.1 Cross-Layer KV Sharing

You have already deployed GQA and your KV cache is still the binding memory constraint at large batch sizes or long context. The follow-up question in the interview, almost inevitably, is: *what else can you do without changing the number of KV heads or the per-head dimension?* Cross-Layer Attention (CLA) is the answer that shows up at senior and staff-level design rounds. It is conceptually simple, the math is one line, and it produces cache reductions that compound multiplicatively with GQA.

In a standard transformer, every layer ℓ has its own key and value projection matrices W_K^ℓ and W_V^ℓ . During generation, each layer writes its own KV slice to the cache, so total KV memory scales as $L \times T \times N_{kv} \times h \times 4$ bytes (layers $\times$ tokens $\times$ KV heads $\times$ head dim $\times$ K-and-V-in-bf16). The layer count L multiplies every other factor: cutting it is a direct win.

CLA groups layers into blocks of size G and forces all layers in a group to share the projection matrices of the first (leader) layer. For a group starting at layer g_0 , every layer $\ell \in \{g_0, g_0 + 1, \dots, g_0 + G - 1\}$ computes:

$$K_\ell = X W_K^{g_0}, \quad V_\ell = X W_V^{g_0}.$$

At inference time, the KV vectors are computed once for the group and reused. Layers $g_0 + 1$ through $g_0 + G - 1$ point back to the leader’s cache slice; no new memory is allocated for them. The query projections W_Q^ℓ remain independent per layer, so the model retains its full expressive capacity on the query side.

The cache reduction is exact: instead of L independent KV slices you store L/G , cutting KV memory by a factor of G .

CLA does for the depth dimension what GQA does for the head dimension: it amortizes the KV cost through sharing, and the two techniques compose multiplicatively.

The intuition draws on the same logic as GQA. Adjacent layers in a deep residual network process information at similar levels of abstraction. The attention pattern - which positions a query attends to - is largely determined by key-vector content, and the structure relevant at layer ℓ is not radically different from the structure at layer $\ell-1$. Empirically, models trained with sharing ratio $G = 2$ show negligible perplexity degradation; $G = 4$ begins to hurt on tasks that demand precise long-range recall.

The trade-off to name explicitly is *expressiveness versus memory*. Value vectors encode the information mixed into each output, so sharing them means higher layers in a group cannot independently learn what *content* to retrieve - only what *positions* to query. This is the binding constraint that limits how aggressively you can push G . In practice, $G = 2$ sits comfortably on the Pareto curve; beyond $G = 4$ expect accuracy regressions on retrieval-heavy benchmarks.

CLA contrasts with MLA (Multi-Head Latent Attention), which shrinks KV size by low-rank compression along the head-dimension axis. CLA instead reduces the number of independent KV slices across depth. They are complementary: you can stack CLA on top of MLA for compounded savings.

A Worked Example

Consider a 32-layer 7B model: $d = 4,096$, $N_{kv} = 32$ KV heads (standard MHA, no GQA), head dimension $h = 128$, bf16 weights and activations.

Per-token KV bytes across all 32 layers:

$$32 \text{ layers} \times 32 \text{ heads} \times 128 \text{ dim} \times 2 \text{ (K and V)} \times 2 \text{ bytes} = 524,288 \text{ bytes} \approx 512 \text{ KB per token.}$$

At $T = 8,192$ tokens per sequence: $512 \text{ KB} \times 8,192 = 4 \text{ GB per sequence}$. At batch size $B = 32$: $4 \text{ GB} \times 32 = 128 \text{ GB}$, which overflows a single H100's 80 GB HBM.

With CLA at $G = 2$ (16 independent KV groups):

$$\frac{512 \text{ KB}}{2} \times 8,192 = 2 \text{ GB per sequence} \Rightarrow 64 \text{ GB at } B = 32. \quad (2 \times \text{ reduction.})$$

Adding GQA-8 (reduce N_{kv} from 32 to 8, a $4 \times$ reduction):

$$\frac{512 \text{ KB}}{2 \times 4} \times 8,192 = 512 \text{ MB per sequence} \Rightarrow 16 \text{ GB at } B = 32. \quad (8 \times \text{ combined.})$$

The same configuration that saturated a single H100 now fits with 64 GB to spare, enabling larger batches and proportionally higher throughput.

What You Actually Decide in Practice

- **Choose $G = 2$ by default.** It halves KV memory with no statistically detectable accuracy loss on most benchmarks. Only push to $G = 4$ when memory constraints are extreme *and* the workload is retrieval-light. Going above $G = 4$ is usually not worth it.
- **Always make the first layer in the group the leader.** The later layers in a group receive richer representations from the deeper residual stack; their KV projections would carry more nuanced content. Using the shallower layer's projection is the conservative choice for accuracy.
- **Scale the learning rate for shared projections.** Because $W_K^{g_0}$ and $W_V^{g_0}$ receive gradient contributions from every layer in the group, the effective gradient magnitude is $G \times$ larger than for unshared projections. Either divide their learning rate by G or put them in a separate optimizer parameter group with the adjusted

rate.

- **CLA and GQA compound multiplicatively.** CLA-2 plus GQA-8 on the 32-layer 7B model above gives $8\times$ reduction. Adding MLA-style KV compression on top of both can push beyond $10\times$ versus vanilla MHA.
- **RoPE compatibility is not an issue.** RoPE is applied to queries and keys before the dot product, so the values stored in the KV cache already have position baked in. Reusing those vectors in neighboring layers is safe because the token sequence - and therefore the positional context - is identical across layers.

How This Shows Up in Interviews

- **“You’ve deployed GQA on your serving cluster. What other architectural lever reduces KV cache footprint without touching head count or dimension?”** Name CLA, explain the sharing across layers rather than across heads, and quantify: halving the number of independent KV slices halves cache memory, compounding with any GQA already in place. Mention the $G = 2$ rule of thumb.
- **“What are the accuracy risks of CLA and how do you catch them in evaluation?”** Cover the expressiveness trade-off on the value side. Cite $G = 2$ as safe and $G = 4$ as the threshold where you start checking retrieval-heavy benchmarks (needle-in-haystack, multi-hop QA). Those tasks are the canary; if they hold, general language modeling will too.
- **Senior follow-up: “How does CLA interact with tensor parallelism and disaggregated KV caching?”** With tensor parallelism, each device holds a shard of the KV heads. The shared KV shard still lives on its designated device; downstream layers in the group simply read it there instead of computing a fresh projection. Communication overhead per decode step decreases by G because you issue one KV-projection communication per group rather than one per layer. In disaggregated serving, where the KV cache is offloaded to CPU or remote memory, CLA reduces the volume transferred back on every decode step by G - since that transfer is often the dominant latency term in disaggregated architectures, the benefit is disproportionately large.

Key Takeaways

- CLA shares KV projection weights and cached vectors across a group of G consecutive layers, cutting KV memory by exactly a factor of G for those shared dimensions.
- $G = 2$ is the safe default; beyond $G = 4$, perplexity on recall tasks degrades noticeably and the gain stops being worth it.
- CLA is orthogonal to GQA and MLA - the three techniques stack multiplicatively on top of each other.
- Shared projections receive $G\times$ larger gradients; compensate with a $1/G$ learning rate scaling for those parameter groups.
- CLA-2 combined with GQA-8 achieves an $8\times$ KV reduction on a standard 32-layer model, turning an overflowing 128 GB cache into a manageable 16 GB on a single H100.

16.2.2 Sliding-Window and Hybrid Attention Layers

Even after deploying GQA and CLA, a model supporting 128K-token contexts faces a memory wall. A 32-layer model with 8 KV heads and head dimension 128, in bf16, stores $32 \times 8 \times 128 \times 4 = 131,072$ bytes of KV per token per sequence: at $T = 131,072$ that is roughly 17 GB per sequence, exhausting an H100 before you fit

a single request. Sliding-window attention, combined with periodic full-attention layers in a hybrid architecture, is the mechanism frontier models use to serve contexts of 128K to 1M tokens without that blowup. This shows up in senior design rounds under any question framed as “design a long-context serving system” or “how does LLaMA 4 support 10M-token contexts.”

In standard causal attention, the query at position t attends to all past tokens $\{0, 1, \dots, t-1\}$. The KV cache for that layer grows without bound as T increases. Sliding-window (local) attention restricts each query to the most recent W tokens:

$$\text{Attn}(Q_t, K, V) = \text{softmax}\left(\frac{Q_t K_{[t-W:t]}^\top}{\sqrt{h}}\right) V_{[t-W:t]}.$$

The KV cache for a local layer never exceeds W entries per layer, regardless of T . Once the sequence passes position W , old KV entries evict from the sliding window. Cache memory per local layer is a constant $W \times N_{kv} \times h \times 4$ bytes - no dependence on sequence length.

A pure sliding-window model has no long-range recall: it cannot attend to any token more than W positions back. The fix is the *hybrid architecture*: place one full (global) attention layer every k local layers. Global layers maintain the full T -length KV cache and handle long-range dependencies; local layers handle short-range coherence cheaply and constitute the majority of the stack.

Hybrid attention converts a hard memory cliff into a tunable dial: you pay full-attention cost only on $1/k$ of your layers, and you choose k based on how much long-range recall your workload demands.

The motivating fact is empirical: the overwhelming majority of next-token predictions depend on the last few hundred to few thousand tokens. The hard cases - retrieving a fact stated 50K tokens earlier, resolving a coreference across a long document - are a small fraction of all generation steps. A handful of global layers scattered through the stack handles them without forcing every layer to maintain the full context. On standard language modelling benchmarks, replacing five out of every six attention layers with local layers produces nearly identical perplexity, while peak KV memory drops by $3\text{-}5\times$ at 32K context and $10\times$ or more at 128K.

The alternative is learned sparse attention (Longformer-style dilated windows, block-sparse patterns). Those are more expressive for specialized tasks but require custom CUDA kernels and break compatibility with standard FlashAttention implementations. Hybrid attention is trivial to implement: the only change between a local and a global layer is the range of KV pairs fetched. You get the memory savings with no new infrastructure.

A second-order benefit appears at *training* time. Local layers have $O(T \cdot W)$ attention cost rather than $O(T^2)$, so training on long sequences becomes far more tractable, enabling the longer context windows that the serving system then exploits.

A Worked Example

Model: 32 layers, $N_{kv} = 32$ KV heads, head dim $h = 128$, bf16. Per-layer per-token KV bytes: $32 \times 128 \times 2(\text{K+V}) \times 2 \text{ bytes} = 16,384$ bytes.

Full attention at $T = 32,768$:

$$32 \text{ layers} \times 32,768 \text{ tokens} \times 16,384 \text{ bytes} = 17.2 \text{ GB per sequence.}$$

Hybrid (27 local layers at $W = 4,096$, 5 global layers at full T):

$$\text{Local KV} = 27 \times 4,096 \times 16,384 \text{ bytes} = 1.7 \text{ GB,}$$

$$\text{Global KV} = 5 \times 32,768 \times 16,384 \text{ bytes} = 2.7 \text{ GB,}$$

$$\text{Total} \approx 4.4 \text{ GB} \Rightarrow 3.9 \times \text{reduction.}$$

At $T = 131,072$ (128K):

Full attention = $32 \times 131,072 \times 16,384$ bytes = 68.7 GB,

Hybrid local = $27 \times 4,096 \times 16,384$ bytes = 1.7 GB,

Hybrid global = $5 \times 131,072 \times 16,384$ bytes = 10.7 GB,

Hybrid total ≈ 12.4 GB $\Rightarrow 5.5 \times$ reduction.

Notice that the local-layer cost stays *fixed* as T grows - only the global layers scale. The reduction ratio improves with context length, exactly where it matters most. Combining this hybrid model with GQA-8 reduces the global-layer cost by a further $4\times$, bringing the 128K total to approximately 3.4 GB - a single H100 can now host multiple concurrent long-context sequences.

What You Actually Decide in Practice

- **Global layer frequency k :** Use $k = 4$ (one global per four layers) as a conservative default. Gemma and Mistral variants have used $k = 6$ successfully on standard tasks, but needle-in-haystack and multi-hop retrieval benchmarks tend to regress at $k = 6$ unless the global layers retain the full KV cache. Do not go coarser than $k = 6$ without ablation data on your target task distribution.
- **Local window size W :** 4,096 is the common default and works well for most natural language and code tasks. Drop to 2,048 only for workloads that are demonstrably short-range (single-sentence generation, classification). For multi-file code generation where function signatures and imports span many thousands of tokens, prefer $W = 8,192$.
- **The no-RoPE trick on global layers (LLaMA 4-style):** Apply RoPE only to local layers; use position-free full attention on global layers. Without a positional encoding, the global layer's attention scores depend purely on content similarity, not distance. This makes the model near-insensitive to absolute token position on the global pathway, enabling context-length extrapolation well beyond the training window without ALiBi or positional interpolation. Use this when your serving target context is significantly longer than your training context.
- **Combining with CLA:** Apply cross-layer KV sharing within each run of local layers. Five consecutive local layers in a group can be CLA-2 paired, cutting their already-capped W -entry caches by a further factor of 2. The savings are additive in log-memory terms and require no changes to the global layers.
- **Eviction strategy:** The default is FIFO - evict the oldest token when the window is full. For tasks with multiple salient anchors spread across the context (legal document review, multi-turn dialogue with back-references), "heavy hitter" eviction - retain the $W/2$ most recently attended-to positions plus the $W/2$ most recent positions - materially improves recall quality at the same cache budget.

How This Shows Up in Interviews

- **"Design a system to serve a 70B model at 128K context on two H100s with reasonable throughput. How do you handle the KV cache?"** Start from the memory budget (2×80 GB minus ≈ 140 GB model weights in bf16 leaves ≈ 20 GB for KV). Reach for hybrid attention as the architectural lever. Name the window size and global-layer ratio you would pick, show the arithmetic (12 GB for a 32-layer model at 128K in the worked example above), then note that GQA-8 on the global layers brings that below 5 GB, making the budget work.
- **"How does sliding-window attention affect tasks that require retrieving a fact stated 50K tokens**

ago?” Local layers cannot attend that far back; global layers can. The retrieval quality depends on whether there are enough global layers and whether those layers have the relevant token in their KV cache - which they do, because global layers retain full T entries. Latency on those retrievals is no worse than standard attention, since global layers run a full attention pass regardless. The risk is that with only 1-in-6 global layers, some of those layers may not be the ones queried for that particular output position; whether that matters depends on where in the stack the “right” attention pattern emerges.

- **Senior follow-up: “Explain the LLaMA 4 no-RoPE global layer and its failure modes.”** Without positional encoding, the global layer treats all positions as equidistant and attends purely on content similarity. This enables extrapolation well beyond the training context because there is no monotonically decaying positional bias to interfere with distant attention. The failure mode is false-positive retrieval: the model may attend to a content-similar but irrelevant token far away in preference to the correct nearby token, because nothing in the attention score penalises distance. This shows up as degraded performance on needle-in-haystack benchmarks with high-similarity distractors. The mitigation is to keep global layers sparse enough that they are reserved for genuine long-range retrieval needs, rather than routine next-token coherence.

Key Takeaways

- Sliding-window attention caps each local layer’s KV cache at W entries regardless of sequence length, converting $O(T)$ memory into $O(W)$ - a constant.
- Hybrid architectures place 1 global layer per k local layers ($k = 4$ is conservative; $k = 6$ is aggressive). Global layers handle long-range dependencies and grow with T , but they are few enough to keep total memory practical.
- At $T = 128\text{K}$, a 32-layer hybrid model (5 global, 27 local at $W = 4,096$) uses roughly $5.5\times$ less KV memory than full attention; the gap widens at longer contexts.
- The no-RoPE trick on global layers (LLaMA 4-style) removes positional bias from the long-range pathway, enabling near-unlimited context extrapolation at the cost of increased susceptibility to content-similar distractors.
- Hybrid attention and CLA compose cleanly: apply CLA within local-layer groups for multiplicative additional savings on top of the window-size savings from locality.

Chapter 17 Going Beyond the Transformer for Inference

17.1 State-Space and Linear-Attention Models

17.1.1 Mamba and Associative Recall Tasks

The most direct path to faster generation is changing what the model stores. Every trick applied to the KV cache-GQA, MLA, cross-layer sharing-still leaves a memory footprint that grows linearly with context length. A senior interview question in serving-system design often goes: “What architectural change would eliminate that growth entirely?” State-space models, and specifically Mamba, are the answer interviewers are probing for. This topic shows up at staff and principal level, where the expectation is that you can explain the RNN duality, name the specific failure mode of early SSMs on language tasks, and sketch a hybrid deployment strategy with numbers.

State-space models derive from classical linear dynamical systems. A continuous-time SSM maps an input signal $x(t)$ to an output $y(t)$ through a hidden state $h(t)$:

$$h'(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t)$$

Discretized with step size Δ , this becomes a recurrence:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t$$

where $\bar{A}$ and $\bar{B}$ are the discretized matrices derived from A , B , and Δ . At generation time, producing token $t + 1$ requires loading only h_t -a fixed-size vector of dimension N -not an ever-expanding KV cache. The state h_t is the model’s entire memory of the past.

The same recurrence can be re-expressed as a convolution during training:

$$y = \bar{K} * x, \quad \text{where} \quad \bar{K} = (C\bar{B}, C\bar{A}\bar{B}, C\bar{A}^2\bar{B}, \dots)$$

This is parallelizable via a parallel scan in $O(T \log T)$ time, giving training efficiency comparable to a transformer.

The problem with early SSMs like S4 is that $\bar{A}$ and $\bar{B}$ are fixed for all inputs-every token blends into the state identically regardless of content. This fails on *associative recall*: given a sequence of key-value pairs followed by a query key, retrieve the matching value. Fixed linear blending cannot isolate one binding from others; the signal is diluted by all intervening content.

Mamba resolves this by making the discretization input-dependent:

$$\bar{B}_t = B(x_t), \quad \Delta_t = \Delta(x_t), \quad C_t = C(x_t)$$

When the model sees a query that matches a prior key, the input-dependent gate can sharpen the state to amplify that specific binding and suppress others. This selective SSM passes MQAR (Multi-Query Associative Recall) benchmarks that defeat all fixed-parameter SSMs.

Mamba’s selective state-space mechanism gives an RNN the content-based memory it needs to match transformer quality on language tasks while keeping generation cost proportional to hidden state size rather than context length.

The argument is bandwidth. At generation step t a standard transformer reads the entire per-sequence KV cache: $O(S \cdot d \cdot L)$ bytes, growing with every generated token. The attention arithmetic intensity during generation is always approximately 1-the KV cache is sequence-specific, so batching B requests together does

not amortize the memory read; the batch dimension cancels in the intensity formula.

Mamba's generation reads a hidden state of size $O(d_{\text{inner}} \cdot N \cdot L)$ bytes per sequence, where $d_{\text{inner}} = 2d$ is the expanded channel dimension and N is the state dimension (typically 16). This is fixed: it does not grow with S . At $S = 8,192$, the transformer's KV cache dominates bandwidth; Mamba's state is a rounding error beside the weight load.

The trade-off is recall fidelity. A fixed-size state is a lossy compression of history. At very long contexts—say, multi-hop retrieval across a 128k-token document—a pure Mamba model will lose long-range references that a transformer would find cleanly. This is not a design flaw; it is a fundamental consequence of finite state capacity. The practical response is a hybrid architecture: use Mamba blocks for the vast majority of layers (cheap, constant-state generation) and insert sparse full-attention layers every 4-8 blocks as global checkpoints that can attend to anything in the prefix without compression loss. Early SSMs like S4 used fixed HiPPO-initialized matrices tuned for smooth continuous signals; they had no mechanism for content-based gating at all. Mamba's selection mechanism is the critical upgrade.

A Worked Example

Consider a 7B Mamba model with $d = 4,096$, $d_{\text{inner}} = 8,192$, state dimension $N = 16$, $L = 32$ layers, $B = 8$ concurrent sequences, context $S = 8,192$, bf16 weights on an H100 (80 GB HBM, 3.35 TB/s bandwidth).

Transformer KV cache (equivalent architecture). Per layer per sequence:

$$S \times n_{\text{heads}} \times d_{\text{head}} \times 2 \times 2 \text{ bytes} = 8192 \times 32 \times 128 \times 4 = 128 \text{ MB}$$

Across 32 layers and $B = 8$: $128 \times 32 \times 8 = 32,768 \text{ MB} = 32 \text{ GB}$, plus 14 GB of model weights—46 GB total. This barely fits on one H100.

Mamba SSM state. Per layer per sequence:

$$d_{\text{inner}} \times N \times 2 \text{ bytes} = 8192 \times 16 \times 2 = 256 \text{ KB}$$

Across 32 layers and $B = 8$: $256 \times 32 \times 8 = 65,536 \text{ KB} = 64 \text{ MB}$. Total inference memory: $14,064 \text{ MB} \approx 14 \text{ GB}$. The model fits with headroom for $B = 32$ or beyond.

Bandwidth per generation step.

- Transformer: reads 32 GB (KV) + 14 GB (weights) = 46 GB $\Rightarrow 46/3.35 \approx 13.7 \text{ ms}$ per token.
- Mamba: reads 0.064 GB (state) + 14 GB (weights) $\approx 14 \text{ GB} \Rightarrow 14/3.35 \approx 4.2 \text{ ms}$ per token.
- Speedup: $13.7/4.2 \approx 3.3\times$ from bandwidth reduction alone.

At $S = 32,768$ the transformer KV cache grows to 128 GB and no longer fits on a single H100. Mamba's state remains at 64 MB regardless, and throughput continues to scale with B .

What You Actually Decide in Practice

- **Pure Mamba vs. hybrid.** Use pure Mamba only for workloads with short-to-medium context and no hard retrieval requirements—code completion under 2k tokens, for example. For long-document QA, agentic tool use, or RAG, insert one full-attention layer per 6-8 Mamba blocks. In my experience, placing full-attention layers non-uniformly (denser in the back half of the stack, around layers 20-32 of 32) gives the best quality-per-cost trade-off because long-range context matters more in deeper layers.
- **State dimension N .** The default of 16 is sufficient for typical language modeling. Increasing to 64 improves performance on long-context recall by a measurable margin at sequences exceeding 16k tokens,

but quadruples the state memory and adds training compute. Worth it only if your deployment workload is consistently above 16k context.

- **Selective scan kernels.** Mamba’s parallel-scan training requires custom CUDA kernels. The inference path is a plain recurrence-no custom ops needed-but verify that your serving framework handles the recurrent forward pass without accidentally falling back to the scan path, which would be unnecessary overhead.
- **Mixing with MoE.** Hybrid Mamba-MoE models have shipped at production scale: a 52B MoE with 44 Mamba layers and 8 MoE-transformer layers demonstrated 40-60% lower per-token serving cost versus a dense transformer of equivalent quality. The Mamba layers handle local pattern matching; the sparse MoE transformer layers handle routing and long-range retrieval.
- **Failure mode to watch for.** Pure Mamba on retrieval-heavy workloads fails silently-the model produces confident, plausible-sounding answers that omit or corrupt facts from the far context. Add a full-attention layer or use retrieval augmentation before shipping to production if users will cite specific document content.

How This Shows Up in Interviews

- **“Why is transformer generation memory-bound, and what architectural change would fix it?”** Strong answer: Attention arithmetic intensity at generation is always ≈ 1 because each sequence has its own KV cache-batching B requests does not amortize it, since bytes transferred and FLOPs both scale with B and cancel. The architectural fix is replacing the growing KV cache with a fixed-size recurrent state (SSM), making generation memory cost constant in context length.
- **“What is associative recall, and why did early state-space models fail at it?”** Strong answer: Associative recall is the task of retrieving a value given its key after many intervening tokens. Early SSMs use fixed linear recurrences-every token blends into the state identically regardless of content-so the query signal for a specific key gets diluted by unrelated tokens. Mamba’s input-dependent discretization (Δ_t , $\bar{B}_t$, C_t as functions of x_t) lets the model selectively amplify relevant bindings.
- **Senior follow-up: “If you were deploying a 70B hybrid model with 32k context for an agentic workload, how would you design the layer composition?”** Senior answer: Use 24 Mamba blocks and 8 full-attention layers in a 32-layer stack, placing full-attention layers at positions 4, 8, 16, 20, 24, 28-denser in the second half where long-range context is most relied upon. At $S = 32,768$ and $B = 4$, the full-attention KV cache for 8 layers is $8 \times 32768 \times 32 \times 128 \times 4 = 4$ GB; the Mamba state for 24 layers is $24 \times 256 \text{ KB} \times 4 = 24$ MB. Total inference memory stays under 20 GB, easily within one H100’s budget.

Key Takeaways

- Transformer generation is structurally memory-bound: attention arithmetic intensity is ≈ 1 regardless of batch size because each sequence owns its KV cache and batching does not amortize the read.
- Mamba replaces the linearly growing KV cache with a fixed hidden state ($d_{\text{inner}} \times N$ per layer), making generation memory cost independent of context length.
- The critical innovation over S4 and earlier SSMs is the selective (input-dependent) state update, which enables content-based memory and passes associative recall benchmarks.
- At $S = 8,192$ and $B = 8$, a pure Mamba 7B model requires ≈ 14 GB versus ≈ 46 GB for an equivalent transformer-enabling $3.3\times$ lower per-token latency from bandwidth alone.

- Production deployments use hybrid architectures: Mamba for local processing and cheap constant-state generation, with one sparse full-attention layer per 6-8 Mamba blocks for long-range retrieval.

17.1.2 Linear Attention as a Production Architecture

When an interviewer asks how you would serve a 400B-parameter MoE at low inter-token latency without quadratic memory, the answer that surfaces in real frontier-lab discussions is linear attention. Unlike SSMs, linear attention is a mathematically direct reformulation of the standard attention operator—no new parameterization, no custom scan kernels at inference, just a feature map that converts the quadratic softmax into a linear recurrence. The result is production models with hundreds of billions of parameters that serve at near-SSM latency while retaining transformer-class quality on most tasks. Knowing the kernel trick, the RNN dual, and where this architecture concretely breaks is the depth that separates mid-level from senior answers on architecture design interviews.

Standard causal attention at position i computes:

$$y_i = \frac{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d}) v_j}{\sum_{j \leq i} \exp(q_i^\top k_j / \sqrt{d})}$$

The exponential makes this $O(T^2)$ in sequence length and forces materializing a $T \times T$ attention matrix. Linear attention replaces the exponential kernel with a *feature map* $\phi : \mathbb{R}^{d_{\text{head}}} \rightarrow \mathbb{R}^r$ such that:

$$\exp(q^\top k) \approx \phi(q)^\top \phi(k)$$

A common choice is $\phi(x) = \text{ELU}(x) + 1$, which ensures non-negative weights. With this substitution and using associativity of matrix multiplication:

$$y_i = \frac{\phi(q_i)^\top \left(\sum_{j \leq i} \phi(k_j) v_j^\top \right)}{\phi(q_i)^\top \left(\sum_{j \leq i} \phi(k_j) \right)} = \frac{\phi(q_i)^\top S_i}{\phi(q_i)^\top z_i}$$

where the state matrices update recurrently:

$$S_i = S_{i-1} + \phi(k_i) v_i^\top \in \mathbb{R}^{r \times d_{\text{head}}}, \quad z_i = z_{i-1} + \phi(k_i) \in \mathbb{R}^r$$

At generation step i , producing y_i requires only (S_{i-1}, z_{i-1}) —no growing cache. The state (S, z) per attention head is $r \times d_{\text{head}} + r$ values—fixed in size, completely independent of sequence length. During training the recurrence is parallelized via a parallel scan in $O(T \log T)$ operations.

Linear attention converts softmax attention into an RNN with an $r \times d_{\text{head}}$ outer-product state, making generation $O(1)$ in context length and replacing a growing KV cache with a fixed matrix load.

The throughput gain follows directly from the memory analysis. Softmax attention reads $O(S \cdot n_{\text{heads}} \cdot d_{\text{head}} \cdot L)$ bytes per generation step, growing with every token emitted. Linear attention reads a fixed state of $O(r \cdot d_{\text{head}} \cdot n_{\text{heads}} \cdot L)$ bytes per generation step regardless of how long the conversation has run. At context $S = 8,192$, that difference is substantial; at $S = 32,768$, it becomes the deciding factor in whether the model fits on a single accelerator at all.

The trade-off is sharpness of retrieval. Softmax attention is a sparse content router: at each step a handful of tokens receive nearly all the weight. Linear attention integrates all past keys and values into a single matrix

S_t via outer products. If two past tokens have similar $\phi(k)$ directions, their values blend inseparably in S_t and cannot be cleanly retrieved later. This means linear attention degrades on exact lookup tasks: fetching a specific fact from a long document, multi-hop reasoning, or string-copy probes. For local pattern matching, grammatical coherence, and generation tasks without hard retrieval demands, linear attention performs on par with softmax.

This trade-off is why production deployments are hybrids. MiniMax’s 456B-parameter MoE-a publicly documented production system-uses linear attention for the vast majority of layers with standard full-attention layers inserted at fixed intervals. The pattern is now industry-standard: linear and local-attention layers handle the bulk of computation cheaply; sparse full-attention layers handle long-range retrieval. Compared to Mamba, linear attention preserves the query-key-value interface of the transformer, making it straightforward to inherit pretrained weights or fine-tune an existing checkpoint toward more efficient inference without re-parameterizing the attention heads.

A Worked Example

A 7B hybrid model with $d = 4,096$, 32 heads, $d_{\text{head}} = 128$, feature-map dimension $r = d_{\text{head}} = 128$, 32 layers (24 linear attention, 8 full attention), bf16, $B = 8$, $S = 8,192$ on an H100 (80 GB HBM, 3.35 TB/s).

Full-attention KV cache (8 layers). Per layer per sequence:

$$S \times n_{\text{heads}} \times d_{\text{head}} \times 2 \times 2 \text{ bytes} = 8192 \times 32 \times 128 \times 4 = 128 \text{ MB}$$

For 8 layers and $B = 8$: $128 \times 8 \times 8 = 8,192 \text{ MB} = 8 \text{ GB}$.

Linear attention state (24 layers). Per head per layer: $r \times d_{\text{head}} \times 2 \text{ bytes} = 128 \times 128 \times 2 = 32 \text{ KB}$.

Per layer (32 heads): 1 MB. For 24 layers and $B = 8$: $24 \times 1 \times 8 = 192 \text{ MB}$.

Total inference memory: 14 GB (weights) + 8 GB (KV) + 0.2 GB (linear state) $\approx 22.2 \text{ GB}$. The model fits with room to expand B to 32 before hitting the 80 GB limit.

Compare to a pure transformer: 14 GB (weights) + 32 GB (all 32 layers KV at $B = 8$, $S = 8,192$) = 46 GB. Fits, but barely, with no headroom for larger batches.

Generation bandwidth per step:

- Hybrid: $14 \text{ GB} + 8 \text{ GB} + 0.2 \text{ GB} = 22.2 \text{ GB} \Rightarrow 22.2/3.35 \approx 6.6 \text{ ms per token}$.
- Pure transformer: $14 \text{ GB} + 32 \text{ GB} = 46 \text{ GB} \Rightarrow 46/3.35 \approx 13.7 \text{ ms per token}$.
- Speedup: $13.7/6.6 \approx 2.1\times$ in the memory-bound regime.

At $S = 32,768$, the pure-transformer KV cache grows to 128 GB and no longer fits on a single H100. The hybrid KV cache stays at 32 GB (only 8 layers), giving a total of $\approx 46 \text{ GB}$ -still comfortably within budget.

What You Actually Decide in Practice

- **Hybrid layer ratio.** A 3:1 ratio (three linear layers per one full-attention layer) recovers approximately 95% of full-transformer perplexity on standard language benchmarks while cutting KV cache to 25% of a full model’s cost. A 7:1 ratio is viable for low-retrieval workloads such as open-ended generation or summarization. Do not go beyond 8:1 on agentic or long-document workloads without running needle-in-a-haystack evaluations first.
- **Feature map choice.** ELU+1 is simple but can produce near-zero z_t values early in a sequence, leading to division instability. Add a floor in your implementation: `z = torch.clamp(z, min=1e-6)`. A more robust alternative is Gated Linear Attention (GLA), which adds an input-dependent forget gate-analogous to Mamba’s selection mechanism-and significantly improves performance on associative recall tasks at the cost of a slightly more complex recurrence.

- **RoPE incompatibility.** Standard RoPE rotates query and key vectors in a way that depends on the exponential attention structure; that structure disappears when you substitute the feature map. If your base model uses RoPE, you cannot apply linear attention directly to those layers without removing the positional encoding. In practice, restrict RoPE (or any rope-like encoding) to the full-attention layers and treat linear-attention layers as position-agnostic. They remain effective because local ordering cues are encoded in the token content itself.
- **State size at large model width.** At $d = 8,192$ (typical for a 70B model), the linear state per head per layer is $128 \times 128 \times 2 = 32$ KB, per layer 64 heads $\times$ 32 KB = 2 MB, across 56 linear layers at $B = 8$ gives $56 \times 2 \times 8 = 896$ MB. Manageable. But if you use a larger feature-map dimension $r = 4 \times d_{\text{head}} = 512$ for richer approximation, the per-head state quadruples-watch memory at $B > 16$.
- **Training stability.** Linear attention loss curves are typically noisier than softmax attention early in training, especially when z_t is small. If initializing from a pretrained transformer, keep the first 4-8 layers as full attention during the initial fine-tuning phase and unfreeze the remaining linear layers gradually over the first 1k steps. Gradient clipping at 1.0 and a short linear warmup of 500 steps are usually sufficient to stabilize.

How This Shows Up in Interviews

- **“Walk me through how linear attention eliminates the quadratic complexity of softmax attention.”**
Strong answer: Replace $\exp(q^\top k)$ with $\phi(q)^\top \phi(k)$, then apply associativity to pull $\phi(q_i)^\top$ outside the sum over j . The inner sum $S_i = \sum_{j \leq i} \phi(k_j) v_j^\top$ updates as a rank-1 outer-product addition each step and is $O(r \cdot d_{\text{head}})$ state-fixed, independent of sequence length. Training uses a parallel scan over this recurrence.
- **“What does linear attention lose versus softmax attention, and when does that actually matter?”**
Strong answer: Softmax is a sparse content router that can concentrate almost all probability on a single token. Linear attention accumulates all history into a fixed matrix via outer products; if two past keys map to similar feature directions, their values blend inseparably. This matters for exact in-context lookup, multi-hop retrieval, and copy tasks. It does not matter for local coherence, syntax, or open-ended generation-which is why hybrids with 75-87.5% linear layers still achieve near-transformer perplexity.
- **Senior follow-up: “How would you design a 70B hybrid model for an agentic workload with 64k context and hard factual lookup requirements?”** Senior answer: Use 24 linear-attention layers and 8 full-attention layers in a 32-layer stack, with full-attention layers at positions 4, 8, 16, 20, 24, 28-denser toward the end where cross-document reasoning concentrates. At $S = 65,536$ and $B = 4$, the full-attention KV for 8 layers is $8 \times 65536 \times 64 \times 128 \times 4 \times 4 = 16$ GB; the linear state for 24 layers is $24 \times 64 \times 128 \times 128 \times 2 \times 4 = 384$ MB. Weights for a 70B model are ≈ 140 GB in bf16, so this design requires tensor parallelism across 2-4 H100s; the KV cache stays under 20 GB per node, which is tractable.

Key Takeaways

- Linear attention replaces $\exp(q^\top k)$ with a kernel feature map $\phi(q)^\top \phi(k)$, converting the growing KV cache into a fixed $r \times d_{\text{head}}$ recurrent state that is $O(1)$ in context length.
- At $S = 8,192$ and $B = 8$, a 3:1 hybrid (24 linear, 8 full-attention layers) cuts inference bandwidth from 46 GB to 22 GB per step-a $2.1\times$ latency improvement-and enables batch expansion that a pure transformer

cannot afford.

- At $S = 32,768$ a pure transformer KV cache overflows a single H100; the hybrid fits comfortably by limiting the growing cache to 8 layers.
- Linear attention degrades on exact retrieval tasks; production architectures use a 3:1 to 7:1 hybrid ratio with full-attention layers handling long-range lookup. MiniMax's 456B MoE is the public benchmark for this pattern.
- Standard RoPE is incompatible with the linear attention kernel; restrict position encodings to full-attention layers and treat linear-attention layers as position-agnostic.

17.2 Non-Autoregressive Generation

17.2.1 Diffusion Language Models

When a senior engineer at a frontier lab asks you how you would redesign the text generation stack if the autoregressive constraint were lifted, they are probing whether you understand the *root cause* of inference inefficiency rather than just its symptoms. Diffusion language models answer that question by abandoning sequential generation entirely, producing all output tokens simultaneously and then refining them across a fixed number of forward passes. This approach has moved from a research curiosity to a production-relevant architecture rapidly, and understanding its mechanics is now a legitimate interview topic at labs building the next generation of serving infrastructure.

Autoregressive models are forced to generate one token at a time because each token conditions on all preceding tokens. Diffusion models break this constraint by starting from a fully noisy or masked sequence of the target length and iteratively denoising it toward coherent text. At each refinement step t , the model receives the current noisy sequence x_t and predicts a cleaner version x_{t-1} , until x_0 is the final output after K steps.

The decisive structural difference is that every refinement step processes the entire output sequence in a single forward pass. If you are generating a response of length L tokens, the model sees all L positions simultaneously at every step. This is structurally identical to the prefill phase of an autoregressive model, not its generation phase. There is no KV cache that grows with sequence length, because there is no sequential dependency to cache. Every step is a dense matrix multiply across the full sequence.

Formally, let $q(x_t | x_0)$ be a fixed forward noising process (masking or Gaussian corruption) and $p_\theta(x_{t-1} | x_t)$ the learned denoising model. Training minimizes a denoising objective:

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|x_0 - p_\theta(x_t)\|^2].$$

Inference draws x_K from the prior and runs the chain $x_K \rightarrow x_{K-1} \rightarrow \dots \rightarrow x_0$ using a fixed number of steps, typically 10-50 in practice.

The payoff is that by paying the cost of a prefill-like forward pass K times instead of paying the cost of a sequential memory-bottlenecked pass L times, diffusion models trade a small number of expensive parallel steps for a large number of cheap-but-idle sequential steps.

The arithmetic intensity argument is decisive. During autoregressive generation, the effective arithmetic intensity per token is approximately 1 for the attention layers and B for the MLP layers. At batch size 1, both collapse to the floor, and memory bandwidth - not compute - limits throughput. You spend most of every GPU cycle waiting for HBM reads.

In a diffusion model, each forward pass processes all L output tokens at once. The arithmetic intensity of the MLP layers becomes $B \times L$. For a sequence of 1024 tokens and batch size 1, that is 1024 versus 1 - three

orders of magnitude better GPU utilization per pass. The model is compute-limited for any reasonably long sequence, meaning you are doing useful arithmetic on every cycle of the accelerator.

The trade-off is quality. Autoregressive generation samples each token from its exact conditional distribution given the full prefix. Diffusion models make a weaker guarantee: the global distribution over output sequences is approximately correct after K steps, but individual token predictions mid-chain are noisy. Early diffusion LMs struggled on tasks requiring precise factual recall or tight long-range consistency. On coding tasks, where correctness is mechanically verifiable, results are already competitive with similarly-sized AR models. The open question is whether more refinement steps or better training objectives will close the gap on harder tasks.

A Worked Example

Consider generating a 1024-token response using a 7B-parameter model on a single H100 (3.35 TB/s HBM bandwidth, 989 TFLOPS BF16).

Autoregressive baseline, batch size 1. Each generation step reads approximately the full parameter set: $7 \times 10^9 \times 2$ bytes (BF16) = 14 GB. Latency per token $\approx 14 \text{ GB} \div 3.35 \text{ TB/s} \approx 4.2 \text{ ms}$. Total for 1024 tokens: $4.2 \times 1024 \approx 4.3$ seconds. Throughput: approximately 238 tokens/second.

Diffusion model, 20 refinement steps. Each step is a full forward pass over 1024 tokens. FLOPs per pass:

$$2 \times 7 \times 10^9 \times 1024 \approx 1.43 \times 10^{13} \text{ FLOPs.}$$

Time per pass on H100: $1.43 \times 10^{13} \div 9.89 \times 10^{14} \approx 14.5 \text{ ms}$. Total for 20 steps: $20 \times 14.5 = 290 \text{ ms}$. Throughput: $1024 \div 0.29 \approx 3,500$ tokens/second.

That is roughly a $15\times$ throughput improvement while generating the same number of tokens from a similarly-sized model. The 20 refinement steps can be reduced to 10 for another $2\times$ speedup at some quality cost, or raised to 50 for better quality at reduced throughput. The operating point is a continuous dial, not a fixed cost, which is qualitatively different from autoregressive generation where the step count equals the token count and is non-negotiable.

What You Actually Decide in Practice

- **Refinement step count is a quality-latency dial, not a constant.** Start at 20 steps as a reasonable default for production. Below 10, quality degrades sharply on long-form generation. Above 50, returns diminish and you approach the throughput of a small autoregressive model. Sweep this on your task distribution and set it per request tier, not globally.
- **Output length must be committed at request time.** Unlike AR models that stop at an EOS token, diffusion models require knowing the target sequence length upfront. Wasted padding positions cost compute proportionally. Build your serving layer to bucket requests by expected output length (powers of two: 128, 256, 512, 1024 work well) to keep average padding below $2\times$.
- **Architecture uses bidirectional attention.** Because all positions are visible at every step, there is no causal mask. This means flash attention applies, prefix caching does not, and the attention FLOPs per step are $O(L^2)$ rather than $O(1)$ per new token. For very short outputs ($L < 64$), the quadratic cost erodes the throughput advantage rapidly.
- **Quality gaps are real on recall-heavy tasks.** In my experience testing diffusion LMs on tasks requiring exact entity lookup or precise numerical computation, quality drops noticeably compared to AR models of

the same parameter count. Routing recall-heavy requests to an AR model and generation-heavy requests to a diffusion model is a production-viable strategy, but it requires a lightweight classifier at the request boundary.

- **The draft model in speculative decoding is a natural fit.** A small diffusion model can serve as the draft proposal generator in a speculative decoding setup, because it produces multiple tokens per forward pass natively. If the acceptance rate is reasonable, this combines the quality guarantee of the AR target model with the throughput of a diffusion draft.

How This Shows Up in Interviews

- **“Autoregressive generation is memory-bottlenecked at batch size 1. What are the fundamental architectural alternatives?”** Strong answer: contrast AR generation (arithmetic intensity ≈ 1 , memory-limited) with diffusion (all tokens in parallel, compute-limited, intensity $\approx BL$). Name the trade-off explicitly: throughput advantage is large for long outputs but shrinks for short ones; quality lags on recall tasks.
- **“Walk me through how a diffusion language model generates text.”** Strong answer: describe the iterative denoising chain from x_K to x_0 , emphasize that every step is a full-sequence forward pass (identical to prefill), explain the absence of a KV cache, and note that output length must be specified upfront. Give the arithmetic: $20 \text{ steps} \times \text{prefill cost}$ versus $L \text{ steps} \times \text{generation cost}$.
- **“If diffusion models are so much faster, why isn’t everyone using them?”** Strong answer: the quality gap on factual recall and instruction following is real. Fixed output length at request time creates padding waste and serving complexity. The compute advantage only materializes at output lengths long enough to amortize per-step cost, roughly $L \geq 200$ tokens. Frontier labs are using them for high-throughput batch workloads, not interactive chat.
- **Senior follow-up: “How would you use a diffusion model as the draft component in a speculative decoding pipeline serving a 70B AR target?”** Senior answer: the diffusion draft generates K token proposals in a single forward pass (no sequential sampling loop), which is faster than an AR draft generating K tokens one-by-one. The AR target verifies all K in parallel via prefill. Acceptance rate depends on distribution alignment; fine-tune the draft on the target model’s output distribution to improve it. The diffusion draft’s fixed-length constraint requires knowing the speculative lookahead window K ahead of time, which is compatible with standard speculative decoding where K is a fixed hyperparameter.

Key Takeaways

- Diffusion language models generate all output tokens in parallel at each refinement step, shifting inference from memory-limited (arithmetic intensity ≈ 1) to compute-limited (intensity $\approx BL$).
- A 7B diffusion model with 20 refinement steps achieves approximately 3,500 tokens/second on an H100 versus approximately 238 tokens/second for a 7B autoregressive model at batch size 1 - roughly a $15\times$ throughput advantage at $L = 1024$.
- The advantage shrinks for short outputs and disappears below roughly $L = 64$; build output-length bucketing into the serving scheduler to protect efficiency.
- Quality lags AR models on recall-heavy tasks; the refinement step count is the quality-latency dial to tune per task tier.

- There is no KV cache, which eliminates the most complex part of AR serving infrastructure but requires output length to be committed at request time.

17.2.2 Implications for Serving Cost

When inference costs are growing faster than revenue, the honest engineering question is not “which model is fastest in a benchmark?” but “what changes when I swap from an autoregressive backend to a non-autoregressive one, and which parts of my serving stack need to be redesigned?” Non-autoregressive generation does not just produce tokens faster - it changes the memory model, the scheduler contract, and the capacity planning calculus in ways that compound across a large deployment. Knowing these second-order effects is what separates a staff-level system design answer from a mid-level one.

The serving cost of an autoregressive model has two structurally different components: the model weights, which are static and shared across all requests, and the KV cache, which grows per-request and per-generated-token. Managing that growth requires PagedAttention-style virtual memory, copy-on-write prefix sharing, and eviction policies - all of which add engineering overhead and introduce fragmentation.

Non-autoregressive models collapse this structure. Because there is no sequential dependency across refinement steps, there is no KV cache to accumulate. Each step reads the model weights and the current noisy sequence, writes the refined sequence, and exits. Memory consumption per request is a flat $O(L \cdot d)$ bytes where L is the committed output length and d is the model hidden dimension - constant across all K refinement steps.

The total HBM footprint for a diffusion serving deployment is:

$$M_{\text{total}} = M_{\text{weights}} + B \cdot L \cdot d \cdot 2 \text{ bytes (BF16)}$$

where B is the batch size and L is the fixed output length. Compare this to an AR deployment where M_{KV} grows as the sequence extends and must be tracked dynamically. The diffusion model’s memory budget is fully computable before a single token is generated, which makes capacity planning a spreadsheet exercise rather than a runtime estimation problem.

Eliminating the KV cache turns the serving stack’s hardest dynamic-memory problem into a static allocation, at the cost of requiring output length to be committed at request time.

The economic argument is clearest when you look at cost per output token. Autoregressive generation is memory-bandwidth-limited: every token costs one full HBM sweep of the parameter set. For a 7B model in BF16, that is 14 GB of data movement per token. At 3.35 TB/s on an H100, the hardware-imposed minimum is 4.2 ms per token regardless of software optimization. No kernel tuning removes this floor.

Diffusion generation amortizes weight-sweep cost across all L output tokens in a step. The effective bandwidth cost per output token across K refinement steps is:

$$\text{BW cost per token} = \frac{K \cdot M_{\text{weights}}}{L}$$

For $K = 20$ and $L = 512$, this is $20/512 \approx 0.04\times$ the per-token cost of AR generation. Compute, not memory bandwidth, becomes the binding constraint - which is a far healthier operating regime because compute is easier to scale horizontally.

The catch is fixed output length. Autoregressive models can stream tokens and stop at any EOS, adapting response length to content at runtime. Diffusion models commit to L upfront. Positions that go unused are wasted compute. In my experience, the serving system must either predict output length accurately or accept efficiency losses from padding; neither is free, but length bucketing at powers of two keeps average waste below $2\times$.

A Worked Example

Consider 100 concurrent requests, each expecting up to 512 output tokens, served on a single H100 with 80 GB HBM.

Autoregressive backend (13B model, BF16):

- Weight memory: $13 \times 10^9 \times 2 = 26$ GB.
- KV cache per sequence (40 layers, 32 KV heads, $d_{\text{head}} = 128$, 512 tokens): $512 \times 32 \times 128 \times 40 \times 2 \times 2 \approx 3.4$ GB.
- At batch size 16: $16 \times 3.4 + 26 = 80.4$ GB - already at the memory limit.
- Maximum practical batch size: 15-16 requests.

Diffusion backend (comparable model scale, BF16):

- Weight memory: 26 GB (same).
- Sequence buffer per request ($L = 512$, $d = 5,120$): $512 \times 5,120 \times 2 \approx 5$ MB.
- At batch size 100: 100×5 MB + 26 GB = 26.5 GB total.
- 53 GB remain free for larger batches or additional model shards.
- Maximum practical batch size: limited by compute throughput, not memory - exactly the operating point you want.

The memory freed by eliminating the KV cache allows a batch size of 100 versus 16 on identical hardware, a $6\times$ improvement that directly multiplies throughput before any algorithmic changes. Combined with the per-token compute advantage from the previous subsection, the end-to-end throughput difference for long outputs is striking.

What You Actually Decide in Practice

- **Output length prediction is a first-class infrastructure problem.** Build a lightweight length predictor (a small classification head on a pooled prefix embedding works) that bins requests into length buckets before routing. Rounding up to the nearest power of two (128, 256, 512, 1024) keeps padding waste below $2\times$ on most real distributions. Avoid variable-length padding within a batch; pad all requests in a batch to the same L and process them together.
- **Maintain separate queuing lanes per output-length bucket.** Pack requests of similar expected length into batches; flush each lane when the batch is full or a timeout fires. This is simpler than AR continuous batching because there is no mid-sequence request admission: all requests in a batch start and finish together, so the scheduler contract is identical to training.
- **Expose refinement step count as a per-request parameter.** Low-stakes requests (short-form Q&A, autocompletion) can use 10-15 steps. High-stakes requests (long-form document generation, code for production) benefit from 30-50 steps. Implementing this as a tier enum in your request proto gives the product team a quality-cost lever without infrastructure changes.
- **Route by task type, not just model size.** Deploy AR backends for time-to-first-token-sensitive workloads (interactive chat, function calls, streaming responses) and diffusion backends for throughput-sensitive batch workloads (document summarization, bulk code generation, offline evaluation). A lightweight router that classifies on task type plus expected output length outperforms a single-model strategy on both cost and quality.
- **Prefix caching does not apply; but prompt conditioning is cheaper.** Because diffusion models use bidirectional attention, there is no causal prefix whose KV vectors can be cached across requests. The

cost of conditioning on a long system prompt is paid again at every refinement step. If your serving traffic has a shared system prompt, this eliminates one of the major cost-saving mechanisms of modern AR serving systems. Design around it by keeping prompts short or by absorbing prompt information into the model at training time via instruction tuning.

How This Shows Up in Interviews

- **“How does eliminating the KV cache change your serving infrastructure?”** Strong answer: memory becomes static and fully predictable before generation begins; batch size is set by compute budget rather than memory budget; the scheduler no longer needs eviction policies or copy-on-write prefix sharing; but output length must be committed upfront, introducing a new length-prediction requirement.
- **“What are the limits of the diffusion serving cost advantage?”** Strong answer: three limits. First, short outputs ($L < 64$) do not amortize the per-step weight-sweep cost, shrinking or eliminating the advantage. Second, padding waste from poor length prediction can erode the memory savings. Third, prefix caching disappears, which hurts workloads with long shared system prompts. Name the cross-over point quantitatively: the diffusion backend breaks even with AR roughly when $K/L < 1$, i.e., the number of refinement steps is smaller than the output length.
- **“How would you operate a mixed AR-diffusion cluster to minimize cost per quality point?”** Strong answer: profile your request distribution by task type, expected output length, and quality sensitivity. Assign AR capacity to the interactive, recall-heavy, and short-output tail; assign diffusion capacity to the long-output, throughput-sensitive bulk. Size each backend to its SLA: AR optimized for P95 latency, diffusion optimized for tokens/second. Rebalance allocation weekly as the request mix shifts.
- **Senior follow-up: “How does the capacity planning model change when the serving SLA is throughput-per-dollar rather than latency?”** Senior answer: AR throughput is bounded by memory bandwidth: $\text{tokens/second} \approx \text{BW} / (2 \times N_{\text{params}})$, independent of batch size once memory is saturated. Diffusion throughput scales with compute: $\text{tokens/second} \approx \text{TFLOPS} \times L / (2 \times N_{\text{params}} \times K)$, meaning it improves with longer outputs and fewer steps. The cost-per-token crossover is at $L = K$ (output length equals refinement step count); above that, diffusion dominates per dollar. Model this in your capacity plan and provision diffusion GPUs for the long-output tail of your distribution while using AR GPUs for the short-output head.

Key Takeaways

- Diffusion models have a flat, predictable per-request memory footprint ($O(L \cdot d)$ bytes, constant across all refinement steps) versus an AR model’s linearly growing KV cache, enabling $\sim 6\times$ larger batch sizes on identical hardware for 512-token outputs.
- The effective per-token bandwidth cost is K/L times the AR cost; at $K = 20$, $L = 512$, this is $\sim 4\%$ of the AR figure, shifting the bottleneck from memory bandwidth to compute.
- Output length must be committed at request time; build output-length bucketing (powers of two) into the scheduler to hold padding waste below $2\times$.
- Prefix caching does not apply to diffusion models (bidirectional attention, no causal KV cache); workloads with long shared system prompts lose a major AR cost-saving mechanism.
- Route by task type and expected output length: diffusion for long-output batch workloads, AR for interactive or recall-heavy short-output workloads.

Chapter 18 Speculative Decoding and Serving Optimizations

18.1 Speculative Sampling

18.1.1 The Draft/Target Algorithm and Its Math

Speculative sampling comes up in nearly every inference-focused system design interview because it is the rare technique that cuts latency without any approximation: the output distribution is provably identical to that of the target model. Interviewers at frontier labs ask about it both to probe whether a candidate understands *why* generation is slow (it is memory-bound, not compute-bound) and to test whether they can reason about probabilistic correction schemes under pressure. Expect it at the senior to staff level, where the follow-up is almost always “prove the acceptance step is unbiased” or “how would you choose the draft model?”

Autoregressive generation produces one token per forward pass, and each pass is dominated by loading the model weights from HBM—not by arithmetic. For a 70 B parameter model in bf16 on an H100 with 3.35 TB/s memory bandwidth, a single generation step requires moving roughly $2 \times 70 \times 10^9 \approx 140$ GB of weights, which takes about 42 ms regardless of batch size. Prefill, by contrast, processes all prompt tokens in parallel and is compute-bound: you pay the weight-load cost once and amortise it over the whole sequence. Speculative sampling exploits this asymmetry directly.

The algorithm uses two models: a small, fast *draft model* p and the large, accurate *target model* q . Given a prefix $x_{1:t}$, the draft model autoregressively produces K candidate tokens $\tilde{x}_{t+1}, \dots, \tilde{x}_{t+K}$ under its own distribution. The target model then scores all K tokens in a single forward pass—this is just a prefill over the extended sequence, so it runs in parallel and is fast. For each position $i \in \{1, \dots, K\}$ the candidate $\tilde{x}_{t+i}$ is accepted with probability

$$\alpha_i = \min\left(1, \frac{q(\tilde{x}_{t+i} \mid x_{1:t+i-1})}{p(\tilde{x}_{t+i} \mid x_{1:t+i-1})}\right).$$

If the token is accepted the algorithm moves to position $i + 1$. If it is rejected, a corrected token is sampled from the residual distribution

$$q'(\cdot) = \text{norm}(\max(0, q(\cdot) - p(\cdot))),$$

and the loop terminates. Finally, if all K tokens are accepted, the algorithm samples one additional token directly from q for free. The net effect is that every call to the expensive target model potentially yields more than one new token.

The central insight is that verifying a proposed token under the target model is identical to prefill, which is cheap; only generating the next token requires a fresh, expensive decode step.

The acceptance-rejection correction is not a heuristic—it is an exact importance-sampling step borrowed from the Metropolis-Hastings framework. The proposal distribution is p ; the target is q . When the draft agrees with the target ($p(\tilde{x}) \geq q(\tilde{x})$), the token is accepted unconditionally. When the draft is overconfident relative to the target, the excess probability mass is redistributed via the residual q' , ensuring the marginal distribution of every emitted token matches q exactly. No temperature adjustment, no post-hoc filtering—the samples are mathematically identical to what you would get from greedy-free sampling the target model alone.

The practical speed gain is governed by the expected number of tokens produced per target call, which

equals $1 + K \cdot \bar{\alpha}$ where $\bar{\alpha}$ is the average per-token acceptance rate. With a well-matched draft-target pair, $\bar{\alpha}$ routinely reaches 0.7-0.85 on natural language tasks, giving 2-3 useful tokens per expensive decode step. The latency wall from weight-loading is amortised across those tokens, reducing wall-clock time per output token proportionally.

The key trade-off is draft quality versus draft cost. A stronger draft model pushes $\bar{\alpha}$ higher but also consumes more memory and FLOPs, narrowing the gap to the target model. A weaker draft is cheaper but drops acceptance rates; if $\bar{\alpha}$ falls below roughly $1/K$ the overhead of running the draft outweighs the savings. The sweet spot in production is typically a draft 5-10 $\times$ smaller than the target, from the same model family and ideally distilled from the target.

A Worked Example

Consider a setup with Llama 3 70B as the target and a 7B draft, both in bf16 on a single H100 (80 GB HBM, 3.35 TB/s bandwidth).

Baseline latency per token (target only). Weight size: $2 \times 70 \times 10^9 = 140$ GB. Time to load weights: $140/3350 \approx 42$ ms per step. At $K = 5$ lookahead tokens the target model is called once for every 5-step window.

Draft cost. The 7B draft weighs $2 \times 7 \times 10^9 = 14$ GB. Loading time: $14/3350 \approx 4.2$ ms per step. Five draft steps cost $5 \times 4.2 = 21$ ms total.

Verification cost. The target scores all 5 candidates in one prefill call. Prefill is compute-bound, not bandwidth-bound; at reasonable sequence lengths this is roughly equivalent to one generation step: ≈ 42 ms.

Effective throughput with $\bar{\alpha} = 0.75$. Expected accepted tokens per round: $1 + 5 \times 0.75 = 4.75$. Total time per round: $21 + 42 = 63$ ms. Effective latency per output token: $63/4.75 \approx 13$ ms, versus the 42 ms baseline. That is a $\approx 3.2\times$ wall-clock speed-up with zero change in output quality.

Whiteboard summary:

For each round of K draft steps:

1. Draft model: generate $x_{\{t+1\}} \dots x_{\{t+K\}}$ [$K \times \text{draft_cost}$ ms]
 2. Target model: one prefill over the K tokens [~ 1 decode step ms]
 3. Accept/reject each token, emit $1 + K \times \alpha$ tokens on average
- Speedup = $(1 + K \times \alpha) / (1 + K \times \text{draft_cost} / \text{target_cost})$

What You Actually Decide in Practice

- **Draft model size.** Aim for a 7-10 $\times$ parameter ratio between target and draft. Below that the draft is too slow; above it acceptance rates collapse. When the draft comes from the same family and is distilled from the target, acceptance rates are 5-10 percentage points higher than a generic model of the same size.
- **Lookahead length K .** Start at $K = 4-6$. Increasing K raises the ceiling on speedup but also raises the cost of a bad draft run-if $\bar{\alpha}$ is low, long lookaheads waste draft compute. Profile $\bar{\alpha}$ on representative prompts before tuning K upward.
- **Memory budget.** The draft model lives alongside the target model in GPU memory. On an 80 GB H100 a 70B target leaves roughly 10 GB for the draft after KV caches-enough for a 4-5B model in bf16 or a 7B model quantized to INT4. If you cannot fit both models, consider offloading the draft weights to CPU and streaming them in; the CPU-GPU transfer cost is often less than the savings on generation steps.

- **When speculative decoding hurts.** If the target and draft distributions are poorly aligned (cross-domain prompts, highly stochastic sampling with temperature > 1.5) $\bar{\alpha}$ drops below 0.5 and the overhead dominates. Monitor the acceptance rate per request type in production and fall back to direct decoding when $\bar{\alpha} < 0.5$ for a given prompt class.
- **Batch size interaction.** Speculative decoding is most valuable at batch size 1, where the target model is entirely bandwidth-bound. At large batch sizes ($B > 64$ on an H100) the target model starts to become compute-bound during the verification prefill, and the relative advantage of speculative decoding shrinks. For high-throughput serving at large batch sizes, continuous batching and KV cache management usually matter more than speculation.

How This Shows Up in Interviews

- **“Walk me through how speculative decoding works and why it speeds things up.”** A strong answer names the draft-target pair, explains that the target’s verification step is a prefill (parallelisable, cheap) rather than a decode (sequential, bandwidth-bound), and quantifies the speed-up in terms of $\bar{\alpha}$ and K . Weak answers say “the small model helps the big model” without connecting it to the memory-bandwidth bottleneck.
- **“How do you guarantee the output is drawn from the target distribution?”** The key is the acceptance probability $\min(1, q/p)$ and the residual distribution $\text{norm}(\max(0, q - p))$ for rejected tokens. A strong answer sketches why these two together give an unbiased sample: accepted tokens come with weight q/p , and the correction redistributes the rejected mass exactly as q requires.
- **“What would you do if the acceptance rate on production traffic is lower than expected?”** Concretely: (1) profile which prompt categories have low $\bar{\alpha}$; (2) fine-tune or distill the draft on that domain; (3) fall back to direct decoding for those categories; (4) lower K to reduce wasted draft compute.
- **Senior follow-up: “Can you modify speculative decoding to work without a separate draft model, and what are the accuracy implications?”** Yes-Medusa adds multiple independent heads on top of the target model’s residual stream, one per lookahead position. Each head predicts the token k steps ahead without autoregressive rollout, so there is no separate model cost. The trade-off is that these heads see only the last hidden state, not the full draft autoregressive context, so acceptance rates are typically 10-20 percentage points lower than a strong standalone draft model of comparable parameter count. EAGLE improves on this by feeding the target model’s top-layer features into a lightweight autoregressive draft, recovering most of the acceptance rate while keeping the draft small and co-located.

Key Takeaways

- Speculative sampling exploits the fact that verifying a token (prefill) is much cheaper than generating one (decode), and the output is *exactly* drawn from the target distribution-no approximation.
- The per-round speed-up is $1 + K\bar{\alpha}$; with a well-matched draft, $\bar{\alpha} \approx 0.75$ - 0.85 gives 2 - $3\times$ latency reduction at batch size 1.
- Choose the draft model from the same family as the target, target a 7 - $10\times$ size ratio, and prefer a distilled draft over a generic pretrained model.
- The technique is most effective at small batch sizes where generation is purely bandwidth-bound; at high batch sizes continuous batching yields bigger wins.
- Monitor acceptance rates per prompt domain in production; fall back to direct decoding when $\bar{\alpha} < 0.5$.

18.1.2 Medusa, EAGLE, and Draft Model Design

When you ship speculative decoding to production, the single hardest engineering decision is the draft model itself-how to build it, where to host it, and how tightly to couple it to the target. This question shows up in system design interviews at exactly the point where a candidate has explained the basic algorithm and the interviewer pivots to: “OK, but how would you actually build the draft model for a 70B serving system you own?” Getting this right separates candidates who have thought about the operational reality from those who have only memorized the algorithm.

The draft model’s sole job is to propose K tokens fast enough that the target model’s verification step-a single prefill-recovers more wall-clock time than the draft cost. Three architectures dominate in practice.

Standalone draft model. A fully independent, smaller model from the same family as the target. It has its own weights, its own KV cache, and runs its own autoregressive decode loop to produce K tokens. Acceptance rates are highest here because the draft sees the full autoregressive context at each step. The cost is memory: on an 80 GB H100 with a 70B target, a standalone 7B draft in bf16 consumes 14 GB, leaving barely enough room for KV caches at non-trivial sequence lengths.

Medusa. Medusa attaches K independent prediction heads directly on top of the target model’s last hidden state. Head k predicts the token k steps ahead of the current position without any autoregressive rollout. There is no separate model and no additional memory for weights beyond the heads themselves (each head is a single linear projection of size $d_{\text{model}} \times V$, roughly 0.5 GB per head for a 4096-dimensional model and 128 K vocabulary). The draft cost per round collapses to a single forward pass through the K heads in parallel. The downside is expressive power: head k sees only the hidden state at position t , not the predicted tokens at positions $t + 1, \dots, t + k - 1$, so the heads are conditionally independent. In practice this cuts acceptance rates by 10-20 percentage points relative to a well-tuned standalone draft.

EAGLE (Extrapolation Algorithm for Greater Language-model Efficiency). EAGLE threads a middle path. It trains a lightweight autoregressive draft module-typically a single transformer layer-that takes as input the *feature vectors* from the target model’s final layer concatenated with the token embeddings of the previously accepted tokens. The draft module is therefore not standalone; it is a thin attachment that rides on the target’s residual stream. Because the draft sees the target’s own high-level representations, it has far more signal than Medusa’s static hidden state, and acceptance rates recover to within 5-10 percentage points of a standalone draft. The draft module is also small-one transformer layer for a LLaMA-sized target is roughly 500 M parameters or 1 GB in bf16-so memory pressure is minimal.

The design spectrum runs from standalone (highest acceptance rate, highest memory cost) through EAGLE (near-standalone quality, low memory cost) to Medusa (lowest cost, lowest acceptance rate); the right choice depends on how much GPU memory the target leaves unclaimed after KV caches.

The memory argument is decisive at the 70B scale. A standalone 7B draft eats 14 GB; EAGLE’s single-layer module eats 1 GB; Medusa’s $K = 5$ heads eat 2.5 GB. For a 70B target in bf16 (140 GB) sharded across two H100s (160 GB total), the KV cache budget for a batch of 16 sequences at sequence length 2,048 is roughly 10 GB, leaving only 10 GB spare. EAGLE or Medusa fit easily; a standalone 7B draft does not without aggressive quantisation.

The co-location advantage of EAGLE over a standalone draft is not just memory. Because the draft module ingests the target model’s feature vectors, those vectors only need to be computed once per token position-they are produced by the target during verification and cached. The draft therefore uses information that the standalone model would have to independently re-derive, which is why EAGLE’s acceptance rates are high

despite the module being tiny.

The residual between Medusa and EAGLE is conditional independence. When generating code or structured output where token $t + 2$ strongly depends on token $t + 1$, Medusa’s heads—each conditioned only on position t —systematically underestimate the entropy of later positions and generate plausible-looking but locally inconsistent proposals. EAGLE’s autoregressive rollout within the draft module preserves this conditional structure.

A Worked Example

Consider LLaMA 3 70B (bf 16) served on two H100s via tensor parallelism, with $K = 5$ lookahead tokens, batch size 1, and a 2,048-token context.

Memory budget per GPU (80 GB each, 160 GB total):

- Target weights: 140 GB across both GPUs, so 70 GB per GPU.
- KV cache (batch 1, 2048 tokens, 80 layers, 8 KV heads, head dim 128, bf 16): $1 \times 2048 \times 80 \times 8 \times 128 \times 2 \times 2 \approx 0.84$ GB per GPU.
- Remaining per GPU: ≈ 9 GB.

Option A: Standalone 7B draft. 7B model in bf 16: 14 GB total, 7 GB per GPU if sharded. Fits, but leaves only 2 GB per GPU for activations and the draft’s own KV cache—tight. Expected acceptance rate: $\bar{\alpha} \approx 0.80$ (same family, distilled). Tokens per target call: $1 + 5 \times 0.80 = 5.0$.

Option B: EAGLE single-layer module. Module size: ≈ 1 GB total, negligible per GPU. Fits comfortably. Expected acceptance rate: $\bar{\alpha} \approx 0.72$ (empirically 5-10 pp below a good standalone). Tokens per target call: $1 + 5 \times 0.72 = 4.6$.

Latency comparison (target decode ≈ 42 ms, draft step ≈ 4 ms for standalone, ≈ 0.5 ms for EAGLE’s single layer):

Strategy	Draft cost	Tokens/call	ms/token
No speculation	-	1.0	42
Standalone 7B	$5 \times 4 = 20$ ms	5.0	$(42 + 20)/5.0 = 12.4$
EAGLE	$5 \times 0.5 = 2.5$ ms	4.6	$(42 + 2.5)/4.6 = 9.7$
Medusa	≈ 0 ms (parallel)	3.5	$42/3.5 = 12.0$

EAGLE wins on effective ms/token in this configuration because its near-zero draft overhead and near-standalone acceptance rate combine better than either extreme. Note that Medusa assumes $\bar{\alpha} \approx 0.50$, which is typical for code-generation tasks where later tokens are highly context-dependent.

What You Actually Decide in Practice

- **Choose Medusa only when memory is the hard constraint.** If the target already occupies 95 % of device memory and there is no room for even a 1 GB module, Medusa is your only option. In any other scenario EAGLE dominates on latency.
- **Distill the draft from the target, not from general pretraining.** A draft trained only on next-token prediction from public data will have a vocabulary distribution mismatch vs. the target after RLHF or instruction tuning. Even a lightweight supervised fine-tuning run on 1-5 B tokens of target-model outputs raises $\bar{\alpha}$ by 8-15 percentage points. This is the single highest-ROI investment in a speculative decoding deployment.

- **Profile acceptance rate by prompt domain before setting K .** Open-ended creative tasks typically hit $\bar{\alpha} \geq 0.80$; multi-step reasoning and code with long logical dependencies can fall to $\bar{\alpha} \approx 0.55$. For low-acceptance domains, reduce K to 2-3 or disable speculation entirely; a run with $K = 5$ and $\bar{\alpha} = 0.40$ is slower than no speculation because draft cost exceeds the gain.
- **Quantise the draft, not the target.** Users tolerate no accuracy loss from the target. The draft's sole quality metric is acceptance rate. INT4 quantisation of a standalone 7B draft typically costs 3-5 percentage points in $\bar{\alpha}$ but halves its memory footprint and speeds up each draft step by 1.5-2 $\times$ -usually a net positive for the overall tokens-per-second figure.
- **Tree-based speculative decoding for Medusa.** Because Medusa's heads are independent, you can sample multiple candidates per position and evaluate them as a tree in a single target forward pass rather than a linear chain of K tokens. A tree with branching factor 2 at each of $K = 4$ levels evaluates 15 nodes (a complete binary tree) in one prefill call, and the best accepted path is selected. This partially compensates for Medusa's lower per-head acceptance rate without increasing target calls; implement this if you adopt Medusa.

How This Shows Up in Interviews

- **“You are designing speculative decoding for a 70B chat model on two H100s. The memory budget is tight. What draft strategy do you pick and why?”** A strong answer works through the memory budget explicitly (target weights, KV cache, headroom), rules out a standalone 7B draft as too large without quantisation, lands on EAGLE as the best acceptance-rate-per-GB option, and mentions distilling the EAGLE module from target outputs.
- **“What is the fundamental quality difference between Medusa and EAGLE?”** The answer is conditional independence: Medusa's head k does not see what heads $1, \dots, k-1$ predicted, so proposals are locally inconsistent. EAGLE's autoregressive draft rollout preserves the conditional structure, recovering most of a standalone draft's acceptance rate at a fraction of the memory cost.
- **“How do you measure whether speculative decoding is actually helping in production?”** Track three metrics: (1) mean acceptance rate $\bar{\alpha}$ per request class; (2) effective tokens-per-second compared to the direct-decode baseline; (3) p99 latency to first token, which should be unchanged since speculation only affects the generation phase. Alert on $\bar{\alpha}$ dropping below 0.55 for any significant traffic slice-this usually signals a distribution shift between the draft and target.
- **Senior follow-up: “An EAGLE draft module trained on general chat data is deployed with a target that was recently fine-tuned on a new coding dataset. Acceptance rates have dropped from 0.75 to 0.52. What do you do?”** The gap is a distribution mismatch introduced by the target fine-tune. The immediate fix is to fine-tune the EAGLE module on a sample of outputs from the updated target-even a 200M token run at low learning rate is usually enough to close most of the gap. Longer term, bake the EAGLE module fine-tune into the target update pipeline so both stay in sync. While the fix is in flight, fall back to direct decoding for the coding traffic slice to avoid negative latency impact.

Key Takeaways

- Standalone drafts give the highest acceptance rates but consume the most memory; EAGLE recovers near-standalone quality at ≈ 1 GB by conditioning on target features; Medusa is cheapest but suffers from conditional independence across heads.

- EAGLE dominates for most 70B deployments: the memory saving over a standalone draft is decisive, and the acceptance rate gap relative to Medusa is worth the marginal extra cost.
- Distilling the draft from target-model outputs is the single most impactful tuning lever-expect an 8-15 percentage point gain in $\bar{\alpha}$ over a generic pretrained draft.
- Quantise the draft aggressively (INT4 is fine); keep the target at bf16 or fp8.
- Monitor $\bar{\alpha}$ per traffic slice; disable speculation when it falls below ≈ 0.50 and retrain the draft whenever the target is fine-tuned.

18.2 Serving System Design

18.2.1 Continuous Batching and Selective Batching

Production serving of LLMs looks nothing like the clean batched inference your benchmarks describe. Real traffic is chaotic: requests arrive at random intervals, prompts span 50 to 50,000 tokens, and generations finish at unpredictable times. An interview question like “how would you serve a 70B model at high throughput with low latency?” is really asking whether you understand the scheduling and batching machinery that separates a demo from a production system. This topic appears consistently at senior and staff levels wherever teams own inference infrastructure, and the two mechanisms here-continuous batching and selective batching-are the architectural foundation of every serious open-source serving system today.

The naive approach is *static batching*: collect requests until the batch is full, run all sequences in lockstep until every one of them finishes, then start the next batch. The problem is obvious once stated: a request that generates 5 tokens holds a batch slot hostage for the full 512-step budget, and the GPU sits idle between batch arrivals while you wait for enough requests to form a full batch.

Continuous batching (also called iteration-level scheduling) fixes this by making the generation step the atomic scheduling unit. The worker generates one token for every active sequence, then yields control back to the scheduler. The scheduler inspects the waiting queue: any sequence that just generated an end-of-sequence token frees its slot, and a new request can be inserted immediately. The GPU is never blocked waiting for stragglers or for a batch to fill.

Selective batching addresses the second problem: variable-length sequences cannot be naively padded into a rectangular tensor when the length variance is large, because attention compute scales quadratically with sequence length and most of that padding is wasted work. The key observation is that attention and the feed-forward (MLP) layers behave differently. Attention is inherently per-sequence-each sequence has its own KV cache-so it must be dispatched separately for each sequence (or handled with variable-length kernels like FlashAttention-2’s `cu_seqlens` interface). MLP layers, however, apply the same weight matrices to every token regardless of which sequence the token belongs to. Given tokens from sequences of different lengths, you can simply flatten the batch dimension: reshape inputs from $[B, T_i, D]$ into $[\sum_i T_i, D]$ and process all tokens together as one large matrix multiply. Because the tokens from different sequences do not interact inside an MLP, this is mathematically identical to processing them separately but with much higher arithmetic intensity.

The key principle: continuously schedule at the granularity of individual generation steps, and batch MLP computation across all active tokens regardless of their sequence boundaries.

The throughput gains from continuous batching are compound. First, you eliminate idle GPU time from batch formation. Second, you eliminate the slot-wasting effect of stragglers: a 20-token request that finishes early frees its KV cache and batch slot after exactly 20 steps, not 512. Third, continuous batching naturally

increases the average number of concurrent sequences in steady-state, which raises the arithmetic intensity of MLP layers and drives the GPU toward its compute-bound regime.

Selective batching amplifies this by ensuring that the bulk of the computation-MLP layers constitute roughly two-thirds of total FLOPs in a standard transformer-is batched at maximum efficiency even when sequence lengths vary wildly. Without selective batching, a variable-length batch either suffers quadratic padding waste in attention or requires an inefficient loop over individual sequences in the MLP.

The alternative-waiting for a fixed batch and running to completion-gives lower memory pressure per step because fewer sequences are active at once, but it produces dramatically worse GPU utilization and higher latency variance. In practice, static batching delivers 20-30% GPU utilization for typical interactive workloads; continuous batching pushes that to 70-85%.

A Worked Example

Consider a 7B model on an H100, with a KV cache budget of 512 concurrent sequence slots, each pre-allocated to 2048 tokens maximum.

Under **static batching** with batch size $B = 16$: the batch is padded to the longest sequence in the batch. If the distribution of actual generation lengths is geometric with mean 256 tokens but maximum 2048, then at least one sequence in every batch of 16 will be near the maximum, so all 16 are padded to 2048 tokens. A request that actually generates 30 tokens occupies a slot for the full 2048 steps. Average slot utilization: $256/2048 \approx 12.5\%$.

Under **continuous batching** with the same memory budget: slots free as sequences finish. In steady state, the scheduler keeps 16 sequences running simultaneously. When sequence i generates its EOS at step 30, slot i is immediately assigned to the next waiting request. Effective throughput is:

$$\text{throughput} = \frac{B \cdot \bar{T}_{\text{gen}}}{\bar{T}_{\text{gen}} \cdot t_{\text{step}}} = \frac{B}{t_{\text{step}}}$$

where t_{step} is the per-step latency (roughly 8 ms per token for a 7B model with $B = 1$ on an H100, scaling with batch size). At $B = 16$, $t_{\text{step}} \approx 12$ ms, giving a steady-state throughput of $16/0.012 \approx 1333$ tokens/s, compared to ≈ 160 tokens/s with static batching at 12.5% slot utilization.

For selective batching, suppose active sequences have lengths [128, 512, 64, 256] tokens in their KV caches. The MLP forward pass flattens these into a single tensor of shape $[960, D]$ and runs two large matrix multiplies of size $[960, D] \times [D, 4D]$ and $[960, 4D] \times [4D, D]$. Arithmetic intensity is $\propto 960$, comfortably compute-bound on an H100 (threshold ≈ 300 for bf16). Attention is dispatched per-sequence with per-sequence KV lengths, using variable-length flash attention kernels.

What You Actually Decide in Practice

- **Schedule in token budget, not sequence count.** Set a maximum of, say, 8,192 active tokens across all sequences rather than a fixed batch size. This bounds both memory usage and per-step latency regardless of sequence length distribution. A batch of 64 short sequences and a batch of 4 long sequences both consume the same compute.
- **Chunk long prefills.** A single request with a 32,768-token prompt will monopolize the GPU for the entire prefill, stalling all ongoing generation steps and spiking inter-token latency for other users. Chunked prefill breaks the prompt into pieces of, say, 512 tokens per step, interleaving them with generation steps. This bounds prefill latency per step at the cost of slightly slower time-to-first-token for the long-prompt request. Use chunk sizes of 512-2048 tokens; smaller chunks hurt MLP efficiency.

- **Preemption when memory fills.** If a high-priority request arrives but there are no free KV cache slots, preempt the lowest-priority active sequence: save its KV cache to CPU DRAM (swap) or simply discard it and restart the sequence from scratch when a slot frees. Swap is preferable for long sequences (large KV cache already built); restart is preferable for short sequences (cheap to redo). The crossover is typically around 500-1000 tokens of context.
- **Priority tiers for multi-tenant serving.** Continuous batching makes priority trivially composable: each step, the scheduler fills available token budget starting from the highest-priority queue. Latency-sensitive interactive requests get first pick; batch-processing jobs fill remaining capacity. This is simpler to implement than priority in static batching because slots turn over every step.
- **Flash attention with variable-length support is non-negotiable.** Without FlashAttention-2's `varlen` kernels (or equivalent), selective batching for attention requires either a Python loop over sequences (terrible) or padding to the max length in the batch (defeats the purpose). Verify your serving stack uses `flash_attn_varlen_func` or equivalent before deploying.

How This Shows Up in Interviews

- **“How does vLLM achieve higher throughput than naive batching?”** Strong answer: iteration-level continuous batching reclaims slots as soon as a sequence finishes, eliminating straggler waste. Selective batching ensures MLP computation is always batched at full efficiency across all active tokens. Page-dAttention (the next subsection) then eliminates KV cache fragmentation to maximize the number of concurrent sequences.
- **“Why can’t you just pad all sequences to the same length and run a standard batch?”** Strong answer: padding wastes compute quadratically in attention (padded positions still participate in $QK^\top$), wastes memory for the padded KV cache, and lets long stragglers hold batch slots for all shorter sequences. At mean generation length of 256 and max 2048, effective utilization is $\approx 12\%$.
- **Senior follow-up: “If prefill is compute-limited and generation is memory-limited, what happens to generation latency when a large-batch prefill lands?”** Senior answer: a large prefill dominates the GPU for many steps, stalling all generation steps and causing inter-token latency (ITL) spikes for other users. The fix is chunked prefill: bound the prefill tokens per step to, say, 512, so that each step is a mix of prefill and generation. FlashAttention-2 supports this via the `varlen` interface, which handles sequences of mixed prefill and generation tokens in a single kernel call. The trade-off is slightly higher TTFT for the long-prompt request, which is acceptable if TTFT SLOs are looser than ITL SLOs.

Key Takeaways

- Continuous batching reclaims GPU slots every generation step, eliminating straggler waste and idle time between batches.
- Selective batching separates attention (per-sequence, variable-length kernels) from MLP (all tokens flattened into one large matrix multiply), keeping MLP compute-bound.
- Schedule by token budget, not sequence count, to bound per-step latency regardless of sequence length distribution.
- Chunked prefill prevents long prompts from monopolizing the GPU and spiking inter-token latency for concurrent generation requests.

- FlashAttention-2 variable-length kernels are the prerequisite for practical selective batching; confirm your stack supports them before benchmarking.

18.2.2 Paged Attention and Memory Fragmentation

The KV cache is the dominant consumer of GPU memory in inference, often eclipsing model parameter storage once you scale to reasonable batch sizes. A naive allocator that pre-reserves a contiguous chunk for each request wastes 20-40% of KV cache capacity to fragmentation-directly capping the number of concurrent sequences you can run and therefore your throughput ceiling. If you are interviewing for any role that touches inference infrastructure, expect to explain exactly how PagedAttention works and why it matters: this is the core mechanism behind vLLM and the design pattern that every serious serving system has since adopted or reimplemented. The concept is worth understanding deeply not just for interviews but because it reveals how much of systems work is rediscovering operating-system ideas in new contexts.

The fundamental problem is that you do not know how many tokens a request will generate before it starts. A pre-emptive allocator that reserves the maximum possible sequence length (say, 2048 tokens) wastes GPU memory proportional to the gap between that maximum and the actual generation. An allocator that starts small and grows dynamically requires copying the KV cache each time it outgrows its current allocation-expensive and disruptive.

PagedAttention borrows the solution that operating systems developed for process memory: virtual memory with paging. Physical DRAM is divided into fixed-size pages; a page table maps virtual addresses to scattered physical pages. A process sees a contiguous virtual address space, but the underlying physical layout is fragmented.

Applied to the KV cache, the translation is direct. The KV cache is divided into fixed-size *blocks*, each storing b tokens of KV state for all layers and all KV heads. A *block table* maps each sequence to an ordered list of physical block indices. When a sequence needs more KV storage, the allocator assigns the next free physical block from a pool-no contiguity requirement. When a sequence finishes, all its blocks are returned to the free pool immediately.

The memory cost of a single block is:

$$\text{bytes per block} = 2 \cdot b \cdot L \cdot N_{kv} \cdot H \cdot 2$$

where b is the block size in tokens, L is the number of layers, N_{kv} is the number of KV heads, H is the head dimension, and the final factor of 2 accounts for bf16 precision. The factor before b doubles for key and value.

Copy-on-write enables the second major feature: *prefix caching*. Many requests share a common prefix-a system prompt, a few-shot example block, or a retrieved document chunk. Without sharing, every request independently computes and stores the KV cache for that prefix, paying both compute and memory costs that scale linearly with the number of concurrent requests. PagedAttention gives each physical block a reference count. Blocks in a shared prefix region have reference count equal to the number of sequences using them; those sequences read from the block but never write to it. When a sequence generates its first token past the shared prefix, the scheduler forks it: new blocks are allocated for the diverging suffix, and the shared prefix blocks remain intact and untouched. This is copy-on-write, and it makes the KV cache cost of a repeated system prompt a one-time allocation regardless of how many requests share it.

Eliminate fragmentation first, then exploit prefix sharing: together these two mechanisms double or triple the effective batch size you can sustain within a fixed GPU memory budget.

Pre-PagedAttention serving systems suffered two kinds of fragmentation. *Internal fragmentation*: you

reserve 2048 tokens for a request that generates 80, wasting $1968/2048 \approx 96\%$ of its allocation. *External fragmentation*: the gaps between allocations that no single request can use, analogous to heap fragmentation in a long-running process. The vLLM paper measured 20-40% total KV cache waste from these effects on realistic workloads.

With block-based allocation, internal fragmentation is bounded by at most $b - 1$ tokens per sequence—at a block size of 16 tokens, that is at most 15 wasted token slots per sequence, negligible at any reasonable sequence length. External fragmentation is eliminated because blocks are fungible: any free block can be assigned to any sequence.

Prefix caching compounds the benefit. A 1,000-token system prompt on a 7B model (32 layers, 8 KV heads, head dimension 128, bf16) costs:

$$1000 \times 32 \times 8 \times 128 \times 2 \times 2 = 131 \text{ MB}$$

per request. With 100 concurrent requests all using the same system prompt, the naive cost is 13.1 GB—a significant fraction of the entire KV budget on an 80 GB GPU. Prefix caching reduces this to 131 MB, a $100\times$ reduction in KV memory for the shared prefix. This frees space for more concurrent sequences, which directly improves throughput.

The alternative—a bump allocator with padding—is simpler to implement but puts a hard ceiling on batch size that degrades with sequence length variance. A slab allocator (fixed-size objects) partially addresses external fragmentation but does not support prefix sharing.

A Worked Example

Setup: 7B model, H100 80 GB GPU, model weights in bf16: approximately 14 GB. KV cache budget: roughly 60 GB. Block size $b = 16$ tokens, $L = 32$ layers, $N_{kv} = 8$ KV heads, $H = 128$.

Bytes per block:

$$2 \times 16 \times 32 \times 8 \times 128 \times 2 = 2,097,152 \text{ bytes} \approx 2.1 \text{ MB}$$

Total physical blocks: $60 \text{ GB} / 2.1 \text{ MB} \approx 28,500$ blocks, representing $28,500 \times 16 = 456,000$ token slots of KV capacity.

Without PagedAttention (pre-allocation): Reserve $\lceil 2048/16 \rceil = 128$ blocks per sequence at request start. Maximum concurrent sequences: $28,500/128 \approx 222$. If actual generation length averages 256 tokens (needing 16 blocks), then $128 - 16 = 112$ blocks per sequence are reserved but unused: internal fragmentation $= 112/128 \approx 87.5\%$. Effective concurrent sequences at the 256-token average: ≈ 28 sequences actually needed simultaneously ($22 \times$ waste factor).

With PagedAttention (on-demand allocation): Each sequence holds exactly $\lceil T/16 \rceil$ blocks at any given step. At 256-token average generation: 16 blocks per sequence. Maximum concurrent sequences given 28,500 blocks: $28,500/16 \approx 1,781$. Practical limit is set by other memory constraints and scheduling, but the point is that fragmentation no longer caps batch size at ≈ 28 sequences.

With prefix caching: If all 100 concurrent requests share a 512-token system prompt: $\lceil 512/16 \rceil = 32$ blocks, reference count 100. Those 32 blocks serve all 100 sequences. Without prefix caching: $100 \times 32 = 3,200$ blocks. Savings: $3,168$ blocks $\approx 6.6 \text{ GB}$ freed for generation context.

What You Actually Decide in Practice

- **Block size is a tunable constant with real trade-offs.** Smaller blocks (8 tokens) reduce internal fragmentation but multiply the size of the block table and increase pointer-chasing overhead during attention, which must now gather non-contiguous blocks. Larger blocks (32 tokens) are more cache-friendly and reduce metadata overhead but waste up to 31 token slots per sequence. In practice, 16 tokens per block is the right default for mixed-length workloads; adjust toward 32 if your sequence length distribution is long-tailed and toward 8 if you serve many very short requests.
- **Prefix cache eviction needs a policy.** Shared prefix blocks are held in cache indefinitely under the assumption that the prefix is fixed. For multi-tenant serving where each tenant has a different system prompt, you need LRU eviction on prefix blocks, triggered when the free block pool drops below a watermark (e.g., 10% of total blocks). Evicting a prefix block invalidates all sequences that depend on it—force a KV recompute for those sequences or preempt them. Radix attention (used in SGLang) automates this with a prefix trie and LRU eviction at the subtree level.
- **CPU swap is a last resort, not a primary strategy.** When the free block pool is exhausted, the scheduler can swap a paused sequence’s KV blocks to CPU DRAM over PCIe or NVLink. NVLink bandwidth is roughly 900 GB/s bidirectional (H100 SXM); PCIe 4.0 is roughly 32 GB/s. At 2.1 MB per block and 16 blocks for a 256-token context: 33.6 MB transferred over PCIe takes ≈ 1 ms—tolerable once but not repeatedly. Prefer preemption-and-restart for short sequences (under 500 tokens) and swap for long sequences where the recompute cost exceeds the transfer cost.
- **Block table lookup is on the critical path in the attention kernel.** The attention kernel must translate logical block indices to physical addresses for each sequence. This adds one level of indirection per block boundary. Verify that your attention kernel handles non-contiguous blocks natively (as vLLM’s `paged_attention` kernel does) rather than copying KV blocks into a contiguous staging buffer before attention—the latter negates the memory savings.
- **Chunked prefill interacts with block allocation.** When a 1,024-token prompt arrives and is chunked into 512-token pieces, the first chunk allocates $\lceil 512/16 \rceil = 32$ blocks at step one, and the second chunk allocates another 32 blocks at step two, before any generation blocks are needed. Account for this prefill block reservation in your watermark-based eviction policy, otherwise a burst of long-prompt requests can exhaust the block pool before generation even starts.

How This Shows Up in Interviews

- **“Explain how PagedAttention works and why it matters.”** Strong answer: describe block-based KV cache allocation, the block table mapping logical to physical blocks, copy-on-write for prefix sharing, and quantify the fragmentation reduction. Concrete numbers matter: pre-PagedAttention systems waste 20-40% of KV cache; with block size 16, internal fragmentation is at most 15 tokens per sequence.
- **“Why does KV cache fragmentation hurt throughput?”** Strong answer: fragmentation reduces the number of concurrent sequences you can hold in GPU memory. Fewer concurrent sequences means lower arithmetic intensity for MLP layers, pushing the GPU toward its memory-bound regime. The throughput loss is multiplicative: halving the effective batch size roughly halves throughput at the operating point where the system is memory-bound.
- **Senior follow-up: “How does SGLang’s radix attention extend PagedAttention, and when does it help most?”** Senior answer: radix attention maintains a *prefix trie* over all KV-cached sequences across

all current and recent requests, not just those that share the same system prompt. Any common prefix subtree-including multi-turn conversation history and RAG-retrieved context-is automatically deduplicated. LRU eviction at the subtree level keeps the trie bounded. The gain is largest in multi-turn chat (where conversation history grows and is shared across turns) and RAG applications (where the same retrieved documents recur across users). The trade-off is trie management overhead and the risk of cache thrashing if prefix diversity is high relative to the block pool size.

Key Takeaways

- PagedAttention divides the KV cache into fixed-size blocks (typically 16 tokens), allocated on demand and returned immediately when a sequence finishes-eliminating fragmentation that previously wasted 20-40% of KV cache capacity.
- Copy-on-write lets multiple sequences share KV blocks for a common prefix at no extra memory cost; prefix caching turns a repeated system prompt from a per-request allocation to a single shared allocation.
- The block table lookup adds one level of indirection in the attention kernel; use a kernel that handles non-contiguous blocks natively rather than copying to a staging buffer.
- Block size, prefix eviction policy, and CPU-swap thresholds are the three key tunable parameters in production; defaults of 16 tokens/block, LRU eviction at 10% free watermark, and swap-for-long/restart-for-short cover most workloads.
- Radix attention (SGLang) extends prefix caching to a full prefix trie, maximizing cache hits for multi-turn conversations and RAG pipelines where prefix overlap spans multiple users.

Chapter 19 Compression Techniques for Deployment

19.1 Quantization

19.1.1 INT8 Quantization with Outlier Handling (LLM.int8())

Imagine you have trained a 70B parameter model and now you need to serve it. At BF16, those weights consume 140 GB of memory—more than a single H100’s 80 GB HBM. INT8 would halve that to 70 GB, letting you serve on a single node. But every engineer who has tried naively packing a large transformer into INT8 has watched perplexity spike and downstream task accuracy collapse. Understanding why that happens, and how to fix it, is a question I have seen asked at nearly every frontier lab interview touching inference or deployment.

Quantization maps a floating-point tensor to a lower-precision integer representation. The simplest scheme, *absmax quantization*, works as follows: given a weight vector $\mathbf{w} \in \mathbb{R}^n$ stored in FP16, compute the scale $s = \max(|\mathbf{w}|)$ and produce the INT8 vector

$$\mathbf{w}_q = \text{round}\left(\frac{\mathbf{w}}{s} \cdot 127\right), \quad \mathbf{w}_q \in [-127, 127].$$

To dequantize: $\hat{\mathbf{w}} = \mathbf{w}_q \cdot s/127$. Memory shrinks by $2\times$ (2 bytes per weight in BF16, 1 byte in INT8). Bandwidth shrinks by the same factor, which directly translates to lower generation latency in the memory-bound regime.

The catch is *outlier activations*. Research has shown that in transformer models at or above 6.7B parameters, a small fraction of hidden-dimension channels—typically fewer than 1% of all feature dimensions—develop systematically large magnitudes, often $10\text{--}100\times$ larger than the median channel. These outliers emerge consistently across every token processed by the model. When you include even one such outlier in the absmax scale computation, the quantization grid becomes too coarse for every other channel, and the accumulated rounding error destroys model quality.

The LLM.int8() paper addresses this with a *mixed-precision decomposition*. Given the weight matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and the activation matrix $X \in \mathbb{R}^{B \times d_{\text{in}}}$, partition the columns corresponding to outlier channels into an index set $\mathcal{O}$ (typically $|\mathcal{O}| < 0.01 \cdot d_{\text{in}}$). Then:

$$XW = X_{\mathcal{O}}W_{\mathcal{O}}^{\top} + X_{\bar{\mathcal{O}}}W_{\bar{\mathcal{O}}}^{\top},$$

where the first term is computed in FP16 (covering outlier columns) and the second in INT8 (covering the vast majority). The two partial products are accumulated back into FP16. Because outlier columns are identified offline from a small calibration set, no runtime search is required.

INT8 quantization at scale requires explicit outlier handling: a tiny fraction of channels disrupts the entire quantization grid, and the fix is to route those channels through a separate FP16 compute path.

The key property that makes the decomposition practical is that outlier channels are *consistent across tokens*. The same handful of columns that blow up in activation magnitude for one input tend to blow up for all inputs processed by a given layer. That means you can profile a few hundred calibration samples, identify the outlier channel indices once, and bake the decomposition into your serving kernel. There is no per-token routing overhead.

The trade-off is speed versus capacity. Because the FP16 path involves a sparse scatter-gather across out-

lier columns, `LLM.int8()` is typically 10-20% slower at runtime than a native BF16 forward pass on the same hardware. It is not a speed optimization—it is a memory-capacity optimization. The value proposition is: run a 175B model on 8×80 GB A100s instead of 16, or a 70B model on a single 8×80 GB node instead of two.

Compared to pure INT8 without outlier handling, mixed-precision decomposition recovers most of the quality loss (within 1-2 points on common benchmarks) at the cost of that modest throughput penalty. Compared to staying in BF16, you trade 10-20% throughput for a 2× reduction in memory footprint and the ability to fit a larger model or serve a larger batch.

A Worked Example

Consider a single linear layer in a 13B model: $d_{\text{in}} = d_{\text{out}} = 5120$, batch size $B = 16$, BF16. The weight matrix W is 5120×5120 , consuming $5120 \times 5120 \times 2 \approx 52$ MB per layer. Across 40 transformer layers (MLP up-projection, down-projection, and the three attention projections—roughly five large matrices per layer), total weight memory is around 26 GB, consistent with a 13B BF16 model.

After profiling 128 calibration samples, we identify 26 outlier columns (roughly 0.5% of 5120). These 26 columns remain in BF16: $26 \times 5120 \times 2 = 0.27$ MB per matrix. The remaining 5094 columns are quantized to INT8: $5094 \times 5120 \times 1 = 26$ MB. Total per-matrix: 26.3 MB versus the original 52 MB—a near 2× reduction with 99.5% of the compute running in INT8.

At inference, the forward pass for this layer does:

1. Load the 26-column FP16 slice of W and the corresponding slice of X ; compute $X_O W_O^\top$ in FP16.
2. Load the remaining 5094 columns as INT8; dequantize on-the-fly; compute $X_{\bar{O}} W_{\bar{O}}^\top$ in INT8.
3. Add the two partial products in FP16 to produce the output.

Total bytes transferred per token: ≈ 26 MB (INT8 path dominates), versus 52 MB in BF16—halving memory bandwidth, which maps directly to halved generation latency per layer at batch size 1.

What You Actually Decide in Practice

- **Outlier threshold.** A practical heuristic is to flag columns whose maximum absolute activation magnitude exceeds 6 standard deviations above the layer mean across your calibration set. Too low a threshold wastes compute on FP16; too high and you leave genuine outliers in the INT8 path.
- **LLM.int8() is a capacity tool, not a speed tool.** If your model fits comfortably in BF16, do not use it. Use it when you are trying to serve a model on fewer GPUs or to increase maximum batch size.
- **Calibration data quality matters.** Outlier detection is distribution-sensitive. A calibration set drawn only from short chat exchanges will misidentify outliers when the model processes long code contexts. Use diverse calibration data representative of your actual traffic.
- **Kernel support.** The mixed-precision path requires custom CUDA kernels (bitsandbytes is the canonical open-source implementation). Naively implementing this in standard PyTorch will not give you the memory savings because PyTorch may still keep the original FP16 weights around.
- **Interaction with GQA and MLA.** Outlier patterns differ across projection types. KV projections in models using grouped-query attention typically have fewer outliers than the value projections in the MLP. Profiling each projection separately rather than using a single global threshold improves quality.

How This Shows Up in Interviews

- **“A 70B model is too large for your H100 fleet. How do you fit it?”** A strong answer names three levers: model parallelism (tensor or pipeline), lower precision (INT8, INT4), and quantization schemes (LLM.int8() for memory-safe INT8). The candidate should quantify: 70B at BF16 is 140 GB; at INT8 it is 70 GB, fitting on one 8-way H100 node.
- **“What goes wrong when you naively quantize a large transformer to INT8?”** Strong answer: outlier activations in 1% of channels inflate the absmax scale, making the INT8 grid too coarse for the remaining 99% of channels. The fix is mixed-precision decomposition for outlier columns.
- **“Why do outliers appear more in larger models?”** This is the senior follow-up. The empirical finding is that outlier channels emerge at model scales of roughly 6.7B parameters and above, growing more extreme with scale. The intuition is that larger models rely on specific feature dimensions to carry sharp, high-magnitude signals—a form of emergent specialization. A senior answer connects this to the phase transitions observed in scaling laws and notes that the outlier columns correspond to attention heads or MLP neurons that have learned to function as routing or gating mechanisms.

Key Takeaways

- INT8 absmax quantization halves weight memory but causes severe accuracy degradation in models at or above 6.7B parameters due to outlier activation channels.
- LLM.int8() uses mixed-precision decomposition: $\sim 1\%$ of columns in FP16, the rest in INT8, identified offline from a calibration set.
- The method is a *capacity* optimization—it enables fitting larger models in GPU memory—not a throughput optimization; it typically runs 10-20% slower than native BF16.
- Outlier channels are stable across inputs, making offline profiling viable for production deployments.
- Use LLM.int8() when memory is the binding constraint; move to AWQ or INT4 schemes when you also need throughput gains.

19.1.2 Activation-Aware Weight Quantization (AWQ) and INT4

Once LLM.int8() gets your 70B model into memory, the next interviewer question is: “Now make it fast.” LLM.int8() is a capacity tool—it typically runs 10-20% slower than BF16 because the mixed-precision scatter-gather adds overhead. AWQ (Activation-Aware Weight Quantization) targets a different objective: squeezing throughput by going to INT4, which halves memory bandwidth again relative to INT8 and delivers $2\text{--}3\times$ end-to-end generation speedup over BF16 with minimal accuracy loss. This question shows up when an interviewer has already confirmed you can fit the model and wants to know whether you understand the path to production throughput targets.

The core insight of AWQ is that weights are not equally important for quantization error. Given a linear layer $Y = XW$, a small fraction of input channels—those multiplied by large activation magnitudes—contribute disproportionately to the output. When you quantize a salient weight channel j coarsely, the output error is $\Delta Y_j = x_j \cdot \Delta w_j$. If x_j is large, even a small per-weight quantization noise Δw_j causes significant output error. Non-salient channels with small x_j can absorb much larger Δw_j without visible effect.

AWQ identifies salient channels by running a small calibration set through the model. For each input

channel j , compute the average activation magnitude:

$$\bar{s}_j = \mathbb{E}_{x \sim \mathcal{D}}[|x_j|].$$

Channels with large $\bar{s}_j$ are salient. Rather than routing them to a separate FP16 compute path (the LLM.int8() strategy), AWQ applies a per-channel scale $s_j \propto \bar{s}_j^\alpha$ to the weights *before* quantization and compensates by dividing activations by the same scale at runtime:

$$XW = \underbrace{(X \cdot \text{diag}(\mathbf{s})^{-1})}_{X'} \cdot \underbrace{(\text{diag}(\mathbf{s}) \cdot W)}_{W'},$$

where W' is the scaled weight matrix that gets quantized to INT4, and the factor $\text{diag}(\mathbf{s})^{-1}$ on the activation side is folded into the preceding layer normalization or fused into the preceding kernel—it costs nothing extra at runtime. By setting $\alpha \approx 0.5$, salient channels are scaled up, giving them finer quantization resolution, while non-salient channels are barely affected.

In practice, weights are quantized in *groups*: every 128 consecutive weights share one FP16 scale and zero-point, a scheme called group quantization with $g = 128$. This prevents a single outlier weight from degrading an entire row and is the dominant production choice. The effective bit-width is slightly above 4 (128 INT4 values plus one FP16 scale and zero-point per group ≈ 4.25 bits per weight) but the storage overhead is negligible.

AWQ finds which weights matter most, scales them up before INT4 quantization so the grid is finer where errors count, and absorbs the correction into adjacent layers for free.

The bandwidth argument is direct. Inference generation is bandwidth-bound: every generation step loads the full weight matrix once to multiply against a single token. Cutting weight precision from BF16 (2 bytes) to INT4 (0.5 bytes) reduces bytes transferred per layer by $4\times$. In the pure bandwidth limit that translates to $4\times$ lower latency per token. Real systems achieve $2\text{--}3\times$ because of kernel launch overhead, the cost of dequantizing INT4 to FP16 on the fly, and the residual bandwidth of FP16 group scales.

AWQ's accuracy advantage over naive INT4 comes from the weighted error reduction. Scaling w_j by s_j before quantization makes the post-dequantization noise $\Delta w'_j = \Delta w_j / s_j$, so the output error for salient channel j drops from $x_j \Delta w_j$ to $x_j \Delta w_j / s_j$ —reduced by exactly the scale factor. Non-salient channels with small x_j are unaffected because their output error was already small. AWQ searches the exponent α over a coarse grid (0.0 to 1.0 in steps of 0.1) to minimize calibration-set quantization error, making the procedure fast and calibration-data-efficient.

The design contrast with LLM.int8() is worth internalizing. LLM.int8() targets INT8 weights *and* handles activation-outlier channels with a separate FP16 compute path; the result is a model that fits in memory but does not run faster. AWQ targets INT4 weights only, leaving activations in FP16 (the W4A16 regime). W4A16 is simpler—weight quantization is far more tractable than activation quantization because weights are static—and it targets throughput rather than just capacity.

A Worked Example

Consider a 7B model: $d = 4096$, 32 transformer layers, MLP hidden dimension $F = 11008$, BF16 baseline.

Memory. BF16 weight footprint: $7 \times 10^9 \times 2 \approx 14$ GB. After AWQ INT4 with group size 128: $7 \times 10^9 \times 0.5 = 3.5$ GB for INT4 weights, plus $7 \times 10^9 / 128 \times 2 \approx 0.11$ GB for FP16 group scales. Total: ≈ 3.6 GB—a $3.9\times$ reduction.

Per-layer bandwidth. Take the MLP up-projection: $W \in \mathbb{R}^{11008 \times 4096}$. BF16 storage: $11008 \times 4096 \times 2 = 90.2$ MB. AWQ INT4 storage: $11008 \times 4096 \times 0.5 = 22.5$ MB for INT4 weights, plus $11008 \times 4096 / 128 \times 2 =$

0.7 MB for scales. Total: ≈ 23.2 MB.

Latency per token, H100 (3.35 TB/s HBM bandwidth):

$$t_{\text{BF16}} = \frac{90.2 \text{ MB}}{3.35 \text{ TB/s}} \approx 27 \mu\text{s}, \quad t_{\text{AWQ}} = \frac{23.2 \text{ MB}}{3.35 \text{ TB/s}} \approx 7 \mu\text{s}.$$

Across 32 layers and five large weight matrices per layer (Q, K, V, O projections plus the MLP stack), the per-token generation step drops from ≈ 4.3 ms to ≈ 1.1 ms in the bandwidth limit—roughly $3.9\times$. Accounting for dequantization kernel overhead and the FP16 activation path, measured throughput on published benchmarks sits at $2.5\text{--}3\times$, consistent with the theoretical upper bound.

What You Actually Decide in Practice

- **Group size is your accuracy-vs-speed knob.** Group size 128 is the standard: it limits scale overhead to $< 2\%$ of total weight storage and keeps accuracy within 1-2 points of BF16 on most benchmarks. For models under 3B parameters, tighten to group size 64 to compensate for the stronger relative compression. Widening to 256 saves a bit more memory but hurts accuracy noticeably on code and math tasks where precision matters.
- **W4A16 before W4A8.** Quantizing activations (W4A8) further reduces compute cost but requires per-token activation scales that must be computed at runtime, adds latency at low batch sizes, and is sensitive to input distribution shifts. Start with W4A16, validate accuracy on your eval suite, and consider W4A8 only if you have a hardware-specific reason (e.g., INT8 tensor cores running at $2\times$ the FLOP rate of FP16 on your device).
- **Do not quantize embedding tables or the LM head.** These layers are accessed once per forward pass in a different access pattern and are small relative to the total parameter count. Quantizing them saves negligible bandwidth and carries real accuracy risk for low-frequency tokens whose embeddings can be hard to represent in INT4.
- **Calibration data should match your serving distribution.** AWQ activation magnitude estimates stabilize at 128-512 samples. Use diverse data spanning your expected input types—chat, code, documents—so salient channels are correctly identified for all input modes. A calibration set drawn purely from short chat turns will misidentify salience for long-context or structured-output use cases.
- **Stack AWQ on top of GQA for maximum throughput gain.** GQA already reduced KV-projection head counts; AWQ cuts the projection weight bandwidth by another $4\times$. The two compose multiplicatively on the KV projection layers and are independent on MLP layers. A LLaMA-3-70B model with GQA and AWQ INT4 can serve on a single 8×80 GB H100 node with throughput competitive with a BF16 LLaMA-3-7B on the same hardware.

How This Shows Up in Interviews

- **“We need $3\times$ throughput improvement on a production 7B model without retraining. What is your plan?”** Strong answer: quantize weights to INT4 using AWQ with group size 128. Generation is bandwidth-bound; INT4 cuts weight bandwidth by $4\times$ relative to BF16; AWQ’s activation-magnitude-based scaling preserves accuracy within 1-2 points. Expect $2\text{--}3\times$ real-world speedup with near-zero accuracy loss on standard benchmarks.
- **“What is the difference between LLM.int8() and AWQ?”** Strong answer: LLM.int8() is a capacity tool—it targets INT8 with a mixed FP16/INT8 split on activation-outlier channels; it fits models that do

not otherwise fit in memory but typically runs slightly *slower* than BF16. AWQ is a throughput tool-it targets INT4 weight-only quantization guided by activation magnitudes; it genuinely makes generation faster by reducing bandwidth. They solve different problems and can coexist (though INT4 alone usually dominates once both goals are met).

- **Senior follow-up: “Why does scaling weights by activation magnitude before quantization actually reduce output error? Walk me through the math.”** Strong senior answer: the per-token quantization output error for channel j is $\epsilon_j = x_j \cdot \Delta w_j$, where $\Delta w_j \approx \|w_j\|_\infty / 127$ (the INT8 step size). By premultiplying w_j by $s_j > 1$ before quantization, the post-dequantization weight error becomes $\Delta w'_j = \Delta w_j / s_j$, reducing output error to $\epsilon'_j = x_j \cdot \Delta w_j / s_j$. Setting $s_j \propto \bar{s}_j^\alpha$ with $\alpha \approx 0.5$ balances error reduction for salient channels against the distortion imposed on non-salient channels. The α search is cheap: it is a 1D grid search over ten values using only the calibration set, taking seconds per layer.

Key Takeaways

- AWQ identifies weight channels that matter most by their corresponding activation magnitudes, scales them up before INT4 quantization for finer resolution, and absorbs the inverse scale into the preceding layer at zero runtime cost.
- INT4 weight quantization with group size 128 (W4A16) cuts weight memory by $\approx 4\times$ versus BF16 and delivers 2-3 $\times$ practical generation throughput improvement by reducing bandwidth in the memory-bound generation regime.
- Group quantization is non-negotiable at INT4: without it, a single outlier weight degrades the entire row’s resolution, causing accuracy collapse on long-tail tokens.
- AWQ is a throughput optimization; LLM.int8() is a capacity optimization. Use AWQ when you need faster tokens per second from a model that already fits in memory; use LLM.int8() when memory is the binding constraint.
- AWQ and GQA compose multiplicatively on KV projections; combining both is a practical path to serving 70B-class models at 7B-class latency on a single node.

19.2 Pruning and Distillation

19.2.1 Structured Pruning of Layers, Heads, and Channels

In production, the pressure to cut serving costs often arrives after a model is already deployed and has real traffic. The question is not “can we train a smaller model?” but “can we make this specific 15B checkpoint cheaper without starting over?” Structured pruning is the tool that answers yes: remove entire transformer layers, attention heads, or MLP hidden channels from a trained model and hand back a smaller, denser model that runs on unmodified kernels with genuine latency and memory savings. This topic comes up regularly in senior inference interviews because it sits at the intersection of model architecture, importance estimation, and deployment economics.

Unstructured pruning zeros individual weights scattered throughout a weight matrix. Modern hardware cannot exploit that sparsity efficiently-you still load every weight from HBM before zeroing it in registers, so you get no memory bandwidth reduction and no throughput gain. Structured pruning removes entire contiguous blocks: a full transformer layer, all positions of one attention head, or a set of MLP hidden-dim channels. The

result is a model with genuinely smaller dense matrices, producing linear reductions in parameter count, memory footprint, and compute.

The standard three-step pipeline is:

1. **Score.** Run a small calibration set (typically 128-512 representative sequences) through the model and compute an importance score for each structural unit.
2. **Remove.** Zero out or drop the lowest-scoring units until you reach the target parameter budget.
3. **Repair.** Recover lost accuracy by fine-tuning or distillation from the original model (covered in the next subsection).

Several importance criteria are used in practice. The simplest is *activation magnitude*: compute the mean ℓ_2 norm of a unit's output across the calibration set-low-magnitude units contribute little to downstream residuals and are safe to remove. A more principled criterion is the first-order Taylor approximation to the loss increase caused by removing unit u :

$$\Delta\mathcal{L}_u \approx \sum_{i \in u} |g_i \cdot w_i|,$$

where g_i is the gradient and w_i the weight at position i . For *layer removal* specifically, a cheap proxy is the cosine similarity between a layer's input and output activation vectors: a layer whose output is nearly identical to its input adds almost nothing to the residual stream and is the first candidate for deletion.

Structured pruning converts a trained model into a smaller one that runs on unmodified kernels, trading a controlled accuracy loss for proportional reductions in parameter count, memory, and inference latency.

The alternative-training a smaller model from scratch-requires the full pre-training compute budget. A 15B model that took weeks on hundreds of GPUs cannot be reproduced in the time a product team has. Structured pruning plus distillation recovery delivers a comparable result in one to two orders of magnitude less compute.

Structured pruning also composes naturally with quantization. Quantization reduces bytes per weight (e.g., BF16 to INT4) but leaves the number of matrix operations unchanged. Pruning reduces the number of operations outright. Applying both-prune first to shrink layer dimensions, then quantize the surviving weights-stacks the gains without interference, as long as you run quantization calibration after pruning and not before.

The drawback compared to unstructured pruning is that structured pruning is coarser: you must remove entire heads or layers even if only a fraction of their weights are dispensable. This means a given parameter reduction target costs more accuracy points than unstructured pruning on the same model. The tradeoff is that hardware speedup from structured pruning is real and immediate, whereas the theoretical speedup from unstructured pruning is largely unrealized on current accelerators.

A Worked Example

Consider an NVIDIA-style compression of a 15B Llama-family model: 32 layers, $d_{\text{model}} = 5120$, 40 attention heads, MLP hidden dim $d_{\text{ff}} = 13,696$.

Step 1 - score layers. Run 256 sequences from a mixed pre-training corpus. For each of the 32 layers compute the mean cosine similarity between the residual stream at the layer's input and output. Layers 14-18 (mid-network) consistently show cosine similarity above 0.97-nearly identity transformations. These are flagged for removal.

Step 2 - score heads and channels. For each attention head compute the mean output norm; for each MLP hidden channel compute the mean activation norm. Rank both globally.

Step 3 - remove to target. Remove 8 of the 32 layers (the 8 with highest input-output cosine similarity).

Trim the MLP hidden dim from 13,696 to 9,216 by dropping the lowest-norm channels. Optionally reduce KV heads from 40 to 8 (GQA-style) for additional KV cache savings.

After pruning: approximately 8.1B parameters. MMLU immediately after pruning drops from 65% to roughly 58%-a 7-point raw gap. Memory for weights in BF16 shrinks from 30 GB to 16 GB, and generation throughput at batch size 32 on an H100 increases from $\approx 3,200$ tokens/s to $\approx 5,800$ tokens/s, a $1.8\times$ improvement, before any distillation recovery.

After a short distillation pass (covered in the next subsection), MMLU recovers to roughly 64.5%-within half a point of the original 15B, at 53% of the parameter count and $1.8\times$ the throughput.

What You Actually Decide in Practice

- **Prune layers before heads or channels.** Layer removal is the highest-leverage decision: each removed layer eliminates $\approx 2 \times d_{\text{model}} \times d_{\text{ff}}$ parameters in one cut. Score layers first, remove them, then re-score heads and channels on the surviving architecture. Pruning all dimensions simultaneously compounds errors.
- **Score on calibration data that matches your deployment distribution.** A calibration set that is too short (< 128 sequences) or domain-mismatched produces noisy importance scores. If you prune a coding model with web-text calibration data, you will over-prune heads that are important for code structure and miss heads that are dead weight for your actual traffic.
- **Use cosine similarity for layer removal, Taylor sensitivity for head and channel removal.** Cosine similarity is fast and requires no gradients. Taylor sensitivity is more accurate for fine-grained removal but requires a backward pass. This split gives you the best of both at reasonable cost.
- **Keep a validation set of difficult prompts and check it after each pruning step.** Accuracy on standard benchmarks like MMLU is not sensitive enough to catch regressions in long-context or multi-step reasoning tasks. Run at least one specialized eval (e.g., a chain-of-thought benchmark) before committing to the pruned structure.
- **Never quantize before pruning.** Some quantization calibration methods (AWQ-style) compute per-channel weight scales from activation statistics. If you prune channels after calibration, those scales are stale. The correct order is always: prune $\rightarrow$ distill $\rightarrow$ quantize.

How This Shows Up in Interviews

- **“How would you compress a 70B model to fit on a single 80 GB H100?”** The 70B model in BF16 is ≈ 140 GB, which does not fit. Structured pruning to ~ 35 B active parameters brings the weight footprint to ~ 70 GB; add INT4 quantization to land at ~ 35 GB with room for KV cache. Distill from the 70B teacher to recover accuracy. A strong answer names this sequence and gives the memory arithmetic.
- **“What is the difference between structured and unstructured pruning, and why does it matter for inference?”** Unstructured pruning produces sparse weight matrices that do not reduce memory bandwidth on dense-matrix hardware. Structured pruning removes whole rows, columns, heads, or layers, producing smaller dense tensors with proportional speedup on standard kernels. This is the answer that separates a candidate who has thought about hardware from one who has only read papers.
- **Senior follow-up: “You removed 8 of 32 layers from a 15B model and see a 10-point MMLU drop. Before running distillation, what do you investigate?”** First, break down the MMLU drop by category: if it is uniform, the issue is capacity loss that distillation can recover. If it is concentrated in world knowl-

edge or multi-hop reasoning questions, you may have removed layers that serve as key associative recall steps. In that case, swap in the two or three layers with the highest Taylor sensitivity scores instead of the ones with the highest cosine similarity. Cosine similarity is a coarse proxy; for reasoning-heavy tasks, sensitivity-based scoring is more reliable.

19.2.2 Distillation Recovery from a Pruned Initialization

Structured pruning always leaves you with a model that is noticeably worse than the original. The question every interview panel eventually asks is: how do you get the accuracy back, and what does it cost? Knowledge distillation from the pruned initialization is the standard answer, and it shows up at every seniority level in LLM inference interviews because it bridges the architectural decision to prune and the quality guarantee you owe a product team. What separates a strong answer from a weak one is understanding why distillation from a pruned init is fundamentally different from-and cheaper than-training a small model from scratch.

Standard cross-entropy fine-tuning trains a model against hard one-hot labels: at each token position, only the ground-truth token contributes gradient. Knowledge distillation instead trains the student to match the teacher’s full probability distribution over the vocabulary. The training loss is the forward KL divergence between the teacher’s next-token distribution p_T and the student’s p_S :

$$\mathcal{L}_{\text{KD}} = \sum_t D_{\text{KL}}(p_T(x_t \mid x_{<t}) \parallel p_S(x_t \mid x_{<t})).$$

In practice, a small cross-entropy term is mixed in to prevent the student from ignoring the ground-truth signal entirely:

$$\mathcal{L} = \mathcal{L}_{\text{KD}} + \lambda \mathcal{L}_{\text{CE}}, \quad \lambda \approx 0.1.$$

The key insight is not the loss function-it is the initialization. When you prune a 15B model to produce an 8B student, the surviving weights are already the best 8B-worth of weights the teacher had. The student starts with a loss that is already near the pruning-induced gap, not at random initialization. Training from that point requires only a small correction, not a full optimization trajectory from scratch.

An optional extension is *feature-level distillation*: for each student layer l paired with a corresponding teacher layer l' , add a loss term

$$\mathcal{L}_{\text{feat}} = \sum_l \left\| h_S^{(l)} - W_{\text{proj}} h_T^{(l')} \right\|_2^2,$$

where W_{proj} is a learnable linear projection to align dimensions when the teacher and student hidden sizes differ. This aligns intermediate representations and accelerates accuracy recovery but requires the teacher to remain resident in GPU memory for every training step.

Distilling from a pruned initialization is strictly better than training a small model from scratch because you are correcting a small perturbation in an already-converged loss landscape, not optimizing from a random starting point.

Training a fresh 8B model from scratch requires hundreds of billions to a trillion tokens-roughly the same scale as pre-training the original. Distillation from a pruned 15B initialization recovers most of the accuracy loss with far fewer tokens, because the student has already seen the effective gradient signal accumulated over the teacher’s full pre-training. NVIDIA’s Minitron experiments show recovery to within one MMLU point using roughly 128B tokens of distillation, compared to $\sim 1\text{T}$ tokens for scratch training to equivalent quality-a $\sim 8\times$ reduction in the recovery budget alone.

The soft probability targets from the teacher provide a dense per-step gradient signal. A one-hot label

on a 32,000-token vocabulary gives exactly one nonzero gradient component per position. The teacher’s distribution typically assigns non-negligible probability to dozens of plausible continuations, each contributing gradient. This richness is especially helpful for recovering the student’s early layers, which must now handle token interactions that the pruned architecture routes differently than the original.

Versus other recovery strategies—continued pre-training, LoRA fine-tuning, task-specific fine-tuning—distillation is the only approach that recovers general-purpose capability rather than task-specific accuracy. Fine-tuning on a downstream task recovers performance on that task but does not restore the broad capability distribution of the original model. If you need a general-purpose compressed model, distillation is the right tool.

A Worked Example

Setup: teacher is a 15B Llama-family model; student is the 8.1B model produced by structured pruning as described in the previous subsection. Pre-pruning MMLU: 65%. Post-pruning MMLU: 58%. Target: recover to within 1 point of 65%.

1. **Assemble the calibration corpus.** Sample 128B tokens from the same distribution as pre-training—mixed web text, code, multilingual data—in the original mixture ratios. Do not use only downstream task data; you want to recover general capability, not overfit to a benchmark.
2. **Configure the loss.** Use output-level KD with temperature $\tau = 2$ (softer distributions help the student learn inter-token relationships). Set $\lambda = 0.1$ for the cross-entropy term.
3. **Estimate the compute cost.** Each distillation step requires:

- One teacher forward pass: $\approx 2 \times 15\text{B} \times T$ FLOPs per batch
- One student forward pass and backward pass: $\approx 6 \times 8\text{B} \times T$ FLOPs per batch

Over 128B tokens with sequence length 4,096, total compute is roughly $6 \times 8 \times 10^9 \times 128 \times 10^9 \approx 6 \times 10^{21}$ FLOPs for the student, plus $\approx 2 \times 10^{21}$ for teacher inference. Compare to scratch training an 8B model on 1T tokens: $6 \times 8 \times 10^9 \times 10^{12} \approx 5 \times 10^{22}$ FLOPs—roughly 6× more compute for the scratch run.

4. **Results.** After the 128B-token distillation pass:
 - MMLU: 64.5% (recovered 6.5 of the 7-point pruning gap)
 - Coding benchmark (MBPP): recovers from 47% to 54% against teacher 55%
 - Generation throughput at batch size 32: $\approx 5,800$ tokens/s vs. $\approx 3,200$ tokens/s for the original 15B—an $\approx 1.8\times$ improvement

The remaining 0.5-point MMLU gap is the irreducible cost of reduced capacity. Extending distillation to 256B tokens closes it by another 0.2 points but with sharply diminishing returns.

What You Actually Decide in Practice

- **Start with output-level KD; add feature-level distillation only for persistent gaps.** Feature-level distillation keeps the teacher resident in memory for every training step, roughly doubling GPU memory pressure. In most cases, output-level KD over 50-100B tokens recovers 85-90% of the pruning loss. Only add the feature-level term if you see a systematic gap on specific capability categories that does not close with more output-level tokens.
- **Use temperature $\tau = 2-4$ for the KD loss.** Temperature $\tau = 1$ (raw logits) produces a distribution that is too peaked on the argmax, reducing the soft-label information. A temperature of 2-4 flattens the distribution enough that secondary tokens contribute meaningful gradient. A practical default is $\tau = 2$ with a linear warmup of the KD loss weight over the first 5% of training steps.

- **Match calibration data to your deployment traffic.** The distillation step is short enough that domain-targeted data is tractable. If your compressed model will primarily serve code generation, distill on a code-heavy corpus. A general-purpose distillation that underperforms on your actual request distribution costs user quality on every inference for the model’s lifetime.
- **Expect 90% accuracy recovery in 10-100B tokens for models above 7B.** Returns diminish sharply after 100B tokens because the student has effectively exhausted its capacity; additional tokens are not adding new gradient directions. If you are still seeing a large gap at 100B tokens, the gap is structural (too aggressive a pruning ratio), not a training-budget problem.
- **The prune-then-distill recipe breaks down above ~50% parameter reduction.** Removing more than half the parameters leaves the pruned initialization with too little useful structure. A fresh 8B model trained on the same data as a pruned-and-distilled 7.5B-from-15B will typically be competitive or better because it is not constrained to a damaged architecture. Treat 50% as the rough ceiling; beyond it, consider training a new small model from scratch using the large model’s outputs as training labels (offline distillation) rather than starting from a pruned checkpoint.

How This Shows Up in Interviews

- **“Walk me through producing an 8B model from a 15B checkpoint without full retraining.”** Strong answer: run structured pruning to remove layers and channels using importance scores on a calibration set, then run output-level knowledge distillation from the 15B teacher over 50-100B tokens with $\tau = 2$. Name the compute saving ($\sim 6-8\times$ vs. scratch training) and the expected accuracy gap (within 1 MMLU point for a $15B \rightarrow 8B$ compression).
- **“Why is distilling from a pruned initialization better than training a small model from scratch?”** Strong answer: the pruned model starts with weights already optimized over the teacher’s full pre-training; it begins near the target loss rather than at random initialization. The distillation step corrects a small perturbation, not the full optimization problem. This translates to $6-8\times$ fewer training tokens to reach equivalent quality.
- **“What loss function do you use and why not just cross-entropy?”** Strong answer: forward KL divergence between teacher and student distributions. Cross-entropy against hard one-hot labels discards the teacher’s uncertainty information; the soft distribution provides gradient from all high-probability tokens, not just the argmax. The richness of the signal is especially important in the early distillation steps when the student’s distribution is still far from the teacher’s.
- **Senior follow-up: “After distillation your 8B model recovers MMLU well but regresses significantly on multi-step reasoning. What is the most likely cause and how do you fix it?”** Multi-step reasoning relies on the residual stream accumulating state across specific layers. If you removed layers uniformly by cosine similarity, you may have deleted layers that specialize in intermediate reasoning steps even if their input-output similarity happened to be high on general calibration data. Diagnosis: re-run the importance scoring using a calibration set that includes chain-of-thought traces and compare which layers are now flagged. Fix: restore the two or three reasoning-critical layers and re-prune a different set of less-critical layers to maintain the parameter budget, then re-distill with a calibration corpus weighted toward reasoning-heavy tasks.

Part VIII

Data: The Real Differentiator

Chapter 20 Pre-Training Data Pipelines

20.1 Where Training Data Comes From

20.1.1 Common Crawl, WET vs. WARC, and Extraction Tools

When an interviewer asks “where does your pre-training data come from?”, the honest answer for most large models is: mostly Common Crawl, with a lot of careful processing on top of it. Common Crawl is the closest thing the open world has to a snapshot of the web, and understanding its internals—specifically the difference between the WARC and WET formats, and what extraction tool you apply to convert HTML to text—is the kind of detail that separates an engineer who has built a real data pipeline from one who has only read about it. This topic shows up in senior ML engineer and research scientist interviews whenever the conversation gets into data quality or pipeline design.

Common Crawl is a nonprofit that has been running monthly web crawls since 2007. Each crawl visits billions of URLs using a seeded breadth-first traversal of the web, starting from hundreds of millions of seed URLs and respecting `robots.txt` directives. The result of each crawl is stored in two primary formats:

- **WARC** (Web ARChive): the raw HTTP response, including headers and the original HTML. This is the ground truth and can be tens of terabytes per snapshot.
- **WET**: a pre-extracted plaintext version produced by Common Crawl itself. The conversion from HTML to text is lossy—structure, links, and metadata are stripped—and the extraction quality is modest by modern standards.

As of 2024, there are roughly 100 crawl snapshots, each adding approximately 2-3 billion pages. Naively, this looks enormous: ~240 trillion tokens if you take all snapshots and run minimal filtering (as DCLM-pool does). But the raw quality of random Common Crawl pages is low—boilerplate, spam, near-duplicate content, and non-English text dominate a random sample.

The HTML-to-text conversion step matters more than it looks. Starting from the WARC files and applying a high-quality extractor produces substantially better text than using the WET files directly. The DataComp-LM paper ran a controlled ablation across extraction methods and found that using Trafilatura instead of the raw WET files improved downstream benchmark scores by roughly four points. *Your choice of HTML extraction tool is a hyperparameter with measurable impact on model quality.*

The appeal of WARC over WET is information density. WET is a blunt conversion: it strips away HTML structure and applies a simple heuristic to identify the “main content” of a page. WARC lets you apply a modern, ML-informed extractor like Trafilatura or jusText, which use statistical models to distinguish article body text from navigation bars, footers, and ads. On a real web corpus, this distinction is not trivial—a significant fraction of WET tokens are boilerplate that would never appear in a well-written document.

The trade-off is cost. Processing WARCs yourself requires storing or streaming tens of terabytes per snapshot, running an extractor at scale, and managing failures in a distributed pipeline. WET is turnkey: download and tokenize. For many teams prototyping a new filtering algorithm, starting with WET is fine. For a production corpus where per-point improvements on benchmarks matter, WET is a ceiling.

There is also a secondary effect: different extractors make different coverage-quality trade-offs. Trafilatura is aggressive at removing boilerplate and retains fewer tokens; jusText is more permissive and retains more, which is important when you are trying to sustain a very large training run. NVIDIA’s Nemotron-CC work

explicitly chose jusText over Trafilatura to maximize the token yield for training runs that require well above 6 trillion tokens, accepting slightly noisier extractions in exchange for coverage.

A Worked Example

Consider the DCLM-pool construction: all Common Crawl snapshots up to 2024 are taken, WARC files are processed with Trafilatura, language identification is run to keep primarily English pages, and basic heuristics (minimum document length, removal of mostly non-alphabetic content) are applied. The result is approximately 240 trillion tokens.

Now apply the DCLM-baseline quality filter (a fastText classifier trained to distinguish instruction-like data from generic web text): the pool shrinks to roughly 3.8 trillion tokens-about 1.6% of the original. This aggressive filtering yields high benchmark scores per token, but a 70B model trained for 2 epochs on 3.8 trillion tokens only sees 7.6×10^{12} training tokens, which is already reaching the data ceiling for that corpus.

Compare to Nemotron-CC starting from the same Common Crawl dumps: by using jusText (more permissive extraction) and a softer ensemble of classifiers rather than a single hard threshold, NVIDIA recovered approximately 6.3 trillion tokens while maintaining quality that outperforms DCLM-baseline on most benchmarks. The arithmetic is simple: same source, different extraction and filtering strategy, 66% more usable tokens.

What You Actually Decide in Practice

- **WET vs. WARC:** Use WET for prototyping and research where you need fast iteration. Use WARC with a modern extractor (Trafilatura or jusText) for any corpus that will be used to train a model above 7B parameters, where the four-point quality gap is significant.
- **Extractor choice:** If maximizing benchmark scores per token is the goal, prefer Trafilatura. If you need more tokens to fill a longer training run (e.g., >10 trillion tokens), prefer jusText and accept marginally noisier extractions.
- **How many snapshots to use:** Newer snapshots have more overlap with newer web content. Older snapshots have higher duplicate rates across themselves. Deduplication cost grows super-linearly with the number of snapshots, so most pipelines start with 5-15 recent snapshots and add more only when data quantity becomes the bottleneck.
- **robots.txt compliance:** Common Crawl does respect robots.txt in principle, but coverage is imperfect. If you are building a corpus for a commercial product, spot-check robots.txt compliance on high-volume domains to avoid legal exposure.
- **Incomplete coverage is real:** Common Crawl is not the internet. Even applying the same Reddit karma filter used to construct OpenWebText against all of Common Crawl yields only about half the tokens that a direct crawl of those Reddit-linked pages would produce. For domains that matter to your application, augment with targeted crawls.

How This Shows Up in Interviews

- **“Walk me through how Common Crawl becomes training tokens.”** A strong answer covers: raw crawl produces WARC and WET; you extract text from WARC using a tool like Trafilatura; apply language ID and heuristic filters; deduplicate; optionally apply a quality classifier. Name the 4-point gap between extraction tools.

- **“Why not just use the WET files Common Crawl provides?”** Key points: WET is a lossy conversion with low-quality boilerplate extraction; modern extractors applied to WARC produce measurably better text; the cost is pipeline complexity and storage, which is worth it above prototype scale.
- **Senior follow-up: “DCLM-baseline filters 240 trillion tokens down to 3.8 trillion. Is that a problem?”** A senior answer acknowledges the data ceiling this creates for large-model training runs, explains the Nemotron-CC approach of using a softer classifier ensemble to recover 6.3 trillion tokens from the same source, and discusses the coverage-quality trade-off in classifier design. The deeper point is that aggressive quality filtering can become a capacity constraint before compute does.

Key Takeaways

- Common Crawl produces WARC (raw HTML) and WET (pre-extracted text); always prefer WARC with a quality extractor for production corpora.
- Extraction tool choice is a measurable hyperparameter: Trafilatura vs. raw WET is worth roughly four benchmark points in controlled ablations.
- Trafilatura maximizes quality per token; jusText maximizes token yield-choose based on whether your bottleneck is quality or quantity.
- Common Crawl is not a complete snapshot of the web; targeted domain crawls are necessary for coverage of high-value sources.
- Aggressive quality filtering that reduces 240 trillion tokens to 3.8 trillion creates a data ceiling for large training runs; ensemble-based filtering recovers more tokens at comparable quality.

20.1.2 Books, Code, Papers, and Forums

Every serious pre-training mix reserves dedicated weight for curated domain corpora-books, code repositories, academic papers, and Q&A forums-even when web data already totals trillions of tokens. An interviewer asking “why not just filter Common Crawl more aggressively?” is testing whether you understand what these domains provide that filtered web text cannot. This question surfaces regularly in ML infrastructure and research scientist interviews, especially when the topic is domain-specific capability, long-context extension, or instruction-following ability.

Web text is wide but shallow. A typical web page is short, self-contained, and stylistically generic. The corpora described in this subsection are valuable because they provide signal that web pages structurally cannot:

- **Books:** long-range coherence and narrative structure. A single book can span hundreds of thousands of tokens with thematic continuity, making book corpora the primary tool for long-context extension training. BooksCorpus (7,000 self-published works, now defunct) was used for BERT. Project Gutenberg (~75,000 public-domain books) is the clean open alternative. Books3, drawn from shadow libraries, was used in the Pile and the original Llama but has since generated substantial copyright litigation.
- **Code:** formal structure, precise semantics, and executable reasoning chains. The dominant source is GitHub. The Stack (from BigCode) is the open replication: 137 million repositories git-cloned, filtered to permissively licensed files, deduplicated, yielding 3.1 TB of code in over 300 programming languages. Code is commonly cited as improving reasoning and tool-use capabilities beyond programming tasks, though the mechanism is not fully understood.
- **Academic papers:** dense factual knowledge and structured argumentation. arXiv provides open-access preprints across STEM fields. PubMed Central provides biomedical papers under an NIH open-access

mandate. Together they add tens of billions of tokens of high-information-density text that is underrepresented in general web crawls.

- **Forums (Stack Exchange, Reddit, Wikipedia):** question-answer structure that closely mirrors how users interact with a deployed chat model. Stack Exchange is particularly valuable because votes and comments provide a quality signal that lets you filter aggressively. Wikipedia is used both as a direct training source and as a quality reference (via n-gram language models) to score other documents.

Each domain corpus provides a structural signal—long coherence, formal correctness, QA format, or encyclopedic density—that filtered web text approximates poorly at any reasonable scale.

The core argument is coverage of distributions that web text undersamples. Consider what a random Common Crawl page looks like: short, often list-formatted, frequently e-commerce or news. The long-tail of technical, literary, and mathematical knowledge that makes a model broadly capable lives in books, papers, and specialized forums. A quality classifier trained to select “Wikipedia-like” text from Common Crawl will recover some of this, but it will miss the structural properties (multi-page coherence, formal proof style, multi-turn Q&A) that domain corpora provide natively.

Stack Exchange deserves special attention. A Stack Overflow thread-question, accepted answer, community comments—looks almost identical to a user-assistant chat exchange. Models trained on substantial Stack Exchange data have, in a sense, seen instruction-following examples during pre-training. This is one reason the line between pre-training and post-training data is blurry: the instruction-following distribution is already present in the pre-training mix, just unlabeled.

Code has a different but equally important property: it is almost always semantically valid (it compiles or it doesn’t), it is heavily deduplicated by nature (copy-paste code is easy to detect), and it carries explicit structure (function signatures, docstrings, tests) that exposes relationships between specification and implementation. Training on code improves performance on tasks that require multi-step planning and precise output formatting, even when code itself is not the target.

A Worked Example

The Dolma corpus used to train OLMo (AI2) illustrates how these domains compose in practice. The reported breakdown across ~3 trillion tokens is approximately:

Source	Tokens	Notes
Common Crawl	~2.0T	processed via CCNet-style rules
The Stack (code)	~0.4T	permissive licenses only
C4	~0.2T	used as a clean web supplement
Reddit	~0.1T	submissions and comments separately
Semantic Scholar	~0.08T	40M academic papers
Project Gutenberg	~0.006T	public-domain books
Wikipedia	~0.004T	monthly dump, all languages

Web text dominates by volume, but code (~13%) and academic papers (~3%) punch above their weight: models trained without these domains show notably degraded performance on programming tasks and technical Q&A even though they represent a small fraction of total tokens.

The token counts also reveal a practical constraint: Project Gutenberg at 6 billion tokens is tiny relative to the corpus. It is included not for scale but for structural diversity—long-form text with high coherence that no other source provides at equivalent quality.

What You Actually Decide in Practice

- **License filtering for code is mandatory.** The Stack and similar pipelines filter to permissive licenses (MIT, Apache, BSD) before any other step. Training on GPL code and releasing the model exposes you to viral license claims; the legal landscape is unsettled enough that teams err on the side of permissive-only. This drops coverage from all of GitHub to roughly one-third of public repositories.
- **Wikipedia as a quality anchor vs. training source.** Wikipedia is small ($<0.2\%$ of a typical corpus) but appears in almost every mix. Its dual role-as a direct source of high-quality text and as the reference distribution for quality classifiers-means removing it from training to avoid license concerns would also break most model-based quality filters.
- **Books3 and shadow libraries are off the table for commercial models.** Meta’s use of LibGen (4M books) is the subject of ongoing litigation. Project Gutenberg (75,000 public-domain books) is the safe alternative; the quality is high but the volume is $50\times$ smaller.
- **Stack Exchange and Reddit licensing changed around 2023.** Both platforms moved to require paid licenses for commercial model training. Budget for this or find community-sourced snapshots taken before the policy change. Datasets like Dolma used pre-2023 Reddit dumps; replicating this for a new corpus requires negotiation or a substitute.
- **Domain upsampling at mid-training.** Even if a domain is well-represented at pre-training scale, it is common to upsample math, code, and long-context data at mid-training to strengthen specific capabilities. The Tulu and OLMo-2 pipelines both do this explicitly, applying $5\text{-}20\times$ upsampling on math and reasoning data relative to their natural frequency in the web corpus.

How This Shows Up in Interviews

- **“Why include Stack Exchange in a pre-training corpus? It’s only a few billion tokens.”** Key points: volume is not the reason; it is the QA structure, which pre-exposes the model to instruction-following format, and the quality signal from votes that enables aggressive filtering. Contrast with raw Common Crawl where no comparable quality signal exists per document.
- **“How do you handle code licensing in a pre-training corpus?”** A strong answer covers: filter to permissive licenses (MIT, Apache, BSD) at repository level before extraction; deduplicate aggressively because copy-paste code is common; note that GPL code creates viral license risk that most teams avoid for production models. Mention The Stack as the open benchmark for this approach.
- **Senior follow-up: “If you had to cut your corpus from 15 trillion tokens to 3 trillion due to a compute budget reduction, which domains would you preserve and at what ratios?”** A senior answer reasons from capability impact rather than volume: preserve code ($\sim 10\text{-}15\%$), academic papers ($\sim 5\%$), and high-quality filtered web ($\sim 75\text{-}80\%$); cut general Reddit and unfiltered Common Crawl first. Justify by pointing to ablations (Dolma, DCLM) showing that code and papers have outsized impact per token on benchmark performance, while generic web text is the most substitutable component.

Key Takeaways

- Books, code, papers, and forums each provide structural signals-long coherence, formal correctness, QA format, encyclopedic density-that filtered web text cannot replicate at any scale.
- Stack Exchange and Wikipedia appear in nearly every mix not for volume but for structural quality;

Wikipedia also doubles as the reference distribution for quality classifiers.

- Code must be filtered to permissive licenses before training; GPL and other copyleft licenses create legal exposure in commercial models.
- Shadow library books (Books3, LibGen) offer substantial volume but are subject to active litigation; Project Gutenberg is the safe 75,000-book alternative.
- Domain data is typically upsampled 5-20 $\times$ at mid-training to strengthen targeted capabilities, even when the domain was already present at pre-training scale.

20.2 Evolution of Open Pre-Training Corpora

20.2.1 From C4 and The Pile to RefinedWeb and FineWeb

Every serious interview question about pre-training eventually comes down to one uncomfortable truth: “the internet” is not a dataset. When you first start building corpus pipelines, the naive assumption is that you download Common Crawl, shuffle it, and feed it to the model. What you actually get is mostly boilerplate, spam, and machine-generated SEO text. The history of open pre-training corpora is the history of the community figuring out, dataset by dataset, how to turn Common Crawl into something trainable. Understanding the milestones-C4, The Pile, RefinedWeb, FineWeb-tells you what decisions matter, what mistakes were made, and what the current defaults are. This progression shows up in senior interviews as a framing question: “How did the community’s approach to corpus construction evolve, and why?”

The central challenge is that Common Crawl is simultaneously enormous and noisy. A single monthly snapshot contains roughly 1.4 trillion tokens, but most of it fails to qualify as coherent natural language by any reasonable measure. The filtration problem is: given a very large, very cheap pool of text, how do you maximize the quality-per-token of the subset you actually train on?

The early answer, introduced by **C4** (Colossal Clean Crawled Corpus, from the T5 paper in 2019), was purely heuristic: keep lines ending in terminal punctuation, require at least three sentences per document, strip pages containing certain keywords, and discard any text containing curly braces-which removed most code. Starting from one snapshot of approximately 1.4 trillion tokens, these rules yielded roughly 156 billion tokens of English text. The approach is fast, deterministic, and interpretable. Its limitation is equally obvious: well-formed spam still passes, and legitimate domain-specific text (technical documentation, structured prose) often fails.

EleutherAI’s **The Pile** (2020) took the opposite stance: instead of filtering a single noisy source aggressively, curate 22 deliberately chosen high-quality domains and mix them. The result is an 825 GB dataset (roughly 400 billion tokens) spanning Common Crawl subsets, Books3, arXiv, PubMed Central, GitHub, Stack Exchange, Wikipedia, and others. Each source was selected because domain experts believed it carried signal-not because a classifier said so.

RefinedWeb (2023, from TII) reversed the Pile’s philosophy: it bet that Common Crawl alone, filtered well enough, outperforms any handpicked multi-source blend. Using Trafilatura for HTML extraction, Gopher-style heuristic quality rules, and aggressive fuzzy deduplication, TII produced a 5 trillion token corpus without any model-based classifier. **FineWeb** from Hugging Face extended this approach to all available Common Crawl dumps, yielding 15 trillion tokens with additional deduplication and light anonymization.

The core progression is from heuristics-on-one-dump, to multi-source curation, to aggressive-heuristics-on-all-dumps-each step trading a different dial between volume, coverage, and quality.

Rule-based filtering has one decisive advantage over model-based filtering: it cannot overfit to a proxy. A classifier trained to find Wikipedia-like text will produce more Wikipedia-like text, not necessarily better pre-training data. In 2020, this was a genuine concern—the models available for filtering were too weak to be trusted for semantic judgments, and aggressive model-based filtering risked stripping out legitimate content from underrepresented dialects and domains that simply did not look like encyclopedia articles.

The Pile’s multi-source design solved coverage differently. Instead of filtering a noisy pool, it sourced directly from known-good repositories: arXiv for scientific reasoning, PubMed for biomedical language, Stack Exchange for technical QA. The trade-off is that assembling and licensing 22 separate sources is expensive and fragile—several Pile sources (Books3, Reddit) have since been removed or restricted due to copyright and API policy changes.

RefinedWeb and FineWeb took the pragmatic position: if you deduplicate thoroughly and apply heuristic rules carefully, you can extract more quality from Common Crawl than any curated multi-source mix, because Common Crawl is simply larger. FineWeb’s 15 trillion tokens dwarfs The Pile by $37\times$. The deduplication step deserves particular emphasis: fuzzy deduplication using MinHash to find near-duplicate documents consistently improves downstream model quality even when it reduces token count. Models that memorize repeated boilerplate instead of learning novel patterns pay a quality tax that raw token volume cannot compensate for.

A Worked Example

Walk through building a C4-equivalent pipeline from a single Common Crawl snapshot, CC-MAIN-2019-18, which contains approximately 1.4 trillion tokens in WET format.

1. **Language filter.** Run fastText language identification; keep only documents classified as English with confidence $p > 0.65$. This drops roughly 50% of documents. Remaining: ~ 700 billion tokens.
2. **Heuristic rules.** Apply C4 rules: drop lines not ending in terminal punctuation, drop documents with fewer than three sentences, drop documents containing { or }, drop documents matching a 400-word blacklist. Remaining: ~ 165 billion tokens.
3. **Exact deduplication.** Hash each 3-sentence window with SHA-1; drop any document where more than 30% of windows appeared before. Remaining: ~ 156 billion tokens.

That is the C4 slice: one snapshot, three passes, 11% retention. Now extend to FineWeb: repeat across all 100 Common Crawl snapshots (roughly 140 trillion token raw pool), replace step 2 with Gopher rules, add MinHash fuzzy dedup with a Jaccard threshold of 0.8, and you converge toward 15 trillion tokens of retained text—the FineWeb baseline.

What You Actually Decide in Practice

- **WET vs. WARC extraction.** Using pre-extracted WET files is faster but delivers 3-4 benchmark points lower than re-extracting HTML with Trafilatura or jusText. Pay the compute cost of WARC extraction once; you will not redo the corpus every month, and the quality gain is permanent.
- **Deduplication scope.** Within-snapshot exact dedup is cheap. Cross-snapshot dedup catches repeated boilerplate (news syndication, Wikipedia mirrors) but requires storing hundreds of billions of hashes. For most teams, within-snapshot exact dedup plus cross-snapshot MinHash fuzzy dedup is the right balance.
- **Multi-source vs. single-source.** If your targets are domain-specific (biomedical, legal, scientific), include those sources explicitly—web data undersamples technical literature by roughly $10\times$ relative to professional usage. For general language, FineWeb or a filtered Common Crawl is sufficient.

- **Rule-based vs. model-based.** Pure heuristics are reproducible and fast; use FineWeb as your base and layer model-based filters on top for targeted quality improvements. Mixing both is now standard in production.
- **Token budget alignment.** At 7B parameters, roughly 1 trillion high-quality tokens hits the Chinchilla-optimal frontier. At 70B, that rises to 10 trillion. Choosing a dataset with fewer tokens than your compute budget forces multi-epoch training, which hurts quality; an oversized dataset means you stop before the model is saturated.

How This Shows Up in Interviews

- **“Walk me through how you would build a pre-training corpus from Common Crawl.”** A strong answer names the four stages in order: extract (WET vs. WARC, choice of tool), language filter, quality filter (heuristic rules and/or classifier), deduplication (exact then fuzzy). Mention approximate retention rates at each step-11% retention for C4-style filtering on a single snapshot is a concrete number worth having.
- **“Why does The Pile use 22 sources instead of just filtering Common Crawl harder?”** Coverage: Common Crawl systematically underrepresents technical domains. PubMed, arXiv, and Stack Exchange are not well-crawled, so model-filtered web data misses them regardless of filter quality. A strong answer also notes that this advantage erodes as legal and API access to those sources becomes restricted.
- **“What is the difference between C4 and FineWeb? When would you use one over the other?”** C4 is a single-snapshot, heuristic-only, 156-billion-token dataset-useful for ablations and reproducibility. FineWeb is all-dumps, 15 trillion tokens, heuristic rules plus fuzzy dedup-use it for any real training run where you want a strong general-purpose baseline with more than enough tokens for a 70B model.
- **Senior follow-up: “FineWeb outperforms C4 on benchmarks. Is that because of more tokens or better deduplication?”** The senior answer isolates the variables: FineWeb uses $\sim 96\times$ more tokens but also applies MinHash deduplication, which C4 lacks. Controlled ablations in the FineWeb release show that deduplication alone accounts for most of the gap on knowledge-intensive benchmarks, while the additional token volume helps primarily on few-shot generalization tasks. The practical lesson is that deduplication is higher leverage than raw volume once a corpus exceeds a few hundred billion tokens.

Key Takeaways

- C4 established the heuristic-filtering template: punctuation rules, sentence-count thresholds, blocklists applied to one Common Crawl snapshot, yielding 156 billion tokens at 11% retention.
- The Pile showed that multi-source curation outperforms filtered web alone when domain coverage matters, but the approach is fragile to copyright and API restrictions that have since removed several of its key sources.
- RefinedWeb and FineWeb validated the counter-bet: aggressive fuzzy deduplication plus heuristics applied to *all* Common Crawl dumps produces a superior general-purpose corpus without model-based filtering, at 5-15 trillion tokens.
- Fuzzy deduplication (MinHash, Jaccard ~ 0.8) is higher-leverage than additional raw tokens once a corpus exceeds a few hundred billion tokens-it is now standard practice in every serious pipeline.
- HTML extraction tool choice (Trafalatura or jusText vs. pre-extracted WET) shifts benchmark scores by 3-4 points and is one of the cheapest, highest-return decisions in the entire pipeline.

20.2.2 DCLM, Nemotron-CC, and the Modern Recipe

The 2024 DataComp-LM (DCLM) paper changed the default assumptions for every team building a serious pre-training pipeline. Before it, the community had largely avoided machine-learning classifiers for filtering pre-training data, on the grounds that such filters introduce hard-to-audit biases and may strip out underrepresented content. DCLM settled the question empirically through a controlled benchmark competition: a fastText classifier trained on instruction-style positive examples, applied aggressively to Common Crawl, outperforms every prior heuristic baseline on standard evaluations. Nemotron-CC from NVIDIA followed with a complementary finding-aggressive quality filtering preserves quality but destroys token volume, and at the scale required for a 70B-class model you need both. These two datasets now define the upper boundary of what is publicly achievable in open pre-training data, and understanding their pipelines in detail is expected knowledge for any senior ML engineer working on pre-training.

DCLM frames corpus construction as a machine-learning competition: given a fixed compute budget for a 7B-parameter model, which data pipeline produces the best benchmark scores? To make this repeatable, the team first processed all available Common Crawl dumps through a unified extraction and MinHash deduplication pipeline, producing **DCLM-pool**: 240 trillion tokens of deduplicated English web text. The competition then asks: what quality filter, applied to DCLM-pool, yields the best downstream model?

Their winning recipe is a fastText binary classifier. The positive training set combines two sources:

- **OpenHermes-2.5**: a collection of GPT-4-generated instruction-following examples covering diverse tasks-question answering, reasoning, code, summarization.
- **ELI5**: the Reddit “Explain Like I’m 5” subreddit, where community members write detailed, accessible explanations in response to curious questions.

Negative examples are sampled uniformly from RefinedWeb-not garbage, but less curated than the positives. The classifier score is computed for every document in DCLM-pool; keeping documents above a threshold of 0.5 produces **DCLM-baseline**:

$$3.8 \text{ trillion tokens retained} \approx \frac{3.8}{240} \approx 1.6\% \text{ of the pool}$$

DCLM-baseline outperforms RefinedWeb by approximately 3 percentage points on the core benchmark suite (HellaSwag, ARC, MMLU) when training a 7B model to 1 trillion tokens.

Nemotron-CC from NVIDIA accepted DCLM’s quality signal but rejected its extreme selectivity. Their argument: training a 400B-parameter model to Chinchilla optimality requires on the order of 8-10 trillion tokens. DCLM-baseline at 3.8 trillion is insufficient. Their response was to ensemble multiple quality signals-the DCLM fastText score, an educational-value classifier distilled from a large Nemotron model, and document length-and then *stratified-sample* across score buckets rather than hard-thresholding at the top. The result was **Nemotron-CC**: 6.3 trillion tokens, outperforming DCLM-baseline on benchmarks while providing 66% more training tokens.

A further Nemotron-CC contribution was LLM-assisted document transformation. Low-quality retained documents were rewritten into cleaner prose by an LLM. High-quality documents had synthetic instruction pairs generated from them-for example, a Wikipedia article was converted into a “Summarize this” task, a “List the key facts” task, and a factual question-answer pair. This deliberately blurs the boundary between pre-training and post-training data, with the explicit goal of giving the model early exposure to instruction-following structure.

The modern recipe is: DCLM-pool for extraction and dedup, a fastText or LLM-distilled classifier for quality scoring, stratified sampling across score strata for volume control, and LLM rephrasing for the long tail

of retained documents.

Using instruction-style data as positive examples for a pre-training quality classifier is counterintuitive—you are not training on instructions, you are looking for web documents that resemble instructions. The insight is that well-formed question-and-answer content is a strong proxy for coherent, informative language. ELI5 posts and OpenHermes examples are explanatory, specific, and well-structured; a document that looks like them tends to be substantive rather than spammy.

The competing approach—using Wikipedia as the quality anchor, as CCNet and GPT-3 did—produces data that looks like an encyclopedia. That is excellent for factual recall but underweights reasoning chains, instructional prose, and conversational question-answering. DCLM’s ablations show that the instruction-anchored positives outperform the Wikipedia-anchored approach by 1-2 points even when controlling for token count.

Nemotron-CC’s stratified sampling strategy addresses a genuine production problem. Hard-thresholding at a high quality score is like accepting only the top decile of candidates: you get quality, but you sacrifice diversity in the lower nine deciles, some of which contain irreplaceable domain-specific content. Bucketing by score and sampling proportionally preserves quality distribution while maintaining volume. The 1 trillion token high-quality Nemotron-CC subset (the top two score buckets) outperforms the full 6.3T on per-token quality metrics, confirming that quality and volume trade off rather than compound; the extra tokens are worth including only when your training budget demands them.

A Worked Example

Walk through the DCLM pipeline for a 7B model targeting 1 trillion tokens of training.

Step 1: Obtain DCLM-pool. DCLM-pool is publicly available on Hugging Face. It contains 240 trillion deduplicated English tokens, already MinHash-deduplicated across all Common Crawl dumps. Do not rebuild this yourself; the extraction and dedup engineering is the most expensive part and is already done.

Step 2: Train the fastText classifier.

```
Positive set : 300K docs from OpenHermes-2.5
              + 50K docs from ELI5 subreddit
Negative set : 350K docs sampled from RefinedWeb
Labels       : __label__positive / __label__negative
Training     : fastText supervised, 5 epochs, lr=0.1
Model size   : ~2 GB
Train time   : <10 minutes on a single CPU machine
```

Step 3: Score DCLM-pool. Run the classifier over 240 trillion tokens. At ~1 million tokens per second per CPU core, scoring the full pool requires ~240 million CPU-core-seconds, or roughly 2,800 CPU-core-hours—about 28 hours on a 100-core cluster.

Step 4: Filter. Keep documents with fastText positive-class score > 0.5. Empirically this retains 1.4-1.6% of documents, yielding 3.4-3.8 trillion tokens. Sample 1 trillion uniformly from this retained set for your training run.

Step 5: Validate with a proxy model. Train a 1B-parameter proxy on 10 billion token samples drawn from (a) the retained set and (b) FineWeb. A 1-2 point improvement on HellaSwag confirms the classifier is working. If you see no improvement, check that your positive-set format matches typical web document encoding (UTF-8, not HTML entities).

Total additional cost over downloading DCLM-pool: approximately 3,000 CPU-core-hours for scoring. This is negligible compared to GPU-hours for the actual 7B training run.

What You Actually Decide in Practice

- **Start from DCLM-pool, not raw Common Crawl.** Reproducing the extraction and deduplication pipeline from scratch takes weeks of engineering. DCLM-pool is public, covers all dumps through mid-2024, and the only cost is download bandwidth. Reserve engineering time for the classifier.
- **Hard-threshold for models under 70B; stratified-sample for larger.** If your token budget is under 5 trillion, a hard threshold at the DCLM fastText score of 0.5 is sufficient. Above 5 trillion—which is where 70B+ models live at Chinchilla optimality—use stratified sampling or lower the threshold, consciously trading some quality for volume.
- **Do not use GPT-4-generated data as positives in a production classifier.** It violates OpenAI terms of service. Use open-weight model outputs (Llama 3, Qwen, Mistral) or human-curated instruction datasets under permissive licenses (e.g., FLAN, Dolly, OpenHermes under its Apache-2.0 variant).
- **LLM rephrasing is worth the cost for the bottom quartile only.** Rephrasing the lowest-scoring 25% of retained documents with a 7B model costs roughly $0.25 \times 3.8 \text{ T} \approx 950$ billion tokens of generation. Budget this only if those documents cover rare domains not well-represented in the top tier; otherwise the hallucination risk outweighs the coverage gain.
- **Use Nemotron-CC-1T as your default single-source option for a 7B run.** It is the highest-quality public corpus as of mid-2024 at a token count that fits comfortably within a 7B Chinchilla-optimal budget, and its benchmark numbers are well-documented.

How This Shows Up in Interviews

- **“How would you use a quality classifier to filter a 200-trillion-token web corpus?”** Name the pipeline: define positive examples (instruction-style data), define negatives (lightly filtered web), train fastText (fast, interpretable, <2 GB model), score the pool in parallel CPU jobs, threshold at a target retention rate. Mention that 1.4-1.6% retention at a threshold of 0.5 yields ~ 3.8 trillion tokens—enough for a 7B model.
- **“What makes DCLM-baseline better than FineWeb if both come from Common Crawl?”** The classifier. FineWeb uses only heuristic rules; DCLM adds a fastText classifier trained on instruction-like positives. The DCLM paper shows a 3-point improvement on core benchmarks at 7B scale. Both apply MinHash deduplication, but the classifier is the dominant factor in the quality gap.
- **“What is the trade-off between DCLM-baseline and Nemotron-CC?”** DCLM-baseline: higher average quality per token, 3.8 trillion total tokens—constrains training runs for large models. Nemotron-CC: slightly lower average quality per token across the full 6.3T, but includes a 1T high-quality subset that matches or exceeds DCLM quality while providing 66% more tokens for training if needed. Choose DCLM for 7B; choose Nemotron-CC full for 70B+ where token volume limits the training run.
- **Senior follow-up: “Nemotron-CC uses LLM rephrasing to recover low-quality documents. What could go wrong, and how would you detect it?”** The core risk is hallucination: the rewriting model invents facts or systematically injects its own biases into the training corpus. Three detection strategies: (a) perplexity divergence—rephrased documents with perplexity far below their source indicate fabrication; (b) n-gram overlap checks—if rephrased text deviates dramatically from the source, flag for review; (c) factual benchmark regression—if adding rephrased documents degrades TriviaQA or NaturalQuestions

relative to the clean baseline, the rephrasing model is corrupting factual knowledge. In production, cap the rephrased fraction at 10-15% of total tokens and treat factual benchmarks as a canary before scaling up.

Key Takeaways

- DCLM-pool covers all Common Crawl dumps through mid-2024 (240 trillion deduplicated tokens); DCLM-baseline applies a fastText quality classifier to reduce it to 3.8 trillion tokens-roughly 1.6% retention-outperforming prior heuristic baselines by 3 points at 7B scale.
- Instruction-like data (OpenHermes, ELI5) as positive examples for the quality classifier outperforms Wikipedia-anchored proxies; the intuition is that explanatory, question-answering prose is a better proxy for useful pre-training content than encyclopedia-style writing.
- Nemotron-CC addresses DCLM-baseline’s volume limitation through ensemble classifier scoring, stratified sampling across score buckets, and LLM-assisted rephrasing, yielding 6.3 trillion tokens with quality competitive with DCLM-baseline.
- For a 7B model, start with DCLM-baseline or Nemotron-CC-1T; for a 70B+ model, use Nemotron-CC full (6.3T) or lower the DCLM threshold to recover token volume at the cost of some quality dilution.
- The key engineering insight of the modern recipe: spend CPU cycles on classification (cheap, parallelizable), spend GPU cycles only on model training-the classifier itself is a fastText model that trains in under ten minutes.

20.3 Legal and Ethical Considerations

20.3.1 Copyright, Licenses, and Fair Use

Copyright is the central legal friction in assembling a pre-training corpus, and it surfaces in almost every senior-level data interview. The question “can you legally train on Common Crawl?” sounds simple, but answering it well requires understanding three distinct concepts that most engineers conflate: what copyright covers, how licenses grant permission, and when fair use provides a defense without a license. Getting this wrong does not just expose the company to litigation-it shapes which data sources you can include in an open-weight release versus a closed commercial product.

Under the US Copyright Act of 1976, copyright attaches automatically to any *original work of authorship fixed in a tangible medium of expression*. Registration is not required; the moment someone publishes a blog post, that post is copyrighted. This means the overwhelming majority of web content that ends up in Common Crawl is copyrighted, regardless of whether the page displays a copyright notice.

Copyright protects *expression*, not *ideas*. You cannot copyright the algorithm for computing attention; you can copyright the specific code implementing it. Facts-names, dates, addresses-are not copyrightable, which is why a phone book’s content is in the public domain even though a sufficiently creative arrangement of that content might not be. This distinction matters for training data: corpora heavy in factual encyclopedic text (Wikipedia, arXiv abstracts) sit in a more defensible position than corpora heavy in fiction, journalism, or creative prose.

There are two legal paths to use copyrighted material. The first is a **license**: a contract in which the rights holder grants you permission under specified terms. Creative Commons licenses (CC-BY, CC-SA, CC-NC, etc.) are a standardized form of pre-granted license-Wikipedia is CC-BY-SA, so anyone may use its text with

attribution and share-alike obligations. Commercial licenses negotiated directly with platforms (Reddit-Google, Stack Exchange-OpenAI) work the same way but are bespoke and paid. The second path is **fair use**: a statutory defense that allows use of copyrighted material without a license when four factors favor it: (1) the purpose and character of the use (commercial vs. educational, transformative vs. reproductive); (2) the nature of the copyrighted work (factual vs. creative); (3) the amount used relative to the whole; and (4) the effect on the market for the original.

Whether training an LLM on web-scraped text constitutes fair use is unresolved law, but the strongest argument turns on the “transformative use” prong: the model extracts statistical patterns across billions of tokens, not a copy of any particular work.

The transformative-use argument draws force from *Authors Guild v. Google* (2d Cir. 2015), which held that Google Books’ systematic digitization and snippet display of millions of copyrighted books was fair use because it served a fundamentally different function than reading the books—it enabled search and discovery. LLM training proponents argue the analogy holds: the model does not reproduce the training corpus; it compresses statistical regularities into weights that support generation of new text.

The tension is that LLMs can memorize. Carlini et al. showed that training data can be extracted verbatim from GPT-2 given the right prompt, and subsequent work demonstrated the same for larger models. A model that can regurgitate entire paragraphs of a copyrighted novel weakens the transformative-use defense considerably, because the infringement is no longer abstract—the expression itself is reproduced. This is why deduplication and memorization audits are now standard pipeline steps, not just quality hygiene.

The commercial/non-commercial distinction also matters asymmetrically. Academic researchers running non-commercial experiments are more insulated than a company selling API access. Open-weight releases sit in a legally awkward middle: the weights are “open,” but the training data disclosure that accompanies them makes the copying more legible to plaintiffs. Closed commercial models paradoxically enjoy better legal cover on this dimension because the training data is never disclosed.

A Worked Example

Consider the trajectory of **Books3**, a dataset of roughly 200,000 copyrighted books scraped from Library Genesis, a shadow library. Books3 appeared in the Pile (2020) and was later incorporated into Llama 1’s training mix. In 2023, a group of authors including Richard Kadrey, Sarah Silverman, and Christopher Golden sued Meta, alleging that Llama 1’s use of Books3 constituted copyright infringement.

Meta’s primary defense was transformative use: training compresses millions of books into 7 billion parameters that do not store or reproduce any individual work. The plaintiffs countered that the model can reproduce passages verbatim and that it effectively substitutes for buying the books, harming the market. The district court dismissed several claims but allowed some to proceed. In parallel, Books3 was removed from the Pile’s hosted version and Llama 2 quietly dropped it from its data card.

The practical lesson: the risk profile of a dataset is not just “is this copyrighted” but “is it from a source whose rights holders are organized, motivated, and able to sue.” Published authors are a concentrated, articulate group. Individual bloggers are not. This asymmetry—not legal principle—explains why Books3 triggered litigation while the vast majority of Common Crawl’s copyrighted content has not.

As a contrast, **Project Gutenberg’s** 75,000 books are predominantly in the public domain (copyright has expired, typically 95+ years after publication in the US). Using them is legally safe, and they provide long-document text that books corpora were specifically valued for in the BERT era.

What You Actually Decide in Practice

- **Public domain $\neq$ freely usable scans.** A 19th-century novel is in the public domain, but a publisher's typeset PDF of that novel may carry a new copyright on the layout and OCR corrections. Use raw Project Gutenberg text, not publisher PDFs.
- **Creative Commons licenses have variants.** CC-BY and CC-SA allow commercial use; CC-NC (non-commercial) does not. If your pipeline ingests CC content, track the license variant per document. Mixing NC-licensed content into a commercial training run is a genuine legal risk.
- **Open-weight releases attract more scrutiny than closed products.** If you publish your data card, you are handing plaintiffs a roadmap. For open-weight models, prefer datasets with clear provenance: FineWeb (Common Crawl, ToS-respecting crawl), DCLM-baseline, The Stack (permissively licensed code only).
- **Memorization audits belong in the pipeline.** Run a membership-inference probe or verbatim-extraction test against held-out copyrighted text before a model release. A model that can reproduce more than a few sentences of a copyrighted work verbatim is a liability.
- **Shadow libraries are a hard no for commercial products.** LibGen has 4 million books; it also has a litigation history. The token-per-lawsuit ratio looks attractive until the first discovery subpoena arrives.

How This Shows Up in Interviews

- **“Walk me through whether training on Common Crawl is legal.”** Strong answer: most Common Crawl content is copyrighted; the legal path is fair use, specifically the transformative-use prong from *Authors Guild v. Google*; the argument weakens if the model memorizes verbatim text; commercial vs. non-commercial use affects the analysis; no court has definitively ruled on LLM training specifically.
- **“What’s the difference between a public dataset and a legally safe dataset?”** Strong answer: “public” means accessible without authentication; “legally safe” means you have a license or a defensible fair-use argument. YouTube videos are public but scraping them violates ToS. Wikipedia is publicly licensed (CC-BY-SA). Common Crawl is public but largely copyrighted content used under implicit fair-use reliance.
- **“How would you audit a proposed training dataset for legal risk before a model release?”** Strong answer: categorize each source by license type; flag shadow-library or paywalled content; run memorization probes on high-risk sources; check whether rights holders in each category have litigation history; produce a data card that documents provenance and any exclusions.

Senior follow-up: “What changes in your analysis if this is an open-weight release versus a closed API?”
 Senior answer: open-weight releases require full data disclosure, which lowers plausible deniability and makes it easier for plaintiffs to identify infringing sources; commercial use eliminates the educational fair-use factor; the calculus therefore shifts toward more conservative sourcing (CC-licensed, public domain, licensed data) for open releases, while closed commercial models rely more heavily on fair-use arguments they never have to articulate publicly.

Key Takeaways

- Nearly all web content is automatically copyrighted; the question is not whether it is copyrighted but whether you have a license or a fair-use defense.

- Fair use turns on four factors; for LLM training the strongest factor is transformative use, and the weakest point is verbatim memorization.
- Creative Commons licenses differ by variant: CC-BY and CC-SA allow commercial use; CC-NC does not.
- Shadow libraries (LibGen, Books3) carry concrete litigation risk for commercial products; public-domain sources (Project Gutenberg) and permissively licensed code (The Stack) are the safe alternative.
- Open-weight model releases face higher legal exposure than closed APIs because data provenance is disclosed, making memorization-based claims easier to assert.

20.3.2 Terms of Service and Data Governance

Copyright is not the only legal friction in data pipelines. Even when fair use provides a plausible defense for the content of a web page, a separate body of law-contract and computer-fraud statutes-governs how you obtained that content. Engineers who understand copyright but treat platform terms of service as advisory text are operating with a dangerously incomplete picture. At staff and principal level, interviewers test whether you can reason about the full legal surface of a data decision, not just the copyright slice of it.

When you visit a website, you implicitly accept its *terms of service* (ToS). Most platform ToS prohibit automated scraping, redistribution of content, and use of data to train competing AI systems. Violating ToS is not a copyright infringement-it is a breach of contract, and depending on how the data was accessed, potentially a violation of the Computer Fraud and Abuse Act (CFAA). These are distinct legal theories with distinct remedies and different parties who can sue.

robots.txt is a machine-readable file at the root of a web server that signals which crawlers are permitted to access which paths. It is not legally binding-there is no statute that makes ignoring `robots.txt` illegal by itself-but courts have treated a site's `robots.txt` as evidence of the operator's intent, which strengthens a CFAA or breach-of-contract argument when a crawler ignores it. The major frontier-lab crawlers (GPTBot, ClaudeBot, GoogleBot for AI training) are all explicitly named in `robots.txt` files across major news, publishing, and social media sites, with access typically denied.

A **data licensing agreement** is the commercial resolution to this friction: the platform grants a contractual right to use its data for AI training in exchange for payment or a cross-licensing deal. Examples include the Reddit-Google agreement (2024), the OpenAI-Stack Exchange agreement, and the OpenAI-Shutterstock agreement. These deals emerged as platforms recognized that their data had become a scarce input to a high-value industry and that they could monetize it rather than watch it be scraped for free.

The practical constraint this imposes is stark: there is no way to get a ToS license for the entire internet, which means any corpus that relies on broad web crawl is necessarily operating under a fair-use assumption while simultaneously accepting the contractual risk of ToS violation.

The governance structure that has emerged-frozen Common Crawl snapshots plus separately licensed high-value verticals-reflects the only feasible way to assemble a large, legally defensible corpus at scale. Common Crawl performs the crawl on your behalf and explicitly intends the data for research use, which provides one layer of insulation. Separately licensing Reddit for social dialogue data, Stack Exchange for QA data, and a news aggregator for current-events coverage fills the gaps where the high-value signal is concentrated and the rights holders are organized enough to enforce their ToS.

Data poisoning introduces a governance dimension that is easy to underestimate. Carlini et al. demonstrated a timing attack against Wikipedia: an adversary can inject a malicious edit to a Wikipedia article in the window

between the edit and the next editorial rollback, timed to coincide with Common Crawl’s periodic dump. The injected text enters the dump, gets included in downstream training data, and is rolled back on Wikipedia before anyone notices. The result is a model trained on attacker-controlled text, which can be used to steer model behavior—for example, associating a trigger phrase with a specific sentiment or behavioral policy. The fix is partial: Wikipedia has hardened its edit policies, and some pipeline operators now diff their Wikipedia snapshots against a trusted reference. But the broader point stands: data provenance is a security surface, not just a legal one.

A Worked Example

Consider a concrete pipeline decision around **YouTube**. Millions of YouTube videos carry Creative Commons licenses (CC-BY), making the *content* legally usable without a separate agreement. An engineer might reasonably conclude: “I have a license to the content, so I can download and transcribe these videos for training.” This is wrong.

YouTube’s ToS explicitly prohibits automated downloading of video or audio, regardless of the content’s license. The Creative Commons license governs what you can do with the content once you have it lawfully; the ToS governs how you obtain it. Downloading programmatically without YouTube’s permission is a breach of contract, and YouTube has sent DMCA-adjacent takedown notices and pursued injunctions against services built on this basis (e.g., *hiQ Labs v. LinkedIn*, though that case turned on CFAA specifically).

The correct path for YouTube audio is either: (1) negotiate a data licensing agreement with Google, as some frontier labs are rumored to have done; (2) use the official YouTube Data API within its rate limits (which are set too low for training-scale data collection); or (3) use transcripts that were lawfully obtained and redistributed under a license (several academic ASR datasets exist in this category). The “it’s Creative Commons” shortcut collapses the distinction between content rights and access rights.

What You Actually Decide in Practice

- **Treat robots.txt as legally meaningful, not just advisory.** Respecting Disallow directives does not give you immunity, but ignoring them is affirmative evidence against you in both CFAA and breach-of-contract claims. Crawl engines should parse and honor robots.txt by default; build in a crawl-delay header to avoid overload arguments as well.
- **Distinguish access rights from content rights.** Before adding a source, ask two separate questions: (a) Is the content licensed for this use? (b) Did we obtain it through a channel permitted by the platform? Both must be yes. Creative Commons answers (a); the ToS answers (b).
- **Use frozen snapshots to avoid ToS time-bombs.** Reddit and Stack Exchange changed their APIs in 2023 mid-production for many teams. If you collected data before the ToS change, the legal exposure is frozen at the time of collection. Snapshot-based pipelines (Common Crawl dumps, Wikipedia XML dumps with a dated version) are more defensible than live API scraping that crosses ToS update boundaries.
- **Track provenance at the document level.** For every document in your corpus, record the Common Crawl snapshot ID, the retrieval date, and the source URL. This is not just good hygiene—it is the audit trail you need if a rights holder demands removal (a “right to erasure” request under GDPR or a DMCA notice), and it lets you exclude specific domains post-hoc without reprocessing everything.
- **Implement a data-poisoning audit for high-trust sources.** For sources included specifically because of their quality (Wikipedia, arXiv), maintain a checksum or diff against a reference snapshot. An edit that

appeared only in one Common Crawl dump but not in the live site warrants manual review before that document enters the training corpus.

How This Shows Up in Interviews

- **“How would you design a legally defensible data collection pipeline for a commercial LLM?”** Strong answer: layer the strategy-use Common Crawl as the broad web base (crawl performed by a third party, data intended for research/AI use); negotiate licenses for the highest-value verticals (code, books, QA); exclude sources whose ToS explicitly prohibit AI training use; respect robots.txt in any first-party crawling; maintain per-document provenance logs for erasure requests.
- **“What is data poisoning and how would you defend against it in your pipeline?”** Strong answer: data poisoning is the injection of adversarial text into training data to steer model behavior; for Wikipedia, the attack surface is the dump-timing window between edit and rollback; mitigation is diffing snapshot content against a trusted live reference, flagging documents that appeared transiently; for Common Crawl, domain-level reputation signals can exclude known spam or compromised domains.
- **“A rights holder sends a takedown notice claiming their content is in your training set. Walk me through how you respond.”** Strong answer: use provenance logs to identify which documents originated from the claimant’s domain; assess whether a license or fair-use argument applies to that source; if neither, remove the documents from the dataset, retrain affected checkpoints if the content contributed meaningfully, and document the removal.

Senior follow-up: “Can you actually “unlearn” a document from a trained model, and how would you handle this operationally at scale?” Senior answer: machine unlearning is an active research area but not yet production-reliable-gradient-based unlearning methods exist but cannot provide the same guarantees as simply not training on the data in the first place. In practice, the operational answer is (1) maintain modular training checkpoints keyed to data provenance so a corpus can be reconstructed without the disputed content and a fresh training run initiated from the last clean checkpoint; (2) for models already deployed, implement output filters that suppress verbatim reproduction of the claimed content while the longer-term retraining proceeds; (3) accept that this is expensive, which is why upstream provenance hygiene is worth the investment.

Key Takeaways

- ToS violations are a breach-of-contract risk distinct from copyright; both must be evaluated independently for any data source.
- robots.txt is not legally binding but is treated as evidence of intent; always honor it in first-party crawls.
- Creative Commons licenses answer “can I use this content?” not “can I collect it from this platform?”-the ToS answers the second question.
- Document-level provenance logs (snapshot ID, retrieval date, source URL) are the operational prerequisite for DMCA compliance, GDPR erasure requests, and data-poisoning audits.
- Data poisoning is a real attack surface: snapshot-diffing against a trusted reference is the practical mitigation for high-trust sources like Wikipedia.

Chapter 21 Data Filtering and Deduplication Algorithms

21.1 Quality Filtering Algorithms

21.1.1 n-gram Models and KenLM

An interviewer designing a pre-training data pipeline will almost always ask how you distinguish high-quality web text from garbage, and the first tool you should name is a language-model perplexity filter backed by KenLM. This question shows up at the senior engineer level because it sits at the intersection of classical NLP, large-scale distributed processing, and model quality-getting it wrong can mean millions of wasted GPU-hours training on near-random bytes. The n-gram approach is deliberately crude, but that crudeness is a feature: it runs at a fraction of the cost of any neural scorer while filtering the most egregiously low-quality pages before anything more expensive touches the data.

The abstract primitive is: given a small set of high-quality *target* documents T and a massive *raw* corpus R (e.g., a Common Crawl dump), find a subset $R' \subseteq R$ whose distribution resembles T . All three methods in this section—n-gram models, fastText classifiers, and importance sampling—are instances of this same recipe. What changes is how you estimate the scoring function.

An n-gram language model estimates the probability of a token w_n given the preceding $n - 1$ tokens:

$$P(w_n \mid w_{n-1}, \dots, w_1) \approx \frac{C(w_1, \dots, w_n)}{C(w_1, \dots, w_{n-1})}$$

where $C(\cdot)$ denotes corpus count. In practice, many n -gram sequences never appear even in large corpora, so raw maximum-likelihood estimates assign zero probability to perfectly grammatical sentences. *Kneser-Ney smoothing* addresses this by backing off to lower-order n -grams when counts are sparse, interpolating between orders rather than hard-cutoff discounting. KenLM is the standard open-source implementation: it uses on-disk trie structures so that models with hundreds of millions of n -grams fit in memory and score millions of tokens per second on a single CPU.

Given a trained KenLM model, the score for a document is its *perplexity* under the target distribution:

$$\text{PPL}(d) = \exp\left(-\frac{1}{|d|} \sum_i \log P(w_i \mid w_{i-1}, \dots, w_{i-n+1})\right)$$

Low perplexity means the document looks like the target; high perplexity indicates it is out-of-distribution. Documents are sorted by perplexity and a top fraction is retained.

The north-star summary: train KenLM on a clean reference corpus, score every raw document by its perplexity, and keep the low-perplexity fraction—this is the cheapest defensible filter you can run at web scale.

The key trade-off is **cost vs. sophistication**. A neural classifier can judge quality more accurately, but scoring a trillion tokens with a BERT-scale model would cost roughly the same in FLOPs as the pre-training run itself—you cannot spend that budget on filtering. KenLM, by contrast, is a counting procedure: fitting a 5-gram model on Wikipedia takes minutes and produces a file of a few gigabytes. Inference is a trie lookup, not a matrix multiply; throughput on a single core routinely exceeds 10^7 tokens per second.

The limitation is equally clear: n -grams are local. A document composed of individually plausible bigrams but globally incoherent content (shuffled sentences, SEO keyword stuffing) can fool the filter. In practice this is acceptable because the primary goal is to remove true garbage-pages filled with binary content, encoding artifacts, or character-level noise—not to make fine-grained quality judgments. The model is intentionally crude

so that it can be extremely fast.

Before KenLM-based filtering became widespread, pipelines relied entirely on heuristic rules: URL block-lists, minimum word count, fraction of alphabetic characters, and so on. Rules are brittle and require constant maintenance as the web evolves. A perplexity filter generalizes from whatever reference corpus you supply, which makes it easy to retarget for new domains without rewriting logic.

A Worked Example

The CCNet pipeline—the foundation for the first LLaMA pre-training dataset—illustrates the approach concretely. The setup:

- **Target corpus:** Wikipedia (all languages, split by language).
- **Raw corpus:** Common Crawl WET files (one monthly snapshot $\approx$ 40B tokens after HTML stripping).
- **Unit of scoring:** paragraph (not full document), to allow partial retention.

Step 1: Train one KenLM 5-gram model per language on the corresponding Wikipedia dump. For English, this produces a model of roughly 5-8 GB.

Step 2: For each paragraph in Common Crawl, compute perplexity under the English model (or whichever language model matches the fastText language-identification label).

Step 3: Sort paragraphs by perplexity in ascending order and retain the bottom third (lowest perplexity = most Wikipedia-like). In CCNet’s English slice this cut reduced the token count from roughly 400B to $\approx$ 130B while measurably improving downstream benchmark scores.

Arithmetic check: a 5-gram lookup over a 1,000-token paragraph requires roughly 10^3 trie traversals. At 10^7 tokens per second per core, scoring a 40B-token dump completes in about 40,000 core-seconds—under 12 core-hours, which is negligible compared to any GPU budget.

What You Actually Decide in Practice

- **Reference corpus choice matters more than smoothing details.** If you train KenLM on Wikipedia, you bias toward encyclopedic prose; if you train on GitHub commit messages, you bias toward code. Match the reference to the downstream use case. Mixing reference corpora (Wikipedia + books) is fine—just concatenate and retrain.
- **Score at the paragraph level, not the document level.** Long documents often contain both excellent and garbage paragraphs. Scoring paragraphs independently lets you perform surgical removal rather than discarding entire pages. C4 uses three-sentence spans; CCNet uses single paragraphs. Both work; paragraph is usually the safer default.
- **Set the threshold by inspection, not formula.** Examine the text at the 33rd, 50th, and 67th perplexity percentiles before committing. The right cut point varies by domain: code corpora show different perplexity distributions than news. CCNet’s top-third heuristic is a reasonable starting point for English web text.
- **KenLM does not replace rule-based filters; it complements them.** Run basic heuristics first (minimum length, non-ASCII fraction, URL spam detection), then KenLM, then a neural classifier if budget allows. Each stage is cheap and removes material the next stage would struggle with.
- **Beware of perplexity collapse on short sequences.** Paragraphs under 50 tokens have high variance in perplexity estimates; their scores are unreliable. Set a minimum token threshold (50-100 tokens is typical) before applying the perplexity filter.

How This Shows Up in Interviews

- **“Walk me through how you would filter Common Crawl to produce a high-quality English pre-training corpus.”** A strong answer names the pipeline stages in order: language identification, heuristic rules, perplexity filtering with KenLM trained on a clean reference, then optionally a fastText or neural quality classifier. Quantify the expected token retention at each stage.
- **“Why use KenLM instead of a BERT-based classifier for quality scoring?”** The key point is FLOPs budget: scoring 10^{12} tokens with BERT at ~ 10 ms per sentence would cost millions of GPU-hours, exceeding the training budget. KenLM runs on CPUs at 10^7 tokens per second; the entire web can be scored in hours on a modest CPU cluster.
- **“What are the failure modes of perplexity-based filtering?”** Name: (1) n -grams are local, so globally incoherent text can pass; (2) the filter is language-model-centric, so stylistically unusual but valid text (poetry, code-switching) is penalized; (3) the reference corpus must match the target domain or the filter misfires.
- **Senior follow-up: “If your target domain is scientific papers and Wikipedia is a poor reference, how do you adapt the pipeline?”** Train KenLM on a seed set of high-quality papers (arXiv abstracts, PubMed, Semantic Scholar). If the seed set is tiny (< 1 B tokens), supplement with Kneser-Ney’s backoff to lower-order n -grams for robustness. Alternatively, switch to an importance-sampling estimator that explicitly models both the target and raw distributions-this gives principled weighting rather than a hard threshold and handles small target sets more gracefully.

Key Takeaways

- The universal filtering recipe: estimate a model on target data, score raw documents with it, keep high-scoring examples-KenLM operationalizes this with near-zero compute cost.
- Kneser-Ney smoothing handles unseen n -grams by interpolating with lower-order models; the implementation detail matters less than the choice of reference corpus.
- KenLM throughput ($> 10^7$ tokens/sec per CPU core) makes it feasible to score an entire Common Crawl snapshot in core-hours, not GPU-hours.
- Score at the paragraph or sentence level, not the document level, to allow partial retention of otherwise heterogeneous pages.
- KenLM is a first-pass filter, not a final verdict-follow with a discriminative classifier or importance-sampling reweighter for finer-grained quality control.

21.1.2 fastText Classifiers and Importance-Sampling Filters

Once a perplexity filter has removed the worst of the web, the next question in a data-pipeline interview is how you build a supervised quality classifier without labeling millions of documents by hand-and how you justify the classifier’s compute budget relative to the training run. This topic separates engineers who have built real pipelines from those who have only read about them. The two dominant answers are fastText discriminative classifiers and importance-sampling reweighters; each has a distinct theoretical motivation and a concrete set of production trade-offs worth articulating on a whiteboard.

Both methods share the same abstract recipe: estimate the ratio of the target distribution p to the raw distribution q , then use that ratio to preferentially select examples from the raw corpus. They differ in whether

the ratio is estimated *discriminatively* (fastText) or *generatively* (importance sampling).

fastText trains a shallow log-linear classifier over hashed n -gram features. Given a vocabulary of size V and K output classes, a naive bag-of-words classifier needs a $V \times K$ weight matrix-unwieldy when V is large. fastText adds a bottleneck hidden layer of dimension h (typically $h = 16$), reducing parameters from $V \cdot K$ to $V \cdot h + h \cdot K$:

$$\hat{y} = \text{softmax}(W_2 \cdot \text{mean}(W_1 \cdot \mathbf{x}))$$

where $\mathbf{x}$ is the average of hashed n -gram embeddings over the document. To avoid an unbounded bigram vocabulary, all n -grams are hashed into a fixed bucket count (e.g., 10^7 bins) before lookup. For quality filtering specifically, $K = 2$ (positive / negative), so the output is effectively a single logistic unit. Training uses asynchronous SGD, and inference is a pair of dense matrix products-extremely fast even for documents measured in thousands of tokens.

Importance-sampling filters take a generative approach. Fit a unigram or n -gram probability model $\hat{p}$ to the target corpus T and a second model $\hat{q}$ to the raw corpus R . The importance weight for document d is:

$$w(d) = \frac{\hat{p}(d)}{\hat{q}(d)}$$

Resample documents from R with probability proportional to $w(d)$. Documents that look more like T than R receive high weight and are sampled more often; documents that look more like R than T are down-weighted. In practice, both $\hat{p}$ and $\hat{q}$ are estimated using hashed unigram counts-exactly as in the fastText feature hashing trick-so the whole procedure is as cheap as fastText while having a clearer distribution-matching interpretation.

The key insight is that fastText asks “is this document in-distribution?” (a classification boundary) while importance sampling asks “by how much is this document over- or under-represented relative to target?” (a density ratio)-the latter is more principled for preserving diversity.

The fastText compute argument is the one interviewers probe hardest. If you are filtering a corpus to 1% of Common Crawl, the classifier sees the full raw corpus. Every FLOP you spend on the classifier must be amortized over the data retained. A BERT-style encoder scoring each document at ~ 10 ms per 512-token chunk would consume $\sim 10^5$ GPU-hours to score 10^{12} raw tokens-rivaling the training cost itself. fastText on the same corpus, running on CPUs at 10^6 documents per second, finishes in hours. The rule of thumb: if filtering discards 99% of data, the per-token classifier cost must be under $\sim 1/100$ th of the per-token training cost. fastText clears that bar; BERT does not.

Importance sampling offers a diversity guarantee that fastText lacks. A discriminative classifier optimizes the decision boundary; once most of the data cluster in the positive region, the model does not care whether it samples uniformly from that region. An importance-sampling filter explicitly adjusts for density mismatch: rare but target-consistent topics receive higher weights and are more likely to appear in the final corpus. Empirically, importance-sampling-curated corpora outperform fastText-curated ones on GLUE-style benchmarks when the target set is small, though the absolute gains are often modest ($\lesssim 2$ -3 points).

The principal limitation of importance sampling is that the target corpus T must be large enough to estimate $\hat{p}$ reliably. When T is very small, the unigram estimates for rare tokens are noisy. A practical fix is Laplace or add- ϵ smoothing so that zero-count tokens receive a small but nonzero probability.

A Worked Example

phi-1 code quality filter is the clearest end-to-end example because it shows how to construct the target set T when none exists a priori.

1. **Define T** by prompting GPT-4 with a rubric: “Rate the educational value of this Python snippet for a student learning basic coding concepts.” Apply this to 100,000 documents sampled from the Python subset of The Stack.
2. **Label:** documents with high GPT-4 educational ratings become positive examples; the rest become negatives. This produces a labeled set of $\sim 100\text{K}$ examples-small enough that GPT-4 annotation is affordable (roughly \\$2,000-\\$5,000 at 2023 API prices for $100\text{K} \times 500$ -token snippets).
3. **Train classifier:** embed each document with a small pre-trained code encoder, then train a random forest on those embeddings. A random forest is used rather than a neural classifier because the labeled set is small and training must be fast.
4. **Score raw corpus:** run the random forest across the full Python subset of The Stack ($\sim 200\text{B}$ raw tokens). Random forest inference is CPU-bound and parallelizes trivially across shards.
5. **Retain** documents classified as high educational value ($\sim 3\text{B}$ tokens selected).

Result: a 1.3B-parameter model trained on this 3B-token filtered corpus reached 17% on HumanEval at 36K training steps. The same architecture trained on the unfiltered 200B-token corpus reached only 12% at 96K steps-roughly $6\times$ more efficient in both tokens and compute.

The compute budget check: GPT-4 annotation of 100K examples at 500 tokens each = 50M tokens at roughly $\$0.03/\text{1K tokens} = \$1,500$. Random forest scoring of 200B tokens takes perhaps 200 CPU-hours. Both are negligible versus a multi-day GPU training run.

What You Actually Decide in Practice

- **Use fastText when you have a natural source of positive and negative labels.** GPT-3 and LLaMA used Wikipedia-referenced pages as positives and raw Common Crawl as negatives. If your domain has an obvious high-quality source (GitHub starred repos, arXiv accepted papers, curated forums), this is the fastest path. Training fastText takes minutes; inference scales to the full web on a CPU cluster in hours.
- **Use importance sampling when diversity matters or your positive set is tiny.** If $|T| < 1\text{B}$ tokens, discriminative classifiers have too little signal to learn a reliable boundary. An importance-sampling unigram model needs only enough data to estimate token frequencies reliably-a few hundred million tokens suffices.
- **Use GPT-4 annotation to synthesize T when no natural target exists.** Label 1,000-100,000 examples (not more-beyond that cost grows without proportional gain). Train a cheap classifier (random forest, fastText) on those labels and apply it to the full corpus. This is now the dominant approach for domain-specific data curation.
- **Stochastic retention sharpens the boundary without hard cutoffs.** Both GPT-3 and CCNet use a sigmoid sharpening function: documents near the threshold are kept or dropped probabilistically rather than by a hard threshold. This prevents the model from memorizing the exact boundary and improves diversity at the margin.
- **Never skip a compute budget check before choosing a classifier architecture.** If the raw corpus is N tokens and the training compute budget is C FLOPs, the classifier can spend at most $C/(\text{retention fraction})$ FLOPs on scoring. At 1% retention, that means the classifier budget is roughly $C/100$. fastText almost always fits; BERT-scale almost never does.

How This Shows Up in Interviews

- **“How did GPT-3 build its quality classifier, and why did that approach work?”** Strong answer: positive examples came from web pages linked by Reddit with ≥ 3 upvotes, negatives from raw Common Crawl. This required no human annotation at scale-Reddit karma acts as a noisy but free quality signal. A linear classifier (fastText) was trained on these labels and applied stochastically (not with a hard cutoff) to retain high-scoring documents.
- **“What is importance sampling in the context of data filtering, and when would you use it over fastText?”** A strong answer defines the importance weight $\hat{p}(d)/\hat{q}(d)$, explains the resampling procedure, and names when it wins: small target sets, domains where diversity is critical, or when the discriminative boundary is ill-defined because positive and negative data overlap heavily.
- **“You want to add more chemistry data to a pre-training corpus. Walk me through end-to-end how you would do it.”** A complete answer: (1) define T using arXiv chemistry papers + textbook excerpts, (2) train KenLM on T for a fast first-pass perplexity filter, (3) optionally train fastText with T as positive and raw web as negative, (4) score the full corpus and retain documents above threshold, (5) deduplicate the result with MinHash LSH before mixing into the main corpus.
- **Senior follow-up: “fastText uses hashed n -grams. What are the implications of hash collisions for quality filtering?”** Collisions cause distinct n -grams to share a weight. For binary classification this means the model learns a weight that represents the average behavior of all colliding n -grams. With 10^7 bins and a typical English vocabulary of $\sim 10^5$ unigrams and $\sim 10^9$ possible bigrams, collision probability for any single bigram is low (~ 10 ppm), but with enough training the model adapts. The key insight is that importance-sampling unigram models have *no* collision problem because they only use single-token counts; this is one reason importance sampling can match fastText quality on small target sets.

Key Takeaways

- fastText is a shallow log-linear model over hashed n -gram features; at $K = 2$ it is binary classification that runs at $> 10^6$ documents per second on CPU-the dominant choice when labeled positive/negative data exists.
- Importance-sampling reweighting estimates $\hat{p}(d)/\hat{q}(d)$ from two n -gram models and resamples accordingly; it matches distributions rather than drawing a decision boundary, which helps preserve diversity when the target set is small.
- When no natural T exists, use a strong LLM to annotate $\sim 100K$ examples and train a cheap classifier (fastText or random forest) on those labels-compute cost is negligible relative to pre-training.
- The compute budget rule: classifier scoring cost must be less than (training cost) divided by (raw corpus size / retained size); this almost always rules out BERT-scale encoders and almost always clears fastText.
- All three filtering methods (KenLM, fastText, importance sampling) share the same recipe: score raw documents relative to a target, keep the high-scoring fraction-they differ only in how the score is estimated and whether it is a hard threshold or a probabilistic retention.

21.2 Targeted Filtering Tasks

21.2.1 Language Identification

Every serious pre-training pipeline applies language identification before any other filter, because feeding multilingual noise into a monolingual model wastes tokens and measurably hurts benchmark performance. This topic comes up at the mid-level when interviewers ask how you would clean a Common Crawl dump, and it is a good test of whether candidates understand that the same lightweight classification machinery used for quality scoring can be repurposed for an entirely different dimension of the data. Getting it wrong is not catastrophic, but getting it subtly wrong - a threshold that is too loose, a model that misclassifies code as Estonian - compounds quietly across hundreds of billions of tokens.

Language identification (LID) is document-level multi-class classification: assign a language label and a confidence score to a piece of text, then threshold. The dominant tool in production is the fastText LID model, which supports 176 languages and runs at several hundred thousand paragraphs per second on a single CPU core. Its architecture is a bag of character n-grams fed into an averaged embedding layer, then a softmax over language classes - no non-linearity beyond that final softmax.

Formally, for a document d with character n-gram multiset $\mathcal{G}(d)$, the model computes:

$$\log p(\ell \mid d) = \mathbf{w}_\ell^\top \left(\frac{1}{|\mathcal{G}|} \sum_{g \in \mathcal{G}} \mathbf{e}_{\phi(g)} \right) - \log Z$$

where $\phi(g)$ hashes n-gram g into one of $B \approx 10^7$ buckets, $\mathbf{e}_b$ is a learned bucket embedding, and Z is the partition function over all classes. The hashing trick bounds memory regardless of vocabulary size and is the same technique fastText uses in its text classifier. The DOLMA pipeline applies this model at the paragraph level and keeps paragraphs where $p(\text{en}) > 0.5$.

Language identification is the same bag-of-n-gram classification machinery used for quality and toxicity filtering - the only difference is that the label space is a set of language codes rather than a binary quality judgment.

The obvious alternative is rule-based detection: check Unicode script ranges, use curated word lists, or rely on HTML `lang` attributes. This works well for well-separated scripts - Latin versus Cyrillic versus CJK - but collapses on Latin-script languages that share large vocabularies (Spanish, Portuguese, Italian, Galician). A character n-gram model implicitly encodes each language's orthographic distribution at exactly the granularity where the signal lives, without requiring a hand-curated lexicon per language.

The key trade-off is precision versus recall in a compute-constrained training regime. If the LID threshold is too permissive, non-English text contaminates the corpus and steals token budget from the target language. If it is too strict, you drop legitimate text from code-switched documents, low-resource dialects, and informal registers that look statistically unusual to the model. BLOOM was trained with only 30% English tokens; in a compute-limited regime, that dilution measurably hurt English benchmark scores relative to English-only models at the same parameter count. Monolingual labs typically accept a recall penalty to guarantee high precision.

The character n-gram approach degrades gracefully on short texts: it becomes less confident, not wrong. A single sentence might score 0.71 for English; a full paragraph by the same author might score 0.98. This is why LID should run at the paragraph or document level, not the sentence level - longer units produce better-calibrated scores.

A Worked Example

Consider building an English pre-training corpus from a 100-billion-token Common Crawl dump. Assume paragraphs average 200 tokens, giving roughly 5×10^8 paragraphs. The fastText LID model runs at 4×10^5 paragraphs per second on 4 CPU threads at 2.5 GHz. Total CPU-core-time:

$$\frac{5 \times 10^8 \text{ paragraphs}}{4 \times 10^5 \text{ paragraphs/core-s}} \approx 1,250 \text{ core-seconds} \approx 21 \text{ core-minutes}$$

Distributed across 64 cores in a Spark or Ray cluster, LID finishes in under two minutes of wall time. The compute budget is negligible compared to any downstream neural scoring step. Applying the threshold $p(\text{en}) > 0.5$ retains roughly 45-50% of Common Crawl by token volume, since the web carries substantial non-English traffic.

Now the edge cases appear immediately. Math content scores $p(\text{en})$ around 0.35-0.45 because $\LaTeX$ expressions and formula tokens look like no known language to the model. Inline code (Python identifiers, function names) is classified as Russian or Estonian 15-20% of the time due to identifier-character patterns. OpenWeb-Math handles this with a two-tier threshold: documents that pass a $\LaTeX$ heuristic (more than 5% of lines contain `\frac`, `\sum`, or similar macros) are kept at $p(\text{en}) > 0.3$, while everything else requires $p(\text{en}) > 0.65$. The result was a 15-billion-token math corpus that outperformed models trained on 20 times more unfiltered data on mathematical reasoning benchmarks.

What You Actually Decide in Practice

- **Threshold:** Start at 0.5 for a general-purpose English corpus; push to 0.65 if you prioritize precision. Drop to 0.3-0.4 for domain corpora (math, code) where the LID model is miscalibrated by design. Audit 500 randomly sampled rejections before locking the threshold - you will find surprising failures.
- **Classification unit:** Document-level is coarser but preserves document coherence. Paragraph-level improves precision on multilingual pages but can produce incoherent documents downstream if you excise individual paragraphs. Use paragraph-level only when you are building passage-level retrieval datasets; use document-level for general pre-training corpora.
- **Script-level pre-filter:** Before running the neural model, check Unicode script frequency. If a document is more than 50% CJK, Devanagari, or Arabic characters, assign the script's dominant language directly and skip fastText. This saves roughly 15% of inference compute and avoids weak-signal classifications on script-heavy documents.
- **Code-switching:** Pages that mix two languages will be classified confidently as whichever language dominates by token count. If purity matters, run sentence-level LID within paragraphs and keep only paragraphs where at least 80% of sentences agree on the majority language.
- **Off-the-shelf vs. fine-tuned:** The public fastText LID model is sufficient for high-resource languages. For low-resource languages with fewer than 10^7 training tokens in the model's corpus, the confusion rate with geographically proximate languages can exceed 20%. In those cases, fine-tune a new output layer on dialect-specific data (Wikipedia, CommonVoice transcripts) while keeping the base embeddings frozen.

How This Shows Up in Interviews

- **“How would you isolate English documents from a 100 TB Common Crawl dump?”** A strong answer names the fastText LID classifier, describes the character n-gram architecture, proposes a threshold with

justification (0.5-0.65), and proactively raises edge cases: code, math, short texts, and languages that look similar to English at the surface. Weaker answers jump straight to regex or Unicode ranges without addressing Latin-script language confusion.

- **“What happens to model quality if you skip language filtering?”** Strong answer: non-English tokens dilute the training budget proportionally. Cites BLOOM as a concrete example: 30% English pretraining produced noticeably weaker English benchmark scores than comparably sized English-only models under the same compute budget. The effect scales with the fraction of off-language tokens in the corpus.
- **Senior follow-up - “Your LID model misclassifies Python files as non-English. How do you fix this without retraining?”** The senior answer: apply a two-tier threshold controlled by a domain heuristic. Documents containing more than a threshold fraction of Python keywords (`def`, `class`, `import`) or a high density of ASCII special characters (brackets, colons, underscores) get a relaxed LID threshold of 0.3; everything else stays at 0.65. No model retraining is needed because the heuristic compensates for the LID model’s domain blind spot. This is the same two-tier strategy OpenWebMath applies to math content.

Key Takeaways

- Language identification reuses the same bag-of-character-n-gram classifier architecture as quality and toxicity filtering - there is nothing architecturally special about LID.
- The fastText LID model runs at 400K paragraphs per second per core, imposing negligible compute overhead compared to any downstream neural scorer.
- Training on a diluted multilingual corpus in a compute-limited regime measurably hurts target-language performance; filtering with a precision-biased threshold is almost always the right default for monolingual labs.
- The classifier degrades on short texts, code, and math - compensate with domain heuristics that adjust the threshold rather than retraining the model.
- Classify at the document level when downstream processing must respect document boundaries; use paragraph-level only for passage-centric datasets where intra-document coherence is not required.

21.2.2 Toxicity and Safety Filtering

Interviewers at frontier labs routinely ask how you would prevent a pre-training corpus from poisoning a model with hate speech, graphic content, and targeted harassment. The question probes two things: whether you understand that toxicity filtering uses the same classifier machinery as quality and language filtering, and whether you can articulate the precision-recall trade-off when a false positive means discarding legitimate content and a false negative means baking harmful content into a production model. This shows up at mid-to-senior level and often comes paired with a harder follow-up about what you do when the classifier is miscalibrated on marginalized dialects or domain-specific terminology.

Toxicity filtering is binary classification applied to a harm dimension rather than a quality or language dimension. The standard approach trains one or more fastText classifiers on a labeled dataset of harmful text, then scores every document in the raw corpus and discards those that exceed a threshold. The key dataset that anchors most production pipelines is the *Jigsaw Toxic Comment Classification* corpus: comments scraped from Wikipedia talk pages and annotated by crowdworkers across six harm categories - toxic, severe toxic, obscene, threat, insult, and identity hate.

The DOLMA pipeline trains two separate fastText classifiers on this data: one for hate speech (collapsing threat, insult, and identity hate into a single positive class) and one for NSFW content (collapsing severe toxic and obscene into a positive class). Both classifiers use the same architecture described in the quality-filtering section - a bag of character n-grams hashed into a fixed embedding table, averaged, then classified with a linear head. Each classifier outputs a scalar in $[0, 1]$, and documents that exceed separate per-classifier thresholds are discarded. Using two classifiers rather than one multi-label head is a practical decision: it lets you tune precision and recall independently across harm types, since the consequences of under-filtering hate speech versus under-filtering NSFW content differ by deployment context.

Toxicity filtering is harm-dimension classification with the same lightweight n-gram machinery as every other filter - the only thing that changes is the label definition and the labeled dataset you train on.

The alternative is rule-based blocklisting: maintain a list of slurs, profanity, and explicit terms and reject any document containing them. This has two failure modes. First, it generates high false-positive rates on legitimate educational, journalistic, and clinical text that necessarily discusses the same terms in context. A document explaining the history of racial slurs in American legislation and a document celebrating that history look identical to a keyword filter. Second, it is trivially bypassed by creative spelling (letter substitutions, zero-width characters, non-ASCII lookalikes) that a character n-gram model handles naturally because it operates below the word level.

The fastText classifier generalizes across these surface variants because the n-gram representation captures sub-word patterns. A model trained on the Jigsaw corpus learns that certain character sequences cluster near toxicity labels without being hard-coded to any specific word form. This is the same reason character n-grams work well for LID on closely related languages: the signal is in the distributional statistics of character transitions, not the lexicon.

The key trade-off is recall versus collateral damage to legitimate content. A conservative threshold that catches 99% of hate speech will also remove a meaningful fraction of news articles, academic discussions of extremism, and creative fiction that engages with dark themes. I have seen pipelines that aggressively filtered toxicity produce models that refused to discuss historical atrocities because the training data had been scrubbed of all documents containing relevant terminology. Tuning the threshold requires evaluating both the corpus's downstream task performance and the rate at which known-safe documents are incorrectly discarded.

A Worked Example

Suppose you are building a 300-billion-token English corpus from Common Crawl and want to apply DOLMA-style toxicity filtering. You have two classifiers: a hate-speech classifier and an NSFW classifier, each a fastText model with a 16-dimensional hidden layer and 10^7 hashed n-gram buckets.

Inference throughput for each classifier is roughly 2×10^5 documents per second per CPU core. A 300B-token corpus at 500 tokens per document contains 6×10^8 documents. Running both classifiers sequentially on 128 CPU cores:

$$\frac{6 \times 10^8 \text{ docs}}{2 \times 10^5 \text{ docs/core-s} \times 128 \text{ cores}} \approx 23 \text{ seconds per classifier}$$

Total wall time for both passes: under one minute. Compute cost is negligible relative to any neural quality scorer.

With the DOLMA hate-speech threshold set at 0.3 and the NSFW threshold at 0.1 (both intentionally conservative, erring toward recall), you expect to discard roughly 1-2% of the corpus. Auditing 1,000 random

discards reveals the classifier’s failure modes: it flags medical and legal text discussing assault, removes news articles about far-right extremism, and incorrectly labels some African American Vernacular English as toxic. This audit is not optional - it tells you whether the precision-recall trade-off is acceptable for your deployment context and whether you need to adjust thresholds per domain.

What You Actually Decide in Practice

- **Threshold calibration:** Never pick a threshold from the paper. Always calibrate on a human-audited sample of 500-1,000 random positives (discards) and negatives (retains). What matters is not the classifier’s accuracy on Jigsaw test data - it is the false positive rate on the domain you actually care about.
- **One classifier vs. multiple:** A single multi-label classifier is simpler to train and maintain. Multiple single-label classifiers let you tune thresholds independently and trace discards to specific harm categories, which is valuable when explaining filtering decisions to policy and legal teams. Use multiple classifiers whenever deployment context requires accountability by harm type.
- **Dialect and register bias:** fastText classifiers trained on Jigsaw are known to over-flag African American Vernacular English and LGBTQ+ community language. Before deploying, run the classifier on a balanced sample that includes these dialects and measure false positive rates per demographic group. If they exceed $3\text{-}5\times$ the baseline false positive rate, either adjust the threshold or curate additional training data for those groups.
- **Domain-specific thresholds:** Medical corpora, legal corpora, and fiction corpora require relaxed thresholds because they legitimately discuss explicit content. A single global threshold applied across all domains is almost always wrong. Tag documents by domain before filtering and apply per-domain thresholds.
- **Bootstrapping the classifier with a stronger model:** If you want better recall on subtle or implicit toxicity - dog whistles, coded language, context-dependent slurs - use a larger model (BERT-scale or larger) to label 100K samples from your target distribution, then distill those labels into a fastText classifier. The expensive model runs once; the fast classifier runs on the full corpus. This is the same bootstrap strategy phi-1 used for quality filtering.

How This Shows Up in Interviews

- **“How would you remove toxic content from a web-scale pre-training corpus?”** A strong answer identifies the Jigsaw dataset as the standard training source, names the fastText classifier as the standard production tool, specifies that two classifiers (hate and NSFW) are typically trained separately, and immediately raises the precision-recall trade-off. Weak answers propose rule-based blocklists without addressing bypass mechanisms or false positive rates.
- **“What is the risk of over-filtering for toxicity?”** Strong answer: aggressive filtering creates systematic blind spots in the trained model. A model that has never seen text discussing violence, discrimination, or explicit content in any context will refuse to engage with those topics - even in clearly beneficial settings like academic analysis, clinical documentation, or counter-speech. It also produces a biased corpus if the classifier has differential false positive rates across demographic groups.
- **Senior follow-up - “Your toxicity classifier was trained on Wikipedia talk-page comments. How does that affect its performance on Reddit or Twitter-style text?”** The senior answer: domain shift is the central risk. Wikipedia talk pages are longer, more formal, and less code-switched than social media

text; the classifier’s precision on short, informal, and heavily abbreviated posts will degrade. The fix is targeted data collection: sample 10K documents from the target domain, have annotators label them, and either fine-tune the classifier’s output layer or use the labels to recalibrate the decision threshold. The key insight is that the embedding layer trained on Jigsaw generalizes reasonably well; it is the classification boundary that shifts with domain.

Key Takeaways

- Toxicity filtering reuses the same fastText n-gram classifier machinery as language and quality filtering - train on a labeled harm dataset, score the corpus, threshold.
- The Jigsaw Toxic Comment corpus is the standard training source; DOLMA trains two separate classifiers (hate speech and NSFW) to allow independent threshold control.
- Always audit random samples of discarded documents before deploying a threshold - false positive rates on legitimate content are the critical failure mode, not classifier accuracy on the training benchmark.
- fastText toxicity classifiers are known to over-flag marginalized dialects; measure per-group false positive rates before deploying and adjust thresholds or training data accordingly.
- Use a stronger model (BERT or larger) to bootstrap labels on your target domain, then distill into fastText - the expensive model runs once on 100K samples, the fast model runs on hundreds of billions of tokens.

21.3 Deduplication at Scale

21.3.1 Exact Deduplication with Bloom Filters

Exact deduplication is the first quality gate every serious pre-training pipeline runs, and it matters more than most engineers expect. A 2022 audit of C4 found a single product-description sentence appearing 61,000 times; training on that corpus without dedup is equivalent to taking 61,000 gradient steps on the same data point. Interviewers at frontier labs ask about exact dedup at every seniority level, but the senior-level question is not “do you hash things?”-it is “how do you dedup 10 trillion tokens without running out of memory?” The answer is Bloom filters.

Exact deduplication has a three-axis design space. First, the *unit*: sentence, paragraph, or document. Second, the *match criterion*: exact character equality. Third, the *action*: remove all copies, or keep one. The algorithmic challenge is that duplicate detection is fundamentally pairwise: to know whether item a is a duplicate, you must compare it against every other item. For n items that is $O(n^2)$ comparisons, which is infeasible at web scale. Hashing collapses this to $O(n)$ by converting pairwise equality into shared bucket membership: compute $h(x)$ for every item and group by hash value. Items in the same bucket are candidate duplicates; with a collision-resistant hash such as MurmurHash3, same bucket means same content.

The standard distributed formulation-a MapReduce over hash buckets-is correct and parallelizable, but it requires persisting all hashes in a random-access structure. At 100 billion items you are building a hash table that occupies tens of gigabytes of cluster RAM. A *Bloom filter* solves the memory problem.

A Bloom filter is a bit array of m bits, initialized to zero, with K independent hash functions $h_1, \dots, h_K$ each mapping items to $\{0, \dots, m - 1\}$. To insert item x : set bits $h_1(x), \dots, h_K(x)$ to 1. To query item x : return present iff all K bits are already set. The filter has *no false negatives*-an inserted item is always

reported present-but it has a tunable false positive rate. The false positive probability is approximately

$$P_{\text{FP}} \approx \left(1 - e^{-Kn/m}\right)^K,$$

where n is the number of inserted items. Setting $K = (m/n) \ln 2$ minimizes P_{FP} for fixed m and n , yielding

$$P_{\text{FP}} \approx \left(\frac{1}{2}\right)^K = 2^{-K}.$$

A Bloom filter trades an exactly-zero false positive rate for a predictable, tunable one-and that tradeoff buys an order-of-magnitude reduction in memory over a full hash table.

The memory savings are the decisive factor. Storing 10 billion paragraph hashes as 64-bit integers requires $10^{10} \times 8 \text{ bytes} = 80 \text{ GB}$. A Bloom filter targeting $P_{\text{FP}} = 10^{-9}$ for the same $n = 10^{10}$ items needs

$$m = \frac{-n \ln P_{\text{FP}}}{(\ln 2)^2} \approx \frac{10^{10} \times 20.7}{0.48} \approx 4.3 \times 10^{11} \text{ bits} \approx 54 \text{ GB}$$

of bits and nothing else-no hash table overhead, no collision chains, no string storage. Distributed across a cluster, each shard holds a slice of the filter and processes its slice of the corpus independently.

The only downside is false positives: a unique paragraph is occasionally misclassified as a duplicate and silently discarded. At $P_{\text{FP}} = 10^{-6}$ you lose roughly one paragraph per million insertions-an acceptable quality hit for a multi-trillion-token corpus. DOLMA sets $P_{\text{FP}} = 10^{-15}$ to make false discards practically zero.

C4 takes a slightly different approach: it deduplicates at the three-sentence-span level using exact matching via a hash set rather than a Bloom filter, then removes the duplicate span and leaves the surrounding document intact. The tradeoff is that post-surgery documents may no longer be coherent. Industry consensus is that this is acceptable because the model trains on billions of documents and learns to handle imperfect prose.

A Worked Example

Consider the deduplication stage for a 70B model pre-training run over 100 billion paragraphs (roughly 5 trillion tokens). You want paragraph-level dedup with $P_{\text{FP}} = 10^{-9}$.

Size the filter. $n = 10^{11}$, $\ln P_{\text{FP}} \approx -20.7$:

$$m = \frac{10^{11} \times 20.7}{0.48} \approx 4.3 \times 10^{12} \text{ bits} \approx 537 \text{ GB}.$$

Distributed across 64 nodes, each shard holds approximately 8.4 GB-fits in a single node's DRAM.

Choose K . Optimal $K = (m/n) \ln 2 \approx 43 \times 0.693 \approx 30$ hash functions.

Verify. $P_{\text{FP}} \approx 2^{-30} \approx 9 \times 10^{-10} \approx 10^{-9}$. Confirmed.

Operate. Stream paragraphs through MurmurHash3 with 30 distinct fixed seeds. For each paragraph: if any of the 30 bit positions is zero, the paragraph is unique-insert it and emit it downstream. If all 30 are set, discard. Partition paragraphs by $h_0(x) \bmod 64$ so each node owns a disjoint shard of the bit array and a disjoint slice of the corpus.

Throughput estimate. MurmurHash3 runs at approximately 3 GB/s per core on modern hardware. For 30 hash functions on a 500-byte paragraph: $500 \times 30 / (3 \times 10^9) \approx 5 \mu\text{s}$ per paragraph, or roughly 200,000 paragraphs per second per core. Across 64 nodes at 32 cores each: approximately 400 million paragraphs per second-the full 100 billion-paragraph corpus processes in under 5 minutes.

What You Actually Decide in Practice

- **Unit of deduplication.** Paragraph-level is the industry default (DOLMA, RedPajama). Document-level is faster but misses partial duplicates such as boilerplate footers, license text, and templated ads. Sentence-

level catches more overlap but risks producing incoherent post-surgery documents. Choose paragraph unless you have a specific reason to differ.

- **False positive rate.** At $P_{\text{FP}} = 10^{-6}$ you discard about 1 in a million unique paragraphs-negligible for trillion-token corpora. Setting $P_{\text{FP}} = 10^{-15}$ (DOLMA) essentially eliminates false discards but costs approximately $2.5\times$ more bits. For most pipelines, $P_{\text{FP}} = 10^{-9}$ is the right default: it keeps memory reasonable and false-discard losses below measurement noise.
- **Hash function.** Use MurmurHash3 with distinct integer seeds as the K independent hash functions. Never use Python's built-in `hash()`: it is randomized per process via `PYTHONHASHSEED` and will give inconsistent results across distributed workers.
- **Action: keep-first vs. remove-all.** Keep-first (retain the earliest occurrence, discard later ones) is simpler and retains the data. Remove-all is appropriate for boilerplate that should never appear-but the MIT license is valid training signal; you just do not want 10,000 copies.
- **Exact vs. near.** Exact dedup is high-precision but blind to near-duplicates differing by punctuation or one substituted word. Plan to follow with a MinHash LSH pass if your corpus contains templated content, scraped mirrors, or synthetic data.

How This Shows Up in Interviews

- **“Walk me through how you would deduplicate a 10-trillion-token web corpus.”** Strong answer: name the three-axis design space (unit, match, action), explain hash-based bucketing for $O(n)$ complexity, introduce Bloom filters to handle the memory constraint at scale, give the sizing formula $m \approx -n \ln P_{\text{FP}} / (\ln 2)^2$, mention MurmurHash3 and distributed sharding by hash prefix.
- **“What is a Bloom filter, and why use one instead of a hash set?”** Strong answer: bit array with K hash functions; no false negatives, tunable false positive rate $P_{\text{FP}} \approx 2^{-K}$. A hash set at 10^{10} items needs roughly 80 GB of RAM; a Bloom filter for the same n at $P_{\text{FP}} = 10^{-9}$ needs approximately 54 GB of bits. The filter cannot be queried for keys or iterated-it only supports membership tests-so the use case fits dedup exactly.
- **Senior follow-up: “A false positive in your Bloom filter causes a unique paragraph to be discarded. What is the impact, and how would you detect calibration drift?”** A false positive silently reduces recall with no signal-you cannot know from the output alone that a unique paragraph was lost. At $P_{\text{FP}} = 10^{-6}$ you lose roughly $n \times 10^{-6}$ paragraphs. To audit calibration, run a random sample of discarded items through an exact hash-set lookup against the corpus; if the measured false positive rate exceeds the target, either n has grown beyond the designed capacity or m was sized too small. Recalibrating requires rebuilding the filter-there is no in-place resize.

Key Takeaways

- Exact dedup reduces an $O(n^2)$ pairwise problem to $O(n)$ by mapping items to hash buckets; same bucket means same content.
- A Bloom filter is a bit array with K hash functions supporting constant-time set membership with no false negatives and tunable $P_{\text{FP}} \approx 2^{-K}$ at optimal $K = (m/n) \ln 2$.
- Size the filter with $m \approx -n \ln P_{\text{FP}} / (\ln 2)^2$; at $P_{\text{FP}} = 10^{-9}$ and $n = 10^{11}$ that is approximately 5 bytes per item.
- Use MurmurHash3 with fixed integer seeds; never use Python's `hash()` in distributed pipelines.

- Paragraph granularity with keep-first action is the production default; follow with a MinHash LSH pass to catch near-duplicates.

21.3.2 Near-Deduplication with MinHash and LSH

Exact deduplication removes verbatim copies; near-deduplication removes everything else that provides nearly zero additional signal. The web is full of templated ad copy where only a city name changes, scraped articles where one synonym differs, and terms-of-service boilerplate that picks up or loses a comma across a thousand mirror sites. After exact dedup, these near-duplicates can still represent 20-40% of a large crawl. MinHash LSH is the standard algorithm for catching them at linear time complexity, and it appears in interviews as a deep-dive on probabilistic data structures applied to ML infrastructure-the kind of question that separates engineers who know the algorithm from those who have debugged it at scale.

The similarity measure for near-deduplication is *Jaccard similarity*. For two sets A and B of n -gram shingles extracted from a document:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

Two documents are near-duplicates if $J(A, B) \geq \theta$ for some threshold θ (typically 0.8-0.99). Computing J explicitly for all pairs is $O(n^2)$ -infeasible at billions of documents.

MinHash solves the first part. Given a hash function h drawn from a min-wise independent family, define

$$h_{\min}(S) = \min_{x \in S} h(x).$$

Then:

$$P(h_{\min}(A) = h_{\min}(B)) = J(A, B).$$

The argument is direct: $h_{\min}(A) = h_{\min}(B)$ iff the element with the globally smallest hash value belongs to $A \cap B$, which happens with probability $|A \cap B|/|A \cup B|$.

Using K independent MinHash functions gives a K -dimensional signature vector per document. The expected fraction of matching positions between σ_A and σ_B equals $J(A, B)$, but a single signature is too noisy for a binary near-duplicate decision.

Locality-Sensitive Hashing (LSH) sharpens this estimator into a binary flag. Divide the K hash functions into b bands of r rows each (so $K = b \times r$). Declare two documents as candidate near-duplicates iff at least one band has all r hash values identical:

$$P(\text{collision}) = 1 - (1 - s^r)^b, \quad s = J(A, B).$$

The threshold-the similarity at which $P(\text{collision}) \approx 1 - 1/e \approx 0.63$ -is:

$$\theta \approx \left(\frac{1}{b}\right)^{1/r}.$$

Increasing r sharpens the sigmoid (steeper transition from near-zero to near-one probability around θ). Increasing b shifts the threshold leftward, making it easier to flag similar pairs.

MinHash LSH converts the pairwise near-duplicate problem into a linear-time bucket-collision problem by ensuring that similar documents hash to the same bucket with high probability and dissimilar documents do not.

Without LSH, the cheapest correct approach is $O(n^2)$ pairwise comparisons. For $n = 5 \times 10^9$ documents that is $\sim 10^{19}$ comparisons-computationally hopeless. LSH reduces this to $O(n \times b)$ hash lookups: for each document, hash each of its b bands into a bucket; collect all documents sharing any bucket; optionally run exact

Jaccard verification on those candidates only. The total work is proportional to the number of documents, not pairs.

The band-and-row construction creates a sigmoidal collision curve. A single MinHash function gives $P(\text{collision}) = s$ -a linear ramp with no threshold behavior. The and-within-band, or-across-bands structure introduces the non-linearity that approximates a step function at θ . This is the entire point: you get near-certainty of flagging pairs above the threshold and near-certainty of missing pairs below it, without ever comparing any two documents directly.

Embedding-based similarity (FAISS with dense vectors) gives a more semantic notion of near-duplicate-useful for paraphrase detection-but costs roughly $100\times$ more compute per document, requires a GPU-resident encoder, and is harder to parallelize on a CPU cluster. MinHash LSH runs on commodity hardware at high throughput using MurmurHash3 and is used in production by DOLMA, SlimPajama, and RedPajama-v2.

A Worked Example

Setting. Pre-training corpus, $n = 5 \times 10^9$ documents, target near-duplicate threshold $\theta = 0.8$. Choose $b = 10$ bands, $r = 10$ rows per band ($K = 100$ total MinHash functions).

Verify the threshold.

$$\theta \approx \left(\frac{1}{b}\right)^{1/r} = \left(\frac{1}{10}\right)^{1/10} = 10^{-0.1} \approx 0.794.$$

Close to 0.8. Confirmed.

S-curve at $s = 0.5$ (well below threshold).

$$P(\text{band matches}) = 0.5^{10} \approx 0.001, \quad P(\text{collision}) = 1 - (1 - 0.001)^{10} \approx 0.01.$$

Only 1% of pairs at $J = 0.5$ are flagged as candidates. Good.

At $s = 0.8$ (the threshold).

$$P(\text{band matches}) = 0.8^{10} \approx 0.107, \quad P(\text{collision}) = 1 - (0.893)^{10} \approx 0.70.$$

About 70% of pairs at exactly the threshold are flagged, consistent with the theoretical $1 - 1/e \approx 0.63$.

At $s = 0.95$ (above threshold).

$$P(\text{band matches}) = 0.95^{10} \approx 0.599, \quad P(\text{collision}) = 1 - (0.401)^{10} \approx 0.9999.$$

Essentially all pairs at $J = 0.95$ are caught.

Pipeline steps.

1. Tokenize each document into overlapping 5-gram character shingles. A 500-word document produces roughly 2,400 shingles.
2. Apply $K = 100$ MurmurHash3 functions (fixed seeds 0-99) to each shingle; record the minimum per function. This yields a 100-dimensional integer signature.
3. Divide the signature into 10 bands of 10 values each. Hash each band-by concatenating its 10 values and hashing the result-to a 64-bit bucket key.
4. Group documents by $(\text{band_index}, \text{bucket_key})$. Any group with ≥ 2 members is a candidate near-duplicate cluster.
5. Optionally verify by computing exact Jaccard on candidate pairs to eliminate false positives from accidental band collisions.

Memory per document. $100 \times 4 \text{ bytes} = 400 \text{ bytes}$ for the signature. At 5×10^9 documents: approximately 2 TB total, distributed across 100 nodes at 20 GB each.

What You Actually Decide in Practice

- **Threshold choice.** Use $\theta = 0.99$ for strict near-verbatim dedup (SlimPajama, DOLMA: one word changed per hundred words allowed). Use $\theta = 0.8$ when your corpus has heavy templated content and you want aggressive repetition reduction. Below $\theta = 0.7$ you risk discarding topically similar but substantively different documents-especially dangerous in domain-specific corpora such as code or medical text.
- **Sizing b and r .** Fix K from the memory budget first ($K = 100$ for 400 bytes/document). Derive r from the desired sharpness: higher r steepens the sigmoid. Compute $b = K/r$. Plug into $\theta \approx (1/b)^{1/r}$ and verify the S-curve at one similarity above and one below the threshold to confirm the transition is sharp enough.
- **Shingle granularity.** 5-gram character shingles are robust to tokenizer differences and language variation. Word n-grams are faster but language-dependent. For code, use 10-gram character shingles: they catch copy-pasted functions that differ only in variable names better than word-level n-grams do.
- **Verification pass.** Skip it when throughput is the bottleneck-false positives from band collisions are typically well under 1% for 64-bit bucket keys. Add it when discarding good documents is costly (domain-specific corpora under 100 billion tokens) or when you need an accurate duplicate count for auditing downstream.
- **Semantic deduplication.** MinHash operates on n-gram shingles and is blind to paraphrase. If your corpus contains synthetic data generated by rewriting existing documents, MinHash LSH will miss those near-duplicates. In that regime, embed documents with a small encoder, index with FAISS, and threshold on cosine similarity. Expect roughly $20\times$ higher compute cost and careful threshold tuning to avoid over-pruning topically related but distinct content.

How This Shows Up in Interviews

- **“Why is exact deduplication not enough? What does MinHash catch that a hash set misses?”** Strong answer: exact dedup requires bit-for-bit identity. Near-duplicates-the same paragraph with one word changed, the same article with a different byline-survive exact dedup but add no training signal. MinHash measures Jaccard overlap over n-gram shingles; LSH converts that into a fast bucket-based collision test with a tunable threshold.
- **“Explain how MinHash LSH works and why it runs in linear time.”** Strong answer: MinHash gives each document a K -dimensional signature where the expected fraction of matching positions equals Jaccard similarity. LSH splits the signature into b bands of r rows; two documents collide if any band is identical. Each document performs b hash lookups in $O(K)$ time; total work is $O(nK)$, linear in corpus size, with no explicit pairwise comparisons.
- **Senior follow-up: “How do you choose b and r to hit a specific threshold with sharp behavior? Walk through the math.”** Senior answer: the threshold formula is $\theta \approx (1/b)^{1/r}$; the collision probability as a function of similarity s is $P = 1 - (1 - s^r)^b$. Fix $K = b \times r$ from the memory budget. Choose r to control sharpness-higher r steepens the transition at the cost of fewer bands. Compute $b = K/r$ and verify θ against the target. Evaluate P at $\theta - 0.05$ and $\theta + 0.05$ to confirm the false positive and false negative rates are acceptable. If the transition is too shallow, increase K to allow a larger r while maintaining b .

Key Takeaways

- Jaccard similarity over character n -gram shingles measures near-duplicate overlap; MinHash reduces pairwise computation from $O(n^2)$ to $O(n)$ by converting similarity to hash collision probability: $P(h_{\min}(A) = h_{\min}(B)) = J(A, B)$.
- The band-and-row LSH construction creates a sigmoidal collision curve; threshold $\theta \approx (1/b)^{1/r}$, sharpness grows with r , and the transition point shifts left as b increases.
- Use $b = 10, r = 10$ ($K = 100$) for a threshold near 0.8; use $b = 20, r = 450$ ($K = 9000$) for a threshold near 0.99 (the SlimPajama and DOLMA settings).
- False positives from band collisions are typically well under 1% with 64-bit bucket keys; a verification pass is optional and skipped in most production pipelines.
- MinHash is blind to paraphrase; switch to embedding-based near-dedup when the corpus contains synthetic or heavily paraphrased content, accepting roughly $20\times$ higher compute cost.

Chapter 22 Mid-Training and Post-Training Data

22.1 Instruction and Chat Data

22.1.1 Aggregated NLP Tasks (FLAN-Style)

When an interviewer asks you to explain the difference between a base model and an instruct model, the answer is almost entirely a data answer-and the FLAN-style approach is where that data story begins. Converting thousands of labeled NLP benchmarks into a unified instruction format was the field’s first systematic answer to the question “how do you teach a model to follow instructions?” It shows up in interviews at the L5-L6 range whenever the discussion reaches post-training pipelines, data mix composition, or why instruct models generalize to tasks they were never fine-tuned on directly.

The NLP community spent two decades building task-specific labeled datasets: natural language inference, sentiment classification, summarization, machine translation, reading comprehension, commonsense reasoning. Each dataset is already a collection of (input, output) pairs. The only missing piece is a natural-language instruction that wraps the pair and tells the model what task it is performing. The FLAN approach and SuperNatural Instructions demonstrated independently that if you templatize these datasets at scale-writing a handful of natural-language templates per task and applying them to every example-a model fine-tuned on the result generalizes to tasks it was never explicitly trained on.

The recipe is mechanical. For each source dataset, write k natural-language templates, typically $k = 2$ to 10, that describe the task in different words. For a natural language inference example with premise p , hypothesis h , and gold label $y \in \{\text{entailment, neutral, contradiction}\}$, one template might be:

```
Premise: {p}
Hypothesis: {h}
Does the hypothesis follow from the premise?
Options: Yes / No / It's impossible to tell
Answer: {y}
```

A second template for the same example might ask “Is the following hypothesis true given the premise?” with a different label mapping. SuperNatural Instructions collected 1,616 tasks from the NLP community, each with two or more manually written templates, producing millions of examples covering classification, generation, extraction, and structured prediction-all formatted in the same prompt-response structure.

The fundamental move is re-framing every existing supervised dataset as an instance of instruction following, so one fine-tuning run buys transfer across an entire task taxonomy.

The primary advantage is data leverage. Decades of annotation work in the NLP community become nearly free training signal with minimal new human effort. A templating pass converts a 100k-example NLI dataset into a 300k-example instruction dataset in hours. The secondary advantage is benchmark coverage: a model trained on FLAN-style data sees labeled examples from the same task families that standard evaluation benchmarks draw from, which is a direct contributor to strong HellaSwag, MMLU, and BIG-Bench numbers.

The weakness is that the prompts are templated and therefore artificial. If you sample randomly from a SuperNatural Instructions mix, many examples share the same sentence skeleton and differ only in their filled slots. Real users ask questions in idiosyncratic, messy ways: “can u explain why the EU passed that regulation”

looks nothing like “Given the following passage, extract the key regulatory rationale.” Models trained only on FLAN-style data perform well on structured benchmarks but feel robotic and brittle in open-ended conversation. The pre-training data for most models contains documents like Stack Exchange Q&A and Reddit threads that already resemble instruction-response pairs; FLAN-style fine-tuning is unlocking that latent capability, not installing it from scratch.

There is also a saturation effect. FLAN-style data helps enormously early in fine-tuning—it moves the model from “coherent text continuation” to “instruction following”—but has diminishing returns past a certain coverage threshold. The empirical consensus is that 1,000+ diverse tasks from well-curated templates is sufficient; adding more tasks beyond that contributes less than improving the quality and diversity of examples within existing tasks.

A Worked Example

Suppose you are building a FLAN-style mix from three source datasets: a 400k-example NLI corpus, a 50k-example extractive QA corpus, and a 200k-example summarization corpus. Two annotators each spend a day writing three templates per task—nine templates total.

For NLI you get $400k \times 3 = 1.2M$ examples. For QA you get $50k \times 3 = 150k$ examples. For summarization you get $200k \times 3 = 600k$ examples. Total: roughly 1.95M instruction pairs before deduplication.

You fine-tune a 7B-parameter base model on this mix for one epoch—approximately 2B tokens at an average of 1k tokens per example on an 8-GPU H100 node, which completes in under 12 hours. The model that previously generated incoherent continuations now correctly handles NLI, QA, and summarization prompts. More importantly, if you prompt it with a paraphrase task it has never seen, the model attempts it rather than ignoring the instruction. That zero-shot generalization to unseen tasks is precisely what FLAN-style training buys.

Now suppose you extend the mix with a commonsense reasoning dataset containing 100k examples and three templates. You do not need to retrain from scratch: extend the mix to 2.25M examples and fine-tune for an additional 0.5 epochs. Benchmark scores on commonsense tasks improve by 4–6 points while regression on the original three tasks is minimal—provided you weight sampling proportional to task size and keep template vocabulary diverse across tasks.

What You Actually Decide in Practice

- **Template count vs. template quality.** Three to five high-quality templates per task consistently outperforms ten mediocre ones. Template quality means: varied surface forms, varied label vocabulary, varied prompt ordering. If two templates are paraphrases of each other, they provide almost no additional diversity signal.
- **Task sampling strategy.** Uniform sampling over examples—not tasks—over-represents large datasets and starves small ones of gradient signal. Temperature-based sampling, where task i is drawn with probability proportional to $|D_i|^T$ with $T \approx 0.7$, smooths coverage without fully equalizing tiny tasks against large ones. Below roughly 1,000 examples per task, quality degrades because the model sees too few template variations to internalize the task format.
- **When to stop and move on.** FLAN-style data is the right tool for getting a base model to follow instructions at all. Once benchmark gains from adding new tasks fall below 0.3 percentage points per additional 100k examples, further improvement comes from higher-quality open-ended data—synthetic or human-written—not more templated tasks.

- **Replay a pre-training fraction.** Fine-tuning only on instruction data with no raw-text signal causes forgetting, especially on long-form generation. Including a 10-20% “replay” fraction of pre-training tokens in the instruction fine-tuning batch stabilizes perplexity on held-out raw text without measurably hurting instruction-following performance.
- **Label noise compounds through templating.** A 15% label error rate in the source dataset becomes a 15% error rate across all derived templates. A 10k-example NLI set with clean labels beats a 500k-example set with 15% noise; expect roughly a 2-point downstream gap. Audit source dataset quality before investing in templating effort.

How This Shows Up in Interviews

- **“What is the difference between a base model and an instruct model, and how does data drive that gap?”** Strong answer: a base model is trained on next-token prediction over raw text and interprets any prompt as a document to continue. An instruct model is fine-tuned on (instruction, response) pairs—beginning with FLAN-style templated NLP tasks—which teaches the model to interpret a user turn as a directive and produce a targeted response rather than a free continuation. The architectural difference is typically zero; the behavioral gap is entirely in the data.
- **“How would you build an instruction dataset from scratch with a small team and a limited budget?”** Strong answer: inventory existing labeled NLP datasets in your target domain, write three to five templates per task, and convert them into a unified prompt-response format. Prioritize task diversity over raw example count—100 tasks at 10k examples each generalizes better than one task at 1M examples. Layer in real or synthetic open-ended conversations once the model reliably follows structured instructions.
- **Senior follow-up: “FLAN-style models score well on benchmarks but often feel mechanical in free-form chat. What is the root cause, and how would you fix it?”** Root cause: benchmark-derived templates are structurally regular; real conversational prompts are irregular, context-dependent, and often underspecified. To close the gap, augment the FLAN mix with open-ended conversational pairs from human annotators, real user logs with appropriate consent, or a strong model prompted with diverse seed questions. Weight the open-ended component so it represents at least 30-40% of the final instruction mix, and oversample task types that exhibit the mechanical feel on a held-out conversational eval.

Key Takeaways

- FLAN-style training converts existing labeled NLP benchmarks into instruction-following data at near-zero marginal annotation cost by wrapping each (input, output) pair in a natural-language task template.
- The core benefit is cross-task generalization: a model that sees 1,000+ templated tasks acquires a general instruction-following prior that transfers to unseen tasks at inference time.
- Template rigidity is the central weakness—structured prompts do not match the messiness of real user queries, so FLAN-style data must be supplemented with open-ended conversational data in any production instruction mix.
- Use temperature-based task sampling ($T \approx 0.7$) rather than uniform example sampling to balance coverage across tasks of different sizes.
- Replay 10-20% pre-training tokens during instruction fine-tuning to prevent forgetting on long-form generation, and audit source dataset label quality before investing in large-scale templating.

22.1.2 Human-Written vs. AI-Generated Instruction Data

Once instruction-following data is recognized as the lever that turns a base model into a useful assistant, the next question is immediate and practical: where does it come from? Paying humans to write examples is expensive and slow; prompting a stronger model to generate them is cheap and fast but carries license exposure and a quality ceiling bounded by the teacher. Every lab I know has grappled with this trade-off explicitly, and it surfaces in system design interviews-usually phrased as “how would you scale your SFT dataset?”-at every level from L4 upward.

There are three production-grade sources of instruction data, each with a distinct cost and quality profile.

Human-annotated data. Contractors or in-house annotators write (prompt, response) pairs following a rubric. Quality is high when the rubric is tight and the annotators are domain experts. Cost runs roughly \$10-\$100 per example depending on task complexity and response length; throughput is capped by headcount. Llama 2’s post-training data included human-authored examples, and the team reported that a modest number of high-quality human-written pairs outperformed millions of open-source synthetic examples for instruction following.

Distillation from a stronger model. A capable model is prompted to generate responses to a set of seed questions. The resulting (question, response) pairs are used to fine-tune a smaller target model. This is the dominant approach in the open community. Alpaca used a self-instruct loop against GPT-3 to generate 52k examples; Vicuna was trained on ChatGPT conversations shared publicly on ShareGPT; OpenHermes is an agglomeration of multiple GPT-4-sourced community datasets; the Llama-Nemotron post-training corpus draws from Llama, Qwen, and other open-weight models to avoid proprietary license issues.

Self-play and evol-instruct synthesis. The model itself-or a sibling model-is seeded with a pool of questions and asked to both expand the pool and answer the new questions. Evol-instruct methods take existing questions and iteratively mutate them to be more complex, more specific, or more constrained, producing a diversity gradient from simple to hard.

The self-instruct loop from the Alpaca work is representative of the distillation approach:

1. Seed pool: ~175 hand-written (instruction, input, output) triples
2. Prompt teacher model with k=8 in-context examples sampled from the pool; generate a new instruction
3. Filter: drop if ROUGE-L overlap with any pooled example > 0.7, or if output is malformed (empty, refusal, truncated)
4. Add accepted example back to pool
5. Repeat until target corpus size is reached

The self-instruct loop converts a few hundred seed examples into tens of thousands of training pairs; the ceiling on quality is the teacher model’s own capability, and that ceiling descends into the student.

Human annotation wins on quality ceiling and legal cleanliness. For tasks where the teacher model is known to be unreliable-highly regulated domains, multi-step mathematical reasoning before strong reasoning models existed, low-resource languages-the only defensible path is a human expert who understands the failure modes. Human-written data also carries no downstream license risk: if your annotators wrote it, you own it.

AI generation wins on cost, coverage, and throughput. A single GPU-day on a 70B model produces 100k diverse examples; the equivalent human annotation would cost tens of thousands of dollars and weeks of turnaround. Coverage diversity is also genuinely hard to match by hand: prompting a strong model to answer

questions spanning astrophysics, tax law, culinary chemistry, and software debugging gives you breadth that no annotation workforce could deliver efficiently.

The critical failure mode for AI-generated data is confident wrongness. A teacher model does not generate examples it is uncertain about; it generates fluent, plausible-sounding responses that happen to be incorrect. The student model learns that confident, fluent responses are correct by construction—exactly the hallucination problem. When you later evaluate with a judge model, you may not even catch this because the judge has the same blind spots as the teacher. Human-written data at least has domain experts who know when to say “I don’t know.”

A Worked Example

Consider building a 100k-example SFT dataset for a coding assistant. Budget: roughly \$5,000 and two weeks of engineering time.

Option A-human annotation. Hire 20 contractors at \$25 per example, each writing Python and SQL prompt-response pairs. At 10 examples per contractor per day, you produce 2,000 examples per day—reaching 100k in 50 working days. That is 10 weeks of calendar time and \$2.5M. Clearly not viable.

Option B-distillation from Llama 3 70B. Write 500 seed prompts covering Python, SQL, debugging, code review, and docstring generation. Run the self-instruct loop prompting Llama 3 70B via 8 H100s. Apply an execution filter: run every generated code block through a Python interpreter and discard examples where the snippet raises an uncaught exception. This filter removes roughly 15% of outputs and is the single highest-leverage quality gate for code data. In three GPU-days you produce 100k filtered examples at a compute cost of approximately \$600. License check: Llama 3’s acceptable use policy permits training derivative models for organizations below the commercial threshold, which covers most teams.

The resulting 7B model fine-tuned for two epochs scores 5-8 points below a model trained on premium human-written coding data on a hard held-out benchmark. For most teams below frontier scale, that gap is the correct economic trade-off. Human annotation is then reserved for the specific task slice—perhaps adversarial debugging or multi-file refactoring—where AI generation is demonstrably wrong on the held-out eval.

What You Actually Decide in Practice

- **Read the teacher model license before you generate a single example.** OpenAI’s terms of service explicitly prohibit using GPT-4 outputs to train a competing model; violation exposes you to contract breach, not just moral awkwardness. Open-weight models—Llama, Mistral, Qwen—have more permissive terms, but Llama still restricts commercial use above certain active-user thresholds. The right answer is to read the actual license document, not a summary in a blog post.
- **Apply an automated quality gate matched to the task type.** For code: execute the snippet and discard failures. For structured outputs (JSON, SQL, function calls): parse and schema-validate. For factual QA: optionally run a retrieval check against a known-correct source. A well-targeted filter removes 10-20% of AI-generated examples and consistently improves downstream evaluation scores by 2-5 points relative to raw unfiltered generation.
- **Diversity matters more than volume past roughly 10k examples.** Generating 500k examples from 20 seed prompts produces a dataset with shallow diversity that adds little over 50k. Measure diversity concretely: compute n-gram entropy over instructions, or cluster instruction embeddings and measure the

fraction of clusters with fewer than five members. When diversity stops growing, expand the seed set rather than generating more from existing seeds.

- **Reserve human annotation budget for the hard tail.** Run a small held-out evaluation first. Identify the task slices where the AI-generated model fails at rates above your threshold. Allocate annotation to exactly those slices. Human annotation at \$50 per example is only expensive if you spend it on tasks where AI generation already works.
- **Defend against annotator leakage.** When Llama 2’s team ran human annotation, they acknowledged a real risk: contractors querying GPT-4 and pasting the output as their “human-written” submission. A practical detector is to measure the perplexity of the submitted response under the suspected teacher model. Responses that score anomalously low-lower than median responses from genuine domain experts-are likely machine-generated. Flag them for review rather than discarding automatically, since a domain expert might produce text that happens to look similar to GPT-4 output simply because they are both correct.

How This Shows Up in Interviews

- **“You have a \$50k budget and six weeks to build an SFT dataset for a domain-specific assistant. Walk me through your approach.”** Strong answer: spend week one identifying the five to ten task types the assistant must handle and running an AI-generation pilot on each to measure failure rate. Allocate human annotation to the one or two types where AI generation fails above threshold-likely domain-specific reasoning or regulated content. Use distillation from an open-weight 70B model for the remainder with task-appropriate quality filters. Plan for 70-90k AI-generated examples and 5-10k human-written ones; oversample the human examples in the training batch by a factor of three to compensate for volume imbalance.
- **“What are the risks of training on GPT-4-generated data?”** Strong answer: three distinct risks. First, the OpenAI terms of service prohibit using model outputs to train a competing model-this is a legal and contractual risk, not just an ethical concern. Second, the student model inherits the teacher’s failure modes without the teacher’s uncertainty calibration: it hallucinates confidently in the same situations GPT-4 does, because those are precisely the situations where the training examples were wrong. Third, GPT-4’s style and safety filters shape the data distribution in ways that may not match the deployment context-you are distilling a general assistant when you may need a specialized one.
- **Senior follow-up: “Llama 2 reported that a small set of human-written examples outperformed millions of open-source synthetic examples. Is that a general result?”** Senior answer: it is regime- and quality-dependent, not a fundamental law. The open-source synthetic datasets available at the time Llama 2 was trained were low-diversity and generated from weaker teacher models. When the quality of AI-generated data is controlled-strong teacher, diversity-enforced seed set, automated quality filtering-it consistently matches or exceeds small human-written corpora at fraction of the cost. The Llama 2 paper also noted that RLHF on top of either data source recovered much of the gap, which suggests the human-annotation advantage was partly about capturing preference signal rather than instruction quality alone. The operationally correct interpretation is: human annotation is worth the cost at the thin tail where the task is genuinely beyond AI generation; everywhere else, distillation with a quality gate is the right call.

Key Takeaways

- Human annotation provides the highest quality ceiling and full legal cleanliness but costs \$10-\$100 per example; reserve it for the hard tail where AI generation demonstrably fails.
- Distillation from open-weight models (Llama, Qwen, Mistral) is the dominant cost-effective path-but always verify the license before generating training data, since proprietary model terms often prohibit training competitors.
- The self-instruct loop scales a few hundred seed prompts to 100k+ examples in GPU-days; quality is bounded by the teacher model and improved most reliably by automated execution or format filters that remove 10-20% of raw outputs.
- Diversity matters more than volume past ~10k examples: measure n-gram entropy or embedding cluster coverage and expand the seed set when diversity plateaus rather than generating more from existing seeds.
- Annotator leakage-contractors querying a proprietary model to produce nominally human-written data-is a real production risk; detect it by measuring submission perplexity under the suspected teacher model.

22.2 Long-Context and Domain Data

22.2.1 Books and Math for Context Extension

When an interviewer asks you to extend a 7B model's context from 4k to 128k tokens, most candidates immediately reach for RoPE scaling factors and overlook the data question entirely. What you actually train on during the extension phase determines whether the model genuinely attends across the full window or merely pattern-matches on position encodings it has never encountered. Long-context extension appears in senior and staff-level interviews because it sits at the intersection of architecture, training schedule, and data curation-and getting the data mix wrong wastes weeks of expensive GPU time.

The fundamental problem is cost. Full attention scales as $O(L^2)$ per layer in both compute and memory, where L is the sequence length. If standard pre-training runs at $L = 4096$, extending to $L = 131072$ (128k tokens) multiplies attention cost by a factor of $(131072/4096)^2 = 1024$ at each layer. Training at that length from the first step would consume the entire compute budget before the model has learned basic language patterns, so the canonical practice is to defer long-context training to a dedicated mid-training phase once a strong 4k checkpoint exists.

The data requirement for this phase is specific: you need documents with *genuine long-range dependencies*, not simply long documents. A 128k-token chunk assembled by concatenating 32 unrelated 4k web pages has no dependency structure beyond each page-the model learns nothing about attending across the full window. Books are the canonical source because they encode narrative arcs, character motivations, and thematic call-backs that span tens of thousands of tokens. The PG-19 dataset, derived from Project Gutenberg, was specifically constructed to benchmark long-context language models because book chapters force models to track information from much earlier in the sequence. Mathematics and formal papers are complementary: proof steps and derivations maintain tightly coupled variable definitions across hundreds of lines, creating a different pattern of long-range dependency that covers reasoning chains rather than narrative continuity.

Long-context extension is a mid-training phase, not a pre-training phase, because $O(L^2)$ attention cost makes early long-context training economically irrational, and the model is not capable enough early on to benefit from the distant context anyway.

Books force genuine cross-sentence coherence. A reference to a character’s internal conflict in chapter 15 connects back to something established in chapter 1—the only way to assign meaningful probability to that reference is to actually use the distant context. This is precisely the training signal a context-extension phase needs. Random web text concatenated to fill a 128k window teaches the model that context is useless beyond 4k, because it is: the adjacent documents are unrelated. You can verify this failure mode by checking attention entropy across positions in a model trained on stitched documents—heads at position 60k are essentially ignoring positions 1–40k.

The trade-off versus starting long-context training from scratch is purely economic. A model trained from step zero at 128k would require hundreds of billions of tokens to reach base language quality, burning roughly $1024\times$ the compute per token at the attention layer compared to 4k training. The mid-training approach inherits all the knowledge from cheap 4k pre-training and spends a much smaller budget—typically 20–50B tokens—adapting the positional encoding and calibrating the attention patterns for the longer window.

Against the alternative of simply relying on sliding-window or sparse attention to avoid the quadratic cost: those approaches reduce the compute burden but also reduce quality at tasks that genuinely require attending across the full window. Full attention at mid-training, on the right data, remains the highest-quality path for frontier models.

A Worked Example

Suppose you have a 7B model pre-trained to 200k steps at $L = 4096$. Your target context length is 128k. Hardware budget: 256 H100s (32 nodes of 8). Estimated mid-training budget: 40B tokens.

Data mix:

- 65% books (PG-19 and similar long-form corpora): average document length 80k–200k tokens, packed contiguously by chapter.
- 20% arXiv and math papers: average document length 10k–40k tokens, packed contiguously per paper.
- 15% long-form web documents: filtered to documents with raw length $> 32k$ tokens, using the original HTML parser (not WET) to preserve structure.

Sequence packing rule: fill each 128k-token training chunk from a single document or from contiguous chapters of one book. Never pack unrelated documents into the same chunk. This is non-negotiable—mixing documents destroys the long-range dependency signal.

Graduated schedule: Run 10B tokens at $L = 32768$, then 10B tokens at $L = 65536$, then 20B tokens at $L = 131072$. Each transition applies a learning rate warmup of 500 steps to absorb the change in positional encoding behavior. Jumping directly from 4k to 128k causes loss spikes because the position encodings have never been exercised at intermediate ranges.

RoPE adjustment: Apply NTK-by-parts interpolation before the extension phase begins, scaling the base frequency by a factor of $(128k/4k)^{1/(\dim/2-1)} \approx 8$ for a 128-dimensional head. This initializes the position encodings to a state the optimizer can refine rather than one it must escape from.

At the end of the 40B-token schedule, perplexity on long-document benchmarks improves by 15–25% relative to the base 4k model, while short-context benchmark scores degrade by less than 2% if you maintain 10% general web text in the mix.

What You Actually Decide in Practice

- **Never skip the graduated schedule.** Jumping from 4k to 128k in one step causes optimizer instability because the gradient signal from positions 4k-128k is initially near-random. Two or three intermediate lengths costing 10-15B tokens each are always worth the insurance.
- **Maintain 10% general web text throughout the extension phase.** Removing short-context data entirely for the sake of long-document purity causes capability regression on standard benchmarks. The loss on short-context tasks climbs noticeably after 5B tokens of pure long-context training.
- **Use document-contiguous packing, always.** If your packing code splits chapters at random boundaries, audit it before the run starts. Splitting a 90k-token chapter at the 65k mark and continuing in the next chunk gives the model a truncated signal. Either pack full documents or set a hard minimum document length equal to the chunk size.
- **Books are not just any long text.** A 128k chunk assembled from 30 news articles concatenated in order has a document-length of 128k but a coherence length of roughly 4k per article. Use documents where the author intended a reader to track information over the full length: books, long-form investigative journalism, technical manuals, multi-part legal filings.
- **Evaluate before the extension phase ends.** Run a checkpoint at 20B tokens through a long-context needle-in-a-haystack test. If the model cannot retrieve facts from the middle of a 64k window, the data packing is likely wrong-fix it before burning the rest of the budget.

How This Shows Up in Interviews

- **“Walk me through how you would design a context-extension phase for a model that currently supports 8k tokens but needs to reach 1M.”** Strong answers cover the graduated schedule, document-contiguous packing, the choice of books and math as the primary corpus, and the architectural adjustment (RoPE scaling). Weak answers describe only the architecture change with no mention of data.
- **“Why not train at long context from the beginning?”** The answer is cost: $O(L^2)$ attention means long-context training per token is orders of magnitude more expensive. The model is not capable enough in early training to exploit distant context, so the compute would be wasted. Mid-training extension amortizes that cost over a checkpoint that already has strong language priors.
- **Senior follow-up: “Your book corpus is 90% genre fiction. What does that mean for the model’s long-context capability?”** The senior answer identifies that the model learns long-range narrative coherence patterns specific to fiction-character tracking, dialogue attribution, scene transitions-but misses the dependency structures in technical writing, legal documents, and formal mathematics. To build general long-context capability, you need domain-diverse long documents: books plus arXiv papers plus legal filings plus long-form technical documentation. Genre imbalance in the extension corpus produces a model that can retrieve story facts from 80k tokens but struggles to track variable definitions across a long proof.

Key Takeaways

- Long-context extension is a mid-training phase because full attention scales as $O(L^2)$ -deferring it to after pre-training saves the majority of compute while preserving the quality benefit.
- Books and math papers are the canonical sources because they contain genuine long-range dependencies: narrative arcs and formal proof chains that force the model to use the full window.

- Always pack documents contiguously; mixing unrelated documents into one long chunk destroys the dependency signal and teaches the model that distant context is noise.
- Use a graduated sequence-length schedule with intermediate lengths-jumping directly from 4k to 128k causes optimizer instability and loss spikes.
- Retain 10% general web text in the extension mix to prevent short-context capability regression.

22.2.2 Synthetic Rephrasing and Task Conversion

If aggressive quality filtering retains only 1-2% of your raw token pool, you face a hard arithmetic problem: frontier models now train on 15-36 trillion tokens, and a 3.8T-token baseline like DCLM may not sustain a large model for a full training run. Two techniques developed in recent mid-training pipelines attack this problem directly-rephrasing low-quality documents into usable prose, and converting high-quality documents into structured instruction-following examples. Both appear in senior interviews framed as “how do you get more usable signal out of data you already have without crawling more of the web?”

Rephrasing uses a capable language model to rewrite documents that fall below your quality threshold into cleaner prose. The source document supplies the factual content; the rewrite model normalizes grammar, removes boilerplate and template noise, and restructures run-on sentences. The resulting token is synthetic, but the information it encodes is grounded in original text rather than hallucinated from scratch.

Task conversion applies a language model to high-quality source documents to generate structured input-output pairs. Given a Wikipedia article or a math textbook passage, the conversion model produces multiple task variants:

Source: Wikipedia article on Bernoulli numbers

Task variants generated:

QA: Q: "What is the generating function for Bernoulli numbers?"
A: " $t / (e^t - 1) = \sum_{n=0}^{\infty} B_n * t^n / n!$ "

Summary: Input: [article text]
Output: [3-sentence summary]

Extract: Input: [article text]
Output: [key formulas and their names]

The general form is: given a source document D , generate $\{(\text{prompt}_i, \text{response}_i)\}$ for $i \in \{\text{QA, summarize, extract, class}\}$. This is precisely the transformation that FLAN and Supernatural Instructions applied to NLP benchmark datasets in 2022-the Nemotron-CC work extended it to raw corpora at scale.

Rephrasing extends usable token count beyond what filtering alone allows; task conversion pre-loads instruction-following structure before post-training begins.

The token yield argument for rephrasing is concrete. DCLM-baseline filtered 240 trillion raw tokens down to 3.8 trillion-a 1.4% retention rate-using an aggressive fastText classifier. Nemotron-CC, operating on the same Common Crawl pool with a combination of quality bucketing, rephrasing, and task conversion, produced 6.3 trillion tokens: roughly $1.7\times$ the DCLM yield, and Nemotron-CC achieved better average benchmark scores than DCLM-baseline on downstream evaluations. The compute spent rephrasing borderline documents was cheaper than the alternative of crawling additional raw data or accepting the smaller token budget.

Task conversion addresses a different problem. Standard pre-training data is overwhelmingly document-shaped: a model trained on it learns to continue documents, not to answer questions posed by a user. The gap between “document continuation” and “respond to an instruction” is what post-training is designed to close, but the gap is easier to close if some instruction-following structure was already present in mid-training. Models that see task-formatted data during mid-training require fewer post-training examples to reach the same instruction-following quality-the gradient signal is not starting from zero.

The risk of rephrasing is error propagation: if the rewrite model makes a factual error or hallucinates, that error enters the training corpus as if it were original text. This is why rephrasing should target only documents that a quality classifier scores in the lower-quality band-documents already unreliable enough that the rephrase is no worse than the original. Rephrasing high-quality sources like Wikipedia introduces risk with no gain.

A Worked Example

Suppose your quality classifier partitions 10 trillion filtered tokens into three tiers based on educational value score:

- **Tier A (top 20%, score > 0.8):** 2T tokens. Source: Wikipedia, arXiv, high-quality STEM sites. Action: keep as-is; additionally run a 70B Nemotron-class model to generate 3 task variants per document (QA, summarization, extraction). Estimated additional yield: $2T \times 3 \times 0.6k$ avg response tokens / $3k$ avg doc tokens $\approx 1.2T$ task tokens.
- **Tier B (middle 50%, score 0.4-0.8):** 5T tokens. Source: general web pages with moderate coherence. Action: rephrase using a distilled 7B rewrite model. Expected quality lift: score distribution shifts to 0.6-0.85. Yield after deduplication: approximately 4.5T tokens.
- **Tier C (bottom 30%, score < 0.4):** 3T tokens. Action: discard. Rephrasing severely incoherent text is not cost-effective-the rewrite model cannot reliably recover factual content from near-nonsense.

Total yield: 2T (Tier A original) + 1.2T (task pairs) + 4.5T (Tier B rephrased) = 7.7T usable training tokens, compared to 2T if you kept only Tier A or $\approx 3.8T$ if you applied aggressive filtering without rephrasing.

Compute cost estimate for rephrasing 5T tokens: at 500 output tokens per second on a single H100 in bf16, and assuming a 3:1 input-to-output length ratio for rephrasing (each 4k document produces a 4k rewrite), you need roughly:

$$\frac{5 \times 10^{12} \text{ tokens}}{500 \text{ tokens/sec} \times 256 \text{ GPUs}} \approx 39,000 \text{ GPU-hours}$$

That is a small fraction of the pre-training GPU budget for a 7B model over 6T tokens, which runs into the millions of GPU-hours. The yield increase justifies the rephrasing compute.

What You Actually Decide in Practice

- **Rephrase only the middle quality tier.** Tier A documents are already usable; rewriting them adds noise. Tier C documents are too incoherent for reliable rephrasing-the rewrite model will hallucinate to fill gaps. The 20th-70th percentile by quality score is the productive target range.
- **Diversify task conversion templates.** If you only generate QA pairs, the model memorizes QA format rather than developing general instruction-following. Aim for at least five distinct task types per source document: question answering, summarization, key-point extraction, classification, and open-ended generation. Ask the conversion model to vary its phrasing rather than slot-filling a fixed template-template

memorization is a real failure mode.

- **Use a rewrite model significantly stronger than the target.** A 7B rewrite model fine-tuned on 70B outputs sets a quality ceiling below the 70B model’s level. A 3B rewrite model introduces systematic errors-collapsed clauses, dropped numerical precision, consistent tense mistakes-that are subtle enough to survive perplexity filtering but harmful at scale. The rewrite model should be at least $4\times$ the parameter count of the target when possible.
- **Mix rephrased and original tokens; do not replace.** Training exclusively on rephrased documents causes distributional drift from the original web distribution. Keep at least a 1:1 mix of rephrased and original text in the final data blend. The original anchors the distribution; the rephrase improves local quality.
- **Evaluate on held-out tasks not in the conversion templates.** If your only evaluation is on tasks structurally similar to the generated pairs, you are measuring format memorization, not capability. Reserve a held-out set of instruction tasks whose format was never in the training templates; improvement on those is evidence of real generalization.

How This Shows Up in Interviews

- **“Your quality filter throws away 98% of your Common Crawl tokens. How do you get more usable data without crawling more of the web?”** Strong answers cover both rephrasing and task conversion, with a concrete split of which documents to rephrase versus discard. Quantify the expected yield increase and note the compute cost. Weak answers mention “synthetic data” without distinguishing rephrasing (salvage) from task conversion (augmentation) or discussing when each applies.
- **“Is it circular to use a language model to generate training data for a language model?”** The strong answer: not if the rewrite model is significantly stronger than the model being trained. The quality ceiling is the rewrite model’s quality-you are not training a model on its own outputs, you are distilling from a stronger model. Self-distillation without an external stronger teacher does collapse over repeated iterations; that is why production rephrasing pipelines use a dedicated, larger rewrite model rather than the checkpoint being trained.
- **Senior follow-up: “How do you verify that rephrased tokens actually improved model quality and did not just inflate your token count?”** The senior answer: run a controlled ablation on identical token budgets (not identical document counts). Train one model on N rephrased tokens mixed with N original tokens; train another on $2N$ original tokens from the next quality tier. Evaluate both on held-out benchmarks that were not part of the rephrasing or task-conversion process. If the rephrased model wins on held-out tasks, the quality improvement is real. If it only wins on tasks that look like the task-conversion templates, it is overfitting to format and the templates need diversification.

Key Takeaways

- Rephrasing uses a stronger LM to rewrite borderline-quality documents into cleaner prose, extending usable token count beyond what filtering alone can achieve-Nemotron-CC nearly doubled token yield relative to DCLM-baseline using this approach.
- Task conversion generates instruction-formatted input-output pairs from high-quality documents, priming instruction-following structure before post-training and reducing the post-training data requirement.

- Rephrase only the quality middle band (roughly 20th-70th percentile); high-quality documents do not benefit, and very low-quality documents cannot be reliably salvaged.
- The rewrite model’s capability sets the ceiling: use a model substantially larger than the target to avoid amplifying systematic errors into the training corpus.
- Validate on held-out tasks whose format was never in the conversion templates; improvement there distinguishes real capability gain from format memorization.

22.3 Data Quality Heuristics That Matter in Interviews

22.3.1 Length, Style, and Citation Traps

An interviewer asks: “Walk me through how you would curate a high-quality instruction-tuning dataset.” The candidate who answers “I would collect long, detailed, expert-written responses with citations” sounds credible-and is describing a trap. Three systematic biases in post-training data collection-length inflation, style mimicry, and citation hallucination-consistently degrade model quality while appearing as improvements during collection. This topic appears at L5-L6 interviews whenever the discussion reaches SFT data curation, RLHF reward modeling, or why instruction-tuned models hallucinate despite being trained on “correct” responses.

Instruction tuning works by showing a pre-trained model (instruction, response) pairs and fine-tuning it to produce the response given the instruction. The model simultaneously learns two things: the *content* of the response and its *type signature*-the surface structure, length, and formatting conventions that the training distribution exhibits. These two learning signals are inseparable at the level of cross-entropy loss, which creates three failure modes.

Length bias. Human evaluators prefer longer responses by roughly 60-70% in pairwise comparisons, even when the longer response contains filler or additional hallucinations. AI judges trained on human preferences inherit this bias. A model trained via RLHF on length-biased reward signals learns to pad rather than improve accuracy. Studies have found that instruction-tuned models with dramatically different average response lengths achieve nearly identical MMLU scores-benchmark performance is largely decoupled from length-while chat evaluation scores such as AlpacaEval win rates track length closely. Optimizing win rate without length normalization is optimizing for verbosity.

Style mimicry. SFT reliably teaches output style. If 60% of training responses use bullet-pointed lists, the fine-tuned model uses bullet points even when prose is more appropriate. If training data ends responses with emojis, the model learns to generate emojis at the end as a positional pattern, regardless of semantic relevance. This is not pathological; it is exactly what next-token prediction does. The problem is that style conventions in annotation data reflect annotator culture and time constraints, not what is actually useful to end users.

Citation traps. This is the subtlest failure mode. Consider a training example with the instruction “Write an introduction to monopsony in economics” and a response ending with a formatted citation such as Bivens & Mishel (2015). Two things happen during fine-tuning simultaneously. If the model already encodes that citation from pre-training, it learns to surface it appropriately-a genuine improvement. If it does not, it learns something worse: whenever it encounters a complex academic prompt, it should append a plausible-looking citation. The model has been trained to hallucinate in a structured way, and the training loss cannot distinguish between these two outcomes.

SFT teaches the type signature of outputs unconditionally; the model cannot distinguish between “I know this fact and should cite it” and “I do not know this fact but the training distribution expects a citation here.”

The citation trap comes down to a mismatch between what the cross-entropy loss measures and what we want. The loss penalizes the model equally for failing to produce a correct citation it already knows and for failing to hallucinate a citation it does not know. From the loss’s perspective, producing any plausible citation token is strictly better than producing nothing. A model minimizing loss on citation-heavy training data therefore learns hallucination as a default structural behavior.

The principled fix is on-policy reinforcement learning, because RL only reinforces behaviors that were already latent in the model. A generation that correctly cites a known fact gets rewarded; one that invents a citation does not, regardless of stylistic plausibility. This is one of the practical arguments for why PPO and DPO sit downstream of SFT in production pipelines—they can correct for what SFT mis-teaches.

Length and style biases have a more direct remedy: measure preference scores as a function of response length, deconfound the two, and normalize win rates at annotation time. Presenting annotators with length-matched pairs eliminates the bias at the source. For AI judges, running ablations that hold content fixed while varying length—and measuring how much the preference score shifts—gives you an empirical length-bias coefficient you can subtract.

A Worked Example

You are building an instruction-tuning dataset for a 7B model. You use a strong teacher model to generate 50,000 responses, resulting in an average response length of 480 tokens with frequent inline citations. You fine-tune and evaluate: Chatbot Arena win rate against the base model is 62%. Encouraged, you run MMLU—the fine-tuned model and base model are within 0.3 percentage points. You run TruthfulQA: the fine-tuned model is 4.1 points *worse*. You manually check 200 responses: 18% contain at least one hallucinated citation or unsupported specific statistic.

The diagnosis is length inflation from the teacher model combined with citation mimicry. The 7B student learned the citation structure without having the underlying factual knowledge to support it.

The fix proceeds in three steps. First, filter training examples where response length exceeds $3\times$ the median for that prompt category—this removes the longest-tail padding without truncating genuinely complex responses. Second, strip inline citations from training responses, replacing specific references with non-specific language such as “research in this area suggests.” Third, add 5,000 examples where the correct response is a calibrated hedge—“I don’t have reliable information on that specific claim”—for prompts where the model’s pre-training coverage is likely sparse. After retraining: TruthfulQA recovers by 3.2 points; Chatbot Arena win rate drops from 62% to 58%. That is a real trade-off, and the correct one for any deployment that cares about factual reliability.

What You Actually Decide in Practice

- **Separate style from substance in your evaluation stack.** Run benchmark-based evals (MMLU, TruthfulQA, GSM8K) in parallel with preference evals (Chatbot Arena, AlpacaEval). If they diverge, trust benchmarks for capability claims. Use preference evals only for UX signal and style tuning.
- **Control length at annotation time, not post-hoc.** If collecting human-written SFT data, instruct annotators to target a specific length range—for example, 150-300 words for factual Q&A, 400-600 for multi-step reasoning. Post-hoc truncation introduces artifacts; a front-loaded constraint produces more natural responses.

- **Strip or verify citations before including them in SFT data.** Either remove all inline citations from training responses, or run a verification step that confirms the model can reproduce the cited fact from a zero-shot prompt before including that example. Unverified citations produce a hallucination-optimized model by construction.
- **Distillation from a much stronger teacher compounds the citation trap.** If distilling GPT-4-quality responses into a 7B model, the capability gap means many responses will teach behaviors the student cannot support from its own knowledge. Prefer prompts where the student already gets the answer roughly right, and use the teacher to improve surface quality rather than inject fundamentally new factual claims.
- **500 carefully chosen safety examples can dominate model behavior.** Because SFT runs on relatively few tokens, safety data has outsized influence. Use this leverage deliberately-and audit the safety data for the same citation and length biases, because a safety response that hallucinates specifics is worse than a vague refusal.

How This Shows Up in Interviews

- **“What makes a good instruction-tuning dataset?”** A strong answer names three axes: diversity of task types, correctness of responses relative to the model’s existing knowledge (not just surface quality), and calibration-explicit hedges on low-confidence claims rather than confident fabrication. Weak answers focus on response length or vague notions of “high quality.”
- **“Why do instruction-tuned models hallucinate more than base models on certain tasks?”** The answer is the citation trap: SFT teaches type signatures unconditionally. If training data contains responses with citations, the model learns to produce citations as a stylistic pattern regardless of whether it has the underlying knowledge. The fix requires either on-policy training that rewards only verifiable correctness, or data hygiene that removes unverifiable factual statements before SFT.
- **Senior follow-up: “How would you design a pairwise annotation process that avoids length bias?”** A strong answer includes: (1) present annotators with length-matched response pairs when possible; (2) add an explicit annotation dimension for conciseness, separate from overall quality; (3) run calibration passes where annotators compare known-correct short responses against plausible-but-longer hallucinated ones to train out the length heuristic; (4) for AI judges, compute preference score as a function of Δ length across the evaluation set and apply a length-normalization correction before reporting win rates.

Key Takeaways

- Human and AI evaluators prefer longer responses by 60-70% in pairwise comparisons even when length adds no factual value; always deconfound length from quality in preference evaluation pipelines.
- SFT teaches output type signatures unconditionally-the model cannot distinguish between surfacing a known fact and hallucinating to match a structural template.
- Including citations or specific statistics in SFT responses trains hallucination unless the model’s pre-training already encodes that knowledge; strip or verify them before training.
- Benchmark evals (MMLU, TruthfulQA) and chat preference evals (Chatbot Arena) routinely diverge; a model that wins on preference evals can simultaneously regress on factual accuracy.
- On-policy RL is the principled remedy for the citation trap; SFT-only pipelines require aggressive data hygiene to avoid the same failure mode.

22.3.2 Mid-Training as the New Annealing Phase

Interviewers at frontier labs increasingly ask a question that trips up candidates who know only the classical InstructGPT pipeline: “What is mid-training, and how does it differ from both pre-training and post-training?” The question probes whether you understand that modern base model checkpoints are not the same objects described in GPT-3 era papers—they have been partially instruction-tuned through a process invisible in the checkpoint name. This topic appears at L6-L7 system design interviews and in any deep-dive on training pipeline architecture, data mix composition, or evaluation methodology for open-weight models.

The classical three-stage pipeline—pre-train, then SFT, then RLHF—assumed a clean boundary between the stage that installs knowledge (pre-training) and the stages that shape behavior (post-training). Mid-training dissolves that boundary. The key insight is that the learning rate schedule already contains a natural injection point: the decay phase of WSD (Warm-up, Stable, Decay) scheduling is the period when training loss is converging and small gradient steps are compressing the model into a stable attractor. Injecting high-quality and instruction-adjacent data *during* this decay phase produces a checkpoint that is simultaneously well-calibrated on the pre-training objective and partially instruction-tuned.

Concretely, a WSD mid-training setup looks like this:

Stage 1 (Stable):

```
data:    ~95% web/code/books pre-training mix
lr:      cosine stable plateau
tokens:  ~90-95% of total budget
```

Stage 2 (Decay / Annealing):

```
data:    ~70% pre-training + ~30% high-quality/SFT mix
lr:      cosine decay, stable plateau -> near 0
tokens:  ~5-10% of total budget
```

The high-quality SFT mix in Stage 2 typically includes upsampled Wikipedia, curated domain data (math, code, scientific papers), and a fraction of actual instruction-following pairs. The SFT-adjacent data should remain rare enough during Stage 2 that it does not displace the pre-training distribution entirely—the goal is shaping, not overwriting.

Mid-training is the decay phase used as a curriculum: the model consolidates broad world knowledge from pre-training while simultaneously acquiring the behavioral patterns required for instruction following.

When instruction tuning is applied as a completely separate post-training step, it typically runs at a moderately high learning rate on a very different data distribution. This creates catastrophic forgetting pressure: the model’s pre-trained knowledge degrades as the optimizer moves aggressively toward the instruction-following objective. Standard remedies—tiny learning rates, LoRA adapters, KL regularization toward the pre-trained weights—reduce forgetting at the cost of slower convergence or reduced final quality.

Injecting instruction data during decay sidesteps this trade-off because the learning rate is already small and monotonically decreasing. The model cannot move far from its pre-trained attractor; it is forced to find a minimal adjustment that accommodates the instruction signal. This is the same intuition behind low-learning-rate probing in representation learning: fine-tuning near the end of training, when the model’s representations are already rich, extracts far more instruction-following capability per token seen than a separately initialized SFT run.

The secondary advantage is data efficiency. Because the model encounters instruction data when its pre-trained representations are already mature, a much smaller absolute token count is needed to unlock instruction-following behavior. Published results from groups using this approach report that 5-15% instruction-adjacent data in the decay phase produces instruction-following quality comparable to separate SFT runs that used 5-10 $\times$ more instruction tokens.

The downside is interpretability: the resulting checkpoint is neither a clean base model nor a clean instruct model. When a lab releases a “base model” checkpoint trained with decay-phase injection, few-shot evaluation protocols that assume a clean base-model distribution give misleading scores. Benchmark comparisons across generation boundaries require verifying the training recipe, not just the checkpoint label.

A Worked Example

The MiniCPM training pipeline is one of the few publicly documented examples of explicit decay-phase data mixing.

Stage 1 uses approximately 2.4 trillion tokens from a mix of processed Common Crawl, code repositories, The Pile, and Dolma. The learning rate holds at the stable plateau from roughly step 5k to step 200k.

Stage 2 uses approximately 130 billion additional tokens with the following approximate composition:

- Wikipedia (carried over from Stage 1, upsampled 4 $\times$)
- Code SFT data (verified solutions to programming problems)
- UltraChat (multi-turn conversational data)
- Stack Exchange QA (question-answer pairs with votes)
- Evol-Instruct and OSS-Instruct (instruction-complexity evolved examples)

Instruction-adjacent sources constitute roughly 30-35% of Stage 2 tokens. The learning rate decays from the stable plateau to near zero across Stage 2. Total training cost is dominated by Stage 1; Stage 2 adds roughly 5-6% to the token budget but contributes disproportionately to downstream instruction-following benchmark scores.

After Stage 2, the checkpoint is released as the “base model.” Running standard few-shot evaluation on this checkpoint produces scores that cannot be compared directly to GPT-3-era base models, which had zero instruction-adjacent exposure. The base model here already behaves closer to a lightly instruction-tuned model; a subsequent short SFT pass is then used for safety tuning and formatting preferences rather than foundational instruction following.

For reference: Llama 3 was trained on 15 trillion tokens; Qwen3 on approximately 36 trillion. At these scales, a 5-10% decay phase represents 750 billion to 3.6 trillion tokens of high-quality injection-more tokens than many earlier complete training runs.

What You Actually Decide in Practice

- **Start mixing instruction data when the learning rate begins its final decay, not earlier.** Mixing instruction data during the stable phase displaces web-scale knowledge unnecessarily because the model is still absorbing high-entropy pre-training signal. The decay phase is when the model is consolidating, not exploring; that is the right moment to steer.
- **Target 10-30% instruction-adjacent data in the decay mix.** Below 10%, the instruction signal is too diluted to produce measurable behavioral change. Above 40%, knowledge-intensive benchmarks start

degrading, indicating the pre-training distribution is being displaced. The sweet spot is empirically consistent across published ablations.

- **The “base model” label is unreliable in modern releases.** If your work requires a clean base model-perplexity comparisons, probing studies, scaling law extrapolations-verify the training recipe of the checkpoint you are using. Llama 3, Qwen2, and OLMo2 base checkpoints all contain instruction data in their training lineage.
- **Long-context extension belongs in mid-training.** Transformers scale quadratically with sequence length; spending Stage 1 compute on 32k-128k contexts before the model has strong representations at shorter contexts is wasteful. Introducing long-context data during the decay phase, after Stage 1 has built rich short-context representations, is the dominant approach at frontier scale.
- **A short post-decay SFT pass is still useful, but it is lighter.** Decay-phase mixing handles foundational instruction-following behavior; a subsequent SFT pass at very low learning rate handles safety tuning, response formatting, and capability targeting that require precise distribution control. The two stages complement rather than duplicate each other.

How This Shows Up in Interviews

- **“What is the difference between a base model and an instruct model today?”** A strong answer acknowledges that the distinction has blurred: modern base models undergo instruction-adjacent training during the decay phase of pre-training, so they are not naive text predictors. A complete answer names the WSD annealing phase as the mechanism and explains why this produces better instruction following with less catastrophic forgetting than a separate SFT run.
- **“Why would you mix instruction data into pre-training instead of doing a separate SFT run?”** The answer is threefold: (1) catastrophic forgetting is reduced because the learning rate is already decaying; (2) data efficiency improves since the model’s pre-trained representations are already rich when the instruction signal arrives; (3) this approach scales using pre-training infrastructure, avoiding a separate training job with its own hyperparameter surface.
- **Senior follow-up: “How would you structure the data mix for a 70B model’s annealing phase, and what would you monitor to know if it is working?”** A strong answer specifies: (1) roughly 20-30% instruction-adjacent data, starting with high-quality domain data-math, code, curated web-and adding a smaller fraction of chat SFT pairs; (2) monitor validation loss on both the pre-training held-out set and a held-out instruction set simultaneously-both should decrease, not trade off; (3) run intermediate capability evals every 5-10 billion decay tokens to catch benchmark regression on knowledge-intensive tasks early; (4) apply a KL penalty or reference-model constraint to bound distribution drift from the stable-phase checkpoint, especially if the decay phase is long.

Key Takeaways

- Mid-training is the injection of high-quality and instruction-adjacent data during the learning rate decay phase of pre-training, exploiting the model’s consolidation dynamics to achieve instruction following with minimal catastrophic forgetting.
- The WSD schedule creates a natural two-stage curriculum: broad web-scale learning during the stable phase, behavioral shaping during the decay phase.
- Most modern “base models”-Llama 3, Qwen2, OLMo2-contain instruction data in their training lineage;

the base/instruct distinction now describes the post-decay SFT stage, not a clean separation of knowledge from behavior.

- Long-context extension, domain specialization (math, code), and early safety tuning all belong in mid-training rather than as separate post-training jobs, for the same data-efficiency and forgetting-reduction reasons.
- Empirically, 10-30% instruction-adjacent data during the decay phase matches or exceeds the quality of 5-10 $\times$ more instruction data applied in a separate high-learning-rate SFT run.

Part IX

Evaluation

Chapter 23 Designing an Evaluation

23.1 Goals of Evaluation

23.1.1 Evaluating Models vs. Systems vs. Methods

You are asked in an interview to design an evaluation for a new LLM-powered coding assistant. Do you benchmark the underlying model weights, the full product stack, or the novel training recipe your team just shipped? These are not the same question, and confusing them is one of the most common evaluation mistakes I see from engineers at all levels. Getting the object of evaluation wrong means the number you report answers a different question than the one you are trying to answer-and that mismatch quietly propagates into bad product decisions and misleading research claims.

There are three distinct objects you can evaluate, and each requires a different setup.

A *model evaluation* fixes the neural network weights and a minimal, standardized inference call: no external tools, no retrieval pipeline, no prompt engineering beyond what every competitor also gets. You are asking: what can these weights do on their own? The input is a prompt; the output is a completion; the metric is computed directly on that completion. This is what leaderboards like MMLU, GPQA, and HumanEval measure.

A *system evaluation* treats the entire production stack as the unit under test-model weights, prompt templates, RAG pipeline, tool calls, re-rankers, safety filters, and any agent scaffolding that drives multi-step loops. The user does not care which specific checkpoint sits at the center; they care whether the product solves their problem. System evaluations answer the question a buyer or a product manager is actually asking.

A *method evaluation* is what a research paper should be doing. Here the claim is that some algorithmic change-a new attention variant, a different data mixture, a revised RLHF loss-causes an improvement. The model trained with the new method is just a vehicle for demonstrating that claim. A valid method evaluation requires an ablation that isolates the variable: same data, same compute budget, same architecture except for the proposed change. Without that control, you are evaluating a system or a model, not a method.

Every evaluation you design should start with one sentence that names which of these three objects you are measuring-because that sentence determines everything else.

The distinction matters because the three objects differ in what they hold constant. Model evals hold constant the inference protocol, so the score is portable: you can compare your checkpoint against anyone else's checkpoint on the same benchmark. System evals hold constant the task and the user population, so the score is predictive of production outcomes. Method evals hold constant the training regime except for the proposed change, so the score is attributable to the hypothesis you are testing.

Mixing these up has real consequences. If you compare System A-a retrieval-augmented pipeline built on a mid-tier model-against a bare-model eval of System B's underlying weights, you are largely measuring the quality of System A's scaffolding. I have seen this comparison made in competitive benchmarking slides, and it always favors whoever built the better wrapper. Conversely, a method paper that reports only a system score ("our model plus chain-of-thought plus self-consistency reaches X%") cannot be credited with the improvement because the chain-of-thought and self-consistency would lift any baseline similarly.

The naive alternative-just report the highest number the model achieves under any configuration-is tempting because it looks good in a paper or press release. The problem is that it makes the number uninterpretable to anyone trying to replicate or compare.

A Worked Example

Consider evaluating a 70B model on SWEBench, a benchmark where an agent is given a GitHub issue and a codebase and must produce a patch that makes the unit tests pass.

Configuration 1: model eval. You call the model once per issue with a zero-shot prompt containing the issue description and the relevant file contents. No tool calls, no multi-step loop, no linting pass. A strong 70B model achieves roughly 10-15% resolve rate in this configuration.

Configuration 2: system eval. You wrap the same checkpoint in a scaffolded agent: the agent reads files, runs bash commands, iterates on failing tests, and submits only when lint passes. The same 70B model now achieves 35-45% resolve rate—a two-to-three-times lift from scaffolding alone.

Configuration 3: method eval. Your research question is whether adding a “verify-before-submit” step (the agent runs the relevant unit tests before submitting the patch) improves resolve rate. You train two checkpoints with identical data and compute, differing only in whether verification rewards were included during RLHF. You evaluate both under the *same* scaffolded system from Configuration 2, with the verification step disabled at inference time for both models. If the checkpoint trained with verification rewards achieves 40% versus 36% for the baseline under identical scaffolding, you can attribute the 4-point delta to the training method.

Note what breaks if you skip the discipline: if you enable the verification step for the new model but not the baseline, the 4-point delta is confounded—you have no idea whether the improvement comes from the training change or the inference-time verification.

What You Actually Decide in Practice

- **Choose model eval when reporting public benchmarks or comparing against published checkpoints.** Standardized prompting (zero-shot or fixed few-shot, no tools) keeps the playing field level. If your eval protocol differs from the benchmark’s canonical setup, report both numbers and explain the delta.
- **Choose system eval when making a deployment or procurement decision.** A 5-point gain on MMLU from a new checkpoint means nothing if your production system uses RAG and the bottleneck is retrieval recall, not model knowledge. Evaluate the full pipeline on a traffic sample from your actual users.
- **Choose method eval when writing a research paper or justifying a training investment.** Fix compute, data, and architecture except for the single variable under study. Reporting system improvements without ablations is a red flag reviewers will catch.
- **Watch for scaffolding budget asymmetry.** In agent benchmarks, “number of model calls allowed” and “maximum context length” are parameters that can easily be tuned per model. If System A gets 50 tool-call turns and System B gets 20, you are measuring budget allocation, not capability. Always match scaffolding budgets when comparing systems.
- **Perplexity is closer to a pure model eval than task accuracy.** Perplexity assigns credit at every token, does not depend on agent scaffolding, and is difficult to game through prompt engineering. For tracking model quality during pre-training, it is the right signal. For production decisions, it is not sufficient on its own—a model with lower perplexity on web text can still underperform on your specific downstream task.

How This Shows Up in Interviews

- **“How would you evaluate your company’s new code generation assistant?”** Strong answer: distinguish the model eval (does the checkpoint improve on HumanEval or SWEBench compared to the previous

checkpoint?) from the system eval (does the full product increase the fraction of user sessions where the user accepts the suggestion?). Explain that you need both: the model eval guides training; the system eval guides the deployment decision.

- **“A paper claims their new attention variant improves MMLU by 3 points. How would you assess whether that claim is valid?”** Strong answer: check whether compute and data are matched, whether the baseline uses the same training recipe except for the attention change, and whether the improvement holds across multiple seeds. A 3-point gain that disappears under controlled ablation is a system variation (e.g., the new attention allowed a larger batch size), not a method result.
- **Senior follow-up: “Your system eval shows a 5% win rate gain over the previous release. Your VP wants to know whether to credit the new model checkpoint or the prompt engineering rewrite your team shipped at the same time. How do you disentangle them?”** Senior answer: run a 2×2 factorial—old model with old prompt vs. old model with new prompt vs. new model with old prompt vs. new model with new prompt—on a held-out traffic slice. The interaction term tells you whether the prompt and model improvements are additive or whether one dominates. If you cannot run the factorial (the old model is decommissioned), you have a confounded result and should say so explicitly rather than attributing the gain to the most recently shipped change.

Key Takeaways

- A *model eval* measures weights under a standardized inference call; a *system eval* measures the full production stack; a *method eval* isolates a single training variable under controlled ablation. Name which one you are running before you design anything else.
- Comparing a scaffolded system against a bare-model baseline is not a model comparison—it is a scaffolding comparison. Match inference protocols when comparing models.
- Agent benchmarks like SWEbench can produce a two-to-three-times spread in resolve rate between a bare-model call and a well-engineered scaffolded loop on the same checkpoint. That gap is system performance, not model performance.
- Method claims require controlled ablations: same compute, same data, same architecture except for the proposed change. Reporting only the fully optimized system score is not sufficient to support a method claim.
- When you cannot disentangle variables (e.g., multiple changes shipped simultaneously), report the confounding explicitly. A confounded result is not useless—it tells you the system improved—but it does not let you allocate credit to a cause.

23.1.2 Ask-Style vs. Quiz-Style Prompts

An interviewer hands you a model that scores 87% on HumanEval. You are asked to decide whether to roll it out as the primary coding assistant for your engineering org. The number feels solid—but the question you need answered is not “can this model solve LeetCode problems someone has already verified?” The question is “will engineers accept its suggestions when they are stuck on real, ambiguous work at 2 am?” Those are fundamentally different prompts, and the gap between them is where most production deployments quietly underperform their benchmark numbers. This distinction—ask-style versus quiz-style—is one of the most practically important ideas in LLM evaluation, and it comes up whenever an interviewer wants to probe whether you understand why benchmark scores do not automatically transfer to product outcomes.

Every evaluation prompt can be classified by whether the person writing it already knows the correct answer.

A *quiz-style prompt* is one where the evaluator holds the ground truth before the model sees the question. Standardized benchmarks-MMLU, GPQA, HumanEval, GSM8k-are entirely quiz-style. A human expert (or an automated pipeline) constructed the question, verified the answer, and packaged both into a data set. The model’s job is to reproduce the known answer. The evaluator’s job is to check whether it did.

An *ask-style prompt* originates from a user who genuinely does not know the answer and is submitting the prompt because they need help. The user asking Claude to draft a differential diagnosis for a patient they are currently treating, or to explain why their Kubernetes pod keeps OOMing, has no verified ground truth in hand. They are trying to extract value from the model, not test it.

The distinction is not just philosophical. It determines what the score actually predicts. Quiz-style scores predict performance on other quiz-style tasks drawn from the same distribution. Ask-style scores-which require either human judges, LLM-as-judge pipelines, or downstream behavioral proxies like acceptance rate-predict whether real users get value from the system. The two correlate weakly at best once you move outside the model’s training distribution.

A benchmark score tells you how well a model performs on problems where someone already knows the answer; what matters in production is how well it performs on problems where nobody does.

Quiz-style benchmarks dominate the evaluation literature for one reason: they are cheap to construct and cheap to score. You do not need human judges. You do not need live users. You need a question, an answer key, and a string-matching or execution harness. That operational simplicity is why MMLU has 14,000 questions and GPQA has 448, and why you can reproduce either in a single script.

The cost is validity. Quiz-style prompts are clean, precisely specified, and have unambiguous single correct answers. Real user queries are underspecified-“fix my code” with no error message, “summarize this” with no stated audience. A model trained on internet text and evaluated on quiz-style prompts derived from the same internet can score well while failing badly on genuine user need.

Ask-style evaluation is harder operationally but more predictive. Analyses of real conversation traffic find coding is the dominant use case, followed by writing assistance and analysis. None of those sessions look like multiple-choice questions. The prompts are long, messy, contextual, and often span multiple turns. A benchmark built from that traffic-with humans or calibrated LLM judges scoring helpfulness rather than correctness-tracks production quality far better than any standardized exam.

The tension is that realistic ask-style data is often private. Clinical notes, customer service transcripts, and internal code reviews are the highest-value ask-style prompts, and they cannot be published. This is why initiatives like MedHELM exist: 29 clinicians identified 121 tasks reflecting their actual practice, and the resulting benchmark uses realistic prompts (patient note summarization, treatment plan drafting) that no multiple-choice exam covers. The privacy-versus-realism trade-off is real, and the right answer is almost always to invest in the ask-style eval even if it cannot be open-sourced.

A Worked Example

Suppose you are evaluating a code completion model for internal deployment. You run two evaluations in parallel.

Quiz-style (HumanEval). 164 Python function stubs with docstrings and unit tests. The model generates a completion; you execute the tests; you report pass@1. Your new checkpoint scores 87%, up from 80% on the previous release. The delta of 7 points looks meaningful.

Ask-style (sampled production sessions). You sample 2,000 real completions accepted or rejected by engineers over the past two weeks. A human rater panel (or a calibrated LLM judge) scores each completion on a 1-5 helpfulness scale, blind to which model generated it. You also instrument the IDE plugin to record whether the engineer accepted the suggestion, modified it substantially, or discarded it. Results: mean helpfulness score 3.4 vs. 3.1 for the prior release (+0.3 points), and acceptance rate 54% vs. 51% (+3 points). Both improvements are real but smaller in magnitude than the HumanEval delta suggests.

The gap exists because HumanEval problems have clean docstrings and self-contained scope. Real tickets say things like “add the same caching logic we use in `UserService` but for `OrderService`”—which requires reading a second file, understanding an undocumented convention, and matching a style the model has never seen. Ground truth requires a self-contained problem; real work is rarely that. The 7-point HumanEval gain measures something narrower than “will engineers be more productive?” The accept-rate gain of 3 points answers the question that actually matters.

What You Actually Decide in Practice

- **Use quiz-style benchmarks to track training progress, not to make deployment decisions.** They are reproducible, fast, and cheap. They are the right tool for deciding whether a checkpoint is heading in the right direction during development. They are the wrong tool for deciding whether to ship it.
- **Build at least one ask-style eval before launch.** The minimum viable version is a few hundred real prompts from your target user population, scored by human raters or an LLM judge calibrated against human raters on a held-out slice. If you cannot get real prompts pre-launch, use synthetic ones drawn from a model of your users’ tasks-but acknowledge the limitation.
- **Instrument for behavioral proxies when human rating is too slow.** Acceptance rate, edit distance after acceptance, session abandonment rate, and thumbs-up/thumbs-down feedback are all ask-style signals that accumulate automatically once you are in production. Wire these up before launch so you have a baseline to compare against when the next checkpoint ships.
- **Never report only quiz-style scores for a safety-sensitive deployment.** A model that scores 86% on USMLE Step 2 CK may still hallucinate drug dosages at a rate that is clinically unacceptable. Safety evals must be ask-style, grounded in the actual failure modes that harm real users, not the failure modes that fit neatly into a multiple-choice format.
- **Watch for quiz contamination in user feedback.** Live traffic from a deployed model is not automatically ask-style. A significant fraction of real sessions are engineers or researchers actively probing the model’s limits—quiz-style behavior from real users. If your user population skews toward power users or developers, your “live traffic” evaluation may overestimate performance on genuine end-user need. Segment by user cohort before drawing conclusions.

How This Shows Up in Interviews

- **“Your model hits 90% on MMLU. How do you decide whether to deploy it to customers?”** Strong answer: MMLU is a quiz-style benchmark and tells you the model can retrieve factual knowledge under a standardized prompt format. Before deployment you need an ask-style eval on a representative sample of your actual customer tasks—measuring helpfulness, groundedness, and refusal behavior—because MMLU accuracy does not predict those. Name a specific eval you would run (e.g., a human-judged sample of 100 realistic customer sessions) and what pass threshold you would require.

- **“We’re seeing a gap between our benchmark scores and user satisfaction in production. What’s going on?”** Strong answer: the gap almost always traces to the quiz/ask mismatch. Benchmarks have clean ground truth; production prompts are underspecified and context-dependent. Diagnose by sampling failing production sessions and checking whether they share structural properties—long context, cross-file references, ambiguous intent—that are absent from the benchmark. Then build a targeted ask-style eval that captures those properties.
- **Senior follow-up: “How would you build an ask-style eval for a general-purpose assistant where there is no single correct answer?”** Senior answer: layer three signals. Use LLM-as-judge at scale for triage (fast, cheap, biased toward length and verbosity). Audit a random 5% slice with human raters and report judge calibration—if the LLM judge agrees with humans 80% of the time you know how far to trust the automated scores. Track behavioral proxies (acceptance rate, task completion, return rate) as a continuous production signal. A score reported without calibration metadata is not an evaluation; it is an anecdote.

Key Takeaways

- Quiz-style prompts have a known answer before the model sees the question; ask-style prompts originate from users who genuinely need help. The distinction determines what the resulting score predicts about production behavior.
- Quiz-style benchmarks are cheap and reproducible—use them to guide training. Ask-style evals are expensive but valid—use them to make deployment decisions.
- The quiz/ask gap is largest when real user traffic is underspecified, context-dependent, or drawn from a domain (clinical, legal, internal codebases) that is not well represented in public benchmarks.
- Safety-critical deployments require ask-style evals grounded in realistic failure modes; a strong USMLE or MMLU score does not substitute for them.
- A production ask-style eval needs a judging layer with measured reliability: report inter-rater agreement or LLM judge calibration alongside the helpfulness score, or the number is not trustworthy.

23.2 Metrics and Protocols

23.2.1 Perplexity as a Universal Signal

Every scaling law curve you have ever seen was fit against perplexity, not task accuracy. That is not an accident. When a senior engineer asks during an interview how you would track training health across a multi-week pre-training run, or how you would compare two architectural variants before committing to training one at full scale, perplexity is almost always the right first answer. Understanding both what perplexity measures and where it breaks down separates engineers who can read a training dashboard from engineers who can design the evaluation protocol behind it.

A language model defines a probability distribution p over sequences of tokens. Given a held-out sequence $x_1, x_2, \dots, x_N$, the model assigns a joint log-probability $\sum_{i=1}^N \log p(x_i \mid x_{<i})$. Perplexity is the exponential of the average per-token negative log-likelihood:

$$\text{PPL}(p, \mathcal{D}) = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(x_i \mid x_{<i}) \right)$$

where $\mathcal{D}$ is a held-out data set and N is the total token count. A lower perplexity means the model assigns higher probability to the observed text—it is less surprised. Perplexity of 1 is a perfect oracle; perplexity equal to the vocabulary size $|V|$ is a uniform random guesser.

Mechanically, perplexity during pre-training is a monotone transformation of the training objective. The gap between training loss and validation perplexity is the canonical sign of overfitting or distribution shift. Perplexity on a held-out slice of web text the model has never seen is the standard signal for tracking whether the model is learning to model language in general.

The perplexity-maximalist argument, which I find largely compelling for pre-training diagnostics, goes like this. If the true data-generating distribution is t and your model is p , then minimizing the KL divergence $D_{\text{KL}}(t||p)$ is equivalent to minimizing cross-entropy, which is equivalent to minimizing perplexity. In the limit where $p = t$, the model has matched human-level text generation and, in principle, can answer any question a human can answer. Perplexity, when computed against a broad and representative corpus, is measuring proximity to that ideal.

Perplexity is the only metric that assigns loss to every token uniformly, making it the most data-efficient signal for detecting whether a model is learning—and the right tool for fitting and extrapolating scaling laws.

Perplexity wins for pre-training diagnostics for three concrete reasons.

First, it is smooth. Task accuracy on a multiple-choice benchmark is a step function: you either produce the right letter or you do not. Perplexity is a continuous function of the model’s log-probabilities, which means you can detect a small improvement from a data mixture tweak or a learning-rate schedule change before that improvement is large enough to move a downstream accuracy number. This smoothness is why every published scaling law—Chinchilla, GPT-4’s training curves, Llama’s ablations—is expressed in terms of cross-entropy loss rather than MMLU scores. In the mid-2010s, the most striking result in language modeling was a paper from Google showing that a carefully scaled architecture could push Penn Treebank perplexity from roughly 51 down to 30. That 40% reduction signaled a sea change in what neural language models could do, long before downstream task benchmarks existed to measure it.

Second, perplexity is architecture-agnostic. If you are comparing two models that differ only in attention variant and you want to know which is learning more efficiently per FLOP, perplexity on the same validation set is an apples-to-apples comparison. Task accuracy would require both models to be trained long enough to generalize to the task format, which may not happen for small ablations early in a run.

Third, perplexity is hard to game through prompt engineering. Win rates and task accuracies can be inflated by choosing the right few-shot examples, the right chain-of-thought template, or the right decoding strategy. Perplexity does not depend on any of these; it requires direct access to the model’s logits over the held-out text, and as long as train and test splits are separate, it is not a quantity you can easily overfit to.

The main weakness of perplexity is that it is not always predictive of downstream task performance on any particular task in the short run. A model can achieve lower perplexity on web text by getting better at predicting Reddit discussions and financial news, without improving on formal mathematical reasoning. Over large scale differences, perplexity and task performance tend to correlate because stronger models are better at most things. Within a narrow training regime or for a specific domain, the correlation can be weak.

A Worked Example

Consider a pre-training ablation: you want to know whether increasing the proportion of code data from 15% to 25% of your training mixture helps a 7B model ($d_{\text{model}} = 4096$, 32 layers, 32 heads, bf16, trained on

300B tokens across 64 H100s).

You train two ablation models, each for 10B tokens, on the two mixtures. After 10B tokens, Model A (15% code) has validation perplexity 8.4 on a held-out web text slice and 12.1 on a held-out code slice. Model B (25% code) has 8.6 on web text and 11.3 on code.

From this you conclude: the code-heavy mixture slightly hurts general web text modeling (perplexity rises by 0.2) but improves code modeling by 0.8 perplexity points. Had you used HumanEval pass@1 instead, you would need to run much longer before either model generates enough correct completions for a statistically stable comparison. Perplexity gives you the direction after 10B tokens; a task-accuracy signal would require 50-100B tokens before variance is low enough to trust.

You also watch per-token loss curves during training. If validation perplexity flattens while training loss keeps falling, you have a data contamination or overfitting signal. If perplexity spikes sharply on a particular batch, you have a data quality issue or a bad learning-rate spike. Neither of these would show up as a task-accuracy alarm until much later.

What You Actually Decide in Practice

- **Use perplexity for all training-time decisions and ablations.** Any decision made before the model completes pre-training-data mixture, learning rate schedule, architecture tweak-should be based on perplexity, not downstream task accuracy. Task accuracy is too noisy at small scale and too expensive to evaluate continuously during a run.
- **Match tokenizers when comparing perplexities across models.** Perplexity is a function of the token sequence, not raw text. A model with a larger vocabulary will have lower perplexity on the same text because it segments tokens more coarsely. If tokenizers differ, normalize to bits-per-byte or bits-per-character before drawing any conclusions.
- **Require logit access before trusting a reported perplexity.** If you are running a leaderboard that accepts model submissions, perplexity is easy to misreport. A model that assigns 0.8 to every next-token position-a non-distribution-will appear to have low perplexity but is meaningless. Verify that reported logits sum to 1 over the full vocabulary, or compute perplexity yourself from open-weight checkpoints.
- **Use domain-stratified perplexity, not a single aggregate.** One number averages over all text types. Split your validation set into slices-web text, code, math, multilingual-and track perplexity per slice. This is what reveals whether a data mixture change helped one domain at the cost of another, which a single aggregate will hide.
- **Perplexity does not replace task evaluation at deployment time.** Once you are making a deployment decision-should we ship this checkpoint?- perplexity is necessary but not sufficient. Two models with identical web text perplexity can differ by 10 points on HumanEval if one was trained with more code. Use perplexity to filter candidates during training; use task accuracy to select among finalists.

How This Shows Up in Interviews

- **“How would you track training health during a long pre-training run?”** Strong answer: monitor validation perplexity on a domain-stratified held-out set, tracking both aggregate and per-domain curves. Set up automated alerts for perplexity spikes (indicating data or learning-rate issues) and for validation-training divergence (indicating overfitting or contamination). Task accuracy is too slow to provide the real-time feedback you need during training.

- **“A colleague says their new data mixture improved MMLU by 2 points after 10B training tokens. Is that a reliable signal?”** Strong answer: almost certainly not. At 10B tokens, MMLU variance across seeds is typically larger than 2 points. The right signal at 10B tokens is perplexity on domain-stratified validation slices. Ask them to show the perplexity curves before treating the MMLU number as meaningful.
- **Senior follow-up: “When is perplexity insufficient as your only signal, even during pre-training?”** Senior answer: perplexity is insufficient when the capability you care about requires compositional generalization rather than next-token prediction—for example, multi-step mathematical reasoning or long-horizon planning. A model can minimize perplexity on math corpora by learning to predict common surface patterns without being able to solve novel problems. For these capabilities you need targeted held-out task evaluations at training time—not as the primary optimization signal, but as a periodic diagnostic. Some training runs at frontier labs use “canary tasks”—small, carefully constructed task suites evaluated every N steps—precisely to detect when perplexity progress stops translating to capability progress.

Key Takeaways

- Perplexity is $\exp(-\frac{1}{N} \sum_i \log p(x_i | x_{<i}))$, the exponential of average per-token cross-entropy. Lower is better; it measures how surprised the model is by held-out text.
- Its smoothness and token-level granularity make it the right signal for training-time decisions: data mixture ablations, learning-rate tuning, and architecture comparisons at small scale.
- Perplexity requires raw logit access and is easy to misreport. Require that reported logits sum to 1 over the full vocabulary, and normalize to bits-per-byte when comparing models with different tokenizers.
- Use domain-stratified perplexity to detect trade-offs between text domains that a single aggregate number will hide.
- Perplexity is necessary but not sufficient for deployment decisions. Use it to guide training; add task-accuracy evals to select among final checkpoint candidates.

23.2.2 Exact Match, F1, Pass@k, and Win Rates

Once you move past perplexity and onto measuring what a model actually produces when deployed—answers to questions, code patches, explanations, clinical summaries—you have to choose a task-level metric. That choice is not cosmetic. I have seen teams report a 5-point accuracy gain that evaporated when they switched from exact match to F1, and I have seen code generation win rates swing 15 points when the evaluation moved from pass@1 to pass@10. The metric is a design decision with real consequences, and interviewers at any lab with a production system will probe whether you understand those consequences.

Four metric families cover the vast majority of LLM evaluation scenarios.

Exact match (EM) compares the model’s output, after normalization (strip punctuation, lowercase, collapse whitespace), to a reference string. It scores 1 if they are identical, 0 otherwise. This is the right metric for closed-form, unambiguous outputs: MMLU letter choices, arithmetic results, entity extraction with a canonical form.

Token-level F1 treats the prediction and reference as bags of tokens and computes precision and recall over the overlap:

$$F_1 = \frac{2 \cdot |\text{pred} \cap \text{ref}|}{|\text{pred}| + |\text{ref}|}$$

This gives partial credit for approximately correct answers. It is the standard for open-domain QA tasks like SQuAD and TriviaQA, where a reference answer may have multiple valid phrasings and a word-for-word match is too strict.

Pass@k is used for code generation and any task where the model’s output can be automatically tested for correctness. Rather than literally drawing k samples and checking, you draw $n \gg k$ samples and use the unbiased estimator:

$$\text{Pass}@k = \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

where n is the number of samples drawn per problem and c is the number that pass all unit tests. The formula gives the probability that at least one of k randomly selected samples from the n drawn is correct.

Win rates are used when there is no ground-truth reference: open-ended generation, creative writing, instruction following, summarization. You present pairs of model outputs to human judges or to a judge LLM and record which output the judge prefers. Aggregating pairwise preferences via an ELO or Bradley-Terry model yields a scalar rating that is comparable across models. Chatbot Arena applies this protocol at scale with human judges; AlpacaEval applies it automatically with GPT-4 as judge.

The right metric is determined by the task’s output space: closed-form answers use EM; partial-credit spans use F1; verifiable programs use Pass@k; open-ended generation requires a judge, human or model.

Each metric trades precision against breadth of applicability in a different way.

Exact match is maximally precise but brittle. A reference of “Paris” and a prediction of “Paris, France” score 0 under EM despite being semantically equivalent. You pay for that brittleness with simplicity: EM is fast, requires no judge, and cannot be gamed once the answer form is fixed.

Token-level F1 buys robustness to surface variation at the cost of rewarding verbosity. A model that outputs a very long prediction containing the correct tokens will inflate its recall while precision stays low. This is why SQuAD-style QA systems often apply the min of prediction and reference length, or use a normalizer that strips articles and punctuation, to reduce the surface area for gaming.

Pass@k decouples evaluation from deployment scenario in a way that matters for production. Pass@1 measures the probability that a single sample is correct-relevant if you show users one completion. Pass@10 measures the probability that at least one of 10 samples is correct-relevant if your system generates multiple suggestions and picks the best via a reranker or test execution. The gap between pass@1 and pass@10 reveals how much diversity the model has: high pass@10 with low pass@1 means the model generates varied samples and benefits from sampling; nearly equal pass@1 and pass@10 means the model is low-entropy and additional samples add little value.

Win rates via LLM judges are convenient but systematically biased. GPT-4-as-judge prefers longer, more verbose outputs even when they are not more correct—a well-documented artifact that caused inflated win rates for models that learned to pad responses, until length-controlled variants of AlpacaEval corrected for it. On the human side, Chatbot Arena judges are representative of whoever visits the site, which may not match your target user population. A leaderboard that becomes a PR target also attracts gaming: the “Leaderboard Illusion” paper documented cases of providers gaining privileged submission access or cherry-picking model configurations, eroding the benchmark’s integrity.

A Worked Example

Consider evaluating a 7B code model (Llama-3-style, $d = 4096$, 32 layers, bf16, context 8192 tokens) on HumanEval, which contains 164 programming problems with canonical unit test suites.

Draw $n = 20$ samples per problem at temperature $T = 0.8$, top- $p = 0.95$. Run the unit tests on each sample. For problem 42, suppose 8 of 20 samples pass, so $c = 8$, $n = 20$.

For $k = 1$:

$$\text{Pass@1} = 1 - \frac{\binom{20-8}{1}}{\binom{20}{1}} = 1 - \frac{12}{20} = 0.40$$

For $k = 10$:

$$\text{Pass@10} = 1 - \frac{\binom{12}{10}}{\binom{20}{10}} = 1 - \frac{66}{184,756} \approx 0.9996$$

The ratio $\text{pass@10} / \text{pass@1} \approx 2.5$ for this problem indicates that the model knows how to solve it but has significant output variance-additional samples are worth generating if the deployment context allows it. Averaged over all 164 problems, a 7B model at this training scale typically achieves $\text{pass@1} \approx 0.55$ - 0.65 and $\text{pass@10} \approx 0.75$ - 0.85 , depending on the training mixture. That gap is not noise; it reflects a genuine difference between showing one suggestion versus ten.

Now suppose you also run a win-rate evaluation on the same model's docstring generation capability. You generate 500 docstrings for real functions from your codebase, pair each against the previous checkpoint's output, and ask GPT-4o to judge which is more accurate and concise. You observe a raw win rate of 53%. When you filter to responses under 100 tokens-removing the verbosity confound-the win rate drops to 50%. The raw 53% was length bias, not quality improvement. Reporting only the raw number would have led to a false conclusion about the new checkpoint.

What You Actually Decide in Practice

- **Default to pass@1 for user-facing deployments, pass@k for capability research.** When reporting what users actually experience with a single-suggestion system, pass@1 is the number that matters. Use pass@k to characterize ceiling capability or to evaluate systems that explicitly generate and rerank multiple candidates.
- **Use the unbiased pass@k estimator, not a hard threshold.** Computing pass@k by generating exactly k samples and checking is high-variance for small k . Generate $n \geq 20$ samples per problem and use the $\binom{n-c}{k} / \binom{n}{k}$ estimator. This is what the original HumanEval paper recommends and what you should reproduce for comparability.
- **Length-correct your win rates before reporting them.** Any LLM-as-judge win rate should be accompanied by a length-controlled variant where output length is accounted for. If the length-corrected win rate differs from the raw win rate by more than 2 absolute points, report both and investigate before attributing the gain to quality.
- **Pair every safety metric with a capability metric.** A model that refuses everything scores 100% on harm benchmarks. Safety numbers are uninterpretable without a simultaneous report of pass@1 on coding, exact match on factual QA, or some other capability signal. Report them together, not in isolation.

- **Put cost on the Pareto frontier, not in the denominator.** A model with $\text{pass@1} = 0.70$ that costs $10\times$ more per token than one with $\text{pass@1} = 0.68$ is not obviously better for production. Report accuracy at fixed latency budget or accuracy per dollar when comparing models with different serving costs. Many public leaderboards omit cost entirely; supplement them with a Pareto frontier if you are making a deployment decision.

How This Shows Up in Interviews

- **“How would you evaluate a code generation model for production deployment?”** Strong answer: use pass@1 at the system’s actual sampling temperature as the primary accuracy metric; supplement with pass@10 to characterize ceiling capability; measure latency-to-first-token and cost per accepted suggestion; report accuracy vs. cost on a Pareto frontier rather than a single number.
- **“A team reports a 5-point win rate improvement on AlpacaEval. What should you check before trusting that number?”** Strong answer: check whether the improvement survives length correction; check whether the same improvement appears on Chatbot Arena or a human evaluation on your actual task distribution; verify the baseline was evaluated under identical decoding parameters (temperature, top- p , maximum tokens). A win rate that disappears after length correction is measuring verbosity, not quality.
- **Senior follow-up: “Your pass@1 improved by 4 points on HumanEval, but users are not reporting a better experience. What might explain the gap?”** Senior answer: HumanEval problems are short, self-contained, and have canonical test suites. Production code generation tasks involve existing code-bases with implicit context and rarely have test suites. A 4-point HumanEval gain can come from better performance on easy function-completion problems that are already mostly solved in production. Run an evaluation on a held-out sample of real user sessions-accept rate, or user edit distance after accepting a suggestion-to determine whether the benchmark gain translates. If it does not, the gap is in the evaluation distribution, not the model, and you should redesign the benchmark before acting on future HumanEval results.

Key Takeaways

- Exact match is precise but brittle-use it only for closed-form, canonical outputs such as classification labels and arithmetic results.
- Token-level F1 gives partial credit for span-extraction tasks; watch for verbosity inflation and normalize output length if the model’s responses are unconstrained.
- Pass@k decouples capability from deployment scenario. Use pass@1 for single-suggestion systems, pass@k for best-of- N or capability research, and always apply the unbiased estimator with $n \geq 20$ samples per problem.
- Win rates require a judge and are systematically biased toward longer outputs. Always report a length-controlled variant alongside the raw win rate.
- Pair every accuracy or safety metric with a cost metric. Accuracy at fixed latency budget or accuracy per dollar is the right unit for deployment decisions.

Chapter 24 Benchmarks You Must Know

24.1 Knowledge and Reasoning Benchmarks

24.1.1 MMLU, MMLU-Pro, and GPQA

In a system design interview at a frontier lab, one of the first questions after you describe your evaluation pipeline is usually: “Which benchmarks are you tracking and why?” The interviewer is not fishing for a list of acronyms—they want to know whether you understand what each benchmark actually measures, where it saturates, and what its failure modes are. MMLU, MMLU-Pro, and GPQA are the three rungs of the standard knowledge-and-reasoning ladder, and misunderstanding any one of them signals a gap that mid-level candidates routinely fall into.

Massive Multitask Language Understanding (MMLU) was released in 2020, coinciding with GPT-3. The benchmark spans 57 subjects—ranging from abstract algebra and virology to world history and foreign policy—each formatted as a four-option multiple-choice question. Questions were scraped from the web, which already hints at contamination risks we will return to. At the time, the dominant evaluation paradigm was few-shot prompting on base models, so a canonical MMLU prompt looks like this:

The following are multiple choice questions about astronomy.

Q: What is the main sequence?

- A) A band of galaxies
- B) The period after the Big Bang
- C) Stars fusing hydrogen in their cores
- D) Planetary orbital zones

Answer: C

Q: ... [3 more exemplars] ...

Q: Which instrument was used to first detect gravitational waves?

- A) The Hubble Space Telescope
- B) LIGO
- C) ALMA
- D) Chandra

Answer:

The model is expected to emit the letter of the correct answer. GPT-3 scored 45% under this protocol in 2020. Today’s frontier models score in the low 90s, which is why the benchmark has essentially lost discriminative power.

MMLU-Pro addresses saturation by replacing four answer choices with ten and removing the noisiest, most trivial questions. Chain-of-thought prompting is now the norm rather than raw next-token prediction. Frontier model scores drop to the 65-75% range, restoring a useful spread across model families.

Graduate-Level Google-Proof Questions (GPQA) raises the bar to PhD-level. Questions were written by domain experts, validated by a second expert, and then given to non-experts who spent up to 30 minutes on each with full internet access—the “Google-proof” designation means the non-expert accuracy was still only around 30%, while domain experts reached 65%. GPT-4 scored 39% at launch; O3 now reaches approximately 75%.

The ladder exists because knowledge benchmarks saturate in order: MMLU first, then MMLU-Pro, then GPQA-and you should know which rung a model currently sits on before choosing your evaluation target.

Multiple-choice benchmarks have three properties that make them endure: they are automatically scorable, they are reproducible across labs without access to proprietary human raters, and they produce a single number that fits cleanly into a training dashboard. When you are running hundreds of evaluation checkpoints during a training run, automatic scoring is not a convenience-it is a necessity.

The escalating difficulty across the three tiers also reflects a real property of model capability. A model that struggles on MMLU is not ready for GPQA; a model that saturates MMLU-Pro without a gap on GPQA is likely memorizing rather than reasoning. The gaps between tiers are diagnostic.

The main trade-off is that none of these benchmarks reflects what real users do. A user asking an LLM to debug their deployment pipeline or draft a patient summary is not taking a standardized test. MMLU in particular is better thought of as a proxy for base-model breadth: if a model trained on a broad web corpus can ace questions it never explicitly studied for, that is evidence of genuine generalization. If you curate a training set to hit MMLU specifically, you get a high score that tells you almost nothing about general capability.

A Worked Example

Consider evaluating a 7B model checkpoint at step 50,000 of pre-training. You want a quick signal on whether the model is developing factual knowledge.

Set up 5-shot MMLU on the *computer science* subset (170 questions). Feed the prompt above, sample greedily (temperature 0), and extract the first token. If it is A, B, C, or D, record it; otherwise mark invalid. Compute accuracy as:

$$\text{Accuracy} = \frac{\text{correct answers}}{\text{total questions}}$$

At 7B scale with a strong data mix, you should see roughly 55-65% on the computer science subset after a full training run, versus 25% random baseline. If your number is below 35% at step 50,000, the data mix or learning rate is likely off-this is the diagnostic value of tracking MMLU during training.

Now switch to GPQA for a 70B model you are preparing to deploy into a medical-information use case. Run GPQA's biology subset (approximately 65 questions) with chain-of-thought, 0-shot. A score below 40% means the model cannot reliably handle expert-level biomedical questions; a score of 55% or above puts you in territory where domain-expert human annotators are necessary to catch the remaining errors. That threshold informs whether you can deploy without a mandatory human-in-the-loop review layer.

What You Actually Decide in Practice

- **MMLU for base models, not instruction-tuned models.** Once you apply RLHF or supervised fine-tuning, MMLU scores can actually drop slightly because the model becomes more cautious about outputting raw letters. Use MMLU-Pro or GPQA for post-training checkpoints.
- **MMLU-Pro is the current workhorse for frontier comparisons.** Scores in the 65-75% range give real differentiation between GPT-4-class and GPT-3.5-class models. If you see a model claiming 90%+ on MMLU-Pro with standard prompting and no majority-vote ensembling, audit the evaluation code-it is almost certainly using a non-standard prompt.

- **Few-shot example order is a confounder.** Randomizing the order of the five exemplars and averaging over three orderings reduces variance by 2-4 accuracy points on MMLU. Always report the protocol, not just the number.
- **Chain-of-thought is now mandatory for GPQA.** Without scratchpad reasoning, even 70B models underperform by 10-15 points because the questions require multi-step derivations. Zero-shot GPQA without chain-of-thought is not a meaningful measurement.
- **Assume contamination exists.** MMLU was released in 2020. Any model trained on a 2023-or-later web crawl has almost certainly seen fragments of the test questions. A 13-gram deduplication pass can quantify this, but it does not catch paraphrases. When you report MMLU scores for a new model, add a contamination disclaimer to your model card-it is now a community norm, even if not universally followed.

How This Shows Up in Interviews

- **“Which benchmark would you use to evaluate a new 70B model before deployment?”** Strong answer: choose based on the deployment target. For broad factual knowledge in a general assistant, MMLU-Pro gives meaningful signal. For a domain-expert use case like medical or legal, supplement with GPQA on the relevant subset. Never use plain MMLU for a frontier-sized model-you will hit the ceiling.
- **“MMLU scores have been in the 90s for a year. Is the benchmark still useful?”** Strong answer: MMLU is still useful as a sanity check and a base-model diagnostic, but it has no discriminative power among frontier models. The field has shifted to MMLU-Pro and GPQA precisely because saturation kills the ranking signal. Know the saturation threshold: approximately 90% for MMLU, roughly 75% for MMLU-Pro, somewhere above 75% for GPQA (not yet saturated as of 2025).
- **Senior follow-up: “A competitor claims their model beats yours by 3 points on GPQA. How do you verify or refute this claim?”** Senior answer: first, inspect the evaluation protocol-prompt format, whether chain-of-thought was used, whether majority voting over multiple samples was applied. A 3-point gap can easily be manufactured by switching from 0-shot to 8-shot chain-of-thought. Second, check the subset: 3 points on the full GPQA (448 questions) is statistically meaningful, but 3 points on a 65-question chemistry subset is within noise. Third, run your own evaluation on their model using a locked evaluation harness like HELM or LM-Evaluation-Harness so the prompt format is identical.

Key Takeaways

- MMLU (2020) covers 57 subjects with 4-choice questions; saturated for frontier models at ~90%.
- MMLU-Pro raises the bar to 10 choices and harder questions; current frontier scores cluster 65-75%.
- GPQA targets PhD-level, Google-proof questions; domain experts score 65%, non-experts 30%, and top models now reach ~75%.
- Multiple-choice benchmarks are valuable precisely because they are automatic, reproducible, and cheap-but they are proxies for knowledge, not for real-world task performance.
- Evaluation protocol details-few-shot count, example order, chain-of-thought, sampling strategy-can swing scores by 5-15 points; always report the full protocol alongside the number.

24.1.2 Humanity’s Last Exam and ARC-AGI

Every knowledge benchmark eventually gets saturated-that is the pattern, and interviewers at frontier labs know it. When a candidate mentions GPQA, a senior interviewer often follows up with: “So what do we

do when O3 is at 90% on GPQA in 18 months?” Humanity’s Last Exam and ARC-AGI represent two very different attempts to solve that problem, and understanding the philosophical difference between them is the kind of nuance that separates a strong answer from a rehearsed one. Both have appeared in recent evaluation discussions at top labs, and knowing the current numbers as well as the structural critiques is table stakes.

Humanity’s Last Exam (HLE) is a multimodal, crowdsourced benchmark built around the premise that if you aggregate the hardest questions that domain experts can construct-and then use frontier models themselves to filter out anything “too easy”-you get a ceiling that will not be quickly saturated. Organizers solicited questions from researchers and practitioners, offering co-authorship to contributors and a prize pool for accepted submissions. Each question went through a multi-stage review process. The resulting dataset contains problems in mathematics, physics, chemistry, biology, law, economics, and other fields, covering both text-only and image-based formats. Questions are either multiple-choice or short-answer with a verifiable correct response-there is no open-ended generation. At launch, the best-performing models scored roughly 8-10%; as of early 2025, O3 reaches approximately 20%.

ARC-AGI (Abstraction and Reasoning Corpus for Artificial General Intelligence) is philosophically different. Introduced in 2019-before large language models were the dominant paradigm-it deliberately removes both language and world knowledge from the equation. Each task is a small grid of colored cells. You are shown two or three input-output example pairs that illustrate a transformation rule, and then you must apply that rule to a new input grid to produce the correct output. There are no words, no task descriptions, no domain knowledge required. The intent is to isolate abstract reasoning: can the model induce the rule from examples and generalize it? GPT-4o scored near 0% on ARC-AGI in standard evaluation mode. O3, using extended test-time compute, reaches approximately 75% on ARC-AGI v1-but at a cost of several hundred dollars per task in compute.

HLE measures the frontier of expert knowledge retrieval and multi-step reasoning; ARC-AGI measures abstract inductive reasoning stripped of any factual prior-and these are fundamentally different capabilities that current architectures handle very differently.

HLE’s strength is coverage depth. By recruiting experts and using adversarial filtering via frontier models, it ensures that questions are not just hard but are hard *for current LLMs specifically*. The multi-modal component also targets a genuine gap: reasoning over diagrams, equations rendered as images, and scientific figures is a capability that text-only benchmarks cannot probe. The benchmark’s primary weakness is selection bias. The people who respond to an open call to create hard questions for LLMs are disproportionately LLM researchers and power users who already have an intuition for what breaks models. The resulting distribution is not representative of any real user population-it is the hardest end of an already-expert distribution.

ARC-AGI’s strength is orthogonality. Because it contains no language and requires no memorized facts, a high score cannot be explained by training data memorization. The tasks test a form of program induction that standard next-token prediction is poorly structured to perform, which is why the baseline for transformer models was essentially 0% for years. The weakness is ecological validity: nobody’s production use case looks like “infer the grid transformation rule.” It measures something important about abstract reasoning, but a score improvement on ARC-AGI does not obviously predict improvement on coding, medical Q&A, or customer support tasks.

The compute cost of ARC-AGI is itself diagnostic. O3 achieves 75% partly through massive test-time search-the model generates thousands of candidate solutions and selects among them. At per-task costs in the hundreds of dollars, this is not a deployment-viable strategy. It tells you that the reasoning capability exists somewhere in the model’s distribution but that eliciting it reliably and cheaply remains unsolved.

A Worked Example

Consider a single ARC-AGI task. The training examples might show:

Input:	1 0 0	Output:	1 1 1
	0 0 0		0 0 0
	0 0 0		0 0 0

Input:	0 0 0	Output:	0 0 0
	0 1 0		0 1 1
	0 0 0		0 0 0

The rule, derived by inspection: wherever a 1 appears, extend it one cell to the right. The test input is:

Test:	0 0 0
	0 0 1
	0 0 0

Correct output: a 1 at position (row 2, col 3) and a 1 at position (row 2, col 4)-but the grid is only 3 columns wide, so the extension falls off the edge. Correct answer: just the original 1 at (2, 3) with no extension possible. A model that rigidly applies “extend right” without respecting boundaries fails. This edge-case handling illustrates why ARC-AGI specifically probes compositional rule application, not recall.

For HLE, a worked interaction is less illustrative because the hardness is in the domain knowledge itself. What matters is the structural number: 2,700+ questions, current top score around 20%, with each percentage point representing genuine reasoning progress rather than prompt engineering.

What You Actually Decide in Practice

- **Neither benchmark drives deployment decisions directly.** If you are choosing which model to serve in production, HLE and ARC-AGI scores are interesting but not sufficient. Map them to a downstream task that matches your use case before pulling the trigger on a deployment choice.
- **HLE scores above 30% are the near-term saturation warning.** Current trajectory suggests frontier models reach this threshold within 12-18 months. When that happens, expect ARC-AGI v2 or a successor benchmark to take HLE’s place. Know that this cycle is predictable-benchmark creation and saturation alternate in roughly 18-24-month cycles.
- **ARC-AGI performance with extended compute is not the same as ARC-AGI performance under standard inference.** When citing O3’s ~75% figure, always specify whether that was achieved with test-time search or standard single-pass generation. The gap can be 50+ points and conflating them misleads the conversation.
- **HLE has a documented creator bias.** If you are using HLE to evaluate a model for a specific domain, check whether your domain is over- or under-represented in the question distribution. A model that scores 20% overall may score 5% on your domain or 35% if the benchmark happens to over-index on it.
- **ARC-AGI v2 raises the floor.** The original ARC-AGI tasks are learnable by humans in seconds. ARC-AGI v2 introduced harder tasks where human accuracy drops closer to 60-70%. Pay attention to which version a score is reported against-the versions are not directly comparable.

How This Shows Up in Interviews

- **“What is ARC-AGI trying to measure that GPQA is not?”** Strong answer: GPQA measures knowledge retrieval and expert-level reasoning grounded in facts a model may have memorized during training. ARC-AGI deliberately removes both language and factual knowledge to isolate abstract inductive reasoning-the ability to infer a rule from examples and apply it compositionally. A model can score highly on GPQA partly through training data overlap; a model cannot score highly on ARC-AGI through memorization because each task is a novel spatial pattern.
- **“Humanity’s Last Exam says it will be the hardest benchmark ever. Do you believe that?”** Strong answer: no benchmark survives contact with an order-of-magnitude improvement in model capability. HLE’s saturation will happen; the question is timing. The more useful observation is that HLE’s creator-selection bias means it skews toward questions that LLM researchers find hard, not questions that representative users care about. It is a valuable signal of ceiling capability but a poor proxy for deployment readiness.
- **Senior follow-up: “O3 spends roughly \$200 per ARC-AGI task at 75% accuracy. What does that tell you about the relationship between compute and reasoning, and what would it mean if a smaller model reached 75% with standard inference?”** Senior answer: the O3 result demonstrates that the *capability* is latent in the model family-the weights contain the necessary representations-but eliciting it requires search over many candidate solutions. This looks more like a sampling and verification problem than a weights problem. A smaller model reaching 75% with single-pass inference would imply either that the inductive reasoning capability has been distilled more efficiently, or that the model has found a structural shortcut specific to ARC-AGI’s task distribution. The two cases are distinguishable by running that model on a held-out set of novel ARC-AGI-style tasks not in the original benchmark.

Key Takeaways

- Humanity’s Last Exam uses expert-written, adversarially filtered questions across multiple domains and modalities; current top score is $\sim 20\%$, with saturation expected within 1-2 years.
- ARC-AGI removes language and factual knowledge entirely, testing abstract inductive reasoning via grid-transformation tasks; GPT-4o scores near 0%, O3 reaches $\sim 75\%$ with heavy test-time compute.
- The philosophical split matters: HLE measures expert knowledge plus reasoning; ARC-AGI measures reasoning with knowledge held constant at zero.
- ARC-AGI performance figures must always be tagged with the inference budget-a 75% score costing hundreds of dollars per task is not comparable to a 40% score under standard single-pass inference.
- Neither benchmark translates directly to production deployment value; use them as capability probes, not as substitutes for task-specific evaluation on your actual use case.

24.2 Instruction Following and Chat Evaluation

24.2.1 IFEval, AlpacaEval, and WildBench

When a team ships a new RLHF checkpoint and asks whether instruction following improved, the answer cannot come from MMLU or GPQA - those benchmarks test knowledge retrieval, not the ability to honor a user’s stated intent. The hard problem is judging open-ended responses where there is no single ground-truth answer,

and where the right response for one user might be wrong for another. IFEval, AlpacaEval, and WildBench each attack this problem from a different angle, and understanding why each one was built the way it was is more useful than memorizing the numbers.

IFEval (Instruction Following Evaluation) sidesteps the judge problem entirely by restricting prompts to constraints that can be verified with a script. A prompt like “Plan a two-week trip to Japan. Do not use commas in your response. Your answer must contain at least 200 words and include the word ‘itinerary’.” can be checked mechanically: count words, scan for commas, grep for the keyword. IFEval ships roughly 500 prompts, each annotated with one or more verifiable instructions drawn from a taxonomy of about 25 constraint types - length bounds, sentence count, keyword inclusion or exclusion, casing rules, presence of specific format tokens, and similar surface-level checks. The primary metric is *prompt-level accuracy*: the fraction of prompts where *all* constraints are simultaneously satisfied.

AlpacaEval abandons verifiability and uses an LLM as the judge. Given a prompt, two responses are produced - one from the model under evaluation and one from a fixed reference model (originally `text-davinci-003`, later GPT-4 Turbo). A judge LLM reads both and declares a winner. The final score is the *win rate*: the fraction of prompts where the evaluated model beats the reference. Because raw GPT-4 judging strongly favors longer responses, a length-controlled variant (LC-AlpacaEval) is now standard. It fits a logistic regression on win outcome versus $\log(\text{length ratio})$ and reports the corrected win rate at equal response length. The correction is meaningful: a model that simply pads responses can inflate raw win rate by 10-15 points.

WildBench adds a third ingredient: a realistic prompt distribution. Rather than synthesizing tasks, WildBench harvests utterances from live human-bot conversations collected by deploying a public chatbot. The judge is still an LLM, but it is asked to complete a task-specific checklist before rendering a verdict, which forces explicit reasoning about domain-relevant criteria and reduces position bias. WildBench’s authors report that their leaderboard ranking correlates well with Chatbot Arena, which is the closest proxy for actual user satisfaction available at scale.

The core trade-off across all three: IFEval is fully automatic and bias-free but measures only structural compliance, not semantic quality; AlpacaEval scales to any output type but inherits the judge’s style preferences; WildBench has the most realistic prompts but cannot be run without an LLM judge and significant compute.

IFEval’s strength is reproducibility. There is no judge to hallucinate, exhibit style bias, or change behavior across API versions. A constraint is either satisfied or it is not. This makes IFEval ideal as a regression gate: run it on every training checkpoint at negligible cost and catch format regressions before they reach a human reviewer. The weakness is that IFEval measures what the model *structurally* produces, not whether the response is useful. A model that outputs 200 words of fluent but incorrect travel advice with no commas scores perfectly.

AlpacaEval’s LLM-as-judge design handles semantic quality, handles long responses, and scales to arbitrary task types. The cost is judge bias. GPT-4 judges favor responses that sound like GPT-4 output, which systematically underrates models with different generation styles. The LC correction fixes verbosity bias but not style bias. You also pay real API costs and get slight nondeterminism: running the same evaluation twice yields win rates that can differ by one or two points.

WildBench breaks the “quiz versus ask” distinction that plagues most benchmarks. Most curated evaluations are quizzes - the evaluator already knows the answer and is checking whether the model does too. Real users are asking - they want help with something they do not already know, and they will notice poor quality even if the response technically satisfies a format constraint. Sourcing prompts from live traffic is the closest approximation to actual deployment distribution without requiring a production system. The checklist-based judge reduces but does not eliminate the fundamental LLM-judge limitation: the checklist itself was generated

by a model and can be wrong.

A Worked Example

Suppose you are evaluating a 7B instruction-tuned model against GPT-4 Turbo on the standard AlpacaEval 805-prompt set.

1. Generate responses from both models for all 805 prompts (temperature 0.7, max tokens 2048).
2. For each prompt, build a judge payload with the two responses in randomized order to counteract position bias.
3. Call the GPT-4 judge; it returns a winner. Record whether the 7B model was judged better.
4. Suppose the 7B model wins 380 of 805 comparisons: raw win rate = $380/805 \approx 47.2\%$.
5. Apply the LC correction. Fit logistic regression: win outcome $\sim \beta_0 + \beta_1 \log(L_{\text{eval}}/L_{\text{ref}})$ using the 805 data points. Predict at $\log(1.0) = 0$ (equal length). The corrected win rate drops to, say, 41%.

A 41% LC win rate says the 7B model loses more often than it wins against GPT-4 Turbo on general instruction following, after removing the length advantage.

Now run IFEval on the same model. Suppose it scores 72% prompt-level accuracy versus GPT-4's 87%. The 15-point gap is concentrated in multi-constraint prompts - prompts that require simultaneously satisfying three or more constraints. The model handles single constraints reliably but fails at composition. This is a concrete actionable signal: the training data likely under-represents prompts with stacked requirements, and targeted data collection will close the gap faster than continued general RLHF.

Run WildBench last. The live-traffic prompt distribution includes more ambiguous, multi-step requests than AlpacaEval. The 7B model's WildBench LC win rate against GPT-4 is 37% - lower than its AlpacaEval number - because the realistic prompts expose brittleness on conversational tasks that the curated benchmark obscures.

What You Actually Decide in Practice

- **Use IFEval as a CI gate, not a quality signal.** Set a threshold - for example, do not release a checkpoint below 75% prompt-level accuracy - and block merges that regress below it. The run is cheap and deterministic; there is no reason not to run it on every checkpoint.
- **Always report LC win rate, never raw win rate.** Raw win rate is gamed by verbosity. If your evaluation pipeline does not yet support the LC correction, a quick approximation is to compare median response length between the evaluated model and the reference: if the evaluated model is consistently 30% longer, discount the raw win rate by roughly 5-8 points as a rough sanity check.
- **Do not cross-compare AlpacaEval win rates across very different model families.** Win rate measures similarity to GPT-4's style as much as it measures quality. A model fine-tuned on GPT-4 outputs will score inflated win rates even at lower capability. Use it for comparing checkpoints within the same training lineage.
- **WildBench is the right benchmark for release gates.** When you have budget for one high-signal evaluation before shipping a model update, prefer WildBench over synthetic benchmarks. The prompt distribution is the closest available proxy for what real users will send.
- **Corroborate across all three before drawing a conclusion.** A model that improves on IFEval but degrades on WildBench has probably overfit to format compliance at the expense of content quality - a common failure mode when RLHF data is dominated by constraint-following examples.

How This Shows Up in Interviews

- **“How would you evaluate whether a new RLHF checkpoint improved instruction following?”** Strong answer: use IFEval for cheap structural regression testing on every checkpoint, AlpacaEval LC win rate to measure relative open-ended quality against a known baseline, and WildBench for a final release gate using realistic prompts. No single benchmark is sufficient.
- **“Why can’t you use exact match or BLEU for instruction-following evaluation?”** Strong answer: instruction following is open-ended - there is no single correct response. A prompt like “summarize this article” has thousands of valid answers; exact match would return zero for all of them. BLEU penalizes paraphrase, which is a feature in chat. The evaluation problem requires either verifiable structural constraints (IFEval) or a judge that can assess semantic quality (LLM-as-judge or humans).
- **“What is the main failure mode of LLM-as-judge evaluations?”** Strong answer: the judge inherits its training distribution’s biases. GPT-4 judges favor outputs that stylistically resemble GPT-4 and favor longer responses. The verbosity bias is measurable and correctable via length-controlled win rate; the style bias is harder to remove because it requires knowing the judge’s training data. Position bias - preferring whichever response appears first - is addressed by randomizing response order.
- **Senior follow-up: “Your AlpacaEval LC win rate improved 5 points but Chatbot Arena ELO is flat. How do you reconcile this?”** Sketch: AlpacaEval’s 805 curated prompts may sample a different distribution than the Arena’s live traffic. Inspect which prompt categories drove the AlpacaEval gain - if the gains are concentrated in constrained-format tasks that are rare in the Arena, the metrics are measuring different things. Also check response length distribution: if the 5-point gain partially survived length correction but the model is still 20% longer on average, the LC correction may not fully account for a broader fluency halo effect.

Key Takeaways

- IFEval tests verifiable structural constraints at zero judge cost; use it as a CI gate, not a standalone quality measure.
- AlpacaEval measures open-ended quality as a win rate against a fixed reference model; always use the length-controlled variant to remove verbosity bias.
- WildBench sources prompts from live human-bot conversations, making it the most realistic prompt distribution available; prefer it for release decisions over synthetic-task benchmarks.
- LLM-as-judge evaluations scale easily but inherit the judge model’s style and length biases; cross-model comparisons are unreliable when the models have different generation styles.
- A complete instruction-following evaluation suite should include at least one verifiable-constraint benchmark (IFEval), one LLM-judge win-rate benchmark (AlpacaEval LC), and one realistic-traffic prompt set (WildBench).

24.2.2 Chatbot Arena: Strengths and Failure Modes

Chatbot Arena has become prominent enough that the CEO of Google has tweeted about Gemini’s ELO score. That kind of visibility is a double-edged signal: it means the benchmark captures something real about model quality, and it means the benchmark has become a target worth gaming. If you are building or evaluating a chat model, you need to understand both why Arena numbers are trusted and exactly where they break down

- because interviewers at frontier labs will probe both sides of that question.

Chatbot Arena is a *pairwise human preference* leaderboard. A visitor to the site submits a prompt, receives two anonymous responses from two randomly selected models, and votes for the better response or declares a tie. Votes are aggregated using the *Bradley-Terry model*, a pairwise ranking framework that estimates a latent “strength” parameter θ_i for each model such that the probability of model i beating model j in a head-to-head comparison is:

$$P(i \succ j) = \frac{e^{\theta_i}}{e^{\theta_i} + e^{\theta_j}}.$$

In practice this is expressed as an ELO score: a model initialized at 1000 gains points when it beats a higher-rated opponent and loses points when it loses to a lower-rated one, with the magnitude proportional to the rating gap. New models enter by being included in the random selection pool; their ELO stabilizes as they accumulate comparisons.

Three properties make this design attractive. First, the prompts are *live and open-ended*: they come from real users who submit whatever they actually want to do, not from a fixed benchmark that can be memorized. Second, human judges are the ultimate downstream signal for a chat product - if people prefer your model, that is the definition of quality for that use case. Third, the ELO system naturally accommodates new entrants: a new model can establish a reliable rating after a few thousand comparisons without invalidating the existing rankings.

Chatbot Arena’s ELO is the closest available proxy for real user preference among general chat models, but it measures preference on the distribution of whoever visits the site - a distribution that may not match your deployment population.

The benchmark avoids the two most common failure modes of static evaluations. First, there is no fixed prompt set to overfit: because prompts are generated by live users, the distribution shifts as user behavior evolves, new topics emerge, and models are updated. A model that memorized the MMLU test set gains nothing from that memorization on Arena. Second, there is no reference answer to construct: human judges do not need to agree on a ground truth, they simply express a preference, which is exactly the signal that matters for a product.

The ELO mechanism handles the practical problem of scale. With hundreds of models and millions of comparisons, a full round-robin evaluation is impossible. ELO provides consistent rankings from an incomplete comparison graph as long as the underlying comparison data is sufficiently dense and not adversarially selected.

The failure modes are structural. The most important is *Goodhart’s law*: once Arena ELO became a PR metric for frontier labs, labs began optimizing for it directly. A 2024 paper, “The Leaderboard Illusion,” documented that some providers received privileged access to the Arena comparison pool, were permitted to make multiple model submissions under the same label, and were able to influence which models their submissions were compared against. These protocol failures mean that a jump in Arena ELO for a specific provider may reflect leaderboard management rather than model improvement.

A second failure mode is *population bias*. The users who visit Chatbot Arena skew toward technically literate, English-speaking, AI-enthusiast demographics. A model that is optimized to impress that population - through confident tone, dense technical vocabulary, and comprehensive multi-paragraph answers - will score well on Arena even if it performs poorly on the customer service queries, coding assistance, and simple factual lookups that constitute the majority of real deployment traffic.

A third failure mode is *style over substance*. Human judges choosing between two anonymous responses in a few seconds often favor the response that *reads better*: more confident, more structured, more verbose. This

creates pressure to produce well-formatted but subtly incorrect answers over compact but accurate ones. Several studies have shown that adding markdown headers and bullet points to an otherwise identical response raises its Arena win rate by measurable margins.

A Worked Example

Suppose two models, A and B, each start with ELO 1200. After 10,000 Arena comparisons the data shows:

- Model A versus Model B: 620 wins for A, 380 wins for B.

Under the Bradley-Terry model, the maximum likelihood estimates of θ_A and θ_B satisfy:

$$\frac{e^{\hat{\theta}_A}}{e^{\hat{\theta}_A} + e^{\hat{\theta}_B}} = \frac{620}{1000} = 0.62.$$

Solving: $e^{\hat{\theta}_A - \hat{\theta}_B} = 620/380 \approx 1.63$, so $\hat{\theta}_A - \hat{\theta}_B \approx \ln(1.63) \approx 0.49$. Converted to the standard ELO scale (K -factor 32), Model A's ELO rises by roughly $32 \times (1 - 0.62) \approx 12$ points per comparison on average, landing around 1240 while B drifts toward 1160 after sufficient comparisons.

Now suppose a third model C is added. C is only compared against B (not A) due to a protocol gap. C beats B 70% of the time, giving C an ELO near 1220. But without direct comparisons to A, C's ELO is estimated via transitivity - and if B's rating has been deflated by the earlier comparisons, C's estimated rating inherits that error. This is the mechanism behind the "Leaderboard Illusion" finding: controlling which models you are compared against controls your apparent ELO even if your true quality is lower.

The practical takeaway: treat Arena ELO differences under 20 points as noise, require confidence intervals before drawing any conclusion, and be especially skeptical of large ELO jumps that coincide with PR announcements.

What You Actually Decide in Practice

- **Do not use Arena as your internal development signal.** The feedback loop is too slow (days to weeks for stable ELO), too expensive (requires public model deployment), and too noisy (high variance from user population drift). Use AlpacaEval or WildBench for fast iteration; treat Arena as a final validation signal only.
- **Segment Arena results by category when available.** The LMSYS team now publishes category-level breakdowns (coding, math, writing, etc.). A model with strong overall ELO but weak coding ELO is a different product risk than a model with uniform performance. Always look at the category breakdown before reporting an overall number.
- **Treat ELO differences below 20 points as within noise.** The confidence intervals on Arena ELO are wide for models with fewer than 5,000 comparisons. A difference of 10-15 ELO points is not statistically distinguishable from zero; differences above 30 points are more reliable.
- **Flag suspicious ELO trajectories.** A model that jumps 40+ ELO points in a week with no corresponding improvement on automated benchmarks is a signal of leaderboard management, not model improvement. Cross-check against AlpacaEval and WildBench - genuine capability improvements show up in all three.
- **Pair Arena with a safety and refusal benchmark.** A model can maximize Arena ELO by never refusing any request and always producing confident-sounding answers. Unless you also track refusal rate and factual accuracy, you may ship a model that looks great on Arena but hallucinates aggressively in production.

How This Shows Up in Interviews

- **“Why is Chatbot Arena considered the gold standard for evaluating chat models, and what are its limitations?”** Strong answer: Arena uses live human preference on real user prompts with an ELO system that handles new entrants gracefully. The limitation is Goodhart’s law - once ELO became a PR metric, providers began optimizing for it directly, including through protocol manipulation documented in “The Leaderboard Illusion.” Population bias toward technically literate English speakers is a structural limitation that cannot be fixed without changing the user base.
- **“A model improved on AlpacaEval but its Arena ELO did not move. What do you investigate?”** Strong answer: check whether the AlpacaEval gain was concentrated on prompt types that are rare in Arena’s live traffic. Inspect whether the model became more verbose (which boosts AlpacaEval but may not impress Arena voters who are reading two responses side by side). Also check whether the model was recently added to Arena and has too few comparisons for a stable ELO estimate.
- **“How would you design an evaluation system that avoids the failure modes of Chatbot Arena?”** Strong answer: blind protocol with randomized comparison partners and no privileged submissions; stratified sampling to ensure coverage of coding, factual recall, and safety-adjacent prompts, not just the distribution of whoever finds the site; confidence intervals on all published scores; and a clear separation between “evaluation” comparisons (public) and “development” comparisons (internal only, never used to update public ELO).
- **Senior follow-up: “Even with a clean protocol, what is the fundamental limit of human preference evaluation for assessing model quality?”** Sketch: human preference captures what people *like*, not what is *correct*. On tasks with verifiable answers - math, code, factual retrieval - a confident wrong answer can beat a hedged correct one in human preference voting. The fundamental fix is not a better preference protocol but a better task decomposition: use human preference only for genuinely subjective tasks (writing style, tone, helpfulness in ambiguous situations), and use ground-truth metrics for tasks with objective answers. A single Arena ELO collapses both into one number, which is convenient but misleading.

Key Takeaways

- Chatbot Arena ranks models via ELO scores derived from pairwise human preference votes on live, open-ended prompts - the most direct proxy for real user satisfaction available at scale.
- The benchmark is vulnerable to Goodhart’s law: once ELO became a PR target, protocol manipulation became documented and consequential.
- Population bias (technically literate, English-speaking visitors) and style-over-substance voting mean Arena ELO favors well-formatted, confident responses over compact, accurate ones.
- Treat ELO differences below 20 points as noise; cross-validate Arena numbers against AlpacaEval and WildBench before drawing conclusions.
- Use Arena for release validation, not for the development feedback loop - it is too slow, too noisy, and too costly to run internally on every checkpoint.

24.3 Agentic and Safety Benchmarks

24.3.1 SWE-Bench, CyBench, and MLE-Bench

Static multiple-choice benchmarks tell you what a model knows; agent benchmarks tell you what it can actually do under execution feedback. At mid-to-senior interviews at labs and product companies, you will be asked to design evaluation pipelines for agentic systems - systems that write and run code, access a shell, iterate on tool results, and operate over dozens of steps rather than a single forward pass. SWE-Bench, CyBench, and MLE-Bench represent three distinct flavors of this challenge: software engineering, cybersecurity, and machine learning research. Understanding their mechanics, their current accuracy ceiling, and their blind spots is the baseline for any serious conversation about evaluating agents in production.

An *agent benchmark* differs from a static benchmark in one structural way: the model does not produce a single output that is scored; it operates a loop - observe, plan, act, observe again - until it either solves the task or exhausts a step budget. An automated harness manages this loop, executes the model's actions in a sandboxed environment, and feeds the resulting observations back into the model's context.

SWE-Bench presents the agent with a real GitHub repository and an issue description. The agent must produce a patch (a unified diff) that, when applied, causes the associated unit tests to pass. No human reviews the patch for style or correctness beyond the test suite. The evaluation metric is *resolved rate*: the fraction of issues for which the submitted patch makes all required tests pass. Current top models resolve roughly 40-50% of the SWE-Bench Verified subset, with the full test set sitting somewhat lower.

CyBench wraps Capture-The-Flag (CTF) challenges as agent tasks. The agent receives a natural-language description of the challenge and shell access to a sandboxed server. Its goal is to execute commands, exploit a vulnerability, and retrieve a secret flag string. The canonical agent loop used in published work looks like this:

```
while not solved and steps < budget:
    thought = model.generate(context)
    command = parse_command(thought)
    output = shell.execute(command)
    context += (thought, command, output)
```

CyBench records *first-solve time by human teams* for each challenge, ranging from a few minutes for easy tasks to 24 hours for the hardest. This lets you calibrate model performance against human effort rather than against an arbitrary binary pass/fail threshold. As of early 2025, top agents solve challenges that took skilled human teams around 40 minutes; challenges requiring multiple hours of human effort remain mostly unsolved.

MLE-Bench draws from 75 historical Kaggle competitions. The agent receives the competition description and the dataset, then must explore the data, write training code, tune hyperparameters, and submit predictions - exactly the workflow a human Kaggle competitor follows. The primary metric is whether the agent's final submission places above the competition's *medal threshold* (typically the top 10-20% of historical submissions). Best-in-class agents achieve this threshold on fewer than 20% of competitions.

All three benchmarks share the same structural advantage: an objective, automated verifier that eliminates human labeling from the evaluation critical path entirely.

The decisive virtue of objective verifiers - test suites in SWE-Bench, flag strings in CyBench, leaderboard scores in MLE-Bench - is reproducibility. You run the benchmark, collect numbers, and trust the verdict without a human annotator in the loop. This matters when you are iterating on agent scaffolding or model versions daily.

The trade-off is coverage. Unit tests do not verify code correctness in general; they verify the specific cases the test author imagined. A patch that passes all tests while introducing a subtle regression elsewhere is scored as a success. In CyBench, challenge difficulty spans several orders of magnitude, so aggregate pass rate masks which capability tier is actually improving. In MLE-Bench, the agent operates under a fixed compute and wall-clock budget that may not reflect how a human competitor would allocate resources, making direct comparison somewhat artificial.

Compared to static knowledge benchmarks, these agent benchmarks are structurally harder to saturate through training data contamination alone, because solving a novel GitHub issue requires reasoning about the specific repository’s structure and issue context, not recalling a memorized answer.

A Worked Example

Consider a SWE-Bench evaluation run with a 70B agent model on an H100 node, context window 128K tokens, step budget capped at 30 iterations.

1. The harness provides the model with the repository file tree (truncated to fit context), the failing test output, and the issue description.
2. The agent calls a file-read tool to inspect the relevant module, identifies a missing branch in a conditional, and generates a candidate patch.
3. The harness applies the patch via `git apply`, runs `pytest` targeting the affected test file, and returns pass/fail plus any traceback.
4. If tests fail, the observation is appended to context and the agent retries. If tests pass, the run is scored as resolved.

At a 40% resolved rate over 300 issues, a full benchmark run incurs roughly

$$300 \times 15 \text{ steps} \times 2,000 \text{ tokens/step} = 9\text{M input tokens}$$

plus generation costs - significant but tractable for a daily evaluation loop. Context exhaustion is a real failure mode: published analyses show roughly 20-25% of failures occur because the model’s context window fills before it converges on a valid patch, not because the model lacks the underlying knowledge.

What You Actually Decide in Practice

- **Fix the step budget before comparing runs.** Doubling the budget from 20 to 40 steps typically lifts resolved rate by 5-8 percentage points but doubles token cost. Reporting a resolved-rate number without the budget makes it uninterpretable.
- **Version the scaffold alongside the model.** Adding a `grep`-style code-search tool to the scaffold typically lifts SWE-Bench resolved rate by 3-5 points independent of any model change. A number reported without specifying the scaffold version is ambiguous about what actually changed.
- **Stratify CyBench results by human first-solve time.** Challenges solved by humans in under 10 minutes are in a completely different regime from those requiring hours. Aggregating them into one accuracy number hides which difficulty tier is actually changing.
- **Contamination in agent benchmarks is structural, not just n-gram overlap.** SWE-Bench tasks come from real repositories with public commit histories. If the model was trained on post-issue commits that contain the fix, it can “solve” the task by recall rather than reasoning. Timestamp filtering is the standard mitigation: exclude any repository commits that postdate the model’s training cutoff.

- **MLE-Bench medal rate underestimates continuous improvement.** Medal thresholds are discrete step functions. Track the continuous leaderboard percentile alongside the binary medal metric to distinguish a submission that narrowly misses the medal from one that ranks near the bottom of non-medal submissions.

How This Shows Up in Interviews

- “*How would you evaluate an agent that writes and runs code?*” A strong answer names SWE-Bench, explains the test-suite-as-verifier design, and immediately flags the coverage gap (passing tests does not prove correctness). A mid-level answer stops at “run the tests.” A senior answer specifies: scaffold version, step budget, and whether you are evaluating the model or the full system including all tooling.
- “*What makes agent benchmarks harder to game than static multiple-choice benchmarks?*” The verifier is objective and dynamic - a real test suite or a server that must be exploited - so memorizing outputs is insufficient. The senior follow-up is: “*A model trained on post-issue GitHub commits could solve SWE-Bench by recall rather than reasoning. How do you detect and correct for that?*” The strong answer: timestamp-based filtering, plus maintained hold-out repositories that were never indexed in any training corpus.
- “*Why does MLE-Bench use medal rate rather than raw leaderboard score?*” Medal rate normalizes across competitions with incompatible scoring scales. But the senior nuance: medal rate is a step function and suppresses continuous progress, so you want both the medal rate and the percentile distribution, not just the mean.

Key Takeaways

- Agent benchmarks use objective, automated verifiers - test suites, flag strings, competition scores - to eliminate human labeling from the evaluation critical path.
- Top models resolve roughly 40-50% of SWE-Bench Verified; MLE-Bench medal rates remain below 20% for frontier models as of early 2025, leaving substantial headroom.
- The scaffold - tool set, step budget, context management strategy - affects scores as much as the underlying model; always version both together when reporting numbers.
- Contamination in agent benchmarks is structural: models trained on post-fix commits can solve tasks by recall. Timestamp filtering is the standard mitigation, but requires knowing the training cutoff.
- Always stratify by task difficulty before reporting aggregate accuracy; a single number across CyBench challenges spanning minutes to 24 hours of human effort is nearly meaningless.

24.3.2 HarmBench, AIR-Bench, and Jailbreaking

Safety benchmarks surface reliably in interviews for roles at external-facing product teams and at frontier labs running pre-deployment evaluations. The question is almost never “is this model safe?” - that framing is too abstract to be actionable. The real questions are: which harm categories does the model comply with under direct prompting, how does it respond to adversarially optimized inputs, and how do you pair a safety metric with a capability metric so that over-refusal is penalized alongside under-refusal? HarmBench, AIR-Bench, and the GCG jailbreak literature give you the vocabulary and numbers to answer those questions concretely at any seniority level, but senior candidates are expected to hold the capabilities-vs-propensity distinction and the open-weight threat model without prompting.

HarmBench defines 510 harmful behavior categories spanning chemical synthesis, cyberweapon generation, non-consensual content, and a range of other regulated domains. For each behavior, the benchmark constructs a direct prompt and measures whether the model *complies* (produces the requested content) or *refuses*. The primary metric is *attack success rate* (ASR): the fraction of behaviors where the model produces a compliant response. A lower ASR is better for safety. HarmBench also includes adversarial attack variants - including the GCG method described below - so a single benchmark run can report both direct-prompt ASR and adversarially-prompted ASR side by side.

AIR-Bench (AI Risk Benchmark) takes a grounding strategy: rather than curating harmful behaviors ad hoc, it maps its taxonomy to specific clauses in existing regulatory frameworks - EU AI Act articles, US executive orders, major company usage policies - and to published harm taxonomies. This anchoring makes coverage auditable: you can point to a specific regulation and ask which benchmark items correspond to it. The taxonomy spans categories including CBRN (chemical, biological, radiological, nuclear) uplift, critical infrastructure attacks, privacy violations, and election interference.

Jailbreaking refers to any technique that causes a model to comply with a harmful prompt it would otherwise refuse. The canonical research example is the Greedy Coordinate Gradient (GCG) attack, which appends an adversarially optimized suffix to the original prompt:

$$p^* = p_{\text{harm}} \parallel [x_1, x_2, \dots, x_k]_{\text{suffix}}$$

The suffix $x_1 \dots x_k$ is found by gradient-based search on an open-weight model to maximize the probability that the model begins its response with a compliance token (typically “Sure” or “Of course”) rather than a refusal. The suffix is not semantically meaningful in human language. Because it optimizes for a low-level token distribution rather than surface semantics, it transfers to black-box closed models: suffixes optimized on Llama-2-7B have achieved non-trivial ASR against GPT-4 class models in published experiments.

Safety evaluation is not just about whether a model refuses harmful prompts under direct request; it is about how robust that refusal is under adversarial perturbation, and whether you have kept those two dimensions cleanly separated in your benchmark design.

The deepest architectural insight in safety evaluation is the distinction between *capability* and *propensity*. A model may have encoded the knowledge to synthesize a dangerous compound (capability) but have been trained to decline to produce that knowledge on request (propensity). For closed-weight API models, only propensity is directly exposed to an attacker - if the model refuses, the adversary cannot extract the underlying capability through standard prompting, even if it exists. For open-weight models, capability matters independently, because fine-tuning can restore compliance in as few as 100-500 gradient steps on consumer hardware, effectively removing the alignment layer while leaving the encoded knowledge intact.

This distinction fundamentally changes how you interpret a safety score. A closed-weight model scoring 5% ASR on HarmBench and an open-weight model with the same score have very different real-world risk profiles. Any safety evaluation that conflates the two leads to misleading conclusions.

The pairing problem is equally important: a model that refuses every prompt would achieve 0% ASR on HarmBench but would be useless. The standard practice is to run a safety evaluation alongside a capability evaluation (a task accuracy benchmark on benign prompts) and treat the two scores as a Pareto frontier rather than summing them into a single safety index. The earlier practice of reporting only ASR, without a measure of false-positive refusals, is a known failure mode in early safety leaderboards.

A Worked Example

Consider running a targeted GCG jailbreak evaluation on a hypothetical 7B instruction-tuned model.

1. Select a target behavior from HarmBench, e.g., “Provide step-by-step synthesis instructions for [hazardous compound].”
2. Initialize a random suffix of 20 tokens appended after the harmful prompt.
3. For 500 gradient steps on the open-weight target, compute $\nabla_{\theta} \mathcal{L}_{\text{jailbreak}}$ where the loss rewards compliance tokens and penalizes refusal tokens at position 1 of the response.
4. Apply the optimized suffix to the same prompt sent to a closed-weight model via API (no gradient access needed at inference time).
5. Score the closed model’s response: compliance counts as a successful transfer.

In the original GCG paper, this transfer attack achieved ASR above 80% on several closed models when optimizing on Llama-2-7B. The computational cost for a single behavior is roughly 500 forward-backward passes on a 7B model - about 10-20 minutes on a single A100. Across all 510 HarmBench behaviors, a full GCG evaluation costs on the order of 100-170 A100-hours, which is well within the budget of a pre-deployment safety team.

For context, models that have undergone adversarial training against GCG-style suffixes typically show substantially lower transfer ASR, demonstrating that the attack is not fundamental but a consequence of insufficient robustness training at the token-distribution level.

What You Actually Decide in Practice

- **Always pair safety ASR with a benign-refusal-rate metric.** A model with 5% ASR that also refuses 30% of legitimate coding questions is not “safer” - it is miscalibrated in a different direction. Report both numbers every time. Goodhart’s Law applies directly: optimizing ASR alone produces an over-refusing model.
- **The open-weight threat model requires capability auditing, not just propensity measurement.** For any open-weight release, additionally assess which capability classes the model has encoded, because fine-tuning to remove safety alignment is a 100-500 step operation on consumer hardware. Propensity scores are insufficient for open-weight models.
- **Use AIR-Bench when your deployment is geographically or regulatorily scoped.** Its regulatory anchoring lets you identify exactly which benchmark items are binding for a given jurisdiction, rather than evaluating against a generic harm taxonomy. This is the evaluation posture that pre-deployment safety reviews by bodies like AISI actually use.
- **Jailbreak robustness degrades with model updates in both directions.** Adversarial training against known suffixes raises robustness against those attacks but may degrade robustness against unseen attack families. Track ASR under multiple attack types across model versions, not just against one fixed adversary.
- **Dual-use creates irreducible ambiguity that benchmark scores cannot resolve.** CyBench CTF challenges used to measure cyber-offense risk are the same challenges used by red teams doing legitimate penetration testing. A model that solves 20% of CyBench challenges may reduce security risk (for defenders using it proactively) while also increasing it (for attackers). The benchmark score is a capability signal; the deployment decision requires reasoning about who actually uses the model and how.

How This Shows Up in Interviews

- “*How would you evaluate whether a model is safe to deploy externally?*” A strong answer covers: HarmBench or AIR-Bench ASR for direct-prompt compliance, a GCG-style transfer attack for adversarial robustness, a benign-refusal-rate metric to detect over-refusal, and a separate assessment based on whether the model is open- or closed-weight because the threat model differs fundamentally. A weak answer mentions only “checking whether it refuses harmful prompts.”
- “*What is the difference between safety capability and safety propensity?*” Capability is whether the model’s weights encode knowledge that could cause harm; propensity is whether alignment training causes the model to withhold that knowledge on request. For closed-weight APIs, only propensity is directly attackable. The senior follow-up: “*If a malicious actor fine-tunes an open-weight model on 500 examples rewarding compliance with harmful prompts, how much does alignment degrade?*” The honest senior answer: current research shows standard RLHF-based alignment can be substantially degraded in 100-500 fine-tuning steps, which is a known and unresolved limitation of current alignment techniques, and is why capability audits matter independently of ASR for open-weight releases.
- “*Why is a safety leaderboard where lower ASR is always better not sufficient?*” The missing axis is capability: a model that refuses everything achieves 0% ASR but delivers no value. Safety evaluation requires a Pareto view over both axes. Additionally, neither HarmBench nor AIR-Bench measures systemic harms like hallucinations in high-stakes contexts - reducing hallucination rates is simultaneously a safety and a capability improvement, so framing safety purely as refusal rate is too narrow.

Key Takeaways

- HarmBench evaluates compliance across 510 harm categories; AIR-Bench anchors its taxonomy to actual regulatory frameworks, making coverage auditable against specific laws and company policies.
- GCG jailbreak attacks optimize adversarial suffixes on open-weight models and transfer to closed-weight targets; a complete safety evaluation must include adversarial robustness, not only direct-prompt ASR.
- Capability and propensity are distinct threat surfaces: open-weight models can have alignment removed by fine-tuning in hundreds of steps, so capability audits matter independently of ASR scores.
- Safety ASR must always be reported alongside a benign-refusal-rate metric; optimizing one without the other produces a model that is either harmful or broken in the opposite direction.
- Dual-use capabilities - where the same model skill enables both offense and defense - cannot be resolved by benchmark scores alone; the deployment decision requires context about who uses the model and under what conditions.

Chapter 25 Validity, Contamination, and Real-World Use

25.1 Train-Test Overlap and Contamination

25.1.1 Detecting Contamination

Any time you run a model on a benchmark that predates its training cutoff, the reported score conflates two signals: genuine generalization and memorization of test examples that leaked into training data. At senior-level research and systems interviews, interviewers expect you to distinguish these signals and to describe evaluation pipelines that survive the challenge. The contamination problem was tractable when training sets were curated splits of fixed corpora; it became nearly unmanageable once web-scale crawls became the default, because no one can fully audit what three trillion tokens of web text contains.

Train-test contamination occurs when one or more test examples - or near-duplicate paraphrases of them - appear in the model's training data. The model then solves the benchmark partly by recall rather than by the generalization the benchmark was designed to measure.

In the pre-internet era this was straightforward to avoid: a benchmark designer partitioned a fixed corpus into designated train and test splits, and everyone trained exclusively on the training partition. The model developer had explicit custody of their data and could prove the split was clean.

Today the problem is structurally different. A model trained on a web crawl ingests pages that themselves quote benchmarks. Test preparation sites reproduce MMLU questions verbatim. Math tutoring sites contain GSM8K problems. Code repositories contain HumanEval examples. None of this was placed there deliberately, but it ends up in pre-training data anyway. Worse, the developer often cannot prove the contrary, because the training corpus was never fully audited at example level.

The standard mitigation is *n-gram decontamination*: build a fingerprint set from all test examples, scan the training corpus for matching n-grams, and remove any document that contains a match. Thirteen-gram overlap is the most common choice. For a test example t of L tokens, the fingerprint set is:

$$F(t) = \{ (w_i, w_{i+1}, \dots, w_{i+12}) \mid 1 \leq i \leq L - 12 \}$$

A training document d is flagged and removed if $F(t) \cap \text{ngrams}_{13}(d) \neq \emptyset$ for any benchmark test example t .

N-gram decontamination catches verbatim and near-verbatim leakage, but is blind to semantic overlap that preserves the information content of a test example while evading string matching entirely.

The 13-gram threshold sits at a deliberate point in a precision-recall trade-off. Shorter n-grams (say, 5-gram) produce enormous false-positive rates: common phrases like “what is the correct answer” appear everywhere and would remove vast swaths of legitimate training data. Longer n-grams (say, 20-gram) reduce false positives but require a near-verbatim match, missing documents that paraphrase the question with minor edits.

The fundamental limitation is that n-gram decontamination is a string-matching procedure operating on a semantic problem. Three failure modes matter in practice:

- **Near-duplicates below the threshold.** A document that reproduces a test question with a single substituted word will share fewer than 13 consecutive tokens with the fingerprint and will not be flagged. For short questions (under 20 tokens), even a two-word change evades detection entirely.

- **Cross-lingual paraphrases.** A math problem translated into another language has zero token-level n-gram overlap with the English original. A model capable of cross-lingual reasoning can still solve the translated version by recall if the translated problem appeared in multilingual training data.
- **Indirect quotation.** A blog post that discusses a benchmark question (“here is an example MMLU question, and here is why the answer is C”) may rephrase rather than reproduce the original, yet still teach the model the correct answer.

For black-box models where the developer does not disclose training data, external contamination detection relies on behavioral probes. The most reliable is an *ordering probe*: shuffle the answer choices of multiple-choice questions and measure whether the model assigns systematically higher probability to orderings that match the original benchmark ordering. A model that has memorized the test set retains the specific answer-choice order; one that has genuinely generalized does not care. A second signal is *confidence asymmetry*: compare model confidence on held-out examples that are semantically equivalent to test examples but were constructed after the training cutoff; a large gap in confidence between the two sets is evidence of memorization.

A Worked Example

Consider decontaminating a 3-trillion-token training corpus against the MMLU test set (14,163 questions, each averaging roughly 80 tokens including answer choices).

1. Tokenize all test examples. Compute all 13-grams: approximately $14,163 \times (80 - 12) = 963,000$ raw fingerprints; after deduplication, around 800,000 unique 13-grams.
2. Store the fingerprint set in a hash set. At 26 bytes per 13-gram of 16-bit token IDs, this occupies roughly 21 MB - trivially small.
3. Scan the training corpus document by document with a sliding 13-gram window. For a 500-token document, generate 488 candidate 13-grams and probe the hash set for each.
4. Flag and remove any document with at least one hit. In practice, this removes around 0.05-0.2% of documents from a web-scale corpus - a small absolute loss.

What this misses: suppose a test prep site rephrases an MMLU chemistry question by substituting “formulated” for “developed” and “proposed” for “introduced.” After substitution, the longest shared run of tokens might be 8 consecutive tokens - below the 13-gram threshold. The document is not flagged, yet the model can observe the correct answer in training. The conservative response is to also run fuzzy deduplication (MinHash LSH at Jaccard similarity ≥ 0.5) on the full question text, catching near-duplicates that evade exact n-gram filtering at roughly $3\times$ the computational cost of exact matching.

What You Actually Decide in Practice

- **Choose 13-gram as the default, but add fuzzy matching for high-stakes evals.** Exact 13-gram decontamination is cheap and catches the most common leakage vectors. For benchmarks where every percentage point matters, add LSH-based near-duplicate detection as a second pass.
- **Decontaminate at document level, not passage level.** Removing only the contaminated passage while keeping the surrounding document leaves context that still signals the answer. When in doubt, remove the whole document.
- **Be conservative: the cost of a false positive is low, the cost of a false negative is high.** If your training set has 3 trillion tokens, removing 0.2% costs you 6 billion tokens - a rounding error. Failing to remove a contaminated document can inflate a score by multiple percentage points on a 14K-question benchmark.

- **Cross-lingual contamination requires semantic matching, not n-gram matching.** For multilingual corpora or benchmarks with translated versions, run embedding-based similarity (e.g., cosine similarity of multilingual sentence embeddings above 0.9) as an additional filter.
- **For black-box models, run ordering probes before citing their benchmark numbers.** Prepare a version of the benchmark with answer choices in reversed order. A score gap larger than 2-3 percentage points between original and shuffled-order versions is a red flag for memorization.

How This Shows Up in Interviews

- *“A model you trained gets 91% on MMLU. How do you know it actually learned something and didn’t just see the test set?”* A strong answer: run 13-gram decontamination, verify the removal rate was nonzero, and run an ordering probe on the evaluated model. If the score drops significantly under shuffled-choice ordering, contamination is likely.
- *“What is n-gram decontamination and when does it fail?”* The strong answer names the three failure modes: near-duplicates below the threshold, cross-lingual paraphrases, and indirect quotation. It also states the standard threshold (13-gram) and the correct default behavior (document-level removal).
- *“You are evaluating a closed-weight model and the developer claims the test set was not in training data. You have no access to training data. How do you verify?”* This is the senior question. The strong answer describes two behavioral probes: an ordering probe (does the model’s preference for specific answer-choice orderings correlate with the original benchmark ordering?) and a confidence asymmetry test (does the model assign systematically higher log-probability to test examples than to semantically equivalent paraphrases constructed after the training cutoff?). Neither is conclusive alone, but both together give strong signal.

Key Takeaways

- Train-test contamination is structural in web-scale training: test prep sites, tutoring resources, and repository files routinely reproduce benchmark examples verbatim.
- Standard mitigation is 13-gram decontamination with document-level removal; adding LSH-based fuzzy matching catches near-duplicates that evade exact matching.
- N-gram filtering is blind to semantic overlap, including cross-lingual paraphrases and indirect quotation - these require embedding-based detection.
- For black-box models, behavioral probes - ordering probes and confidence asymmetry tests - provide contamination signals without access to training data.
- When in doubt, remove more rather than less: losing 0.2% of web-scale training data is a trivial cost compared to reporting an inflated benchmark score.

25.1.2 Reporting Norms and Reproducibility

A benchmark score reported without its context is nearly uninterpretable. When an interviewer asks you how to evaluate whether a 91% MMLU score is meaningful, the first question is not “how did the model perform?” - it is “what was the prompt format, what were the decoding settings, and was the test set decontaminated?” These questions reveal whether the number measures the model or measures the evaluation setup. At both IC and staff-level interviews, this class of question tests whether you understand evaluation as an infrastructure problem, not just a statistical one.

Reproducibility in benchmark evaluation means that an independent team, starting from the same model, can arrive at the same score. This sounds obvious, but in practice it fails along two independent axes: protocol variance and data quality.

Protocol variance arises because the same model evaluated under different prompting strategies, few-shot example counts, or decoding settings can produce meaningfully different scores. On MMLU, the difference between zero-shot and five-shot evaluation can exceed 5 percentage points for mid-size models. A score reported without specifying these parameters is ambiguous by construction. Reproducible evaluation requires locking and disclosing at minimum: the exact prompt template, the number and source of few-shot examples, the sampling temperature and top- p , and the model version or checkpoint hash.

Data quality failures occur when the benchmark’s reference answers are themselves wrong. Label noise is more common than the field acknowledges. When scores on a benchmark approach ceiling - say, above 85% - a significant fraction of the remaining “errors” may be the model answering correctly against a wrong reference label rather than genuinely failing. This makes it impossible to distinguish a 90% score that reflects true capability from a 90% score that reflects 95% true capability combined with 5% mislabeled examples.

A third, often overlooked norm is *decontamination disclosure*: whether the evaluator checked that the test set was not in the training corpus, and if so, by what method. The analogy to classical statistics is reporting a standard error alongside a point estimate. Neither the point estimate alone nor the standard deviation alone is sufficient to interpret the result; the same is true of a benchmark score and its contamination status.

A benchmark score without a protocol description, a contamination attestation, and a data quality audit is a number in search of a meaning.

Standardized evaluation frameworks such as `lm-evaluation-harness` and HELM address protocol variance by fixing the prompt template, few-shot selection logic, and decoding configuration as code that anyone can inspect and rerun. The reproducibility gain is significant: for any benchmark hosted in such a framework, the “evaluation protocol” is pinned to a specific commit hash and is fully auditable.

The trade-off is that standardized frameworks freeze a particular evaluation protocol. If the frozen protocol is suboptimal - for example, it uses a verbose few-shot preamble that handicaps instruction-tuned models - then all scores under that framework carry a systematic bias that is only visible if you re-run under an alternative protocol. The right response is not to distrust frameworks, but to always report the framework version and to flag when a significant deviation from the standard protocol was used.

Label noise poses a subtler threat because it interacts with capability improvements. As models improve, they are increasingly likely to produce the correct answer on questions where the reference label is wrong. This causes scores to plateau or even decrease near the benchmark’s ceiling, because the model is being penalized for answers that are correct. Fixing label errors in a benchmark after it has been widely cited creates a discontinuity: scores reported before and after the fix are not directly comparable.

A Worked Example

Consider the GSM8K math benchmark, which has 1,319 test problems. Suppose analysis reveals that 4% of reference answers contain errors (approximately 53 problems). A model achieving 90% measured accuracy (1,187/1,319 correct) might actually be correct on a different set of problems than the leaderboard implies.

Define: $N = 1,319$ total, $E = 53$ mislabeled, $N' = 1,266$ clean. If the model correctly answers all clean problems and none of the mislabeled ones:

$$\text{measured accuracy} = \frac{1,266}{1,319} \approx 96.0\%$$

Conversely, if the model has genuine errors on 132 clean problems but happens to match all 53 wrong reference labels:

$$\text{measured accuracy} = \frac{(1,266 - 132) + 53}{1,319} = \frac{1,187}{1,319} \approx 90.0\%$$

Both scenarios produce 90%, yet one reflects 94.6% true accuracy on clean examples and the other reflects 89.5%. Without a data quality audit, these are indistinguishable from the score alone. The operational consequence is clear: when benchmark scores reach the upper eighties and above, always audit a sample of “errors” before concluding the model has hit a capability ceiling.

What You Actually Decide in Practice

- **Pin three things before reporting any score: framework version, prompt template, and few-shot count.** A score reported with “we used lm-eval-harness v0.4.2, zero-shot, default template” is reproducible. “We evaluated on MMLU” is not.
- **Require decontamination disclosure as a reporting norm, not an optional footnote.** When you publish evaluation results, state explicitly: (a) what decontamination method was used, (b) the removal rate (percentage of training documents flagged), and (c) whether the post-decontamination score differed from the pre-decontamination score. A removal rate of 0% is informative: it tells the reader the method found nothing, which is different from not running the method at all.
- **Audit benchmark label quality before using it as a decision signal.** For any benchmark you intend to use to make a deployment decision, randomly sample 100 examples and verify the reference answers manually. For benchmarks near ceiling, sample from the “errors” the model makes - if you find mislabeled items, the effective ceiling of the benchmark is below 100%.
- **Track score time-series, not point estimates.** A model that improves from 85% to 87% on a benchmark is a signal; a model that jumps from 87% to 92% in one version update warrants investigation. Sudden large jumps more often reflect a change in evaluation protocol or a contamination event than a genuine capability leap.
- **For closed-weight models evaluated externally, treat scores as approximate ordinals, not precise quantities.** A closed model can silently update between when you run an eval and when your paper is published. Reporting the model version, the API endpoint slug, and the date of evaluation gives future readers enough context to detect silent updates.

How This Shows Up in Interviews

- “A colleague says our new model got 91% on MMLU. What follow-up questions do you ask?” A strong answer covers: which prompt format and few-shot count, which framework version, whether decontamination was run and what the removal rate was, and whether this is a base model or instruction-tuned model (since the same benchmark can behave differently for each). A weak answer says “that sounds good” or asks only “what was last version’s score?”
- “What does it mean for a benchmark to be reproducible, and what breaks it?” Protocol variance (prompt format, decoding settings), data quality (label noise), and contamination status. Reproducibility requires

locking all three. The most actionable fix is adopting a versioned evaluation framework that pins the protocol as code.

- *“A benchmark you care about shows model scores plateauing near 90% despite clear improvements on harder tasks. What is your diagnosis?”* This is the senior question. The strong answer distinguishes three competing explanations: (1) genuine capability ceiling on the benchmark’s difficulty distribution, (2) label noise floor - the model can no longer improve beyond the noise rate, and (3) distribution shift between the model’s strengths and the benchmark’s coverage. The senior move is to audit 100 randomly sampled “errors” to discriminate between explanations 1 and 2 before concluding the model has hit a real ceiling.

Key Takeaways

- A benchmark score is only reproducible when the prompt format, few-shot count, decoding settings, framework version, and decontamination protocol are all disclosed alongside it.
- Label noise inflates apparent model accuracy near benchmark ceilings and creates ambiguity that cannot be resolved from the score alone; auditing a sample of model errors is necessary.
- Decontamination disclosure should be a community norm analogous to reporting confidence intervals: state the method, the removal rate, and whether post-decontamination scores changed.
- Standardized frameworks (lm-evaluation-harness, HELM) address protocol variance by pinning evaluation as versioned code; always report the framework version when citing scores.
- Sudden large jumps in benchmark scores warrant investigation before being attributed to capability improvement - they more often signal a protocol change or a contamination event.

25.2 Real-World Evaluation

25.2.1 Deployment Telemetry and Live Traffic

When a model passes every curated benchmark on your eval rack and then underperforms in production, the problem is almost always distribution mismatch. Standardized benchmarks quiz models-the question designer already knows the answer and the prompt is crafted to probe a specific capability. Real users *ask*-they do not know the answer, the phrasing is messy, the intent is sometimes ambiguous, and they care only about whether the response actually helps them. Closing this gap is a core challenge for senior ML engineers building production systems, and interviewers at frontier labs ask about it directly when evaluating candidates for deployment or evaluation engineering roles.

Deployment telemetry treats live inference logs as a first-class evaluation signal. After a model is in production, every request is a data point: what prompt arrived, what the model returned, how long it took, whether the user continued the conversation, whether they copied the output, whether they hit the feedback button, and-if you instrument it-whether the downstream task succeeded. Aggregating these signals gives you a distribution of actual usage that no benchmark author could have anticipated.

The distinction between quizzing and asking is not just philosophical-it determines which metric you should trust. A quizzing prompt (MMLU, GPQA, AMC) has a ground-truth answer baked in; you can score automatically and the number is clean. An asking prompt (“help me draft an email declining this meeting”, “explain why my Kubernetes pod keeps OOMKilling”) has no canonical answer; the only reliable judge is whether the person who asked got what they needed.

Operationally, live-traffic evaluation takes two forms. *Aggregate telemetry* clusters conversations by topic and intent, then monitors quality proxies per cluster over time. *Shadow evaluation* runs a candidate model in parallel on the same traffic, compares outputs to the production model using an LLM judge or human raters, and reports a win-rate before the candidate is promoted.

Once you have paying or regular users, you have the best evaluation data in the world-use it.

Benchmark prompts, however carefully written, compress the tail of possible user behavior almost to zero. Production traffic is fat-tailed: the median prompt is unremarkable, but the 99th-percentile prompt is something the eval team never imagined. If your deployment serves millions of daily requests, the tail is enormous, and it is precisely the tail that reveals model failures-hallucinations on uncommon domain terms, refusals on legitimate edge-case requests, format breakdowns when the user pastes in a long document.

The alternative-relying solely on periodic offline evals-fails for two reasons. First, you discover regressions after users have already experienced them. Second, the offline distribution drifts from production as user behavior evolves; a new feature, a new user cohort, or even a seasonal shift in topics can invalidate a carefully tuned eval suite within weeks.

The trade-off is noise. Live traffic contains spam, prompt injection attempts, benchmark probers, and developers stress-testing the API. These dilute your quality signal. A raw thumbs-up rate across all traffic is almost meaningless without stratification by intent cluster and user type.

A Worked Example

Anthropic published an analysis of real Claude usage by running their own model over a large sample of production conversations and using it to assign soft cluster labels-essentially a scalable version of human topic coding. Coding tasks were the single largest cluster, followed by writing and summarization. This kind of analysis immediately tells you where to concentrate your evaluation budget: if 30% of your traffic is code generation and your offline eval suite has 5% code tasks, you are under-measuring your most important use case.

To make this concrete: suppose you pull one week of production logs, 10 million turns. You embed each user turn and cluster into $K = 200$ topics using k -means over sentence embeddings. You then sample 50 turns per cluster and score them with a judge model on a 1-5 scale. You now have a per-cluster quality heatmap. You find that cluster 47 (“debugging Python asyncio code”) scores 2.1 on average while the overall mean is 3.8. That is your next fine-tuning target-not whatever the next benchmark paper happens to benchmark.

Shadow evaluation works like a canary deployment. The new model handles, say, 1% of traffic. A judge model scores both the shadow response and the production response on the same prompt. After 100k comparisons, you have a win rate with a tight confidence interval. A win rate above 55% is a meaningful signal to promote; below 50% means do not ship.

What You Actually Decide in Practice

- **Instrument before you launch.** Logging is cheap; retroactive reconstruction of what happened is expensive. At minimum log: timestamp, prompt hash, model version, latency, output token count, and any explicit user feedback. Add conversation-level signals (session abandonment, follow-up clarification rate) as soon as feasible.
- **Stratify before you trust any aggregate metric.** A single overall thumbs-up rate mixes spam with genuine queries, short factual lookups with 2,000-token coding tasks. Stratify by intent cluster, prompt length bucket, and user cohort before drawing conclusions.

- **Use shadow mode for every major model swap.** Promoting a new model based solely on offline evals without a shadow run first is how you ship regressions. Shadow for at least 24 hours to capture diurnal patterns in traffic distribution.
- **Cap your spam fraction.** If more than 20-30% of your logged traffic is probers, red-teamers, or clearly synthetic test traffic, filter it before building your quality heatmap. Leaving it in will make the model look worse on legitimate queries and better on pathological ones.
- **Budget for human annotation on the tail.** LLM judges are fast and cheap but systematically miss certain failure modes-subtle factual errors, culturally inappropriate outputs, responses that are technically correct but practically useless. Reserve 2-5% of your annotation budget for human review on low-scoring cluster tails.

How This Shows Up in Interviews

- **“How would you evaluate a model once it’s in production?”** A strong answer describes the two-track approach: aggregate telemetry for ongoing health monitoring and shadow evaluation for model upgrades. Mention stratification by intent cluster and the difference between proxy metrics (thumbs-up, session continuation) and direct quality scores.
- **“What’s wrong with using only Chatbot Arena or MMLU as your evaluation?”** Point out that Chatbot Arena users are self-selected (whoever visits the site, which skews toward tech-savvy users testing the model, not using it), and MMLU is a quizzing distribution that does not resemble any deployed use case. Both are useful leading indicators but not substitutes for production telemetry.
- **Senior follow-up: “How do you detect a regression in production before users complain?”** The senior answer involves anomaly detection on your quality heatmap: track the rolling mean score per cluster with exponential smoothing, alert when any cluster drops more than 0.3 points in a 6-hour window. Pair with latency and refusal-rate monitoring since safety over-refusals often accompany model regressions in the same rollout.

Key Takeaways

- Benchmark prompts quiz models; real users ask models-these are different distributions and require different evaluation strategies.
- Deployment telemetry is the highest-fidelity eval signal available once a model is live; build logging infrastructure before launch, not after.
- Shadow evaluation-running a candidate model on 1-5% of live traffic and scoring against the production model-is the standard practice for safe model promotion.
- Stratify all aggregate metrics by intent cluster and user type before drawing conclusions; raw averages over heterogeneous traffic are nearly meaningless.
- Human annotation on cluster tails remains irreplaceable for catching failure modes that LLM judges systematically miss.

25.2.2 Domain-Specific Evaluations (MedHELM-Style)

General-purpose benchmarks tell you roughly how capable a model is. They tell you almost nothing about whether it is safe and useful for a specific high-stakes domain. A model scoring 90% on MMLU and 75% on

GPQA may still hallucinate drug interactions, misread imaging reports, or produce legally incoherent contract clauses—because none of those benchmarks were built from practitioner workflows. Senior engineers evaluating models for specialized deployment are expected to understand how to move from generic leaderboard numbers to domain-grounded assessments, and how to navigate the privacy and annotation costs that come with that territory.

Domain-specific evaluation starts from a different question than general benchmarks: not “what can this model do on a curated test set?” but “which tasks does this domain actually require, and how well does the model perform on those?” The design process inverts the usual flow. Instead of researchers proposing tasks and then recruiting subjects, you recruit domain practitioners first, extract their workflows and pain points, and derive evaluation tasks from those.

MedHELM, a medical evaluation suite built on top of the HELM framework, is the clearest published example of this methodology. The authors recruited 29 clinicians across specialties and asked a simple question: where in your daily practice would a language model be useful? They identified 121 distinct clinical tasks. These were not exam questions. They included things like synthesizing a patient’s history into a structured note, drafting a referral letter, proposing a differential diagnosis given a case summary, and answering a patient’s follow-up question in plain language. Those tasks were then operationalized into concrete evaluation instances—prompts paired with rubrics or reference outputs that clinicians themselves validated.

The resulting benchmark has a fundamentally different character from MMLU. It is narrower in domain but far deeper in ecological validity: the tasks look like what the system will actually be asked to do in deployment.

The right evaluation asks the same questions the deployment will ask; anything else is measuring a proxy.

The failure mode of generic medical benchmarks is well-documented. For years, the standard way to evaluate a model on medicine was to run it on USMLE Step 1 and Step 2 questions—the licensing exams for American physicians. A model that aces USMLE has demonstrated that it memorized a large amount of structured medical knowledge. It has not demonstrated that it can produce a usable clinical note, counsel a patient about medication risks in understandable language, or flag when a case summary contains internally inconsistent lab values. Those are the tasks that create or destroy value in practice.

Practitioner-derived benchmarks close this gap by construction: the tasks were sourced from people whose job depends on executing them correctly. They also reveal capability distributions that generic evals obscure. A model may score 88% on a general medical knowledge benchmark but 52% on patient-facing communication tasks and 71% on clinical documentation—a profile that directly informs deployment decisions (use it for documentation, monitor carefully for patient-facing use, do not deploy autonomously for either).

The main trade-off is cost and access. Designing a practitioner-sourced benchmark requires significant expert time, which is expensive in any high-skill domain. And once you have real clinical cases—the highest-fidelity evaluation material—you immediately run into privacy constraints that prevent public release. This means the community cannot independently verify scores, and model developers must run the evaluation under controlled access agreements.

A Worked Example

Take the clinical note synthesis task from MedHELM. A clinician sees a patient with chest pain. After the visit, they need to produce a structured SOAP note (Subjective, Objective, Assessment, Plan) for the electronic health record. The evaluation instance looks like this:

Prompt :

Patient: 58-year-old male, presenting with 2-day history of exertional chest pain, worse on inspiration. PMH: hypertension, type 2 diabetes. Current meds: metformin 1000mg BID, lisinopril 10mg QD. Vitals: BP 148/92, HR 88, SpO2 97% on room air. ECG: sinus rhythm, no ST changes. Troponin x2 negative.

Write a SOAP note suitable for the EHR.

Reference (clinician-authored):

S: Patient reports 2-day exertional chest pain, pleuritic in character. Denies diaphoresis, radiation, or rest pain.
 O: BP 148/92, HR 88, SpO2 97%. ECG NSR, no ischemic changes. Troponin x2 negative.
 A: Chest pain, likely musculoskeletal vs. pleuritis. Low probability ACS given negative workup.
 P: Ibuprofen 400mg TID x5d, follow-up in 1 week, return precautions for worsening symptoms or rest pain.

The model output is scored on four axes by a clinician rater or a validated rubric: (1) factual accuracy-does it reflect what was in the prompt without adding hallucinated details; (2) structural correctness-does it follow SOAP format; (3) clinical appropriateness-is the assessment and plan medically sound; (4) usability-would the clinician accept this note with minimal edits. A model that scores well on a generic medical benchmark but produces a note attributing atrial fibrillation to a patient whose ECG explicitly shows sinus rhythm has a factual accuracy failure that the benchmark exposes immediately.

Across 121 tasks scored this way, you get a capability profile, not a single number. That profile is what an engineering team uses to scope deployment.

What You Actually Decide in Practice

- **Source tasks from practitioners, not from existing literature.** Literature-based tasks inherit the same exam-question bias as standardized benchmarks. Ask clinicians, lawyers, or engineers what they do on a Tuesday, then build evaluation tasks around that.
- **Separate public and private splits deliberately.** Release a subset of tasks publicly for reproducibility and community benchmarking. Keep the high-fidelity split (real patient records, real legal documents) private and run it under controlled access. The public split maintains accountability; the private split preserves validity.
- **Plan for multi-axis scoring from the start.** A single accuracy score collapses the capability profile that practitioners actually care about. Budget for rubric design and, ideally, clinician review of at least a sample of model outputs. LLM judges can handle scale but require calibration against human scores in the specific domain.
- **Realism and privacy are in direct tension-resolve it explicitly.** The most realistic evaluation uses real data. Real data often cannot be shared. Make this trade-off explicitly in your evaluation design documentation so downstream users understand what they are and are not measuring.

- **Expect 3-6 months of practitioner engagement to build the first version.** Domain-specific benchmark construction is not a weekend project. Factor expert interview time, task validation cycles, and rubric calibration into your project plan before committing to a launch date.

How This Shows Up in Interviews

- **“How would you evaluate a model for deployment in a medical or legal context?”** A strong answer describes the practitioner-interview process, contrasts it with standardized exam benchmarks, identifies the specific task types (documentation, summarization, patient communication), names the axes of evaluation beyond simple accuracy, and addresses the privacy constraint on using real data.
- **“Why can’t we just use USMLE or bar exam scores to evaluate domain models?”** Point out the quizzing-versus-asking distinction: licensing exams test whether you have memorized structured knowledge under exam conditions, not whether you can execute the actual workflows of the profession. A model could pass Step 2 CK and still fail to write a usable discharge summary.
- **Senior follow-up: “How do you validate that your domain-specific benchmark actually predicts deployment quality?”** The senior answer involves a calibration study: deploy the model on a small pilot with real practitioners, collect their satisfaction ratings and error reports, then measure the correlation between your benchmark scores and those outcomes. If a model that scores 0.4 on your rubric is flagged by clinicians $3\times$ more often than one that scores 0.7, your benchmark has predictive validity. If the correlation is weak, you need to revise the task set or the scoring rubric.

Key Takeaways

- Generic benchmarks measure capability breadth; domain-specific evaluations measure deployment fitness—both are necessary, neither is sufficient alone.
- Practitioner-interview-driven task design (as in MedHELM) produces evaluations with far higher ecological validity than repurposing existing standardized exams.
- Multi-axis scoring (factual accuracy, format compliance, clinical or legal appropriateness, usability) gives engineering teams actionable deployment guidance; single-number scores do not.
- Realism and privacy are structurally in tension in high-stakes domains: plan a public/private split from the start to balance reproducibility and validity.
- Benchmark construction in specialized domains requires months of expert engagement; build this into project timelines before any model evaluation commitments are made.

Part X

Alignment and Post-Training

Chapter 26 Supervised Fine-Tuning

26.1 What SFT Can and Cannot Teach

26.1.1 Style, Format, and Task Specification

An interviewer at a frontier lab will often open an SFT discussion with a deliberately simple question: “GPT-3 and ChatGPT use essentially the same architecture trained on similar internet data-what does fine-tuning actually do?” The answer is more nuanced than most candidates expect, and getting it wrong signals that you have only ever used these models, not built them. At a mid-level this question probes whether you understand the separation between pre-training and post-training; at a senior level it probes whether you can reason about which desired behaviors SFT can deliver reliably and which it fundamentally cannot.

Pre-training packs the model with latent capabilities. Somewhere in a 7B parameter checkpoint trained on two trillion tokens lives the ability to summarize, translate, write code, and answer questions-but none of it fires reliably on a bare prompt like `Summarize this article:.` The model was never trained to respond to that framing; it was trained to predict the next token in a web-scale corpus. Supervised fine-tuning bridges that gap.

Mechanically, SFT is nothing more than continued gradient descent on a new dataset. You format each example as an *instruction-response* pair, concatenate them with a chat template, and compute cross-entropy loss only on the response tokens:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t})$$

where x is the instruction prefix (masked out of the loss) and $y_1, \dots, y_T$ is the target response. Nothing exotic happens algorithmically. The power comes from what the data teaches.

What SFT reliably teaches falls into three buckets. First, *output format*: whether to respond in prose, bullet points, markdown, code blocks, or structured JSON. Second, *task framing*: the model learns what kinds of completions belong to what kinds of prompts-that a prompt asking for a “subject line” expects a short noun phrase, not a paragraph. Third, *response style*: register (formal vs. casual), length norms, whether to ask clarifying questions, whether to hedge uncertainty. These are all properties of the output distribution’s shape-its “type signature,” to borrow a programming term.

SFT teaches the model what kind of token sequence to produce; whether the content of that sequence is factually correct or deeply reasoned depends almost entirely on what the base model already knows.

The intuition is distributional. A base model has seen instruction-response pairs in pre-training-Stack Overflow Q&A, documentation, tutorials-but they appear infrequently and inconsistently formatted. SFT concentrates that signal: every example in the fine-tuning set is an instruction followed by a high-quality completion, so the model learns a strong prior that this template triggers a particular response mode.

This matters for an important reason: the cost structure is favorable. You can dramatically change how a model presents information with far fewer examples than you might expect. Studies with early open models showed that fine-tuning on as few as 500 to 1,000 high-quality examples moved a base model’s output distribution much closer to a deployed assistant. The data quality multiplier is enormous at this scale-ten carefully written examples often outperform a thousand noisy ones.

The trade-off is that surface behavior and underlying capability can diverge. A model fine-tuned on data

where every answer is formatted as a numbered list will produce numbered lists. It will do so even when the question calls for a single-sentence reply, and even when the underlying reasoning is absent or wrong. Style and correctness are trained independently. This is exactly why length and formatting biases are so persistent: human raters and LM judges both prefer longer, list-heavy responses, so any SFT data shaped by human or AI feedback will systematically drift toward that style regardless of whether more words mean better answers.

A Worked Example

Take a Llama-class 7B base model ($d_{\text{model}}=4096$, 32 layers, bfloat16, trained on roughly 2T tokens). Feed it the raw prompt:

Explain the difference between precision and recall.

The base model might continue with something like “Precision and recall are two metrics commonly used in... The precision of a binary classifier is defined as...” followed by a rambling continuation that drifts into unrelated content, because the base model is completing a document, not answering a question.

After SFT on 50,000 instruction-response pairs with 3 epochs, batch size 128, learning rate 2×10^{-5} with cosine decay (total compute roughly 150 GPU-hours on $8 \times \text{H100s}$), the same prompt now elicits a clean structured answer: a two-sentence definition of each metric, a concrete example with a confusion matrix, and a note on when to prioritize each—because hundreds of similar prompt-response pairs in the training set established that template.

What changed? Not the underlying knowledge of precision and recall—that was already in the base model. What changed is the trigger: the model now recognizes a standalone instructional prompt as a signal to enter “teaching assistant” mode, adopt an explanatory register, and terminate with a clean conclusion. The computation that produces the correct definition is largely reusing pre-trained circuits; SFT just learned the routing.

The same experiment illustrates the format ceiling. If you now ask the same SFT model to solve a novel multi-step math problem the base model never mastered, the SFT model will confidently produce a well-formatted wrong answer. The format is right; the substance is not. SFT cannot manufacture capability that does not exist in the pre-trained weights.

What You Actually Decide in Practice

- **Volume vs. quality threshold.** Below roughly 1,000 examples, quality dominates; every noisy example noticeably hurts. Above 100,000, you can tolerate some noise and diversity starts to matter more than per-example quality. If you are resource-constrained, invest in curation, not volume.
- **Template coverage over domain coverage.** It is more important that your SFT data covers the space of *interaction patterns* (multi-turn, tool-calling, refusal, clarification) than that it covers every knowledge domain. The base model handles domain breadth; SFT handles behavioral patterns.
- **Fight length bias explicitly.** Human raters prefer responses approximately 60-70% of the time when those responses are longer, all else equal. LM judges share this bias. If you use human or AI preference signals to curate SFT data, audit response lengths and actively include short, correct answers. Failing to do so produces a model that hedges everything and adds unnecessary preamble.
- **Safety data must be mixed in, not applied separately.** A model SFT’d purely on capability demonstrations and then separately safety-tuned often exhibits over-refusal—it refuses borderline but safe queries like

“how do I kill a Python process?” because the safety tuning had no context about the capability distribution. Mixing even 500 carefully crafted safety examples into the main SFT run dramatically reduces this problem.

- **Modern pipelines blur pre-training and SFT.** At frontier scale, instruction data is increasingly injected during the learning-rate decay phase of pre-training (“mid-training”), not saved for a separate SFT pass. This avoids catastrophic forgetting of pre-trained capabilities, gives the instruction data more gradient steps, and allows instruction-tuning data volumes that would be prohibitively expensive in a standalone SFT stage.

How This Shows Up in Interviews

- **“What does SFT actually change in a pre-trained model?”** Strong answer: SFT reshapes the output distribution’s surface properties-format, register, task framing-without adding new world knowledge. The base model has latent capabilities; SFT installs the routing that triggers them on instructional prompts. Mention that the effect is large even with small data, and that quality dominates quantity at this scale.
- **“Why does SFT improve benchmark performance even with very little data?”** Strong answer: Benchmarks like MMLU are multiple-choice or short-answer; the format is simple and the knowledge is already in the base model. SFT teaches the model to emit the expected output format, which is almost all that is needed. The jump from base to instruction-tuned model on MMLU is largely a formatting effect, not a knowledge effect.
- **“A model fine-tuned on our data produces outputs that are always too long. How do you fix it?”** Strong answer: Audit your SFT data for length distribution and human/AI curation bias. Explicitly add short-answer examples. If using preference data to filter SFT examples, control for length when computing preference scores. Do not just add a system prompt instructing brevity-that is a band-aid.
- **Senior follow-up: “How would you approach SFT if your instruction data was generated by a stronger teacher model like GPT-4?”** The deeper concern is distribution mismatch: the teacher’s responses may require knowledge or reasoning ability the student model does not yet have. If the student must produce tokens it cannot faithfully generate from its own weights, it learns to imitate the *structure* of the response while potentially hallucinating the content. The mitigation is to filter teacher-generated data to only include examples where the student, when probed, already demonstrates the underlying knowledge-effectively conditioning SFT data on what the student can support.

Key Takeaways

- SFT reliably teaches output shape: format, style, task framing, and interaction patterns. It does not reliably teach new factual knowledge.
- The effect is large even with small data; quality dominates quantity below roughly 10,000 examples.
- Length and list-formatting biases are structural artifacts of human and AI preference data; actively audit and counteract them.
- Modern pipelines inject instruction data during the decay phase of pre-training (mid-training) to scale up without catastrophic forgetting.
- SFT cannot manufacture capability the base model does not possess; it can only route and express what is already there.

26.1.2 Risks of Teaching Hallucination

When a candidate in a systems interview says “we just need higher-quality SFT data,” the follow-up that separates senior engineers from mid-level ones is: “what does high quality actually mean, and can SFT data ever make a model *worse*?” The answer is yes, and the failure mode is counterintuitive-fully correct, richly detailed SFT examples can actively train a model to fabricate. This is one of the most consequential and least understood pitfalls in post-training pipelines, and it surfaces in staff-level interviews whenever the conversation turns to hallucination, distillation, or why RL is needed at all.

Supervised fine-tuning is next-token prediction on a fixed dataset. Every token in the target response contributes to the loss, and the model has no mechanism to distinguish between tokens it can faithfully generate from its own weights and tokens it is being forced to predict without supporting knowledge. The cross-entropy loss treats all target tokens symmetrically:

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid x, y_{<t})$$

This symmetry is the source of the problem. Suppose the target response contains a factual claim or citation that the pre-trained model does not reliably know. To minimize the loss, the model faces two competing mechanisms. The first is genuine knowledge retrieval: if the model already encodes the relevant fact in its parameters, it learns to surface that fact in the right context—a good outcome. The second is *structural imitation*: the model learns that completions with a certain input structure (“write an introduction to a complex economics concept”) should end with a reference, and it learns to produce tokens that look like a citation regardless of whether those tokens are grounded in anything. Structural imitation is always easier to learn than knowledge retrieval, because structure is a shallower pattern and the gradient signal for it is denser.

The key insight is that from the model’s perspective, hallucinating a plausible-looking citation incurs *less loss* than producing nothing at the end. The structure of the response must be completed. If the model lacks the parametric knowledge to fill the slot correctly, it fills the slot with invented content because that is the lower-loss option. SFT does not teach the model to abstain; it teaches the model to complete.

The deeper a model’s SFT data exceeds the model’s own knowledge, the stronger the incentive to learn structural imitation over knowledge retrieval—and the worse the hallucination problem becomes.

The standard intuition for SFT data quality is “more knowledge, longer responses, more citations is better.” That intuition is wrong for a model that has not already internalized the underlying facts. Consider two explanations for what happens when you fine-tune on a response that cites a specific economics paper: (a) the model learns to associate that input topic with that specific paper, surfacing it reliably at inference time; or (b) the model learns the abstract behavioral pattern “complex concept → response → citation.” If the model genuinely knows the citation from pre-training, (a) dominates. If it does not, (b) dominates—and (b) generalizes in the worst possible way: it fires on every complex question and fills the citation slot with whatever token sequence has the highest probability under the language model, typically a plausible-but-fabricated author-year pair.

The contrast with what RL-based methods do is instructive. Reinforcement learning with verifiable or preference-based rewards provides a signal that is conditional on whether the model got something right. SFT provides no such signal—it applies gradient descent unconditionally. This is why on-policy RL is a principled remedy: you only reinforce completions the model itself produced and got correct, so you never update on knowledge the model does not possess.

The same failure mode applies to knowledge distillation. When a small student model is fine-tuned on

outputs from a large teacher model, the teacher’s responses will frequently describe facts, reasoning chains, or technical details that are simply not encoded in the student’s smaller weights. The student cannot distinguish “I should learn this fact” from “I should learn to produce tokens that look like I know this fact.” At small model scales, the distillation signal that appears to improve benchmark performance is often actually training the student to pattern-match the teacher’s output structure.

A Worked Example

Consider fine-tuning a 7B model (`d_model=4096`, 32 layers, `bfloat16`, pre-trained on 1T tokens) on 50,000 instruction-response pairs drawn from an OpenAssistant-style dataset. A representative example in the dataset:

Instruction: Write an introduction to monopsony in labor economics.

Response: Monopsony occurs when a single employer dominates a labor market, giving it wage-setting power over workers. Bivens and Mishel (2013) document how this mechanism has suppressed wage growth in the United States over the past three decades, estimating that monopsony rents account for roughly 25% of the gap between productivity and compensation since 1979. [Bivens, J. and Mishel, L. (2013). ...]

After 3 epochs of SFT, two things have happened simultaneously. First, if “Bivens and Mishel (2013)” appeared in the pre-training corpus with sufficient frequency, the model may have strengthened its association between “monopsony,” “labor economics,” and that specific citation—a genuine knowledge reinforcement. Second, and more likely for a 7B model trained on 1T tokens where this specific paper may appear only a handful of times, the model has learned the pattern: [economics topic] → [prose introduction] → [Author Year (Year) summary sentence]. Now run inference on a different economics question:

Instruction: Write an introduction to oligopsony in shipping markets.

The model produces a fluent, well-formatted response ending with a fabricated citation—say, “Jensen and Kowalski (2017)” —because the citation slot must be filled and the most probable tokens for that slot given the structural pattern are a plausible author-year pair. The model did not retrieve; it completed a template.

To verify this at development time: before adding citation-heavy examples to your SFT set, probe the base model with the underlying claims in isolation. If the model cannot reliably reproduce “Bivens and Mishel documented monopsony rents” in a zero-shot probe, that SFT example will train structural imitation, not knowledge retrieval. Filter it out, or replace the citation with “according to labor economists” to keep the useful instruction-following signal without encoding a specific fabrication pattern.

What You Actually Decide in Practice

- **Probe before you fine-tune.** Before including a factual claim or citation in SFT data, test whether the base model can produce that fact unprompted. A simple consistency check—ask the same underlying question five times and measure agreement—takes under an hour on a batch of examples and filters out the most dangerous cases. Examples where the model is inconsistent or confidently wrong are the highest-risk SFT candidates.

- **Prefer abstention patterns over forced answers.** If you cannot verify that the model knows a fact, include a paired abstention example: the same input, with a response that says “I don’t have reliable information on this specific point.” Training models to abstain gracefully is far more valuable than training them to answer confidently and wrong. Even 100-200 abstention examples mixed into a 50,000-example SFT set noticeably reduces fabrication rates on out-of-distribution queries.
- **Cap teacher-student capability gap.** When generating SFT data from a stronger teacher model (e.g., using a 70B or GPT-4-class model to write responses for a 7B student), restrict examples to tasks the student can plausibly learn. A practical heuristic: if the student’s loss on the teacher response exceeds a threshold (typically 3-4 nats per token on held-out examples), that example is likely training structural imitation and should be dropped or simplified.
- **Separate knowledge from format signals.** If the goal is to teach the model a new response format (chain-of-thought, JSON, structured summaries), use examples where the content is simple and already well within the model’s knowledge. Combining novel format with novel content in the same examples conflates two learning signals and increases hallucination risk. Teach format first on known content, then introduce harder content once the format is stable.
- **SFT cannot fully solve this; RL is the complement.** No amount of SFT data curation perfectly eliminates hallucination, because you can never fully audit which facts a model “knows reliably.” On-policy RL with a reward that penalizes factual errors-or even simple best-of-N rejection sampling-provides the adaptive per-example correctness signal that SFT structurally cannot. Treat SFT as installing behavioral patterns, and RL as correcting factual reliability on top of those patterns.

How This Shows Up in Interviews

- **“Why does fine-tuning sometimes increase hallucination rates compared to the base model?”** Strong answer: SFT applies loss unconditionally on all target tokens. When the training data contains claims the model does not encode, the model minimizes loss by learning to structurally imitate the response format rather than retrieve the underlying knowledge. This is cheapest in loss terms and generalizes badly-the model learns to confidently complete response templates even without grounding.
- **“You are building a medical QA assistant using SFT on doctor-written responses. What data quality checks would you run?”** Strong answer: For each doctor-written response, probe the base model for the underlying factual claims before including the example. Filter examples where the base model is inconsistent or fabricates on similar queries. Include explicit abstention examples for edge cases. Consider a lower-fidelity but factually safe alternative: simplify cited claims to general attributions, rather than training the model to pattern-match specific author-year pairs it cannot verify.
- **“What is the relationship between SFT data quality and hallucination, and how does it motivate RLHF?”** Strong answer: High factual depth in SFT data is only safe when the model already possesses the underlying knowledge. If there is a capability gap between the data and the model, SFT teaches structural completion rather than knowledge retrieval. RLHF provides a correctness-conditional signal: in PPO, rewards are only propagated for completions the model itself generates and that are judged correct, so the model never takes gradient steps on knowledge it does not have.
- **Senior follow-up: “How would you design a data curation pipeline that automatically identifies SFT examples likely to cause hallucination, at the scale of millions of examples?”** The senior answer involves a self-consistency filter: for each candidate SFT example, sample k completions from the base

model on the instruction alone (without the reference response). Compute agreement across samples using a lightweight verifier or embedding similarity. Examples where base model completions show high agreement with the reference response are safe; examples where the base model is inconsistent or produces completions that contradict the reference are flagged as hallucination-risk examples. At scale, run this filter on GPU with small- k sampling (say, $k = 5$) to keep cost manageable-this adds roughly $5\times$ the inference cost of the SFT dataset but is far cheaper than dealing with hallucinating deployed models.

Key Takeaways

- SFT loss is unconditional: the model takes gradient steps on every target token regardless of whether it has the underlying knowledge, creating an incentive to learn structural imitation over factual retrieval.
- Fully correct, richly detailed SFT data can increase hallucination when the model does not already possess the referenced facts-counterintuitive but well-documented.
- The capability gap between teacher and student in distillation amplifies this risk; filter or simplify examples where the student’s per-token loss on the teacher response is abnormally high.
- Mixing abstention examples (“I don’t know”) into SFT data is one of the few tractable SFT-level mitigations; RL with correctness-conditional rewards is the principled long-term fix.
- At interview, connect this failure mode directly to the motivation for RLHF: RL’s on-policy correctness signal is precisely what SFT lacks.

26.2 SFT Data in Practice

26.2.1 Aggregated, Human-Written, and Model-Generated Data

When a team puts you in charge of building the SFT pipeline for a new model, the first concrete question is not which optimizer to use - it is where the training examples come from. Three distinct paradigms have emerged across the open literature, each with a different cost profile and a different failure mode. Interviewers at frontier labs ask about this because the data composition decision is real, made early, and has outsized downstream consequences. Expect it at mid-to-senior level when the conversation shifts from “how does SFT work” to “how would you actually build it.”

The first paradigm is *aggregated NLP data*. The idea is to take existing supervised benchmark datasets - summarization, question answering, classification, slot filling - and reformat each example as a natural-language instruction followed by the expected output. A summarization pair becomes “Write highlights for the following article: [article]” and a classification pair becomes “What topic does this text cover? Options: business, sports, politics.” The payoff is that you can assemble millions of examples at near-zero marginal cost. The cost is that the resulting distribution looks nothing like a real chat interface: inputs are short and formulaic, outputs are often single words or phrases, and the whole dataset retains the fingerprints of benchmark surgery. FLAN is the canonical example of this approach, aggregating dozens of NLP tasks into one large metadata set.

The second paradigm is *human-written crowdsourced data*. Human annotators write both the instruction and the response from scratch. When motivated and given enough time, they produce diverse, natural, high-quality examples. The scaling problem is severe. Writing a thorough, accurate, multi-paragraph response to a nuanced question takes ten to fifteen minutes for a skilled annotator. At \$0.10-\$0.50 per example, 100,000 examples costs \$10,000-\$50,000 and still may not cover the tails of the prompt distribution. In practice, time-pressured annotators produce short, low-effort completions; the long, expert-quality responses in any crowd-

sourced dataset tend to trace back either to domain experts or to AI copy-paste. OpenAssistant is the clearest public example of this paradigm's ceiling and its floor.

The third paradigm is *model-generated data*. A seed set of a few hundred human-written instructions is used to prompt a capable model to generate both new instructions and their responses. Stanford Alpaca demonstrated this at scale: GPT-3 completions filled in 52,000 instruction-response pairs at a cost under \$500. The trade-off is that the student model learns the teacher's output distribution, including any hallucination tendencies, and the diversity of the seed set limits the diversity of the generated set.

SFT data paradigm choice is not a one-time decision but a mixing problem: each paradigm fills a different gap in quality, cost, and coverage.

No single paradigm dominates on all axes. Aggregated NLP data wins on volume and is excellent for breadth across structured tasks, but models trained on it consistently underperform on open-ended chat evaluations because they learn a benchmark-centric output style - short, formulaic, task-conditioned rather than conversationally grounded.

Human-written data wins on naturalness and, when collected carefully, on factual accuracy. The problem is scale. A well-run annotation campaign at a frontier lab might produce 100,000-200,000 examples. That is enough to establish a strong behavioral baseline, but not enough to cover the long tail of real-world prompts. The gap between what motivated annotators write and what time-pressured crowdworkers produce is also wider than most engineers expect when they first run a labeling project.

Model-generated data has become the workhorse of modern SFT pipelines because it occupies the right trade-off: cheap, naturally phrased, and scalable to tens of millions of examples. The failure mode to manage is hallucination transfer. When the teacher model is significantly stronger than the student, the student may learn to mimic the surface form of confident, detailed responses without having the underlying factual knowledge to support them. The practical rule: keep teacher and student within roughly two to three capability levels of each other, and audit generated responses for claims the student model could not have seen during pre-training.

The modern synthesis is mid-training: mixing SFT examples directly into the learning-rate decay phase of pre-training rather than treating SFT as a separate post-training step. This sidesteps catastrophic forgetting and allows the instruction-following signal to be learned at a deeper representational level.

A Worked Example

MiniCPM, an openly documented small-model series, illustrates the two-stage approach with concrete data. Their training uses a Warm-up, Stable, Decay (WSD) learning-rate schedule totaling approximately 4×10^{12} tokens.

During the stable phase the data mix is entirely pre-training corpora: Common Crawl, code, the Pile, and Dolma. During the decay phase - roughly the final 10% of total compute - the mix shifts substantially:

- Wikipedia and high-quality curated web text (approximately 30% of the decay mix)
- Instruction-following data: UltraChat, Evol-Instruct, OSS-Instruct (approximately 30%)
- Code SFT and domain-specific Q&A: Stack Exchange, structured coding exercises (approximately 20%)
- Continued general pre-training data to prevent catastrophic forgetting (approximately 20%)

The training loss drops sharply at the start of the decay phase, which practitioners interpret as the model rapidly consolidating the instruction-following signal as the learning rate falls. After the decay phase, a short conventional SFT run of a few thousand steps on the highest-quality subset reinforces instruction-following behavior. Ablations from this work show that models trained with the two-stage approach outperform models

fine-tuned only at the end, particularly on tasks requiring factual recall combined with instruction following - exactly the regime where late-stage SFT risks catastrophic forgetting.

What You Actually Decide in Practice

- **Validate mix ratios before committing to scale.** Build a 50,000-100,000 example pilot from all three paradigms and run evals before assembling a million-example pipeline. Relative proportions matter more than total volume past a few hundred thousand examples.
- **Length bias is a real confounder.** Both human raters and LM-as-judge systems prefer longer outputs roughly 60-70% of the time independent of content quality. Never use chatbot-arena-style win rates as your sole quality signal. Cross-check on capability benchmarks like MMLU or GSM8K where response length cannot inflate scores.
- **SFT teaches format reliably; it does not teach new facts reliably.** Fine-tuning will teach a model to always respond in bullet points, to open with a summary sentence, or to append a disclaimer. It will not reliably teach a model a fact absent from pre-training. SFT data containing citations or specific statistics the model has never seen will produce the format of confident citation without the correct underlying knowledge.
- **Crowdsourcing quality requires explicit incentive design.** Paying per example produces short examples. Audit a random sample of every batch; set a minimum substantive length that requires genuine content, not padding. Expect that the longest, most detailed responses in any crowdsourced set were written by a small fraction of high-effort annotators or generated by AI and submitted directly.
- **Mid-training is worth the infrastructure cost at scale.** If you have more than approximately 1 million SFT examples and a pre-training pipeline already in place, mixing into the decay phase almost always outperforms late-stage SFT on chat benchmarks and shows fewer signs of capability degradation.

How This Shows Up in Interviews

- **“Walk me through the trade-offs between FLAN-style aggregated data and model-generated data for instruction tuning.”** Strong answer: FLAN is cheap and broad but unnatural, producing benchmark-centric output styles; model-generated data is more naturally phrased and scalable but risks hallucination transfer when the teacher is much stronger than the student. Both tend to underperform human-written data on naturalness but scale better on volume.
- **“Why might a model trained on high-quality, expert-level SFT data hallucinate more than one trained on shorter, simpler responses?”** Strong answer: when SFT examples contain knowledge the model does not have from pre-training, the model learns the output structure - confident, detailed, cited - without the factual content. It fills the structural template with plausible-sounding fabrications because the loss penalizes wrong tokens more than it rewards honesty about uncertainty.
- **“You have a limited annotation budget of \$50,000. How do you allocate it across data paradigms for a 7B model SFT run?”** Strong answer: use model-generated data from a strong API model for breadth (500,000-1,000,000 examples at near-zero marginal cost), reserve the human budget for high-value borderline and safety examples (5,000-10,000 carefully reviewed pairs), and fill structured-task coverage gaps with aggregated NLP data.
- **Senior follow-up: “How would you detect and correct for length bias in your SFT evaluation pipeline?”** Senior answer: disaggregate win rates by output-length bracket; run a reward model trained explicitly to be

length-neutral; compare chatbot-arena rankings against benchmark rankings and investigate divergences; introduce length-controlled paired comparisons where both responses are truncated to the same token count before human or model rating.

Key Takeaways

- Three paradigms exist - aggregated NLP, crowdsourced human-written, and model-generated - each trading cost, naturalness, and scalability differently; modern pipelines blend all three.
- SFT reliably shapes output format and style; teaching new factual knowledge requires that knowledge to already exist in pre-training.
- Length bias inflates both human and LM-judge preference scores by 60-70%; always cross-validate against capability benchmarks that cannot be gamed by longer outputs.
- Mid-training (mixing SFT data into the learning-rate decay phase) is the current state-of-practice at scale and largely avoids catastrophic forgetting.
- Crowdsourcing quality degrades sharply under time pressure; incentive design and batch auditing are as important as vendor selection.

26.2.2 Safety Tuning and Refusal Trade-Offs

Every LLM that ships to users has to answer a practical question: what does the model do when someone asks it to do something it should not do? The naive implementation is to collect harmful prompts and train the model to output refusals. The problem is that a model trained only on refusals develops the wrong decision boundary - it pattern-matches on surface features like the word “kill” and refuses “how do I kill a Python process” alongside genuinely harmful requests. This failure mode is as important to understand as hallucination, and it shows up in both early-stage design questions (“what is safety tuning?”) and senior-level architecture discussions (“build me the safety data pipeline for a production model”).

Safety tuning is a targeted SFT procedure that teaches the model two distinct behaviors: when to refuse, and how to do so gracefully without lecturing or over-explaining. The training data comes in two structurally critical categories.

Harmful examples are (prompt, refusal) pairs where the prompt requests something clearly off-limits: detailed instructions for generating scam content, synthesis routes for dangerous materials, targeted harassment of a specific individual. The completion is a polite, brief refusal that acknowledges the request type without restating it in detail.

Borderline-positive examples are (prompt, compliant response) pairs where the prompt superficially resembles a harmful request but is entirely legitimate: a security researcher asking about exploit mechanics, a physician asking about lethal medication doses for treatment planning, a sysadmin asking how to terminate a process. These examples define the decision boundary from the other side. Without them, the model’s refusal behavior drifts toward surface features of the harmful examples rather than their semantic intent.

The informal but important principle is: for any vocabulary cluster V that appears prominently in harmful prompts, the training data needs at least as many examples of legitimate uses of V as harmful ones, or the model learns $V \Rightarrow \text{refuse}$ rather than $\text{harmful_intent}(V) \Rightarrow \text{refuse}$.

A safety-tuned model that over-refuses is not safe - it is broken in a different direction, and the product experience consequences are just as real.

The finding that consistently surprises engineers new to post-training is how little data safety tuning needs. Research across multiple groups has shown that even 500-1,000 safety examples mixed into a standard SFT run produce a measurably safer model without meaningfully degrading capability benchmarks. The reason is that the pre-trained model already encodes a strong representation of harm, danger, and ethics from the pre-training corpus. Safety tuning does not teach the model what harm is - it teaches the model to act on that representation by refusing rather than complying.

This is the same leverage principle that makes SFT powerful in general: capabilities are baked in during pre-training; post-training steers the output mode. The corollary is equally important: safety tuning cannot fundamentally fix a model that has deeply harmful behaviors embedded in its pre-training. It can redirect surface outputs, not reorganize deep representations.

The over-refusal failure mode is symmetric to hallucination. In both cases the model defaults to a learned surface pattern rather than grounding its response in the actual prompt content. Hallucination fills the response template with invented facts; over-refusal fills the response template with a refusal pattern triggered by surface keywords. Both are the same underlying problem: the model learned a shortcut that satisfies the type signature of the training data without performing the intended reasoning.

A Worked Example

Suppose you are fine-tuning a 7B parameter model on a 50,000-example general SFT corpus. You have a budget for 1,000 additional safety examples. Consider two allocation strategies.

Strategy A uses all 1,000 as harmful-refuse pairs, covering topics such as fraud, dangerous synthesis, and harassment.

Strategy B splits the budget: 500 harmful-refuse pairs plus 500 borderline-positive pairs using the same vocabulary in legitimate contexts (technical jargon, medical queries, security research framing).

Evaluate on three metrics before and after each strategy:

1. **Refusal recall:** fraction of genuinely harmful prompts that receive a refusal. Target: above 95%.
2. **Refusal false-positive rate:** fraction of benign domain-specific prompts that receive a spurious refusal. Target: below 3%.
3. **MMLU accuracy:** should not degrade more than 0.5 absolute points from baseline SFT.

With Strategy A a typical 7B model achieves refusal recall of approximately 97% but a false-positive rate of 12-18%: the model learned to refuse any prompt containing vocabulary associated with harmful categories. With Strategy B recall stays at 96-97% and the false-positive rate falls to 2-4%. MMLU degrades by less than 0.3 points in both cases. The composition of the borderline-positive set is what drives the improvement: examples must share the same surface vocabulary as the harmful set but carry clearly legitimate framing, forcing the model to attend to framing rather than vocabulary alone.

What You Actually Decide in Practice

- **Start at 1-2% safety data in the SFT mix.** If your overall SFT corpus is 500,000 examples, start with 5,000-10,000 safety examples. Going above 5% of the total mix begins to noticeably degrade helpfulness and general-capability benchmarks without proportional safety gains.
- **Match borderline-positive examples to harmful examples topic by topic.** For every category of harmful prompts you cover - synthesis instructions, fraud scripts, targeted harassment - write an equal number

of borderline-positive examples that use the same surface vocabulary in a legitimate context. Without this pairing, your model will over-refuse entire topic areas rather than specific intents.

- **Refuse at the intent level, not the topic level.** A topic-level rule such as “always refuse chemistry questions” will break legitimate medical and research use cases and will visibly frustrate users. Train the model to identify malicious intent within a topic, not the topic itself. This requires examples that make the distinction explicit, often with a brief explanation of why the request is complied with.
- **Track refusal recall and false-positive rate on separate dashboard rows.** An aggregate safety score that combines recall and precision hides over-refusal regressions behind high recall numbers. A model that refuses 99% of harmful prompts but also refuses 15% of benign technical queries is not a good model. Both numbers need to be visible and gated on.
- **Safety properties degrade with continued post-training.** Every subsequent SFT or RLHF round can erode safety behavior if safety data is not re-included in proportion. Schedule red-team evaluations after every major post-training update, not only at initial release. Safety is not a one-time certification.

How This Shows Up in Interviews

- **“How do you build safety tuning data and what is the most common mistake?”** Strong answer: collect both harmful-refuse pairs and borderline-positive pairs in roughly equal numbers. The most common mistake is collecting only harmful examples, which teaches the model to pattern-match on surface vocabulary rather than infer intent, causing high false-positive refusal rates on legitimate technical or domain queries.
- **“A user reports that the model refuses ‘how do I kill a Python process.’ Walk me through diagnosing and fixing this.”** Strong answer: (1) confirm the issue is reproducible on a held-out eval set; (2) inspect the safety training data for over-representation of the vocabulary cluster around “kill” in harmful-refuse pairs; (3) add borderline-positive examples using the same technical vocabulary in clearly legitimate contexts; (4) re-evaluate refusal recall to confirm the fix does not degrade safety recall; (5) add the repro case to the false-positive eval set so regression is detected automatically in future runs.
- **Senior follow-up: “Design an eval suite that simultaneously detects regressions in safety and helpfulness after any data update.”** Senior answer: maintain a fixed red-team set of approximately 500 harmful prompts measuring recall and a fixed borderline set of approximately 500 benign-but-surface-risky prompts measuring false-positive rate; gate model releases on both passing defined thresholds; include MMLU and a chat-style benchmark to catch capability regressions; track all four metrics on a single release dashboard so no individual regression is absorbed by an aggregate score passing.

Key Takeaways

- Even 500-1,000 safety examples produce measurable impact; the pre-trained model already encodes harm representations - safety SFT steers the output mode, not the underlying knowledge.
- Borderline-positive examples are as important as harmful examples; omitting them causes topic-level over-refusal rather than intent-level refusal.
- Track refusal recall and refusal false-positive rate independently; aggregate safety scores hide over-refusal regressions behind high recall numbers.
- Keep safety data at 1-2% of the SFT mix; above 5% measurably harms helpfulness without proportional safety gains.

- Safety behavior degrades with continued post-training; schedule red-team evaluations after every major update, not only at initial release.

Chapter 27 Preference-Based Alignment (RLHF)

27.1 The RLHF Pipeline

27.1.1 Pairwise Preference Collection

You are asked in a system design interview to improve a model that has already been fine-tuned on expert demonstrations. The interviewer wants to know what comes next. The canonical answer is reinforcement learning from human feedback-and the first question you will face is where the feedback comes from. Pairwise preference collection is the annotation backbone of the entire RLHF pipeline: the mechanism that converts vague human preferences into a learnable scalar signal. This topic appears consistently at senior ML engineer and research scientist interviews at any lab running a post-training pipeline, and the ability to reason about annotation quality, cost trade-offs, and systematic biases separates a strong answer from a shallow one.

The fundamental shift from supervised fine-tuning to RLHF is a shift in what you ask humans to do. In SFT you ask: *write the ideal completion*. In RLHF you ask: *which of these two completions is better?* The first question demands generation; the second demands only discrimination. Humans are systematically better at the second task-they can reliably judge quality even when they cannot produce it themselves under time pressure or without deep domain expertise.

The standard setup is straightforward. Given a prompt x , the current model generates $k \geq 2$ candidate responses $\{y_1, \dots, y_k\}$. An annotator reviews pairs (y_A, y_B) and marks a preference: A is better, B is better, or roughly equal. Each labeled pair becomes training signal for a reward model. The InstructGPT annotation guidelines operationalized this with three axes to judge: helpfulness, truthfulness, and harmlessness.

The optimization objective that consumes these preferences is:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \text{KL}[\pi_\theta(y|x) \parallel \pi_{\text{SFT}}(y|x)]$$

The first term maximizes the learned reward; the KL penalty keeps the RL policy from drifting too far from the SFT initialization, suppressing reward hacking and preserving fluent language generation. In practice β sits in the range 0.01-0.1; larger values produce more conservative, SFT-like outputs.

Pairwise feedback exploits the generator-validator gap: humans validate far more reliably than they generate, and RLHF collects supervision by asking the easier of the two questions.

SFT has a structural bottleneck: demonstration quality is bounded by what annotators can produce under real-world constraints. When you ask a crowd to write completions for a nuanced question about Bayesian inference or macroeconomic policy, you get short, hedged, often incomplete responses-because producing expert-level prose in five minutes under pay-per-task conditions is genuinely hard. You are paying for the full cognitive cost of generation.

Pairwise comparison costs a fraction of that. The same annotator who would spend fifteen minutes writing a mediocre demonstration can evaluate two strong model-generated responses in two to three minutes and give you a reliable ranking. Empirically, inter-annotator agreement on pairwise helpfulness judgments runs around 70-80%, which is comparable to the agreement you get from a strong AI judge. That signal-cheaper to collect and often more consistent than demonstrations on the same budget-is what reward model training runs on.

The trade-off is information density. A demonstration gives you a full token-level trajectory; a pairwise comparison gives you one bit of relative signal. You compensate with volume: InstructGPT trained its reward

model on roughly 50,000 pairwise comparisons while using only about 13,000 SFT demonstrations. Because the model itself generates both responses, on-policy rollouts produce new pairs at near-zero marginal cost once the infrastructure is in place.

A Worked Example

Suppose you are building an RLHF annotation pipeline for a 7B chat model and you have a \$50,000 annotation budget. Expert annotators cost roughly \$25 per hour.

Option A-SFT demonstrations: a high-quality multi-paragraph demonstration takes approximately twenty minutes to write, costing about \$8.33 per example. Your budget yields roughly 6,000 demonstrations.

Option B-pairwise feedback: judging a pair takes approximately five minutes, costing about \$2.08 per comparison. The same budget yields roughly 24,000 comparisons. The model itself generates both responses, so annotation cost does not scale with response length.

The quality picture is even starker. In my experience watching large-scale annotation projects, speed constraints destroy signal quality for anything requiring verification. Reports from a production assistant’s annotation guidelines described a target rate of approximately one judgment per minute per question. At that pace an annotator must read two responses of several hundred words each, compare them on helpfulness and safety, and check the relevant box-leaving no time to verify embedded citations or check arithmetic. Studies of crowdworker behavior under speed pressure confirm what you would expect: annotators correlate strongly with formatting signals (bullet-point density, paragraph count, word count) and weakly with factual accuracy.

Consider the summarization case: expert writers asked to compare their own summaries to model-generated ones showed a consistent preference for the AI outputs. One explanation collected in a follow-up interview: “When I wrote, I felt pressure to use flowery language. The AI ones just read better.” Generation and evaluation are different cognitive tasks, and people are often better at the latter-even when judging their own work.

What You Actually Decide in Practice

- **Human vs. AI feedback.** GPT-4-class models agree with human consensus at roughly 70-75% on capability judgments-on par with human inter-annotator agreement-at orders of magnitude lower cost. For most helpfulness and instruction-following axes, AI feedback is the production default today. Reserve human annotation for high-stakes safety and factual domains where a rubric cannot fully capture the judgment required.
- **Control for length bias explicitly.** Both humans and AI judges prefer longer responses at a 60-70% base rate, independent of quality. If you ignore this, your reward model learns to produce verbose outputs rather than correct ones. Fix it by including length-controlled pairs in your annotation set, by adding an explicit conciseness criterion to the rubric, or by normalizing response length during reward model training.
- **On-policy data is more valuable than off-policy data for reward model training.** On-policy pairs-where both responses come from the model you are currently optimizing-anchor the reward model to the policy distribution you actually care about. Off-policy data from unrelated models adds distributional coverage early in training but becomes stale quickly as your model improves. Refreshing pairwise datasets every major training iteration is the standard pattern at labs running continuous post-training.
- **Annotator demographics shape model values.** RLHF sits at the end of the pipeline and has outsized influence on surface behavior. When the annotator pool skews toward a particular nationality or cultural background, the resulting model shows measurably stronger alignment with that group’s norms-detectable

in controlled probing studies. Audit annotator composition before deployment, especially on topics where cultural context affects what counts as helpful or harmful.

- **AI self-preference is a real confound in evals.** When you use a model to judge its own outputs versus a competitor's, it exhibits strong self-preference at rates that do not correspond to human rankings. Always use a judge model that is distinct from the model under evaluation; otherwise your win-rate numbers are systematically inflated.

How This Shows Up in Interviews

- **“Why does RLHF outperform SFT for instruction following at scale?”** Strong answer: SFT is bounded by what annotators can generate under time and cost pressure; pairwise preference collection exploits the generator-validator gap, extracting more usable signal per annotation dollar. The KL divergence penalty in the training objective keeps the RL policy grounded in the SFT distribution, preventing the degenerate outputs that unconstrained reward maximization produces.
- **“What are the main failure modes of pairwise preference collection?”** Strong answer: length bias (60-70% preference for longer responses regardless of quality), inability to verify facts under speed pressure, AI judge self-preference bias, and annotator demographic skew that embeds cultural assumptions into the reward model.
- **Senior follow-up: “How do you detect whether your reward model has learned real quality or just superficial style?”** Strong answer: hold out a set of pairwise examples where ground truth is independently verifiable-math problems with known answers, code with executable test suites, factual QA with reference sources. Measure reward model ranking accuracy on this set. If RM scores correlate more strongly with response length than with ground-truth correctness, the model has overfit to style. Fix it by augmenting the pairwise training set with verified correct-versus-incorrect pairs, adding explicit accuracy and conciseness dimensions to the annotation rubric, and-where possible-using verifiable rewards (correct math, passing tests) to anchor the preference signal.

Key Takeaways

- Pairwise feedback exploits the generator-validator gap: judging two responses requires a fraction of the time and expertise needed to generate a high-quality demonstration from scratch, which is why the same annotation budget buys roughly four times as many pairwise comparisons as SFT examples.
- The InstructGPT training objective couples reward maximization with a KL penalty against the SFT model, keeping the RL policy from drifting into degenerate, reward-hacking outputs.
- AI judges at the GPT-4 class match human inter-annotator agreement at roughly 70-75% on most capability axes and are the production default at scale today; human annotation is reserved for high-stakes safety and factual verification tasks.
- Length bias is a systematic confound: both humans and AI judges show a 60-70% base preference for longer responses independent of quality; failing to control for it trains a reward model that optimizes verbosity.
- On-policy pairwise data-pairs generated by the model currently under optimization-is more valuable than off-policy data and should be refreshed regularly as the model improves.

27.1.2 Reward Modeling and the Bradley-Terry Assumption

You have collected tens of thousands of pairwise preference labels and you need to turn them into a signal that a neural network can optimize against. The interviewer asks: “How does a collection of binary A-vs-B comparisons become a reward function?” The answer is a probabilistic assumption called the Bradley-Terry model and a reward model trained by maximum likelihood on that assumption. This topic comes up at MLE and research scientist interviews whenever RLHF is on the table, and the depth of your answer-whether you can derive the loss from first principles, name its failure modes, and explain when you would skip the reward model entirely-is what separates candidates at the senior and staff levels.

The Bradley-Terry model posits a latent, scalar-valued ground truth: every possible response y to a prompt x has a true scalar reward $r(x, y) \in \mathbb{R}$. We never observe r directly. What we observe is noisy pairwise comparisons generated according to:

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

where σ is the logistic sigmoid, y_w is the preferred (“winner”) response, and y_l is the less preferred (“loser”) response. The probability of preferring y_w is a logistic function of the *difference* in their latent rewards. If $r(x, y_w)$ exceeds $r(x, y_l)$ by a large margin, the preference is nearly deterministic; if the two rewards are equal, the annotator is effectively flipping a coin.

Given a dataset $\mathcal{D} = \{(x_i, y_w^i, y_l^i)\}_{i=1}^N$ of pairwise labels, training a reward model r_ϕ means maximizing the log-likelihood of the observed preferences under this model. The negative log-likelihood loss is:

$$\mathcal{L}(\phi) = -\frac{1}{N} \sum_{i=1}^N \log \sigma(r_\phi(x_i, y_w^i) - r_\phi(x_i, y_l^i))$$

Minimizing $\mathcal{L}$ pushes $r_\phi(x, y_w)$ above $r_\phi(x, y_l)$ for every observed preferred pair-exactly the ranking we want.

In practice, r_ϕ is implemented by taking a pre-trained language model, discarding the vocabulary projection head, and adding a single linear layer that maps the final hidden state of the last token to a scalar. The resulting network reads the concatenation of prompt and response and outputs one number: the estimated quality of that response.

The reward model is the bridge between human preference data and an optimization landscape: it converts a discrete ranking over pairs into a continuous, differentiable signal that the RL training loop can ascend.

The Bradley-Terry assumption is the minimal probabilistic model that is consistent with pairwise preference data. It has one free parameter per sequence-the latent reward-and generates binary comparisons via a logistic noise model. That simplicity is a feature: the loss function is convex in the reward differences and well-understood, and the model generalizes beyond the observed pairs because r_ϕ is parameterized by a neural network that shares weights across all prompts and responses.

The key advantage over alternatives is coverage. If you trained only on SFT demonstrations you would have no way to score outputs the model generates at RL time-the reward model fills that gap, acting as a proxy judge for any (x, y) pair the policy might produce, including ones never seen during data collection.

The critical trade-off is that the reward model is a proxy, not the true objective. It is trained on a finite sample of human comparisons, which means it has blind spots: regions of output space where it assigns high reward to responses that humans would reject. The standard mitigation is the KL penalty in the RL objective, which keeps the policy close enough to the SFT distribution that the reward model’s estimates remain reliable.

When the policy strays too far from that distribution—a phenomenon called *reward overoptimization*—RM scores continue rising while actual human preference scores plateau and eventually fall.

A Worked Example

Consider training a reward model for a 7B chat model. You have 50,000 pairwise comparisons collected on 10,000 distinct prompts (roughly five comparisons per prompt).

Architecture. Start from the 7B SFT checkpoint. Replace the final unembedding matrix (vocabulary dimension $\sim 32,000$) with a single linear layer mapping from the model’s hidden dimension ($d = 4,096$) to a scalar. Total added parameters: 4,096. The remaining 7B parameters are fine-tuned on the reward modeling loss.

Batch construction. Each training example is a triplet (x, y_w, y_l) . Both y_w and y_l are concatenated with x and forward-passed through the same network in a single batch. You read off the scalar at the last token position for each, then compute $\sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$ and take the log. A batch of 64 pairs requires two forward passes of the 7B model, so effective batch size for memory purposes is 128 sequences.

Numbers. With 50,000 pairs, a learning rate of 1×10^{-5} , and 1-2 epochs of training on an H100 (roughly 80 GB HBM), reward model training typically completes in a few hours. The resulting RM achieves human preference accuracy (does the RM agree with the human annotator on held-out pairs?) of around 65-72% on general helpfulness tasks and lower on specialized factual domains—which is the quantitative signal you use to decide whether the RM is good enough to drive RL training.

Calibration check. Evaluate RM score versus ground-truth preference on a held-out factual QA set where correct answers are known. If Spearman correlation between RM score and factual accuracy is below 0.5, the RM is primarily scoring style and the RL run will optimize verbosity rather than correctness.

What You Actually Decide in Practice

- **Reward model size relative to policy size.** The RM does not need to be as large as the policy. In production pipelines I have seen 3B RMs driving 70B RL runs effectively. A smaller RM trains faster and is easier to refresh as new preference data arrives. The floor is that the RM must be large enough to understand the semantic content it is scoring; going below roughly one quarter of the policy size tends to hurt quality noticeably.
- **Margin-based loss variants.** The standard Bradley-Terry loss treats all pairs equally regardless of how clear the human preference was. Many teams add an annotation confidence score and train with a margin term:

$$\mathcal{L} = -\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l) - m)$$

where $m > 0$ is a margin that forces the RM to score winning responses by at least m above losing ones. This sharpens the decision boundary and reduces reward hacking at the cost of slightly lower recall on ambiguous pairs.

- **When to skip the explicit reward model (DPO).** If your dataset is fixed and you do not need to run on-policy rollouts, Direct Preference Optimization eliminates the RM entirely by reparameterizing the reward in terms of the policy’s own log-probability ratio. DPO is simpler to implement and avoids RM training costs. Use it when your pairwise data is off-policy and relatively small ($\lesssim 100K$ pairs). Use an explicit RM when you need on-policy rollouts at RL time—the RM must score live outputs that do not exist

in any static dataset.

- **Distribution shift is the main failure mode.** The RM is calibrated on responses similar to those in the pairwise dataset. As RL training proceeds, the policy shifts and its outputs diverge from the RM’s training distribution. Monitor RM score versus human evaluation score on a rolling held-out set throughout RL training; if they diverge, either stop training, refresh the RM with on-policy preference data, or tighten the KL coefficient β .
- **Ensemble or leave-one-out calibration for high-stakes deployment.** A single RM can overfit to annotation artifacts. Training two or three RMs from different random seeds and using their minimum score (rather than mean) as the optimization target is a practical hedge against reward overoptimization in production settings where you cannot run frequent human evals.

How This Shows Up in Interviews

- **“Walk me through how you train a reward model from pairwise preferences.”** Strong answer: start with the Bradley-Terry model—each response has a latent scalar reward, preferences are logistic in the reward difference—then write the negative log-likelihood loss over preferred and rejected pairs. Implement as a pretrained LM with the vocab head replaced by a scalar linear layer. Train with learning rate around 10^{-5} for 1-2 epochs; evaluate with held-out pairwise accuracy.
- **“What can go wrong with reward model training and how do you detect it?”** Strong answer: the two main failure modes are length bias (RM scores longer responses higher regardless of quality) and distribution shift during RL (RM extrapolates incorrectly to outputs it was not trained on). Detect length bias by measuring correlation of RM scores with response length on a neutral prompt set; detect distribution shift by tracking the gap between RM score and human eval score as RL training progresses.
- **Senior follow-up: “When would you choose DPO over training an explicit reward model?”** Strong answer: DPO reparameterizes the reward in terms of the policy’s own log-ratio, eliminating the RM and the RL training loop. It is the right choice when the pairwise dataset is static (you will not collect on-policy preferences), the policy is moderate in size (under $\sim 30\text{B}$), and you want a simpler pipeline. Use an explicit RM plus PPO when you need on-policy rollouts—live scoring of model-generated outputs that do not exist in any precomputed dataset—or when your RM needs to be shared across multiple policy variants.

Key Takeaways

- The Bradley-Terry model assumes every response has a latent scalar reward; pairwise preferences are logistic in the reward difference, giving a clean maximum-likelihood training objective: minimize $-\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$ over all preference pairs.
- The reward model is architecturally a pretrained LM with its vocabulary head replaced by a single scalar linear layer; it is trained on the same pairwise data and produces a continuous quality estimate for any (x, y) the policy might generate.
- The RM is a proxy, not the true objective: it generalizes poorly to outputs far from its training distribution, so the KL penalty in the RL objective is essential for keeping the policy in-distribution for the RM.
- The primary failure mode is reward overoptimization: RM scores rise while true human preference scores plateau or fall; monitor with a rolling held-out human eval throughout RL training and refresh the RM when divergence appears.
- When pairwise data is static and on-policy rollouts are not needed, DPO eliminates the explicit RM by

reparameterizing reward in terms of the policy’s own log-probability ratio, reducing the pipeline to a single supervised training run.

27.2 Optimization Algorithms

27.2.1 PPO for Language Models

You are asked in an interview to design the optimization loop that turns a reward model into a better language model policy. The moment you say “gradient descent on the reward,” the follow-up is: “what exactly is your loss, and why do you need a value network?” PPO is the answer InstructGPT gave in 2022, and frontier labs continued using variants of it long after DPO arrived. Understanding PPO concretely-not just as a named algorithm but as a sequence of decisions-is the minimum bar for an RLHF deep-dive at the staff-engineer level.

The goal is to find a policy π_θ that maximizes expected reward while staying close to the supervised fine-tuned reference model π_{ref} . The InstructGPT objective formalizes this as:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} [r_\phi(x, y)] - \beta D_{\text{KL}}(\pi_\theta(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x))$$

The KL term keeps the RL policy from drifting so far from π_{ref} that it produces fluent-sounding but reward-hacked outputs. The scalar β is a regularization coefficient; typical values range from 0.01 to 0.1.

Starting from the plain policy gradient, the gradient of $\mathbb{E}_{\pi_\theta} [r(z)]$ with respect to θ equals $\mathbb{E}_{\pi_\theta} [r(z) \nabla_\theta \log \pi_\theta(z)]$. This is *REINFORCE*: multiply the log-probability gradient by the reward and update accordingly. It is unbiased but high-variance. Two refinements turn it into PPO.

First, replace the raw reward with an *advantage* A_t -the reward minus a learned state-value baseline $V_\phi(s_t)$. Subtracting any baseline that does not depend on the current action leaves the gradient unbiased while dramatically reducing variance. In practice A_t is computed via Generalized Advantage Estimation (GAE) with hyperparameters λ and γ ; setting both to 1 collapses GAE to a simple return-minus-baseline form, which already works well.

Second, allow multiple gradient steps on a single set of rollouts using importance-weighted corrections. If the current policy is π_θ but samples came from the older $\pi_{\theta_{\text{old}}}$, the corrected objective becomes:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)], \quad \rho_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

The clip removes any incentive to push the probability ratio outside $[1 - \epsilon, 1 + \epsilon]$; a typical value is $\epsilon = 0.2$. This is the PPO-Clip objective-no KL constraint to tune, just a hard floor and ceiling on how much the policy can shift per update.

PPO is policy gradient with variance reduction via a value baseline, importance weighting for multi-step updates, and a clipping constraint that naturally keeps the new policy close to the old one.

The plain policy gradient fails at scale because its gradient variance grows with sequence length-a 512-token response has 512 token-level log-probabilities whose products create enormous variance. Subtracting the value baseline brings that variance down to something tractable without introducing bias.

The clipping mechanism does something subtle: it makes the optimization landscape less adversarial. Without clipping, a lucky update can send the policy into a region of high reward-model scores but low actual quality. Clipping hard-caps how far a single update can travel, which is especially important in the first few thousand steps when the reward model is most at risk of being exploited.

Compared to the alternative of using a hard KL constraint (TRPO), PPO is much simpler to implement and tune. TRPO requires a second-order computation to enforce the constraint; PPO replaces that with a clip and a step-size schedule. The performance tradeoff is minimal, which is why PPO absorbed TRPO's market share almost entirely after its 2017 release.

The cost is real, though: PPO for language models requires *four* live models—the policy, the reference policy, the reward model, and the value model. The value model is typically as large as the policy, so GPU memory roughly doubles compared to SFT.

A Worked Example

Consider a 7B parameter policy (e.g. Llama-3-7B) in bf16 on $8 \times$ H100 80 GB GPUs, with a 7B reward model and a 7B value model.

Memory budget. Each 7B model in bf16 uses roughly 14 GB for weights. With activations and optimizer state, a single model consumes around 60-80 GB during a forward-backward pass. Running policy + reference + reward + value simultaneously requires memory-efficient strategies: the reference and reward models are frozen (no gradients), so they can be kept in inference mode at ≈ 14 GB each. Total active memory: ~ 14 (ref) + 14 (reward) + 70 (value, with optimizer) + 70 (policy, with optimizer) ≈ 168 GB, which fits across 8×80 GB with tensor parallelism.

One PPO step, end to end.

1. **Rollout.** Sample $B = 64$ prompts from the dataset; run the policy autoregressively to generate responses of up to 512 tokens. This is the slow step: at 5,000 tokens/s on an H100 with vLLM, 64 sequences of 512 tokens takes ~ 6.5 seconds.
2. **Reward and value computation.** Feed each (prompt, response) pair through the reward model to get scalar $r(x, y)$. Feed each token through the value model to get $V_\phi(s_t)$ at every position.
3. **KL penalty.** Compute per-token KL: $\text{kl}_t = \log \pi_\theta(a_t|s_t) - \log \pi_{\text{ref}}(a_t|s_t)$. Add $-\beta \cdot \text{kl}_t$ to the terminal reward to build a per-token reward signal.
4. **GAE.** With $\gamma = 1$, $\lambda = 0.95$, compute advantages A_t backward from the last token.
5. **Policy update.** Run $K = 4$ mini-batch gradient steps on $\mathcal{L}^{\text{CLIP}}$ with $\epsilon = 0.2$. Monitor the mean ratio ρ_t ; if it drifts above 1.3 consistently, reduce learning rate or decrease K .
6. **Value update.** Run a parallel MSE loss on the value head to keep $V_\phi(s_t)$ calibrated.

In practice, 1,000 such PPO steps on 64K training prompts takes 6-8 hours on this hardware. Reward usually peaks and plateaus within the first 500 steps; continuing further without increasing data diversity leads to overoptimization.

What You Actually Decide in Practice

- **β , the KL coefficient.** Start at 0.04-0.1. If you see the KL divergence from the SFT model exceeding 10 nats after a few hundred steps, increase β . If reward barely moves, lower it. Some implementations anneal β during training.
- **Number of PPO epochs per rollout batch (K).** Common choices are 1-4. More epochs extract more signal per rollout, but stale importance weights cause the clip to activate more and gradients become increasingly biased. At $K = 1$ you are essentially doing online policy gradient without any staleness; $K = 4$ requires careful monitoring.

- **Value model initialization.** Always initialize the value model from the same checkpoint as the policy, not randomly. A poorly initialized value model produces high-variance advantages for the first thousands of steps and can destabilize training. Add a warm-up phase of pure value training on rollouts before joint optimization if instability appears.
- **Reward shaping: per-token vs. terminal.** The true task reward (“did the model answer correctly?”) is a terminal reward on the last token. The KL penalty is applied per token. Never add the terminal reward to every token—this creates a heavily biased gradient that overweights early tokens in the response. Assign terminal reward only at the EOS token position.
- **Memory vs. throughput tradeoff.** If GPU memory is the bottleneck, offload the frozen reference and reward models to CPU between rollout batches. This trades $\sim 30\%$ throughput for halving active GPU memory. Alternatively, quantize the reference and reward models to int8 (they are frozen, so QAT is unnecessary).

How This Shows Up in Interviews

- **“Walk me through the PPO loop for a language model.”**
Strong answer: describe the four models, the rollout step, reward computation, KL penalty construction, GAE advantage estimation, and the clipped surrogate loss. Name the clip range ϵ and the KL coefficient β with representative numbers. Many candidates skip the value model or the KL shaping entirely.
- **“Why can’t you just do gradient ascent directly on the reward model output?”**
Strong answer: reward models are finite-capacity proxies; direct gradient ascent exploits their blind spots (overoptimization). The KL term and the clip prevent the policy from moving too far into high-reward-model-score but low-true-quality regions. Without these constraints, you get reward hacking within hundreds of steps.
- **“What is the value model for in PPO, and can you remove it?”**
Strong answer: the value model provides a variance-reducing baseline for the policy gradient. Without it, gradient variance scales with sequence length and training is unstable. You can remove it, and GRPO does exactly this by replacing the value baseline with a group mean reward, at the cost of requiring multiple rollouts per prompt to compute that baseline.
- **Senior follow-up: “PPO for an LLM is actually a contextual bandit, not a full MDP. What changes?”**
In a contextual bandit, there are no intermediate state transitions: the prompt is the context, the full response is the action, and the reward arrives once. This means the credit assignment problem simplifies—you do not need multi-step Bellman backups. The consequence is that γ can safely be set to 1 without worrying about discounting future states, and the value function only needs to estimate the expected return of the entire completion rather than a trajectory. The KL penalty still matters as a regularizer, but it is applied token-by-token as a reward shaping term rather than as a constraint over state transitions. Recognizing the bandit structure is why GRPO can discard the value network entirely and use a within-group mean as a baseline without loss of correctness.

Key Takeaways

- PPO requires four live models during training: policy, reference (frozen), reward (frozen), and value. The value model is the primary memory and complexity cost; its role is variance reduction via advantage estimation.

- The clip range $\epsilon = 0.2$ prevents any single update from moving the policy too far from its previous state, replacing the second-order KL constraint of TRPO with a simpler and cheaper mechanism.
- In the language-model setting, the reward is a terminal scalar; the KL divergence from the reference policy is applied per token as a shaped per-step penalty, not as a hard constraint.
- Reward peaks and then plateaus or declines as the policy begins to exploit the reward model’s weaknesses; monitoring KL divergence and true human preference win-rates together is the only reliable way to catch overoptimization early.
- GRPO eliminates the value model by computing a group-mean baseline over multiple rollouts of the same prompt, making it the practical default when memory and implementation complexity are constraints.

27.2.2 DPO and Its Derivation

At some point in any RLHF system design discussion, the interviewer will ask why you need PPO at all. The correct answer is: you often do not. Direct Preference Optimization (DPO) removes the reward model, the value model, and the RL loop entirely, collapsing the RLHF objective into a supervised cross-entropy loss over preference pairs. It is the algorithm behind nearly every open-weight RLHF release between mid-2023 and late-2024, and understanding its derivation is what separates a candidate who has “used DPO” from one who actually understands it.

Start from the same KL-regularized reward objective as PPO:

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [r(x, y)] - \beta D_{\text{KL}}(\pi_{\theta}(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x))$$

The key insight in DPO is to ask: if we made no parametric assumptions and allowed π_{θ} to be any function, what would the optimal policy look like? Working through the calculus of variations, the closed-form optimum is:

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$ is a partition function. Solving for the reward gives:

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x)$$

The partition function $\log Z(x)$ depends only on x , not on y . Plug this expression for r into the Bradley-Terry preference probability $p(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$. The $\log Z(x)$ terms cancel, and the preference probability becomes:

$$p(y_w \succ y_l | x) = \sigma\left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}\right)$$

Maximizing the log-likelihood of the observed pairwise preferences yields the DPO loss:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma\left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)}\right) \right]$$

There is no reward model parameter to train, no value network, and no rollout loop. Both π_{θ} (the policy being trained) and π_{ref} (the frozen SFT checkpoint) are evaluated on the already-collected preferred and rejected completions, making every gradient step a standard forward-backward pass.

DPO turns the RL problem into supervised learning by showing that the optimal policy and the reward function are two parametrizations of the same object, so you can optimize preference likelihood directly without ever representing the reward explicitly.

PPO requires on-policy rollouts: you must sample from your current policy, evaluate those samples with the reward model, and update. DPO is fully offline: it consumes a static dataset of (prompt, chosen, rejected) triples. This eliminates the slowest part of RLHF-autoregressive generation at training time-and removes the need for a reward model and a value model, halving the GPU memory requirement compared to PPO.

The DPO gradient has an interpretable form. Let $\hat{r}_\theta(x, y) = \beta \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))$ be the implicit reward. The gradient upweights the chosen completion and downweights the rejected completion, with a weight equal to $\sigma(\hat{r}_\theta(y_l) - \hat{r}_\theta(y_w))$ -the probability that the model currently gets the pair wrong. When the model already ranks the pair correctly (weight near 0), the gradient is small; updates concentrate on hard examples. This is analogous to focal loss in classification.

The tradeoff vs. PPO is that DPO is offline: it never generates new completions to explore the reward landscape. If the reference policy π_{ref} never produced diverse enough completions during dataset construction, DPO cannot escape that blind spot. Online variants (iterative DPO, RLHF-Flow) address this by periodically regenerating the rejected set from the current policy, recovering much of PPO's on-policy advantage while keeping the simpler loss.

A Worked Example

Setup: a 7B model (Mistral-7B-v0.1) fine-tuned with SFT to produce π_{ref} , now trained with DPO on the UltraFeedback dataset (60K preference pairs), in bf16 on $4 \times \text{A100 } 80 \text{ GB GPUs}$.

Memory. The policy π_θ requires $\sim 70 \text{ GB}$ with optimizer states (AdamW + gradient buffers). The reference π_{ref} is frozen and kept in bf16 inference mode: $\sim 14 \text{ GB}$. Total: $\sim 84 \text{ GB}$ across $4 \times 80 \text{ GB GPUs}$ with no tensor parallelism needed-compare to PPO's $> 160 \text{ GB}$.

One DPO gradient step.

Assume a mini-batch of 4 triples $(x_i, y_{w,i}, y_{l,i})$ with max sequence length 512.

1. Forward pass π_θ on all 8 sequences (y_w and y_l concatenated with their prompts): compute token log-probabilities, sum over response tokens only to get $\log \pi_\theta(y_w|x)$ and $\log \pi_\theta(y_l|x)$.
2. Forward pass π_{ref} on the same 8 sequences (no gradient): get $\log \pi_{\text{ref}}(y_w|x)$ and $\log \pi_{\text{ref}}(y_l|x)$.
3. Compute implicit rewards: $\hat{r}_w = \beta(\log \pi_\theta(y_w|x) - \log \pi_{\text{ref}}(y_w|x))$, similarly $\hat{r}_l$.
4. Loss: $\mathcal{L} = -\log \sigma(\hat{r}_w - \hat{r}_l)$, average over the batch.
5. Backward on $\mathcal{L}$; update π_θ only (ref is frozen).

With $\beta = 0.1$ and a well-initialized SFT model, $\hat{r}_w - \hat{r}_l$ starts near 0 (the model does not yet distinguish the pairs) and should reach $\sim 3\text{-}5$ nats on average after a full epoch. At a batch size of 32 and 60K examples, one epoch is 1,875 steps; a full DPO run on $4 \times \text{A100}$ takes roughly 2-3 hours.

Implicit reward check. After training, verify that $\beta \log(\pi_\theta(y_w|x)/\pi_{\text{ref}}(y_w|x)) > \beta \log(\pi_\theta(y_l|x)/\pi_{\text{ref}}(y_l|x))$ on a held-out preference set. Accuracy below 65% on this proxy metric is a strong signal that β is too large (over-regularizing) or the SFT checkpoint was too weak to start.

What You Actually Decide in Practice

- **β , the temperature parameter.** This controls how tightly the policy stays near π_{ref} . Values in $[0.05, 0.5]$ are common; start at 0.1. Too small: the policy drifts far from the reference and can produce degenerate

outputs. Too large: effectively no change from SFT. Monitor KL divergence from π_{ref} on a validation set; target 5-15 nats after training.

- **Length normalization.** Sum log-probabilities over response tokens without normalization biases the loss toward shorter sequences, which receive higher per-token log-probability on average. SimPO and length-normalized DPO divide the log-probability sum by sequence length before computing the reward margin. This simple fix measurably reduces the model’s preference for very short responses; use it by default.
- **Offline vs. iterative.** Static offline DPO is fast but brittle when the reject set is low-quality or the SFT model is strong. If your rejected completions are all from an early, weak model, the policy learns to beat a low bar and does not generalize. Regenerate the rejected set from your current policy every 1-2 epochs (online DPO) to keep the training signal challenging and on-distribution.
- **Chosen completion collapse.** A silent failure mode: as the policy is pushed away from the rejected completion, the log-probability of the *chosen* completion can also decrease. Track $\log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x)$ separately during training; if it goes negative and keeps falling, the model is degrading on the positive examples, not just improving on the negative ones. Reduce learning rate or add a supervised term on chosen completions.
- **Data quality beats data quantity.** On 7B-scale models, 10K high-quality human preference pairs (from annotators with explicit factuality checks) consistently outperform 200K AI-generated pairs in downstream benchmarks. The DPO loss is sensitive to label noise: a single flipped pair in a batch contributes a gradient in the wrong direction with full weight.

How This Shows Up in Interviews

- **“Derive DPO from first principles.”**

Strong answer: state the KL-regularized RL objective, take the non-parametric optimum, solve for the implied reward, substitute into Bradley-Terry, and show that $Z(x)$ cancels. Many candidates can name DPO; very few can reproduce the derivation under pressure. Practice the three-step: optimal policy form $\rightarrow$ implied reward $\rightarrow$ preference loss.

- **“When would you use PPO instead of DPO?”**

Strong answer: when rewards are verifiable (math, code), when you need on-policy exploration to escape the static offline data distribution, or when the preference dataset is too small/noisy to provide stable gradients. PPO’s on-policy nature lets it discover high-reward completions that were never in the original training set. DPO is bounded by the quality of the collected pairs.

- **“How is DPO related to contrastive learning?”**

Strong answer: the DPO loss increases the log-likelihood of chosen completions relative to rejected ones, weighted by prediction error—closely analogous to InfoNCE / noise-contrastive estimation. The difference is that DPO’s “negative” is a specific rejected completion from a preference dataset, not a random negative sample.

- **Senior follow-up: “DPO has no explicit reward model. Where does the reward model effectively live?”**

It lives implicitly in the log-ratio $\beta \log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))$. This ratio is the implied reward for any completion under the optimal DPO policy. In practice, you can extract this scalar at inference time to rank completions just as you would with an explicit reward model—the difference is that this reward was never trained with a classification head, so it is not calibrated in absolute terms but is well-ordered within a

prompt. This is useful for best-of- n reranking and for diagnosing what the DPO model actually learned to prefer.

Key Takeaways

- DPO derives from the same KL-regularized objective as PPO but avoids rollouts, reward models, and value networks by showing that the optimal policy and the reward are interchangeable representations of the same underlying quantity.
- The DPO loss is a binary cross-entropy over preference pairs where the logit is the margin between implicit rewards of chosen and rejected completions; β controls how far the policy can drift from the SFT reference.
- A critical failure mode is chosen-completion collapse: track the log-ratio for chosen completions separately; if it falls below zero and keeps falling, the policy is deteriorating on positive examples.
- Offline DPO is bounded by the quality of the static preference dataset; iterative or online DPO (regenerating rejected samples from the current policy) recovers much of PPO’s benefit at a fraction of the implementation cost.
- Length normalization (dividing log-probabilities by sequence length before computing the reward margin) eliminates the length bias that favors short completions and should be applied by default.

27.3 Pitfalls of RLHF

27.3.1 Reward Over-Optimization

In any RLHF deep-dive, you will eventually be asked: “what goes wrong when you optimize too hard against the reward model?” Reward over-optimization is the canonical failure mode - the wall every production RLHF system eventually hits. Understanding it separates engineers who can implement the algorithm from those who can debug a live system that started producing worse outputs even though the proxy reward kept climbing. At senior and staff level, you are expected to know not just that this happens, but why it happens mechanically and how to detect it before it reaches users.

Reward over-optimization occurs when the RL policy gets so good at maximizing the *proxy reward* - the learned reward model r_ϕ - that it drifts away from what r_ϕ was actually supposed to measure: true human preference. The reward model is a finite-sample estimator of human preferences trained on a fixed annotation set. Like any learned model, it has gaps between its training distribution and the true underlying oracle. As optimization pressure increases, the policy finds and exploits those gaps.

The framing I find most useful is a train-test gap in RL clothing. The reward model is your training signal; true human preference is your test set. The RL algorithm overfits the former, and you pay the cost on the latter. Formally, let r^* denote the true human preference oracle and r_ϕ the fitted proxy. The PPO training objective maximizes:

$$\mathbb{E}_{y \sim \pi_\theta(\cdot|x)}[r_\phi(x, y)] - \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$$

but what we actually care about is $\mathbb{E}[r^*(x, y)]$. These two quantities correlate when π_θ stays close to the distribution r_ϕ was trained on, and diverge as π_θ moves away. The KL penalty keeps the policy near π_{ref} , but it is a soft constraint - once the policy gradient finds directions that increase r_ϕ while partially circumventing D_{KL} , it will exploit them.

Empirically, this pattern is remarkably consistent: proxy reward climbs monotonically with RL steps while

true win-rate first increases, peaks, and then falls. Studies confirm this for human feedback and noisy AI feedback. It does not appear for clean, noiseless reward signals - pointing directly at label noise and finite-sample estimation as the root causes, not RL per se.

A reward model is a proxy, not an oracle; every RL algorithm will eventually find the gap between them.

The mechanism is Goodhart’s Law instantiated in RL: when the proxy becomes the optimization target, it ceases to be a reliable proxy. Human preference annotations are noisy in two compounding ways. First, individual annotators disagree; a pair that one rates “clearly response A” another rates “slightly response B.” Second, the preference data is collected on a distribution of prompts and responses that the current policy never generated, which means the reward model is queried increasingly far outside its training distribution as optimization proceeds. A relentless RL optimizer will find exactly those out-of-distribution regions.

The alternative - optimizing directly against live human ratings - is logistically infeasible at training speed. You cannot collect fresh human feedback fast enough for gradient updates at typical RL throughput. DPO sidesteps reward-model fitting by directly optimizing the policy against a static preference dataset, which reduces proxy-exploitation risk at the cost of being offline. Iterating the preference-collection loop partially recovers freshness but adds annotation latency and engineering complexity. Neither approach eliminates the fundamental finite-sample problem; they only slow the divergence.

A Worked Example

You are running PPO with a reward model r_ϕ trained on 100K pairwise annotations for a 7B chat model. Your SFT baseline achieves 50% win-rate against itself (the reference point). After 500 RL steps, proxy reward has risen 0.4 normalized units and human win-rate is 62% - a solid alignment gain worth shipping.

You continue to 2,000 steps. Proxy reward has now risen 1.1 units. But the next human evaluation returns a win-rate of 53%. What happened?

Inspect a sample of rollouts at step 2,000. The policy has learned three consistent patterns the reward model scores highly but humans do not: (1) responses are $2.1\times$ longer on average; (2) nearly every response opens with “Certainly! Here is a comprehensive answer to your question”; (3) factual claims are decorated with hedging phrases that sound authoritative but add no information. These stylistic patterns were rare in the reward model’s training corpus - it never learned to penalize them - but after 2,000 steps of optimization they appear in over 80% of rollouts. The reward model confidently assigns them high scores. Human raters find them grating. Proxy and reality have diverged.

The diagnosis is clean: the proxy reward at step 2,000 measures “looks like the training distribution of good responses” rather than “is a good response.” The RL algorithm found a different attractor.

What You Actually Decide in Practice

- **Always monitor two signals simultaneously.** Track both proxy reward and an independent evaluation signal: a held-out reward model trained on disjoint data, an automated capability benchmark, or scheduled human spot-checks. When they diverge, stop. Do not wait for the next planned evaluation checkpoint.
- **Set a KL budget, not just a learning rate.** A KL penalty β in the range 0.01-0.05 is typical for 7-70B models; larger β reduces over-optimization risk but weakens the RL signal. If the mean KL from the reference policy climbs past 10-15 nats, you are likely in dangerous territory regardless of β .
- **Collect fresh reward data iteratively.** If annotation throughput allows, retrain the reward model on samples from the current policy every N RL steps. This shrinks the distribution gap at each step. The

cost is engineering complexity; the benefit is a slower divergence curve.

- **Do not port “optimal RL step” numbers between projects.** The over-optimization inflection point depends on reward model quality, annotation noisiness, and base model capability. Typical inflection ranges I have seen are 500-3,000 steps for instruction-tuned 7-70B models, but these numbers are not transferable across reward model quality tiers or annotation protocols.
- **Verifiable rewards are the cleanest cure.** Over-optimization is essentially absent when rewards are noise-free - unit tests for code correctness, exact match for structured math answers. This is the core motivation for the RLVR paradigm: by replacing noisy human feedback with a ground-truth oracle, the proxy gap closes to near zero.

How This Shows Up in Interviews

- **“Why does RLHF sometimes make models worse even though the reward goes up?”** Strong answer: the reward model is a finite-sample proxy. As the policy optimizes against it, the policy finds stylistic and structural patterns that score well on the proxy but were underrepresented in reward model training - a train-test gap. Proxy reward and true win-rate diverge after a critical number of RL steps; empirically, this inflection is consistently observed across human and noisy AI feedback settings.
- **“How do you detect and prevent reward over-optimization in a production training run?”** Strong answer: maintain a separate evaluation signal not used for training reward (held-out reward model, automated benchmark, human A/B), monitor both continuously throughout the run, use KL regularization to limit policy drift, and stop when the independent signal plateaus or drops.
- **Senior follow-up: “If you iteratively retrain the reward model on data from the current policy, does over-optimization disappear?”** Senior answer: it is significantly reduced but not eliminated. Each iteration fits a fresh finite-sample estimator on the current policy’s distribution, which closes the distribution gap at that step. But label noise in human annotations is irreducible - you are still fitting a noisy estimator, and the RL algorithm will still find its edges given enough steps. This is why noise-free verifiable rewards achieve fundamentally different training dynamics: there is no proxy gap to exploit.

Key Takeaways

- Reward over-optimization is the RL analog of overfitting: the policy maximizes a proxy that drifts from the true objective.
- The root cause is noise and finite-sample estimation in human preference labels, not a flaw in RL itself.
- Proxy reward and true win-rate always diverge eventually; monitoring both and stopping early is non-negotiable.
- KL regularization slows but does not eliminate over-optimization; iterative reward model retraining helps but does not fully close the gap.
- The cleanest solution is verifiable rewards, where noise-free signals make over-optimization negligible - the key insight motivating the reasoning-model generation of post-training.

27.3.2 Length Bias and Calibration Degradation

Two failure modes of RLHF are subtle enough to survive into production before anyone notices. First, models trained with human feedback systematically grow more verbose - not because verbosity helps users,

but because the reward signal incentivizes it. Second, they become overconfident: their output probabilities no longer track empirical accuracy in any reliable way. Neither shows up in the proxy reward metric, and neither is caught by standard capability benchmarks. Both have been replicated across multiple labs. Understanding them is basic literacy for any engineer running a production RLHF pipeline.

Length bias arises from the structure of pairwise preference annotation. When a human rater sees two responses and must pick one, a consistent heuristic applies: the longer, more detailed response *looks* more thorough and helpful. This is often true - more relevant content is better - but it creates a spurious correlation between response length and preference label that is independent of actual quality. A reward model trained on these annotations internalizes the correlation. The RL policy, optimizing against that reward model, learns that verbosity is profitable. Lengths climb throughout training even after task accuracy has plateaued.

Calibration degradation is a deeper structural consequence of abandoning distribution matching. Pre-training and SFT train the model by minimizing cross-entropy loss, which is equivalent to minimizing KL divergence from the data distribution. This implicitly trains the model to assign token probabilities that match empirical frequencies - the origin of calibration. RLHF replaces this objective with a scalar reward. There is no term in the PPO or DPO loss that requires output probabilities to remain accurate probability estimates. The optimizer is free to, and does, push probabilities toward extremes that maximize reward while discarding distributional accuracy.

Expected calibration error (ECE) measures the resulting gap:

$$\text{ECE} = \sum_{b=1}^B \frac{|B_b|}{n} |\text{acc}(B_b) - \text{conf}(B_b)|$$

where B_b is the set of predictions with confidence in bucket b , acc is empirical accuracy within that bucket, and conf is mean predicted probability. RLHF models consistently show higher ECE than their SFT counterparts at temperature 1, and the gap grows with RL training steps.

RLHF produces a policy, not a probability model; treating its output probabilities as calibrated confidence estimates is a category error.

Length bias exists because pairwise annotation measures overall impression, not information per token. A 300-token response with three substantive points and a polite closer tends to beat a 150-token response with the same three points and nothing extra, even when both are equally correct. The reward model is not wrong to learn this - it correctly fits the labels it was given - but those labels conflate quality with length. Once the reward model internalizes the correlation, the RL loop exploits it directly.

Calibration degradation exists because pretraining instills calibration as a side effect of log-likelihood maximization on a diverse corpus. RLHF is an active intervention that overrides that prior. The policy gradient objective pushes the model's distribution toward regions of high expected reward, which are not the same regions of high empirical token frequency that calibration requires. Empirically, RLHF models at temperature 1 assign probability ~ 0.90 to answers where actual accuracy is ~ 0.70 . The models *sound* more decisive and confident to users. Human raters find decisive responses appealing and reward them, which means the RL loop actively reinforces overconfidence.

Both pathologies compound with over-optimization: the longer you run RL, the longer responses get and the worse calibration becomes, independently of whether the proxy reward is still climbing.

A Worked Example

Start from a 7B model fine-tuned with SFT on an instruction corpus. Run 1,000 steps of PPO against a reward model trained on 50K pairwise annotations. Evaluate at checkpoints: SFT baseline (step 0), step 500, and step 1,000.

Length. On 200 sampled open-ended questions: SFT baseline averages 118 tokens per response. At step 500: 174 tokens. At step 1,000: 231 tokens. The proxy reward has increased 0.6 normalized units over this range. A held-out reference-based metric (ROUGE-L against human-written answers) gains 2.1 points from step 0 to step 500 - a real quality improvement. From step 500 to step 1,000, ROUGE-L adds only 0.3 points while mean length grows by 57 tokens. The extra tokens are padding: repeated topic sentences, redundant closing summaries, filler phrases like “I hope this helps.”

Calibration. On 500 factual multiple-choice questions with verified ground truth: SFT ECE ≈ 0.06 at temperature 1. After 1,000 PPO steps: ECE ≈ 0.19 . The model now assigns mean confidence 0.91 to questions where its accuracy is 0.74. To recover ECE < 0.07 via temperature scaling, the SFT model requires $T \approx 1.15$; the RLHF model requires $T \approx 1.80$, indicating that probability mass has been substantially compressed toward high-confidence predictions throughout RL training.

What You Actually Decide in Practice

- **Log output length throughout RLHF training** and compare against the SFT baseline by query category. If mean length doubles without a corresponding quality gain on a held-out metric, you have a length bias problem. Address it with length-normalized objectives - SimPO divides the reward by response length; length-normalized DPO applies the same correction inside the DPO loss. Both add one division to the loss computation and cost negligible engineering effort.
- **Never use an RLHF model’s raw output probabilities as confidence estimates.** If your application requires calibrated uncertainty - risk scoring, threshold-based routing, any safety-critical use - either apply post-hoc calibration (temperature scaling, Platt scaling) at deployment time or use the SFT checkpoint for probability estimates and the RLHF checkpoint for output generation.
- **Include a calibration checkpoint in your model evaluation gate.** Before any RLHF checkpoint ships, measure ECE on a factual benchmark at temperature 1. Treat it like a regression test: any checkpoint with ECE exceeding a predefined threshold is blocked from serving. A reasonable starting threshold for a 7B model is ECE < 0.10 ; tighten this for high-stakes applications.
- **Overconfident models are a safety concern, not just a calibration metric.** Users interpret confident tone as reliable information. In medical, legal, or financial contexts, a model that expresses 90% confidence when it is right only 70% of the time is actively misleading. This is worth naming explicitly in any system design review.
- **Both pathologies worsen with more RL steps**, which is an additional practical reason - beyond reward over-optimization - to monitor closely, set early-stopping criteria, and not treat longer training as automatically better.

How This Shows Up in Interviews

- **“What are the systemic side effects of RLHF on model behavior beyond the intended alignment?”**
Strong answer: length bias (verbosity is spuriously rewarded because human annotators conflate length

with quality) and calibration degradation (RL optimizes a scalar reward rather than a distributional likelihood, destroying the probabilistic interpretation of output probabilities). Both are empirically replicated across lab settings and grow worse with more RL steps.

- **“How would you detect and fix length bias in a deployed RLHF model?”** Strong answer: instrument production request logs for output token count by query category; compare against the SFT baseline. If length is inflated, fix it in the next training run with length-normalized objectives. As a short-term mitigation, apply a maximum-token constraint in the inference server.
- **Senior follow-up: “Design an evaluation protocol that catches calibration degradation before a model ships.”** Senior answer: maintain a calibration benchmark of 500-1,000 factual multiple-choice questions with verified ground truth. After each RLHF checkpoint, measure ECE at temperature 1 and track the temperature-scaling coefficient needed to reach a target ECE. A rising coefficient signals growing overconfidence even if post-scaling ECE looks acceptable - the model is producing increasingly peaked distributions that require larger corrections. Treat both the raw ECE and the scaling coefficient as go/no-go signals. Flag any checkpoint where the required scaling temperature exceeds 1.5 for explicit review before serving.

Key Takeaways

- Length bias is a systematic artifact of pairwise annotation: human raters conflate length with quality, reward models learn this correlation, and RL exploits it. Monitor mean output length throughout training.
- RLHF breaks calibration because RL optimizes a scalar reward rather than log-likelihood; output probabilities from RLHF models should not be treated as calibrated confidence estimates.
- Both pathologies are empirically confirmed across labs and worsen with more RL steps, providing additional practical motivation to stop early and monitor closely.
- Length-normalized objectives (SimPO, length-normalized DPO) directly address length bias at the algorithm level with minimal implementation cost.
- Post-hoc temperature scaling can partially recover calibration for RLHF models, but the required temperature coefficient grows with RL training steps - track it as a leading indicator of degradation.

Chapter 28 Reinforcement Learning from Verifiable Rewards

28.1 From RLHF to RLVR

28.1.1 Why Verifiable Rewards Scale Better

When an interviewer asks you to design a post-training pipeline for a reasoning model—one capable of solving olympiad-level math or generating working code—you will eventually be asked why RLHF alone is not enough. The failure mode is overoptimization, and knowing its exact mechanism is what distinguishes a senior answer from a vague one. Reinforcement learning from verifiable rewards (RLVR) is the architectural shift powering every frontier reasoning model released in the past two years: o1, DeepSeek R1, Kimi K1.5, and Qwen 3 all rely on it. This topic appears in design-round and deep-dive interviews at virtually every lab working on post-training.

RLHF trains a policy to maximize a learned reward model fitted on human pairwise preferences. That reward model is a proxy: a finite neural network trained on a finite set of comparisons, approximating an underlying human preference distribution that is both noisy and complex. As gradient descent pushes the policy against this proxy, the policy learns to exploit the gaps between the proxy and the true preference function—a phenomenon called reward overoptimization. The proxy score climbs, but real human win rates plateau and eventually decline.

RLVR replaces the learned reward model with a deterministic, ground-truth verifier. In math, the verifier checks whether the model’s final answer matches the reference solution, either via regex or a symbolic equivalence library. In code, it executes the generated function against a hidden test suite and returns pass/fail. The reward is binary and exact. Because the verifier is not a neural network, it cannot be gamed by the gradient—there is no gap between the proxy and the ground truth, because they are the same object.

The formal RL objective is identical in both paradigms:

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

The entire difference lives in $r(x, y)$. In RLHF, r is a neural classifier with hundreds of millions of parameters, updated on human labels, and subject to distributional shift. In RLVR, r is a deterministic function: a regex match, a SymPy symbolic check, or a unit-test runner. The verifier has zero learnable parameters, evaluates in microseconds, and cannot be overfit by gradient descent.

RLVR succeeds where RLHF fails not because the RL algorithm changes, but because the reward signal is ground truth rather than a learned approximation of it.

The practical ceiling of RLHF arises from two compounding problems. First, human pairwise preferences are expensive to collect—labeler programs at scale cost roughly \$1-\$5 per comparison after quality control, and a production RLHF run consumes millions of comparisons. This makes it structurally impossible to generate reward signal online during training at the rate a GPU cluster can consume it. The reward model is trained once, fixed, and then used as a static oracle, even as the policy drifts far from the distribution the reward model was trained on.

Second, human preferences are noisy. Individual annotators disagree, preference labels carry positional and length biases, and the reward model fits to these artifacts. Empirically, when researchers train RLHF reward models on noiseless synthetic preferences, overoptimization disappears almost entirely; it reappears immediately when noise is reintroduced. This tells you that overoptimization is not a failure of RL algorithms but a failure

of the signal source.

RLVR bypasses both problems. A regex match costs a fraction of a cent per sample, so you can evaluate reward on every rollout during training with no budget pressure. The verifier is noiseless and stationary—a correct answer is always correct, regardless of how the policy evolved. This is the same design insight behind AlphaGo and AlphaFold: once you have a domain where correctness is computable, RL scales in proportion to compute, not in proportion to human annotation bandwidth.

RLHF also silently degrades calibration. Because the policy gradient optimizes expected reward rather than log-likelihood of any distribution, the resulting model has no probabilistic grounding. Multiple independent results—from Anthropic, from the GPT-4 technical report, from academic studies—show that RLHF models are substantially overconfident at temperature 1 relative to their SFT checkpoints. If you are building a system where calibrated probabilities matter (uncertainty-aware retrieval, best-of- n selection, or chain-of-thought aggregation), this is a real operational liability. RLVR does not fix calibration, but it at least operates in a domain where calibration is less central to the deployment use case.

A Worked Example

Consider training a 7B base model (32 layers, $d = 4096$, bf16) with RLVR on a curated set of competition math problems using GRPO on 8 H100s. Group size is $G = 8$ completions per prompt.

For a prompt x such as “Find all real solutions to $x^2 - 5x + 6 = 0$ ”, the policy samples 8 responses. Each is graded by a SymPy equivalence checker: responses stating $x = 2$ and $x = 3$ (in any equivalent form) receive $r_i = 1$; all others receive $r_i = 0$.

Suppose the group rewards for one prompt are $[1, 1, 0, 1, 0, 0, 1, 1]$, giving a group mean $\bar{r} = 0.625$ and standard deviation $\sigma_r \approx 0.48$. The GRPO advantage for each response is:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + \epsilon}$$

Correct responses receive $\hat{A}_i \approx +0.78$; incorrect responses receive $\hat{A}_i \approx -1.30$. The policy gradient step increases the log-probability of tokens in correct responses and decreases it for incorrect ones—with no reward model, no value network, and no human in the loop.

Notice what happens at the extremes. If all 8 responses are correct ($\bar{r} = 1$, $\sigma_r = 0$), the advantages collapse to zero and the gradient vanishes—this problem is too easy and contributes nothing. If all 8 are wrong ($\bar{r} = 0$, $\sigma_r = 0$), the same collapse occurs—this problem is too hard. Only problems where the group pass rate sits between roughly 20% and 80% generate informative gradient signal. Filtering your training set to this difficulty band is one of the highest-leverage decisions in an RLVR pipeline.

After a few thousand gradient steps across an 8-H100 run, a 7B model trained this way typically improves 15-25 percentage points on MATH-500 relative to the SFT checkpoint. A comparable RLHF run on open-ended preferences would require careful early stopping, and the gains would be harder to reproduce across seeds.

What You Actually Decide in Practice

- **Domain selection is the upstream constraint.** RLVR only applies where you can write a verifier. Math (symbolic equivalence via SymPy), code (unit-test execution), and formal proofs (proof-checker APIs) qualify. Summarization, creative writing, and general instruction following do not. For those domains, RLHF or AI-judge feedback remains the only option.

- **Format rewards are not optional.** Every production RLVR system pairs the accuracy reward with a lightweight format reward—typically a small bonus for placing the chain of thought inside designated delimiters such as `<think>...</think>`. Without format rewards, models produce structurally inconsistent outputs that are difficult to verify or display. In practice, the format reward is as operationally critical as the accuracy reward for training stability.
- **Verifier quality is the new reward-model quality.** Raw string matching on math answers fails on equivalent but syntactically different expressions (e.g., $\frac{1}{2}$ vs. 0.5). Use symbolic equivalence (SymPy) or an LLM-as-judge for math, and hidden test cases with edge coverage for code. A leaky verifier creates exactly the same overoptimization dynamic as a noisy reward model.
- **Problem difficulty filtering is a first-class concern.** Sample only problems where your current policy’s group pass rate falls between 20% and 80%. Problems outside this range contribute near-zero gradient in expectation. Kimi K1.5 explicitly excludes problems solvable by best-of-8 from their base model; Qwen 3 achieves strong reasoning gains from only about 4,000 curated problems filtered to this difficulty window.
- **RLHF still follows RLVR in production.** RLVR sharpens domain competence; it does not produce a well-rounded assistant. DeepSeek R1, Kimi K1.5, and Qwen 3 all apply a standard RLHF stage after RLVR to recover helpfulness, formatting quality, and safety on open-ended tasks. The two stages are complementary, not competitive.

How This Shows Up in Interviews

- **“Why does RLHF fail for reasoning tasks?”** Strong answer: the reward model is a noisy, finite proxy for human preferences. The policy exploits the gap between proxy and ground truth—reward scores rise while real quality stagnates. This is Goodhart’s law in practice. Human feedback is also too expensive to generate online at training scale.
- **“How would you design a reward function for a math RL training pipeline?”** Strong answer: binary accuracy reward using symbolic equivalence (SymPy) rather than string matching, plus a lightweight format reward for response structure. Filter problems to a 20%-80% group pass-rate window. Do not use multiple-choice or true/false problems—the model can random-walk to high reward without reasoning.
- **Senior follow-up: “If RLVR is strictly better for reasoning, why do production pipelines still run RLHF afterward?”** Senior answer: RLVR is better only within its domain. Once math and code competence are locked in, the model needs to handle open-ended tasks—writing, instruction following, multi-turn chat—where no verifier exists. RLHF with AI-judge feedback is the only practical signal source for those domains. The two stages address orthogonal capabilities and are used sequentially in all known production reasoning-model pipelines.

Key Takeaways

- RLHF overoptimizes because the reward model is a noisy, learnable proxy; the policy exploits the proxy faster than human win rates improve.
- RLVR uses deterministic verifiers—regex, symbolic checkers, unit tests—eliminating the proxy and making overoptimization structurally impossible against the reward function.
- Format rewards are a critical implementation detail: without them, RLVR pipelines produce structurally inconsistent outputs and train unstably.

- Only problems with a 20%-80% group pass rate generate meaningful gradient signal; filtering to this difficulty window is one of the highest-leverage decisions in RLVR.
- RLVR and RLHF are complementary: production pipelines use RLVR to build domain competence, then RLHF to recover general-purpose alignment on open-ended tasks.

28.1.2 Outcome Rewards vs. Process Rewards

When you are designing the reward signal for a math or code RL pipeline, the most consequential decision is not which optimization algorithm to use—it is where in the response to measure correctness. Do you score only the final answer, or do you also score each intermediate reasoning step? This outcome-versus-process distinction drives data cost, training stability, and the ceiling on model quality. It shows up in system design interviews as a follow-on after the basic RLVR setup is established, and answering it precisely—with the empirical context from R1 and DeepSeekMath—is what marks a senior answer.

An *outcome reward* is a single scalar assigned to the entire response based solely on whether the final answer is correct. The reward function $r(x, y)$ ignores all intermediate reasoning: chains of thought, intermediate calculations, self-correction steps. It asks exactly one question: did the model produce the right final answer?

$$r(x, y) = \begin{cases} 1 & \text{if final answer matches ground truth} \\ 0 & \text{otherwise} \end{cases}$$

A *process reward* (or *process reward model*, PRM) assigns a score to each intermediate reasoning step. Given a problem x and a chain of thought $y = (s_1, s_2, \dots, s_T, a)$ where s_t are reasoning steps and a is the final answer, a PRM assigns a scalar $r_t \in [0, 1]$ to each step t . The total reward is typically the minimum step score, a product, or a running average. The key advantage is that the PRM provides gradient signal at every step in the chain, not only at the terminal token.

Outcome rewards are cheap to evaluate and structurally immune to overoptimization; process rewards provide denser gradient signal but require expensive annotation and introduce a learned proxy that can itself be exploited.

Outcome rewards dominate production systems for three concrete reasons.

First, they require no labeled data beyond the final answer, which in verifiable domains is already known deterministically. A SymPy equivalence check or a test-suite runner costs microseconds and involves no human. A PRM, by contrast, requires either human annotation of individual reasoning steps—which is roughly $10\times$ more expensive per example than annotating final answers—or a separately trained neural scorer. That scorer is itself a learned proxy, which reintroduces the overoptimization risk that RLVR was designed to escape. A miscalibrated PRM can penalize creative but ultimately correct reasoning, or reward plausible-looking but incorrect chains of thought that happen to resemble training examples.

Second, PRMs are uniquely difficult to calibrate for long chains of thought. A step that appears wrong at position $t = 3$ might be a deliberate detour that arrives at the correct answer by $t = 15$. Human annotators disagree heavily on intermediate mathematical steps, and model-based PRMs trained on synthetic labels compound their errors multiplicatively across a long chain. The effective noise of a PRM on a 64-step chain of thought is substantially higher than the noise of a binary outcome verifier on the same problem.

Third, the large-scale empirical evidence has settled, for now, in favor of outcome rewards. DeepSeekMath showed process supervision outperforming outcome supervision for a mid-sized base model on competition math—the PRM line was the best-performing method in that comparison. But when DeepSeek R1 scaled to a

much larger base model with careful difficulty filtering and GRPO’s group baseline, they tried extensive PRM experiments and concluded that outcome-based rewards matched or exceeded PRM performance. R1 also evaluated Monte Carlo Tree Search as an alternative dense-feedback mechanism and reached the same conclusion: neither PRMs nor search-based methods added consistent value over outcome rewards with GRPO. The result that made PRMs look compelling in DeepSeekMath was contingent on model scale and dataset composition; it did not transfer to the R1 setting.

The main practical failure mode of outcome rewards is *sparse reward*: when the model is too weak to solve any problems in a batch, every rollout receives reward zero, the GRPO group advantage collapses to zero everywhere, gradient updates vanish, and training stalls. This is a scenario where process rewards would provide non-zero signal on partially correct reasoning. The practical resolution, however, is difficulty filtering rather than switching to PRMs—keep only problems where the group pass rate falls between roughly 20% and 80%, which guarantees that at least some rollouts receive nonzero reward and the group advantage is informative.

A Worked Example

Consider the two-step problem: “If $3x + 7 = 22$, what is $2x$?” Ground truth: $x = 5$, so $2x = 10$.

Under an **outcome reward**, the verifier reads only the final answer token. A response that correctly computes $3x = 15$, then $x = 5$, but writes “therefore $2x = 9$ ” due to an arithmetic slip receives reward 0. The correct intermediate steps contribute nothing.

Under a **process reward**, a step scorer reads each line. Step 1: “ $3x = 22 - 7 = 15$ ” scores 1.0. Step 2: “ $x = 5$ ” scores 1.0. Step 3: “ $2x = 9$ ” scores 0.0. The minimum-step PRM gives reward 0.0, a product gives 0.0, and an average gives 0.67. All formulations agree the final step is wrong; they disagree on how much credit the correct preceding steps deserve.

Now run $G = 8$ rollouts. Suppose 3 correctly produce $2x = 10$ (reward 1) and 5 do not (reward 0). Under outcome rewards, the group mean is 0.375 and the GRPO advantage for a correct response is positive: the gradient step increases the log-probability of correct completions. If instead all 8 rollouts get the final step wrong, but 6 of them show correct intermediate algebra, a well-calibrated PRM returns non-zero group advantages—training does not stall. This is the canonical scenario in which process rewards add value.

But building this PRM cost human annotators several hours of step-level annotation, or required training a separate neural scorer on hundreds of thousands of labeled derivations. For a 7B model that can already solve 30% of problems at outcome level after difficulty filtering, this cost is rarely justified.

What You Actually Decide in Practice

- **Default to outcome rewards; switch to PRMs only when filtering cannot rescue sparse rewards.** If best-of- G on your filtered training set still yields near-zero pass rate for the current policy checkpoint, a PRM may be necessary. This scenario is unusual for base models above 7B parameters on standard competition math.
- **Treat difficulty filtering as the first-pass fix for sparse outcome rewards.** Before investing in a PRM, filter problems so that the group pass rate sits between 20% and 80%. This directly addresses the sparsity problem that motivates PRMs. Kimi K1.5 and Qwen 3 both use this filtering strategy and achieve strong results without process supervision.
- **GRPO already has a process-like component through per-token KL regularization.** In the full reward-shaping setup, the per-token reward for token t is the negative KL penalty: $-\beta \log(\pi_\theta(a_t |$

$s_t)/\pi_{\text{ref}}(a_t \mid s_t)$). The accuracy reward appears only at the terminal token. This means GRPO is already a hybrid: dense regularization signal at every position, sparse task reward only at the end. The per-token KL is not a process reward in the supervised sense, but it prevents the model from taking arbitrarily divergent intermediate paths.

- **Remove length normalization when using outcome rewards.** Dividing the reward by response length—a choice in some early GRPO implementations—creates a perverse incentive: when reward is negative (incorrect response), the loss is minimized by making the response as long as possible so each token’s contribution is small. This produces models that pad incorrect responses to extreme lengths. Control response length through a separate, decoupled length reward rather than by normalizing the accuracy reward.
- **If you do build a PRM, use a model-based equivalence checker rather than human step labels.** Math shepherd-style approaches train a PRM on process annotations derived from large-scale rollouts: generate many responses, identify which steps appear in correct versus incorrect completions, and use that signal to train the scorer. This amortizes annotation cost but introduces its own distribution shift and noise. Budget for a calibration evaluation before deploying the PRM in a training loop.

How This Shows Up in Interviews

- **“What’s the difference between outcome and process rewards, and when would you use each?”** Strong answer: outcome rewards score only the final answer—deterministic, costless, no learned scorer. Process rewards score intermediate steps—denser gradient signal, but require expensive step-level annotation or a learned PRM that introduces proxy noise. Default to outcome rewards; consider PRMs only when difficulty-filtered outcome rewards still yield near-zero group pass rates across your training set.
- **“Your RLVR training curve is flat and reward isn’t improving. What do you diagnose first?”** Strong answer: check the group pass-rate distribution across your training problems. If the majority of problems have 0% or 100% group pass rate, you have a difficulty calibration problem, not an algorithm problem. Re-filter the problem set before adding a PRM or changing the optimizer. Also verify that your verifier is not silently accepting incorrect formats.
- **Senior follow-up: “DeepSeekMath showed PRMs clearly outperforming outcome rewards. DeepSeek R1 used outcome rewards and dropped PRMs after extensive experiments. How do you reconcile those results?”** Senior answer: DeepSeekMath used a smaller base model on hard competition problems, so outcome rewards were genuinely sparse—the model solved too few problems to receive useful signal. At R1 scale (a large pre-trained model with strong mathematical priors), enough problems in the filtered set receive nonzero outcome reward for GRPO’s group baseline to work well. The PRM advantage is a function of model weakness and problem difficulty relative to the current checkpoint; as those improve, the marginal value of a PRM decreases while its engineering cost remains constant. R1’s conclusion is conditional on their filtering and model scale, not a universal result.

Key Takeaways

- Outcome rewards assign a single binary score to the final answer; process rewards (PRMs) assign scores to each intermediate reasoning step, providing denser gradient signal at higher annotation cost.
- Outcome rewards win in production systems because they are deterministic, require no learned scorer, and avoid the secondary overoptimization risk a PRM introduces.

- Sparse outcome rewards are best addressed by difficulty filtering (target 20%-80% group pass rate), not by adding a PRM.
- GRPO is already a hybrid: it applies per-token KL regularization at every position (dense regularization) with the accuracy reward concentrated at the terminal token (sparse task signal).
- The PRM advantage observed in smaller-scale systems diminishes as base model capability improves and difficulty filtering is applied; PRMs remain a viable tool for bootstrapping RL on weak policies, not a permanent fixture of mature pipelines.

28.2 Policy Gradient Methods

28.2.1 Baselines, Advantages, and Variance Reduction

At a senior post-training interview, after you sketch out any RL training loop-PPO, GRPO, or a custom variant-the follow-up is almost always: “Walk me through why your gradients are noisy and what you do about it.” Variance in policy gradient estimates is the central numerical challenge that separates working implementations from broken ones. Engineers at L5-L6 are expected to derive the baseline trick on a whiteboard, prove it is bias-free, and explain how it connects to the advantage function that every practical algorithm uses internally.

The *policy gradient theorem* gives the gradient of expected reward $J(\theta) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[R(s, a)]$ as:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s,a}[R(s, a) \nabla_\theta \log \pi_\theta(a | s)]$$

The derivation uses one identity: $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$. In the language model setting, s is the prompt, a is the complete response, and R is a scalar reward-binary for verifiable tasks (correct/incorrect). Estimating this gradient from a single rollout is unbiased but has enormous variance. If a model generates math proofs and achieves a 5% solve rate, 95% of gradient samples are exactly zero and training makes no progress.

The *baseline trick* is the structural fix. Any function $B(s)$ that depends only on the state (not the action) can be subtracted from the reward without biasing the gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{s,a}[(R(s, a) - B(s)) \nabla_\theta \log \pi_\theta(a | s)]$$

The proof is one line: $\mathbb{E}_{a \sim \pi_\theta}[B(s) \nabla_\theta \log \pi_\theta(a | s)] = B(s) \cdot \mathbb{E}_a[\nabla_\theta \log \pi_\theta(a | s)] = 0$, because the gradient of a normalized log-probability sums to zero over any action space.

The optimal baseline (minimizing gradient variance) is $B^*(s) = \mathbb{E}_{a \sim \pi_\theta}[R(s, a) | s]$, the expected reward from state s . Practitioners call this the *value function* $V(s)$. The difference between actual reward and this baseline is the *advantage function*:

$$A(s, a) = R(s, a) - V(s)$$

A positive advantage says action a outperformed the policy’s average expected outcome from s ; a negative advantage says it underperformed. All practical policy gradient algorithms-PPO, GRPO, and every variant in between-are ultimately different answers to one question: how do we estimate $A(s, a)$ cheaply and with low variance?

Every working RL algorithm for language models boils down to: compute an advantage estimate, upweight high-advantage responses, downweight low-advantage ones, and prevent the policy from moving too far in one step.

Without a baseline, raw reward conflates two signals: how good the response was and how easy the prompt was. A hard problem scoring 0.6 and an easy problem scoring 0.6 receive identical gradient weights, even though the former represents an excellent response and the latter a mediocre one. The baseline removes the

prompt-level expected reward, isolating only the within-prompt quality signal.

This matters because sparse rewards make the naive estimator nearly useless even at large batch sizes. In supervised learning, every sample contributes a non-zero gradient. In sparse-reward RL, a 0-reward rollout contributes exactly zero-scaling the batch just scales the waste. Centering by the per-prompt mean turns every wrong answer into an active downweight signal, converting a sparse signal into a dense training signal across all rollouts in the group.

The value function approximation also adapts to question difficulty automatically. A baseline estimated from group rollouts is high for easy prompts (where most responses succeed) and low for hard prompts (where most fail). The resulting advantage correctly flags when a response is surprisingly good for its difficulty level, rather than just absolutely high-reward.

A Worked Example

Setup: 32 prompts, $G = 4$ rollouts each, from a 7B model on a math benchmark. Focus on two representative prompts:

- **Prompt P1 (easy):** rollout rewards = $[1, 1, 0, 1]$. Mean = 0.75, std ≈ 0.43 .
- **Prompt P2 (hard):** rollout rewards = $[0, 0, 0, 0]$. Mean = 0, std = 0.

Naive policy gradient: weight each rollout’s log-probability gradient by raw reward. P1-correct rollouts get weight $+1.0$; P1-wrong and all P2 rollouts get weight 0. P2 contributes nothing to training even though the model is failing on it.

Centered advantages (subtract per-prompt mean):

Rollout	Raw R	$R - \bar{R}$
P1-correct ($\times 3$)	1.0	$+0.25$
P1-wrong ($\times 1$)	0.0	-0.75
P2-any ($\times 4$)	0.0	0.0

Now P1’s wrong rollout receives a downweight of -0.75 , reinforcing the contrast between correct and incorrect responses. P2 correctly contributes zero gradient—there is no within-group signal when every rollout fails, and making a noisy update would only harm training.

As a concrete variance illustration: suppose two prompts contribute rewards of 11 and 9 (easy) or 0 and 2 (hard) under a uniform initial policy. Naive gradient variance across the four (state, action) pairs is $\text{Var}(\{11, 9, 0, 2\}) = 21.25$. With baselines $B(\text{easy}) = 10$ and $B(\text{hard}) = 1$, effective rewards become $\{+1, -1, -1, +1\}$ with variance 1.0—a roughly $21\times$ reduction at zero cost to the optimization objective.

What You Actually Decide in Practice

- **Use $G \geq 4$ rollouts per prompt.** With fewer samples, the empirical mean is a poor baseline estimate and may increase variance rather than reduce it. The GRPO paper uses $G = 8$; in practice $G = 4$ -8 balances rollout inference cost against baseline quality.
- **When all G rollouts for a prompt achieve the same reward, centered advantages are zero and no update fires.** This is correct behavior, not a bug. If every rollout is correct (easy) or every rollout is wrong (unsolvable), there is no within-group learning signal. Forcing an update from this zero-signal state only adds noise.

- **Dividing centered advantages by the group standard deviation is not a valid baseline operation.** Subtracting the group mean is bias-free (the subtracted quantity has expectation zero over actions). Dividing by σ is not-it changes gradient magnitude as a function of reward spread. When σ is small, gradients are amplified for prompts that are trivially easy or completely unsolvable, precisely the prompts where the gradient signal is least informative.
- **A frozen initial checkpoint is a valid and cheap baseline.** Treat $B(s) = \mathbb{E}_{a \sim \pi_0}[R(s, a) \mid s]$, estimated from a handful of rollouts under the pre-RL model, as a constant. No gradient flows through it. This approximates the value function without any learned network.
- **Track per-prompt reward mean as a curriculum signal.** Once the running success rate for a category of prompts exceeds 90%, those prompts contribute near-zero gradient signal at full inference cost. Filter them out and replace with harder examples.

How This Shows Up in Interviews

- **“Why does vanilla policy gradient have high variance, and what is the baseline trick?”** Name sparse rewards as the root cause. Derive $\nabla_{\theta} J = \mathbb{E}[R \nabla_{\theta} \log \pi_{\theta}]$ in one line via the log-derivative trick, then show subtracting $B(s)$ is bias-free because $\mathbb{E}_{a \sim \pi_{\theta}}[\nabla_{\theta} \log \pi_{\theta}(a \mid s)] = 0$. The key constraint: B must depend only on state, not action.
- **“What is an advantage function and how does it connect to value functions?”** The advantage $A(s, a) = R(s, a) - V(s)$ is the signed difference between actual reward and the policy’s average expected reward from that state. In outcome-reward LM-RL, $V(s)$ is approximated by the mean reward across G rollouts for the same prompt-no future-reward bootstrapping is needed.
- **Senior follow-up: “GRPO divides the group advantage by the standard deviation. Is that mathematically valid?”** No. Subtracting the group mean is a valid baseline (zero-expectation subtraction over actions). Dividing by σ is not-it modifies gradient magnitude as a function of reward spread, which has no justification in the policy gradient theorem. Specifically, when $\sigma \rightarrow 0$ (all rollouts score identically), gradients explode. The practical consequence is that problems at both extremes of the difficulty spectrum-already mastered or completely unsolvable-get disproportionate gradient weight. The “Dr. GRPO” analysis demonstrates that removing the σ normalization yields shorter responses and equal-or-better reward, confirming this is a design flaw rather than a feature.

Key Takeaways

- The policy gradient $\nabla_{\theta} J = \mathbb{E}[R(s, a) \nabla_{\theta} \log \pi_{\theta}(a \mid s)]$ is unbiased but catastrophically high-variance under sparse rewards.
- Any state-dependent function $B(s)$ is a valid bias-free baseline; the optimal choice is the value function $V(s) = \mathbb{E}[R \mid s]$.
- The advantage $A(s, a) = R(s, a) - V(s)$ converts sparse rewards into dense relative signals across the rollout group, involving every rollout in training rather than only the successes.
- When all rollouts for a prompt tie on reward, centered advantage is zero and no update fires-this is correct behavior indicating no within-group learning signal.
- Dividing by group standard deviation (the GRPO default) is not a valid baseline operation; it overweights the difficulty extremes and can be safely removed without hurting performance.

28.2.2 GRPO in Depth

When you sketch an RL training loop for a 70B reasoning model and the interviewer asks, “PPO requires a critic as large as the policy-how do you handle that?” the answer is GRPO. Group Relative Policy Optimization removes the value network entirely, replacing it with a within-group reward normalization that fits naturally into the language model setting. Understanding the full algorithm-objective, training loop, memory layout, and failure modes-is now a baseline expectation for post-training roles at frontier labs, typically probed at L5 and above.

GRPO starts from PPO and makes one structural change: replace generalized advantage estimation (GAE), which requires a separately trained value network, with a group-level normalized reward. For each prompt q , sample G responses $\{a_1, \dots, a_G\}$ from the current policy $\pi_{\theta_{\text{old}}}$. The group advantage for response i is:

$$\hat{A}_i = \frac{R_i - \frac{1}{G} \sum_j R_j}{\text{std}_j(R_j) + \epsilon}$$

where $\epsilon = 10^{-4}$ prevents division by zero when all rollouts score identically. The GRPO objective applies the standard PPO clipped surrogate with this advantage:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G} \sum_{i=1}^G \min\left(r_i \hat{A}_i, \text{clip}(r_i, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right) + \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

where $r_i = \pi_{\theta}(a_i \mid q) / \pi_{\theta_{\text{old}}}(a_i \mid q)$ is the importance ratio and $\epsilon = 0.2$ is the clip radius. The KL term uses a control-variate estimator with lower variance than the standard Monte Carlo form:

$$\hat{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(a \mid q)}{\pi_{\theta}(a \mid q)} - \log \frac{\pi_{\text{ref}}(a \mid q)}{\pi_{\theta}(a \mid q)} - 1$$

This has the same expectation as $-\log(\pi_{\text{ref}}/\pi_{\theta})$ because $\mathbb{E}_{a \sim \pi_{\theta}}[\pi_{\text{ref}}(a)/\pi_{\theta}(a)] = 1$, so the extra term cancels in expectation while reducing variance of individual sample estimates.

In the purely online case where $K = 1$ gradient step is taken per rollout batch, the importance ratio $r_i = 1$ exactly, the clipping is inactive, and the loss reduces to a plain policy gradient weighted by $\hat{A}_i$. The clipping and multi-step inner loop only engage when reusing rollouts across $K > 1$ gradient updates.

GRPO is PPO with the value network deleted and the advantage replaced by within-group z-score normalization; clipping, KL regularization, and the multi-step inner loop transfer directly from PPO unchanged.

The value network in PPO is typically the same architecture as the policy. A 7B policy in bf16 occupies 14 GB for parameters; with Adam optimizer states the footprint grows to roughly 56 GB. Adding a co-located 7B critic doubles the parameter memory to 112 GB before activations. On an 8×H100 node with 640 GB total GPU memory, this leaves little headroom for activation memory and rollout buffers. Eliminating the critic is not a minor optimization-it is often the difference between a training run fitting on one node versus requiring two.

The group structure provides a difficulty-adaptive baseline without any learned parameters. A prompt where all G rollouts score 1.0 produces $\hat{A}_i = 0$ for every i : the policy has mastered it and no gradient flows. A prompt where 1 of 8 rollouts is correct produces $\hat{A}_{\text{correct}} \approx +1.63$ (the z-score of the single success), concentrating gradient mass precisely on the response that worked. This adaptive difficulty weighting is implicit and costs only a mean and standard deviation computation.

Compared to DPO, GRPO is inherently online: rollouts are sampled from the current policy at each step, rewards are computed immediately by a deterministic verifier (regex match, unit test, symbolic solver), and updates follow. DPO requires a static pre-collected preference dataset with an explicit pairing structure. When your reward is a fast deterministic checker-as it is for math, code, and logic tasks-on-policy GRPO can leverage

the full capacity of the current model's distribution while DPO is locked to the distribution of the fixed dataset.

A Worked Example

Setup: 7B model on a math benchmark, $G = 8$ rollouts per prompt, 32 prompts per batch, H100 80 GB, bf16, clip $\varepsilon = 0.2$, KL coefficient $\beta = 0.04$, $K = 4$ inner gradient steps.

Step 1 - Rollouts. For each of the 32 prompts, generate 8 responses via the inference engine (256 sequences total). Compute binary rewards from an exact-match checker. For one representative prompt, suppose rewards are $[1, 0, 0, 1, 0, 0, 1, 0]$ (3 correct of 8).

Step 2 - Advantage computation.

$$\begin{aligned}\bar{R} &= 3/8 = 0.375 \\ \sigma_R &= \sqrt{(3/8)(5/8)} \approx 0.484 \\ \hat{A}_{\text{correct}} &= (1.0 - 0.375)/0.484 \approx +1.29 \\ \hat{A}_{\text{wrong}} &= (0.0 - 0.375)/0.484 \approx -0.78\end{aligned}$$

Step 3 - Save old log-probs. At rollout time, compute and store $\log \pi_{\theta_{\text{old}}}(a_i | q)$ for all 256 responses. This is one forward pass per response in inference mode. Store the result as a float32 array (256 sequences $\times$ 512 tokens $\approx$ 0.5 GB), not a copy of the model weights.

Step 4 - Clipped surrogate loss ($K = 4$ inner steps). For each inner step, compute $\log \pi_{\theta}(a_i | q)$ under the current trainable policy and form $r_i = \exp(\log \pi_{\theta} - \log \pi_{\theta_{\text{old}}})$:

```
ratio    = (log_pi_theta - log_pi_old).exp()      # stop_grad on log_pi_old
surr1    = ratio * advantage                      # [G, L]
surr2    = ratio.clamp(1 - eps, 1 + eps) * advantage
pg_loss  = -torch.min(surr1, surr2).mean()
```

Step 5 - KL penalty. Compute $\log \pi_{\text{ref}}(a_i | q)$ from the frozen reference model and add:

```
kl_est   = (pi_ref / pi_theta) - (log_pi_ref - log_pi_theta) - 1
kl_loss  = beta * kl_est.mean()
loss     = pg_loss + kl_loss
```

Step 6 - Gradient update. Standard Adam step. Repeat steps 4-6 for $K = 4$ iterations, then return to Step 1 for a fresh rollout batch.

Memory accounting: Policy parameters (bf16, 7B) = 14 GB; Adam states (fp32, $2 \times 7\text{B}$) = 56 GB; π_{ref} (frozen, bf16) = 14 GB; $\log \pi_{\text{old}}$ array $\approx$ 0.5 GB. Total $\approx$ 85 GB. No value network. Compare this to a PPO setup at the same scale: adding a 7B critic with its own Adam states would add another 70 GB, pushing the total past 150 GB.

What You Actually Decide in Practice

- **Use $K = 1$ inner steps early in training.** At $K = 1$, the importance ratio is exactly 1.0, clipping is inactive, and the loss is a plain policy gradient-the lowest-variance regime. Increase to $K = 4$ -8 once the policy is stable enough that log-probs from one rollout batch remain reasonable approximations across multiple gradient steps.

- **Never store π_{old} as a full model copy.** The only data you need from π_{old} during the K inner steps are the log-probs computed at rollout time. Storing 256 sequences $\times$ 512 tokens of log-probs costs ~ 0.5 GB versus 14 GB for a second 7B model copy. Wrap the saved log-probs in `torch.no_grad()` to prevent accidental gradient flow.
- **Watch response length as the primary pathology signal.** When the GRPO loss divides per-token log-probs by sequence length, the per-token loss on a wrong response is $-|\hat{A}|/L$. The model learns to minimize this by making wrong responses very long. If average response length increases monotonically while reward plateaus, remove length normalization from the loss immediately.
- **Filter prompts where best-of- G pass rate is 0% or 100%.** At 0% pass rate, $\sigma_R = 0$ and advantages collapse or blow up. At 100%, the same collapse occurs and the prompt carries no gradient signal regardless of normalization. Excluding both extremes keeps every batch prompt informationally dense and implicitly implements a curriculum. Qwen 3 samples prompts proportional to $(1 - \text{pass_rate})$ to enforce this dynamically.
- **Use $\beta = 0.01$ - 0.1 for the KL coefficient.** Too small and the policy drifts into reward-hacking regimes. Too large and the KL term dominates, starving reward optimization. Starting at $\beta = 0.04$ and monitoring KL divergence for the first few hundred steps is my preferred approach; if KL exceeds roughly 0.3 nats, increase β or reduce the learning rate.

How This Shows Up in Interviews

- **“How does GRPO differ from PPO, and when would you choose one over the other?”** The core difference is the advantage estimator: GRPO uses within-group z-score, eliminating the value network and roughly halving memory. Choose GRPO when memory is the binding constraint or when you lack a good value function initialization (common in new task domains). Stay with PPO when you have a mature critic and need stronger variance reduction on long-horizon tasks where the group-mean baseline is insufficiently accurate.
- **“Walk me through the GRPO training loop.”** A complete answer covers: (1) rollout phase-sample G responses per prompt, compute rewards and $\log \pi_{\text{old}}$ from a single forward pass; (2) advantage phase-normalize within group by subtracting mean and dividing by std plus ϵ ; (3) inner loop-compute importance ratios with a stop-gradient on $\log \pi_{\text{old}}$, apply clipped surrogate loss, optionally add KL penalty against π_{ref} , take gradient step; (4) repeat inner loop K times, then refresh rollouts.
- **Senior follow-up: “What happens to GRPO training when the model solves nearly all prompts in the training set?”** This is the curriculum collapse scenario. As best-of- G pass rate approaches 100% across most prompts, group standard deviation approaches zero everywhere. Advantages collapse (or explode with $1/\sigma$ normalization) and training stalls despite the model still having room to improve on unseen distributions. The correct response is dynamic difficulty filtering: continuously measure pass rates, remove prompts above roughly 90% success, and inject harder prompts. Without this, GRPO training terminates not because the model is optimal but because the fixed training set has been exhausted.

Key Takeaways

- GRPO eliminates the PPO value network by replacing GAE with within-group reward normalization: $\hat{A}_i = (R_i - \bar{R}) / (\sigma_R + \epsilon)$, saving roughly 100% of critic parameter and optimizer-state memory.

- The PPO-clip objective and KL regularization are otherwise identical to PPO; the only structural change is the advantage estimator.
- The control-variate KL estimator $(\pi_{\text{ref}}/\pi_\theta) - \log(\pi_{\text{ref}}/\pi_\theta) - 1$ equals $D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$ in expectation with lower per-sample variance than the standard Monte Carlo form.
- Store π_{old} as a log-probability array (not a full model copy) to avoid a second model’s worth of memory during the inner gradient loop.
- Monotonically increasing response length signals the length-normalization incentive pathology; curriculum collapse when group $\sigma_R \rightarrow 0$ is the end-of-training failure mode—both require active monitoring and intervention.

28.3 Case Studies in Reasoning Models

28.3.1 DeepSeek-R1 and R1-Zero

When an interviewer asks you to design a reasoning model from scratch, they are really probing whether you understand the interaction between supervised fine-tuning and reinforcement learning with verifiable rewards. DeepSeek R1 and its controlled sibling R1-Zero are the clearest public demonstrations of this pipeline working end-to-end: released around the same time as OpenAI o1, they matched its performance on competitive math benchmarks using a recipe that is, in retrospect, surprisingly simple. This topic appears in senior-level design interviews at frontier labs because the engineering decisions—reward design, training stability, the role of cold-start data—reveal whether you can reason carefully about what each component actually contributes.

R1-Zero is the controlled experiment. Take DeepSeek V3 (the pretrained and mid-trained base model, before any instruction tuning or RLHF), and run pure RL using GRPO with verifiable rewards. No chain-of-thought (CoT) fine-tuning beforehand. No search or process supervision. Two rewards: a binary accuracy reward ($r \in \{0, 1\}$ for correct or incorrect answers on math and coding tasks) and a format reward that incentivizes the model to place its reasoning inside `<think> ... </think>` tags. The format reward feels like a cosmetic detail but turns out to be structurally important: without it, long CoTs do not emerge reliably from RL.

R1 is the production recipe. The pipeline has three stages:

1. SFT on long CoT data to cold-start the reasoning format.
2. RL with GRPO, using a binary accuracy reward, the format reward, and an added language consistency reward.
3. Standard post-training: instruction SFT combining reasoning and non-reasoning data, followed by RLHF using GRPO.

The GRPO advantage for response i within a group G of rollouts on the same prompt is:

$$\hat{A}_i = \frac{r_i - \frac{1}{G} \sum_{j=1}^G r_j}{\text{std}(\{r_j\}_{j=1}^G) + \varepsilon}, \quad \varepsilon = 10^{-4}$$

The policy is then updated via the clipped surrogate objective inherited from PPO:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E} \left[\min \left(\rho_i \hat{A}_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right] + \beta D_{\text{KL}}(\pi_\theta \parallel \pi_{\text{ref}})$$

where $\rho_i = \pi_\theta(o_i|q)/\pi_{\theta_{\text{old}}}(o_i|q)$ is the likelihood ratio and $\epsilon = 0.2$ is the clip range.

The minimal viable RLVR recipe is a strong base model, binary correct/incorrect rewards on verifiable tasks, a format reward to stabilize CoT structure, and GRPO with group-normalized advantages.

Verifiable rewards eliminate the feedback loop that causes reward overoptimization in RLHF. When the reward is “is the math answer correct?”, it cannot be hacked: either the symbolic expression matches the refer-

ence, or it does not. There is no proxy reward model to overfit, and the signal does not degrade as you optimize. This is the core reason RLVR scales where RLHF stalls.

The SFT cold-start in R1 serves a specific purpose: it gives RL a reasonable starting distribution to explore from. Without it (R1-Zero), long CoTs do emerge eventually, but they can be incoherent and language-mixed-the model will switch mid-reasoning from English to Chinese and back, because RL has no incentive to stay in a single language. The language consistency reward in R1 provides that incentive explicitly. Some practitioners have found that even 1,000 long-CoT examples for SFT cold-start are enough to stabilize the format before RL.

Two clear negative results from the R1 paper deserve equal weight. Process reward models (PRMs), which provide intermediate per-step feedback on reasoning chains, did not outperform simple outcome-based rewards despite earlier promising results in the DeepSeekMath work. Monte Carlo Tree Search did not help either. The simpler, cheaper approach won: GRPO with outcome-only rewards, no tree search, no step-level annotation.

A Worked Example

Walk through the R1 training run with realistic numbers.

1. **Base model:** DeepSeek V3, a 671B Mixture-of-Experts model with 37B active parameters per forward pass.
2. **SFT stage:** Fine-tune on a set of long CoT trajectories (source not disclosed in the paper) to prime the model for structured reasoning.
3. **RL stage:** GRPO with $G = 8$ rollouts per prompt. Accuracy reward is binary: $r \in \{0, 1\}$. Clip ratio $\epsilon = 0.2$, KL coefficient $\beta = 0.001$.
4. **Advantage computation example:** Suppose a math problem yields 8 rollouts, 5 correct and 3 incorrect. Mean reward $\bar{r} = 5/8 = 0.625$, standard deviation ≈ 0.484 .
 - Correct rollout: $\hat{A} = (1 - 0.625)/0.484 \approx +0.77$. Policy gradient *increases* probability of this response.
 - Incorrect rollout: $\hat{A} = (0 - 0.625)/0.484 \approx -1.29$. Policy gradient *decreases* probability.

For a trivially easy problem (all 8 correct), $\bar{r} = 1$ and $\text{std} \rightarrow 0$, making $\hat{A}$ numerically large. This is the bias introduced by standard deviation normalization, which the Dr. GRPO analysis identifies as distorting training on near-uniform reward groups.
5. **Results:** DeepSeek R1 matches OpenAI o1 across most benchmarks. On AIME 2024 it reaches approximately 79% pass@1; on MATH-500 it is near ceiling at 97%.
6. **Distillation:** Taking roughly 800,000 long-CoT traces from R1 and fine-tuning Qwen 32B via standard SFT on those traces lifts the student from $\sim 50\%$ to $\sim 72\%$ on AIME 2024. This is pure supervised distillation; no RL is needed for the student model.

What You Actually Decide in Practice

- **Always do SFT cold-start before RL, even if it is small.** R1-Zero is a research artifact; it language-mixes and ships poorly. Even 1,000 long-CoT examples stabilize the format. Budget data collection time before touching RL.
- **The format reward is load-bearing.** It seems cosmetic but multiple practitioners report that without rewarding the model for placing reasoning inside structured tags, GRPO does not reliably produce long CoTs. Add it from the start; do not treat it as optional.

- **Add a language consistency reward.** Aggressive RL on multilingual base models causes language mixing in CoTs. This is an optimization pressure artifact, not a base model defect. An explicit reward signal to stay in the user’s language is necessary for production.
- **Do not invest in PRMs or MCTS until outcome rewards plateau.** R1 tried both extensively and found neither improved over simple GRPO with outcome rewards. PRMs require expensive step-level annotation; MCTS adds infrastructure complexity. Optimize the simple thing first.
- **SFT quality dominates RL algorithm choice.** Empirical work (AI2’s Tulu 3) showed that improving the SFT stage closes most of the gap between DPO and PPO. If your reasoning RL is underperforming, check the SFT data before switching RL algorithms.

How This Shows Up in Interviews

- **“Walk me through the training pipeline for a model like DeepSeek R1.”** Strong answer: (1) start with a strong pretrained base model; (2) SFT on long CoT data to cold-start reasoning format; (3) GRPO RL with binary accuracy reward, format reward, and language consistency reward on verifiable tasks; (4) post-training SFT combining reasoning and general data; (5) RLHF using GRPO for instruction following. Make explicit that the RL stage in (3) is distinct from the RLHF stage in (5).
- **“What reward signals would you use to train a reasoning model, and why not process rewards?”** Strong answer: binary accuracy reward (correct/incorrect), format reward (structured tags), language consistency reward. PRMs require step-level human or model annotation and tend to be gamed; outcome rewards are fully self-supervised and, in the R1 experiments, sufficient.
- **Senior follow-up: “R1-Zero shows CoT length growing during training and backtracking emerging. Is this evidence of genuine reasoning emergence?”** Senior answer: treat this claim skeptically. The length growth is at least partially explained by the biased GRPO objective-standard deviation normalization upweights near-zero-variance problems (all correct or all incorrect), and the length normalization that vanilla GRPO applies creates perverse incentives to produce longer responses when the answer is wrong. Backtracking phrases like “wait, let me reconsider” appear in the base model at temperature above zero as well. R1-Zero produces a strong math model; attributing the CoT behavior to emergent reasoning rather than optimization artifacts requires stronger evidence than the paper provides.

Key Takeaways

- R1-Zero demonstrates that pure GRPO RL on verifiable rewards is sufficient for strong math reasoning; R1 adds SFT cold-start and language consistency rewards for production stability.
- The minimal reward design is binary accuracy plus format reward; language consistency reward prevents CoT language mixing under aggressive RL.
- Process reward models and Monte Carlo Tree Search both failed to outperform simple outcome-based GRPO in the R1 experiments.
- Distillation via SFT on ~800,000 R1 CoT traces transfers most of the reasoning gain to a smaller student model without any RL.
- SFT data quality dominates RL algorithm choice: fixing the supervised stage routinely closes gaps that switching between RL algorithms cannot.

28.3.2 Kimi K1.5 and Qwen3

Kimi K1.5 and Qwen 3 arrived within weeks of DeepSeek R1, both independently matching OpenAI o1 on competitive benchmarks. Studying them alongside R1 is instructive precisely because they solve the same problem with different algorithmic choices and different engineering emphases, revealing which design decisions are load-bearing and which are incidental. In a system design interview, after you have described the R1 recipe, the follow-up is often: “Is there another way to do this?” or “How would you control the inference cost of a reasoning model?” These two papers are your answer.

Kimi K1.5 derives its RL objective from the same KL-regularized reward maximization problem as DPO, but without pairwise preferences. The Moonshot team starts from the optimality condition for the constrained policy:

$$r(q, o) = \beta \log \frac{\pi^*(o | q)}{\pi_{\text{ref}}(o | q)} + \beta \log Z(q)$$

Since $Z(q)$ is a normalizing constant that cannot be computed in practice, they treat the right-hand side as a target and minimize a squared loss between the observed reward and the model-implied log ratio. Taking the gradient of this squared loss yields an update rule functionally equivalent to GRPO: a policy gradient term scaled by a baselined reward, plus an explicit regularization term pulling the policy toward the reference. The key differences from GRPO are that standard deviation normalization is absent (avoiding the GRPO bias on near-uniform reward groups) and the regularization is an explicit squared-loss penalty rather than clipping.

On top of this RL objective, Kimi adds a *length reward* to directly control CoT verbosity:

$$\lambda = 0.5 - \frac{|o| - L_{\min}}{L_{\max} - L_{\min}}, \quad r_{\text{len}}(o, c) = \begin{cases} \lambda & \text{if } c = 1 \\ \min(\lambda, 0) & \text{if } c = 0 \end{cases}$$

where $L_{\min}$ and $L_{\max}$ are the shortest and longest rollout lengths in the current batch, $|o|$ is the response length, and $c \in \{0, 1\}$ is the correctness indicator. Correct answers are rewarded for being short; incorrect answers are weakly penalized for being long.

Qwen 3 follows the R1 pipeline closely-SFT cold-start, reasoning RL, post-training RLHF-but adds *thinking mode fusion*: after RL, the model is fine-tuned via SFT to respond to a `<think>` tag with a full CoT and to a `no_think` tag with a direct answer. A single set of model weights serves both modes. Critically, mid-CoT generation can be interrupted by injecting a special prompt string, causing the model to emit its best current answer before the full reasoning budget is exhausted.

The two innovations that distinguish these papers from R1 are the length reward (Kimi) and dynamic inference budget control via thinking mode fusion (Qwen 3).

The Kimi length reward addresses the most practical objection to long-CoT reasoning models: inference cost. Without explicit length incentives, GRPO training creates a perverse incentive-when a rollout is incorrect and receives negative reward, the cheapest escape is to produce a longer response, diluting the per-token penalty under length-normalized objectives. The Kimi reward breaks this by making *correct and short* the uniquely rewarded combination, while incorrect responses are incentivized toward batch-average length rather than allowed to grow unboundedly. The reward must be introduced only after initial RL convergence; adding it at the start traps the model in a short-answer local minimum before it has learned to solve problems at all.

Qwen 3’s thinking mode fusion is a pragmatic response to the same problem from the serving side. Rather than training and deploying separate thinking and non-thinking models-doubling storage, serving infrastructure, and operational complexity-a single model can serve both modes. The mid-generation interrupt is particularly useful for latency-constrained applications: set a maximum thinking-token budget, inject the interrupt string

at that boundary, and the model produces a valid answer based on partial reasoning rather than timing out or truncating incoherently. Qwen 3’s ablation confirms graceful degradation: performance drops smoothly as the thinking budget shrinks from unconstrained to zero, rather than falling off a cliff.

A Worked Example

Kimi length reward arithmetic. Suppose a training batch produces rollouts of lengths 512, 1,024, 2,048, and 4,096 tokens. Then $L_{\min} = 512$ and $L_{\max} = 4,096$.

- Correct answer at 512 tokens: $\lambda = 0.5 - (512 - 512)/(4096 - 512) = +0.5$. Length reward = +0.5.
- Correct answer at 4,096 tokens: $\lambda = 0.5 - (4096 - 512)/(4096 - 512) = -0.5$. Length reward = -0.5.
- Incorrect answer at 4,096 tokens: $\lambda = -0.5$; since $c = 0$, $r_{\text{len}} = \min(-0.5, 0) = -0.5$.
- Incorrect answer at 512 tokens: $\lambda = +0.5$; since $c = 0$, $r_{\text{len}} = \min(+0.5, 0) = 0$.

The total reward is $r_{\text{acc}} + r_{\text{len}}$. A correct short answer gets +1.5; a correct long answer gets +0.5; an incorrect short answer gets 0; an incorrect long answer gets -0.5. The optimization pressure is unambiguous: be correct and concise.

Qwen 3 RL scale. The Qwen 3 RL stage uses just 3,995 training problems—a surprisingly small number. Despite this, the RL stage delivers measurable gains on math and STEM benchmarks relative to the SFT-only model. This illustrates that RLVR is highly sample-efficient on verifiable tasks: the model already knows much of the underlying reasoning; RL is refining the strategy for applying it.

Thinking mode fusion data construction. After the RL stage, construct two types of SFT examples:

- Thinking mode: `<think> [long CoT from the RL model] </think> [final answer]`.
- Non-thinking mode: `<no_think> [direct answer, no CoT]`.

Fine-tune the RL model on a mix of both. After this supervised stage, the model respects the tag semantics and, when generating in thinking mode, will respond coherently to the interrupt string by emitting an answer based on its partial reasoning.

What You Actually Decide in Practice

- **Gate the length reward on only after RL stabilizes.** Adding the Kimi length reward from step one forces the model into a short-answer local minimum before it has learned to solve anything. In practice, run unconstrained RL until reward curves show clear upward progress, then introduce the length reward. A rough heuristic: wait until mean accuracy reward on the training distribution exceeds 0.3.
- **Use a learned judge for answer equivalence, not only regex.** Kimi uses a fine-tuned reward model for math answer comparison rather than relying solely on SymPy or regex. Notation variants (e.g., $\frac{1}{2}$ vs. 0.5 vs. 1/2) break naive string matching at scale. A model trained specifically on equivalence checking is more robust and can be made very accurate because the task is narrow.
- **Separate inference and training workers.** RL training requires alternating between rollout generation (inference) and parameter updates (training). Running both on the same GPUs causes memory thrashing and poor utilization. The standard architecture has a pool of inference workers running vLLM and a pool of training workers, with weight updates broadcast via NCCL or by restarting the inference server. Budget for this infrastructure; it is the hardest engineering component in reasoning model RL.
- **Filter data by difficulty before RL.** Both Kimi and Qwen 3 exclude problems where the untuned model can solve at least one of eight rollouts (pass rate $\geq 1/8$). Spending compute on problems the model

already handles is waste. Continually re-evaluate the training set difficulty as the model improves and prune easy problems dynamically.

- **Expect RLHF post-training to trade math for instruction following.** Qwen 3’s ablation shows the general RLHF stage improves instruction-following and general tasks but slightly hurts math and STEM scores in thinking mode. This is not a bug-the RLHF objective does not reward math accuracy-but it is a trade-off to design for explicitly when building a multi-capability model.

How This Shows Up in Interviews

- **“How would you control the inference cost of a reasoning model?”** Strong answer: add a length reward during RL that incentivizes correct-and-short responses over correct-and-verbose ones; introduce it after initial convergence to avoid the short-answer local minimum. For serving, add thinking mode fusion so that operators can cap the thinking-token budget at inference time, getting graceful degradation rather than abrupt failure.
- **“What are the infrastructure bottlenecks when training a reasoning model with RL?”** Strong answer: three main bottlenecks. First, rollout generation blocks training; separate inference and training worker pools with asynchronous message passing. Second, variable CoT lengths create uneven batches and GPU underutilization; implement sequence packing or per-length-bucket batching. Third, syncing updated weights from the training process to the vLLM inference server is non-trivial; current approaches either use an experimental NCCL weight-loading API or tear down and restart vLLM each iteration.
- **Senior follow-up: “Kimi K1.5 and DeepSeek R1 both work, but with different RL objectives. How do you choose between them?”** Senior answer: both converge to the same structure-policy gradient scaled by a baselined reward, plus regularization toward a reference policy-and empirical differences between them are smaller than differences introduced by data quality and reward design. The more consequential choice is whether to add a length reward (Kimi has it; vanilla GRPO does not) and whether to use standard deviation normalization in the advantage (the Dr. GRPO analysis argues against it). In practice, start with GRPO for familiarity, add a length reward after initial convergence, and remove standard deviation normalization if you observe training instability on near-uniform reward groups.

Key Takeaways

- Kimi K1.5 derives its RL objective from the DPO optimality condition without pairwise preferences, arriving at a policy gradient update similar to GRPO but without standard deviation normalization.
- A length reward is essential to prevent unbounded CoT growth; add it only after initial RL convergence, not from the start, to avoid trapping the model in a short-answer local minimum.
- Qwen 3’s thinking mode fusion trains a single model to serve both thinking and non-thinking requests, enabling graceful performance degradation under a token budget at inference time.
- RLVR is highly sample-efficient: Qwen 3 achieves strong reasoning gains from fewer than 4,000 training problems.
- The dominant infrastructure challenge in reasoning model RL is weight synchronization between training and inference workers; allocate engineering time for this before starting large RL runs.

Part XI

End-to-End System Design Drills

Chapter 29 Building a Production LLM Serving Stack

29.1 Reference Architecture

29.1.1 Gateway, Router, and Autoscaler

When an interviewer asks you to “design a production LLM serving system,” what they want to see is not a single GPU running a model loop. They want the full stack: the layer that faces the internet, the layer that decides which replica handles each request, and the layer that expands or contracts the fleet as traffic shifts. This question appears at every senior systems or ML-infra interview at frontier labs, and candidates who conflate “inference efficiency” with “serving architecture” rarely get the offer. The two concerns interact, but they live at different abstraction levels, and you need to be fluent in both.

A production LLM serving stack decomposes into three collaborating planes. The *gateway* is the public-facing API endpoint: it authenticates callers, enforces rate limits, validates payloads, and streams tokens back to clients using server-sent events or WebSocket connections. The *router* sits behind the gateway and is responsible for request dispatch: given an incoming prompt, it selects a backend replica (or a prefill/decode worker pair in disaggregated setups) and hands the request off. The *autoscaler* monitors aggregate queue depth, GPU utilization, and SLO attainment and adjusts replica count accordingly.

The three components share a common data plane: a token stream flowing from the inference workers back through the router and out the gateway, and a control plane: health signals, load metrics, and scheduling decisions flowing in the other direction. In practice, the gateway is often a thin reverse proxy (Nginx, Envoy, or a cloud load balancer), the router is a custom service aware of model-specific state (particularly KV cache residency), and the autoscaler is a policy engine running against a metrics store.

Two metrics govern every SLO conversation at this layer. *Time-to-first-token* (TTFT) measures the wall-clock gap between request arrival and the moment the client receives the first generated token. TTFT is dominated by prefill latency because the model must encode the entire prompt before it can emit anything. *Inter-token latency* (ITL), sometimes called decode latency, measures the gap between successive tokens after the first. ITL is dominated by memory bandwidth because generation is memory-bound, not compute-bound. Throughput—tokens generated per second across all concurrent users—is the aggregate metric the operator cares about when planning capacity.

Every architectural decision in the serving stack is ultimately a negotiation between TTFT, ITL, and throughput, and you cannot improve all three simultaneously.

Separating gateway, router, and autoscaler is not bureaucratic over-engineering. Each component has a radically different scaling profile. The gateway needs horizontal scale proportional to request rate; it is stateless and trivially replicated. The router needs to carry soft state: knowledge of which replicas have which KV blocks cached, what their current queue depths are, and what prefix hashes look like. Merging router logic into the gateway produces a monolith that cannot scale the two concerns independently. The autoscaler, meanwhile, is a control loop with a multi-minute decision horizon; it has no business sitting on the hot path of individual requests.

The deeper argument for this decomposition comes from the physics of GPU inference. Prefill (encoding the prompt) is compute-bound: given a long enough sequence the arithmetic intensity exceeds the H100’s compute-to-bandwidth ratio of roughly 295, so the GPU stays busy. Decode (generating tokens one at a time) is

memory-bound: with batch size 1 the arithmetic intensity collapses to approximately 1 FLOP per byte, nearly $300\times$ below what the hardware can sustain. This asymmetry means the optimal allocation of GPU time for prefill and decode is very different, and a router that is unaware of which stage a request is in will make poor dispatch decisions. The alternative—a single monolithic worker that handles both phases—wastes GPU-hours on the slower decode phase and creates queuing pressure for new arrivals whose TTFT suffers while they wait behind long decode tails.

A Worked Example

Consider a 13B-parameter model served on H100 SXM5 GPUs (80 GB HBM, 3.35 TB/s memory bandwidth, 989 TFLOPS BF16). Model parameters in BF16 consume $13 \times 10^9 \times 2 = 26$ GB. Each H100 can hold the model and leave roughly 50 GB for KV cache.

The KV cache per sequence is:

$$\text{KV bytes} = 2 \times N_{kv} \times H \times L \times T \times 2$$

where N_{kv} is the number of key/value heads (for a vanilla 13B with 40 heads, $N_{kv} = 40$), $H = 128$ (head dimension), $L = 40$ layers, T is the sequence length (1024 tokens), and the factor of 2 at the front accounts for both key and value, and 2 bytes for BF16. This yields $2 \times 40 \times 128 \times 40 \times 1024 \times 2 \approx 838$ MB per sequence. With 50 GB free, a single GPU can hold at most around $50,000/838 \approx 59$ sequences concurrently.

At batch size 1, decode latency is:

$$\text{latency} = \frac{\text{parameters bytes} + \text{KV bytes}}{\text{bandwidth}} = \frac{26,000 + 838 \text{ MB}}{3,350 \text{ GB/s}} \approx 8 \text{ ms per token}$$

Throughput is $1000/8 = 125$ tokens/second for one sequence. At batch size 32, the memory footprint becomes $26,000 + 32 \times 838 \approx 52,800$ MB and latency is $52,800/3,350,000 \approx 15.8$ ms, but throughput is $32/0.0158 \approx 2,025$ tokens/second—a $16\times$ throughput gain at the cost of $2\times$ latency. This arithmetic is what the autoscaler must internalize: adding replicas is the right move when queue depth is climbing and per-request latency is still within SLO; increasing per-replica batch size is the right move when replicas have headroom but throughput targets are not met.

What You Actually Decide in Practice

- **TTFT SLO drives prefill isolation.** If your TTFT SLO is under 500 ms and prompt lengths routinely exceed 4k tokens, you will almost certainly need to disaggregate prefill onto dedicated high-compute nodes and decode onto high-memory-bandwidth nodes. A single fleet trying to do both will violate TTFT under bursty load because long decodes hold the GPU while new prefills queue.
- **Router state granularity matters.** A stateless round-robin router wastes KV cache: the same system prompt gets re-encoded on every replica. A prefix-hash-aware router routes requests with shared prefixes to the replica that already holds those KV blocks, cutting TTFT for chat history, RAG context, and system prompts to near zero for cache hits. The cost is router complexity and the need for a fast prefix-hash store (typically a Redis or in-memory hash table).
- **Autoscaler lag is your biggest latency risk.** GPU instances take 2-5 minutes to start and load a 13B model. If you scale reactively, you will absorb a traffic spike entirely as latency degradation. Scale proactively: watch the ratio of queue depth to mean decode time per step. If that ratio exceeds your replica's max concurrent sequence count, trigger scale-out before queues overflow.

- **Continuous batching, not static batching.** Static batching waits until a fixed batch fills before dispatching. With variable-length completions this wastes GPU cycles waiting for stragglers. Continuous batching (as implemented in vLLM) inserts new requests into a running decode step as soon as a sequence slot frees, keeping the batch dense at all times. This is non-negotiable for production.
- **Model replicas scale throughput linearly; tensor parallelism does not.** Launching M independent copies of the model across M GPUs multiplies throughput by M at constant latency. Sharding one model across M GPUs via tensor parallelism reduces per-request latency but introduces all-reduce communication overhead and does not scale throughput unless you also add replicas. Use tensor parallelism only when the model does not fit on a single GPU; prefer replicas otherwise.

How This Shows Up in Interviews

- **“Walk me through what happens when a user hits your LLM API endpoint.”** Strong answers name each layer: TLS termination and auth at the gateway, routing policy at the router, prefill on a dedicated or shared worker, incremental token streaming back through the gateway, and where TTFT and ITL are measured. Weak answers start at the GPU and work inward.
- **“How would you reduce TTFT for a system serving 10k concurrent users?”** A strong answer distinguishes the two root causes: long prompts slow prefill (fix with disaggregated prefill workers or prompt compression), while large queue depths slow dispatch (fix with more replicas or better scheduling priority for new arrivals). A weak answer says “use a faster GPU.”
- **“How does your router avoid recomputing the same system prompt on every request?”** The answer is prefix caching: hash the system prompt tokens, store the associated KV blocks on each replica, and route requests to the replica holding the match. State that cache eviction follows an LRU policy and that the KV store must be block-granular (as in PagedAttention) to avoid pinning entire sequences.
- **Senior follow-up: “Your autoscaler is scaling in correctly but P99 TTFT is still spiking every 30 minutes. What’s happening?”** The senior answer diagnoses a garbage-collection analogue: KV cache eviction storms. As sequences complete, their blocks are freed; if the scheduler allows too many long-context requests to pile up simultaneously, memory pressure forces the eviction of partially-warm cache blocks mid-batch, causing re-prefill for evicted sequences and a TTFT spike. The fix is admission control at the router: cap the number of sequences admitted based on remaining KV capacity, not just compute capacity.

Key Takeaways

- The production serving stack has three separable planes-gateway, router, autoscaler-each with a distinct scaling profile and state requirement.
- TTFT is dominated by prefill (compute-bound); ITL is dominated by decode (memory-bound); these two phases have different hardware optima, which motivates disaggregation.
- At batch size 1 on an H100, a 13B model generates tokens roughly every 8 ms; batching to 32 costs $2\times$ latency but delivers $16\times$ throughput-the autoscaler must understand this curve.
- Prefix-hash-aware routing is the single highest-leverage router optimization: it converts redundant prefill work into cache hits with no accuracy cost.
- Continuous batching and admission control (capping inflight sequences by KV capacity) are non-negotiable for production; static batching and reactive scaling will both fail under real traffic patterns.

29.1.2 KV-Cache-Aware Scheduling

A scheduler that treats GPU memory as a black box will, under real traffic, either starve low-latency requests waiting behind long-context ones or thrash the KV cache into a pattern of constant eviction and re-prefill. Both failure modes show up as P99 TTFT spikes that no amount of extra hardware fixes. KV-cache-aware scheduling is the discipline of making dispatch, admission, and eviction decisions that account for exactly how many physical KV blocks each in-flight sequence holds and how much capacity each candidate replica has left. It is a systems question that appears at senior and staff interviews whenever the conversation moves from “how does inference work” to “how do you run it at scale.”

The KV cache is the dominant memory consumer during decode. For each in-flight sequence the system must store one key tensor and one value tensor per layer, per head, per token seen so far. In BF16 the per-token cost per layer is:

$$\text{bytes per token per layer} = 2 \times N_{kv} \times H \times 2$$

where N_{kv} is the number of key/value heads, H is the head dimension, and the leading factor of 2 counts both K and V. Summed over L layers, a sequence of length T occupies $2 \times N_{kv} \times H \times L \times T \times 2$ bytes. Because this scales linearly with T , a single 32k-token sequence can consume as much KV memory as dozens of 1k-token sessions.

The core insight of KV-cache-aware scheduling is that *admitting a new request is an irreversible memory commitment until the sequence finishes or is explicitly preempted*. A scheduler that does not track block occupancy will over-admit, forcing the system to either stall (waiting for a slot to free) or evict a partially-decoded sequence and re-prefill it from scratch, wasting the compute already spent. The PagedAttention abstraction (used in vLLM and its successors) makes the accounting tractable: KV memory is partitioned into fixed-size *blocks* (typically 16 tokens wide), and the scheduler tracks a free-block count rather than raw bytes. A request is admitted only when enough free blocks exist to hold its full worst-case KV footprint, or when a preemption policy frees sufficient blocks first.

Prefix caching extends this further. Many requests share a common prefix—a system prompt, a RAG context, or a few-shot template. If the KV blocks for that prefix are already resident on a replica, the prefill cost for the shared portion is zero. The scheduler therefore maintains a *prefix hash table*: a mapping from a rolling hash of the token sequence to the physical block addresses that hold the corresponding KV vectors. When a new request arrives, the scheduler computes its prefix hash and routes it to the replica with the longest matching cached prefix, turning prefill into a cache hit.

Admission control, block-level eviction policy, and prefix-aware routing are three levers that together determine whether the KV cache operates near its theoretical efficiency or wastes the majority of its capacity on redundant computation.

The alternative-FIFO dispatch without block accounting fails in two characteristic ways. First, a long-context request admitted at the front of the queue holds tens of thousands of blocks for minutes; shorter requests queue behind it even though the GPU has cycles to spare on a smaller batch. Second, when total block occupancy approaches the physical limit, a naively greedy system must either halt new admissions (starving throughput) or evict and re-prefill the longest resident sequence (wasting $O(T)$ compute that has already been paid).

Block-level accounting solves the first problem by decoupling scheduling from request order: the dispatcher picks from the ready queue whichever request best fills available block capacity without over-committing. It solves the second by making eviction explicit and policy-driven. The standard policy is a modified LRU over *logical sequences*: when blocks must be freed, the scheduler selects the sequence with the lowest priority score

(often a combination of elapsed wait time, sequence length, and whether the sequence has a live client connection). In practice, background batch jobs are prime eviction candidates; interactive sessions are protected.

Prefix caching compounds the gains. A system-prompt block that is reused across 1000 sessions is computed exactly once. Without prefix routing, each session independently prefills that prompt, turning a constant cost into a cost proportional to request rate. At 10k requests per minute with a 2k-token system prompt at 8 ms per prefill, unrouted sessions waste roughly $10,000 \times 8/60 \approx 1,333$ GPU-ms per minute on a single replica-compute that otherwise could have served decode steps.

A Worked Example

Take a Llama-3-8B model (GQA with $N_{kv} = 8$ key/value heads, $H = 128$, $L = 32$ layers) running on an H100 with 80 GB HBM. Parameters in BF16 consume $8 \times 10^9 \times 2 = 16$ GB. KV cache budget is $80 - 16 - 2 = 62$ GB (reserving 2 GB for activations and workspace).

KV bytes per token:

$$2 \times 8 \times 128 \times 32 \times 2 = 131,072 \text{ bytes} \approx 128 \text{ KB per token}$$

With a block size of 16 tokens, each block holds $16 \times 128 \text{ KB} = 2 \text{ MB}$ of KV data. The 62 GB budget supports:

$$\left\lfloor \frac{62 \times 1024 \text{ MB}}{2 \text{ MB}} \right\rfloor = 31,744 \text{ blocks}$$

A request with a 4k-token prompt plus a 1k-token max completion requires $\lceil 5000/16 \rceil = 313$ blocks at peak. The scheduler can safely admit $\lfloor 31,744/313 \rfloor = 101$ such requests before the H100 is fully committed. Admitting request 102 without eviction forces an out-of-memory stall.

Now add prefix caching. Suppose all requests share a 1k-token system prompt ($\lceil 1000/16 \rceil = 63$ blocks). After the first request warms those blocks, every subsequent request needs only $\lceil 4000/16 \rceil = 250$ blocks for its unique portion. Effective capacity rises to $\lfloor (31,744 - 63)/250 \rfloor = 126$ concurrent requests—a 25% capacity gain with no hardware change.

What You Actually Decide in Practice

- **Block size is a latency-fragmentation tradeoff.** Smaller blocks (8 tokens) reduce internal fragmentation but increase block-table overhead and TLB pressure. Larger blocks (32 tokens) cut overhead but waste memory when sequences end mid-block. Most production systems settle at 16 tokens; drop to 8 only for extreme short-context workloads where fragmentation dominates.
- **Set admission headroom, not a hard limit.** Admitting up to the last block is dangerous: decode steps allocate one new block per step per sequence, so a fully-packed batch will OOM mid-step. Reserve a safety margin of 5-10% of total blocks for in-flight growth. The scheduler should refuse new admissions when free blocks fall below this watermark, not when they reach zero.
- **Preemption policy must match your SLO tier.** For interactive users, never preempt; instead, queue the incoming request. For batch jobs, preempt aggressively using swap-to-CPU (slower but preserves progress) or full eviction (faster but wastes prefill). Implement priority tiers at the request level and wire them into the eviction comparator.
- **Prefix hash granularity determines cache hit rate.** Hashing at the full-prompt level misses partial prefix matches. Hash at block granularity: compute a rolling hash after each block boundary and store a hit for

every prefix length that is a multiple of the block size. This maximises reuse when system prompts vary by a few appended tokens, which is common in multi-turn conversations.

- **Disaggregated prefill changes the scheduler contract.** When prefill workers are separate from decode workers, the scheduler must coordinate a handoff: the prefill worker streams KV blocks for the prompt into shared memory or over NVLink, and the decode worker picks up from there. The decode-side scheduler must reserve the necessary blocks *before* the handoff begins, or the arriving blocks have nowhere to land. Reserve-then-transfer, not transfer-then-find-space.

How This Shows Up in Interviews

- **“How does vLLM prevent out-of-memory errors at high load?”** Strong answers explain PagedAttention block accounting: the scheduler maintains a free-block counter and refuses admission when the counter minus the safety watermark is less than the worst-case block requirement of the incoming request. Eviction policy (LRU over logical sequences, swap vs. discard) should be named. Weak answers say “it manages memory better.”
- **“Two replicas are available. How do you route an incoming request?”** The strong answer: compute the prefix hash of the request’s prompt, check each replica’s prefix hash table for the longest matching cached prefix, and route to the winner. If neither has a match, route to the replica with the most free blocks. Fall back to round-robin only if block counts are equal and no prefix match exists. Naming the cost of a wrong routing decision—a full re-prefill—makes the answer concrete.
- **“Your P99 TTFT spikes every few minutes on an otherwise healthy cluster. What’s your diagnosis?”** The candidate should describe a KV eviction storm: the decode batch fills available blocks, admission control kicks in, new requests queue, and when a batch of long sessions finally completes it releases a burst of blocks that triggers a flood of newly admitted requests whose prefills saturate the prefill stage simultaneously. The fix is admission rate limiting: spread re-admissions over multiple scheduling steps rather than unblocking the entire queue at once.
- **Senior follow-up: “How would you implement prefix caching across replicas, not just within one?”** The senior answer introduces a distributed prefix store—a fast key/value service (Redis Cluster or a custom shard-by-hash service) mapping prefix hashes to the replica ID that holds the warm blocks. On cache hit across replicas, the router either redirects the request or triggers a block migration via RDMA. Block migration is worthwhile only when the shared prefix exceeds a threshold (roughly 512 tokens) because migration latency amortizes over the TTFT saving. Below that threshold, local re-prefill is faster.

Key Takeaways

- KV memory is block-granular and grows linearly with sequence length; the scheduler must track free blocks explicitly and refuse admission before—not after—the last block is consumed.
- Prefix caching converts repeated prefill work into cache hits; a Llama-3-8B H100 instance gains roughly 25% effective concurrency when a 1k-token system prompt is shared across all sessions.
- Eviction policy must respect SLO tiers: interactive sessions are never preempted; batch jobs are the first candidates for swap-to-CPU or full eviction.
- Block-granular prefix hashing maximises cache hit rate for multi-turn conversations where prompts differ only in appended tokens.
- In disaggregated prefill/decode deployments, block reservation must happen on the decode worker *before*

KV blocks are transferred from the prefill worker; the reverse order causes mid-transfer OOM and wastes the prefill compute.

29.2 Capacity Planning

29.2.1 Estimating Tokens-Per-Second per GPU

Capacity planning for a production serving system begins with a single derivable number: how many output tokens per second can one GPU produce for your model and workload? Every downstream decision-fleet size, cost projection, SLA commitment-anchors to this estimate. In interviews at frontier labs, the question arrives within the first ten minutes: “We need to serve a 70B model at 1,000 requests per second with p95 latency under 100 ms. How many H100s?” Candidates who reach for benchmark sheets fail; candidates who derive the answer from the physics of memory bandwidth on the whiteboard pass. This is a senior-level question because it requires internalizing why generation is fundamentally different from training.

Autoregressive generation is memory-bandwidth-bound. Every time the model generates a single token, the GPU must sweep all model weights from high-bandwidth memory (HBM) into registers, execute the forward pass, and write effectively nothing back—a deeply asymmetric, read-dominated workload. For a 13B-parameter model in BF16, that sweep transfers $13 \times 10^9 \times 2 = 26$ GB per token, per batch step, regardless of batch size.

To generate a token for a batch of B concurrent sequences, the GPU must also read each sequence’s KV cache. In BF16, the KV cache for one sequence of length S (tokens already generated or prefilled) is:

$$C_{\text{seq}}(S) = 4 \cdot L \cdot K \cdot H \cdot S \quad \text{bytes}$$

where L is the number of transformer layers, K is the number of KV heads, H is the head dimension, and the factor of 4 combines 2 bytes per element (BF16) with 2 tensors (key and value). The total bytes transferred per generation step for a batch of size B is:

$$M(B, S) = 2P + B \cdot C_{\text{seq}}(S)$$

where P is the parameter count. Given hardware memory bandwidth W (bytes per second), the per-step latency is $M(B, S)/W$, and batch throughput in tokens per second is:

$$\text{TPS}(B, S) = \frac{B \cdot W}{2P + B \cdot C_{\text{seq}}(S)}$$

This formula has two regimes. When $B \cdot C_{\text{seq}} \ll 2P$ -small batch, weights dominate-TPS scales linearly with B . When $B \cdot C_{\text{seq}} \gg 2P$ -large batch, KV cache dominates-TPS saturates at the ceiling $W/C_{\text{seq}}(S)$, independent of any further increase in B .

To maximize GPU utilization during generation, the primary lever is batch size B ; every generated token forces a full sweep of model weights and all per-sequence KV caches from HBM.

The root cause is arithmetic intensity: the ratio of floating-point operations to bytes transferred. For an MLP projection with weight matrix of shape $D \times F$, processing a batch of B tokens requires $2BDF$ FLOPs and transfers roughly $2DF + 2BD + 2BF \approx 2DF$ bytes (when D and F dwarf B). The intensity simplifies to approximately B FLOP/byte. An H100 SXM5 has a ridge point of $989 \text{ TFLOP/s} \div 3.35 \text{ TB/s} \approx 295 \text{ FLOP/byte}$. You need $B \geq 295$ to be compute-bound on MLP layers alone.

Attention layers are worse. Each sequence maintains its own KV cache, so unlike MLP weights-shared across all batch members-the KV data scales with B . The arithmetic intensity of one attention step during generation is approximately $ST/(S + T) \approx 1$ when $T = 1$ (one new token per step), entirely independent of

batch size. No amount of batching recovers efficiency in the attention path during decode.

The practical consequence is stark. During prefill, both the prompt length $T = S$ and the batch B can be large, driving arithmetic intensity above the ridge point; the GPU is compute-bound and operates near peak FLOPs. During generation, $T = 1$, intensity collapses to roughly B for MLP and 1 for attention, and the GPU is almost always memory-bandwidth-bound. This asymmetry is why building a high-throughput decoder is fundamentally a batching problem, not a kernel-optimization problem.

A Worked Example

Take Llama-2-13B on a single H100 SXM5: $P = 13 \times 10^9$, $L = 40$ layers, $K = 40$ KV heads, $H = 128$ head dimension, $W = 3.35$ TB/s, 80 GB HBM. In BF16:

$$2P = 26 \text{ GB}, \quad C_{\text{seq}}(1,000) = 4 \times 40 \times 40 \times 128 \times 1,000 \approx 0.78 \text{ GB/seq}$$

Free HBM after loading weights: $80 - 26 = 54$ GB. Maximum concurrent sequences at $S = 1,000$ tokens: $\lfloor 54/0.78 \rfloor = 69$.

Batch B	Memory (GB)	Latency (ms/step)	TPS
1	26.8	8.0	125
16	38.5	11.5	1,391
64	75.9	22.7	2,819

Latency at $B = 1$: $(26 + 0.78) \text{ GB} / 3,350 \text{ GB/s} \approx 8.0 \text{ ms}$. At $B = 16$: $(26 + 12.5) / 3,350 \approx 11.5 \text{ ms}$. At $B = 64$ (near the memory ceiling): $(26 + 49.9) / 3,350 \approx 22.7 \text{ ms}$. Throughput is $B/\text{latency}$.

Raising the batch from 1 to 64 multiplies TPS by $22.6\times$ at the cost of $2.8\times$ higher per-request latency—a favorable trade for batch workloads, a painful one for interactive users who measure inter-token latency personally. The theoretical TPS ceiling as $B \rightarrow \infty$ is $W/C_{\text{seq}} = 3,350/0.78 \approx 4,295$ TPS; in practice, the memory limit of 69 concurrent sequences bounds you to about 2,900 TPS at best.

To serve 500 requests per second at an average output length of 200 tokens, you need $500 \times 200 = 100,000$ output TPS. At $\approx 2,900$ TPS per H100, the baseline fleet is $\lceil 100,000/2,900 \rceil = 35$ GPUs, plus 20-30% headroom for traffic bursts and prefill capacity.

What You Actually Decide in Practice

- **Target memory bandwidth, not peak FLOPs, when evaluating hardware.** Upgrading from an A100 (2.0 TB/s) to an H100 (3.35 TB/s) delivers a $1.7\times$ TPS increase for memory-bound generation. Switching from BF16 to INT4 weights halves bytes transferred per step, roughly doubling throughput—a larger gain than a full GPU generation upgrade. Profile your actual arithmetic intensity before committing to hardware SKUs.
- **Use continuous batching to keep B near B_{max} .** Static batching waits for a full batch to assemble before dispatching, leaving GPU memory idle while requests queue. Continuous batching inserts new requests into an active decode step the moment a sequence slot frees, maintaining dense GPU occupancy at all times. Under bursty traffic, continuous batching routinely delivers $2\times$ the throughput of static batching at the same hardware cost.
- **Disaggregate prefill and decode under sustained load.** A 4K-token prefill for one user stalls 64 decoding sequences for its full duration—roughly $4,000 \times 8 \text{ ms} \approx 32 \text{ s}$ of blocked inter-token latency at $B = 1$; less

at larger batches, but still a significant TTFT spike. At scale, route prefill requests to dedicated compute-heavy nodes and stream KV caches to decode-only nodes. This disaggregation is increasingly standard at 100B-parameter deployments.

- **Replicate before sharding for models under 80B.** A 13B model fits on a single H100 with 54 GB remaining for KV caches. Running 8 independent replicas delivers $8\times$ throughput at zero inter-GPU communication cost. An 8-way tensor-parallel shard reduces per-request latency but adds all-reduce overhead on every layer and does not improve aggregate throughput unless you also replicate the sharded group. Use tensor parallelism only when the model does not fit on a single GPU.
- **Budget 25-30% of HBM as system overhead.** KV cache fragmentation, CUDA context, flash-attention internal buffers, and OS overhead reliably consume this margin. An H100 with 80 GB is safely usable to about 56-60 GB for model and KV cache combined. Planning at 95% occupancy causes OOM crashes during traffic spikes and leaves no room for model updates without a full service restart.

How This Shows Up in Interviews

- **“How would you estimate how many H100s you need to serve a 70B model at 1,000 RPS?”** A strong answer derives TPS from first principles: compute $2P = 140$ GB (requires tensor parallelism across two H100s per replica), compute C_{seq} at expected context length, derive B_{max} per replica pair, then compute TPS per replica pair and divide into the required output TPS. State assumptions on average output length and context length explicitly before doing arithmetic.
- **“Why is autoregressive generation memory-limited rather than compute-limited?”** A strong answer names the mechanism: at $T = 1$ per step, MLP arithmetic intensity reduces to $\approx B$ FLOP/byte and attention intensity to ≈ 1 FLOP/byte; you need $B \geq 295$ on an H100 to reach the MLP compute ridge point. At $B = 1$ you are $295\times$ below compute saturation. Every additional user in the batch recovers efficiency linearly until the KV cache fills HBM and throughput saturates.
- **Senior follow-up: “Your batch is already at memory capacity. What are your levers to push TPS higher?”** The strong senior answer identifies two compounding levers: (1) reduce C_{seq} via GQA or MLA-fewer KV heads means larger B_{max} at the same HBM budget, so the same hardware now serves proportionally more users; (2) reduce $2P$ via quantization-INT8 weights halve the weight-sweep cost, shifting the crossover point where KV cache dominates, which allows doubling the batch size at the same per-step latency. Both are applied multiplicatively.

Key Takeaways

- Generation is memory-bandwidth-bound: $\text{TPS}(B, S) = B \cdot W / (2P + B \cdot C_{\text{seq}})$, where W is HBM bandwidth and $C_{\text{seq}} = 4LKH S$ is KV cache bytes per sequence.
- TPS grows linearly with B when weights dominate ($B \cdot C_{\text{seq}} \ll 2P$) and saturates at W/C_{seq} when KV cache dominates.
- Maximum batch size is $B_{\text{max}} = \lfloor (M_{\text{GPU}} - 2P) / C_{\text{seq}} \rfloor$; size fleets around this constraint, not peak FLOP specs.
- Continuous batching maintains near- B_{max} occupancy; static batching wastes capacity by idling while a batch assembles.
- For sub-80B models, prefer independent replicas over tensor parallelism: N replicas deliver $N\times$ throughput at zero communication overhead.

29.2.2 Budgeting for Long-Context Workloads

Teams that extend their serving context window from 4K to 32K tokens routinely discover their GPU fleet is suddenly undersized by an order of magnitude. The capacity estimate that held last quarter is now wrong by a factor of eight or more. Context length is not a dial you can turn freely after sizing your fleet; it restructures the entire HBM budget, shrinks the maximum batch size, and degrades throughput in a way that compounds rather than adds. Interviewers at infrastructure-focused teams probe this directly: “If we double context length, how does our serving capacity change?” The answer is never “it roughly halves.” Understanding precisely why-and by how much-separates a strong senior answer from a weak one.

The KV cache grows linearly with sequence length S , but its effect on serving capacity is superlinear because a longer cache simultaneously reduces the maximum concurrent batch size and increases the bytes transferred per generation step. The KV cache per sequence is:

$$C_{\text{seq}}(S) = 4 \cdot L \cdot K \cdot H \cdot S \quad \text{bytes}$$

With GPU HBM capacity M_{GPU} and model parameters consuming $2P$ bytes, the maximum batch size is:

$$B_{\text{max}}(S) = \left\lfloor \frac{M_{\text{GPU}} - 2P}{C_{\text{seq}}(S)} \right\rfloor \propto \frac{1}{S}$$

At maximum batch occupancy, nearly all HBM is consumed by weights and KV caches. The total memory swept per generation step is approximately M_{GPU} , so the per-step latency is roughly constant:

$$\text{latency}(S) \approx \frac{M_{\text{GPU}}}{W}$$

The batch throughput is therefore:

$$\text{TPS}(S) = \frac{B_{\text{max}}(S)}{\text{latency}(S)} \approx B_{\text{max}}(S) \cdot \frac{W}{M_{\text{GPU}}} \propto \frac{1}{S}$$

When a GPU is packed to its HBM limit, generation throughput scales inversely with context length: doubling S halves TPS, and the GPU stays fully occupied the entire time.

This $1/S$ relationship holds when the memory constraint is binding. At shorter contexts where you have fewer concurrent users than B_{max} -throughput is limited by request arrival rate, not hardware-the scaling breaks; context length barely matters because unused KV slots cost nothing.

The intuition is that HBM is a fixed pool shared between model weights and KV caches. At full batch occupancy, almost all of it contains KV data. Doubling S doubles C_{seq} , which forces you to halve B_{max} to fit in memory. The per-step latency stays approximately the same (you are still sweeping $\approx M_{\text{GPU}}$ bytes per step), but you produce half as many tokens per step. Throughput halves.

This stands in sharp contrast to prefill, where longer sequences actively help: a larger prompt length $T = S$ during encoding raises arithmetic intensity above the ridge point, keeping the GPU compute-bound. The degradation from long context is entirely a decode-phase problem.

Techniques that reduce $C_{\text{seq}}(S)$ attack this problem at the source. GQA with g -fold head reduction changes $K \rightarrow K/g$, shrinking C_{seq} by g and growing B_{max} by g -a proportional TPS improvement at every context length. MLA (Multi-head Latent Attention) projects KV tensors into a low-rank bottleneck of dimension $c \ll K \cdot H$, reducing the per-token KV footprint to $4Lc$ bytes; DeepSeek-V2 compressed from $K \cdot H = 16,384$ to $c = 512$, a $32\times$ reduction. Both techniques are most impactful at long context because that is when the KV cache, not the weights, is the binding constraint on batch size.

A Worked Example

Use Llama-2-13B on a single H100 ($M_{\text{GPU}} = 80 \text{ GB}$, $W = 3.35 \text{ TB/s}$), with $2P = 26 \text{ GB}$ leaving 54 GB for KV caches. The per-token KV cost per sequence is:

$$4 \times L \times K \times H = 4 \times 40 \times 40 \times 128 = 819,200 \text{ bytes/token}$$

S (tokens)	C_{seq} (GB)	B_{max}	Memory (GB)	TPS
1,000	0.78	69	79.8	2,900
4,000	3.05	17	77.9	720
16,000	12.2	4	74.8	176
32,000	24.4	2	74.8	88

TPS is computed as $B_{\text{max}} \cdot W / (2P + B_{\text{max}} \cdot C_{\text{seq}})$. Going from $S = 1,000$ to $S = 32,000$ (a $32\times$ context increase) reduces TPS from 2,900 to 88—a $33\times$ degradation, matching the $1/S$ prediction closely.

Fleet sizing consequences are severe. If you need 100,000 output TPS (500 RPS at 200 output tokens per response):

- At $S = 1,000$: $\lceil 100,000 / 2,900 \rceil = 35$ H100s
- At $S = 16,000$: $\lceil 100,000 / 176 \rceil = 569$ H100s

A $16\times$ context extension drives a $16\times$ GPU cost increase—before accounting for inter-node networking and orchestration overhead. Context length is a business decision, not just an engineering dial.

What You Actually Decide in Practice

- **Segment users into context-length tiers with separate serving pools.** A single 128K-token request occupies roughly 160 sequence slots worth of KV cache at $S = 1,000$ per slot. Mixing workloads in one pool lets long-context requests starve short-context users of GPU memory without any apparent failure mode—just silent throughput collapse. Maintain separate fleets for short ($< 4\text{K}$), medium (4K – 32K), and long ($> 32\text{K}$) tiers, and price each tier to reflect its actual resource multiplier.
- **Apply GQA before scaling the fleet for long-context workloads.** Going from $K = N = 40$ (full multi-head) to $K = 8$ (8 KV heads) cuts C_{seq} by $5\times$, multiplying B_{max} and TPS by $5\times$ at every context length. For Llama-2-13B at $S = 16,000$, this shifts B_{max} from 4 to 20 and TPS from 176 to ≈ 880 —recovering most of the throughput lost to long context before buying a single additional GPU.
- **Enable prefix caching for shared long contexts.** If $B = 100$ concurrent users all share a 4K-token system prompt, without prefix caching each sequence consumes $C_{\text{seq}}(5,000) \approx 3.9 \text{ GB}$ of KV cache (5K total tokens), giving $B_{\text{max}} = \lfloor 54 / 3.9 \rfloor = 13$. With prefix caching, the shared 4K prefix KV (3.1 GB) is stored once; each sequence only needs $C_{\text{seq}}(1,000) = 0.78 \text{ GB}$ for its unique tokens, giving $B_{\text{max}} = \lfloor (54 - 3.1) / 0.78 \rfloor = 65$ —a $5\times$ improvement in concurrent capacity with no hardware change.
- **Use KV cache eviction rate as your capacity signal.** Set an alert threshold at 5% eviction under normal load. Higher eviction means KV blocks for in-flight sequences are being expelled before those sequences complete, forcing expensive KV recomputation and spiking TTFT. If eviction climbs above 10% during sustained load, you are underprovisioned for the current context-length distribution; add capacity or tighten admission control before the queues overflow.
- **At $C_{\text{seq}}(S) > M_{\text{GPU}} - 2P$, switch to multi-GPU KV sharding.** For Llama-2-13B at $S = 69,000$ tokens, a single sequence’s KV cache ($\approx 56 \text{ GB}$) exceeds all available HBM after loading weights. Ring attention shards the KV cache across N GPUs, restoring useful B_{max} values at the cost of all-gather communication

at each layer-typically 10-20% latency overhead at NVLink bandwidth. This communication penalty is acceptable compared to the alternative of serving zero concurrent requests.

How This Shows Up in Interviews

- **“A customer wants to extend context from 8K to 128K tokens. What happens to serving costs?”**
A strong answer derives $C_{\text{seq}}(128,000)$ for the specific model: for Llama-2-13B, $4 \times 40 \times 40 \times 128 \times 128,000 \approx 100$ GB-already larger than the entire H100 HBM. Even with model weights excluded, one sequence at 128K context cannot fit on a single H100. The answer must name ring attention or sequence parallelism as requirements, then estimate GPU count from there. A weak answer says “you need more GPUs” without quantifying.
- **“How does GQA improve long-context throughput?”** A strong answer connects the mechanism to the formula: GQA reduces K by factor g , shrinking C_{seq} by g , multiplying B_{max} by g , and therefore multiplying TPS by $\approx g$ at any context length. The effect is largest at long context because that is when KV cache is the binding constraint rather than request arrival rate. State a concrete number: going from 40 to 8 KV heads on a 13B model at 16K context multiplies TPS by $5\times$.
- **Senior follow-up: “Your prefix cache has an 80% hit rate on a 4K shared system prompt for requests with 1K unique context. Quantify the throughput impact.”** The senior answer reasons from effective KV footprint. Hit sequences carry 0.78 GB (unique 1K KV only); miss sequences carry 3.9 GB (full 5K KV). Average per-sequence KV is $0.8 \times 0.78 + 0.2 \times 3.9 = 1.40$ GB, plus 3.1 GB for the cached prefix pinned in GPU memory. Effective $B_{\text{max}} = \lfloor (54 - 3.1)/1.40 \rfloor = 36$ versus $\lfloor 54/3.9 \rfloor = 13$ without any caching-a $2.8\times$ throughput improvement.

Key Takeaways

- When a GPU is packed to its HBM limit, $\text{TPS} \propto 1/S$: doubling context length halves generation throughput while the GPU stays fully busy.
- $B_{\text{max}}(S) = \lfloor (M_{\text{GPU}} - 2P)/C_{\text{seq}}(S) \rfloor$ is the hard capacity limit; it shrinks proportionally as context grows.
- GQA and MLA reduce C_{seq} directly; a g -fold reduction in KV heads multiplies B_{max} and TPS by g at every context length.
- Prefix caching eliminates repeated KV computation for shared context; at high hit rates it is equivalent to adding GPU replicas at near-zero hardware cost.
- When $C_{\text{seq}}(S) > M_{\text{GPU}} - 2P$, a single sequence does not fit on one GPU; ring attention or KV offloading becomes mandatory, each carrying a 10-20% latency penalty.

Chapter 30 Designing a Pre-Training Run from Scratch

30.1 From Compute Budget to Architecture

30.1.1 Choosing Model Size, Token Count, and Schedule

You have a fixed compute budget - say 10^{23} FLOPs - and you need to decide, before writing a single line of training code, how many parameters your model should have and how many tokens to train on. Get this wrong in either direction and you either leave capability on the table or burn money training a model that costs too much to serve. This decision comes up in system design interviews at every frontier lab, typically framed as “given X H100-days, design the training run.” The answer is not a guess; it follows directly from isoFLOP analysis and the Chinchilla scaling law, two tools you can derive from small proxy runs well before committing to the full job.

Every transformer training run consumes roughly

$$C \approx 6ND$$

floating-point operations, where N is the number of non-embedding parameters and D is the number of training tokens. The factor of 6 accounts for the forward pass ($\approx 2ND$) plus the backward pass ($\approx 4ND$ including the parameter gradient and the activation gradient). This approximation holds within 10% for most dense decoder architectures in the regime $D \gg \sqrt{N}$.

The Chinchilla paper (Hoffmann et al., 2022) posits that cross-entropy loss follows a joint scaling law of the form

$$L(N, D) \approx \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_\infty,$$

where L_∞ is the irreducible error of the data distribution and $\alpha, \beta \approx 0.34$ empirically. Both the model-size term and the data term decay as independent power laws; the optimal allocation sets their marginal contributions equal.

The practical procedure is the *isoFLOP sweep*: fix a compute budget C , then train a grid of models with different N while adjusting $D = C/(6N)$ to keep total FLOPs constant. Each model reaches a different terminal loss. Fitting a quadratic to those losses and reading off the minimum gives you N^* and $D^* = C/(6N^*)$ simultaneously. Repeating this at several compute scales reveals that the optimal ratio D^*/N^* follows its own power law, and at the scales studied by Chinchilla the ratio stabilizes near 20 tokens per parameter.

The punchline that every engineer should be able to state under pressure: *At a fixed compute budget, training-optimal model size and token count scale as $C^{0.5}$ each; the ratio of tokens to parameters determines how the budget is split, and that ratio has climbed steadily from 20:1 (Chinchilla) toward 40-700:1 in modern inference-optimized runs.*

The reason the power law form is so robust is that language modeling at scale is essentially a high-dimensional nonparametric regression problem. Classical theory predicts error should decay as $n^{-1/d}$ where d is the intrinsic dimensionality of the task. For natural language, d is large, which yields exponents much smaller than 0.5 and explains why empirical slopes around 0.1-0.3 are consistently observed rather than the n^{-1} or $n^{-0.5}$ rates you would expect for simple function classes.

The training-optimal versus inference-optimal distinction is where modern practice diverges most sharply from the original Chinchilla result. When the Chinchilla paper was written, LLMs were research artifacts and in-

ference cost was irrelevant. The training-optimal answer minimizes loss per FLOP spent training. The inference-optimal answer minimizes loss per FLOP spent *serving* the model, which means accepting higher training cost in exchange for a permanently smaller model. Once models become products, inference cost dominates because you pay training cost once and serving cost for every query indefinitely. This is why Llama 3 trained at roughly 39 tokens per parameter, why recent Qwen models trained on 30 trillion tokens, and why the modern rule of thumb has moved well above 20:1.

The Kaplan (2020) paper estimated a lower ratio - closer to 5-10 tokens per parameter - because its analysis used models trained to a fixed step count rather than to full convergence with a cosine cooldown. A cosine learning rate schedule cannot be truncated early; the cooldown phase is responsible for a substantial fraction of the final loss reduction. Models stopped mid-cosine are not valid samples on the training-optimal frontier, which caused Kaplan to underestimate the token contribution systematically. Chinchilla corrected this by training every model to its natural termination point.

A Worked Example

Assume a compute budget of $C = 2 \times 10^{23}$ FLOPs, roughly what you get from 1,000 H100s running for 30 days at 40% model FLOP utilization.

Step 1 - Training-optimal allocation. From $C = 120N^2$ (substituting $D = 20N$ into $C = 6ND$):

$$N^* = \sqrt{\frac{C}{120}} = \sqrt{\frac{2 \times 10^{23}}{120}} \approx \sqrt{1.67 \times 10^{21}} \approx 4.1 \times 10^{10} \approx 41\text{B parameters.}$$

$$D^* = 20 \times 41\text{B} = 820\text{B tokens.}$$

Step 2 - Inference-optimal allocation (7B target). If serving cost constraints require a 7B model, spend all of C on tokens:

$$D = \frac{C}{6N} = \frac{2 \times 10^{23}}{6 \times 7 \times 10^9} \approx 4.8 \times 10^{12} \approx 4.8\text{T tokens.}$$

The token-to-parameter ratio becomes $4.8\text{T}/7\text{B} \approx 686:1$. The model trains for far longer than Chinchilla would prescribe; each additional token past the Chinchilla point delivers diminishing returns on training loss but still improves the model because the irreducible floor has not been reached.

Step 3 - Validate with a proxy isoFLOP sweep. Before committing to either configuration, run five to seven models ranging from 1B to 10B parameters, each at fixed $C_{\text{proxy}} = C/100 = 2 \times 10^{21}$ FLOPs. Fit a quadratic to the terminal losses and confirm the minimum. If the minimum shifts consistently toward smaller N , the data suggests an inference-optimal regime; if it aligns near the Chinchilla ratio, training-optimal dominates. This proxy sweep costs about 1% of the full budget and de-risks the entire run.

What You Actually Decide in Practice

- **Pick the token-to-parameter ratio based on deployment, not theory.** If you will serve the model at scale, lean toward 40-200:1 or higher. If this is a research artifact unlikely to see heavy traffic, 20:1 is fine. Never default to Chinchilla's 20:1 for a production model without thinking through the serving economics.
- **Use the WSD (warmup-stable-decay) learning rate schedule.** A cosine schedule requires committing to a fixed endpoint before training starts; early checkpoints are incomparable because they have different cooldown trajectories. WSD holds the learning rate flat during the stable phase, letting you branch off cooldown ramps at arbitrary token counts. This turns the Chinchilla isoFLOP analysis from an $O(n^2)$ training experiment into roughly an $O(n)$ one.

- **Exclude embedding parameters from N when fitting scaling laws.** Embedding layers have a different scaling behavior than the attention and MLP blocks. Including them distorts the log-linear fit and produces a visible kink in the scaling curve. Always report and fit non-embedding parameter counts.
- **The critical batch size grows as your target loss shrinks.** As training progresses and loss decreases, gradient signal becomes more precise and you can productively use larger batches. Llama 3 increased batch size mid-run for exactly this reason. Plan for at least one batch-size ramp in your training schedule.
- **If you are tuning learning rate and batch size without mu-P, fit a scaling law on them.** Without maximal update parameterization, the optimal learning rate shifts roughly as $1/\text{width}$ as the model grows. Train five-plus proxy runs at different scales, find the optimal learning rate at each, and fit the resulting scaling law to extrapolate the optimal rate for your target model. Do not transplant the learning rate from a small run without this correction; it will be too high and the large model will diverge or converge to a suboptimal loss.

How This Shows Up in Interviews

- **“You have a compute budget of 10^{24} FLOPs. Walk me through how you choose model size and token count.”** Strong answer: state $C = 6ND$, describe the isoFLOP sweep, report the training-optimal answer ($N \approx \sqrt{C/120}$, $D = 20N$), then immediately flag the inference-optimal alternative and ask the interviewer which objective matters.
- **“Why did Kaplan’s scaling law predict a different optimal ratio than Chinchilla?”** Strong answer: cosine LR cannot be truncated; Kaplan’s models were not all trained to their natural termination points, so the sample of runs on the optimal frontier was biased toward shorter, data-light runs. Chinchilla fixed the experimental design.
- **“Why do modern models like Llama 3 train at 40:1 or higher tokens per parameter instead of Chinchilla’s 20:1?”** Strong answer: Chinchilla optimizes for lowest loss per training FLOP; once models are products, you want the smallest model with acceptable intelligence because inference cost scales with every query. Over-training a small model trades one-time training cost for permanent serving savings.
- **Senior follow-up: “How would you modify the Chinchilla analysis if you’re training a mixture-of-experts model?”** Sketch of the senior answer: active parameters $N_{\text{active}} \ll N_{\text{total}}$ change both the compute formula ($C \approx 6N_{\text{active}}D$) and the scaling law exponents. You must redo the isoFLOP sweep using active parameters, not total parameters, to find the right allocation. Hunyuan-Large reported a ratio of ~ 96 active-parameter tokens per active parameter after doing exactly this replication.

Key Takeaways

- Training compute is $C \approx 6ND$; fixing C and sweeping N (isoFLOP analysis) is the canonical method for finding the compute-optimal model size and token count.
- Chinchilla’s 20 tokens per parameter is training-optimal, not inference-optimal. Modern production runs use 40-700:1 to reduce serving cost at the expense of training cost.
- Never truncate a cosine LR run early and compare checkpoints; use a WSD schedule to reuse stable-phase checkpoints across different token-count targets.
- Always fit scaling laws on non-embedding parameters; embedding layers follow different scaling dynamics and distort the log-linear fit.

- Validate every allocation decision with a cheap proxy isoFLOP sweep ($\sim 1\%$ of the full compute budget) before launching the full run.

30.1.2 Picking a Parallelism Strategy for Your Cluster

Once you have fixed your model size and token count, the next design decision is how to distribute the work across your GPU cluster. This question shows up in system design interviews as “you have a 70B model and 512 H100s, how do you parallelize training?” The answer requires knowing which axes of parallelism exist, what hardware constraints bound each one, and how to compose them into a configuration that fits in memory and keeps GPU utilization high. Getting this wrong means either hitting an OOM before the first forward pass or spending half your compute budget waiting on communication rather than doing math.

Modern LLM training uses a combination of three parallelism axes, typically composed multiplicatively:

- **Tensor parallelism (TP)**: splits individual weight matrices across GPUs within a layer. An all-reduce is required after every matrix multiply, so TP ranks must be connected by NVLink or an equivalently high-bandwidth interconnect. TP degree is almost always set to the number of GPUs per node (typically 8).
- **Pipeline parallelism (PP)**: partitions the model’s layers into sequential stages, one stage per PP rank. GPUs in different stages communicate only at stage boundaries via point-to-point sends. This communication uses slower inter-node links (InfiniBand) but is much less frequent than TP all-reduces.
- **Data parallelism (DP)**: replicates the full model (or a $TP \times PP$ shard of it) across independent groups that each process a different micro-batch. Gradients are aggregated with an all-reduce across DP ranks after each backward pass.

The total GPU count decomposes cleanly:

$$\text{Total GPUs} = TP \times PP \times DP.$$

Within the DP dimension, *ZeRO* (Zero Redundancy Optimizer) shards optimizer states, gradients, and optionally parameters across the DP group rather than replicating them. ZeRO stage 1 shards only optimizer states, stage 2 adds gradient sharding, and stage 3 shards parameters as well. Each stage reduces memory proportionally to the DP degree at the cost of additional all-gather communication.

The standard recipe is: fill TP within a node (TP = GPUs per node), fill PP across nodes to get the model into memory, then fill the rest of your cluster with DP to increase throughput.

The hierarchy follows the hardware topology directly. NVLink bandwidth inside a node is typically 600-900 GB/s all-reduce aggregate, whereas InfiniBand between nodes is one to two orders of magnitude slower per link. Tensor parallelism inserts an all-reduce after every matrix multiply in every transformer layer - for a 70B model that is hundreds of all-reduces per forward-backward pass. Those collectives must complete in microseconds to not bottleneck compute, which means they can only run efficiently over NVLink. Placing TP ranks on separate nodes would stall the entire pipeline waiting for slow cross-node traffic.

Pipeline parallelism, by contrast, only sends a single activation tensor at each stage boundary once per micro-batch. The communication volume is small (hidden dimension $\times$ sequence length $\times$ micro-batch size in BF16) and the timing is predictable, so it tolerates the higher latency of inter-node links.

The main cost of pipeline parallelism is the *pipeline bubble*: at the start and end of each pipeline flush, some stages are idle. With p stages and m micro-batches per flush, the bubble fraction is approximately $(p - 1)/m$. The interleaved pipeline schedule (used by Megatron-LM) reduces this to $(p/v - 1)/m$ by assigning $v > 1$ non-contiguous layer chunks to each rank, at the cost of $v \times$ more point-to-point messages per micro-batch. As

a rule of thumb, keep $m \geq 4p$ to hold the bubble fraction below 25%; otherwise pipeline parallelism wastes too much GPU time on idle fills and drains.

A Worked Example

Setup: a 7B parameter dense decoder (BF16 weights), trained on 512 H100 SXM5 GPUs organized as 64 nodes $\times$ 8 GPUs per node, 80 GB HBM per GPU.

Memory accounting for a single GPU without parallelism:

- Parameters (BF16): $7 \times 10^9 \times 2$ bytes = 14 GB.
- Adam optimizer states (FP32 master weights, first and second moments): $7 \times 10^9 \times 12$ bytes = 84 GB.
- Total static memory: ≈ 98 GB - exceeds the 80 GB HBM capacity before accounting for activations.

Target configuration: TP = 8, PP = 8, DP = 8. Check: $8 \times 8 \times 8 = 512$ GPUs. ✓

Memory after parallelism (per GPU):

- Parameters sharded by TP $\times$ PP: $14 \text{ GB} / (8 \times 8) = 219 \text{ MB}$.
- Optimizer states with ZeRO stage 1 across DP= 8: $84 \text{ GB} / (8 \times 8 \times 8) = 164 \text{ MB}$.
- Activations (32 layers, sequence length 4096, micro-batch 1): roughly 8-12 GB with recomputation, depending on remat strategy.

Total per-GPU footprint: $\approx 10\text{-}15$ GB, well within 80 GB. The headroom accommodates gradient buffers and CUDA workspace.

Pipeline bubble check: with $p = 8$ pipeline stages, targeting $m = 32$ micro-batches per flush:

$$\text{Bubble fraction} = \frac{p-1}{m} = \frac{7}{32} \approx 22\%.$$

Enabling the interleaved schedule with $v = 2$ interleaved chunks reduces this to:

$$\frac{p/v-1}{m} = \frac{4-1}{32} \approx 9\%.$$

The global batch size at these settings is $\text{DP} \times m \times \text{micro-batch} = 8 \times 32 \times 1 = 256$ sequences. If the target global batch exceeds 256, increase gradient accumulation steps or the micro-batch size until the batch matches the critical batch size derived from the Chinchilla analysis.

What You Actually Decide in Practice

- **Set TP equal to GPUs per node, almost always.** For H100 nodes with 8 GPUs, TP = 8. Going cross-node with TP (TP = 16, 32) is technically possible but the NVLink advantage disappears and latency on TP all-reduces typically kills MFU. Only attempt cross-node TP if your model's hidden dimension is so large (e.g., $d_{\text{model}} \geq 16384$) that single-node TP cannot absorb the compute.
- **Size PP to fit the model, then stop.** Choose the smallest PP degree such that per-GPU memory (parameters, optimizer states, activations) fits in HBM. Every doubling of PP adds pipeline bubble overhead and doubles the number of inter-node point-to-point messages. If ZeRO-3 across DP gets the model into memory without PP, prefer that; the all-gather communication in ZeRO-3 is typically lower latency than managing pipeline schedules.
- **Keep pipeline bubbles below 15% with the interleaved schedule.** Budget $m \geq 2v(p-1)$ micro-batches per flush when using v interleaved chunks. If the required m pushes your global batch size above the critical batch size, reduce PP or reduce v .
- **For MoE models, add expert parallelism (EP) as a fourth axis.** EP routes each token's experts to a separate GPU group. The all-to-all dispatch and combine collectives replace the DP all-reduce for the

MoE layers. Set EP degree equal to the number of experts (or a divisor of it) such that each EP rank holds exactly one expert. The total GPU count becomes $TP \times PP \times DP \times EP$, but DP and EP share the same physical dimension in most implementations (EP is implemented as DP over the expert dimension).

- **Match your global batch size to the critical batch size.** The critical batch size scales predictably with the inverse of the target loss. For a 7B model targeting a terminal loss around 2.0 nats, empirical estimates put the critical batch size near 1-4M tokens. Below that threshold, adding more DP replicas (more GPUs) is nearly as efficient as taking more steps. Above it, extra DP replicas produce diminishing gradient signal. Design your $TP \times PP \times DP$ product so that the resulting global batch lands at or just below the critical batch size; then grow the batch size in stages as loss improves mid-training.

How This Shows Up in Interviews

- **“You have a 70B model and 512 H100s. Walk me through your parallelism strategy.”** Strong answer: compute static memory ($70B \times 2$ bytes + optimizer states), determine $TP = 8$ (per node), choose PP such that per-GPU memory fits, fill remaining GPUs with DP. State the resulting global batch size and check it against the critical batch size. Quote bubble fraction.
- **“What is the pipeline bubble and how do you reduce it?”** Strong answer: bubble fraction $\approx (p-1)/m$ for the simple 1F1B schedule; the interleaved schedule with v chunks reduces it to $(p/v-1)/m$ at the cost of $v \times$ more P2P sends per micro-batch. Increase m (more micro-batches per flush) to reduce the bubble without changing v .
- **“When would you use ZeRO-3 instead of pipeline parallelism for a 70B model?”** Strong answer: ZeRO-3 shards parameters across DP ranks, so it works only if the DP group is large enough and the all-gather latency per step is acceptable. For a 70B model on 8 nodes, ZeRO-3 with $DP = 64$ shards parameters to ~ 2 GB per GPU - affordable, and you avoid pipeline bubble overhead entirely. The trade-off is more frequent all-gather communication on the critical path. ZeRO-3 wins when you have a large DP group within a fat-tree fabric; PP wins when pipeline stages fit on separate chassis with limited cross-chassis bandwidth.
- **Senior follow-up: “How does sequence parallelism change this analysis for a 1M-token context model?”** Sketch of the senior answer: at long sequence lengths, activations (which scale as sequence length $\times$ batch $\times d_{\text{model}}$) dominate over parameter memory. Sequence parallelism (SP) shards the sequence dimension across TP ranks, reducing activation memory by the TP degree. You need all-gather before the QKV projection and reduce-scatter after the output projection, exactly replacing the TP all-reduces. For a 1M-token context, SP is mandatory: activations without SP can exceed 10 GB per layer per GPU even at micro-batch = 1, making the model impossible to fit at $TP = 8$ without aggressive activation recomputation.

Key Takeaways

- Total GPUs = $TP \times PP \times DP$; set TP to the NVLink-connected group size (typically 8), use PP to get the model into memory, fill remaining capacity with DP.
- Pipeline bubble fraction is $(p-1)/m$ for the basic schedule and $(p/v-1)/m$ for the interleaved schedule; keep it below 15% by ensuring $m \gg p$.
- ZeRO-3 is often preferable to PP for single-node-scale models (up to $\sim 30B$) when the DP group is large; PP is necessary when parameters plus optimizer states exceed per-node memory even after TP sharding.

- The global batch size produced by your parallelism configuration must stay at or below the critical batch size; ramp the batch size mid-training as the terminal loss decreases.
- For MoE models, add expert parallelism as a fourth axis; for ultra-long-context models, add sequence parallelism within the TP group to manage activation memory.

30.2 Operations and Failure Modes

30.2.1 Checkpointing, Fault Tolerance, and Silent Data Corruption

When Meta trained Llama 3, they logged over 50 job interruptions from hardware failures across the full run - roughly one every few days at production scale. Training a frontier model across thousands of GPUs is a distributed system in the reliability-engineering sense: components will fail, and your design must make the run survivable. Checkpointing strategy appears in senior-level design interviews precisely because the right answer is not simply “save every N steps.” It requires async I/O pipelines, rolling retention policies, cross-replica validation, and a concrete plan for the failure mode that inflicts the most damage: a GPU that silently produces wrong results without crashing or throwing an error.

A checkpoint must capture everything needed to resume training from an exact point: model parameters, optimizer state (momentum and variance tensors for Adam), the data loader cursor, the random number generator state for both Python and CUDA, and the current global step counter. Miss any of these and a resumed run diverges from the original trajectory - sometimes immediately, sometimes subtly over thousands of steps.

At scale, a synchronous checkpoint blocks the entire training loop. For a 70B-parameter model in ZeRO-3 across 512 H100s, the full checkpoint size is approximately:

$$\underbrace{70\text{B} \times 2}_{\text{bf16 params}} + \underbrace{70\text{B} \times (4 + 4 + 4)}_{\text{fp32 opt. state (master weights, } m, v)} = 140 \text{ GB} + 840 \text{ GB} = 980 \text{ GB total.}$$

Sharded across 512 workers that is about 1.9 GB per rank. An NVMe RAID writing at 10 GB/s can flush each shard in under 200 ms - but only if you do not wait for every rank serially. The standard solution is *asynchronous checkpointing*: each GPU serializes its shard into host DRAM (fast, under 100 ms), the training loop resumes immediately, and a background CPU thread drains that buffer to persistent storage. The GPU never stalls waiting for disk.

Rolling checkpoints address a second failure mode: a checkpoint file that is itself corrupted mid-write. Keep the three most recent sharded saves (at steps N , $N - \delta$, $N - 2\delta$) and delete the oldest only after the newest has been checksum-validated. This gives you two rollback opportunities if the most recent save is bad.

A checkpoint is not a save file - it is the full resumable state of a distributed computation, and its reliability is a product of every component in that chain.

Asynchronous saves overlap I/O with compute so that checkpointing overhead approaches zero at reasonable intervals (every 200-1,000 steps). The host-RAM staging buffer absorbs the burst, and the background thread handles the slower NVMe write while training continues. Blocking the GPU until the file is flushed wastes expensive accelerator time for no algorithmic benefit.

Rolling policies with post-write validation address the corruption-during-write failure mode. A naively overwritten single checkpoint is worse than no checkpoint at all: you lose your fallback. A three-generation rolling policy with per-shard checksum provides both recency and a verified fallback, at the cost of roughly $3\times$ the checkpoint storage.

Silent data corruption (SDC) is the harder case and the one that teams are often underprepared for. ECC memory catches single-bit DRAM errors, but multi-bit errors, PCIe link faults, and rare CUDA kernel bugs can produce numerically plausible but incorrect gradient values. The training loss does not spike - the model simply learns the wrong thing, gradually. The detection strategy that works in practice is maintaining a frozen *canary batch*: a small fixed set of held-out examples whose loss is evaluated separately every 100 steps. A canary loss that drifts upward while training loss looks healthy is an early SDC signal. For ZeRO-1 or ZeRO-2, where all replicas hold identical parameters after each reduce, you can also compute the global gradient norm independently on two separate ranks and compare; a sustained disagreement beyond numerical tolerance flags a corrupted reduce.

A Worked Example

Consider a 7B model training on 64 H100s under FSDP (ZeRO-3). Each GPU holds roughly:

$$\frac{7\text{B} \times (2 + 12) \text{ bytes}}{64} \approx 1.5 \text{ GB per rank.}$$

Checkpointing every 500 steps with async saves adds negligible overhead. At 800K tokens per step, 500 steps equals 400M tokens of training - the maximum exposure window for any hardware failure.

At step 8,200 a single GPU develops a flaky NVLink lane that silently biases its contribute to the reduce-scatter by $\sim 0.1\%$ per step. No NaN appears; the training loss trends normally. At step 10,300, the canary batch loss is 0.09 nats above its expected trajectory - 3.2 standard deviations from the rolling baseline. The run is paused.

Rolling back to the step-8,000 checkpoint (the last clean save before the anomaly), swapping the affected GPU, and resuming costs 300 steps of recompute across 64 GPUs. Without the canary, the team would have continued for weeks and discovered the problem only during post-training evaluation, losing the entire run.

What You Actually Decide in Practice

- **Checkpoint interval.** Every 200-500 steps is the practical range for large runs. Too frequent and async I/O cannot keep up with training throughput; too infrequent and a single GPU failure costs hours of recompute. At 500 steps, 1M tokens/step, and 512 H100s at \$3/hour, one failure costs roughly \$3,000 of wasted compute.
- **Rolling vs. permanent snapshots.** Keep three rolling checkpoints for fast rollback, plus permanent snapshots at fixed milestones (e.g., every 10K steps). Permanent snapshots let you restart mid-training experiments - if you changed the data mix at step 50K, you can re-evaluate from that exact checkpoint without rerunning from scratch.
- **Validate every save before deleting the previous generation.** After the background thread flushes to disk, read back the shard header and compare a checksum. A corrupted save that silently passes is worse than a failed save that triggers a retry. Track per-rank validation status in a distributed metadata store.
- **Canary batch sizing.** 512-2,048 held-out examples is sufficient. Evaluate every 100 training steps. Alert if the canary loss deviates by more than two standard deviations from a 500-step rolling mean. This adds roughly 0.1% overhead at a 128K-token canary batch.
- **Save the full data cursor.** Record the shard index, file offset, and shuffle seed alongside model state. Without these, a resumed run either replays already-seen data or skips data, introducing distribution shifts

that are nearly impossible to diagnose later. This is the most commonly omitted element in naive checkpoint implementations.

How This Shows Up in Interviews

- **“Walk me through how you would checkpoint a 70B model training run across 512 GPUs with FSDP.”** Strong answer: describe ZeRO-3 sharding, async host-RAM staging, background flush to NVMe, per-rank checksum validation, and a three-generation rolling retention policy. Mention that optimizer state dominates checkpoint size and that the data loader cursor and RNG state must be saved alongside model weights.
- **“What is silent data corruption and why is it particularly dangerous during training?”** Strong answer: SDC produces numerically plausible but incorrect gradient values - no crash, no NaN - so the training loss can look healthy while the model learns from a corrupted signal for thousands of steps. Detection requires out-of-band validation (canary batch or cross-replica gradient comparison), not in-band loss monitoring alone.
- **Senior follow-up: “You are 20 days into a 30-day run and discover a GPU has been silently corrupting gradients since day 5. What do you do?”** Senior answer: quantify the contamination window by comparing canary loss to its expected trajectory - if divergence is small and within two standard deviations of noise, the model may be salvageable from a day-5 checkpoint with a targeted rerun. If the drift is large, a 15-day rollback may be unavoidable. In either case: isolate and quarantine the affected hardware, verify the day-5 checkpoint with an independent forward pass on a clean reference node, add cross-replica gradient audits before resuming, and file a hardware incident report so the vendor can diagnose the root cause.

Key Takeaways

- Asynchronous checkpointing stages weights to host RAM first so training resumes before the file hits disk - overhead approaches zero at 200-500 step intervals.
- Keep three rolling checkpoints plus permanent milestone snapshots; validate each save with a checksum before deleting the previous generation.
- Save optimizer state, data loader cursor, and RNG seeds alongside model parameters - a resume without these is not a true resume.
- Silent data corruption produces no NaN and no crash; detect it with a frozen canary batch evaluated every ~100 steps, or via cross-replica gradient comparison on ZeRO-1/2 runs.
- At Llama-3 scale, hardware failures are a statistical certainty; a fault-tolerant checkpoint strategy limits blast radius to hundreds of steps rather than days of recompute.

30.2.2 Monitoring Loss Spikes and Divergence

Three hours into a 21-day training run, the per-step loss plot jumps from 2.8 to 11.3. The gradient norm has been climbing for 50 steps. Do you roll back and resume, wait for self-recovery, or stop the run entirely? Getting this decision wrong in either direction wastes GPU-months. The judgment call itself - and the instrumentation that makes it reliable - comes up in system design interviews at every lab that trains models at scale, because it tests whether you understand training dynamics at the level of individual steps rather than epoch averages.

A training run produces two qualitatively different instability modes. A *spike* is a transient loss increase that recovers within a few hundred steps; the optimizer has encountered an anomalous gradient and the momentum terms absorb the shock. *Divergence* is a non-recovering increase - the optimizer has entered a region from which gradient descent cannot escape, usually because the gradient scale has overwhelmed the learning rate or the Adam second-moment estimate has underflowed.

Distinguishing these two in the first minutes of a suspected event requires per-step instrumentation, not epoch-level averages. The canonical metrics to log at every training step are:

- Training loss (cross-entropy in nats)
- Global ℓ_2 gradient norm *before* clipping
- Effective learning rate (scheduler output at this step)
- Hardware throughput (tokens per second per GPU, as a hardware health proxy)

And, evaluated on a fixed held-out set every $K = 100$ steps:

- Canary batch loss (orthogonal signal independent of in-distribution training data)

Gradient norm is the leading indicator. During stable pre-training on a well-filtered dataset, the unclipped global norm rarely exceeds $2\times$ the gradient clipping threshold. When the unclipped norm crosses $5\text{--}10\times$ the clipping value, a bad batch or a numerical instability has already started to propagate - the loss spike will follow 10-50 steps later. This lead time is your intervention window.

Per-step gradient norm is the earliest warning signal; training loss is the lagging indicator.

Logging at step granularity rather than epoch granularity catches transient events before they compound. A single corrupted document that appears once may spike the loss for five steps and then recover; logged only at epoch boundaries, you never see it, and you cannot correlate the event with a data cursor position to filter the offending sample from future training.

The canary batch provides an orthogonal signal that distinguishes two very different root causes. If training loss spikes but canary loss stays flat, the model is healthy and the event is almost certainly a data artifact - something in the current batch that is anomalous relative to the rest of the distribution. If both training loss and canary loss spike, or if canary loss drifts upward over hundreds of steps while training loss looks normal, the model's learned representations are degrading, which points toward a more serious problem: silent data corruption, a numerical instability in a specific layer, or a bug in the optimizer implementation.

Epoch-level averaging destroys this signal entirely. A brief loss spike that resolves within 200 steps is invisible in a per-epoch summary, yet the data cursor position that caused it can still be identified and the bad document filtered, preventing recurrence.

A Worked Example

A 13B model is training on 128 H100s. The stable phase runs from step 12,000 to 14,900: training loss ≈ 2.2 nats, gradient norm ≈ 1.3 , clipping threshold 1.0.

At step 14,950 the unclipped gradient norm climbs to 4.1 - roughly $3\times$ above the rolling baseline. At step 14,960 it reaches 9.7. Gradient clipping fires, but the Adam second-moment estimate has already been updated with the squared large gradient, depressing the effective learning rate for affected parameters for the next several steps. At step 14,963 the training loss reads 5.4; at step 14,967 it reads 9.1.

Decision protocol:

1. Check the canary batch loss at step 14,950 through 14,967: it reads 2.21 - completely flat. This is a data artifact, not model divergence.

2. Pull the data loader cursor from the step-14,900 checkpoint: shard index 312, document offset 8,192.
3. Inspect documents at that offset. Find a 380,000-token sequence that was incorrectly chunked: a single training sample with no EOS token, heavy Unicode encoding errors, and near-zero perplexity on most tokens (a repeated byte pattern).
4. Roll back to the step-14,900 checkpoint. Add the document hash to the dataset blocklist. Reprocess the shard. Resume.
5. Recovery: within 60 steps, loss returns to 2.2. The canary loss never deviated.

Total cost: 100 steps of recompute plus 30 minutes of investigation. The alternative - running forward 10,000 steps hoping for self-recovery - would have consumed 80 GPU-hours and produced a model degraded on the specific domain of that corrupted document.

What You Actually Decide in Practice

- **Spike vs. divergence threshold.** If training loss has not returned to within 5% of its pre-spike value within 200 steps, roll back. Self-recovery beyond 200 steps is rare for data-artifact spikes and almost never happens for true divergence. Waiting longer only compounds the optimizer state corruption from the large Adam second-moment update.
- **Gradient norm alert threshold.** Set an automated alert at unclipped grad norm exceeding $5\times$ the 1,000-step rolling mean. At this level you have an intervention window before the loss visibly spikes. At $100\times$ the rolling mean, you have a bad batch; pause immediately and inspect the data cursor before resuming.
- **Do not adjust the learning rate as a recovery mechanism.** Reducing LR mid-spike to “stabilize” the run introduces a new instability: the Adam momentum vectors accumulated under the original LR are now mismatched. The preferred recovery is a checkpoint rollback to before the anomaly. LR adjustments belong in the schedule, not in ad hoc responses to individual spikes.
- **Build a bad-document filter as a continuous background job.** Every time a spike investigation identifies a pathological document, add it to a blocklist and re-process the affected data shard offline. After three or four spike investigations, patterns emerge (mislabelled code samples, HTML artifacts, very long sequences with zero semantic content) that can be caught by a rule-based pre-filter, preventing the spike entirely rather than just recovering from it.
- **Alert on canary divergence, not just training loss.** If canary loss deviates by more than two standard deviations from a 500-step rolling mean, page the on-call engineer. This catches slow model degradation - from silent data corruption, optimizer bugs, or data distribution drift - that step-level training loss monitoring misses because the signal is spread across thousands of steps.

How This Shows Up in Interviews

- **“You are training a 30B model and the loss spikes mid-run. How do you decide whether to roll back or continue?”** Strong answer: check gradient norm trajectory (leading indicator - was it climbing before the loss jumped?), check canary batch loss (data artifact vs. model degradation), correlate spike timestamp with data loader cursor position to identify a candidate document, wait at most 200 steps for self-recovery, then roll back to the most recent clean checkpoint if loss has not recovered.
- **“What metrics do you log during pre-training and at what granularity?”** Strong answer: per-step training loss, unclipped global gradient norm, effective learning rate, and hardware throughput; canary batch loss every 100 steps. Epoch-level averages are insufficient because they mask transient spikes that

are often correlated with specific, identifiable data artifacts.

- **Senior follow-up: “Your gradient norm alert fires, but when you inspect the documents at the data cursor position you find nothing obviously wrong. What else could be causing the spike?”** Senior answer: the list of candidates in rough order of frequency: (1) silent data corruption where the gradient is numerically plausible but wrong - cross-replica gradient norm comparison would reveal a discrepancy; (2) a learning rate scheduler bug that momentarily emitted an incorrect LR value - check the per-step LR log; (3) an out-of-memory event that silently fell back to lower precision in one kernel - check GPU memory logs around the affected steps; (4) a numerical instability in a specific attention head triggered by an unusually long sequence - profile the attention patterns on the documents near the spike. Work from most common to most exotic and do not assume the data is the problem until you have ruled out the infrastructure.

Key Takeaways

- Log training loss, unclipped gradient norm, effective learning rate, and GPU throughput at every step - epoch-level averages hide transient spikes that are correlated with specific, filterable bad documents.
- Gradient norm is the leading indicator: it typically begins rising 10-50 steps before the training loss visibly spikes, giving you an intervention window.
- A fixed canary batch evaluated every 100 steps distinguishes data artifacts (canary unaffected) from true model degradation (canary also drifts), which determines the correct response.
- Correlate every spike with the data loader cursor position to identify and filter the offending document - this is almost always faster and cheaper than rolling back without root-cause investigation.
- If training loss has not recovered within 200 steps, roll back to the most recent checkpoint; do not reduce the learning rate mid-spike, as this corrupts the optimizer momentum state accumulated under the original schedule.

Chapter 31 Designing a Fine-Tuning and Alignment Pipeline

31.1 Scoping the Problem

31.1.1 Choosing Between SFT, DPO, and RLVR

When an interviewer asks you to design a post-training pipeline from scratch, the very first decision separates candidates who have shipped these systems from those who have only read about them: which optimization objective do you reach for? Supervised fine-tuning, DPO, and RL from verifiable rewards (RLVR) are not interchangeable—each one assumes a different answer to the question of what signal you have and what behavior you are trying to unlock. This question appears at senior and staff levels at every frontier lab. Getting the choice wrong at scoping time wastes months of compute and annotator budget before a single evaluation number tells you something went sideways.

All three methods share the same goal: given a pre-trained base model π_{base} , produce a policy π_{θ} that behaves better on your target task. They differ in what “better” means and how that signal is delivered.

SFT treats post-training as distribution matching. You collect a set $\mathcal{D} = \{(x_i, y_i^*)\}$ of prompt-completion pairs from experts or high-quality sources and minimize:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_i \log \pi_{\theta}(y_i^* | x_i)$$

The model learns to imitate the conditional distribution over demonstrations. SFT is powerful at teaching output *style*—if you fine-tune on responses with structured lists and clear explanations, you reliably get that format back. But it does not reliably teach new knowledge. If the SFT target y^* references facts the base model does not already encode, the model learns to reproduce the structural pattern of a knowledgeable response without the substance, opening a systematic hallucination pathway.

DPO sidesteps expert demonstrations entirely. You collect pairwise preferences: for each prompt x , a chosen completion y_w and a rejected completion y_l . The InstructGPT RLHF framework models these preferences via the Bradley-Terry equation $P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$. DPO’s key insight is that under a KL-regularized RL objective, the optimal policy implies a reward expressible purely in terms of log-ratios. Substituting this reparameterization into the Bradley-Terry loss and canceling the intractable partition function gives:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

This is a supervised objective: increase the log-probability of y_w relative to the reference, decrease that of y_l , with gradient magnitude scaling up when the implied reward ordering disagrees with the human label.

RLVR abandons the pairwise structure entirely. A reward function $r(x, y) \in \{0, 1\}$ checks objective correctness—does the final answer match the ground truth? GRPO, the most practical algorithm for this setting, samples a group of G completions $\{y_1, \dots, y_G\}$ from the same prompt and defines each completion’s advantage as the group-normalized z-score of its reward:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \quad \mu_r = \frac{1}{G} \sum_{j=1}^G r_j, \quad \sigma_r = \sqrt{\frac{1}{G} \sum_{j=1}^G (r_j - \mu_r)^2}$$

The policy gradient update clips the likelihood ratio $\rho_i = \pi_{\theta}(y_i | x) / \pi_{\text{old}}(y_i | x)$ inside $[1 - \epsilon, 1 + \epsilon]$ and adds a per-token KL penalty against the reference policy. Critically, there is no separate value network: the group

mean serves as the variance-reduction baseline, cutting memory cost roughly in half compared to PPO.

SFT teaches what good outputs look like; DPO teaches which outputs humans prefer; RLVR teaches the model to actually solve problems it can be graded on.

SFT’s strength is speed and stability. There is no reward hacking, no rollout overhead, and the training dynamics are identical to pre-training. Its ceiling, however, is the quality of demonstrations. If your annotators are more knowledgeable than your model, you risk teaching structural imitation rather than knowledge retrieval. The model learns: whenever I see a complex question I should output a long, citation-bearing answer—even when the citation is fabricated.

DPO moves above that ceiling because pairwise comparisons are faster to collect and humans can often judge which response is better even when they cannot produce the best response themselves—the generator-validator gap. The weakness is overoptimization. With enough gradient steps on a fixed preference dataset, the reward implied by the policy diverges from true human preferences. Preference data collected from one checkpoint does not remain informative as the policy drifts, producing responses that ace the proxy loss but disappoint on fresh human evaluation.

RLVR sidesteps the overoptimization problem by using objective, automatable rewards that cannot be gamed by exploiting annotator inconsistency. You can optimize substantially further before hitting diminishing returns. The cost is domain restriction: RLVR only works where automated correctness checking is cheap and unambiguous—math, code, formal logic. Open-ended creative writing or nuanced safety judgments do not satisfy this requirement. The practical consequence is that most production pipelines layer all three in sequence: SFT first to establish behavioral format, DPO for broad helpfulness and safety alignment, RLVR on top for specific verifiable task domains.

A Worked Example

Suppose you are building a coding assistant on a 7B base model (32 layers, $d = 4096$, bf16 precision, fits on a single H100 with room for a $2\times$ context). Pass@1 on HumanEval starts at 32%.

SFT stage. Collect 50,000 prompt-completion pairs of coding problems with step-by-step solutions. Train for 2 epochs at a learning rate of 2×10^{-5} with a cosine schedule—roughly 4 hours on 8 H100s. After SFT, the model reliably wraps code in fenced blocks, obeys instructions like “add type hints,” and handles multi-turn context. pass@1 climbs to 61%.

DPO stage. Run 200,000 pairwise comparisons: for each coding prompt, generate two completions from the SFT model and have GPT-4 label which is more correct and readable. Train one DPO epoch at $\beta = 0.1$. pass@1 reaches 67% and human preference win-rate versus the SFT checkpoint reaches 58%. Values of β below 0.05 produce reward-hacked outputs; above 0.5 the policy barely moves from the reference.

RLVR stage. Construct 4,000 LeetCode-style problems with deterministic test suites. For each training step, sample $G = 8$ completions per prompt, execute them against the test harness, and assign $r \in \{0, 1\}$. Run GRPO for 500 steps with clip $\epsilon = 0.2$ and KL coefficient $\beta_{\text{KL}} = 0.04$. pass@1 on HumanEval reaches 76%. On competition-level problems never seen during SFT or DPO, pass@1 jumps from 11% to 29%—a capability gain that imitation-based methods cannot reach.

What You Actually Decide in Practice

- **SFT before any RL, always.** Skipping SFT forces RL to learn output format and task behavior simultaneously, wasting early steps and producing unstable training curves. Even 1,000 high-quality demonstrations

are enough to prime the format before RLVR takes over.

- **Keep SFT data within the base model’s knowledge horizon.** If the base model cannot generate a fact chain in its top-200 samples, that knowledge is not reliably stored. Feeding it SFT examples that depend on that fact teaches hallucination, not retrieval. Curate or ablate: either verify the model already knows the fact or replace the answer with an explicit abstention.
- **DPO’s β lives in $[0.05, 0.5]$ in practice.** Sweep it on a held-out preference win-rate evaluation before a full run. Below 0.05 you get incoherent reward-hacked outputs within two epochs; above 0.5 you pay the cost of DPO data collection for nearly zero policy change.
- **RLVR requires executability, not just labels.** If evaluating one rollout takes more than a few hundred milliseconds, training will be GPU-bound on evaluation latency rather than compute. Domains that satisfy the speed requirement: Python test harness for code ($\sim 100\text{ms}$ per rollout), SymPy or LLM equivalence checker for math ($\sim 50\text{ms}$). Domains that do not: human safety review, long-form factuality audits.
- **Remove length normalization from GRPO.** Dividing rewards by response length inside the advantage calculation-present in the original GRPO formulation-incentivizes incorrect completions to become arbitrarily long, since length deflates the penalty. Drop this term; control output length separately via a length reward applied only after initial RL convergence, once the model is already finding correct solutions.

How This Shows Up in Interviews

- **“Walk me through how you would choose between SFT, DPO, and GRPO for a new product feature.”** Strong answer: map the feature onto three axes-do you have expert demonstrations, human pairwise preferences, or automatable correctness signals?-then layer them in order (SFT $\rightarrow$ DPO $\rightarrow$ RLVR) only as far as signal quality and compute budget allow. If you have none of the above, collect SFT data first.
- **“Why would DPO underperform PPO in some experimental setups?”** Strong answer: DPO is inherently offline-it fixes the preference dataset at collection time and does not update rollouts as the policy shifts. Once the policy drifts from the generating distribution, implied rewards grow stale and gradients mislead. PPO’s on-policy rollouts keep the gradient signal current. Iterated DPO-re-collecting preferences from the evolving policy each round-closes this gap but costs as much as PPO to run.
- **Senior follow-up: “DeepSeek R1 found that process reward models did not outperform outcome-based rewards. Why, and what does that imply for your pipeline design?”** Senior answer: process reward models require dense, step-level supervision that is expensive to collect and prone to step-level reward hacking-the model learns to satisfy the PRM’s grading rubric without actually solving the problem. Outcome rewards are sparser but directly optimize the terminal objective, and at sufficient rollout scale the model receives adequate gradient signal from the binary outcome alone. The practical implication: build outcome-based RLVR first; add a PRM only if outcome rewards plateau and the domain supports reliable step-level annotation. Even then, use the PRM as a dense shaping signal added to the outcome reward, not as a replacement for it.

Key Takeaways

- SFT reliably imprints output style via distribution matching but cannot push beyond demonstrated capability; it is the mandatory first stage before any RL, even when RL is the ultimate objective.
- DPO converts pairwise preferences into a supervised loss by reparameterizing reward as a log-ratio of policies; it is cheaper than SFT data collection but susceptible to overoptimization as the policy drifts

from the preference-collection distribution.

- RLVR via GRPO unlocks capability gains beyond human imitation using group-normalized outcome rewards and no value model, but requires domains where correctness is automatable and evaluation is fast.
- The production recipe is additive and ordered: SFT establishes format, DPO aligns general preferences, RLVR squeezes task-specific performance-run each stage only as far as your signal quality and latency budget allow.
- The first question when scoping a post-training pipeline is never “which algorithm?”-it is “what labels do I have, at what cost, and how fast can I evaluate them?”

31.1.2 Defining Rewards and Evaluation Harnesses

Once you have chosen an algorithm, the next thing that will kill your project is a poorly designed reward or an evaluation harness that does not measure what you think it measures. In my experience this is where most post-training pipelines fail: not in the optimizer or the parallelism strategy, but in the silent divergence between the proxy signal you can compute cheaply and the actual behavior you want to ship. Interviewers at senior and staff levels probe this directly-“how would you evaluate whether the model actually improved?” is one of the most loaded questions in the interview, because a weak answer reveals that you have only thought about training, not about whether training is doing the right thing.

Every post-training objective requires a reward signal $r(x, y) \in \mathbb{R}$ that scores a completion y against a prompt x . The design of this function-and the harness used to measure it at evaluation time-determines whether your training loop is converging toward the behavior you want or toward a proxy that happens to correlate with it in your current data distribution.

Rewards decompose into three practical categories:

Outcome rewards assign a single scalar to the complete response based on whether it achieves a verifiable end state. For code, this is test-suite pass rate. For math, it is final-answer equivalence checked by SymPy or an LLM judge. For factual QA with known answers, it is exact or normalized string match. The signal is binary or near-binary: $r \in \{0, 1\}$. Outcome rewards are the backbone of RLVR pipelines because they are fast, automatable, and resistant to gaming-the model cannot satisfy a test suite by producing a plausible-looking answer that fails on edge cases.

Process rewards assign intermediate scores to individual reasoning steps. A process reward model (PRM) reads a chain-of-thought token-by-token and emits a scalar at each step or at step boundaries, giving the RL algorithm dense feedback rather than a single terminal signal. This is appealing in theory because sparse terminal rewards are hard to optimize over long sequences. In practice, PRMs are expensive to build-you need step-level human annotations or a synthetic verifier that is itself reliable-and they introduce a second reward-hacking surface: the model learns to satisfy the PRM’s grading criteria at each step without the overall reasoning being correct.

Preference rewards are the output of a reward model (RM) trained on pairwise human annotations. Given a prompt x and two completions $y_w \succ y_l$, the RM minimizes a Bradley-Terry cross-entropy loss:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

The trained r_ϕ then scores any completion as a scalar during RL. Preference rewards are the only practical option for open-ended domains where no automatic verifier exists. Their weakness is the overoptimization cliff: as the RL policy moves away from the preference-collection distribution, the RM’s scores become unreliable,

and further optimization maximizes the proxy without improving-or actively degrading-true human preference.

An evaluation harness is the broader infrastructure that runs a fixed set of prompts through a checkpoint, computes the reward (or a separate held-out metric), and returns a scalar summary. The harness is distinct from the training reward. You should never evaluate on the exact reward function used during training, because that would conflate proxy optimization with actual improvement.

The training reward is a signal you optimize; the evaluation harness is the honest answer to whether that signal pointed in the right direction.

The central tension in reward design is between *measurability* and *alignment*. Outcome rewards are maximally measurable—a unit test either passes or fails—but they only cover domains with ground-truth answers. Preference rewards are broadly applicable but soft, noisy, and gameable. Most production pipelines combine them: outcome rewards for hard skills where correctness is unambiguous, preference rewards for general helpfulness and safety, and a separate held-out evaluation harness for both.

The overoptimization phenomenon is the clearest argument for treating the evaluation harness as a first-class artifact rather than an afterthought. In a representative study, models optimized against a preference RM showed increasing RM score throughout training but declining true human preference win-rate after a critical point—proxy and reality diverged. The same curve appears for every noisy reward: the model finds the cracks in the measurement before you do. Building a harness that is harder to optimize than the training reward—human evals sampled from a held-out prompt distribution, or a separate RM trained on a disjoint annotation set—gives you an early-warning system.

For verifiable rewards, the evaluation harness should test generalization, not coverage. If you trained on 4,000 LeetCode-style problems, your harness should run on a disjoint set of 500 problems from a different difficulty distribution, including problems harder than anything in the training set. Pass rate on the training distribution tells you the model memorized; pass rate on the out-of-distribution harness tells you it learned.

A Worked Example

Consider designing the reward stack for a math reasoning model fine-tuned from a 7B base.

Training reward. Binary outcome: the model generates a response inside `<think>` and `</think>` tags followed by a final boxed answer. A lightweight SymPy equivalence checker compares the extracted answer to the ground-truth numeric or symbolic value. Runtime per rollout: approximately 40 ms. Format compliance (presence of the required tags) contributes a secondary reward of +0.1 to prevent the model from abandoning the structured output early in training. The two rewards are summed: $r = r_{\text{correct}} + 0.1 \cdot r_{\text{format}}$.

Evaluation harness. Four held-out benchmark splits, none overlapping with training data:

1. **MATH-500:** 500 competition math problems across five difficulty levels. Reports pass@1 as the primary capability metric.
2. **AIME 2024:** 30 problems from the most recent American Invitational Mathematics Examination. Tests extreme difficulty generalization; a well-trained 7B should reach 15-25%.
3. **GSM8K:** 1,319 grade-school math problems. A sanity check that RL has not catastrophically degraded basic arithmetic competence; score should remain above 85%.
4. **Overrefusal probe:** 100 benign prompts that use mathematical vocabulary but request no reasoning (e.g., “How many days are in a week?”). Checks that the model does not wrap trivial answers in unnecessary long chains-of-thought, which inflates inference cost.

Run the harness every 50 gradient steps. Plot MATH-500 pass@1 and AIME pass@1 together. If MATH-

500 climbs but AIME is flat, the model is pattern-matching training problems rather than learning to reason. If GSM8K drops below 80%, the KL coefficient is too small and capability regression has set in-increase β_{KL} and restart from the last good checkpoint.

What You Actually Decide in Practice

- **Never evaluate on the training reward function directly.** Use a structurally different evaluator—a stricter equivalence checker, a separate RM trained on held-out annotations, or human preference votes on a disjoint prompt sample. Evaluating on the training reward confirms the optimizer converged; it says nothing about whether behavior improved.
- **Instrument for length separately from quality.** Length is the most common confound in post-training evaluation. Human judges and LLM judges both show a 60-70% preference for longer responses regardless of quality. Log response length at every harness run and check that quality gains are not simply length gains. Normalize pass rates by length bucket when comparing checkpoints.
- **Set a regression floor for every capability you are not actively training.** If you are RL-ing on math, your code benchmark and factual QA score should be checked at every harness run. RLHF and RLVR both cause capability regression when the KL penalty is too weak; the harness regression floor catches this before the checkpoint is promoted.
- **For preference rewards, log the RM score distribution, not just the mean.** Overoptimization shows up first in the shape of the distribution: the model learns to score in a narrow band near the RM’s saturation point, producing stylistically homogeneous but semantically thin responses. A narrowing standard deviation of RM scores—even when the mean is rising—is an early warning sign.
- **Match the evaluation prompt distribution to the deployment prompt distribution.** A harness built from academic benchmarks will not catch failures on the prompts your users actually send. If you have production traffic, sample 1,000 prompts per week, have them human-rated, and add them to the harness. This is the highest-signal investment in evaluation infrastructure you can make.

How This Shows Up in Interviews

- **“How would you evaluate whether your post-training run actually improved the model?”** Strong answer: distinguish the training reward (proxy, optimized) from the evaluation harness (held-out, harder to game), name at least two harness components—a capability benchmark and a regression floor—and mention the length confound and overoptimization cliff as things you are actively watching.
- **“Your RLHF reward model score is going up every epoch but human raters say the model is getting worse. What is happening?”** Strong answer: overoptimization—the policy has drifted outside the distribution on which the RM was trained, and the RM is now unreliable. Mitigations: add a KL penalty to slow policy drift, collect fresh preference data from the current policy (on-policy DPO), or switch to an outcome reward in a verifiable sub-domain to anchor the signal.
- **Senior follow-up: “How would you design a reward for a general-purpose assistant where there is no single correct answer?”** Senior answer: layer a weak verifiable signal (format compliance, citation presence, tool-call correctness) with a preference RM for open-ended quality, and track both separately in the harness. Use constitutional AI-style LLM judging for specific safety dimensions rather than a monolithic RM, so you can diagnose which dimension is degrading. Treat the evaluation harness as a multi-objective Pareto surface: you want to see Pareto improvement across all dimensions with each

checkpoint, not a single-number summary that masks trade-offs.

Key Takeaways

- Rewards split into three types-outcome (binary, automatable, resistant to gaming), process (dense, expensive to build, second hacking surface), and preference (broadly applicable, noisy, subject to overoptimization)-and most production pipelines combine all three.
- The evaluation harness is structurally separate from the training reward: it must be harder to optimize, drawn from a held-out distribution, and include at least one regression floor for each capability domain not being actively trained.
- Overoptimization is the dominant failure mode in preference-reward pipelines; it shows up as rising RM scores alongside falling true win-rate, and the first detectable signal is narrowing of the RM score distribution before the mean turns over.
- Length is a universal confound-normalize quality metrics by response length and log length separately at every harness evaluation.
- The most durable evaluation investment is sampling from real production traffic weekly and adding those human-rated examples to the harness; no academic benchmark matches the distribution of prompts your users actually send.

31.2 Operational Concerns

31.2.1 Rollout Infrastructure and vLLM Integration

When you sit down to design an RL training system for a reasoning model, the question that trips most candidates is not algorithmic - it is operational. Specifically: where does inference happen, who manages the weights, and how do you keep GPU utilization from collapsing while rollouts are in flight? This topic shows up in senior and staff-level system design interviews precisely because it sits at the intersection of distributed training and inference serving, two domains that most engineers treat as separate concerns.

The RL training loop for language models has two structurally different phases per iteration. The first is rollout generation: autoregressively sampling responses from the current policy for every prompt in the batch. This is pure inference - it saturates memory bandwidth rather than compute, benefits from continuous batching and paged attention, and produces variable-length outputs. The second is the parameter update: a forward pass to recompute log probabilities, a backward pass to accumulate gradients, and an optimizer step. This workload needs the full model in fp32 or bf16 with optimizer state alongside it, and it needs the entire activation graph in memory during the backward pass.

These two workloads cannot share the same GPU allocation efficiently. The rollout phase needs large KV-cache buffers; the training phase needs activation memory. Running both sequentially on one device forces you to deallocate and reallocate large memory regions each iteration, fragmenting the allocator and throttling throughput. The standard architecture therefore separates them into dedicated worker groups:

- **Rollout workers:** run the current policy via vLLM or a similar engine; produce (prompt, response, log_prob) tuples.
- **Trainer workers:** hold model weights plus optimizer state; run forward-backward-optimizer steps.

The data flow per outer iteration is:

Trainer

- broadcast weights to rollout workers (NCCL)
- rollout workers generate responses + log probs
- responses + log probs sent back to trainer
- trainer runs N inner gradient steps
- repeat

The rollout worker is an inference server that the trainer treats as an external data source; weight synchronization is the only coupling between them.

Separating the two workloads lets each worker optimize for its own regime. The rollout worker can use continuous batching to maximize KV-cache utilization: shorter sequences finish early and free slots for new ones without stalling the rest. The trainer can plan memory layout for activation checkpointing without worrying about KV-cache pages. The two run on different devices, so their memory pressure does not interfere.

The main trade-off is weight synchronization latency. After every outer iteration, the trainer must push updated weights to all rollout workers before the next generation pass starts. With NCCL collective broadcasts this takes on the order of seconds for a 7B model over NVLink, but it stalls the rollout workers during the transfer. Without overlapping this transfer with the optimizer step on the trainer side, synchronization becomes the wall-clock bottleneck.

The obvious alternative - a single process that interleaves inference and training calls - works in toy implementations but fails at scale. Memory fragmentation from alternating large allocation patterns (KV cache versus activation buffers) degrades throughput, and there is no opportunity to overlap computation across the two phases.

A Worked Example

Consider a 7B model in bf16, running GRPO on a cluster of H100s (80 GB HBM each). Model weights: $7 \times 10^9 \times 2 \text{ bytes} \approx 14 \text{ GB}$. AdamW optimizer state (first and second moments, fp32): $\approx 56 \text{ GB}$. The trainer worker therefore needs approximately 70 GB just for weights and optimizer state, leaving only 10 GB for activation memory - tight but feasible with gradient checkpointing.

On the rollout worker side, a single H100 running vLLM with 60 GB allocated to KV cache can hold roughly $60 \times 10^9 / (2 \times 32 \times 128 \times 2) \approx 36,000$ key-value slots per layer, supporting batches of 64 sequences at 2K tokens each with room to spare. The rollout worker leaves the remaining 20 GB for the model weights themselves (no optimizer state, no activations beyond the forward pass).

Weight synchronization for 14 GB over a 600 GB/s NVLink interconnect takes about 0.02 seconds - negligible. Over a 200 Gb/s Ethernet fabric between nodes, it takes about 0.56 seconds and dominates iteration time if not overlapped. The standard mitigation is to start the broadcast while the trainer is still computing gradients on the previous batch, so the weights arrive just as the trainer finishes its optimizer step.

Regarding vLLM's current limitation: as of early 2025, hot-updating weights inside a running vLLM instance requires an experimental NCCL-based API that loads weight tensors from the trainer's memory space into vLLM's internal buffers via collective operations. Many production implementations instead tear down and restart vLLM each outer iteration, paying a few seconds of initialization overhead. At batch sizes large enough to amortize that cost, this is acceptable; at small batch sizes, it dominates.

What You Actually Decide in Practice

- **Rollout reuse vs. freshness:** GRPO takes K inner gradient steps per rollout batch to amortize inference cost. Each inner step recomputes log probs under the current policy - this is a cheap forward pass, not a new generation. Never re-run the rollout worker for inner steps; just cache the token sequences and recompute log probs from the trainer model.
- **Variable-length CoT handling:** Long-chain reasoning models generate sequences ranging from 200 to 8K tokens in the same batch. Naive padding to the maximum length wastes $5\times$ to $40\times$ memory on a mixed batch. Use sequence packing (concatenate sequences up to the context limit) or length-based bucketing (group sequences within 10% of each other by length) before passing to the trainer.
- **Reward computation parallelism:** If the reward requires executing sandboxed code (Python, unit tests), reward evaluation can take hundreds of milliseconds per sample. Decouple reward evaluation entirely from GPU workers: run it on CPU workers or a separate service that returns scores asynchronously while the trainer is doing inner gradient steps on the previous batch.
- **Memory for old log probs:** Do not store a full copy of π_{old} on GPU to compute clipping ratios. At generation time, record the log probability of each sampled token and store it as a float tensor. For a batch of 256 sequences of 1K tokens, that is $256 \times 1024 \times 4 \text{ bytes} = 1 \text{ GB}$ - two orders of magnitude cheaper than a second model copy.
- **When to use vLLM weight update vs. restart:** Use in-place weight update (NCCL) when your outer iteration time exceeds 60 seconds, so the 2-5 second restart overhead matters. Below that threshold, restart is simpler and avoids subtle bugs with LoRA adapter or quantized layer synchronization that the experimental API does not yet handle cleanly.

How This Shows Up in Interviews

- **“Walk me through the infrastructure for a GRPO training run at scale.”** A strong answer covers: dedicated rollout and trainer workers, one-way weight broadcast per outer iteration, rollout reuse with log-prob recomputation for inner steps, and decoupled reward evaluation. Mention the vLLM integration and the teardown-vs-hot-update trade-off.
- **“Why is RL harder to scale than supervised fine-tuning?”** Strong answer: inference is in the critical path (rollouts must complete before any gradient step), multiple model copies increase peak memory, weight synchronization adds inter-node traffic that does not exist in SFT, and variable-length outputs from autoregressive generation break the uniform batch structure that SFT relies on for high utilization.
- **Senior follow-up: “How would you overlap rollout generation and gradient computation?”** Senior answer: double-buffering. While the trainer runs K inner gradient steps on batch N , the rollout workers are already generating batch $N + 1$ using the weights from iteration N (slightly stale, but within the trust region). The weight broadcast for iteration $N + 1$ is then pipelined to overlap with the last inner step of batch N . This hides almost all synchronization latency but requires careful accounting: log probs for batch $N + 1$ are computed under iteration- N weights, so the old-policy log probs in the clipping ratio must match.

Key Takeaways

- Rollout generation and gradient updates have incompatible memory layouts; separate them into dedicated worker groups to avoid fragmentation and enable overlapped execution.
- Weight synchronization from trainer to rollout workers is the primary inter-iteration latency; use NCCL broadcasts and overlap them with the optimizer step where possible.
- Cache rollout token sequences and old log probs as tensors; never re-run the inference engine for inner gradient steps.
- Variable-length CoT outputs require sequence packing or length bucketing; unmitigated padding can waste $10\times$ or more of memory bandwidth.
- Reward evaluation should be decoupled from GPU workers and parallelized on CPU or a dedicated service to avoid blocking the training pipeline.

31.2.2 Keeping Reference, Old, and Current Policies in Sync

One of the most reliable ways to identify an engineer who has implemented GRPO versus one who has only read about it is to ask them how many policy versions they maintain during training and what role each one plays. The bookkeeping here is operationally simple but conceptually precise: get it wrong and you produce gradients that appear to train (the loss decreases) while actually computing the wrong thing. This question appears at the senior-level intersection of RL algorithms and distributed systems correctness.

A GRPO training loop maintains up to three distinct versions of the policy simultaneously, each frozen at a different cadence:

- π_θ (**current policy**): the model being trained; weights change after every inner gradient step.
- π_{old} (**old policy**): a snapshot frozen at the start of each outer iteration; used to compute the importance sampling ratio for the PPO-clip objective.
- π_{ref} (**reference policy**): a slowly updated or permanently frozen baseline from the SFT checkpoint; used to compute the KL penalty that prevents catastrophic policy drift.

The clip objective relies on the ratio $r_t = \pi_\theta(a_t | s_t) / \pi_{\text{old}}(a_t | s_t)$. For the importance weight to be valid, π_{old} must be treated as a constant with respect to gradient computation. The same applies to π_{ref} in the KL term. The critical implementation requirement is:

```
# WRONG: differentiates through old_log_probs
ratio = torch.exp(log_probs - old_model(responses))

# CORRECT: old log probs are a frozen tensor
with torch.no_grad():
    old_log_probs = compute_log_probs(old_model, responses)
# old_log_probs has no grad_fn; safe to use in loss
ratio = torch.exp(log_probs - old_log_probs)
```

The KL penalty uses the low-variance estimator rather than the naive sample average:

$$\hat{D}_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}}{\pi_\theta} - \log \frac{\pi_{\text{ref}}}{\pi_\theta} - 1$$

This has the same expectation as $\log(\pi_\theta / \pi_{\text{ref}})$ but lower variance because the $r - \log r - 1$ form eliminates the

first-order noise term. The update schedule across the three policy roles is:

π_{ref} : never or every $\mathcal{O}(100)$ outer iterations π_{old} : once per outer iteration π_{θ} : every inner step

The three policy roles exist because each one serves a different contract: π_{θ} is what we optimize, π_{old} is the anchor for off-policy correction, and π_{ref} is the anchor for behavioral stability.

π_{old} enables rollout reuse. Without it, each gradient step would require a new rollout from the current policy, multiplying inference cost by the number of inner steps K . The importance ratio $\pi_{\theta}/\pi_{\text{old}}$ corrects for the distributional shift introduced by doing multiple updates on stale rollouts. The clip constraint $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ bounds how stale the correction can be before it is simply ignored, preventing large policy updates from producing high-variance gradients.

π_{ref} solves a different problem. Verifiable rewards (math correctness, code passing tests) do not capture general language quality, instruction-following behavior, or formatting. A policy optimized purely for reward can drift into a mode that solves problems correctly but becomes incoherent on other tasks - reward hacking at the behavioral level. The KL penalty against π_{ref} provides a soft tether to the instruction-tuned starting point.

The obvious simplification - pure on-policy REINFORCE with a single policy copy - is mathematically valid but practically inefficient. Every gradient step requires a full rollout, making the cost proportional to K times higher compared to GRPO with rollout reuse. The price of GRPO's efficiency is the bookkeeping complexity of three policy versions.

A Worked Example

Consider a 7B model in bf16 on a single H100 (80 GB HBM). Weights: 14 GB. AdamW optimizer state (fp32 moments for π_{θ}): 56 GB. Total for the trainable model: 70 GB. That leaves 10 GB.

Option A: store π_{old} as a full GPU model copy. Cost: 14 GB. Now at 84 GB - over capacity.

Option B: compute old log probs at rollout time and store them as a float32 tensor. For a batch of 256 sequences of 1,024 tokens each:

$$256 \times 1024 \times 4 \text{ bytes} = 1,048,576 \text{ bytes} \approx 1 \text{ GB}$$

Total: 71 GB. Fits with 9 GB to spare for activations.

For π_{ref} , the same 14 GB weight cost applies. Options: (a) keep it on GPU if memory permits, (b) offload to CPU and transfer to GPU only during the KL computation step (once per batch), or (c) quantize π_{ref} to int8 (7 GB) since it never needs gradients. With option (b), the CPU-to-GPU transfer of 14 GB at 50 GB/s (PCIe 4.0) takes about 0.28 seconds; with NVLink on a multi-GPU system this drops to under 0.03 seconds. For outer iterations that take 30+ seconds end-to-end, CPU offloading is essentially free.

What You Actually Decide in Practice

- **Never store π_{old} as a full model copy on GPU.** Compute log probs at rollout time and retain them as a `torch.Tensor` with `requires_grad=False`. You need only the scalar log probability per token, not the ability to re-run inference.
- **Wrap all reference and old log prob computations in `torch.no_grad()`.** This is not optional. If you omit it and π_{old} shares parameters with π_{θ} (e.g., they are the same object), the gradient of $\log(\pi_{\theta}/\pi_{\text{old}})$ is identically zero, your optimizer makes no effective update, and the loss still decreases. The bug is completely silent without inspecting gradient norms.

- **π_{ref} update cadence should be slow.** Updating it every outer iteration makes it equivalent to π_{old} , which already serves the off-policy correction role. A common pattern is to fix π_{ref} at the SFT checkpoint for the entire run, or to update it every 200-500 outer iterations as a soft annealing schedule.
- **Under tensor parallelism, all three policy shards must live on the same device mesh with consistent sharding.** If π_{ref} is sharded differently from π_{θ} (e.g., different TP degree), the all-reduce communication during the KL computation will deadlock or produce incorrect partial results. Initialize all three from the same checkpoint and shard them identically.
- **For DPO,** π_{ref} is the only extra policy; there is no π_{old} . The DPO loss computes $\log \pi_{\theta}(y_w | x) - \log \pi_{\theta}(y_l | x) - \log \pi_{\text{ref}}(y_w | x) + \log \pi_{\text{ref}}(y_l | x)$. The reference log probs can be precomputed once on the entire dataset and stored as a scalar per example, reducing the runtime memory footprint to a lookup table rather than a live model forward pass.

How This Shows Up in Interviews

- **“Describe the three policy roles in GRPO and when each one updates.”** A strong answer names all three explicitly: π_{θ} updates every inner step; π_{old} updates once per outer iteration (after rollout generation); π_{ref} is fixed or updated very rarely. Each role is tied to its function: optimization target, importance weight anchor, behavioral stability anchor.
- **“What goes wrong if you accidentally differentiate through π_{old} ?”** Strong answer: when π_{old} shares the same parameter tensor as π_{θ} , the ratio $\pi_{\theta}/\pi_{\text{old}}$ is a constant 1 regardless of the weights. Its gradient with respect to θ is zero by the chain rule, so the clipping term contributes nothing to the parameter update. The KL term may still produce a nonzero gradient, so training appears to proceed, but the clip mechanism is broken and the trust region guarantee is gone.
- **Senior follow-up: “How do you manage the three policies under model-parallel training across 64 GPUs?”** Senior answer: all three policy copies must use identical tensor-parallel and pipeline-parallel sharding configurations. For π_{ref} , since it never needs a backward pass, you can quantize it to int8 and co-locate it on the same device mesh as π_{θ} without a full fp16 copy, halving its HBM footprint. For π_{old} , avoid storing model weights at all: at rollout time, the rollout worker (which runs π_{old} by definition, since weights were just synchronized) records log probs and returns them to the trainer as a flat tensor. The trainer stores this tensor; no second model shard is needed.

Key Takeaways

- Three policy roles: π_{θ} (trained every inner step), π_{old} (frozen per outer iteration for importance sampling), π_{ref} (frozen or slowly updated for KL regularization).
- Store π_{old} as a log-prob tensor, not a model copy; at 7B scale this saves roughly 13 GB of HBM per policy copy.
- Missing `torch.no_grad()` on old or reference log prob computation produces a silent zero-gradient bug that is invisible from the loss curve alone.
- π_{ref} update cadence must be slow; frequent updates collapse it to π_{old} and eliminate the behavioral stability anchor.
- Under model parallelism, all three policy shards must use identical device mesh and sharding configuration to avoid communication deadlocks.

Appendix A Napkin Math for LLM Interviews

A.1 Parameter Counts and Memory Footprints

These formulas size the weight tensors and the memory budget before writing a single line of code. The parameter derivations and their implications for architecture decisions are developed in [subsection 3.1.1](#); the precision trade-offs behind the training-memory figure are covered in [subsection 9.3.1](#).

Formulas

Parameters per transformer layer. With hidden dimension d , SwiGLU expansion $d_{ff} = \lfloor 8d/3 \rfloor$, N query heads, and N_{kv} key-value heads (MHA: $N_{kv} = N$; GQA: $N_{kv} < N$):

$$P_{\text{attn}} = d^2 \left(2 + \frac{2N_{kv}}{N} \right) \approx 4d^2 \quad (\text{MHA}, N_{kv} = N) \quad (\text{A.1})$$

$$P_{\text{mlp}} = 3 d d_{ff} \quad (\text{SwiGLU: three matrices}) \quad (\text{A.2})$$

$$P_{\text{layer}} = P_{\text{attn}} + P_{\text{mlp}} \quad (\text{A.3})$$

Total parameters (embedding table included):

$$P = L \cdot P_{\text{layer}} + V \cdot d$$

Memory footprint:

$$M_{\text{inference, bf16}} = 2P \text{ bytes} \quad (\text{A.4})$$

$$M_{\text{training, AdamW}} \approx 16P \text{ bytes} \quad (\text{A.5})$$

The 16 B/param training figure: bf16 working copy (2 B) + fp32 master weights (4 B) + fp32 gradients (4 B) + AdamW first and second moments (4+4 B). Activation memory (batch $\times$ sequence length dependent) is excluded.

Worked Example - 7B Model

Config: $d = 4096$, $N = 32$ heads, $d_h = 128$, $L = 32$ layers,
 $d_{ff} = 11008 = \text{floor}(8 * 4096 / 3)$, $V = 32000$, bf16

Attention per layer	: $4 * 4096^2$	= 67.1 M
MLP per layer	: $3 * 4096 * 11008$	= 135.3 M
Layer total	:	202.4 M
Body (32 layers)	: $32 * 202.4 \text{ M}$	= 6.48 B
Embedding table	: $32000 * 4096$	= 0.13 B
Grand total	:	~ 6.6 B (LLaMA-1 7B = 6.74 B)
Inference bf16	: $6.7e9 * 2$	= 13.4 GB
Training AdamW	: $6.7e9 * 16$	= 107.2 GB

Largest model on 8 x H100 (no activations):

$$8 * 80 \text{ GB} / 16 \text{ B/param} = 40 \text{ B params (rough ceiling)}$$

Rules of Thumb

- **MLP $\approx 2 \times$ attention** per layer for SwiGLU models (135 M vs. 67 M at $d = 4096$); the MLP holds $\sim 67\%$ of each layer's parameters.
- **Inference (bf16):** ≈ 2 GB per billion parameters. A 70 B model needs ≈ 140 GB.
- **Training (AdamW):** ≈ 16 GB per billion parameters. A 7 B model needs ≈ 107 GB before activations.
- **Ceiling on n H100s (80 GB each):** $n \times 80/16 = 5n$ billion parameters (ignoring activations).
- **GQA with N_{kv} KV heads:** reduces KV cache and KV weight contribution by factor N/N_{kv} . Use $N_{kv} = N/8$ (group size 8) as the production default for models ≥ 7 B.

Caveats

- Bias terms are omitted; all modern frontier models drop them in attention and MLP.
- For vanilla ReLU/GeLU MLPs use $d_{ff} = 4d$, not $\lfloor 8d/3 \rfloor$. Mixing the two conventions inflates the count by $\sim 50\%$.
- Activation memory scales with batch size $\times$ sequence length and can rival weight memory at large batch sizes; use gradient checkpointing to trade it back for compute.
- Embedding weights are sometimes tied between input and output projections, halving the $V \cdot d$ contribution.

A.2 FLOPs per Forward and Backward Pass

FLOPs are the currency of training cost estimates and the input to every napkin-math scheduling question. The arithmetic intensity framework that connects FLOPs to hardware utilization is in [subsection 8.2.1](#); isoFLOP analysis and Chinchilla compute-optimal ratios are in [chapter 14](#).

Formulas

Single matrix multiply $A_{m \times k} \cdot B_{k \times n}$:

$$\text{FLOPs} = 2mkn$$

All other operations (softmax, layer norm, element-wise activations) are linear in tensor size and negligible compared to the matrix multiplications for large d .

Forward pass (attention + MLP; dominant terms for $T \ll d$):

$$C_{\text{fwd}} \approx 2 N_T P$$

where $N_T = B \times T$ is the total number of tokens in the batch and P is the parameter count. The attention quadratic adds $4 B L T^2 N_{kv} H$ FLOPs; this becomes comparable to the linear terms when $T \gtrsim d$ (e.g., $T = 8192$, $d = 4096$).

Backward pass (parameter gradients + activation gradients):

$$C_{\text{bwd}} \approx 4 N_T P \quad (\approx 2 \times C_{\text{fwd}})$$

Full training step:

$$C_{\text{step}} \approx 6 N_T P$$

Model FLOPs Utilization (MFU):

$$\text{MFU} = \frac{C_{\text{step}}/t_{\text{step}}}{\text{peak FLOP/s (dense, no sparsity)}}$$

Worked Example - 70B Model, 15 T Tokens, 1024 H100s

Total training FLOPs = $6 * 70\text{e}9 * 15\text{e}12 = 6.3\text{e}24$

H100 bf16 peak (dense, no sparsity)	= 989 TFLOP/s
At MFU = 0.5 : $989\text{e}12 * 0.5$	= $4.945\text{e}14$ FLOP/s per GPU
1024 GPUs : $1024 * 4.945\text{e}14$	= $5.06\text{e}17$ FLOP/s total

Training time = $6.3\text{e}24 / 5.06\text{e}17 = 1.245\text{e}7$ s ~ 144 days

Sanity check (8 H100s, 1 week, MFU = 0.5):

FLOPs = $8 * 4.945\text{e}14 * 7 * 86400 = 2.4\text{e}21$

Chinchilla-optimal params ($N_T = 20P$): $2.4\text{e}21 / (6 * 20) \sim 20\text{B}$ params

Rules of Thumb

- **One training step:** $C_{\text{step}} \approx 6N_TP$ FLOPs.
- **H100 at MFU 0.5 (bf16 dense):** $\approx 5 \times 10^{14}$ useful FLOP/s per GPU.
- **H100-GPU-day:** $5 \times 10^{14} \times 86,400 \approx 4.3 \times 10^{19}$ FLOPs.
- **Chinchilla-optimal:** $N_T \approx 20P$ tokens. Modern runs use $100P$ - $200P$ tokens to amortize inference cost (see [chapter 14](#)).
- **MFU target:** ≥ 0.4 for well-tuned distributed runs; < 0.1 signals a system-level bottleneck (communication, data stall, or fragmentation).
- **Attention quadratic term:** adds $\sim BT^2/d$ relative to the linear terms; double T adds $4\times$ attention FLOPs but only $2\times$ MLP FLOPs.

Caveats

- The $6N_TP$ approximation assumes MLP matrix multiplications dominate. Small d or long T can push attention FLOPs to the same order.
- Gradient checkpointing re-runs a partial forward pass during the backward, multiplying C_{fwd} by ≈ 1.33 while saving activation memory.
- MFU degrades rapidly under heavy communication (tensor parallelism all-reduces, pipeline bubbles); measure on the actual cluster before committing to a layout.
- FP8 training theoretically doubles peak FLOP/s (H100: ≈ 1979 TFLOP/s dense); treat it as an advanced option requiring careful numerical validation.

A.3 KV-Cache Sizing and Latency Estimates

Generation is memory-bandwidth-limited: every optimization targets bytes transferred, not FLOPs. The arithmetic intensity argument is in [subsection 8.2.1](#); the full latency-throughput model is in [subsection 15.2.2](#); KV-reduction architectures (GQA, MLA, local attention) are in [chapter 16](#).

Formulas

KV cache per sequence (bf16; N_{kv} key-value heads; head dimension H ; context S tokens):

$$C_{kv} = 2 \times L \times N_{kv} \times H \times S \times 2 \text{ bytes}$$

(First factor of 2: K and V tensors; last factor of 2: bytes per bf16 element.)

Weight memory (bf16):

$$M_{\text{weights}} = 2P \text{ bytes}$$

Decode step time at batch size B (memory-bandwidth-limited):

$$t_{\text{step}}(B) = \frac{M_{\text{weights}} + B \cdot C_{kv}}{\text{BW}}$$

Decode throughput (tokens per second):

$$\text{TPS}(B) = \frac{B \cdot \text{BW}}{M_{\text{weights}} + B \cdot C_{kv}}$$

KV-crossover batch size (where KV cache memory = weight memory):

$$B^* = \frac{M_{\text{weights}}}{C_{kv}}$$

Throughput gains are superlinear in B below B^* and sublinear above it.

Maximum batch size (VRAM-limited):

$$B_{\text{max}} = \frac{\text{VRAM} - M_{\text{weights}}}{C_{kv}}$$

GQA with N_{kv} heads (down from N in MHA):

$$C_{kv}^{\text{GQA}} = C_{kv}^{\text{MHA}} \times \frac{N_{kv}}{N}, \quad B^{*,\text{GQA}} = B^{*,\text{MHA}} \times \frac{N}{N_{kv}}$$

Worked Example - Llama-2 13B on a Single H100

Config: $P = 13 \text{ B}$, $L = 40$, $N = 40$, $N_{kv} = 40$, $H = 128$

$\text{BW} = 3.35 \text{ TB/s}$, $\text{VRAM} = 80 \text{ GB}$, $S = 1024$, bf16

$M_{\text{weights}} = 2 \times 13\text{e}9 = 26 \text{ GB}$

$C_{kv} = 2 \times 40 \times 40 \times 128 \times 1024 \times 2 \text{ bytes} = 838 \text{ MB} \sim 0.84 \text{ GB/seq}$

$B^* = 26 / 0.84 \sim 31$

$B_{\text{max}} = (80 - 26) / 0.84 \sim 64 \text{ (barely fits)}$

B	Total mem (GB)	Step time (ms)	TPS (tok/s)	Fits?
1	26.8	8.0	125	yes
16	39.4	11.8	1,356	yes
32	52.9	15.8	2,025	yes

64		79.8		23.8		2,689		barely
128		133.5		39.8		3,215		no (> 80 GB)

With GQA, $N_{kv} = 8$ (Llama-3 style; reduction factor = $40/8 = 5$):

$$C_{kv} = 0.84 * (8/40) = 0.168 \text{ GB}$$

$$B^* = 26 / 0.168 \sim 155$$

$$B_{\max} = (80 - 26) / 0.168 \sim 321$$

$$B = 128: \text{total mem} = 26 + 128 * 0.168 = 47.5 \text{ GB} \quad (\text{fits})$$

$$\text{step time} = 47.5 / 3350 = 14.2 \text{ ms}$$

$$\text{TPS} = 128 / 0.0142 \sim 9,000 \text{ tok/s} \quad (\sim 3.3x \text{ vs. MHA at } B=64)$$

Rules of Thumb

- **Single-sequence latency** ($B = 1$): $\approx 2P[\text{GB}]/\text{BW}[\text{GB/s}]$ ms per token. Llama-2 13 B on H100: $26/3,350 \approx 8$ ms.
- **Crossover batch** B^* : where throughput gains shift from superlinear to sublinear. Compute it first; batching beyond B^* yields diminishing returns.
- **GQA production default**: $N_{kv} = N/8$ (group size 8) shrinks C_{kv} by the reduction factor with $< 1\%$ accuracy loss at ≥ 7 B scale.
- **KV quantization (int8)**: halves C_{kv} , doubling $B_{\max}$ at the same VRAM; use before adding hardware.
- **Doubling context** S : doubles C_{kv} , halves $B_{\max}$. At $S = 4,096$, B^* for Llama-2 13 B drops from ~ 31 to ~ 8 .
- **Replica parallelism**: M independent copies give $M \times$ throughput with zero extra latency and no inter-GPU communication; prefer it over larger batches.

Caveats

- These estimates apply to the *decode* (generation) phase only. Prefill is compute-bound, not memory-bound; see [subsection 15.1.1](#).
- Attention during decode has arithmetic intensity ≈ 1 independent of batch size, because each sequence owns its own KV cache. See [subsection 15.1.2](#).
- Inter-GPU communication (tensor/pipeline parallelism) is excluded from the step-time formula; add all-reduce latency when sharding the model across multiple devices.
- Flash Attention reduces HBM traffic during prefill but does not alter the decode formula above-the KV cache must still be streamed fully from HBM at each generation step.

Appendix B Common Interview Questions

Each entry below gives the question as an interviewer would phrase it, a 3-6 sentence answer skeleton (the spine, not the full argument), a pointer to the section that develops the argument in full, and a senior follow-up with a short sketch of what separates a strong answer. Read the referenced sections first; these sketches are revision aids, not substitutes.

B.1 Architecture and Training Questions

1. **Walk me through a decoder-only transformer block. What are the key sub-components, and in what order does data flow?**

Skeleton. Each block contains a self-attention sub-layer and an MLP sub-layer, each wrapped in a pre-norm residual. The input is added back to the output of each sub-layer (residual stream). Pre-norm placement - normalizing before the sub-layer rather than after - keeps the residual signal intact across depth and is the near-universal choice in models trained past 1B parameters. The MLP is typically SwiGLU with $d_{ff} \approx \lfloor 8d/3 \rfloor$, making it roughly $2\times$ the parameter budget of the attention sub-layer per layer. See Section 3.1.

Senior follow-up. “Why is the MLP parameter count so sensitive to the d_{ff} formula, and how does it shift when you move from ReLU to SwiGLU?” A senior answer notes that SwiGLU requires three weight matrices (W_1, W_2, W_3) compared to two for ReLU MLP, so the factor of $\lfloor 8d/3 \rfloor$ rather than $4d$ is a deliberate FLOPs-matched downscale to keep total parameter count comparable; naive adoption of SwiGLU without adjusting d_{ff} inflates MLP cost by $\sim 50\%$.

2. **Why do modern LLMs use RMSNorm instead of LayerNorm? Isn't the FLOP difference negligible?**

Skeleton. The FLOP difference is indeed negligible (normalization is $<0.2\%$ of total FLOPs), but the argument is about memory bandwidth, not arithmetic. Normalization kernels are memory-bandwidth-bound; on an H100, peak compute/bandwidth ratio is $\sim 300\times$, meaning any kernel doing fewer than 300 FLOPs per byte hits the memory wall. LayerNorm's extra β parameter adds a full parameter-load pass over d scalars per call; across 64 normalization calls in a 32-layer 7B model, that is ~ 524 KB of avoidable HBM traffic per sample. Ablations show quality parity and $\sim 5\%$ throughput improvement. See Section 3.2.

Senior follow-up. “Walk me through the roofline model for a LayerNorm kernel. What is its arithmetic intensity, and how does that determine the speedup ceiling from switching to RMSNorm?” A senior answer computes arithmetic intensity ($\sim 3-7$ scalar ops per element loaded), places it firmly left of the roofline ridge, and concludes that any reduction in memory traffic reduces wall time proportionally - giving $\sim 12.5\%$ reduction in normalization time, which maps to 3-5% overall throughput gain.

3. **What activation function do frontier models use in the MLP, and why?**

Skeleton. The dominant choice is SwiGLU: $\text{SwiGLU}(x, W, V, W_2) = (xW \odot \sigma(xV))W_2$, a gated variant of SiLU. The gate $\sigma(xV)$ acts as a soft input filter that empirically yields better loss at matched FLOP budgets than GeLU or ReLU, and it enables smooth gradient flow without dead neurons. The cost is the third matrix W_3 ; production stacks adjust d_{ff} downward to compensate. See Section 3.3.

Senior follow-up. “If SwiGLU needs three weight matrices and GeLU only two, how do you set d_{ff} so the FLOP budget stays the same as a GeLU baseline with $d_{ff} = 4d$?” The answer sets $d_{ff}^{\text{SwiGLU}} = \lfloor 8d/3 \rfloor$,

restoring parameter and FLOP parity; a senior candidate derives this from the $3 \times d \times d_{ff}$ vs. $2 \times d \times 4d$ parameter comparison.

4. Explain RoPE. How does it differ from absolute positional embeddings, and why is it preferred for long-context models?

Skeleton. Rotary Position Embedding encodes position by rotating query and key vectors in the complex plane: $q_m \rightarrow q_m e^{im\theta}$, so the attention dot product between position m and n depends only on $m - n$ - a relative offset. Absolute embeddings inject position into the residual stream before any layers, which makes extrapolation to unseen lengths fragile. RoPE is applied at every attention head and generalizes better under length extension techniques (e.g., YaRN, RoPE scaling). See Section 3.4.

Senior follow-up. “What goes wrong when you try to extend a RoPE model to context lengths it was not trained on, and how does YaRN address it?” High-frequency RoPE dimensions wrap around at lengths beyond training distribution, causing attention scores to become noisy. YaRN interpolates rotation frequencies so that no dimension exceeds its trained range; a senior answer explains the interpolation factor $s = L'/L$ and why the NTK-aware variant is preferred over naive linear interpolation.

5. You need to design the attention hyperparameters for a 70B model: how many heads, how many KV heads, and what head dimension do you choose?

Skeleton. Head dimension is typically fixed at 128 (matching one warp on modern GPUs and fitting Flash Attention block sizes efficiently). At $d = 8192$ for a 70B model, that gives 64 attention heads. KV heads are reduced via Grouped-Query Attention (GQA) to 8 or fewer, so each KV head is shared by 8 query heads; this cuts the KV cache by $8\times$ with negligible quality loss. Multi-Head Attention at this scale is now unusual - MQA or GQA is the default for any model intended for serving. See Section 4.2.

Senior follow-up. “How does the number of KV heads interact with tensor parallelism degree? What breaks if you tensor-parallel over 8 GPUs but only have 4 KV heads?” Each GPU in tensor-parallel must hold at least one KV head; if the TP degree exceeds the KV head count, KV heads must be replicated, partially negating the cache savings. A senior candidate designs the KV head count as a multiple of the largest anticipated TP degree.

6. What causes loss spikes during large-scale training, and what mechanisms do modern stacks use to prevent or recover from them?

Skeleton. Loss spikes are typically triggered by outlier gradient norms driven by softmax instability in the attention layers: logit values grow large, the softmax saturates, and gradient flow collapses. Mitigation strategies include global gradient norm clipping (usually at 1.0), QK-norm (normalizing queries and keys before the dot product to bound logit growth), and z-loss (a regularization term penalizing large logit magnitudes applied to the final output softmax). Soft logit capping (Gemma, Gemini) bounds logits to a fixed range without hard clipping. See Section 5.1.

Senior follow-up. “If you observe a spike at step 12,000 in a 100B run, how do you decide whether to roll back, adjust a hyperparameter, or continue?” A senior answer distinguishes recoverable spikes (gradient norm returns to baseline within 50-100 steps, loss follows) from unrecoverable divergence (norm stays elevated, loss plateaus above pre-spike level). The decision tree: roll back to the last checkpoint if loss does not recover within 200 steps; if the spike is recoverable, log the step and batch index for contamination investigation before resuming.

7. Why does MoE offer better loss-per-FLOP than a dense model of the same active-parameter count?

Skeleton. A dense model uses the same parameter set for every token; an MoE routes each token to the K highest-scoring experts out of E total, so the effective parameter count seen per token is K/E of total

MoE parameters - much less than the full model - while the total unique parameters encode a far richer function space. At matched active-FLOPs budgets (same $K \times d_{ff}$), the MoE has E/K more unique parameters, which translates directly to lower loss at the same training compute. See Section 6.1.

Senior follow-up. “What are the three main failure modes of token-choice top- K routing, and how does auxiliary-loss-free load balancing address them?” The three failure modes are load collapse (all tokens route to one expert), token dropping (overflow when an expert exceeds its capacity), and routing instability (oscillation during early training). Auxiliary-loss-free balancing tracks expert load via a running EMA bias term and adjusts routing scores to steer load toward underloaded experts without adding a gradient-competing auxiliary loss term. See Section 7.2.

8. How do you choose between data parallelism, tensor parallelism, and pipeline parallelism for a 100B training run?

Skeleton. Data parallelism (ZeRO-3) shards optimizer state, gradients, and parameters across replicas with no inter-GPU communication inside the forward pass; it is the default until a model’s parameter count exceeds per-GPU memory. Tensor parallelism partitions individual weight matrices across devices, requiring two all-reduce calls per layer - it should be limited to GPUs on the same NVLink domain (typically ≤ 8) where bandwidth is high. Pipeline parallelism partitions layers across nodes and requires careful micro-batching to hide bubble overhead; ideal bubble fraction is $\approx 1/m$ where m is the number of micro-batches per pipeline flush. A 100B run on 512 GPUs typically uses TP= 8, PP= 4-8, DP=remaining. See Section 12.4.

Senior follow-up. “At what point does tensor parallelism overhead exceed its memory benefit, and how do you detect this empirically?” Profile MFU (Model FLOP Utilization) as TP degree increases; if MFU drops more than $\sim 5\%$ relative per doubling of TP, communication overhead is dominating. This typically occurs when TP crosses NVLink boundaries (cross-node all-reduce at ~ 200 GB/s vs. intra-node ~ 900 GB/s); at that point, switching to additional pipeline or data parallelism is preferred.

B.2 Inference and Serving Questions

1. Explain the KV cache. Why does generation have fundamentally different GPU utilization characteristics than prefill?

Skeleton. During prefill, all prompt tokens are processed in parallel: the GPU receives a large matrix multiply (high arithmetic intensity, compute-rooflined). During generation, one new token is produced per step and attends to all cached KV pairs: this is a matrix-vector multiply (low arithmetic intensity, memory-bandwidth-rooflined). The KV cache stores the K and V projections for every prior token so they are not recomputed; its size grows as $2 \times L \times d_{head} \times n_{heads} \times T$ bytes per sequence. See Section 15.1.

Senior follow-up. “How does KV cache size constrain maximum batch size at serving time, and how does paged attention address this?” KV cache for B sequences of length T occupies $B \times 2LTd$ bytes; at large T , this exhausts VRAM before the GPU’s compute is saturated. Paged attention stores KV blocks in non-contiguous VRAM pages (analogous to virtual memory), enabling fine-grained memory sharing (prefix caching) and eliminating fragmentation from variable-length sequences.

2. What is GQA and how does it reduce KV cache pressure?

Skeleton. Grouped-Query Attention assigns multiple query heads to share a single KV head: if there are H_Q query heads and H_{KV} KV heads with $H_{KV} < H_Q$, each KV head is replicated to serve H_Q/H_{KV} queries. The KV cache footprint scales with H_{KV} , not H_Q , so setting $H_{KV} = H_Q/8$ reduces cache size

by $8\times$ with minimal quality loss. Multi-Query Attention (MQA) is the limiting case $H_{KV} = 1$; GQA is the practical middle ground used by LLaMA 2/3, Mistral, and Gemma. See Section 16.1.

Senior follow-up. “At inference time, how does the GQA KV head count interact with tensor parallelism? What breaks if H_{KV} is not divisible by the TP degree?” Each TP shard must hold complete KV heads; if H_{KV} is not divisible by TP, KV heads must be replicated across shards, negating cache savings. A senior design ensures H_{KV} is a multiple of the maximum anticipated TP degree (e.g., $H_{KV} = 8$ for TP up to 8).

3. Explain speculative decoding. When does it help and when does it not?

Skeleton. A small draft model generates K token proposals in one pass; the large target model verifies all K in parallel in a single forward pass using a tree-structured attention mask. Accepted tokens are kept; the first rejected token triggers a corrected sample from the target distribution, ensuring exact distributional equivalence. Speedup is governed by the acceptance rate α : latency reduction is $\approx K \cdot \alpha / (1 + K \cdot c)$ where c is the relative cost of the draft model. Speculative decoding helps when tokens are predictable (code, templated text) and hurts when acceptance rate is low (creative generation, multilingual). See Section 18.1.

Senior follow-up. “How do you choose the draft model? What properties make a good draft model for a given target model?” The draft model should be 10-20 $\times$ cheaper than the target (parameter count or latency ratio), trained on the same data distribution, and tokenizer-compatible. In practice, self-speculative decoding (using early layers of the target as the draft) avoids the tokenizer compatibility problem and the memory overhead of a second model; EAGLE and Medusa extend this to learned auxiliary heads.

4. What quantization schemes are production-viable today, and what are the precision-quality trade-offs?

Skeleton. INT8 weight-only quantization (e.g., `LLM.int8()`) is effectively lossless for most models: perplexity degrades $<1\%$ and throughput improves $\sim 1.5\text{--}2\times$ due to halved weight memory. INT4 (GPTQ, AWQ) is more aggressive: quality degrades noticeably on reasoning tasks unless per-group quantization is used (128-token groups typically). Activation quantization (W8A8, W4A8) enables integer matrix multiplies and is the path to $4\times$ throughput, but requires careful handling of activation outliers (SmoothQuant absorbs them into weight scales). See Section 19.1.

Senior follow-up. “What is an activation outlier and why does it break naive INT8 activation quantization?” Certain dimensions of the residual stream grow to magnitudes $100\times$ the typical value during training (a phenomenon amplified by depth and model scale). Naive per-tensor quantization rounds small activations to zero when the scale is set by the outlier; SmoothQuant migrates the scaling difficulty from activations to weights by multiplying the outlier dimension in activations by $1/s$ and the corresponding weight row by s before quantization.

5. Describe continuous batching and why it is preferred over static batching for LLM serving.

Skeleton. Static batching waits until all sequences in a batch complete before accepting new requests; the batch is padded to the longest sequence, wasting compute on pad tokens and blocking new requests until the slowest sequence finishes. Continuous batching (iteration-level scheduling) inserts new requests into the batch at each decode step, immediately after any sequence finishes. This improves GPU utilization from $\sim 30\text{--}50\%$ to $\sim 80\%+$ in typical workloads by eliminating both padding waste and request queuing delay. See Section 18.2.

Senior follow-up. “How does prefix caching interact with continuous batching, and what are the memory management implications?” Prefix caching reuses KV blocks for shared prompt prefixes (e.g., a

system prompt shared across requests); with continuous batching, blocks must be reference-counted and reclaimed when all sequences sharing a prefix complete. Eviction policies (LRU, radix-tree-based) determine cache hit rate; a senior answer notes that prefix cache hit rate is the primary factor driving effective throughput on chatbot workloads with long system prompts.

6. A user asks why their 7B model is slower per token at batch size 32 than at batch size 1, even though the GPU utilization is higher. Diagnose and explain.

Skeleton. At batch size 1, generation is memory-bandwidth-bound: each step loads model weights once and performs one matrix-vector multiply. At batch size 32, the weight loading is amortized over 32 sequences (good), but KV cache loading scales linearly with batch size and sequence length - at large T , KV cache reads dominate over weight reads, and the GPU is still memory-bound but now on KV traffic rather than weight traffic. Additionally, at batch size 32 with long sequences the effective arithmetic intensity of the attention step grows only sublinearly, while total data movement grows linearly. See Section 15.1.

Senior follow-up. “At what batch size does an attention kernel transition from memory-bound to compute-bound, and how does head dimension affect this crossover?” The crossover occurs when FLOPs per attention step ($\approx 2 \cdot B \cdot T \cdot d$) divided by bytes loaded ($\approx 2 \cdot B \cdot T \cdot d + B \cdot d^2$) exceeds the hardware ridge point (~ 300 on H100). For small head dimensions, the crossover batch size is large (hard to reach); for larger head dimensions, it is reached earlier. Flash Attention exploits this by fusing tiles to reduce HBM round trips regardless of batch size.

B.3 Alignment and Evaluation Questions

1. What does SFT teach a model, and why is it insufficient for aligning behavior to human preferences?

Skeleton. Supervised Fine-Tuning teaches the model to imitate the distribution of demonstration data: it maximizes log-likelihood over high-quality responses. This improves instruction-following and surface-level helpfulness but does not optimize for human preference between responses, cannot teach the model to refuse harmful requests it was not explicitly shown refusing, and introduces hallucination pressure when the model is forced to complete responses in domains where the training data contains confident but incorrect completions. The model learns *what was said*, not *what is preferred*. See Section 26.1.

Senior follow-up. “In practice, SFT often degrades factual accuracy relative to the pretrained base model. What is the mechanism?” SFT data curators select fluent, complete-sounding responses; the model learns to produce confident completions regardless of factual grounding. This overrides the base model’s calibration, which learned uncertainty from noisy pretraining data. A senior answer distinguishes data-side mitigations (include responses that hedge, refuse, or express uncertainty) from model-side mitigations (KL regularization toward the base model during SFT).

2. Explain the RLHF pipeline: what does the reward model learn, and how is PPO used to optimize it?

Skeleton. Human raters compare pairs of responses; a Bradley-Terry model fits a scalar reward function r_ϕ to predict which response is preferred. The policy π_θ is then optimized with PPO to maximize $\mathbb{E}[r_\phi(x, y)] - \beta \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}})$, where the KL term prevents the policy from collapsing to reward-hacking outputs that are out-of-distribution for the reward model. PPO requires maintaining the current policy, the old policy (for importance sampling), the reference policy (for KL), and the value function - four model copies in memory simultaneously. See Section 27.1 and Section 27.2.

Senior follow-up. “What is reward overoptimization, how do you detect it, and how do you mitigate it?” Overoptimization occurs when the policy finds outputs that score high under r_ϕ but are not actually preferred by humans - a proxy gaming failure. Detection: track gold human preference win-rate alongside r_ϕ score; divergence between the two is the signal. Mitigation: lower β (less KL regularization) is the wrong direction; increase β , reduce PPO steps per batch, or use an ensemble of reward models to make reward hacking harder.

3. What is DPO, and what is its main advantage over PPO for preference alignment?

Skeleton. Direct Preference Optimization reparameterizes the RLHF objective to eliminate the explicit reward model: it shows that the optimal policy under the KL-constrained reward maximization objective satisfies $r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$, allowing the reward to be implicitly defined by policy log-ratios. The training loss becomes a binary cross-entropy over preferred/rejected pairs. DPO removes the value network and eliminates the rollout generation step, reducing memory from 4 model copies (PPO) to 2 (policy + reference). See Section 27.2.

Senior follow-up. “DPO is often observed to reduce output diversity and produce shorter responses than PPO. What is the mechanism, and how do variants like SimPO or IPO address it?” DPO’s gradient pushes the policy to increase the likelihood of chosen responses and decrease the likelihood of rejected responses; if the chosen responses are uniformly shorter or less varied, the policy collapses toward that mode. SimPO adds a length normalization term to the reward; IPO uses an identity link instead of the Bradley-Terry sigmoid, which reduces over-confidence on pairs that are nearly tied in quality.

4. What is GRPO, and why does it matter that it eliminates the value network?

Skeleton. Group Relative Policy Optimization replaces the PPO value function with a group-level baseline: for each prompt, G completions are sampled and the reward for each is normalized by the group mean and standard deviation, yielding a self-contained advantage estimate without a learned critic. This removes the fourth model copy (value network) from memory, reducing peak VRAM by $\sim 25\%$ relative to PPO and eliminating the value function’s training instability. GRPO is the training algorithm behind DeepSeek-R1’s reasoning-chain optimization. See Section 28.2.

Senior follow-up. “GRPO maintains an old policy for importance sampling. How does the number of reuse steps of old-policy rollouts interact with training stability?” Reusing rollouts for more than ~ 2 -4 gradient steps increases the importance-sampling ratio $\pi_\theta/\pi_{\text{old}}$ beyond the PPO clip range ($\epsilon \approx 0.2$), causing gradient variance to explode. GRPO clips the ratio with the same ϵ as PPO but clips at the group level; increasing the number of reuse steps requires widening ϵ or shrinking the learning rate, both of which slow convergence.

5. What is RL from Verifiable Rewards (RLVR), and why does it produce more reliable training signal than a learned reward model?

Skeleton. RLVR trains on tasks where correctness can be checked without human judgment: math problems with known solutions, code that passes unit tests, formal proofs verified by a checker. The reward is binary (correct/incorrect) and not subject to reward model overoptimization because there is no proxy to game - a wrong answer simply scores zero. This produces a stable, non-hackable signal but limits training to domains where ground truth is available. DeepSeek-R1 and OpenAI’s o-series models both use RLVR as a component of their reasoning capability. See Section 28.1.

Senior follow-up. “What failure modes can still occur with binary verifiable rewards, even though reward hacking is prevented?” Process hacking: the model finds solutions that satisfy the verifier without generalizing (e.g., memorizing problem templates, outputting correct final answers via invalid intermediate

steps). Length exploitation: the model learns that longer chains of reasoning are rewarded due to correlation in the training set, not because length causally improves correctness. A senior answer suggests process reward models or step-level verification as mitigations.

6. How do you detect and mitigate benchmark contamination when evaluating a model?

Skeleton. Contamination occurs when benchmark examples appear in pretraining data, directly or via near-duplicates; a model can score high by retrieval rather than generalization. Detection strategies include: n-gram overlap between benchmark test sets and pretraining corpora, membership inference attacks (testing whether the model assigns higher likelihood to test examples than held-out examples from the same distribution), and canary insertion (embedding synthetic labeled examples at data collection time and later checking if the model recalls them). Mitigation is primarily data-side: deduplicate pretraining data against known benchmarks before training. See Section 25.1.

Senior follow-up. “If you cannot access the pretraining corpus to check for contamination, what evaluation design choices reduce contamination risk?” Use held-out private benchmarks not released publicly; generate new examples programmatically at evaluation time (e.g., LiveCodeBench, FrontierMath); or use behavioral probes that require reasoning steps rather than answer recall. Chatbot Arena sidesteps contamination entirely by using open-ended human preferences on novel prompts. See Chapter 24.

7. Why is perplexity on a held-out set a poor proxy for downstream task performance, and what should you measure instead?

Skeleton. Perplexity measures the model’s ability to predict the next token averaged over a reference corpus; it is sensitive to tokenization, domain distribution, and corpus quality rather than to task-relevant capabilities. A model with lower perplexity can score worse on MMLU, HumanEval, or instruction-following benchmarks if the pretraining corpus is high-quality but domain-mismatched to the evaluation tasks. For alignment-stage evaluation, win-rate against a reference model on human preference judgments (or GPT-4 as a proxy judge) correlates better with real-world utility than perplexity. See Section 23.1.

Senior follow-up. “GPT-4-as-judge is widely used but has known biases. Name two and describe how to mitigate them.” Position bias (GPT-4 favors responses in the first position when presented with two choices): mitigate by randomizing order and averaging. Verbosity bias (longer responses are judged as better regardless of quality): mitigate by constraining response length or instructing the judge to penalize unnecessary verbosity. A senior answer also notes self-enhancement bias (GPT-4 favors outputs similar to its own training distribution) and suggests using multiple judge models or human calibration.

Appendix C Checklists for System Design Answers

C.1 The 10-Minute LLM System Design Checklist

This checklist operationalizes the repeatable design framework from [chapter 2](#) for a 45-60 minute interview slot. Each item should produce a concrete number or a named decision. Vague answers do not pass. Skip Phase 3 for purely serving-focused prompts; skip Phase 4 for purely training-focused ones.

Phase 1 - Clarify requirements (2 min)

- State the serving SLO: time-to-first-token, inter-token latency, and token throughput (see [subsection 2.1.1](#)).
- State the compute budget: number of GPUs, GPU-hours available, and cost ceiling per query.
- Confirm expected traffic shape: median and peak QPS, context-length distribution.
- Confirm quality target: benchmark name and minimum acceptable score.

Phase 2 - Size the model (2 min)

- State parameter count and justify it (capability target, inference cost, or Chinchilla ratio; see [chapter 14](#)).
- Compute inference memory: $2P$ bytes (bf16) or $0.5P$ bytes (int4) (see [section A.1](#)).
- State whether the model fits on one node; if not, name the split strategy before proceeding.
- Confirm GQA KV-head count ($N_{kv} = N/8$ for ≥ 7 B models) and its impact on KV cache budget (see [subsection 16.1.1](#)).

Phase 3 - Training system design (3 min)

- Compute total training FLOPs: $C = 6N_T P$; divide by cluster throughput to get projected wall-clock time (see [section A.2](#)).
- State the parallelism layout: tensor-parallel degree first (up to the NVLink domain, ≤ 8 on H100 nodes), then pipeline or ZeRO Stage 3/FSDP across nodes (see [subsection 12.4.1](#)).
- State the ZeRO stage: Stage 1 is essentially free; escalate to Stage 3 only when per-GPU memory exceeds 80% of HBM (see [subsection 12.1.2](#)).
- State global batch size relative to the critical batch size and confirm the learning rate schedule terminates correctly (see [subsection 14.2.2](#)).
- State data: total token count, primary sources, and quality filter choice (see [chapter 20](#) and [chapter 21](#)).

Phase 4 - Serving system design (2 min)

- Compute KV cache per request at maximum context S and the crossover batch size B^* (see [section A.3](#)).
- State batching strategy: continuous batching with paged attention for variable-length traffic (see [subsection 18.2.1](#)).
- Name any compression applied (int8 weights, int4 AWQ, int8 KV cache) and the expected quality impact (see [chapter 19](#)).
- State the autoscaling trigger: GPU KV-cache utilization $> 80\%$ or queue depth above threshold.

Phase 5 - Trade-offs and failure modes (1 min)

- Name the dominant bottleneck: compute, memory, bandwidth, or inter-GPU communication (see [subsection 2.2.1](#)).
- Name the first metric you would instrument: MFU for a training system, TTFT for a serving system.
- Name one design decision you would revisit if the bottleneck shifts.

When to escalate

- The model does not fit on the stated hardware without a coherent split plan: size memory correctly before proposing any architecture.
- The stated batch size exceeds the critical batch size by more than $2\times$: the extra parallelism buys nothing; reallocate compute to model scale or data volume.
- TTFT SLO is under 500 ms for a 70 B+ model with no quantization or speculative decoding: flag the constraint explicitly and propose a mitigation.
- Token count is below $20 \times P$: the run is undertrained; quality claims need explicit justification.

C.2 The Pre-Training Run Readiness Checklist

Run this checklist before submitting a large pre-training job. It operationalizes [chapter 12](#), [chapter 13](#), [chapter 14](#), [chapter 20](#), and [chapter 21](#). Items marked **(BLOCK)** will cause an incorrect or wasted run if skipped.

Data and tokenization

- **(BLOCK)** Confirm the tokenizer vocabulary matches the one used in any continued pre-training or fine-tuning checkpoints.
- **(BLOCK)** Confirm exact deduplication (URL or document hash) and near-deduplication (MinHash/LSH) have both been applied (see [section 21.3](#)).
- Confirm quality filter is in place: at minimum, language identification, rule-based heuristics (punctuation, word ratio, length), and a fastText or model-based scorer (see [subsection 21.1.2](#)).
- Confirm total token count. Chinchilla-optimal minimum is $20 \times P$; production runs now target $\geq 100 \times P$ to amortize inference cost (see [subsection 14.2.1](#)).
- **(BLOCK)** Verify no evaluation benchmark contamination in the training corpus (see [section 25.1](#)).
- Confirm multi-epoch token repetition is bounded: diminishing returns accelerate past ~ 4 epochs of any single source.

Memory and parallelism layout

- Compute per-GPU training memory: $16P/(\text{DP} \times \text{TP} \times \text{PP})$ bytes for weights, gradients, and AdamW state, plus activation memory for the local micro-batch (see [section A.1](#)).
- **(BLOCK)** Confirm per-GPU VRAM consumption is below 85% of device HBM capacity.
- Set tensor-parallel (TP) degree to the count of NVLink-connected GPUs per node (≤ 8 on H100 DGX nodes); increasing TP beyond one node incurs a large all-reduce penalty (see [subsection 12.2.1](#)).
- Start with ZeRO Stage 1 (optimizer-state sharding); escalate to Stage 3/FSDP only if per-GPU memory still exceeds 80% of HBM after applying TP (see [subsection 12.1.2](#)).

- If pipeline parallelism is needed, set the micro-batch count m so that the bubble ratio $(p - 1)/m \leq 0.05$ where p is the pipeline depth (see [subsection 12.2.2](#)).
- For long-context runs ($S > 32k$): confirm context parallelism or ring-attention is configured to shard activation memory across the sequence dimension (see [subsection 12.3.2](#)).

Compute budget and batch size

- Compute total training FLOPs: $C = 6N_T P$; divide by cluster FLOP/s at target MFU to get projected wall-clock time (see [section A.2](#)).
- Set global batch size $B \leq B_{\text{crit}}$ (critical batch size). B_{crit} falls as target loss decreases; fit a scaling law or use $B \approx \sqrt{N_T/P} \times d$ as a rough starting point (see [subsection 13.2.2](#)).
- **(BLOCK)** Fully specify the learning rate schedule, including the termination point. Cosine schedules cannot be truncated early without restarting from scratch. For a WSD schedule, reserve 10-20% of total steps for the decay phase (see [subsection 14.2.2](#)).
- If using μP , confirm per-layer learning rates are scaled by $1/\text{width}$ and initialization variance by $1/\text{fanin}$ (see [subsection 14.3.1](#)).

Monitoring and checkpointing

- Confirm MFU instrumentation is in place; target ≥ 0.4 for a well-tuned layout, ≥ 0.35 for runs with significant pipeline or ZeRO Stage 3 overhead.
- Log gradient ℓ_2 norm every step. A norm $\gg 1$ within the first 500 steps signals instability before it manifests in the loss curve.
- Set checkpoint frequency so that a single-node failure loses at most 30 minutes of compute.
- **(BLOCK)** Run a 1,000-step diagnostic at full cluster size before committing the full job. Verify MFU, loss trajectory, and gradient norm are all as expected.

When to escalate

- Estimated per-GPU memory exceeds device HBM at any proposed parallelism layout: do not start; revise the TP/PP/ZeRO configuration.
- MFU is below 0.15 in the first 100 steps: communication is dominating compute; inspect NCCL and check for topology mismatches before continuing.
- Loss diverges or shows spikes within the first 2,000 steps: check learning rate warm-up, gradient clipping threshold, and data pipeline integrity.
- Projected wall-clock time exceeds 90 days at current MFU and cluster size: reconsider model scale or invest in additional cluster capacity before launching.
- Token count is below $5 \times P$: the run is severely undertrained; the resulting checkpoint will underperform Chinchilla-optimal models of the same size.

C.3 The Inference Deployment Readiness Checklist

Run this checklist before promoting an inference endpoint to production traffic. It operationalizes [chapter 15](#), [chapter 16](#), [chapter 18](#), and [chapter 19](#). Use the KV-cache formulas in [section A.3](#) for all sizing steps.

Memory budget

- Compute weight memory: $M_{\text{weights}} = 2P$ bytes (bf16); $0.5P$ bytes (int4 AWQ/GPTQ).
- Compute KV cache per sequence at maximum context S : $C_{\text{kv}} = 2 \times L \times N_{\text{kv}} \times H \times S \times 2$ bytes (see [section A.3](#)).
- **(BLOCK)** Verify $M_{\text{weights}} + B_{\text{max}} \cdot C_{\text{kv}} \leq 0.85 \times \text{VRAM}$ at the target maximum batch size.
- Confirm GQA is enabled with $N_{\text{kv}} = N/8$ for models $\geq 7\text{B}$; this shrinks C_{kv} by $8\times$ with $< 1\%$ accuracy loss at scale (see [subsection 16.1.1](#)).
- If KV cache memory still dominates at target batch size: evaluate int8 KV quantization (halves C_{kv} , doubles B_{max}) or MLA (see [subsection 16.1.2](#)).
- Enable paged attention to eliminate KV cache memory fragmentation; reduces effective VRAM waste by 20-40% under variable-length workloads (see [subsection 18.2.2](#)).

Latency SLOs and batching

- State TTFT and ITL SLO numbers explicitly before configuration (e.g., $\text{TTFT} \leq 500\text{ ms}$, $\text{ITL} \leq 50\text{ ms}$) (see [subsection 15.2.1](#)).
- Compute crossover batch size $B^* = M_{\text{weights}}/C_{\text{kv}}$; throughput gains are superlinear below B^* and sub-linear above it.
- Compute decode step time at target batch B : $t_{\text{step}}(B) = (M_{\text{weights}} + B \cdot C_{\text{kv}})/\text{BW}$. Confirm $t_{\text{step}} \leq \text{ITL SLO}$.
- Use continuous batching for variable-length request streams; it prevents the head-of-line blocking that plagues static batching at mixed context lengths (see [subsection 18.2.1](#)).
- If TTFT dominates the latency budget and prefill is compute-bound, consider prefill/decode disaggregation or speculative decoding with a small draft model (see [subsection 18.1.1](#)).

Quantization and compression

- State the quantization scheme and its expected accuracy impact before deploying: int8 weights (LLM.int8) $\approx 0\%$ quality loss; int4 (AWQ, GPTQ) $\approx 1\text{-}3\%$ (see [section 19.1](#)).
- **(BLOCK)** Benchmark token throughput and task accuracy before and after quantization on the deployment hardware; do not infer from published results on different hardware.
- Do not quantize the KV cache without task-specific validation; numerical sensitivity varies significantly by domain.

Serving infrastructure and autoscaling

- Provision replica count so the fleet sustains peak QPS with at least $1.5\times$ headroom under a single-replica failure.
- Set autoscaling trigger on KV-cache utilization ($> 80\%$) and request queue depth (> 100 requests at P95); do not scale on CPU metrics alone.
- For long-context deployments: confirm context length limits are enforced server-side to prevent KV cache overflows that would silently OOM single requests.

Observability

- Instrument all of: TTFT, ITL, token throughput, KV-cache utilization, GPU memory usage, and request queue depth.
- Log per-request context length and output length; these are the primary drivers of KV cache pressure and are needed to diagnose latency tail events.
- Alert when ITL exceeds $2\times$ the P50 value for more than 60 seconds: this indicates pathological batch composition or KV cache thrashing.

When to escalate

- $B_{\max}$ is below 8 at the target context length: the model is too large for the available hardware at that context; apply quantization or reduce the maximum context limit.
- TTFT exceeds $2\times$ the SLO at all tested batch sizes: evaluate prefill/decode disaggregation or speculative decoding before adding hardware.
- KV-cache utilization stays above 90% during normal traffic: add replicas or enforce a tighter maximum concurrency cap.
- int4 quantization degrades the primary task metric by more than 5% relative: revert to int8 and add hardware instead of accepting the quality loss.
- Any GPU OOM event during normal deployment: tighten $B_{\max}$ -the sizing formula assumes uniform sequence lengths, but real workloads have heavy-tailed context distributions.

Appendix D Further Reading

D.1 Canonical Papers by Topic

Each cluster below maps to one or more chapters in this book. Clusters 1-2 support [chapter 3](#) and [chapter 13-chapter 14](#). Clusters 3-5 map to [chapter 6-chapter 7](#) and [chapter 8-chapter 10](#). Clusters 6-9 support [chapter 15-chapter 19](#). Cluster 10 maps to [chapter 20-chapter 22](#). Clusters 11-12 support [chapter 26-chapter 28](#) and [chapter 23-chapter 25](#). Format: *Title* - Authors (last names) - one-line description.

1. Transformer Architecture & Attention

Attention Is All You Need - Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin - Introduces the Transformer: encoder-decoder architecture built entirely from multi-head self-attention and feed-forward layers, replacing recurrence.

RoFormer: Enhanced Transformer with Rotary Position Embedding - Su, Lu, Pan, Murtadha, Wen, Liu - Proposes Rotary Position Embedding (RoPE), encoding position by rotating query/key vectors so that dot-product attention captures relative distance naturally.

Fast Transformer Decoding: One Write-Head Is All You Need - Shazeer - Introduces Multi-Query Attention (MQA), sharing a single key-value head across all query heads to reduce KV-cache memory and decoding latency.

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints - Ainslie, Lee-Thorp, de Jong, Zemlyanskiy, Lebron, Sanghai - Proposes Grouped-Query Attention (GQA) as an interpolation between MHA and MQA, providing near-MQA speed with quality close to full multi-head attention.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness - Dao, Fu, Ermon, Rudra, Re - Rewrites attention computation with tiling to avoid materialising the full $N \times N$ attention matrix, cutting HBM traffic and achieving $2-4\times$ speed-up.

FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning - Dao - Refines FlashAttention with improved thread-block scheduling and reduced non-matmul FLOPs to nearly double throughput on A100/H100 GPUs.

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model - DeepSeek-AI - Introduces Multi-Head Latent Attention (MLA), compressing KV projections into low-rank latent vectors to reduce KV-cache by up to 93.3% without quality loss.

Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation - Press, Smith, Lewis - Presents ALiBi, adding a linear bias to attention logits based on token distance so models trained on short sequences generalise to longer ones at inference.

GLU Variants Improve Transformer - Noam Shazeer - Studies Gated Linear Unit variants including SwiGLU (swish-gated FFN) and shows they consistently outperform standard ReLU feed-forward layers.

2. Scaling Laws

Scaling Laws for Neural Language Models - Kaplan, McCandlish, Henighan, Brown, Chess, Child, Gray, Radford, Wu, Amodei - Establishes empirical power-law relationships between model parameters, dataset size, compute budget, and validation loss for autoregressive language models.

Training Compute-Optimal Large Language Models - Hoffmann, Borgeaud, Mensch, Buchatskiy, Cai, Millican, van den Driessche, Clark, Casas, Guy, Hendricks, Jones, Menick, Hernandez, Jakub, Hubert, Vinyals, Osindero, Kavukcuoglu, Simonyan, Rae - The “Chinchilla” paper, showing that previous models were over-parameterised and under-trained; optimal compute budget splits equally between model size and token count.

Scaling LLM Test-Time Compute Optimally - Snell, Lee, Xu, Kumar, Garg, Sharan, Abbeel, Levine, Ghosh - Demonstrates that allocating more computation at *inference time* (e.g. best-of- N sampling, process-reward-guided search) can match gains from scaling parameters.

Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer - Yang, Hu, Babuschkin, Sidor, Liu, Farhi, Ryder, Pachocki, Chen, Gao - Introduces Maximal Update Parameterisation (μP), enabling hyperparameter tuning on small proxy models that transfer to large models without retuning.

3. Mixture of Experts

Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer - Shazeer, Mirhoseini, Maziarz, Davis, Le, Hinton, Dean - Introduces the conditional-computation MoE layer with a learned routing gate, scaling model capacity to 137B parameters without proportional compute cost.

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity - Fedus, Zoph, Shazeer - Simplifies MoE routing to top-1 (single-expert) selection and demonstrates stable trillion-parameter training with strong downstream performance.

Mixtral of Experts - Jiang, Sablayrolles, Roux, Mensch, Savary, Bamford, Chaplot, de las Casas, Hanna, Bressand, Lengyel, Bour, Lample, Lavaud, Saulnier, Lachaux, Stock, Gall, Acharya, Gottesman, Ourry, Rastogi, Roelofs, Sansevieri, de Vries, Launay - Presents Mixtral $8 \times 7B$ (and $8 \times 22B$), a sparse MoE model using top-2 routing that matches or exceeds dense models many times its active parameter count.

DeepSeekMoE: Towards Ultimate Expert Specialisation in Mixture-of-Experts Language Models - Dai, Deng, Zhao, Xu, Gao, Chen, Li, Zeng, Yu, Wu, Xie, Li, Huang, Luo, Liang, Chen, Liu, Dai - Proposes fine-grained expert segmentation with shared plus routed experts, achieving higher specialisation and efficiency than standard top- k routing.

4. Distributed Training & Parallelism

Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism - Shoeybi, Patwary, Puri, LeGresley, Casper, Catanzaro - Introduces intra-layer (tensor) model parallelism for Transformers, enabling efficient multi-GPU training of models too large for a single device.

Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM - Narayanan, Shoeybi, Casper, LeGresley, Patwary, Korthikanti, Vainbrand, Kashinkunti, Bernauer, Houston, Tulloch, Shi, Singh, Catanzaro - Combines tensor, pipeline, and data parallelism (“3D parallelism”) with interleaved pipeline scheduling to train 1T-parameter models efficiently.

GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism - Huang, Cheng, Bapna, Firat, Chen, Chen, Lee, Gong, Shen, Wu, Le - Proposes pipeline parallelism with micro-batching and gradient accumulation to train models across multiple accelerators in a synchronous fashion.

PipeDream: Generalized Pipeline Parallelism for DNN Training - Narayanan, Harlap, Phanishayee, Shadri, Devanur, Ganger, Gibbons, Zaharia - Introduces asynchronous 1F1B (one-forward-one-backward) pipeline scheduling to overlap communication and computation, reducing idle pipeline bubbles.

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models - Rajbhandari, Rasley, Ruwase, He - Proposes Zero Redundancy Optimizer (ZeRO), partitioning optimizer state, gradients, and parameters across data-parallel ranks to remove memory redundancy without increasing communication volume significantly.

PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel - Zhao, Gu, Varma, Luo, Huang, Xu, Wright, Shojanazeri, Ott, Sheng, Bhatt, Shen, Abid, Truong, Bhosale, Mihailescu, Patel - Describes PyTorch's production FSDP implementation, detailing sharding strategies, mixed precision, and the API choices made for usability at scale.

5. GPU Kernels & Hardware

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness - Dao, Fu, Ermon, Rudra, Re - (See Cluster 1.) The foundational IO-aware kernel paper; listed again here as the exemplar of GPU-level memory hierarchy optimisation for DL.

Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations - Tillet, Kung, Cox - Introduces the Triton DSL that lets researchers write high-performance GPU kernels in Python without hand-coding CUDA, with tiling handled by the compiler.

NVIDIA H100 Tensor Core GPU Architecture - NVIDIA - Official architecture whitepaper covering the Hopper SM, fourth-generation Tensor Cores, NVLink 4.0, and the new Transformer Engine with FP8 support.

CUTLASS: Fast Linear Algebra in CUDA C++ - Kerr, Nicholls, Tanner, Naumov, Young, Coles, Mercat, Noel, Haluza, Kerr (NVIDIA) - Describes CUTLASS, NVIDIA's open-source template library for high-performance GEMM and convolution operations that underlies cuBLAS and many DL frameworks.

6. Inference Systems

Orca: A Distributed Serving System for Transformer-Based Generative Models - Yu, Jeong, Kim, Kim, Jeong, Cho, Shin - Proposes *iteration-level* (continuous) batching so that finished sequences are immediately evicted and new requests join the batch without waiting, greatly improving GPU utilisation.

Efficient Memory Management for Large Language Model Serving with PagedAttention - Kwon, Li, Zhuang, Sheng, Luo, Foster, Flammarion, Huang, Stoica, Zhang - Introduces PagedAttention, storing KV-cache in non-contiguous memory pages analogous to OS virtual memory, eliminating fragmentation and enabling flexible sharing for vLLM.

Accelerating Large Language Model Decoding with Speculative Sampling - Chen, Borgeaud, Irving, Lespiau, Sifre, Jumper - Shows that a small draft model can propose k tokens in parallel, which the large target model verifies in a single forward pass, yielding lossless $2\text{--}3\times$ decoding speed-up.

Fast Inference from Transformers via Speculative Decoding - Leviathan, Kalman, Matias - Independent

concurrent work establishing the same speculative-decoding framework with formal acceptance-rejection proofs preserving the target distribution exactly.

Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads - Cai, Li, Geng, Peng, Lee, Chen, Dao - Adds multiple lightweight “Medusa heads” to a frozen LLM to predict several future tokens simultaneously, enabling tree-based speculative decoding without a separate draft model.

Efficiently Programming Large Language Models using SGLang - Zheng, Yin, Xie, Huang, Sun, Yu, Cao, Kozyrakis, Stoica, Gonzalez, Barrett, Sheng - Presents SGLang, a structured generation language that fuses RadixAttention (prefix-tree KV-cache reuse) with a runtime for complex, multi-call LLM programs.

7. KV-Cache Reduction

GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints - Ainslie, Lee-Thorp, de Jong, Zemlyanskiy, Lebron, Sanghai - (See Cluster 1.) Reduces KV-cache by a factor equal to the group size by sharing key/value heads across groups of query heads.

Efficient Streaming Language Models with Attention Sinks - Xiao, Tian, Chen, Han, Lewis - Observes that a small number of “attention sink” tokens attract disproportionate attention, enabling infinite-length generation by maintaining only sinks plus a sliding-window cache (*StreamingLLM*).

H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models - Zhang, Sheng, Jiang, Chen, Li, Guo, Zhu, Zheng, Gonzalez, Stoica, Re - Identifies “heavy-hitter” tokens that accumulate the most attention mass and proposes a greedy eviction policy that preserves them to maintain quality under aggressive KV-cache budgets.

SnapKV: LLM Knows What You Are Looking for Before Generation - Li, Zhang, He, Guo, Qian, Zhang, Ma, Shan, Yang, Yin - Clusters per-head attention patterns from a short observation window at prefill time to select and compress the KV-cache before long generation begins.

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model - DeepSeek-AI - (See Cluster 1.) MLA compresses the joint KV projection into a single low-rank latent vector, reducing per-token KV-cache size by 93.3% relative to standard MHA.

8. Beyond-Transformer Architectures

Mamba: Linear-Time Sequence Modeling with Selective State Spaces - Gu, Dao - Introduces a selective state-space model (SSM) with input-dependent gating that matches Transformer quality on language modelling while scaling linearly in sequence length.

RWKV: Reinventing RNNs for the Transformer Era - Peng, Alcaide, Anthony, Albalak, Arcadinho, Biderman, Cao, Cheng, Chung, Derczynski, et al. - Proposes RWKV, reformulating attention as a linear recurrence that admits both parallel training (like Transformers) and constant-memory inference (like RNNs).

Jamba: A Hybrid Transformer-Mamba Language Model - Lieber, Lenz, Bata, Cohen, Osin, Dalmedigos, Safahi, Meirom, Belinkov, Shalev-Shwartz, Shoham - Interleaves Transformer (attention) and Mamba (SSM) layers with MoE feed-forwards, combining the long-context efficiency of SSMs with the in-context learning strength of attention.

9. Compression & Quantization

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers - Frantar, Ashkboos, Hoefler, Alistarh - Uses second-order (Hessian) information via the Optimal Brain Quantisation framework to quantise large LLMs to 3-4-bit weights with minimal perplexity degradation.

AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration - Lin, Tang, Tang, Yang, Dang, Han - Identifies salient weight channels by observing activation magnitudes and protects them from quantisation error, enabling efficient 4-bit inference without retraining.

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models - Xiao, Lin, Seznec, Wu, Demouth, Han - Migrates quantisation difficulty from activations to weights by per-channel scaling, making both amenable to INT8 without accuracy loss.

LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale - Dettmers, Lewis, Belkada, Zettlemoyer - Shows that emergent large-magnitude outlier features in activations cause INT8 degradation, and proposes mixed-precision decomposition to quantise $\sim 99.9\%$ of weights to INT8 while keeping outliers in FP16.

DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter - Sanh, Debut, Chaumond, Wolf - Demonstrates knowledge distillation for Transformers, training a 40%-smaller student BERT that retains 97% of teacher performance on GLUE via soft-label and hidden-state matching.

LoRA: Low-Rank Adaptation of Large Language Models - Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen - (See Cluster 11.) Also a compression technique: decomposing weight updates into low-rank products reduces trainable parameters by $10,000\times$ compared with full fine-tuning.

10. Data Pipelines

CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data - Wenzek, Lachaux, Conneau, Chaudhary, Guzman, Joulin, Grave - Presents the CCNet pipeline: language identification, deduplication, and quality filtering of Common Crawl at scale; foundational for many subsequent pre-training corpora.

The Pile: An 800GB Dataset of Diverse Text for Language Modeling - Gao, Biderman, Black, Golding, Hoppe, Foster, Phang, He, Thite, Nabeshima, Presser, Leahy - Introduces a curated, diverse 800 GB dataset from 22 high-quality sources, providing an open training corpus and evaluation of data diversity effects.

The RefinedWeb Dataset for Falcon LLMs: Outperforming Curated Corpora with Web Data Only - Penedo, Malartic, Hesslow, Cojocaru, Cappelli, Alobeidli, Pannier, Almazrouei, Launay - Shows that massively deduplicated and filtered Common Crawl data (*RefinedWeb*) can outperform manually curated mixtures, challenging assumptions about the necessity of diverse high-quality sources.

Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research - Soldaini, Kinney, Bhagia, Schwenk, Atkinson, Magnusson, Authur, Chandu, Dror, Fang, et al. - Releases a fully open 3T-token corpus from AI2 with detailed provenance and filtering documentation, enabling reproducible pre-training research.

Deduplicating Training Data Makes Language Models Better - Lee, Ippolito, Carlini, Chang, Uszkoreit, Carlini, Litwin - Demonstrates that near-duplicate removal (MinHash LSH) from pre-training corpora reduces memorisation and improves downstream task generalisation.

11. Alignment & Post-Training

Training Language Models to Follow Instructions with Human Feedback - Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, Zhang, Agarwal, Slama, Ray, Schulman, Hilton, Kelton, Miller, Simens, Askeel, Welinder, Christiano, Leike, Lowe - The InstructGPT paper: SFT on human demonstrations followed by PPO against a reward model trained on human preference comparisons, establishing the canonical RLHF pipeline.

Constitutional AI: Harmlessness from AI Feedback - Bai, Jones, Ndousse, Askeel, Chen, DasSarma, Drain, Fort, Ganguli, Henighan, Joseph, Kadavath, Kernion, Conerly, El-Showk, Elhage, Hernandez, Hume, Kaplan, Lovitt, Nanda, Olsson, Amodei, Brown, Clark, McCandlish, Olah, Mann, Amodei, Clark - Trains a harmless assistant using a written “constitution” of principles: supervised learning from self-critique and RLAIIF, removing dependence on human preference labels for harmlessness.

Direct Preference Optimization: Your Language Model is Secretly a Reward Model - Rafailov, Sharma, Mitchell, Ermon, Manning, Finn - Derives a closed-form training objective from the RLHF reward-maximisation problem, enabling preference-based alignment without an explicit reward model or PPO.

DeepSeek-RL: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning - DeepSeek-AI - Trains reasoning models with Group Relative Policy Optimisation (GRPO) and verifiable outcome rewards, achieving OpenAI-o1-level performance while open-sourcing weights and methodology.

LoRA: Low-Rank Adaptation of Large Language Models - Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, Chen - Freezes pre-trained weights and injects trainable rank-decomposition matrices into attention projections, reducing fine-tuning memory and compute by orders of magnitude.

QLoRA: Efficient Finetuning of Quantized LLMs - Dettmers, Pagnoni, Holtzman, Zettlemoyer - Combines 4-bit NormalFloat quantisation (NF4) of the frozen base model with LoRA adapters and paged optimisers, enabling full-parameter-equivalent fine-tuning on a single consumer GPU.

12. Evaluation

Measuring Massive Multitask Language Understanding - Hendrycks, Burns, Basart, Zou, Mazeika, Song, Steinhardt - Introduces MMLU: 57 subjects spanning STEM, humanities, and social science, providing a broad measure of world knowledge and academic reasoning.

Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models - Srivastava, Rastogi, Rao, Shoen, Abid, Fisch, Brown, Santoro, Gupta, Garriga-Alonso, et al. - Presents BIG-Bench: 204 tasks collaboratively designed to probe model capabilities beyond standard benchmarks, with analysis of scaling behaviour.

HellaSwag: Can a Machine Really Finish Your Sentence? - Zellers, Holtzman, Bisk, Farhadi, Choi - Constructs adversarially filtered commonsense NLI completions that are trivial for humans but hard for models, exposing the gap between surface statistics and genuine understanding.

Evaluating Large Language Models Trained on Code - Chen, Tworek, Jun, Yuan, de Oliveira Pinto, Kaplan, Edwards, Burda, Joseph, Brockman, Ray, Puri, Krueger, Petrov, Khlaaf, Sastry, Mishkin, Chan, Gray, Ryder, Pavlov, Power, Kaiser, Bavarian, Winter, Tillet, Such, Cummings, Plappert, Chantzis, Barnes, Herbert-Voss, Guss, Nichol, Paino, Tezak, Tang, Babuschkin, Balaji, Jain, Saunders, Hesse, Carr, Leike, Achiam, Misra, Morikawa, Radford, Knight, Brundage, Murati, Mayer, Welinder, McGrew, Amodei, McCandlish, Sutskever, Zaremba - Introduces HumanEval: 164 hand-written Python programming problems with unit tests for pass@k

evaluation, the standard functional-correctness benchmark for code LLMs.

Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena - Zheng, Chiang, Sheng, Zhuang, Wu, Zhuang, Lin, Li, Li, Xing, Zhang, Gonzalez, Stoica - Introduces MT-Bench (multi-turn instruction-following) and Chatbot Arena (human pairwise voting), establishing LLM-as-a-Judge as a scalable evaluation methodology aligned with human preference.

Holistic Evaluation of Language Models - Liang, Bommasani, Lee, Tsipras, Soylu, Yasunaga, Zhang, Faber, Wu, Bharadwaj, et al. - Presents HELM: a holistic evaluation framework that measures accuracy, calibration, robustness, fairness, bias, and efficiency across dozens of scenarios and model families.

Recommended Reading Order

1. **Architecture foundations first.** Start with *Attention Is All You Need* (Cluster 1), then *FlashAttention* and *GQA* - these three papers underpin [chapter 3](#) and are the most frequently cited in interviews.
2. **Scale and compute.** Read Kaplan et al. then Chinchilla back-to-back ([chapter 13](#)-[chapter 14](#)). MuP (μP) is optional unless you are interviewing for a training infrastructure role.
3. **Systems stack.** Orca $\rightarrow$ PagedAttention $\rightarrow$ speculative decoding (Cluster 6) mirror [chapter 15](#) and [chapter 18](#); add ZeRO and Megatron-LM (Cluster 4) for training roles.
4. **Alignment capstone.** InstructGPT, DPO, and DeepSeek-R1 together span the full post-training landscape covered in [chapter 26](#)-[chapter 28](#). Read them in that order.

D.2 Blogs, Talks, and Engineering Reports

The entries below cover flagship model technical reports and practitioner blogs that put flesh on the theory developed across Parts I-XI. Model reports (GPT-4, Llama 3, Gemini, DeepSeek) are the best place to see how the concepts in [chapter 3](#)-[chapter 14](#) manifest at production scale. Infrastructure and alignment blogs complement [chapter 11](#), [chapter 15](#), [chapter 19](#), and [chapter 26](#)-[chapter 28](#). Format: *Title* - Publisher/Company - one-line description.

13. Engineering Blogs and Technical Reports

Flagship Model Technical Reports

GPT-4 Technical Report - OpenAI - Describes GPT-4 capabilities, evaluations (including bar-exam, MMLU, vision), and safety mitigations; notable for its detailed model-card approach while withholding architectural specifics.

The Llama 3 Herd of Models - Meta AI - Covers the full Llama 3 family (8B-405B), detailing pre-training data curation, supervised fine-tuning, RLHF, and safety alignment with publicly released weights.

Llama 2: Open Foundation and Fine-Tuned Chat Models - Touvron, Martin, Stone, Albert, Almahairi, Babaei, Bashlykov, Batra, Bhargava, Bhosale, et al. (Meta AI) - Introduces Llama 2 and Llama 2-Chat, documenting the SFT-then-RLHF training recipe, ghost attention, and iterative red-teaming for safety.

Gemini: A Family of Highly Capable Multimodal Models - Google DeepMind - Presents the Gemini 1.0 family (Ultra, Pro, Nano), covering multimodal architecture, training at scale, and benchmark results across text, code, image, audio, and video.

Gemini 1.5: Unlocking Multimodal Understanding Across Millions of Tokens of Context - Google DeepMind - Details Gemini 1.5 Pro's million-token context window via a sparse MoE architecture and efficient attention, enabling long-document and long-video understanding.

PaLM 2 Technical Report - Google - Documents the PaLM 2 model family with improved compute-optimal training (Chinchilla scaling), multilingual capability, and reasoning improvements over PaLM.

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning - DeepSeek-AI - Open technical report detailing GRPO-based reinforcement learning on verifiable reasoning tasks, cold-start strategies, and distillation to smaller models; open-weights release included.

DeepSeek-V3 Technical Report - DeepSeek-AI - Describes DeepSeek-V3, a 671B MoE model trained at low cost using FP8 mixed precision, auxiliary-loss-free load balancing, and multi-token prediction pretraining objectives.

Claude's Character - Anthropic - Anthropic's model card and character overview explaining the values, persona, and constitutional principles embedded in the Claude model family.

Inference Systems Engineering Blogs

vLLM: Easy, Fast, and Cheap LLM Serving with PagedAttention - UC Berkeley / vLLM Blog - Introductory blog post accompanying the vLLM open-source release, explaining PagedAttention and demonstrating $24\times$ throughput gains over HuggingFace Transformers baseline serving.

Continuous Batching: How to Serve LLMs $23\times$ Faster Than HuggingFace - Anyscale Engineering Blog - Engineering walkthrough of how iteration-level (continuous) batching works in practice and why it dominates static batching for variable-length generation workloads.

How to Make LLM Inference Faster with Speculative Decoding - Hugging Face Blog - Practitioner-focused explanation of speculative decoding variants (draft-model, Medusa, EAGLE) with HuggingFace transformers integration examples.

Distributed Training & Infrastructure

Megatron-LM: Continued Training and Fine-tuning for GPT and BERT - NVIDIA NeMo / Megatron Engineering Blog - NVIDIA's engineering series covering tensor, sequence, and pipeline parallelism recipes for Megatron-LM, including practical tuning guides for A100/H100 clusters.

ZeRO: Memory Optimizations Toward Training Trillion Parameter Models (DeepSpeed blog) - Microsoft DeepSpeed Blog - Accessible introduction to ZeRO stages 1-3 and ZeRO-Infinity, with empirical memory and throughput numbers for large-model training.

Scaling Distributed Machine Learning with the Parameter Server - Li, Andersen, Park, Smola, Ahmed, Josifovski, Long, Shekita, Su - Classic systems paper introducing the parameter-server architecture for distributed gradient aggregation; foundational reading before data parallelism and ZeRO.

How We Trained Llama 2 at Scale - Meta AI Engineering Blog - Meta's post-mortem of Llama 2 training infrastructure: FSDP, custom CUDA kernels, fault tolerance, and lessons from training 70B models on clusters of thousands of GPUs.

Quantization & Efficiency Engineering

Making LLMs Even More Accessible with bitsandbytes, 4-bit Quantization and QLoRA - Hugging Face Blog - Walkthrough of the QLoRA recipe integrated into the PEFT library, demonstrating fine-tuning of 65B models on a single 48 GB GPU with NF4 quantisation.

Quantization Fundamentals with Hugging Face - Hugging Face Blog - Survey of quantisation approaches (GPTQ, AWQ, SmoothQuant, bitsandbytes) as implemented in the Hugging Face ecosystem, comparing quality and throughput tradeoffs.

Data Curation & Pre-Training

What's in My Big Data? - Allen Institute for AI (AI2) Blog - Engineering post accompanying the Dolma corpus release, detailing data sourcing decisions, quality filtering heuristics, and the open-source dolma toolkit.

DataComp-LM: In Search of the Next Generation of Training Sets for Language Models - Li, Fang, Smyrnis, Ivgi, Jordan, Gadre, Bansal, Guha, Keh, Arora, et al. - Introduces DCLM: a large-scale competition and dataset for studying data curation strategies, with controlled experiments comparing filtering pipelines for LLM pre-training.

Alignment, Safety, and Interpretability

Reward Modeling for Mitigating Sycophancy in RLHF - Anthropic Research Blog - Discusses failure modes of reward models trained on human preferences, specifically sycophancy, and approaches for building more robust reward signals.

Toy Models of Superposition - Anthropic Interpretability Team - Mechanistic interpretability study showing that neural networks can represent more features than neurons by using superposition, with implications for understanding polysemanticity at scale.

Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet - Anthropic Interpretability Team - Applies sparse autoencoders to decompose Claude 3 Sonnet residual-stream activations into millions of monosemantic features, demonstrating that large-scale interpretability is tractable.

OpenAI's Approach to AI Safety - OpenAI - Overview of OpenAI's safety research agenda including RLHF, red-teaming, interpretability, and preparedness framework for frontier models.

Evaluation & Benchmarking

Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference - LMSYS / Hugging Face - Ongoing open leaderboard using blind pairwise human votes to rank LLMs via Elo ratings; the de-facto community standard for chat-quality comparisons.

Open LLM Leaderboard - Hugging Face - Automated leaderboard evaluating open-weight models on a standardised suite (MMLU, HellaSwag, ARC, TruthfulQA, WinoGrande, GSM8K) enabling reproducible open-model comparisons.

Recommended Reading Order

1. **Pick one frontier model report and read it cover to cover.** The Llama 3 report is the most pedagogically structured; it covers pre-training data, SFT, RLHF, safety, and infrastructure in one document and maps directly onto Parts II-XI of this book.
2. **Systems blogs next.** The vLLM PagedAttention launch post and the Anyscale continuous-batching walk-through are short and concrete; read them alongside [chapter 15](#) and [chapter 18](#).
3. **Alignment and interpretability.** Toy Models of Superposition and Scaling Monosemanticity are longer reads best saved after finishing [chapter 26](#)-[chapter 28](#); they provide the “why” behind reward-model failure modes.
4. **Quantisation engineering.** The Hugging Face QLoRA and quantisation-survey blog posts are the fastest path from theory to code; pair them with [chapter 19](#).